Bitcoin Research

Methods Inventory

EXPLAINED EDITION: how the methods work, their mathematical formulations where assigned, and what the evidence means.

244	16	264
METHOD ENTRIES	RESEARCH GROUPS	EVIDENCE FILES

244 method explanations, with explicit formulas for 225 entries and paper context from 27 local research papers.

Includes execution records, negative results, corrected claims and unexecuted proposals. Inclusion is not independent certification of a historical result.

CORY L. THORBECK / RESEARCH ARCHIVE

LINKEDIN https://www.linkedin.com/in/cory-thorbeck-8a21a1146/ / X https://x.com/ioevno

WORKSPACE EDITION • SOURCE INDEX + SEARCHABLE COMPANIONS

PREFACE / CONTEXT

A map of a long research program

This inventory gathers the Bitcoin puzzle and related cryptography work spread across code, saved experiments, audits, notes, and research papers. It records 244 methods in 16 research groups, alongside 264 directly linked evidence files and 27 non-empty local papers. Those counts describe the archive as it exists here; they are not counts of solved targets or independent discoveries.

PUBLICATION DISCLAIMER

This is a historical evidence inventory of methods, runs, and audits performed up to October 2026. Inclusion of a method or run does not constitute independent verification or certification. No private key for Puzzle #71 or any other unsolved puzzle is claimed or disclosed. Corrected or retracted claims supersede earlier stronger statements.

CURRENT CTHEORY STATUS / 27 OCT 2026

CTheory is useful as a scalar-coordinate decomposition and search-planning framework. The current evidence does not establish a predictive shortcut for Puzzle #71. The inspected D1 CUDA path is a stub/unavailable implementation, not a completed solver. Forecasts require forward or held-out validation; target-derived Delta2 or predictor fitting is leakage. Reported work must include the total cost of every plausible branch, not only the cheapest leaf.

THE CENTRAL QUESTION

Can a target key be recovered or its search space meaningfully reduced from public information, a reproducible generator weakness, a valid signature relation, or a bounded discrete-log input? The projects approach that question from several directions. Some methods verify candidates directly. Others model scalar coordinates, examine historical key generators, inspect public artifacts, or estimate the resources a different machine would require. Each direction has a different input contract and a different standard for success.

WHAT THE ARCHIVE PRESERVES

A method record names the operation, what was actually implemented or run, the inputs it requires, and the meaning of the saved result. The evidence label matters: an executed test, a working mechanism, a conditional model, a corrected claim, and a proposal do not carry the same weight. Where an old claim was revised, the correction travels with it. Negative results remain useful because they define which candidate families and budgets were already tested.

WHY THE PAPERS ARE HERE

The papers supply definitions, motivations, experiments, and historical interpretation for many methods. Paper IDs appear on the relevant method sheets, and their stated abstracts are printed later in this volume. Some papers describe earlier project status or stronger claims than later audits support. Read the paper chapter with the dated method evidence and corrections; the book preserves both parts of that record.

THE RESEARCH MAP

Five lines of inquiry

01 DIRECT VERIFICATION AND INTERVAL SEARCH

Address and public-point checkers establish whether a tested scalar matches a target. Small intervals can be exhausted. Kangaroo and BSGS solve bounded discrete logs when the target public point is available; an address hash has a different input contract.

02 STRUCTURED SCALAR COORDINATES

CTheory, root bands, remainder carries, and slot exports reorganize candidate space. Exact reconstruction is arithmetic. A useful forecast needs independent containment and total-work accounting across every plausible slot.

03 GENERATOR AND ENTROPY HYPOTHESES

Seed, recurrence, wallet, firmware, and weak-entropy studies test specific historical generation rules. Their scope is the named recipe, version, state size, target set, and verification procedure recorded with each experiment.

04 PUBLIC ARTIFACTS AND SIGNATURE RELATIONS

On-chain records, distinguished points, metadata, and ECDSA signatures can provide testable clues. Their value depends on provenance and on whether the exact algebraic relation or leakage required by the method is present.

05 SIMULATION AND RESOURCE ESTIMATES

Classical order-finding benchmarks, Shor-style simulation loops, and quantum resource models answer different questions. The inventory keeps measured runs, simulated outcomes, and architecture-dependent estimates separately identifiable.

Use the method ID to follow an idea through its explanation, formula where assigned, evidence files, and paper context. A paper ID leads to the paper chapter; a source ID leads to the file index. Together they make the archive navigable without turning a historical claim into a new result.

EXECUTIVE SUMMARY

The useful results—and the missing step

The archive contains a substantial body of implemented searches and corrective tests. It does not establish a newly recovered Puzzle #71 key or a generally reliable CTheory predictor that removes most unknown candidates.

WORKING MECHANISMS

Address verification, exact partitions, checkpointed no-repeat traversal, incremental public-point updates, known-key tests and small-range exhaustive recovery.

VALID CONDITIONAL TOOLS

Kangaroo/BSGS with a known public point; nonce attacks with the required signature relation or leakage; weak-entropy replay when the generator and seed scope are actually applicable.

UNESTABLISHED SHORTCUTS

Cubic/root re-encodings, metadata anchors, byte novelty, numerical proximity and pattern labels have not supplied a reliably correct unknown scalar.

IMPORTANT CORRECTIONS

Preserve the CTheory speedup retraction, Delta2 calibration leak, transformed-root null correction and distinction between conditional slot cost and total search cost.

Read this as an evidence inventory, not a new experiment. Existing artifacts were inspected; historical searches, GPU claims, balances and external solve status were not rerun or freshly verified for this report.

SCOPE & EVIDENCE

How to read this report

EXECUTED RECORD

A saved report or output describes a run. Inclusion does not independently reproduce or certify it.

IMPLEMENTED / REGISTRY

Relevant source code exists. A completed target run is not inferred.

SYNTHETIC / APPLICABILITY

A conditional mechanism was tested or its required inputs reviewed.

AUDITED / CORRECTED

A later correction supersedes an earlier stronger claim.

BRANCH / CONVERSATION PROPOSAL

Historical documentation or supplied narrative; execution may be absent or limited.

LIVE SNAPSHOT

Mutable worker status is captured at compilation time, not a permanent final count.

Coverage: the main crypto repository, both Puzzle #71 projects, the conversation-atlas Bitcoin audit, Shor archive, PEC37 Bitcoin data, root/near-miss/modular-difference scratch folders, Downloads and home-level artifacts. The companion file index covers 2,188 files, with duplicate-content markers.

Only master is present as a local Git branch. Fourteen historical unmerged branches are represented by their saved wiki pages, not newly checked-out revisions. Existing chat indexes are navigational evidence, not proof of execution.

Excluded: wallet contents, credentials, browser databases, dependency mirrors and unrelated application code. Large data files were identified without treating every generated row as research prose. Deleted, remote and unmounted sources are outside scope.

THE COMPUTATION

From a candidate to a verified match

01 Generate an integer

Choose k inside the stated interval. A sampling rule or predicted band decides which integer is tried next. This step alone makes no statement about the target address.

02 Encode exactly 32 bytes

key.to_bytes(32, 'big') adds zero bytes on the left when needed. Padding does not change the integer or add entropy. The curve operation consumes that scalar.

03 Compute the public point

Calculate Q = k*G on secp256k1. The optimized live worker reuses Q from the previous step by adding the precomputed point for the scalar difference.

04 Serialize and hash

For each tested public-key format, compute RIPEMD160(SHA256(serialized Q)). Compressed and uncompressed forms normally produce different address hashes.

05 Compare with the target

Decode and checksum-check the target P2PKH address, then compare its 20-byte payload with the computed hash. Direct hash comparison avoids Base58 encoding every candidate and is equivalent for this target.

06 Record the outcome

Increment counters, checkpoint progress and save any exact match. A miss rejects this candidate/format; it does not reject the rest of its shard.

READING THE RESULTS

Four different kinds of work

GENERATION

A million rows mean a million generated records. If each row describes a huge interval and one sample, only that sample is available for a candidate check.

VERIFICATION

A candidate becomes an actual tested key only when its derived public point/address is compared with the target. A correct match is the success criterion.

INTERVAL SEARCH

Exhausting all integers in an interval rules out that interval. Kangaroo/BSGS can instead solve a bounded discrete log when the target public point is known; an address hash alone is not that input.

PREDICTION

A model proposes a smaller region before seeing the answer. Its usefulness depends on coverage versus candidate cost on independent evaluation data, not on how narrow the region looks.

A SMALL ROOT EXAMPLE

1234 = 10^3 + 2*10^2 + 34. The coordinates are easy to calculate when 1234 is known. Knowing only that the cube root is 10 still leaves every integer from 1000 through 1330 possible. Rewriting an unknown key into smaller digits has not told us those digits.

SIMULATION IS A DIFFERENT INPUT CONTRACT

Some simulators are given a planted key or an assumed success probability. They can test arithmetic or tuning logic. A random model draw labeled success is not discovery of an unknown key. Each method entry below identifies what is really computed.

MATHEMATICAL FORMULATIONS

Explicit editorial equations are assigned to 225 of 244 methods. They explain an operation, diagnostic, or input contract, with symbol definitions and conditions. Newer entries without a checked formulation are described in prose. These explanations are not additional historical execution records or novelty claims.

CONTEXT / FROM SEARCH TO AUDIT

How the research program developed

The archive did not begin as a finished theory. It grew through repeated attempts to turn visible structure—puzzle tiers, solved-key sequences, scalar decompositions, wallet behavior, signatures, and public-chain artifacts—into a testable reduction of unknown-key work. The important history is therefore not a straight line of discoveries. It is a sequence of hypotheses, implementations, measurements, and corrections.

1 / BUILD A TRUSTWORTHY CHECKER

The first requirement is mechanical: a candidate scalar must reproduce the declared public point or address in the correct serialization. Partitioning, streaming, checkpointing, and incremental point updates make candidate testing reproducible and measurable. These tools improve execution without claiming that any candidate is more likely.

2 / SEARCH FOR STRUCTURE

The work then asks whether puzzle tiers, cubic coordinates, remainders, root bands, bit patterns, generator families, or historical wallet software place the answer in a smaller region. This stage produces many useful coordinate systems and finite hypotheses, but every apparent pattern needs a range-matched baseline and a held-out test.

3 / SEPARATE CONDITIONAL POWER FROM DISCOVERY

Kangaroo, BSGS, nonce algebra, and generator replay can be powerful when their required inputs exist. Their success under a planted key, known interval, leaked nonce relation, or fixed seed family demonstrates the mechanism. It does not show that the required condition exists for an unsolved target.

4 / PRESERVE CORRECTIONS

Later audits found calibration leakage, incomplete slot accounting, transformed-range effects, and implementations that were thinner than their surrounding claims. Those corrections are part of the research result. The inventory keeps them beside the earlier route so a reader can see what changed and why.

CONTEXT / CLAIM LADDER

What the evidence can establish

A method becomes clearer when its claim is placed on the right rung. Strong evidence at a lower rung does not automatically establish the rung above it. The method sheets use this ladder to keep arithmetic, execution, forecasting, and recovery distinct.

01 IDENTITY

An equation reconstructs the same integer or transforms the same candidate exactly.

Boundary: Proves the arithmetic under its definitions; does not predict an unknown coordinate.

02 IMPLEMENTATION

Code performs the stated transformation, search step, or verifier operation.

Boundary: Establishes a mechanism exists; does not establish that a target run completed.

03 EXECUTION

A saved run records inputs, limits, outputs, and an exact verification rule.

Boundary: Establishes tested work within that scope; says nothing about untested candidates.

04 REJECTION

Observed data contradict a fully specified generator or relation.

Boundary: Rejects that configuration; does not reject every member of a broad family.

05 FORECAST

A rule fixed without the target answer contains held-out targets at a stated cost.

Boundary: Supports prioritization only to the measured coverage and evaluation design.

06 RECOVERY

A candidate independently passes the target public-point or address check.

Boundary: Establishes the scalar-target match when provenance and exact verification are preserved.

CONTEXT / READING ROUTES

Four ways into the inventory

The book is intentionally expansive, so it does not need to be read only from page one to the source index. Choose a route based on the question you are trying to answer, then use method IDs, paper IDs, and evidence IDs to move between explanation and provenance.

THE OPERATOR: WHAT CAN ACTUALLY RUN?

Start with verification, interval search, checkpointing, public-point updates, and bounded solver interfaces. Pay attention to target type and interval width.

M001-M037 → M199-M205 → evidence files.

THE THEORIST: DOES THE STRUCTURE REDUCE UNCERTAINTY?

Read the cubic decompositions, root geometry, remainder branches, held-out predictors, and null controls together. Exact identities and predictive evidence answer different questions.

M038-M086 → math appendix → CTheory evidence map.

THE FORENSIC READER: COULD A HISTORICAL GENERATOR BE WEAK?

Follow firmware, wallet, PRNG, seed, and entropy hypotheses. Require an exact version, state/seed scope, derivation path, and target comparison.

M087-M121 → M181-M205 → related paper entries.

THE CRYPTOGRAPHER: ARE PUBLIC ARTIFACTS SUFFICIENT?

Trace signatures, nonce relations, chain metadata, public points, and resource estimates. Check whether the method has the precise algebraic input it assumes.

M150-M180 → M229-M244 → glossary and sources.

On every method sheet, read in this order: mechanism figure, evidence status, what was actually done, required inputs, result meaning, mathematical formulation, evidence IDs, and paper IDs. That order prevents a visually elegant mechanism from carrying a stronger claim than the saved record.

CONTENTS

Research map

01 Verification, sampling and live search 12

02 Kangaroo, interval ECDLP and execution backends 32

03 CTheory decompositions and scalar geometry 49

04 Predictors, held-out validation and corrective audits 67

05 Root-difference, carry, interpolation and near-miss routes 83

06 Generator reconstruction, masking and deterministic sequences 98

07 Firmware entropy and read-only wallet forensics 114

08 Dated generator and decomposition follow-up audits 123

09 Sparse keys, bit walks and synthesis orbits 133

10 HP-RTE, topology and pattern-avoidance experiments 146

11 ECDSA, nonce relations and signature-data searches 161

12 On-chain, metadata and clue relationships 176

13 Weak-entropy wallet and PRNG research 191

14 Exploratory number theory, statistics and literature-inspired probes 213

15 Modular differences and general transformation frameworks 232

16 Shor, classical factoring and quantum-resource research 243

Machine systems preview 266

Classified script preview 268

Research papers in the inventory 256

How to use the mathematical appendix 289

Mathematical tools (not new lemmas) 290

CTheory: full radix-tower description 292

CTheory: extended tower c_6–c_8 293

CTheory references: what each file is for 297

Extended glossary 299

Source index 303

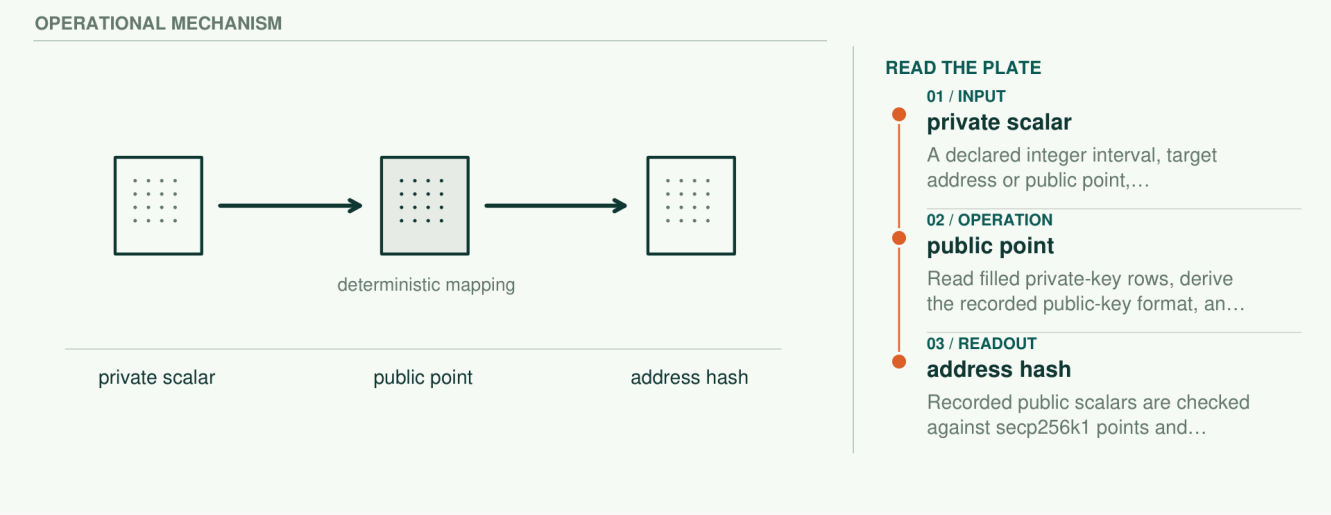
SECTION 01 / VERIFICATION, SAMPLING AND LIVE SEARCH

Public puzzle corpus verification

M001

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

A candidate scalar determines a public point and address hash. Recomputing those values detects an incorrect published key or mismatched database row.

WHAT WAS ACTUALLY DONE

Read filled private-key rows, derive the recorded public-key format, and compare with stored point/HASH160 fields. Empty rows remain unknown; passing the check does not discover their keys.

INPUTS NEEDED

A declared integer interval, target address or public point, candidate-generation parameters and checkpoint/output settings.

WHAT THE RESULT MEANS

Recorded public scalars are checked against secp256k1 points and HASH160; validates the dataset, not a prediction.

MATHEMATICAL FORMULATION

$$Q = kG, \quad h = \mathrm{HASH160}(\mathrm{ser}(Q))$$

G is the curve generator; ser is the declared public-key serialization. Compare Q or h with the recorded target.

EVIDENCE S133 / S024

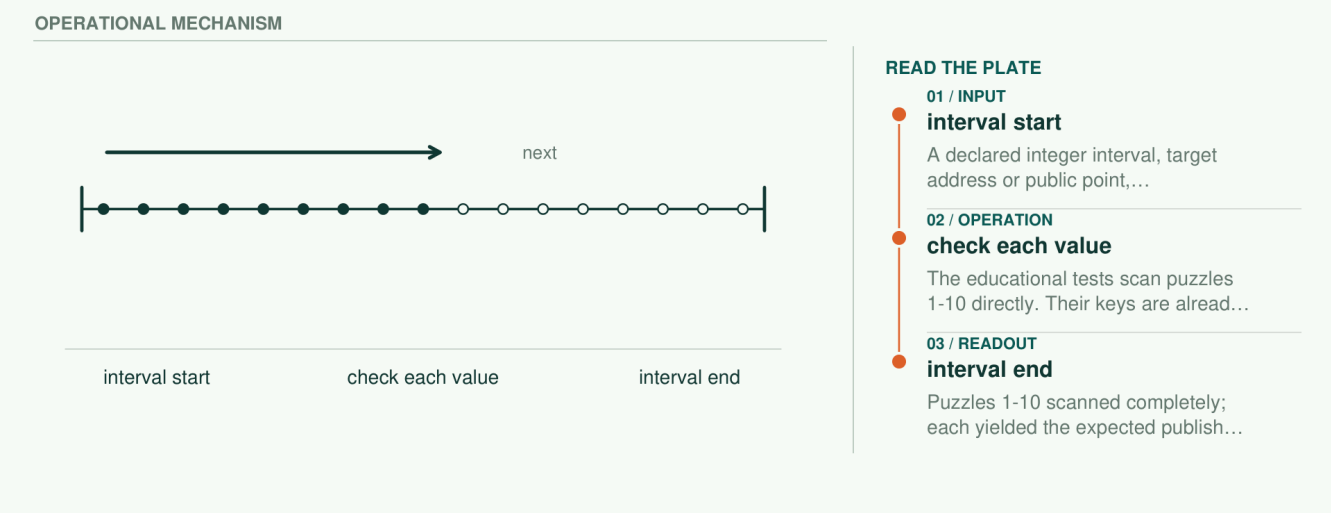
SECTION 01 / VERIFICATION, SAMPLING AND LIVE SEARCH

Small-range exhaustive address search

M002

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

Enumerate every integer in a genuinely small interval and accept only an address match. Exhaustion gives a complete answer for that interval.

WHAT WAS ACTUALLY DONE

The educational tests scan puzzles 1-10 directly. Their keys are already public and used afterward to validate the results; this demonstrates a correct checker at small scale.

INPUTS NEEDED

A declared integer interval, target address or public point, candidate-generation parameters and checkpoint/output settings.

WHAT THE RESULT MEANS

Puzzles 1-10 scanned completely; each yielded the expected published key. Small-range recovery is not an unknown-key breakthrough.

MATHEMATICAL FORMULATION

$$\mathcal{K} = \{L, L+1, \ldots, U\}, \quad W = U - L + 1$$

An exhaustive scan checks all W scalars in the inclusive interval; each candidate requires an exact target comparison.

EVIDENCE S023 / S024

PAPERS PAPER-014

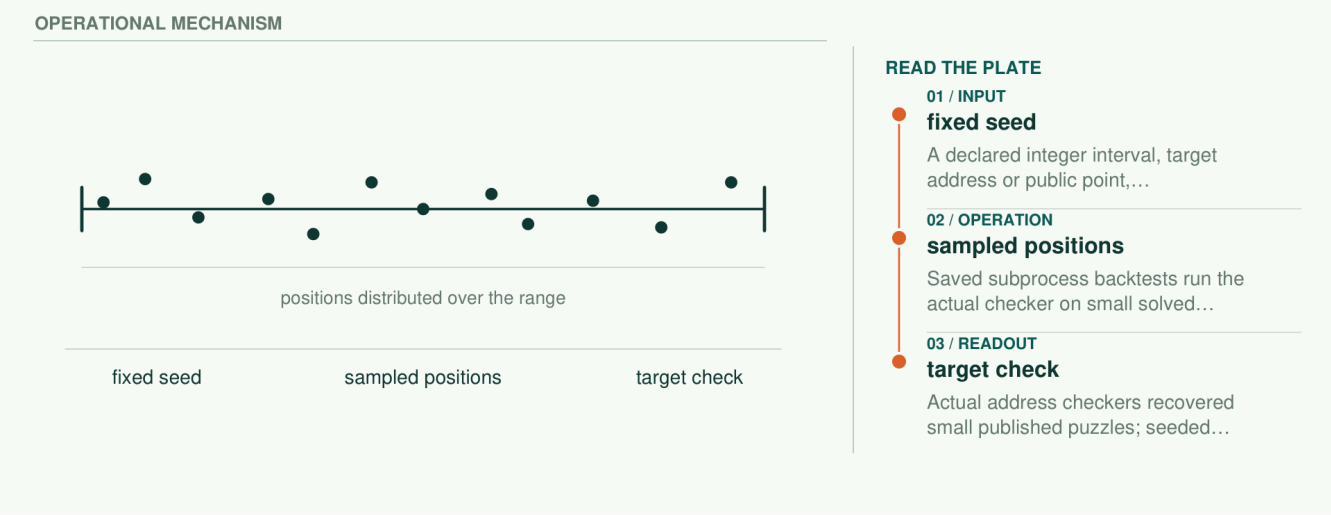
SECTION 01 / VERIFICATION, SAMPLING AND LIVE SEARCH

Full-range seeded sampling backtests

M003

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

A seeded sampler chooses a reproducible order of candidates. The verifier stops when a candidate produces the target address.

INPUTS NEEDED

A declared integer interval, target address or public point, candidate-generation parameters and checkpoint/output settings.

WHAT WAS ACTUALLY DONE

Saved subprocess backtests run the actual checker on small solved puzzles and compare recovered values afterward. Counts describe work until a match, not a learned shortcut.

WHAT THE RESULT MEANS

Actual address checkers recovered small published puzzles; seeded search order does not establish predictive narrowing.

MATHEMATICAL FORMULATION

$$k_t \sim \mathrm{Unif}\{L, \ldots, U\}, \quad \Pr(\text{hit in } t) = 1 - (1 - 1/W)^t$$

W=U-L+1. The probability is the ideal independent, with-replacement baseline for one fixed target; the saved sampler determines the actual sequence.

EVIDENCE S021 / S025

PAPERS PAPER-014

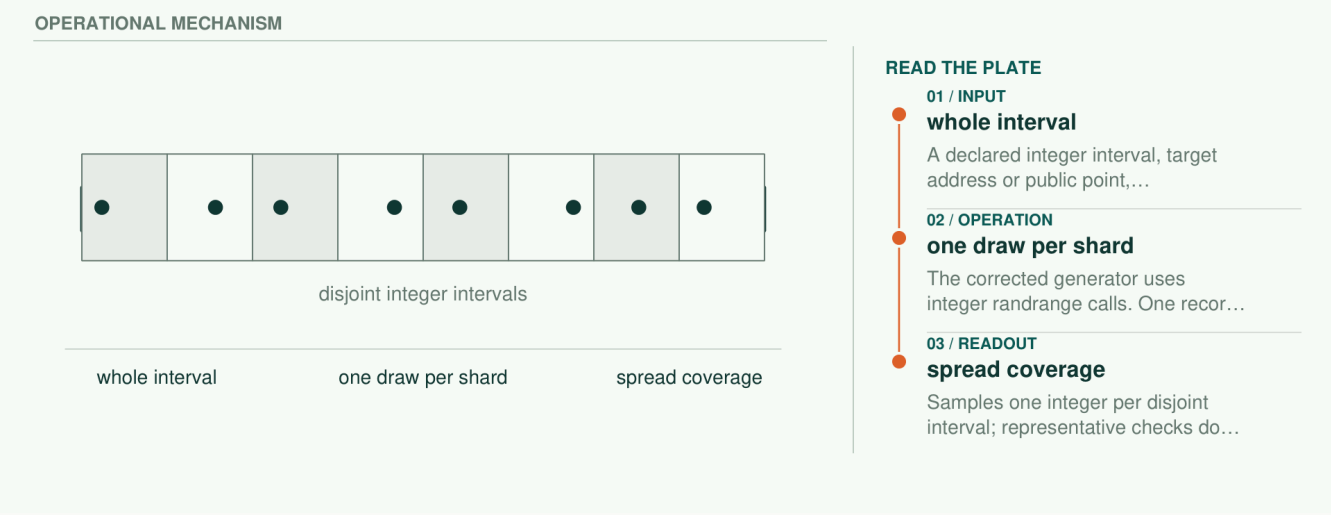
SECTION 01 / VERIFICATION, SAMPLING AND LIVE SEARCH

Uniform within-shard representatives

M004

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

Divide an interval into nonoverlapping shards and select one integer uniformly within each. This spreads a batch over the advertised range.

INPUTS NEEDED

A declared integer interval, target address or public point, candidate-generation parameters and checkpoint/output settings.

WHAT WAS ACTUALLY DONE

The corrected generator uses integer randrange calls. One record is one sampled candidate plus its shard boundaries; no other integer in that shard is thereby checked.

WHAT THE RESULT MEANS

Samples one integer per disjoint interval; representative checks do not exhaust their shard.

MATHEMATICAL FORMULATION

$$k_i \sim \mathrm{Unif}\{b_i, \ldots, b_{i+1} - 1\}$$

One sampled representative is drawn from each nonempty shard. The rest of that shard remains unchecked.

EVIDENCE S026 / S011

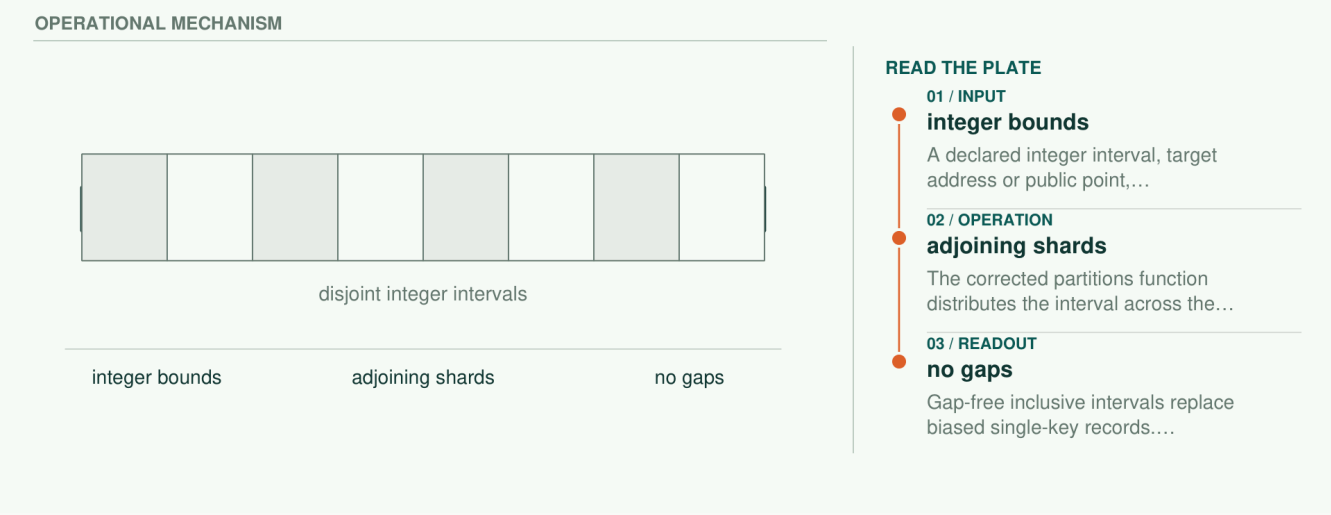
SECTION 01 / VERIFICATION, SAMPLING AND LIVE SEARCH

Exact integer partition coverage

M005

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

Integer division places adjoining boundaries so the first shard starts at the lower limit and the last ends at the upper limit.

INPUTS NEEDED

A declared integer interval, target address or public point, candidate-generation parameters and checkpoint/output settings.

WHAT WAS ACTUALLY DONE

The corrected partitions function distributes the interval across the requested record count. Tests enumerate small examples to check every integer appears exactly once in the intervals.

WHAT THE RESULT MEANS

Gap-free inclusive intervals replace biased single-key records. Exhaustive small-range checks verify the partition arithmetic.

MATHEMATICAL FORMULATION

$$b_i = L + \left\lfloor \frac{iW}{s} \right\rfloor, \quad I_i = [b_i, b_{i+1})$$

W=U-L+1 and s is the number of shards. These half-open integer intervals partition [L,U+1); empty shards require handling when s exceeds W.

EVIDENCE S017 / S003

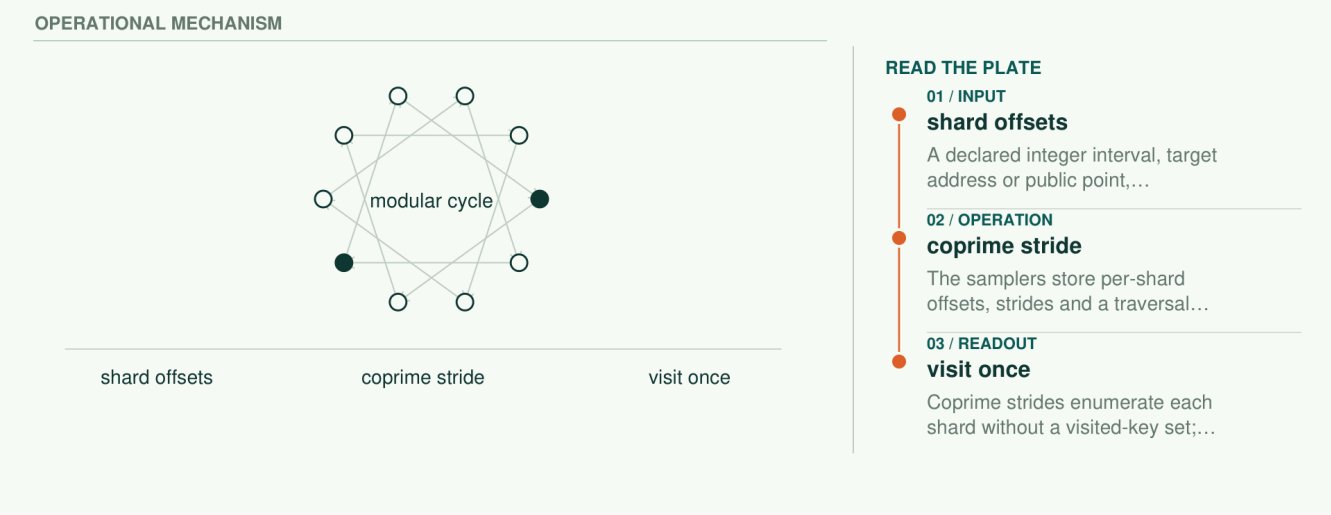
SECTION 01 / VERIFICATION, SAMPLING AND LIVE SEARCH

No-repeat affine shard permutations

M006

EVIDENCE STATUS IMPLEMENTED

FIGURE / METHOD MECHANISM



HOW IT WORKS

An affine permutation visits offsets using (offset + index*stride) modulo shard size. A coprime stride prevents repetition before exhausting the shard.

INPUTS NEEDED

A declared integer interval, target address or public point, candidate-generation parameters and checkpoint/output settings.

WHAT WAS ACTUALLY DONE

The samplers store per-shard offsets, strides and a traversal position. They reconstruct candidates without storing all previously visited keys; this improves coverage accounting, not target likelihood.

WHAT THE RESULT MEANS

Coprime strides enumerate each shard without a visited-key set; retains full support.

MATHEMATICAL FORMULATION

$$k_{i,t} = b_i + (a_i + tq_i \bmod w_i), \quad \gcd(q_i, w_i) = 1$$

A shard has width w_i. Its coprime stride q_i visits every offset exactly once before repeating.

EVIDENCE S026 / S031

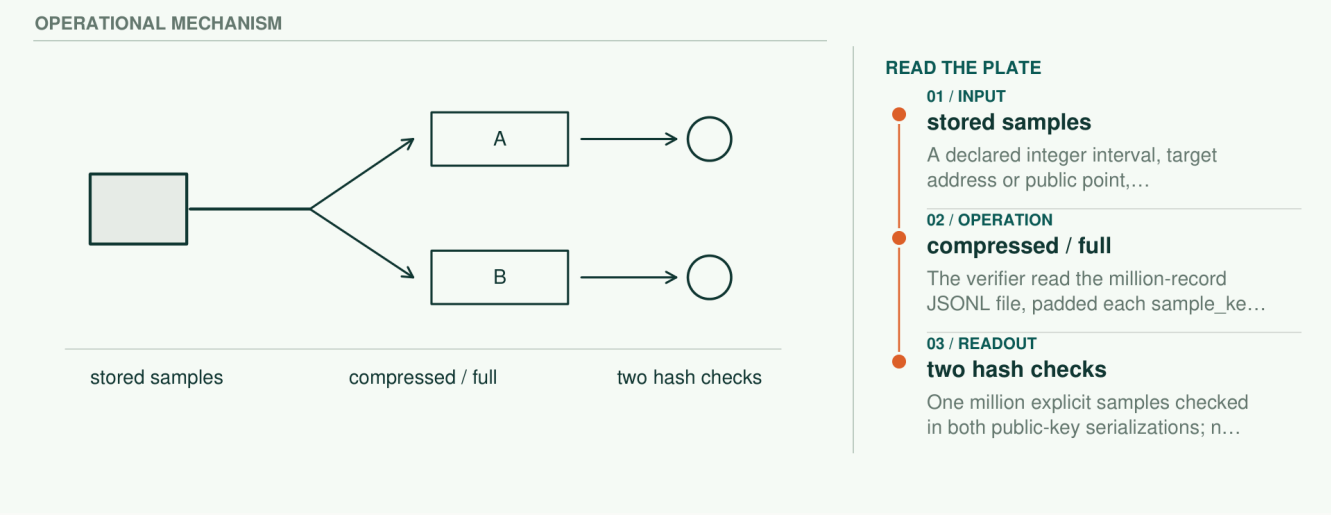
SECTION 01 / VERIFICATION, SAMPLING AND LIVE SEARCH

One-million sample-key verification

M007

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

Derive both public-key serializations for each stored sample and compare their HASH160 values with the decoded target hash.

INPUTS NEEDED

A declared integer interval, target address or public point, candidate-generation parameters and checkpoint/output settings.

WHAT WAS ACTUALLY DONE

The verifier read the million-record JSONL file, padded each sample_key to 32 bytes, and performed two comparisons per scalar. The saved report records no match.

WHAT THE RESULT MEANS

One million explicit samples checked in both public-key serializations; no target match.

MATHEMATICAL FORMULATION

$$h_{i,f} = \mathrm{HASH160}(\mathrm{ser}_f(k_i G)), \quad f \in \{c, u\}$$

c and u denote compressed and uncompressed formats. Each stored sample produces two hash comparisons, not a search of its whole shard.

EVIDENCE S018

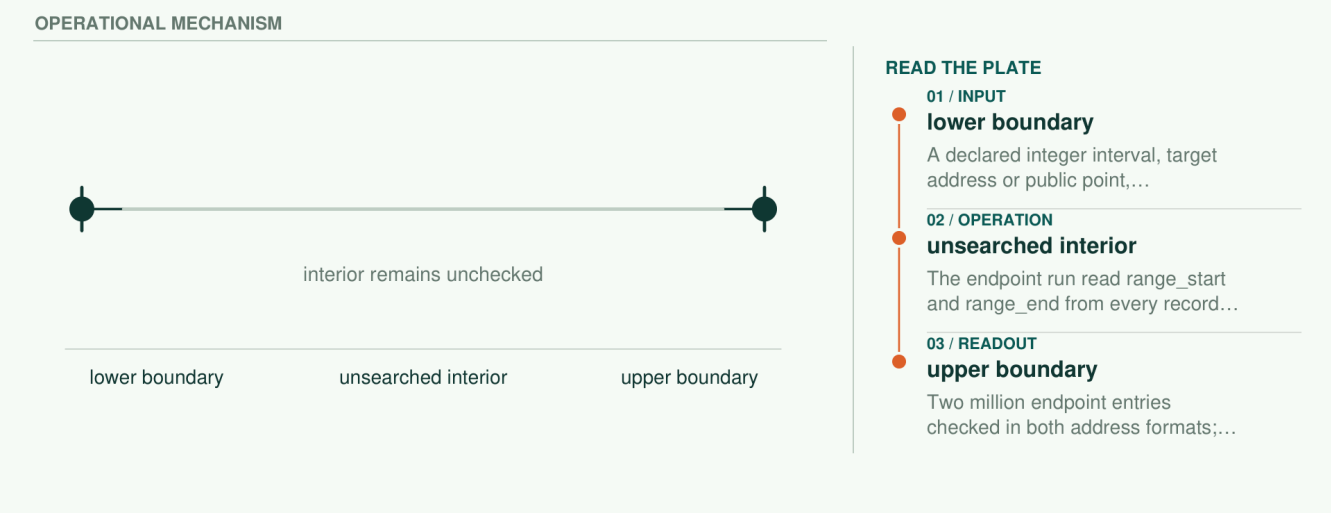
SECTION 01 / VERIFICATION, SAMPLING AND LIVE SEARCH

Range-start and range-end verification

M008

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

A range boundary is itself a possible private scalar and can be checked exactly like a sample. Checking it does not search its neighboring integers.

INPUTS NEEDED

A declared integer interval, target address or public point, candidate-generation parameters and checkpoint/output settings.

WHAT WAS ACTUALLY DONE

The endpoint run read range_start and range_end from every record and checked both address formats. It performed two million scalar checks, separately from the earlier samples.

WHAT THE RESULT MEANS

Two million endpoint entries checked in both address formats; no target match. Interiors were not searched.

MATHEMATICAL FORMULATION

$$\mathcal{C} = cup_i \{L_i, U_i\}$$

Check the endpoints of each inclusive record interval. Duplicated endpoints do not represent new scalar coverage.

EVIDENCE S019

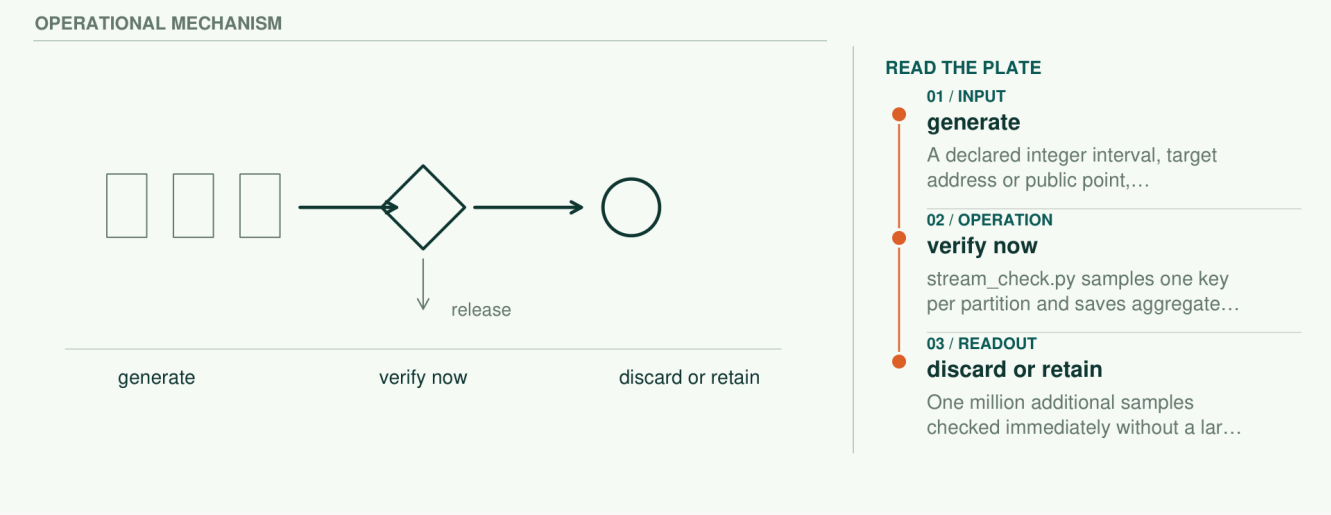
SECTION 01 / VERIFICATION, SAMPLING AND LIVE SEARCH

Generate-and-verify streaming batches

M009

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

Generate a candidate, verify it immediately, then discard it unless it matches. Memory does not grow with the batch size.

INPUTS NEEDED

A declared integer interval, target address or public point, candidate-generation parameters and checkpoint/output settings.

WHAT WAS ACTUALLY DONE

stream_check.py samples one key per partition and saves aggregate counts, a deterministic stream digest and any matches. The seed permits regeneration without a large candidate file.

WHAT THE RESULT MEANS

One million additional samples checked immediately without a large dataset; no matches in the saved batch.

MATHEMATICAL FORMULATION

$$k_t = F(s,t), \quad A_t = 1\{h(k_t) = h_*\}$$

A deterministic seed s and position t regenerate the candidate stream; A_t records an exact target-hash match.

EVIDENCE S016 / S015

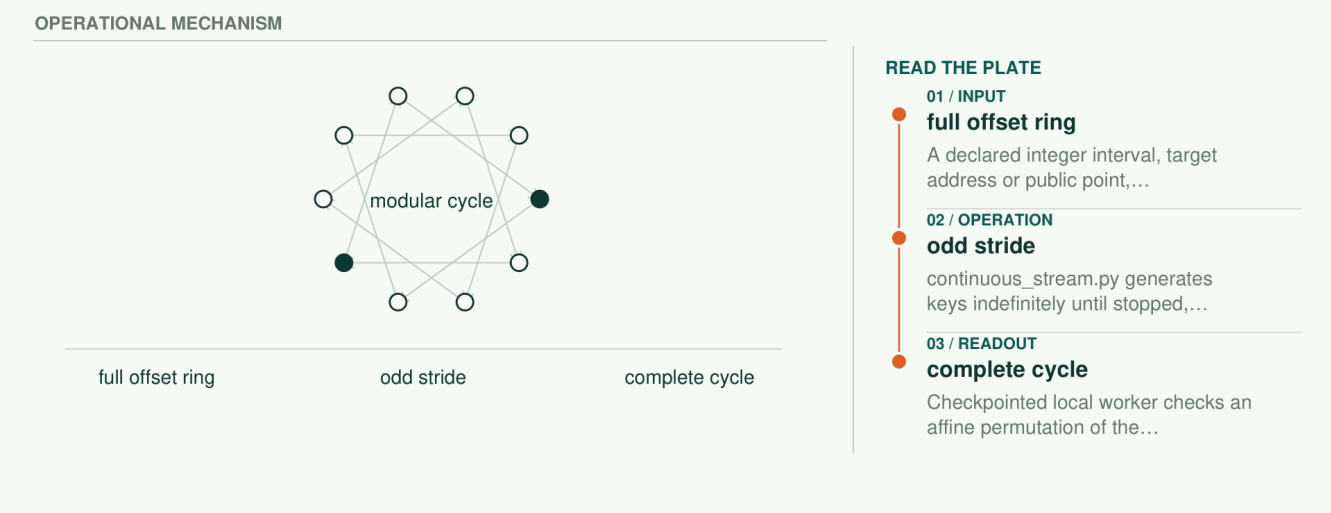
SECTION 01 / VERIFICATION, SAMPLING AND LIVE SEARCH

Continuous affine full-support search

M010

EVIDENCE STATUS LIVE SNAPSHOT

FIGURE / METHOD MECHANISM



HOW IT WORKS

Apply one odd-stride affine permutation to all 2^70 interval offsets. Each offset occurs once over a complete cycle.

INPUTS NEEDED

A declared integer interval, target address or public point, candidate-generation parameters and checkpoint/output settings.

WHAT WAS ACTUALLY DONE

continuous_stream.py generates keys indefinitely until stopped, matched or exhausted. It checkpoints counters and permutation parameters; the process performs actual address checks, not predictor training.

WHAT THE RESULT MEANS

Checkpointed local worker checks an affine permutation of the 70-unknown-bit interval. Status is mutable and must be time-stamped.

MATHEMATICAL FORMULATION

$$k_t = L + (a + tq \bmod 2^{70}), \quad q \text{ odd}$$

An odd stride permutes all offsets of the 71-bit puzzle interval. Only the traversed prefix has been checked before completion.

EVIDENCE S007 / S006

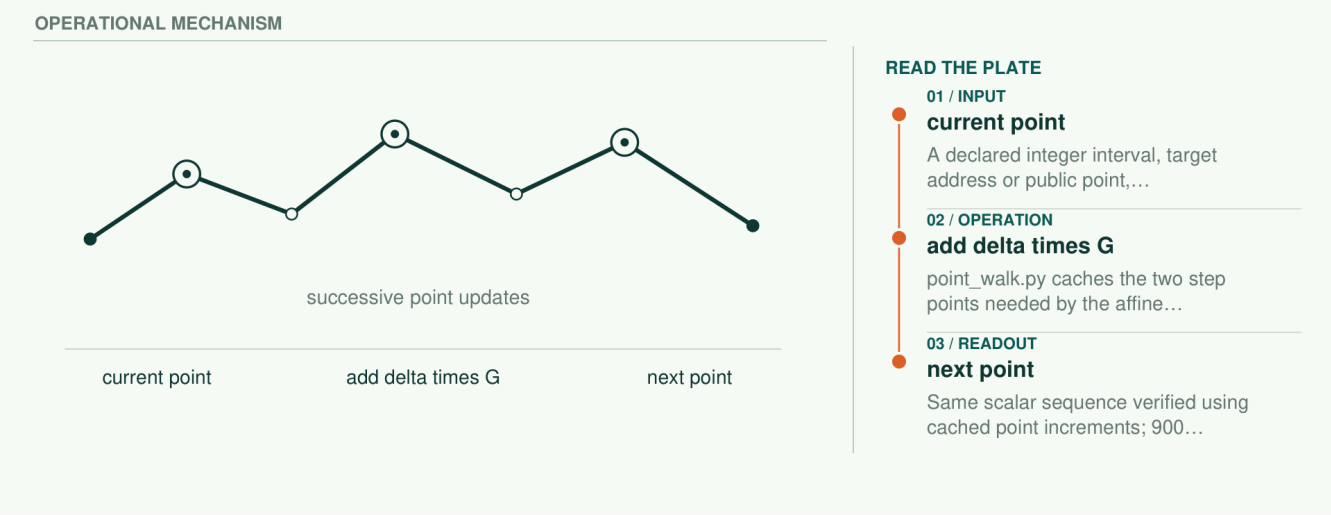
SECTION 01 / VERIFICATION, SAMPLING AND LIVE SEARCH

Incremental public-point verification

M011

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

If the next scalar differs by delta, its public point is the old point plus delta*G. Reusing that point avoids repeated full scalar multiplication.

INPUTS NEEDED

A declared integer interval, target address or public point, candidate-generation parameters and checkpoint/output settings.

WHAT WAS ACTUALLY DONE

point_walk.py caches the two step points needed by the affine traversal, including wraparound. It still serializes and hashes every point; the candidate sequence is unchanged.

WHAT THE RESULT MEANS

Same scalar sequence verified using cached point increments; 900 comparison cases and paired benchmarks passed.

MATHEMATICAL FORMULATION

$$Q_{t+1} = Q_t + (k_{t+1} - k_t)G$$

Reusing the previous public point replaces a new scalar multiplication with a point addition; serialization and hashing still occur.

EVIDENCE S013 / S012

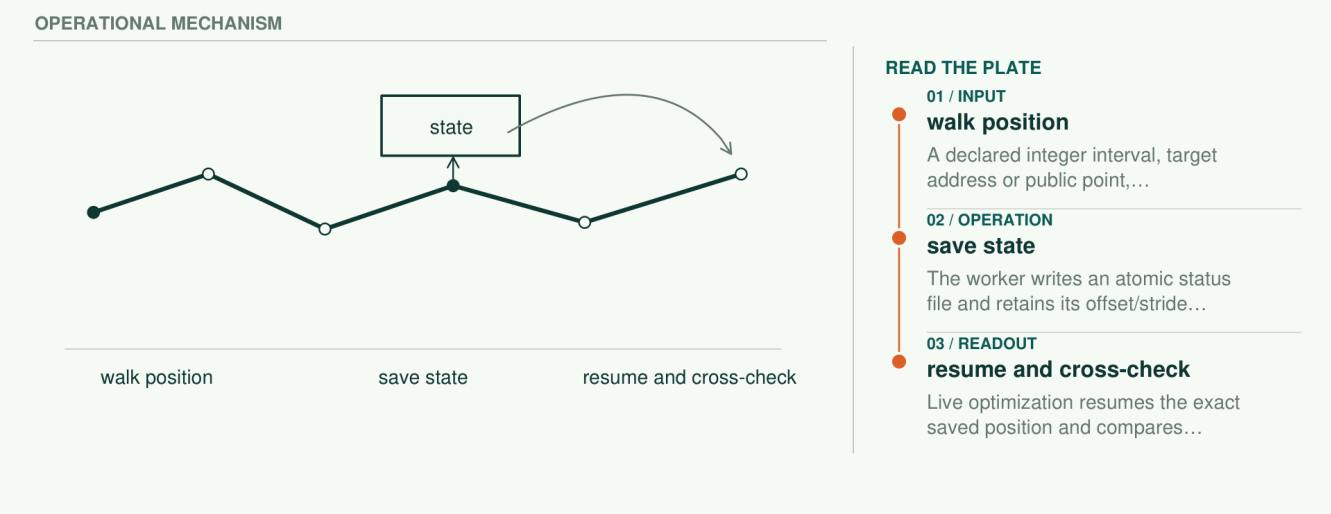
SECTION 01 / VERIFICATION, SAMPLING AND LIVE SEARCH

Checkpoint resume and scalar cross-checks

M012

EVIDENCE STATUS IMPLEMENTED

FIGURE / METHOD MECHANISM



HOW IT WORKS

Saving traversal state allows a restart at the next unchecked position. Independent scalar multiplication can detect incremental-point drift.

INPUTS NEEDED

A declared integer interval, target address or public point, candidate-generation parameters and checkpoint/output settings.

WHAT WAS ACTUALLY DONE

The worker writes an atomic status file and retains its offset/stride across restarts. The optimized version checks the first and each 10,000th point against the original verifier.

WHAT THE RESULT MEANS

Live optimization resumes the exact saved position and compares against full scalar multiplication every 10,000 entries.

MATHEMATICAL FORMULATION

$$\sigma_t = (t, a, q), \quad Q_t \stackrel{?}{=} k_t G$$

The checkpoint stores the next traversal position and permutation parameters; periodic independent multiplication checks point-walk consistency.

EVIDENCE S007 / S004

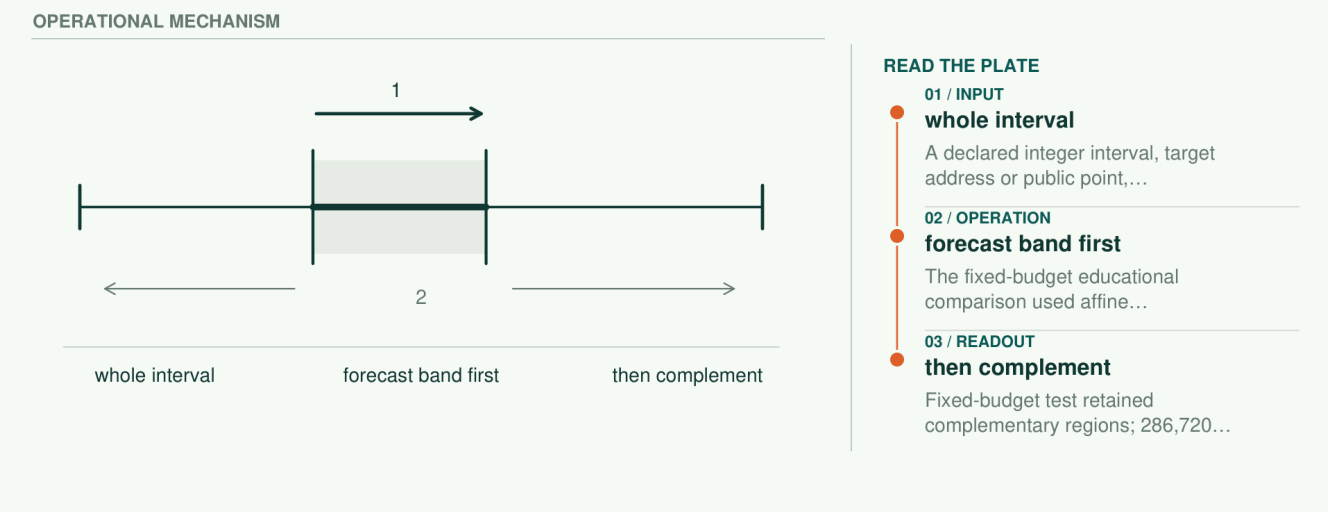
SECTION 01 / VERIFICATION, SAMPLING AND LIVE SEARCH

FUSED priority-band traversal with fallback

M013

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

Search a forecast band first, then its complement. This can improve order under a good predictor without permanently excluding any key.

INPUTS NEEDED

A declared integer interval, target address or public point, candidate-generation parameters and checkpoint/output settings.

WHAT WAS ACTUALLY DONE

The fixed-budget educational comparison used affine permutations within the band and fallback regions. It checked bounded prefixes, not every integer in the forecast band.

WHAT THE RESULT MEANS

Fixed-budget test retained complementary regions; 286,720 checks across both methods found no keys.

MATHEMATICAL FORMULATION

$$I = B \dot{\cup} (I \setminus B)$$

B is the forecast band. Traversing B followed by its complement retains complete support when both parts are eventually exhausted.

EVIDENCE S030
PAPERS PAPER-008

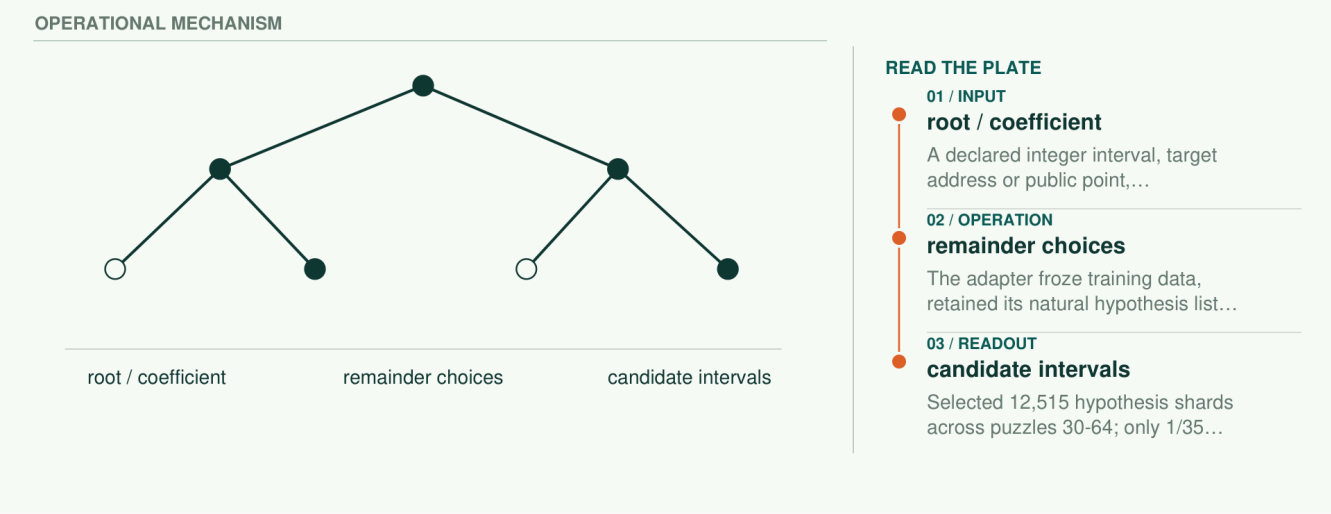
SECTION 01 / VERIFICATION, SAMPLING AND LIVE SEARCH

Advanced selective FUSED/Delta2 adapter

M014

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

Turn predicted root/coefficient/remainder combinations into scalar intervals and sample those hypotheses instead of the entire range.

INPUTS NEEDED

A declared integer interval, target address or public point, candidate-generation parameters and checkpoint/output settings.

WHAT WAS ACTUALLY DONE

The adapter froze training data, retained its natural hypothesis list and checked one representative per shard. A later containment check found that most known evaluation keys were outside those selected intervals.

WHAT THE RESULT MEANS

Selected 12,515 hypothesis shards across puzzles 30-64; only 1/35 known targets was contained.

MATHEMATICAL FORMULATION

$$\mathcal{C} = cup_{\theta \in \Theta}[L_\theta, U_\theta]$$

Each root/coefficient/remainder hypothesis theta defines an interval. Verify target containment separately from samples drawn within those intervals.

EVIDENCE S029

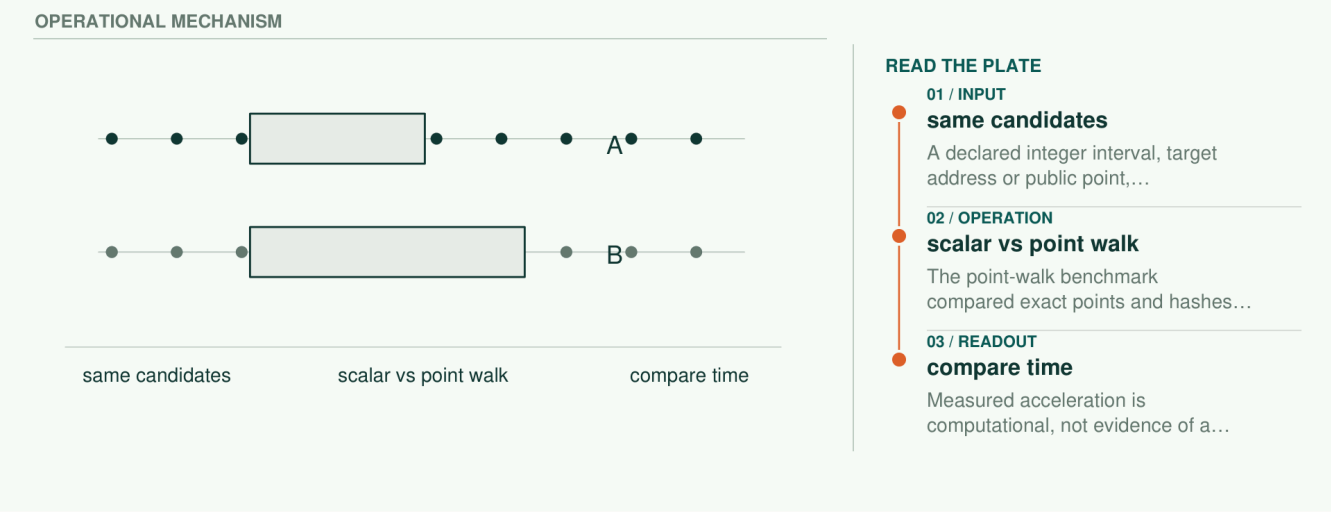
SECTION 01 / VERIFICATION, SAMPLING AND LIVE SEARCH

CPU scalar versus point-walk benchmarking

M015

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

Compare two verifiers on identical scalar streams. Matching outputs isolate performance changes from candidate-selection differences.

INPUTS NEEDED

A declared integer interval, target address or public point, candidate-generation parameters and checkpoint/output settings.

WHAT WAS ACTUALLY DONE

The point-walk benchmark compared exact points and hashes, then timed alternating paired trials with matching stream digests. It measures local throughput, not a higher probability per tested key.

WHAT THE RESULT MEANS

Measured acceleration is computational, not evidence of a more likely candidate distribution.

MATHEMATICAL FORMULATION

$$v = \frac{N_{\text{checks}}}{t}, \quad S = \frac{t_{\text{scalar}}}{t_{\text{walk}}}$$

Compare elapsed times on identical candidate streams and verified outputs. S measures implementation speedup, not a change in success per candidate.

EVIDENCE S005 / S028

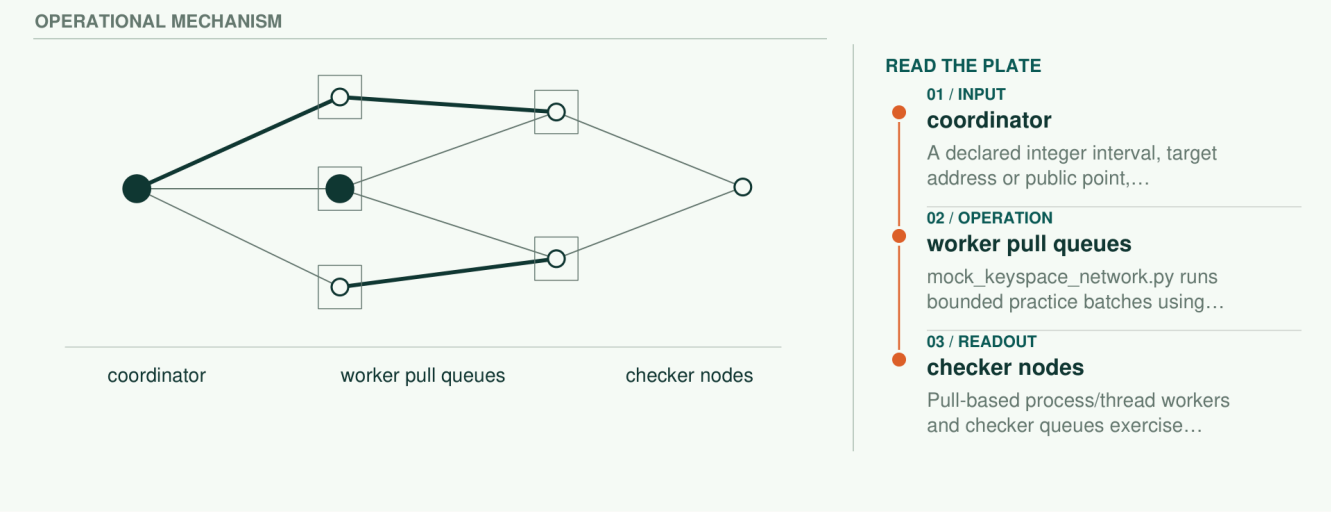
SECTION 01 / VERIFICATION, SAMPLING AND LIVE SEARCH

Mock distributed key-generation network

M016

EVIDENCE STATUS IMPLEMENTED

FIGURE / METHOD MECHANISM



HOW IT WORKS

A coordinator hands work units to generator nodes while optional checker nodes consume results. Pull-based scheduling balances unequal workers.

INPUTS NEEDED

A declared integer interval, target address or public point, candidate-generation parameters and checkpoint/output settings.

WHAT WAS ACTUALLY DONE

mock_keyspace_network.py runs bounded practice batches using processes or threads. Its counters and coverage checks demonstrate orchestration; the mock network is not evidence of a full 71-bit sweep.

WHAT THE RESULT MEANS

Pull-based process/thread workers and checker queues exercise scheduling on bounded mock work; not a full puzzle solve.

MATHEMATICAL FORMULATION

$$N_{\text{total}} = \sum_{j=1}^{p} N_j$$

Worker counts may be added as distinct coverage only when work units are disjoint or duplicate checks are removed.

EVIDENCE S130 / S077

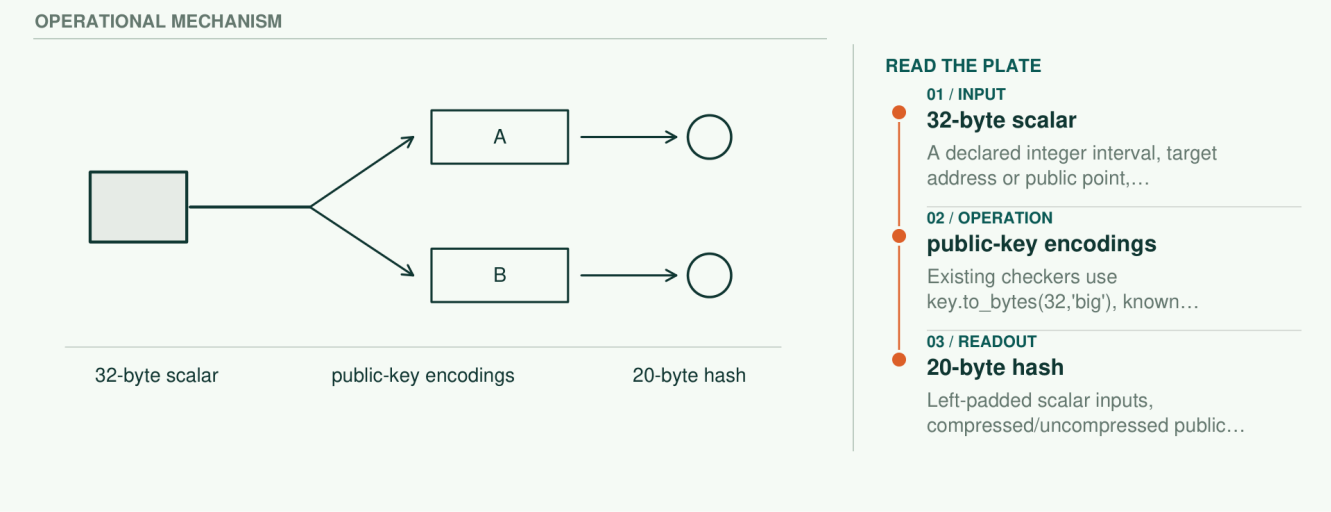
SECTION 01 / VERIFICATION, SAMPLING AND LIVE SEARCH

32-byte padding and dual-format P2PKH checks

M017

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

A scalar is encoded as exactly 32 big-endian bytes, multiplied by G, serialized and hashed. P2PKH comparisons use the address's decoded 20-byte hash.

WHAT WAS ACTUALLY DONE

Existing checkers use key.to_bytes(32,'big'), known scalar-one vectors and compressed/uncompressed formats. They compare HASH160 directly rather than Base58-encoding every generated address, an equivalent target check.

INPUTS NEEDED

A declared integer interval, target address or public point, candidate-generation parameters and checkpoint/output settings.

WHAT THE RESULT MEANS

Left-padded scalar inputs, compressed/uncompressed public keys and known scalar-one address vectors guard verifier correctness.

MATHEMATICAL FORMULATION

$$b = \mathrm{I2OSP}(k, 32), \quad h_f = \mathrm{HASH160}(\mathrm{ser}_f(kG))$$

I2OSP writes exactly 32 big-endian bytes. Leading zero padding preserves the scalar; f selects the public-key format.

EVIDENCE S020 / S027

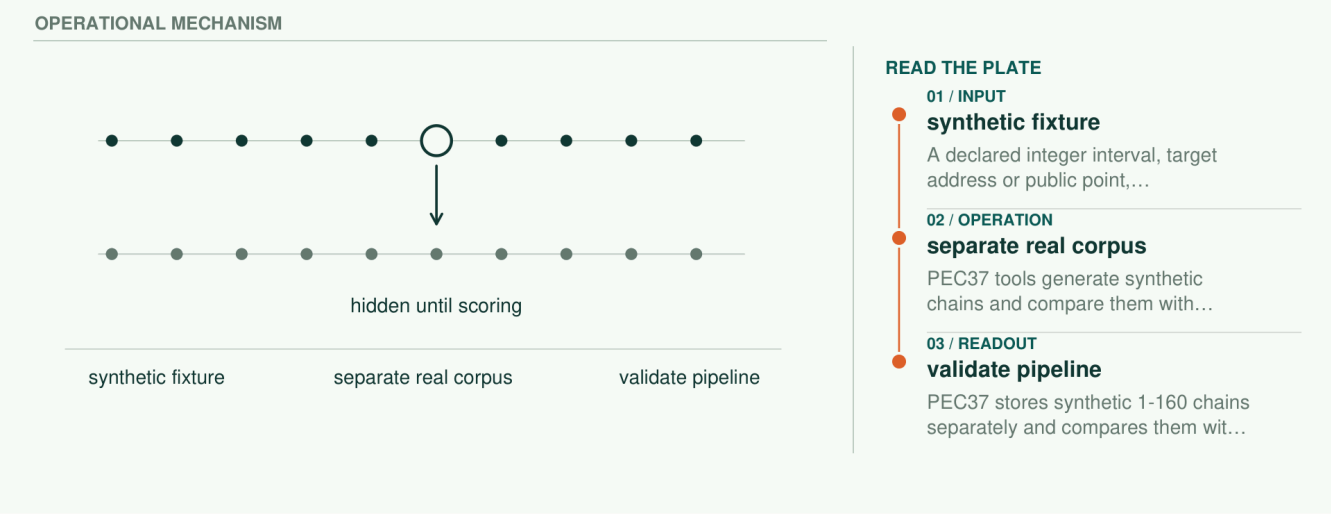
SECTION 01 / VERIFICATION, SAMPLING AND LIVE SEARCH

Synthetic fixtures versus disclosed puzzle data

M018

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

A synthetic fixture can validate pipeline logic because its generation rule is known. It must remain separate from actual published puzzle secrets.

INPUTS NEEDED

A declared integer interval, target address or public point, candidate-generation parameters and checkpoint/output settings.

WHAT WAS ACTUALLY DONE

PEC37 tools generate synthetic chains and compare them with disclosed puzzle data. Successful synthetic reconstruction tests the fixture rule; it cannot populate undisclosed puzzle-key rows.

WHAT THE RESULT MEANS

PEC37 stores synthetic 1-160 chains separately and compares them with actual disclosed keys. Synthetic keys are not puzzle recoveries.

MATHEMATICAL FORMULATION

$$k_i^{\mathrm{syn}} = F(s,i), \quad \widehat{k}_i \stackrel{?}{=} k_i^{\mathrm{syn}}$$

A known fixture generator supplies synthetic ground truth. Agreement validates that fixture pipeline, separately from disclosed puzzle observations.

EVIDENCE S261 / S260

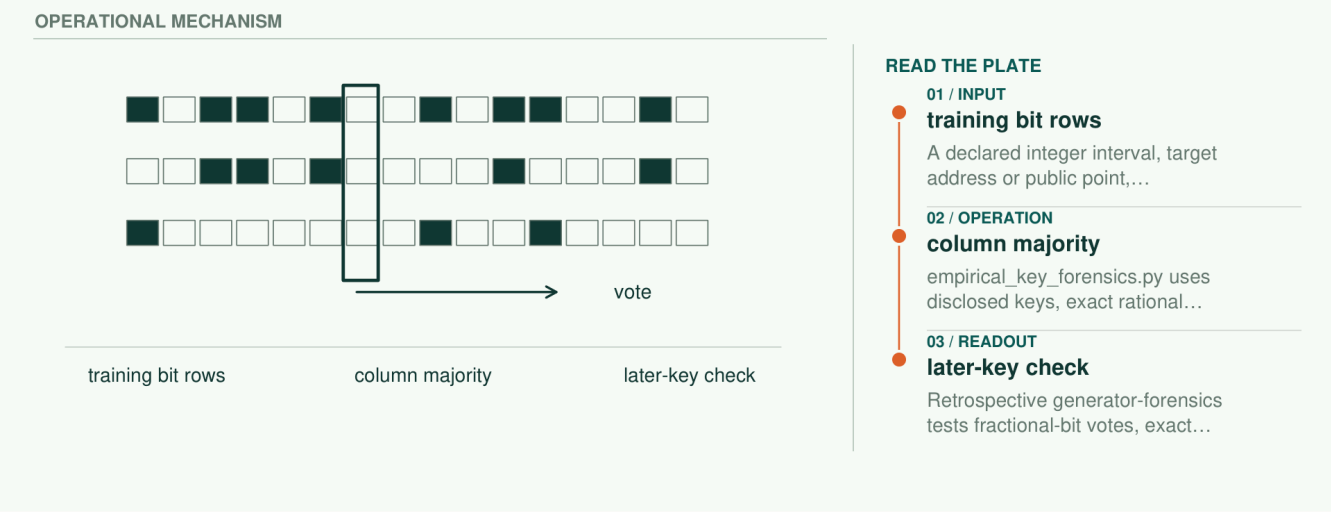
SECTION 01 / VERIFICATION, SAMPLING AND LIVE SEARCH

Empirical bit-vote and position forecasting

M019

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

Train simple position trends or majority votes for fractional bit positions, then evaluate them on later known keys.

INPUTS NEEDED

A declared integer interval, target address or public point, candidate-generation parameters and checkpoint/output settings.

WHAT WAS ACTUALLY DONE

empirical_key_forensics.py uses disclosed keys, exact rational regressions and bit-vote diagnostics. It measures retrospective prediction accuracy; it does not search unknown Bitcoin addresses.

WHAT THE RESULT MEANS

Retrospective generator-forensics tests fractional-bit votes, exact regressions and baseline predictors on disclosed keys.

MATHEMATICAL FORMULATION

$$\hat{b}_j = 1\left\{\sum_{i \in T} b_{ij} > \frac{|T|}{2}\right\}$$

This illustrates a majority bit predictor trained on T; positional regressions are alternative rules. Evaluate each on later, excluded observations.

EVIDENCE S258 / S257

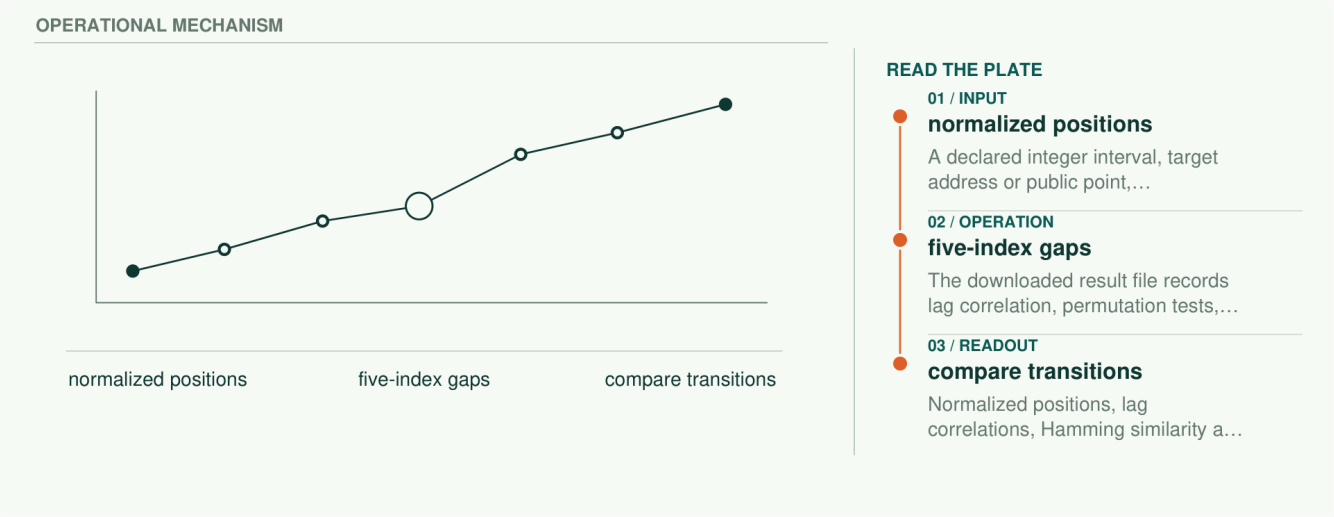
SECTION 01 / VERIFICATION, SAMPLING AND LIVE SEARCH

Five-index scale/transition analysis

M020

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

Normalize every-fifth-puzzle keys to comparable interval positions and ask whether one predicts the next better than a baseline.

INPUTS NEEDED

A declared integer interval, target address or public point, candidate-generation parameters and checkpoint/output settings.

WHAT WAS ACTUALLY DONE

The downloaded result file records lag correlation, permutation tests, prefix Hamming distances and held-out transition errors. These are saved statistical results, not additional private-key discoveries.

WHAT THE RESULT MEANS

Normalized positions, lag correlations, Hamming similarity and held-out transition forecasts are recorded; no new key is recovered.

MATHEMATICAL FORMULATION

$$u_i = \frac{k_i - 2^{i-1}}{2^{i-1}}, \quad \rho_5 = \mathrm{Corr}(u_i, u_{i+5})$$

Normalize scalars to their tier positions before testing every-fifth-index transitions. The correlation is descriptive until held-out prediction is assessed.

EVIDENCE S001

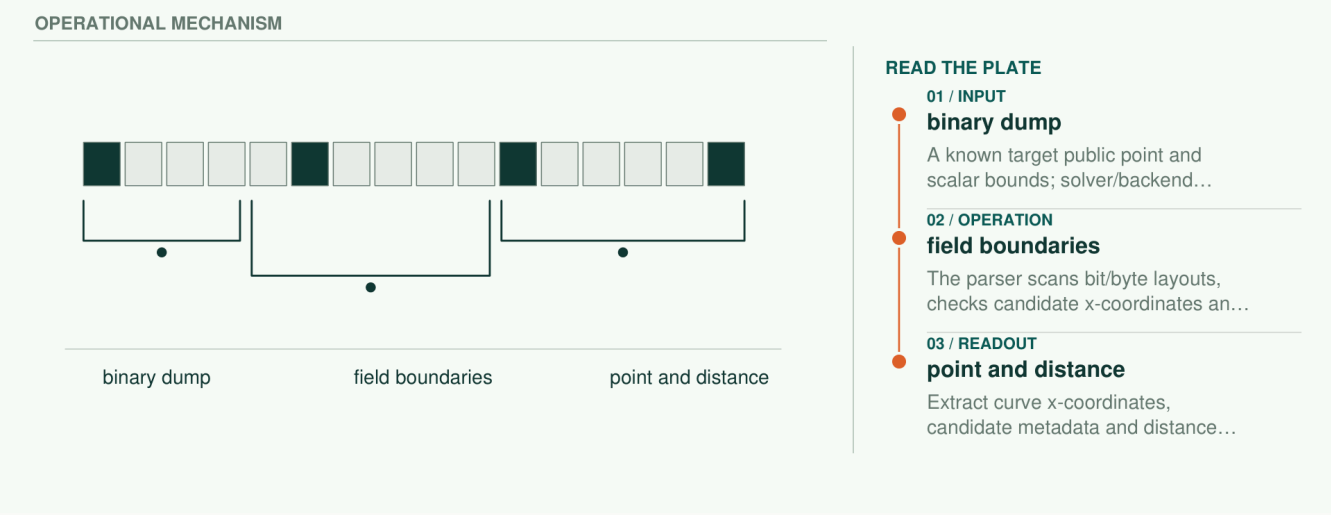
SECTION 02 / KANGAROO, INTERVAL ECDLP AND EXECUTION BACKENDS

Binary distinguished-point parsing

M021

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

Distinguished-point dumps encode selected curve points and possibly walk distances. Recovering their layout is necessary before collision algebra has meaning.

WHAT WAS ACTUALLY DONE

The parser scans bit/byte layouts, checks candidate x-coordinates and extracts metadata slices. Later scripts test interpretations; a parseable integer is not automatically a valid walk distance.

INPUTS NEEDED

A known target public point and scalar bounds; solver/backend configuration. Address hashes alone do not supply the wild-walk point.

WHAT THE RESULT MEANS

Extract curve x-coordinates, candidate metadata and distance fields from the supplied binary. Interpretation of metadata remains a hypothesis.

MATHEMATICAL FORMULATION

$$x = \sum_{j=0}^{w-1} b_j 2^{8j}$$

This is a little-endian interpretation of w bytes; reverse ordering for big endian. Candidate layouts need length, curve-point and distance consistency checks.

EVIDENCE S102 / S171

PAPERS PAPER-011 / PAPER-019 / PAPER-026

SECTION 02 / KANGAROO, INTERVAL ECDLP AND EXECUTION BACKENDS

Tame/wild collision recovery

M022

EVIDENCE STATUS IMPLEMENTED

FIGURE / METHOD MECHANISM

HOW IT WORKS

A tame walk starts from a known scalar; a wild walk starts from the target point. Their meeting relates recorded distances and yields a candidate scalar.

INPUTS NEEDED

A known target public point and scalar bounds; solver/backend configuration. Address hashes alone do not supply the wild-walk point.

WHAT WAS ACTUALLY DONE

collision.py groups same-x entries, tries tame/wild assignments and derives candidates. Because x alone has two point parities, the final scalar must pass exact public-key verification.

WHAT THE RESULT MEANS

Same-point opposite-herd distances yield a scalar when the walk origins and distances are valid. No real target collision is established here.

MATHEMATICAL FORMULATION

$$tG = Q + wG \quad \Longrightarrow \quad k \equiv t - w \pmod{n}$$

Q=kG, n is the subgroup order, and t,w are total tame/wild scalar coordinates at a same-point collision. Same-x observations also require testing the opposite sign.

EVIDENCE S097 / S077

PAPERS PAPER-011 / PAPER-019 / PAPER-026

SECTION 02 / KANGAROO, INTERVAL ECDLP AND EXECUTION BACKENDS

Single-process bounded Pollard kangaroo

M023

EVIDENCE STATUS IMPLEMENTED

FIGURE / METHOD MECHANISM

HOW IT WORKS

Pollard kangaroo uses deterministic pseudo-random jumps so tame and wild walks can merge. Distinguished points reduce the amount of stored walk data.

WHAT WAS ACTUALLY DONE

kangaroo.py implements bounded CPU walks, jump tables, point selection and verification. It requires the target public point and a budget; an address hash cannot initialize its wild walk.

INPUTS NEEDED

A known target public point and scalar bounds; solver/backend configuration. Address hashes alone do not supply the wild-walk point.

WHAT THE RESULT MEANS

Standard interval discrete-log implementation for targets with known public points.

MATHEMATICAL FORMULATION

$$X_{t+1} = X_t + s(X_t)G, \quad d_{t+1} = d_t + s(X_t)$$

The point-selected jump s updates both the walk and its tracked scalar distance. Bounded interval kangaroo has expected work proportional to the square root of interval width.

EVIDENCE S109
PAPERS PAPER-010

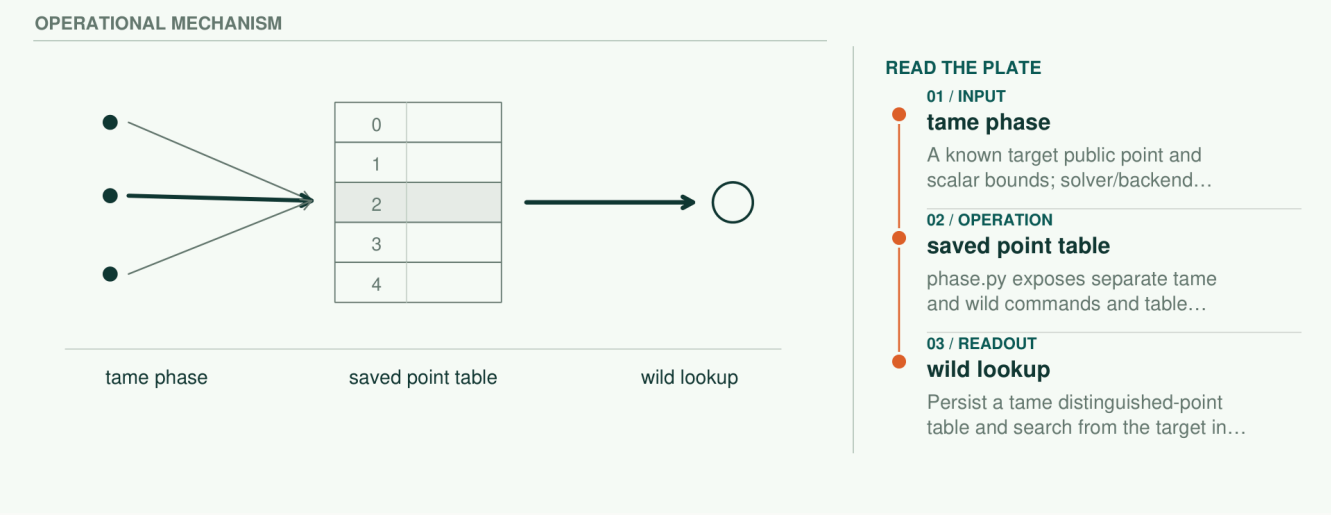
SECTION 02 / KANGAROO, INTERVAL ECDLP AND EXECUTION BACKENDS

Separated tame and wild phases

M024

EVIDENCE STATUS IMPLEMENTED

FIGURE / METHOD MECHANISM



HOW IT WORKS

Build the tame table once, save it, and later walk from the target until a compatible distinguished point is reached.

INPUTS NEEDED

A known target public point and scalar bounds; solver/backend configuration. Address hashes alone do not supply the wild-walk point.

WHAT WAS ACTUALLY DONE

phase.py exposes separate tame and wild commands and table export/loading. This supports separate machines or stages without converting a missing public key into an ECDLP target.

WHAT THE RESULT MEANS

Persist a tame distinguished-point table and search from the target in a separate phase.

MATHEMATICAL FORMULATION

$$\mathcal{T} = \{(X_t, d_t) : X_t \in \mathcal{D}\}$$

Store tame distinguished points in table T, then match compatible wild points. D is the declared distinguished-point predicate, not every visited point.

EVIDENCE S113 / S224

PAPERS PAPER-006 / PAPER-011 / PAPER-019

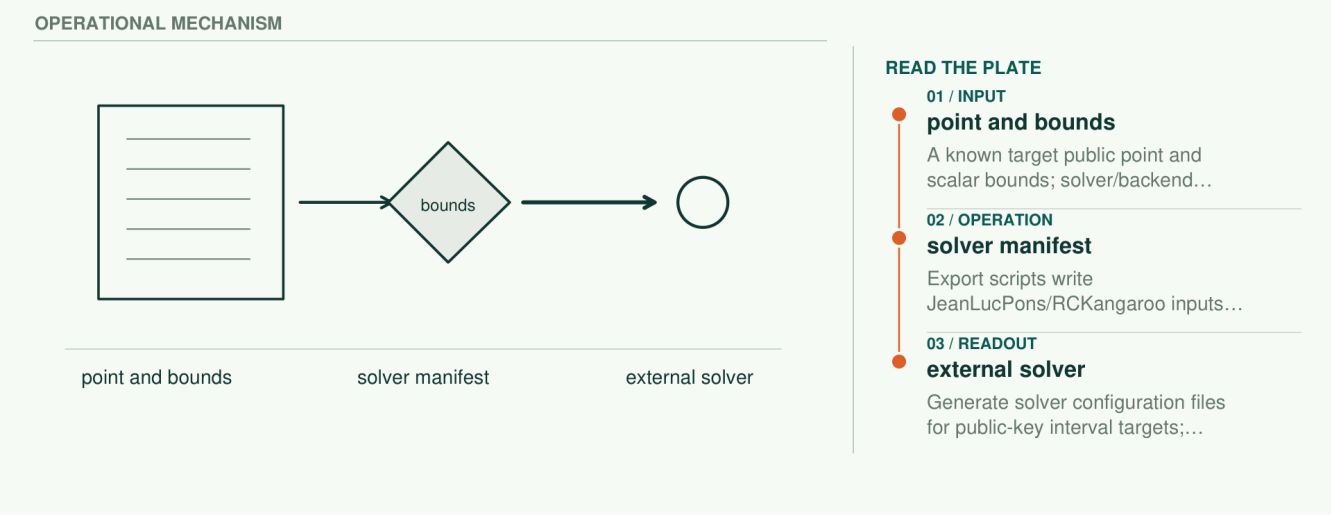
SECTION 02 / KANGAROO, INTERVAL ECDLP AND EXECUTION BACKENDS

JeanLucPons / RCKangaroo export

M025

EVIDENCE STATUS IMPLEMENTED

FIGURE / METHOD MECHANISM



HOW IT WORKS

External interval solvers need bounds, a target point and runtime settings in their expected configuration format.

INPUTS NEEDED

A known target public point and scalar bounds; solver/backend configuration. Address hashes alone do not supply the wild-walk point.

WHAT WAS ACTUALLY DONE

Export scripts write JeanLucPons/RCKangaroo inputs from puzzle or slot metadata. They prepare jobs; whether the external executable ran and found a match requires separate logs.

WHAT THE RESULT MEANS

Generate solver configuration files for public-key interval targets; configuration existence is not execution evidence.

MATHEMATICAL FORMULATION

$$\mathcal{J} = (Q, L, U), \quad Q = kG, \quad L \le k \le U$$

The exported solver job declares the target point and inclusive scalar bounds. A configuration file is not a completed interval search.

EVIDENCE S105 / S225

PAPERS PAPER-002 / PAPER-004 / PAPER-005 / PAPER-006 / PAPER-007

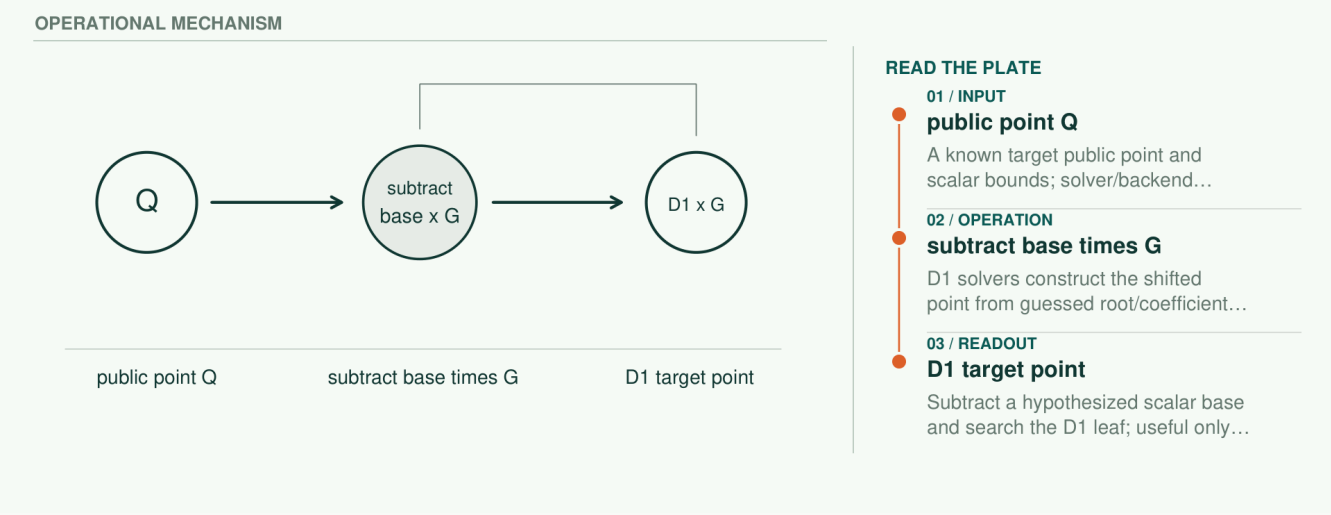
SECTION 02 / KANGAROO, INTERVAL ECDLP AND EXECUTION BACKENDS

Direct D1 interval kangaroo

M026

EVIDENCE STATUS IMPLEMENTED

FIGURE / METHOD MECHANISM



HOW IT WORKS

If k=base+D1, then Q-base*G=D1*G. A known base converts recovery into a smaller interval discrete log.

INPUTS NEEDED

A known target public point and scalar bounds; solver/backend configuration. Address hashes alone do not supply the wild-walk point.

WHAT WAS ACTUALLY DONE

D1 solvers construct the shifted point from guessed root/coefficient coordinates. A wrong base usually makes the true key absent from that interval, so leaf cost alone is incomplete.

WHAT THE RESULT MEANS

Subtract a hypothesized scalar base and search the D1 leaf; useful only if the hypothesized coordinates are correct.

MATHEMATICAL FORMULATION

$$B = R_1^3 + C_1 R_1^2, \quad Q_D = Q - BG = D_1 G$$

Fixing the correct R1,C1 turns the target into a bounded D1 discrete log. The equation does not determine those higher coordinates.

EVIDENCE S129 / S152

PAPERS PAPER-002 / PAPER-012

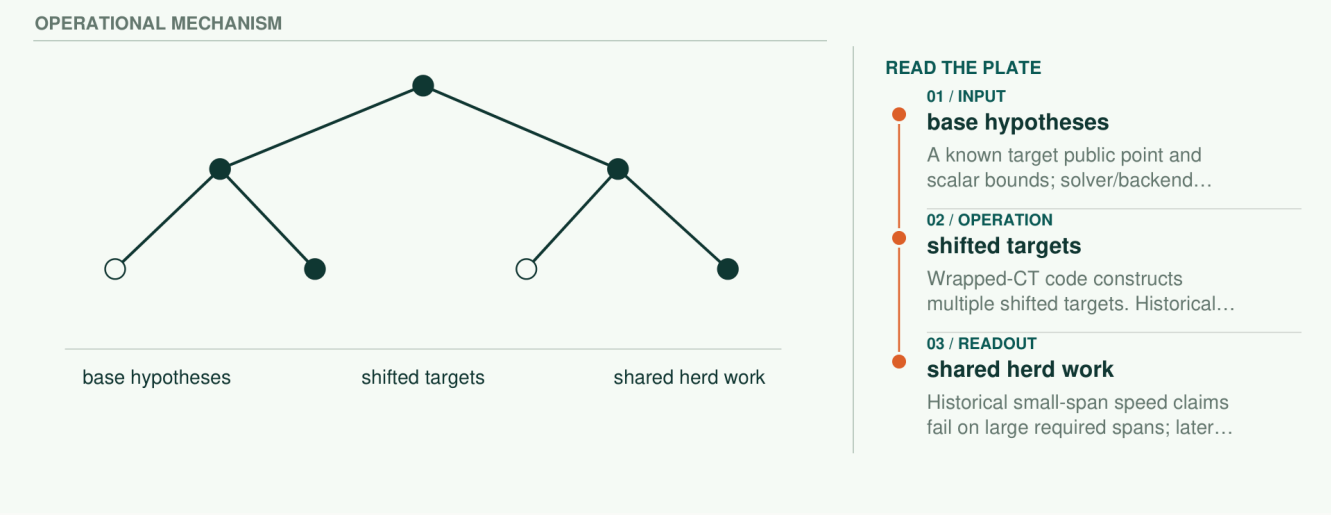
SECTION 02 / KANGAROO, INTERVAL ECDLP AND EXECUTION BACKENDS

Wrapped CT multi-target kangaroo

M027

EVIDENCE STATUS AUDITED/CORRECTED

FIGURE / METHOD MECHANISM



HOW IT WORKS

Share walks or targets across several related base hypotheses, attempting to reuse work instead of solving each independently.

INPUTS NEEDED

A known target public point and scalar bounds; solver/backend configuration. Address hashes alone do not supply the wild-walk point.

WHAT WAS ACTUALLY DONE

Wrapped-CT code constructs multiple shifted targets. Historical audits inspect span/setup growth and find that the advertised tiny-span benefit does not transfer unconditionally to high-index targets.

WHAT THE RESULT MEANS

Historical small-span speed claims fail on large required spans; later audits reject unconditional recovery-cost reductions.

MATHEMATICAL FORMULATION

$$Q_j = Q - B_j G = (k - B_j)G$$

Each proposed wrapped base B_j defines a shifted residual. Its assumed short search interval must actually contain k-B_j; historical span assumptions failed at high tiers.

EVIDENCE S129 / S152

PAPERS PAPER-002 / PAPER-012 / PAPER-026

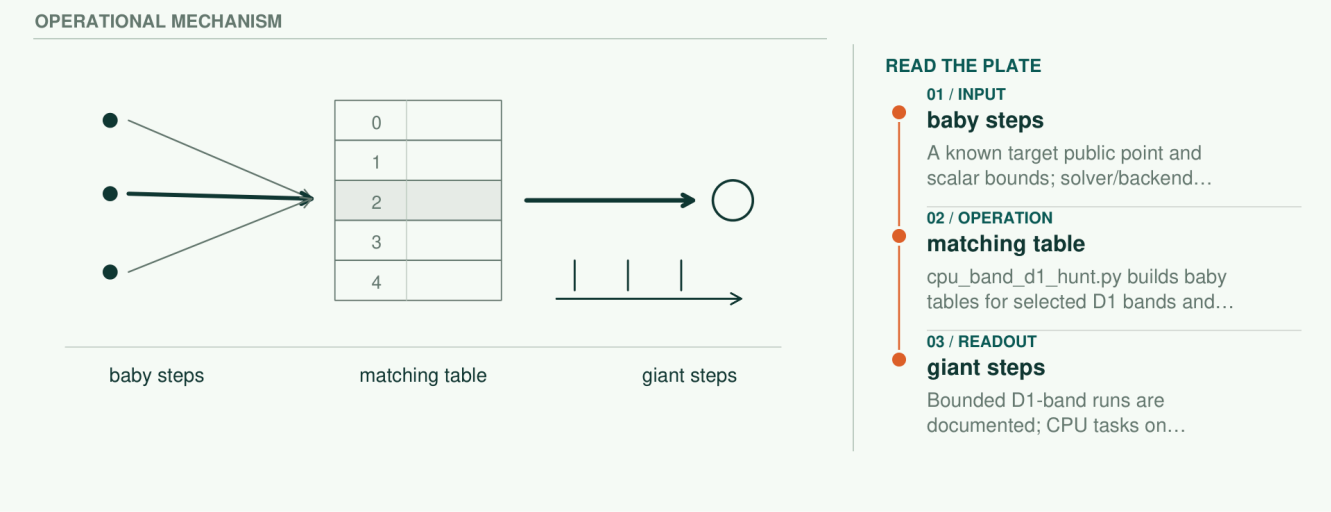
SECTION 02 / KANGAROO, INTERVAL ECDLP AND EXECUTION BACKENDS

CPU banded baby-step/giant-step D1 hunts

M028

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

BSGS stores small point multiples, then subtracts larger multiples from the target to find a match in a bounded interval.

INPUTS NEEDED

A known target public point and scalar bounds; solver/backend configuration. Address hashes alone do not supply the wild-walk point.

WHAT WAS ACTUALLY DONE

cpu_band_d1_hunt.py builds baby tables for selected D1 bands and scans giant steps. Saved deployments used CPU work even on some GPU-equipped hosts; that is not a CUDA benchmark.

WHAT THE RESULT MEANS

Bounded D1-band runs are documented; CPU tasks on GPU-equipped pods must not be described as GPU kernel execution.

MATHEMATICAL FORMULATION

$$Q' = (D - a)G, \quad D - a = im + j, \quad m = \lceil \sqrt{W} \rceil$$

For D in [a,a+W), match Q'-imG with jG using baby steps and giant steps. This is conditional on the chosen residual band.

EVIDENCE S223 / S132

PAPERS PAPER-010 / PAPER-021

SECTION 02 / KANGAROO, INTERVAL ECDLP AND EXECUTION BACKENDS

Prior-weighted stratified kangaroo

M029

EVIDENCE STATUS IMPLEMENTED

FIGURE / METHOD MECHANISM

HOW IT WORKS

Split a known-point interval into strata and allocate more tame-walk effort where a supplied prior assigns greater probability.

INPUTS NEEDED

A known target public point and scalar bounds; solver/backend configuration. Address hashes alone do not supply the wild-walk point.

WHAT WAS ACTUALLY DONE

pwsk.py builds a smoothed histogram, assigns budgets using prior mass and width, and searches against a shared wild herd. Its benefit depends on the supplied prior actually predicting targets.

WHAT THE RESULT MEANS

Reweights interval effort when a genuine prior exists; flat priors reduce to ordinary allocation. Prior quality is unestablished for the unknown puzzle.

MATHEMATICAL FORMULATION

$$\Pr(k \in I_i) = p_i, \quad \sum_i p_i = 1$$

A justified prior assigns probability masses to disjoint strata I_i and guides effort. The formula declares the required prior; it does not establish one for the puzzle.

EVIDENCE S116 / S091

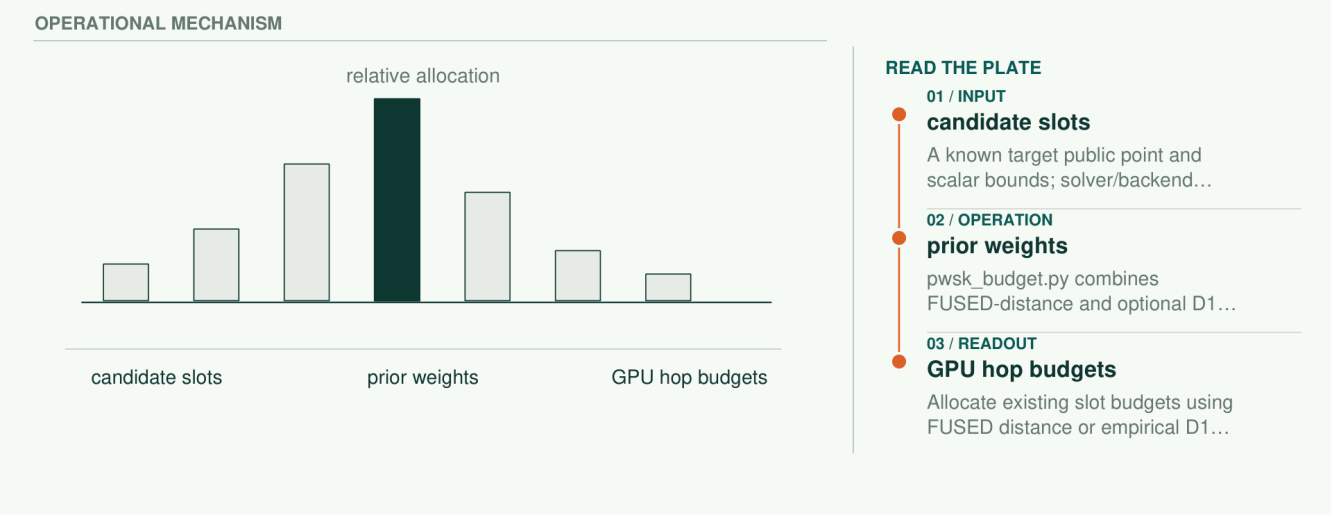
SECTION 02 / KANGAROO, INTERVAL ECDLP AND EXECUTION BACKENDS

Prior-weighted GPU slot budgets

M030

EVIDENCE STATUS IMPLEMENTED

FIGURE / METHOD MECHANISM



HOW IT WORKS

Reweight work across already constructed slots without changing what each slot represents. Higher prior mass receives a larger hop budget.

INPUTS NEEDED

A known target public point and scalar bounds; solver/backend configuration. Address hashes alone do not supply the wild-walk point.

WHAT WAS ACTUALLY DONE

pwsk_budget.py combines FUSED-distance and optional D1 histogram weights, rounds allocations and annotates manifests. It changes scheduling, not the set of possible scalar equations.

WHAT THE RESULT MEANS

Allocate existing slot budgets using FUSED distance or empirical D1 histograms without changing slot geometry.

MATHEMATICAL FORMULATION

$$B_i = B_{\mathrm{tot}} \frac{w_i}{\sum_j w_j}$$

Nonnegative heuristic weights w_i distribute a total budget; integer rounding must preserve the budget. Reweighting does not alter the underlying slot geometry.

EVIDENCE S117

PAPERS PAPER-004

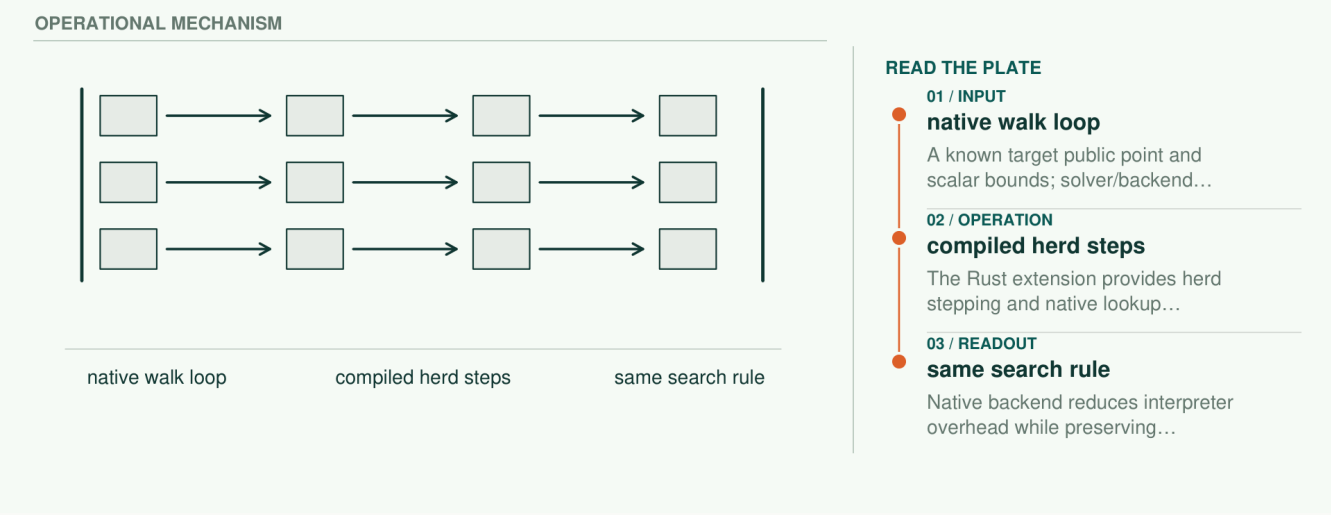
SECTION 02 / KANGAROO, INTERVAL ECDLP AND EXECUTION BACKENDS

Rust native herd stepping

M031

EVIDENCE STATUS IMPLEMENTED

FIGURE / METHOD MECHANISM



HOW IT WORKS

Move repeated curve-walk operations out of the Python interpreter into compiled native code. The mathematical search remains the same.

INPUTS NEEDED

A known target public point and scalar bounds; solver/backend configuration. Address hashes alone do not supply the wild-walk point.

WHAT WAS ACTUALLY DONE

The Rust extension provides herd stepping and native lookup operations called by the Python runtime. Actual speed must come from a build-specific benchmark, not merely the presence of Rust source.

WHAT THE RESULT MEANS

Native backend reduces interpreter overhead while preserving interval-search semantics.

MATHEMATICAL FORMULATION

$$(X, d) \longmapsto (X + s(X)G, d + s(X))$$

Native code performs the same herd transition as the reference walk. Performance changes must preserve point and distance agreement.

EVIDENCE S242 / S121

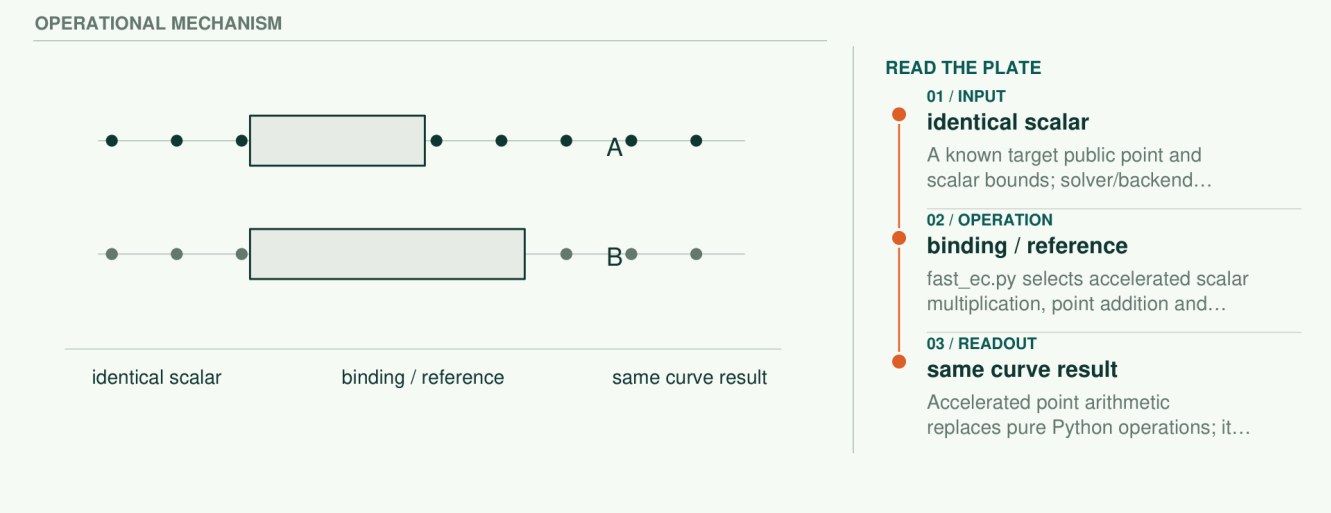
SECTION 02 / KANGAROO, INTERVAL ECDLP AND EXECUTION BACKENDS

Coincurve/libsecp256k1 fast backend

M032

EVIDENCE STATUS IMPLEMENTED

FIGURE / METHOD MECHANISM



HOW IT WORKS

Call optimized libsecp256k1 operations through a Python binding instead of performing every field operation in Python.

INPUTS NEEDED

A known target public point and scalar bounds; solver/backend configuration. Address hashes alone do not supply the wild-walk point.

WHAT WAS ACTUALLY DONE

fast_ec.py selects accelerated scalar multiplication, point addition and optional native herd routines. It is a backend substitution, not an algorithm that predicts private keys.

WHAT THE RESULT MEANS

Accelerated point arithmetic replaces pure Python operations; it does not reduce unknown key entropy.

MATHEMATICAL FORMULATION

$$\mathrm{mul}_{\mathrm{fast}}(k, G) = \mathrm{mul}_{\mathrm{ref}}(k, G)$$

Equality of curve results is the backend contract. Faster multiplication changes elapsed cost, not the number of candidate scalars.

EVIDENCE S106 / S081

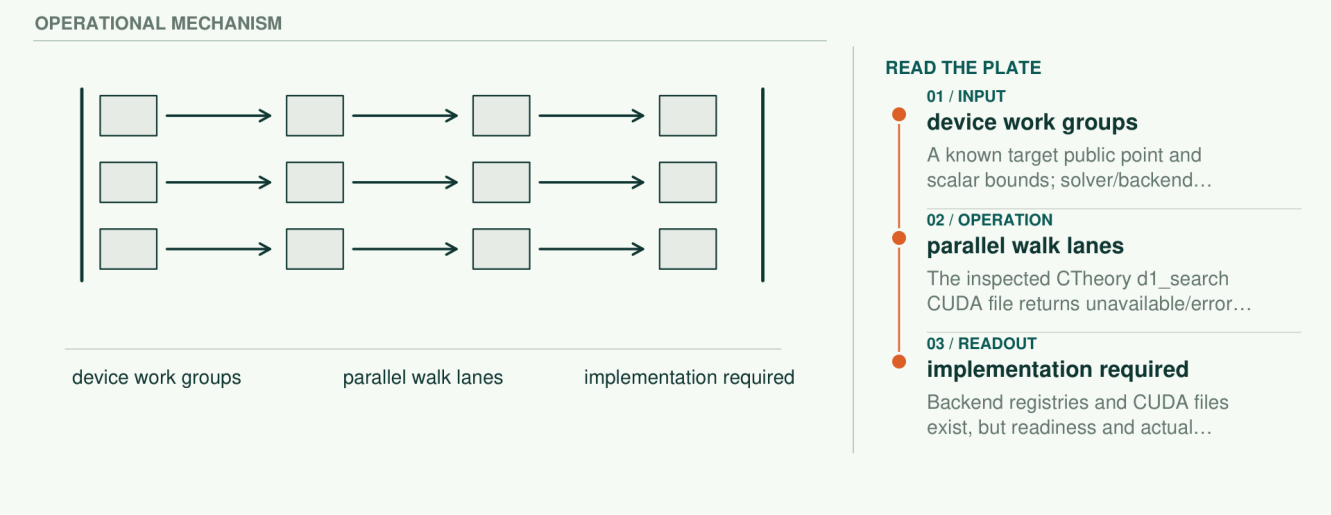
SECTION 02 / KANGAROO, INTERVAL ECDLP AND EXECUTION BACKENDS

GPU modular herd/D1 backends

M033

EVIDENCE STATUS IMPLEMENTATION/STUB MIX

FIGURE / METHOD MECHANISM



HOW IT WORKS

GPU kernels would execute many curve steps concurrently. A backend interface alone is insufficient; the device search must actually be implemented.

WHAT WAS ACTUALLY DONE

The inspected CTheory d1_search CUDA file returns unavailable/error and zeroes its output. The Python registry can report ready from markers/environment flags, which do not by themselves prove a working kernel.

INPUTS NEEDED

A known target public point and scalar bounds; solver/backend configuration. Address hashes alone do not supply the wild-walk point.

WHAT THE RESULT MEANS

Backend registries and CUDA files exist, but readiness and actual execution differ by module. Do not infer a complete device implementation from a registry.

MATHEMATICAL FORMULATION

$$\mathcal{W} = sqcup_{j=1}^{p} \mathcal{W}_j$$

Parallel workers receive work-unit sets W_j. This is an allocation model; actual device readiness and execution require separate evidence.

EVIDENCE S120 / S090

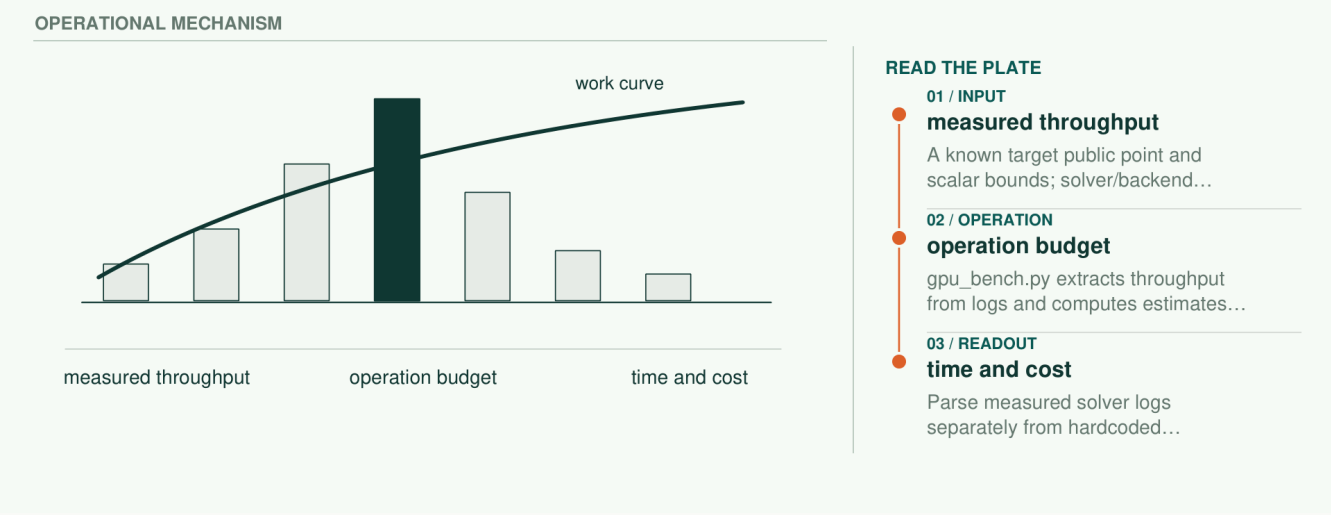
SECTION 02 / KANGAROO, INTERVAL ECDLP AND EXECUTION BACKENDS

GPU throughput and cost calibration

M034

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

Parse actual solver rates, discard warmup effects and combine measured work rates with declared operation counts.

INPUTS NEEDED

A known target public point and scalar bounds; solver/backend configuration. Address hashes alone do not supply the wild-walk point.

WHAT WAS ACTUALLY DONE

gpu_bench.py extracts throughput from logs and computes estimates. Other tables contain assumed rates/prices; those must not be confused with current hardware measurements or target success probabilities.

WHAT THE RESULT MEANS

Parse measured solver logs separately from hardcoded estimates; historical prices and throughput are not current quotations.

MATHEMATICAL FORMULATION

$$t = \frac{N_{\mathrm{ops}}}{v}, \quad \mathrm{cost} = r_{\mathrm{hour}} \frac{t}{3600}$$

Use measured operation throughput v and the dated hourly rate r. Historical estimates are conditional on their hardware and workload.

EVIDENCE S107 / S254

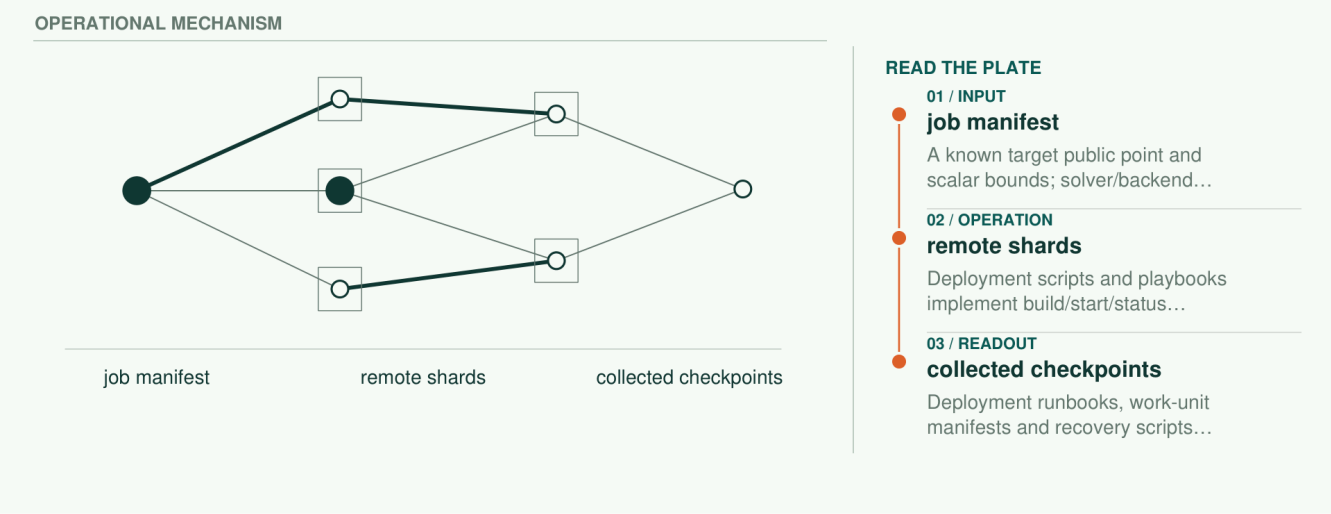
SECTION 02 / KANGAROO, INTERVAL ECDLP AND EXECUTION BACKENDS

RunPod/Vultr sharding and orchestration

M035

EVIDENCE STATUS IMPLEMENTED

FIGURE / METHOD MECHANISM



HOW IT WORKS

Partition solver jobs across remote workers, transfer manifests, run executables and collect outputs/checkpoints.

INPUTS NEEDED

A known target public point and scalar bounds; solver/backend configuration. Address hashes alone do not supply the wild-walk point.

WHAT WAS ACTUALLY DONE

Deployment scripts and playbooks implement build/start/status procedures. This review documents those artifacts without launching instances or treating a successful cloud allocation as a successful key search.

WHAT THE RESULT MEANS

Deployment runbooks, work-unit manifests and recovery scripts coordinate bounded jobs; availability is distinct from a successful search.

MATHEMATICAL FORMULATION

$$\mathcal{J}=\left\{(Q_j,L_j,U_j,\mathrm{id}_j)\right\}_{j=1}^{s}$$

A manifest assigns bounded jobs with stable identifiers for scheduling and recovery. Coverage depends on which jobs actually finish.

EVIDENCE S247 / S254

PAPERS PAPER-002 / PAPER-004 / PAPER-005 / PAPER-006 / PAPER-007 / PAPER-008 / PAPER-012 / PAPER-013 / PAPER-014 / PAPER-015 / PAPER-019 / PAPER-026

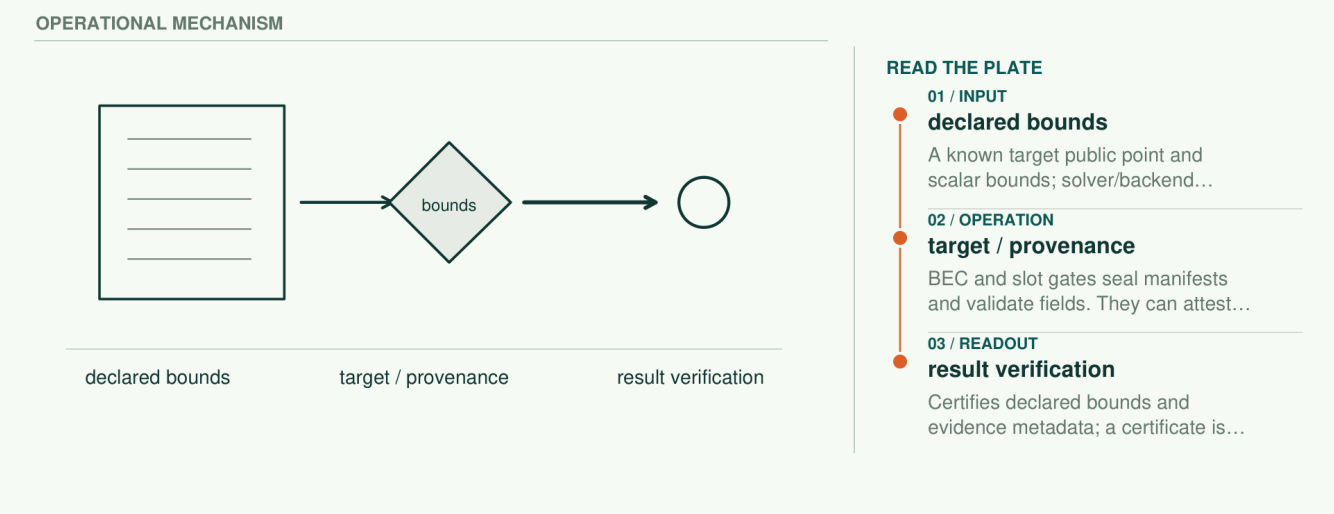
SECTION 02 / KANGAROO, INTERVAL ECDLP AND EXECUTION BACKENDS

Bounded ECDLP certificates and slot gates

M036

EVIDENCE STATUS IMPLEMENTED

FIGURE / METHOD MECHANISM



HOW IT WORKS

Attach a machine-readable statement of bounds, target, provenance and job identity so a result can be checked against its declared task.

INPUTS NEEDED

A known target public point and scalar bounds; solver/backend configuration. Address hashes alone do not supply the wild-walk point.

WHAT WAS ACTUALLY DONE

BEC and slot gates seal manifests and validate fields. They can attest what was searched, but cannot certify that a heuristic guess contains the unknown key.

WHAT THE RESULT MEANS

Certifies declared bounds and evidence metadata; a certificate is not proof that the target lies in a guessed slot.

MATHEMATICAL FORMULATION

$$\mathcal{C} = (Q, L, U, \text{provenance}), \quad \text{verify}(k) : kG = Q$$

A certificate records the input contract and evidence. Recording bounds does not prove the unknown target is inside a guessed interval.

EVIDENCE S094 / S123

PAPERS PAPER-009 / PAPER-010 / PAPER-011

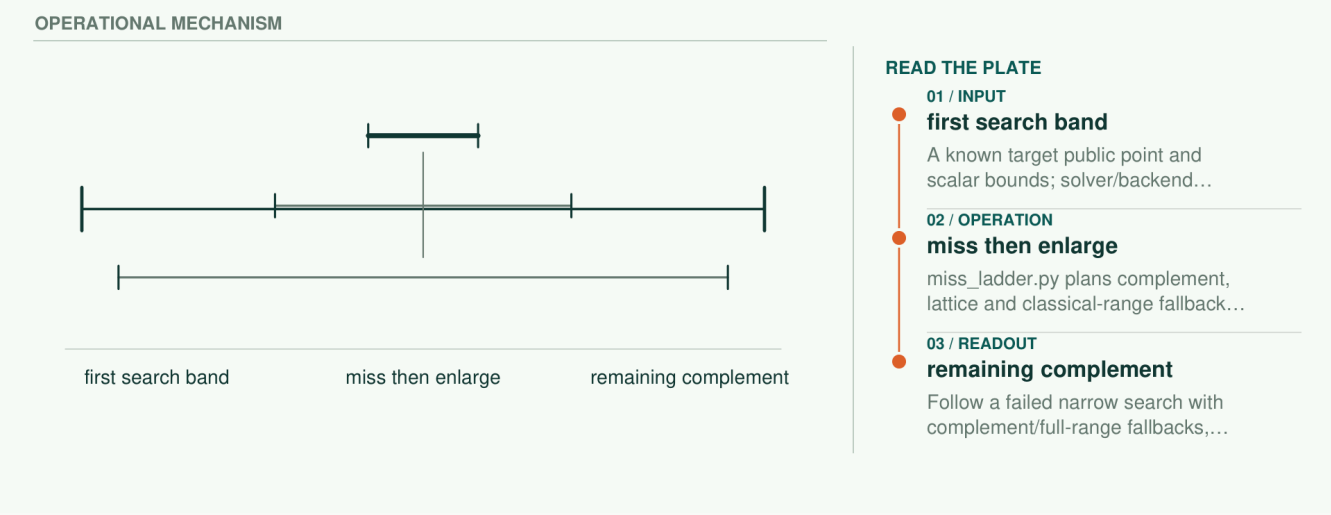
SECTION 02 / KANGAROO, INTERVAL ECDLP AND EXECUTION BACKENDS

Miss ladder and complement widening

M037

EVIDENCE STATUS IMPLEMENTED

FIGURE / METHOD MECHANISM



HOW IT WORKS

After a prioritized search misses, enlarge the hypothesis set in an explicit order rather than silently assuming the key was impossible.

INPUTS NEEDED

A known target public point and scalar bounds; solver/backend configuration. Address hashes alone do not supply the wild-walk point.

WHAT WAS ACTUALLY DONE

miss_ladder.py plans complement, lattice and classical-range fallbacks. Following the ladder spends additional work; it does not retroactively make the first narrow guess complete.

WHAT THE RESULT MEANS

Follow a failed narrow search with complement/full-range fallbacks, rather than treating heuristic misses as exclusions.

MATHEMATICAL FORMULATION

$$B_1 \subseteq B_2 \subseteq \cdots \subseteq I, \quad E_j = B_j \setminus B_{j-1}$$

A miss ladder searches newly admitted regions E_j as bounds widen. Exhaustion of one region excludes only that region.

EVIDENCE S111

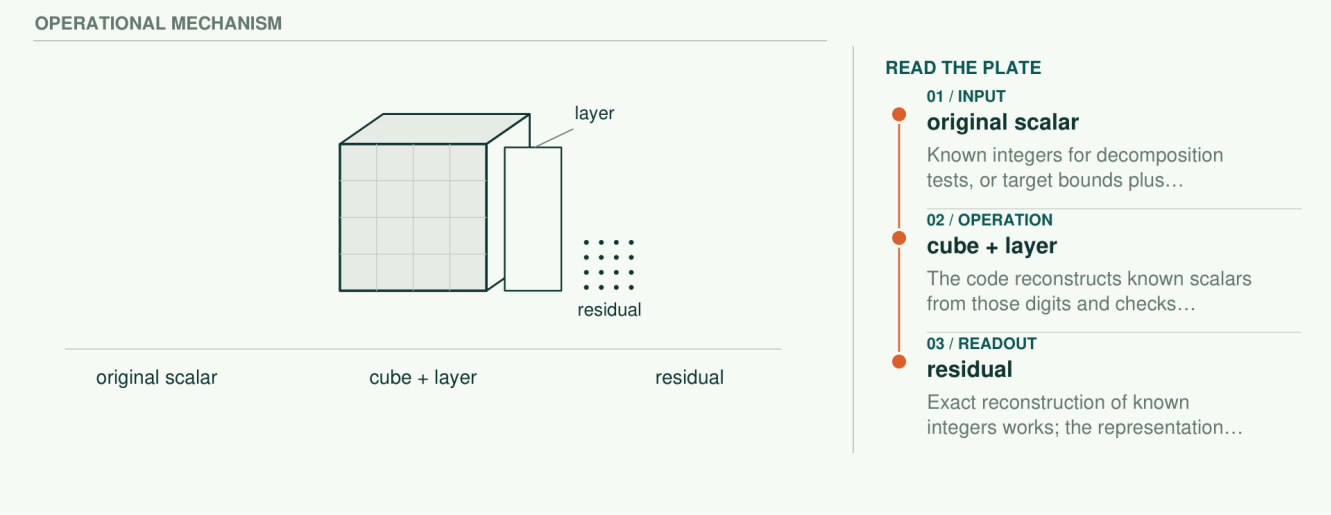
SECTION 03 / CTHEORY DECOMPOSITIONS AND SCALAR GEOMETRY

Cubic scalar decomposition

M038

EVIDENCE STATUS AUDITED/CORRECTED

FIGURE / METHOD MECHANISM



HOW IT WORKS

Set R1=floor(cuberoot(k)), divide k-R1^3 by R1^2, and retain quotient C1 and remainder D1. This exactly re-encodes k.

INPUTS NEEDED

Known integers for decomposition tests, or target bounds plus hypothesized root/coefficient digits; a public point for shifted-point searches.

WHAT WAS ACTUALLY DONE

The code reconstructs known scalars from those digits and checks bounds. For an unknown key the digits are also unknown; an exact identity does not disclose them from the address.

WHAT THE RESULT MEANS

Exact reconstruction of known integers works; the representation itself supplies no unknown digits or entropy reduction.

MATHEMATICAL FORMULATION

$$R = \lfloor \sqrt[3]{k} \rfloor, \quad k = R^3 + CR^2 + D, \quad 0 \le D < R^2$$

C and D are the quotient and remainder of k-R^3 divided by R^2. The identity uniquely re-encodes a known positive integer.

EVIDENCE S099 / S135

PAPERS PAPER-002 / PAPER-004 / PAPER-005 / PAPER-006 / PAPER-007 / PAPER-008

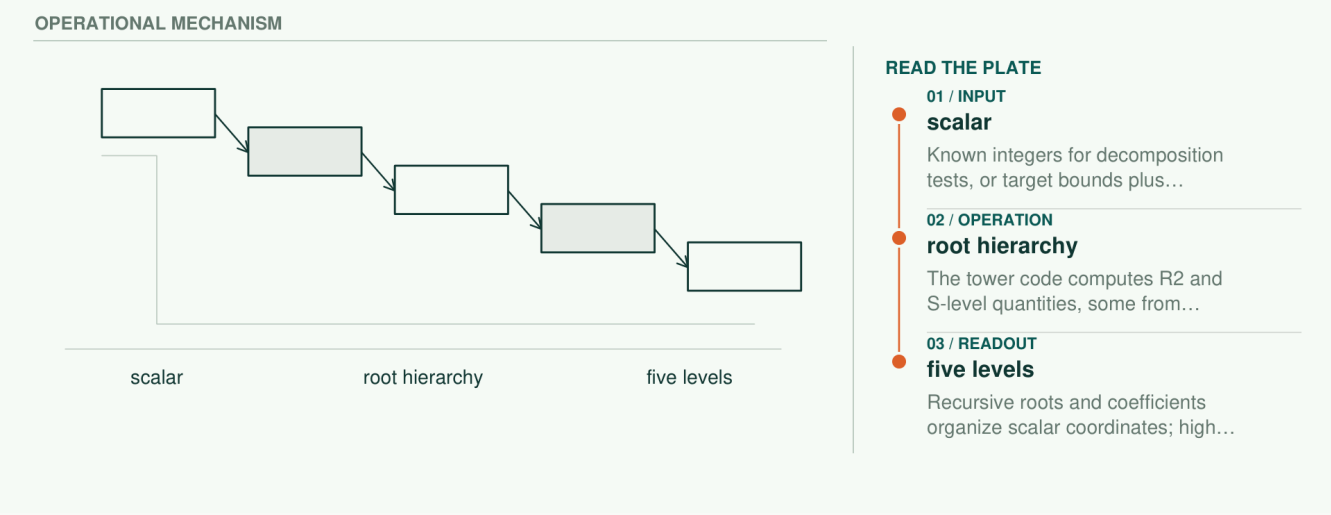
SECTION 03 / CTHEORY DECOMPOSITIONS AND SCALAR GEOMETRY

Five-level R1/R2/S3/S4/S5 tower

M039

EVIDENCE STATUS IMPLEMENTED

FIGURE / METHOD MECHANISM



HOW IT WORKS

Reapply root/division decompositions to R1 and later components, producing a hierarchy of smaller coordinates.

INPUTS NEEDED

Known integers for decomposition tests, or target bounds plus hypothesized root/coefficient digits; a public point for shifted-point searches.

WHAT WAS ACTUALLY DONE

The tower code computes R2 and S-level quantities, some from bit-length formulas. Its tables describe known-key coordinates or target-size anchors, not independently observed secret digits.

WHAT THE RESULT MEANS

Recursive roots and coefficients organize scalar coordinates; high levels often depend only on advertised bit width.

MATHEMATICAL FORMULATION

$$Y_{j-1} = Y_j^3 + c_j Y_j^2 + d_j, \quad Y_0 = k, \quad 1 \le j \le 5$$

Y1,Y2,Y3,Y4,Y5 correspond to R1,R2,S3,S4,S5. Each Y_j is the floor cube root of the previous value; retain all remainders for reconstruction.

EVIDENCE S219

PAPERS PAPER-002

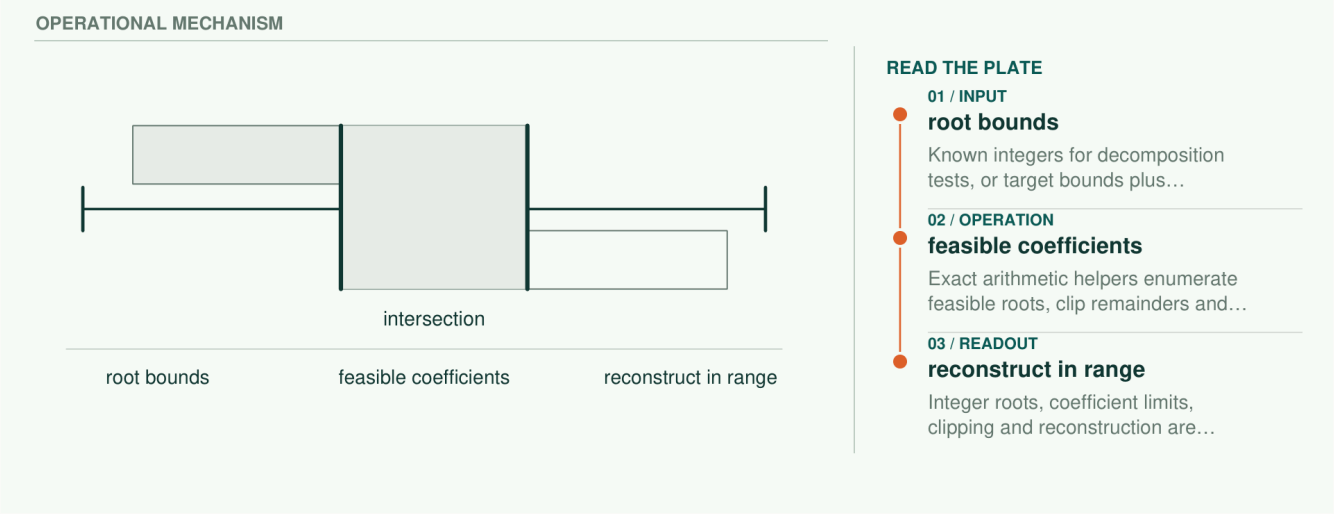
SECTION 03 / CTHEORY DECOMPOSITIONS AND SCALAR GEOMETRY

Exact root bounds and coefficient feasibility

M040

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

Integer roots and inequality bounds determine which digit combinations could reconstruct an integer inside the advertised interval.

INPUTS NEEDED

Known integers for decomposition tests, or target bounds plus hypothesized root/coefficient digits; a public point for shifted-point searches.

WHAT WAS ACTUALLY DONE

Exact arithmetic helpers enumerate feasible roots, clip remainders and verify reconstruction. They reject impossible combinations, while still retaining every feasible unknown scalar unless a separate heuristic cuts further.

WHAT THE RESULT MEANS

Integer roots, coefficient limits, clipping and reconstruction are tested arithmetically.

MATHEMATICAL FORMULATION

$$C_{\max}(R) = \left\lfloor \frac{(R+1)^3 - R^3 - 1}{R^2} \right\rfloor$$

The maximum coefficient comes from the last integer before the next cube. In particular, C=3 is possible even when R is large.

EVIDENCE S100 / S087

PAPERS PAPER-001

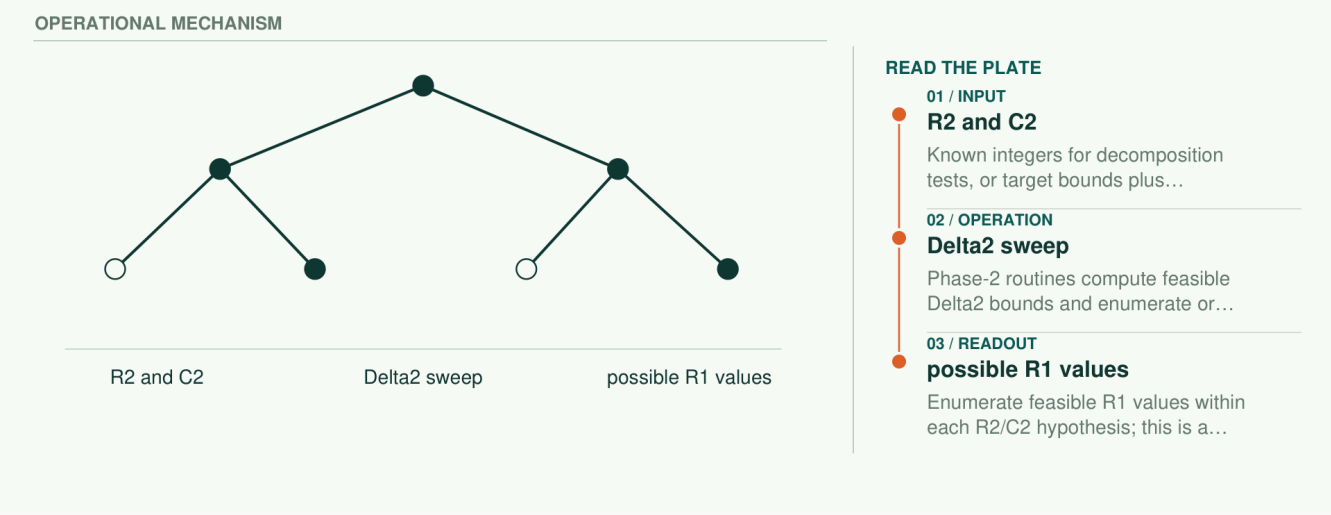
SECTION 03 / CTHEORY DECOMPOSITIONS AND SCALAR GEOMETRY

Phase-2 Delta2 enumeration

M041

EVIDENCE STATUS IMPLEMENTED

FIGURE / METHOD MECHANISM



HOW IT WORKS

Given R2 and C2, each Delta2 selects an R1 through R1=R2^3+C2*R2^2+Delta2.

WHAT WAS ACTUALLY DONE

Phase-2 routines compute feasible Delta2 bounds and enumerate or stride through them. The smaller-looking parameter still represents many distinct scalar bases that may each require a D1 search.

INPUTS NEEDED

Known integers for decomposition tests, or target bounds plus hypothesized root/coefficient digits; a public point for shifted-point searches.

WHAT THE RESULT MEANS

Enumerate feasible R1 values within each R2/C2 hypothesis; this is a search coordinate, not an independently observed point.

MATHEMATICAL FORMULATION

$$R_1 = R_2^3 + C_2 R_2^2 + \Delta_2, \quad 0 \le \Delta_2 < R_2^2$$

Enumerate feasible Delta2 values for each R2,C2 and intersect the resulting R1 possibilities with puzzle-range constraints.

EVIDENCE S114

PAPERS PAPER-006 / PAPER-011 / PAPER-013

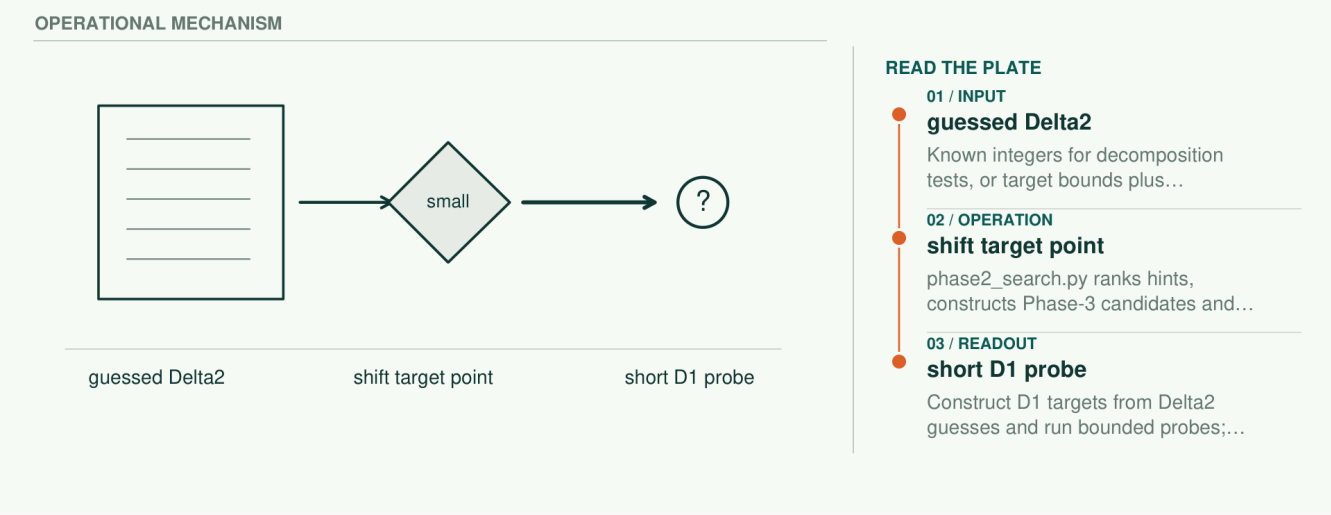
SECTION 03 / CTHEORY DECOMPOSITIONS AND SCALAR GEOMETRY

Phase-2 public-key-guided light probes

M042

EVIDENCE STATUS IMPLEMENTED

FIGURE / METHOD MECHANISM



HOW IT WORKS

For a guessed Delta2, build the corresponding shifted public point and test a small D1 budget before investing in a full leaf hunt.

INPUTS NEEDED

Known integers for decomposition tests, or target bounds plus hypothesized root/coefficient digits; a public point for shifted-point searches.

WHAT WAS ACTUALLY DONE

phase2_search.py ranks hints, constructs Phase-3 candidates and optionally probes them. It does not extract Delta2 directly from Q; every probe remains conditional on its guessed coordinates.

WHAT THE RESULT MEANS

Construct D1 targets from Delta2 guesses and run bounded probes; no independent Delta2 oracle is obtained.

MATHEMATICAL FORMULATION

$$Q_D = Q - (R_1^3 + C_1 R_1^2)G$$

A Delta2 hypothesis determines R1 and hence this residual target. A short D1 probe tests that branch; Delta2 itself is not an independently supplied discrete log.

EVIDENCE S115

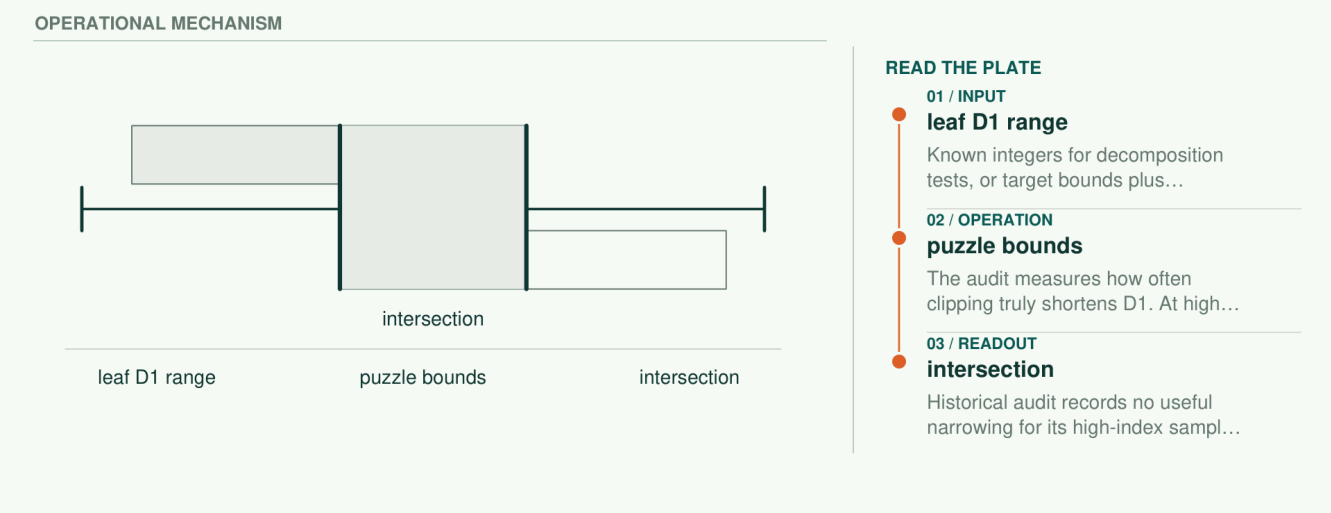
SECTION 03 / CTHEORY DECOMPOSITIONS AND SCALAR GEOMETRY

Puzzle-bound clipping of D1

M043

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

Intersect a leaf's D1 limits with the public puzzle bounds after subtracting its proposed base.

WHAT WAS ACTUALLY DONE

The audit measures how often clipping truly shortens D1. At high indices the public interval often contains the whole leaf, so the original D1 uncertainty remains.

INPUTS NEEDED

Known integers for decomposition tests, or target bounds plus hypothesized root/coefficient digits; a public point for shifted-point searches.

WHAT THE RESULT MEANS

Historical audit records no useful narrowing for its high-index sample; advertised puzzle bounds are too broad at this leaf scale.

MATHEMATICAL FORMULATION

$$D_{\min} = \max(0, L - B), \quad D_{\max} = \min(R_1^2 - 1, U - B)$$

B=R1^3+C1 R1^2. Clipping the D1 interval uses inclusive puzzle bounds [L,U]; an empty interval rejects the branch.

EVIDENCE S152

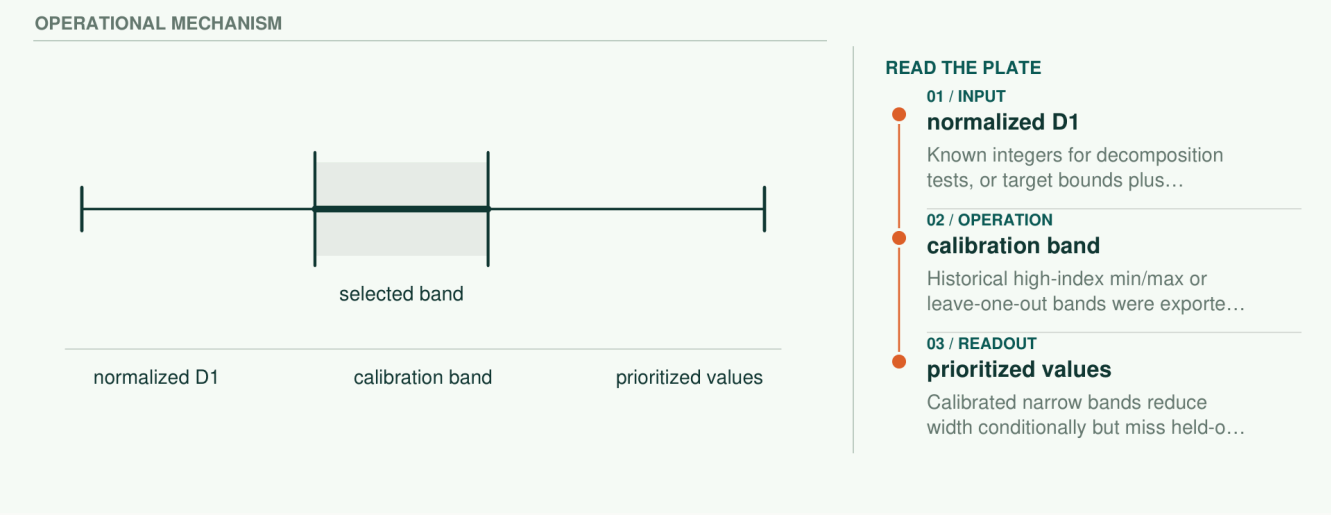
SECTION 03 / CTHEORY DECOMPOSITIONS AND SCALAR GEOMETRY

D1 high-index ratio bands

M044

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

Estimate normalized D1 positions from solved examples and prioritize a common ratio band for new slots.

INPUTS NEEDED

Known integers for decomposition tests, or target bounds plus hypothesized root/coefficient digits; a public point for shifted-point searches.

WHAT WAS ACTUALLY DONE

Historical high-index min/max or leave-one-out bands were exported as search presets. Their reduced width is real for the chosen band, but held-out misses show it is not guaranteed containment.

WHAT THE RESULT MEANS

Calibrated narrow bands reduce width conditionally but miss held-out keys; their complement is required for complete support.

MATHEMATICAL FORMULATION

$$\rho = \frac{D_1}{R_1^2}, \quad D_1 \in [\lceil aR_1^2 \rceil, \lfloor bR_1^2 \rfloor]$$

A calibrated ratio band [a,b] induces an integer D1 band, intersected with legal bounds. Its held-out containment must be measured.

EVIDENCE S152

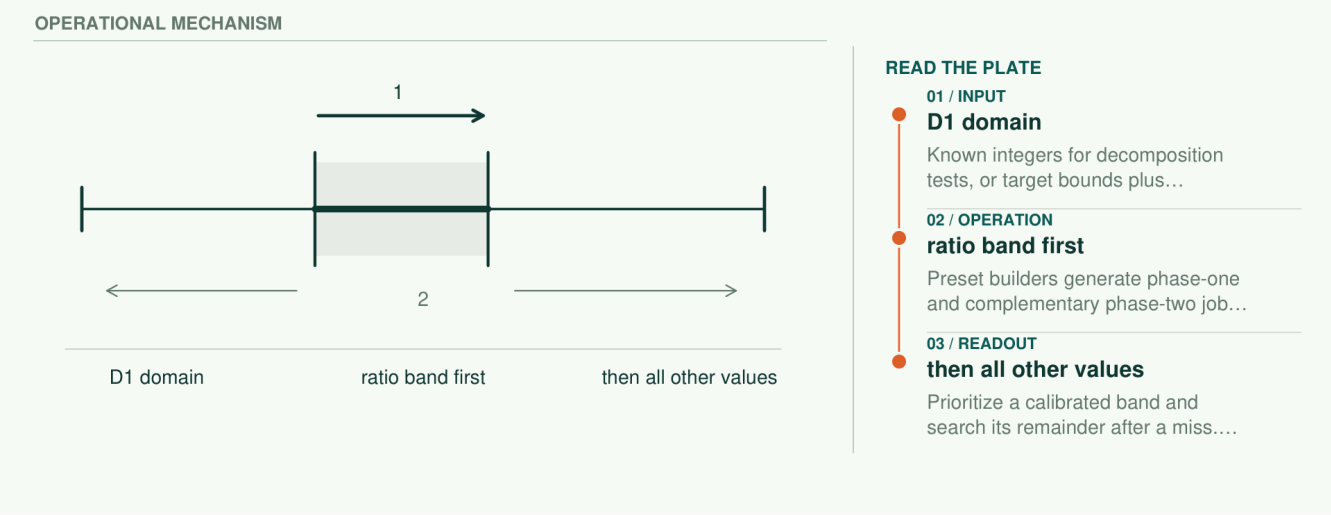
SECTION 03 / CTHEORY DECOMPOSITIONS AND SCALAR GEOMETRY

Two-phase D1 band/complement policy

M045

EVIDENCE STATUS IMPLEMENTED

FIGURE / METHOD MECHANISM



HOW IT WORKS

Search the ratio band first; if it misses, search all remaining D1 values. This preserves eventual full leaf coverage.

INPUTS NEEDED

Known integers for decomposition tests, or target bounds plus hypothesized root/coefficient digits; a public point for shifted-point searches.

WHAT WAS ACTUALLY DONE

Preset builders generate phase-one and complementary phase-two jobs. The combined policy restores eligibility but can cost more than a single unpartitioned interval search.

WHAT THE RESULT MEANS

Prioritize a calibrated band and search its remainder after a miss. Union coverage restores full range rather than reducing entropy.

MATHEMATICAL FORMULATION

$$\mathcal{D} = B \dot{\cup} (\mathcal{D} \setminus B)$$

First search the selected D1 band B, then the rest of the legal D1 set. The union, rather than the initial band alone, supplies full coverage.

EVIDENCE S101 / S152

PAPERS PAPER-002 / PAPER-004 / PAPER-005 / PAPER-006 / PAPER-007 / PAPER-008 / PAPER-012 / PAPER-013 / PAPER-026

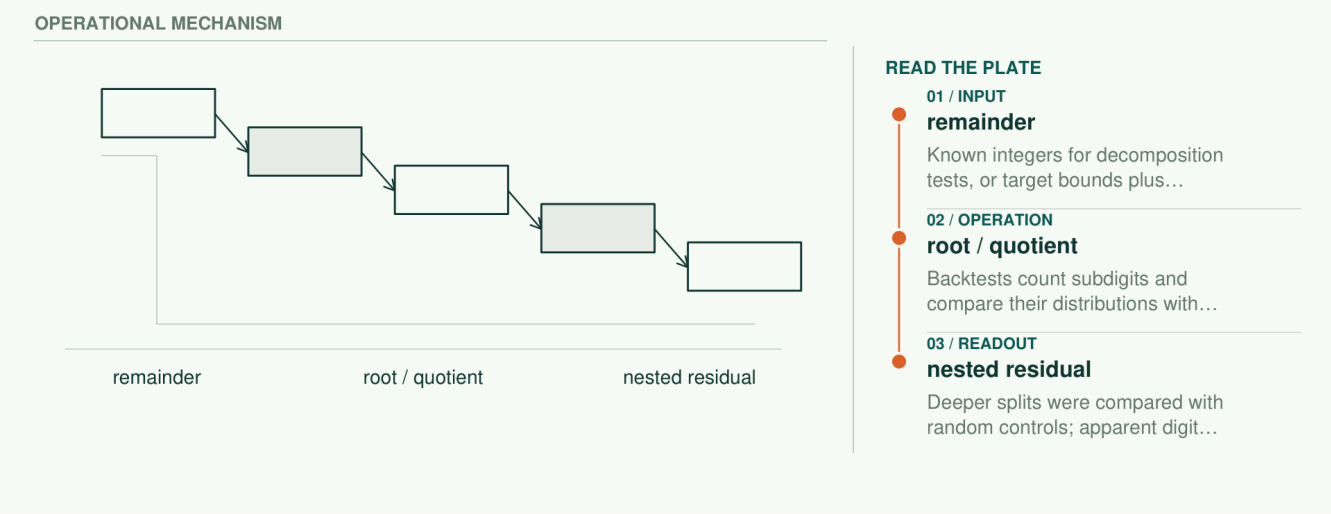
SECTION 03 / CTHEORY DECOMPOSITIONS AND SCALAR GEOMETRY

Recursive remainder/sub-digit decomposition

M046

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

Decompose a remainder again into its own root, coefficient and residual, then repeat recursively.

WHAT WAS ACTUALLY DONE

Backtests count subdigits and compare their distributions with equally transformed random integers. Most apparent simplification is a change of coordinates; omitted subdigits remain unknown.

INPUTS NEEDED

Known integers for decomposition tests, or target bounds plus hypothesized root/coefficient digits; a public point for shifted-point searches.

WHAT THE RESULT MEANS

Deeper splits were compared with random controls; apparent digit biases can arise from the map itself.

MATHEMATICAL FORMULATION

$$d = u^3 + au^2 + \epsilon, \quad u = \lfloor \sqrt[3]{d} \rfloor, \quad 0 \le \epsilon < u^2$$

Apply the same decomposition to a positive remainder d; handle d=0 separately. The tuple retains the original remainder's information.

EVIDENCE S153

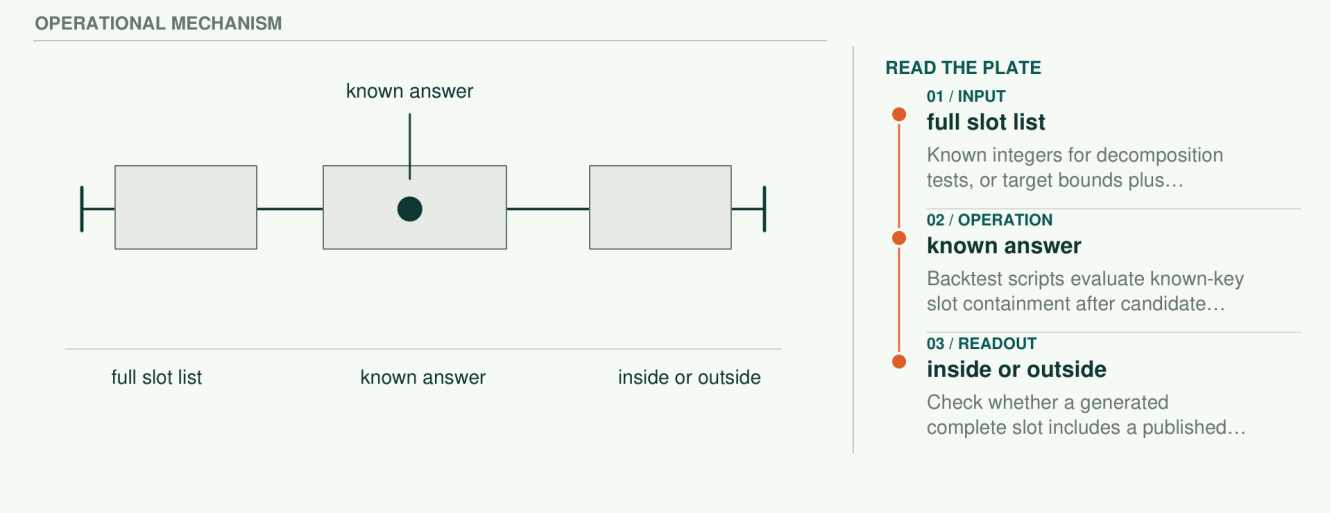
SECTION 03 / CTHEORY DECOMPOSITIONS AND SCALAR GEOMETRY

All-known-key slot containment

M047

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

A complete hypothesis includes every base digit needed to define the final searched interval. Test whether the published answer lies in that interval.

INPUTS NEEDED

Known integers for decomposition tests, or target bounds plus hypothesized root/coefficient digits; a public point for shifted-point searches.

WHAT WAS ACTUALLY DONE

Backtest scripts evaluate known-key slot containment after candidate construction. Knowing the answer for evaluation must not leak into hint generation or be reported as an actual cryptographic recovery.

WHAT THE RESULT MEANS

Check whether a generated complete slot includes a published answer; containment is not independent discovery of that answer.

MATHEMATICAL FORMULATION

$$H = \sum_{i=1}^{m} 1\{k_i \in \mathcal{S}_i\}$$

A slot-containment test counts published keys inside their generated scalar sets S_i. Inclusion is different from independently finding those keys.

EVIDENCE S080 / S136

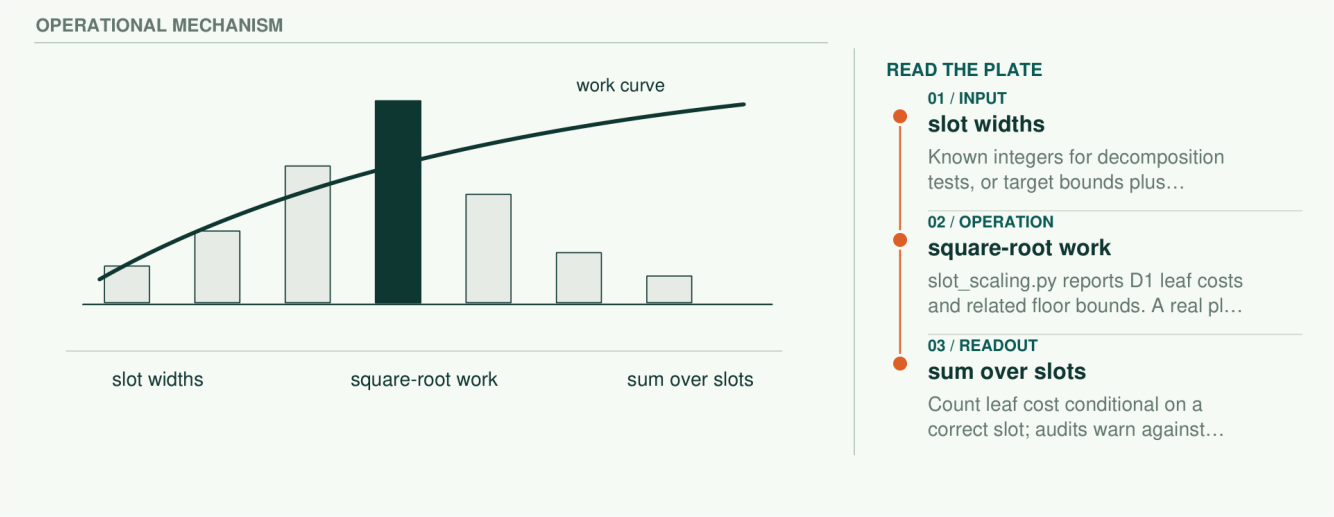
SECTION 03 / CTHEORY DECOMPOSITIONS AND SCALAR GEOMETRY

Post-slot square-root cost scaling

M048

EVIDENCE STATUS IMPLEMENTED

FIGURE / METHOD MECHANISM



HOW IT WORKS

Kangaroo leaf work scales roughly with the square root of its interval width, conditional on the correct target interval.

INPUTS NEEDED

Known integers for decomposition tests, or target bounds plus hypothesized root/coefficient digits; a public point for shifted-point searches.

WHAT WAS ACTUALLY DONE

slot_scaling.py reports D1 leaf costs and related floor bounds. A real plan must additionally count all root/remainder hypotheses and its probability of including the correct leaf.

WHAT THE RESULT MEANS

Count leaf cost conditional on a correct slot; audits warn against omitting the number of unknown slots.

MATHEMATICAL FORMULATION

$$T_{\text{total}} \asymp \sum_{j=1}^{s} \sqrt{W_j}$$

W_j is the width of each separately searched residual interval. Counting only one leaf omits the work of finding the correct branch.

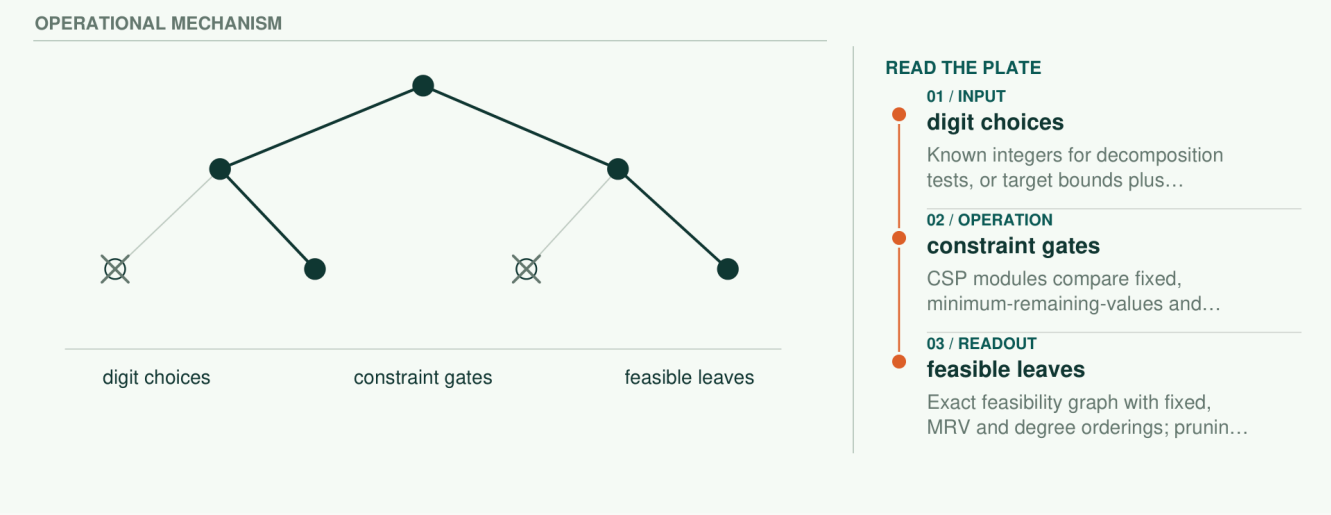
EVIDENCE S124

SECTION 03 / CTHEORY DECOMPOSITIONS AND SCALAR GEOMETRY

Constraint-satisfaction slot construction M049

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

Express digit domains and interval inequalities as constraints, then select variable orderings to prune impossible partial assignments.

INPUTS NEEDED

Known integers for decomposition tests, or target bounds plus hypothesized root/coefficient digits; a public point for shifted-point searches.

WHAT WAS ACTUALLY DONE

CSP modules compare fixed, minimum-remaining-values and degree strategies. They reduce wasted inconsistent tuples but do not add new information about which consistent tuple is the secret.

WHAT THE RESULT MEANS

Exact feasibility graph with fixed, MRV and degree orderings; pruning invalid tuples does not predict the valid target tuple.

MATHEMATICAL FORMULATION

$$\mathcal{F} = \{\theta : g_a(\theta) \text{ holds for all } a\}$$

A constraint solver retains tuples theta satisfying all declared arithmetic/range constraints. Variable ordering changes traversal cost, not which feasible tuple is the target.

EVIDENCE S098 / S150

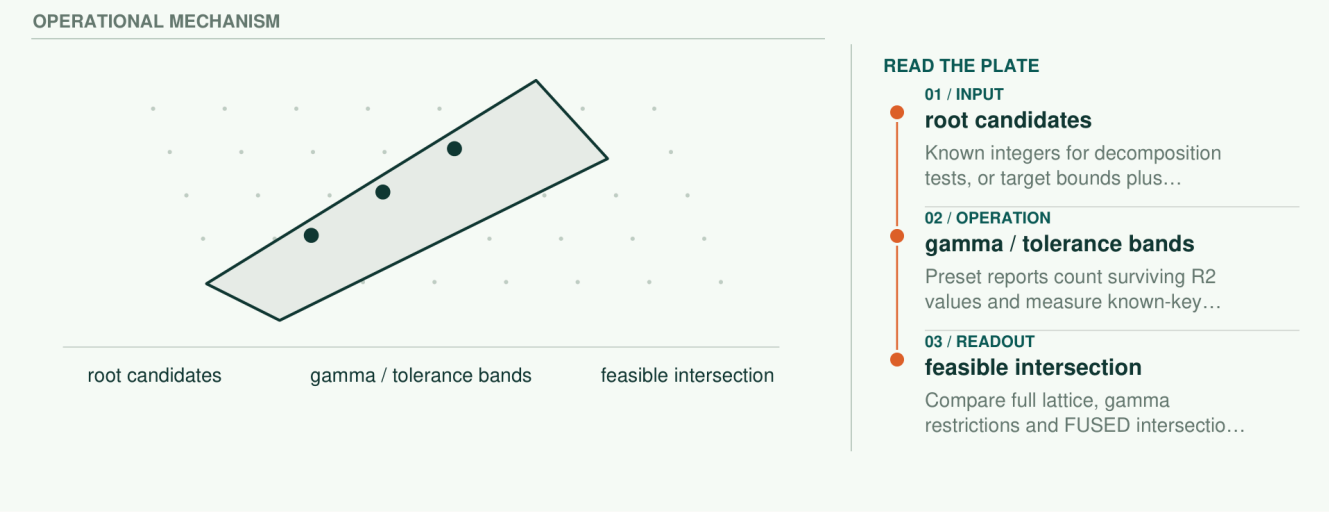
SECTION 03 / CTHEORY DECOMPOSITIONS AND SCALAR GEOMETRY

Gamma-band and lattice narrowing presets

M050

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

Intersect feasible roots with selected gamma bands, FUSED tolerance bands or a two-stage union.

WHAT WAS ACTUALLY DONE

Preset reports count surviving R2 values and measure known-key coverage. A large candidate reduction can simply mean many true keys were excluded by the heuristic.

INPUTS NEEDED

Known integers for decomposition tests, or target bounds plus hypothesized root/coefficient digits; a public point for shifted-point searches.

WHAT THE RESULT MEANS

Compare full lattice, gamma restrictions and FUSED intersections; stronger cuts reduce measured target coverage.

MATHEMATICAL FORMULATION

$$\mathcal{R}_\gamma = \left\{ S_3^3 + \gamma S_3^2 + \delta : 0 \le \delta < S_3^2 \right\}$$

A fixed anchor and coefficient define an R2 band. Intersect with canonical-root, puzzle-range and predictor bounds before claiming coverage.

EVIDENCE S183
PAPERS PAPER-007

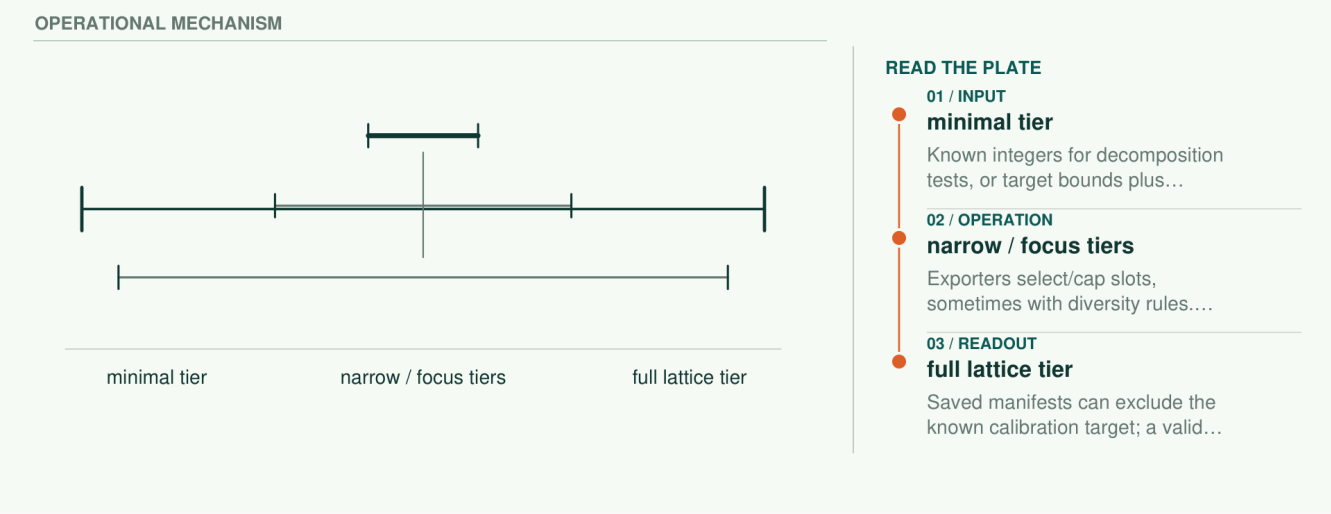
SECTION 03 / CTHEORY DECOMPOSITIONS AND SCALAR GEOMETRY

Tiered minimal/narrow/focus/lattice exports

M051

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

Package different breadths of the same hypothesis family as minimal, narrow, focus or full-lattice tiers.

WHAT WAS ACTUALLY DONE

Exporters select/cap slots, sometimes with diversity rules. Saved calibration reports explicitly check whether the known true R2 and full tuple were omitted from each exported job set.

INPUTS NEEDED

Known integers for decomposition tests, or target bounds plus hypothesized root/coefficient digits; a public point for shifted-point searches.

WHAT THE RESULT MEANS

Saved manifests can exclude the known calibration target; a valid export is not proof of target inclusion.

MATHEMATICAL FORMULATION

$$\mathcal{S}_{\mathrm{preset}} = cup_{j \in J_{\mathrm{preset}}} [L_j, U_j]$$

Each export preset selects a job union. Calculate its scalar mass and known-key containment; the preset name does not imply guaranteed inclusion.

EVIDENCE S151

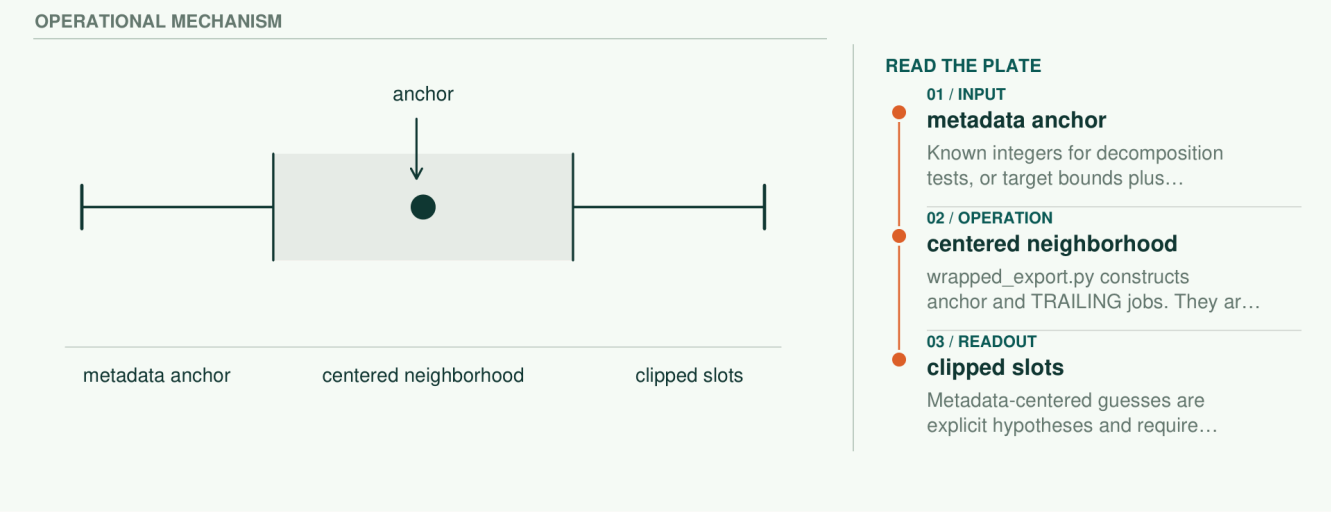
SECTION 03 / CTHEORY DECOMPOSITIONS AND SCALAR GEOMETRY

Anchor/TRAILING-centered slot construction

M052

EVIDENCE STATUS IMPLEMENTED

FIGURE / METHOD MECHANISM



HOW IT WORKS

Use a supplied metadata number as a plausible center, or derive CTheory coordinates from it to define nearby slots.

INPUTS NEEDED

Known integers for decomposition tests, or target bounds plus hypothesized root/coefficient digits; a public point for shifted-point searches.

WHAT WAS ACTUALLY DONE

wrapped_export.py constructs anchor and TRAILING jobs. They are guesses informed by an artifact, not proven high bits or offsets of the target scalar.

WHAT THE RESULT MEANS

Metadata-centered guesses are explicit hypotheses and require cryptographic verification.

MATHEMATICAL FORMULATION

$$\mathcal{B}(a,w)=\{k:|k-a|\le w\}\cap I$$

Metadata anchor a and radius w propose a scalar neighborhood. Its relation to the unknown scalar is a hypothesis requiring exact verification.

EVIDENCE S128

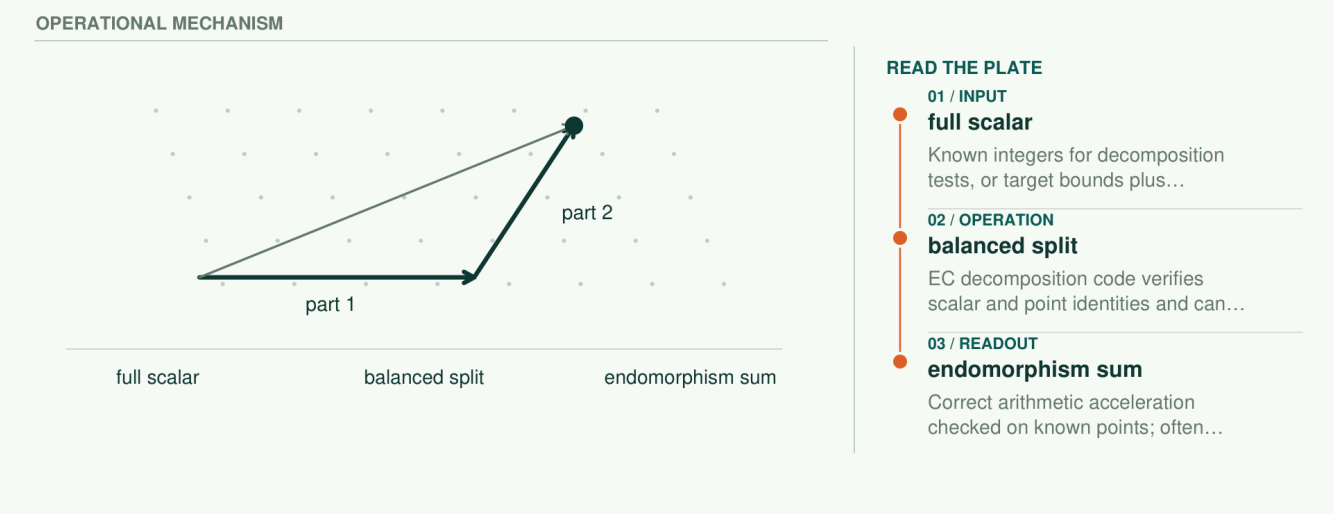
SECTION 03 / CTHEORY DECOMPOSITIONS AND SCALAR GEOMETRY

GLV scalar decomposition and endomorphism

M053

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

The secp256k1 endomorphism permits k congruent to r1+lambda*r2 modulo group order, with a related point map.

INPUTS NEEDED

Known integers for decomposition tests, or target bounds plus hypothesized root/coefficient digits; a public point for shifted-point searches.

WHAT WAS ACTUALLY DONE

EC decomposition code verifies scalar and point identities and can accelerate multiplication. Computing a split of a known scalar does not recover that scalar from a public point.

WHAT THE RESULT MEANS

Correct arithmetic acceleration checked on known points; often degenerate for small scalars and not a generic entropy cut.

MATHEMATICAL FORMULATION

$$k \equiv k_1 + \lambda k_2 \pmod{n}, \quad kG = k_1 G + k_2 \phi(G)$$

GLV uses the curve endomorphism phi with phi(G)=lambda G to accelerate arithmetic. The decomposition does not supply a new target equation.

EVIDENCE S202 / S139

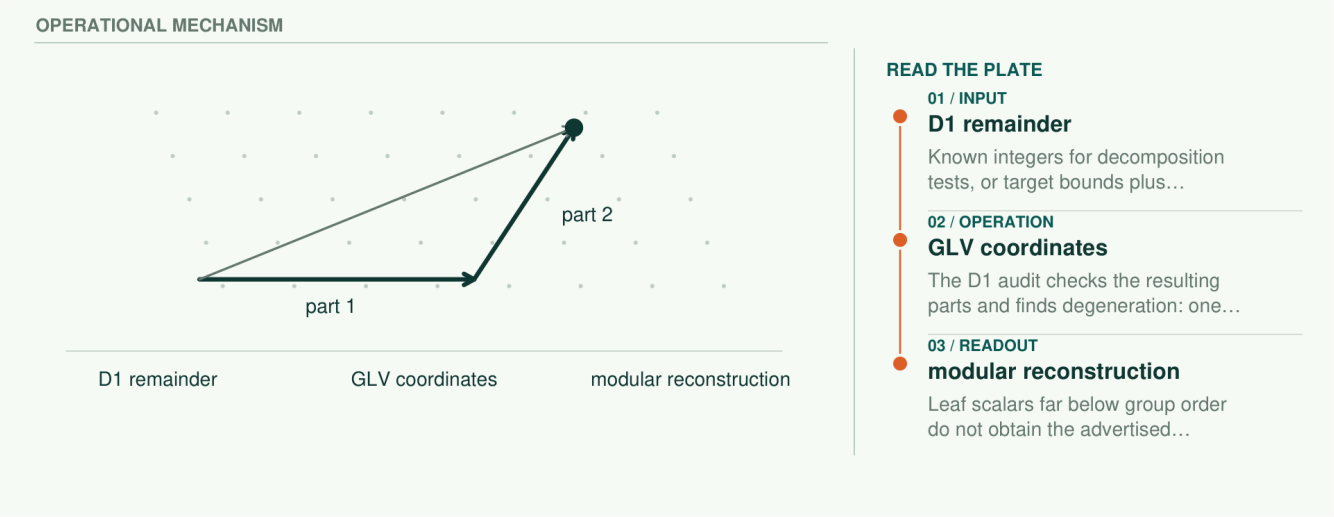
SECTION 03 / CTHEORY DECOMPOSITIONS AND SCALAR GEOMETRY

GLV splitting specifically on D1

M054

EVIDENCE STATUS AUDITED/CORRECTED

FIGURE / METHOD MECHANISM



HOW IT WORKS

Attempt to apply the same balanced GLV split to the much smaller D1 scalar instead of a full-width scalar.

WHAT WAS ACTUALLY DONE

The D1 audit checks the resulting parts and finds degeneration: one component can retain effectively the original small scalar. A nominal two-part representation supplies no automatic search-width reduction.

INPUTS NEEDED

Known integers for decomposition tests, or target bounds plus hypothesized root/coefficient digits; a public point for shifted-point searches.

WHAT THE RESULT MEANS

Leaf scalars far below group order do not obtain the advertised balanced split; no free narrowing follows.

MATHEMATICAL FORMULATION

$$D_1 \equiv d_a + \lambda d_b \pmod{n}$$

Applying GLV to a small leaf D1 is the same modular scalar representation; a balanced small pair is not automatically obtained for that leaf scale.

EVIDENCE S152

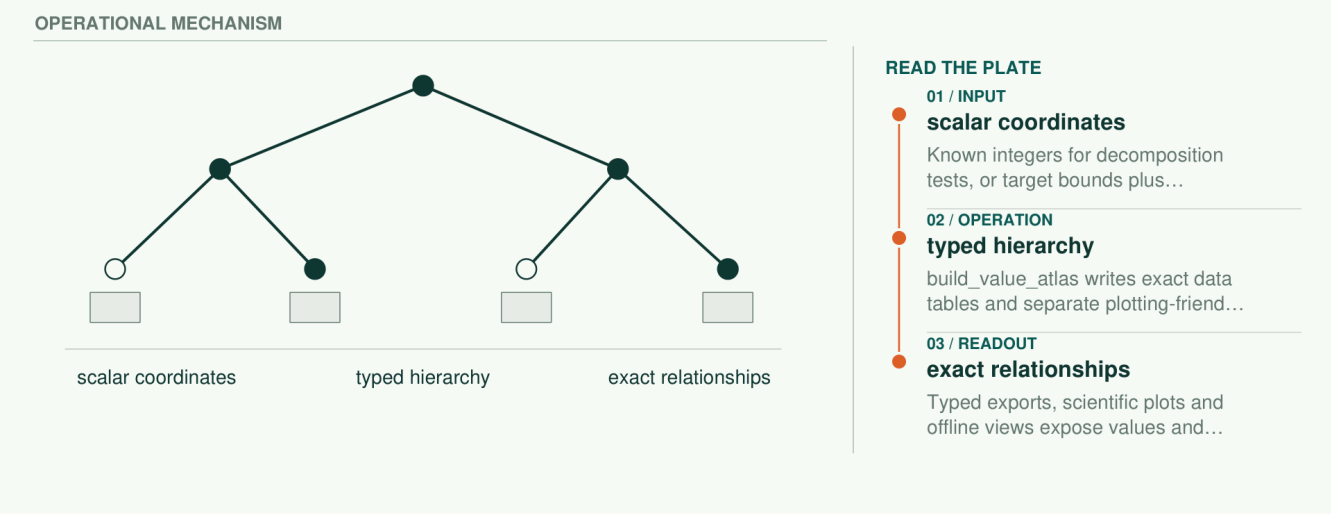
SECTION 03 / CTHEORY DECOMPOSITIONS AND SCALAR GEOMETRY

Exact value atlas and hierarchy visualizations

M055

EVIDENCE STATUS IMPLEMENTED

FIGURE / METHOD MECHANISM



HOW IT WORKS

Export hierarchy values with explicit types, sizes and relationships so arithmetic and hypothesis ranges can be inspected visually.

INPUTS NEEDED

Known integers for decomposition tests, or target bounds plus hypothesized root/coefficient digits; a public point for shifted-point searches.

WHAT WAS ACTUALLY DONE

build_value_atlas writes exact data tables and separate plotting-friendly values. The atlas explains known quantities; a chart that looks clustered is not an independent predictor.

WHAT THE RESULT MEANS

Typed exports, scientific plots and offline views expose values and bounds; visualization itself is not a predictor.

MATHEMATICAL FORMULATION

$$\mathcal{A}(k) = (R_1, C_1, D_1, R_2, C_2, \Delta_2, \ldots)$$

The atlas exports exact coordinates and bounds for known inputs. Plotting A(k) does not infer A(k) from a target address.

EVIDENCE S083

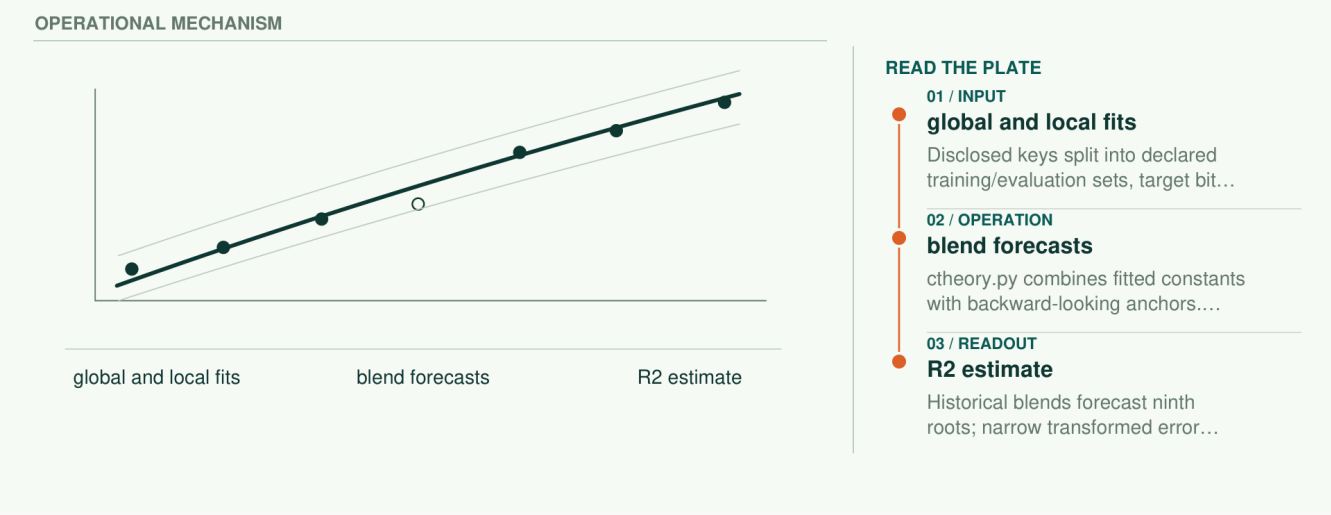
SECTION 04 / PREDICTORS, HELD-OUT VALIDATION AND CORRECTIVE AUDITS

FUSED global/local R2 regression

M056

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

Fit root growth globally and locally, then blend forecasts and sequence terms to predict R2 for a target index.

INPUTS NEEDED

Disclosed keys split into declared training/evaluation sets, target bit widths, fixed model choices and correctly matched null controls.

WHAT WAS ACTUALLY DONE

ctheory.py combines fitted constants with backward-looking anchors. Audits refit or exclude known answers to measure leakage; a forecast supplies a root guess, not Delta2 or D1.

WHAT THE RESULT MEANS

Historical blends forecast ninth roots; narrow transformed error must be converted to actual scalar coverage.

MATHEMATICAL FORMULATION

$$\log_2 \widehat{R}_2(N) = \frac{\ell_g(N) + w\ell_l(N)}{1+w}$$

This describes the global/local log-space anchor blend; ell_g and ell_l are fitted log2 root predictions. Other FUSED components depend on the implementation version.

EVIDENCE S221 / S163

PAPERS PAPER-002 / PAPER-005

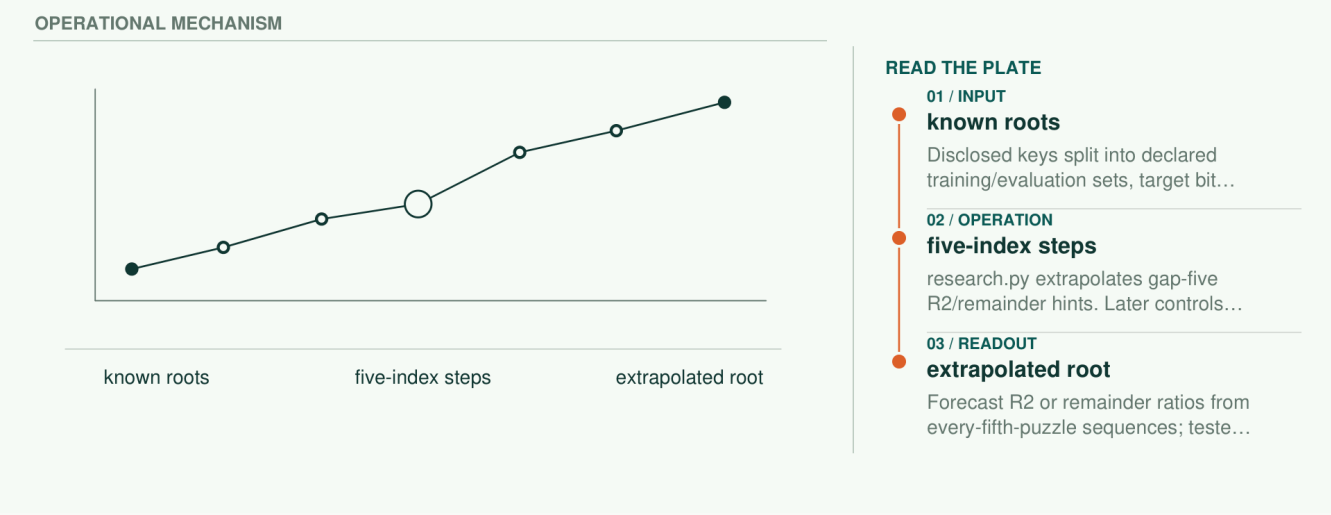
SECTION 04 / PREDICTORS, HELD-OUT VALIDATION AND CORRECTIVE AUDITS

Gap-five sequence extrapolation

M057

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

Use the regularly spaced disclosed high-index keys to estimate trends or differences five puzzle numbers apart.

INPUTS NEEDED

Disclosed keys split into declared training/evaluation sets, target bit widths, fixed model choices and correctly matched null controls.

WHAT WAS ACTUALLY DONE

research.py extrapolates gap-five R2/remainder hints. Later controls run the same fitting on synthetic corpora to test whether apparent predictability exceeds the interval geometry alone.

WHAT THE RESULT MEANS

Forecast R2 or remainder ratios from every-fifth-puzzle sequences; tested against forward-only and uniform controls.

MATHEMATICAL FORMULATION

$$\widehat{R}_2(N) = \frac{R_2(N-5)^2}{R_2(N-10)}$$

Geometric gap-five extrapolation needs both prior roots and a nonzero denominator. Its target root still requires held-out validation.

EVIDENCE S119 / S085

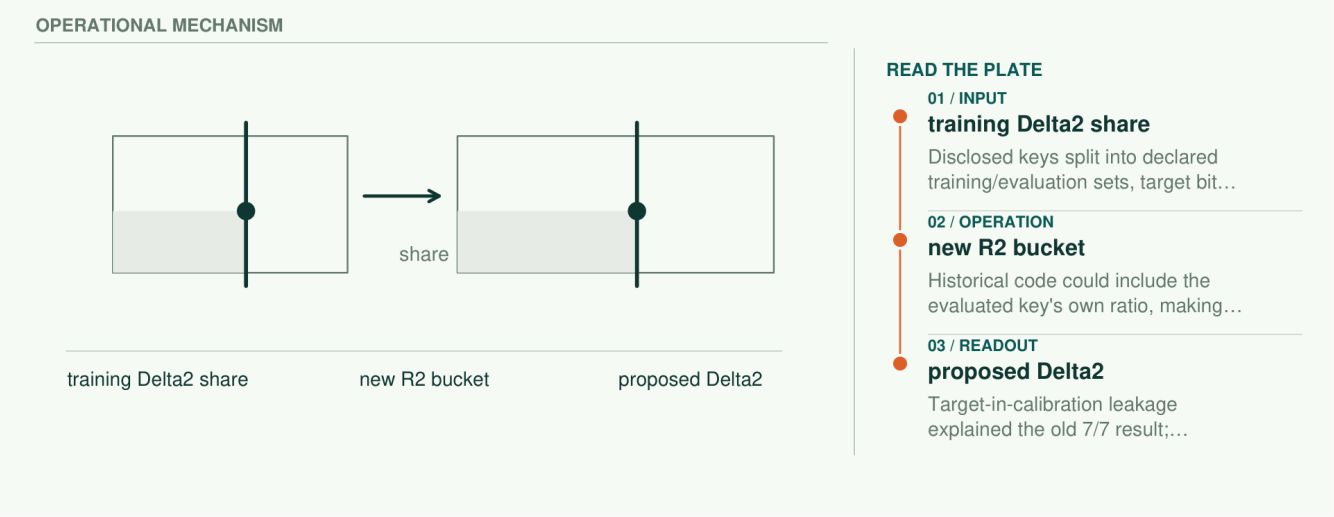
SECTION 04 / PREDICTORS, HELD-OUT VALIDATION AND CORRECTIVE AUDITS

Strategic Delta2 ratio hints

M058

EVIDENCE STATUS AUDITED/CORRECTED

FIGURE / METHOD MECHANISM



HOW IT WORKS

Reuse normalized Delta2 ratios from calibration keys to propose a short list of exact Delta2 values.

WHAT WAS ACTUALLY DONE

Historical code could include the evaluated key's own ratio, making recovery self-confirming. Corrected exclusion removes that shortcut and exposes the much weaker blind exact-hint performance.

INPUTS NEEDED

Disclosed keys split into declared training/evaluation sets, target bit widths, fixed model choices and correctly matched null controls.

WHAT THE RESULT MEANS

Target-in-calibration leakage explained the old 7/7 result; corrected blind high-index results were 0/7.

MATHEMATICAL FORMULATION

$$\widehat{\Delta}_2 = \lfloor \rho_{\text{train}} R_2^2 \rceil$$

A previous normalized remainder ratio proposes Delta2. The target's own ratio must be excluded; rounding to an integer is not evidence of correctness.

EVIDENCE S134

PAPERS PAPER-013

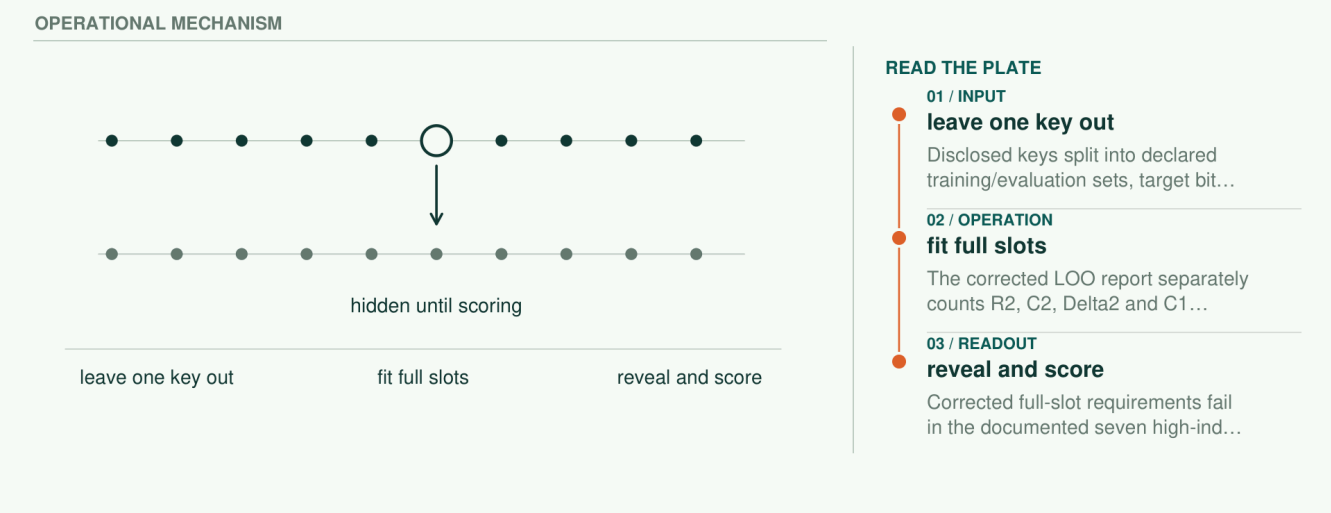
SECTION 04 / PREDICTORS, HELD-OUT VALIDATION AND CORRECTIVE AUDITS

Leave-one-out full-slot backtest

M059

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

Hide one key from every calibration source, build its full slot list, then reveal the answer only to score containment.

INPUTS NEEDED

Disclosed keys split into declared training/evaluation sets, target bit widths, fixed model choices and correctly matched null controls.

WHAT WAS ACTUALLY DONE

The corrected LOO report separately counts R2, C2, Delta2 and C1 success. Coarse root coverage cannot compensate for a missing exact Delta2 coordinate.

WHAT THE RESULT MEANS

Corrected full-slot requirements fail in the documented seven high-index holdouts, despite coarse R2 coverage.

MATHEMATICAL FORMULATION

$$\hat{\theta}_i = \mathrm{Fit}(\mathcal{D} \setminus \{i\}), \quad H = \sum_i 1\{k_i \in \mathcal{S}(\hat{\theta}_i)\}$$

Leave-one-out coverage evaluates the complete predicted slot. Hardcoded parameters fitted on the held-out target would still leak information.

EVIDENCE S141

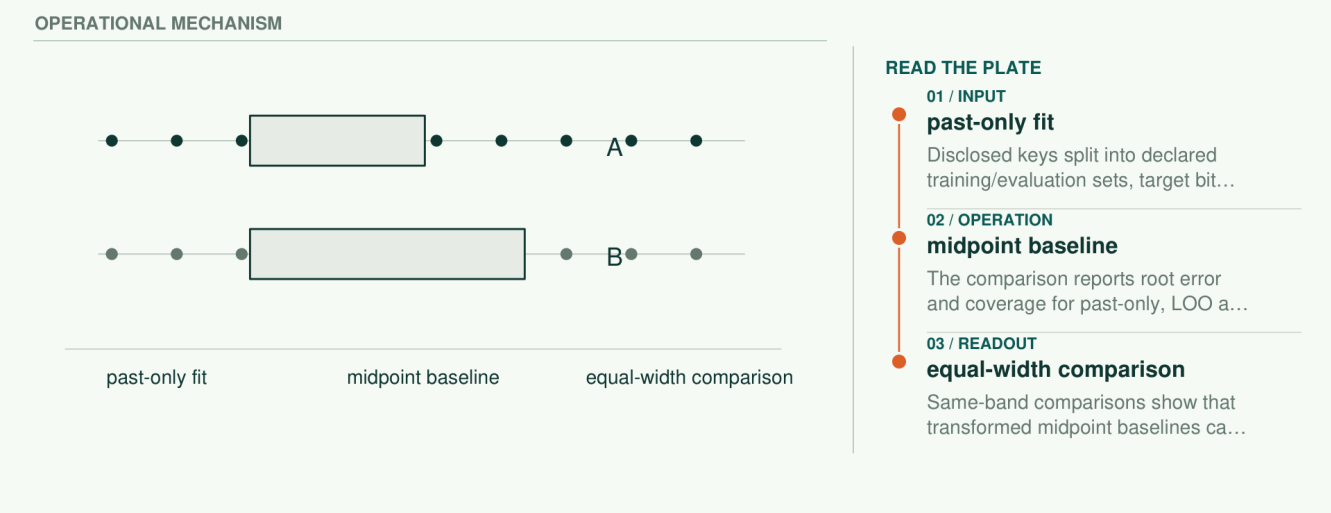
SECTION 04 / PREDICTORS, HELD-OUT VALIDATION AND CORRECTIVE AUDITS

Past-only FUSED versus midpoint

M060

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

Compare a learned R2 forecast with the ninth root of the scalar interval midpoint using equal tolerances and actual scalar mass.

INPUTS NEEDED

Disclosed keys split into declared training/evaluation sets, target bit widths, fixed model choices and correctly matched null controls.

WHAT WAS ACTUALLY DONE

The comparison reports root error and coverage for past-only, LOO and production fits. Later solved keys or fixed full-corpus coefficients can make an ostensibly held-out fit optimistic.

WHAT THE RESULT MEANS

Same-band comparisons show that transformed midpoint baselines can rival or beat apparent R2 coverage.

MATHEMATICAL FORMULATION

$$\widehat{R}_{\mathrm{mid}} = \left\lfloor \frac{r_{\min} + r_{\max}}{2} \right\rfloor$$

Compare the fitted rule with a declared midpoint baseline using the same root-band width. Translate both root bands back into scalar coverage.

EVIDENCE S163

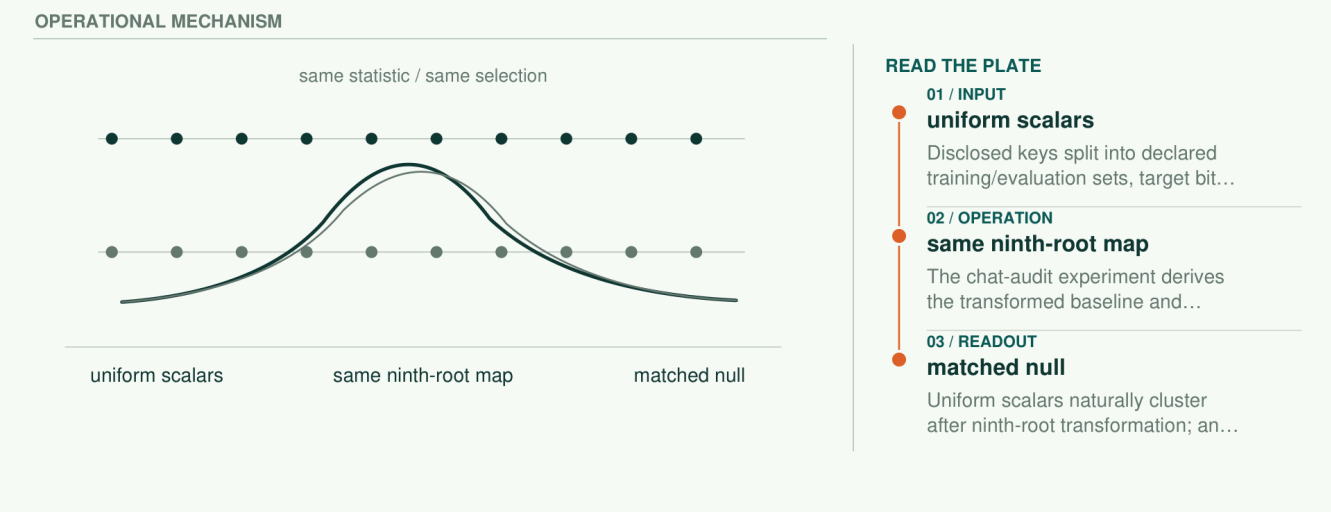
SECTION 04 / PREDICTORS, HELD-OUT VALIDATION AND CORRECTIVE AUDITS

Correct uniform null for ninth-root clustering

M061

EVIDENCE STATUS AUDITED/CORRECTED

FIGURE / METHOD MECHANISM



HOW IT WORKS

Transform uniform random scalars through the same ninth-root map as the real keys before comparing clustering.

INPUTS NEEDED

Disclosed keys split into declared training/evaluation sets, target bit widths, fixed model choices and correctly matched null controls.

WHAT WAS ACTUALLY DONE

The chat-audit experiment derives the transformed baseline and simulates matched-bit-width keys. It corrects an earlier null that mistakenly treated the compressed root coordinate as nearly uniform.

WHAT THE RESULT MEANS

Uniform scalars naturally cluster after ninth-root transformation; an earlier overly broad null standard deviation was corrected.

MATHEMATICAL FORMULATION

$$\mathrm{E}[X^j] = \frac{18}{9+j}\left(1 - 2^{-(9+j)/9}\right), \quad X = U^{1/9}$$

U is continuously uniform on [1/2,1); j=1,2 yields the transformed mean and variance. The discrete tier calculation weights exact ninth-root fibers.

EVIDENCE S068

SECTION 04 / PREDICTORS, HELD-OUT VALIDATION AND CORRECTIVE AUDITS

Forward-only synthetic-corpus null

M062

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM

HOW IT WORKS

Run the entire forward-fitting procedure on synthetic uniform corpora, not just a single fixed prediction under a new null.

INPUTS NEEDED

Disclosed keys split into declared training/evaluation sets, target bit widths, fixed model choices and correctly matched null controls.

WHAT WAS ACTUALLY DONE

The frozen experiment counts exact covered scalar fractions and compares aggregate hits-minus-coverage with replicated null corpora. This tests that named predictor under stated historical-index conventions.

WHAT THE RESULT MEANS

Repeat the predictor-fitting process on uniform synthetic keys and count exact scalar coverage.

MATHEMATICAL FORMULATION

$$T = \sum_i \left(1\{k_i \in B_i\} - \frac{|B_i|}{|I_i|} \right)$$

Refit the same forward-only procedure on synthetic tier-uniform corpora to calibrate T. B_i is the proposed scalar subset of tier I_i.

EVIDENCE S085 / S131

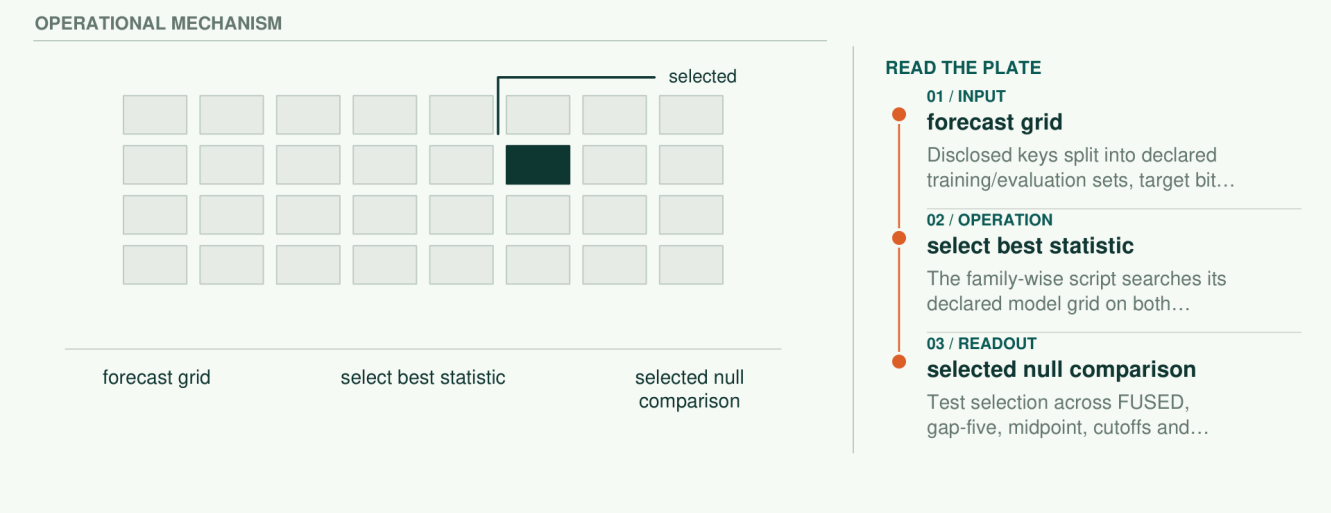
SECTION 04 / PREDICTORS, HELD-OUT VALIDATION AND CORRECTIVE AUDITS

Family-wise forward R2 grid

M063

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

When many cutoffs, widths and forecasts are examined, compare the best-looking observed result with equally selected null results.

INPUTS NEEDED

Disclosed keys split into declared training/evaluation sets, target bit widths, fixed model choices and correctly matched null controls.

WHAT WAS ACTUALLY DONE

The family-wise script searches its declared model grid on both observed and synthetic keys. Adjusted evidence is weaker than quoting the smallest unadjusted p-value.

WHAT THE RESULT MEANS

Test selection across FUSED, gap-five, midpoint, cutoffs and window widths with family-wise controls.

MATHEMATICAL FORMULATION

$$M = \max_{a \in \mathcal{A}} T_a\,, \quad \widehat{p} = \frac{1 + \sum_{q} 1\{M_q \geq M_{\mathrm{obs}}\}}{B+1}$$

Each of B synthetic replicates repeats the entire model grid A and its maximum selection. This calibrates the selected statistic, not a single prespecified rule.

EVIDENCE S084

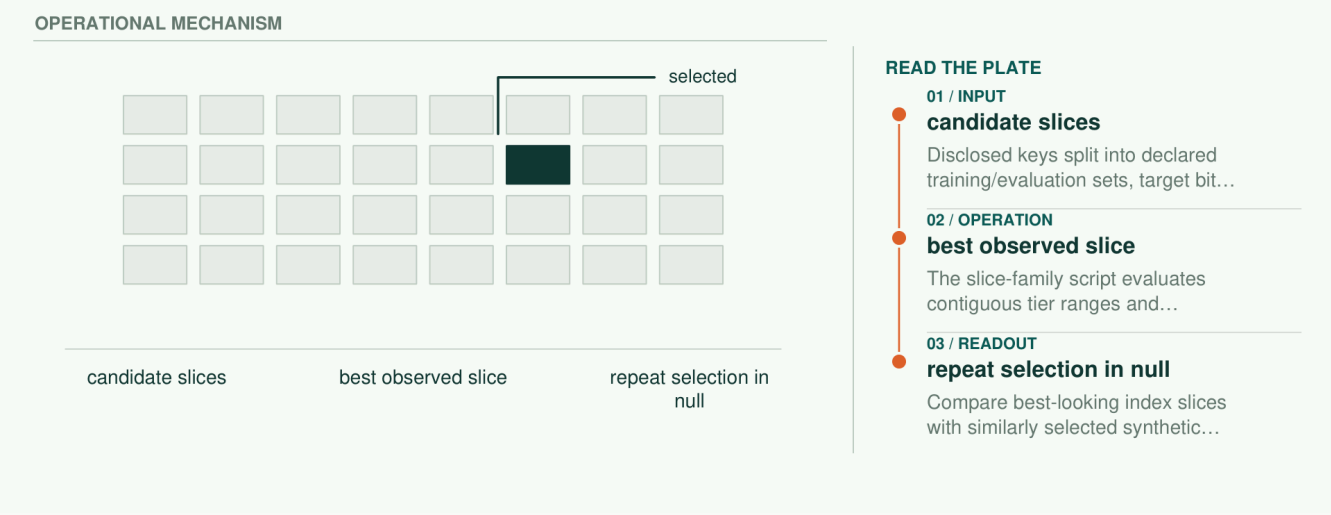
SECTION 04 / PREDICTORS, HELD-OUT VALIDATION AND CORRECTIVE AUDITS

Contiguous-slice selection adjustment

M064

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

A favorable consecutive slice can be selected by chance. The null comparison must also search candidate slices.

INPUTS NEEDED

Disclosed keys split into declared training/evaluation sets, target bit widths, fixed model choices and correctly matched null controls.

WHAT WAS ACTUALLY DONE

The slice-family script evaluates contiguous tier ranges and selection-adjusts their scores. It measures the effect of choosing a convenient evaluation interval after seeing outcomes.

WHAT THE RESULT MEANS

Compare best-looking index slices with similarly selected synthetic controls; guards against picking favorable subsequences.

MATHEMATICAL FORMULATION

$$M = \max_{[a,b]\in\mathcal{I}} T_{[a,b]}$$

Search the declared family of contiguous index slices I; synthetic controls must repeat this same maximization to account for slice selection.

EVIDENCE S086

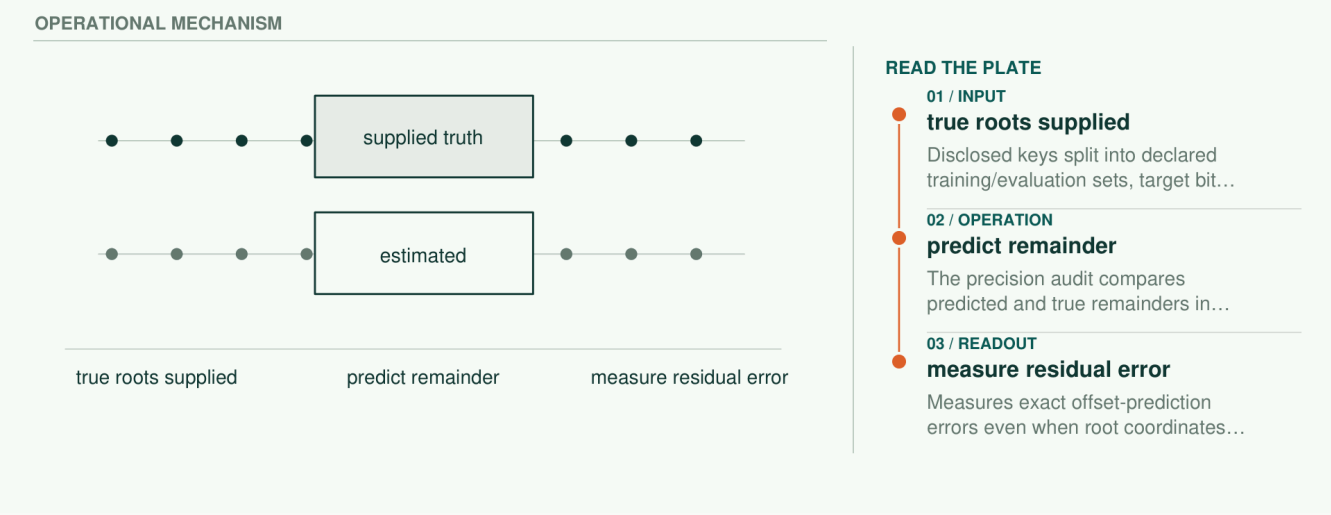
SECTION 04 / PREDICTORS, HELD-OUT VALIDATION AND CORRECTIVE AUDITS

Remainder precision granted true roots

M065

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

Give a predictor the true root coordinates to isolate whether it can predict the remaining offset accurately enough.

INPUTS NEEDED

Disclosed keys split into declared training/evaluation sets, target bit widths, fixed model choices and correctly matched null controls.

WHAT WAS ACTUALLY DONE

The precision audit compares predicted and true remainders in integer/key units. It is an oracle diagnostic, deliberately easier than an unknown-key attack, and still exposes large residual errors.

WHAT THE RESULT MEANS

Measures exact offset-prediction errors even when root coordinates are supplied; coarse root prediction does not supply leaf precision.

MATHEMATICAL FORMULATION

$$e_i = |\widehat{d}_i - d_i|, \quad \mathrm{hit}_i = 1\{d_i \in B_i\}$$

True higher roots are supplied in this diagnostic. Exact remainder error and band containment measure the precision still missing under that favorable assumption.

EVIDENCE S088

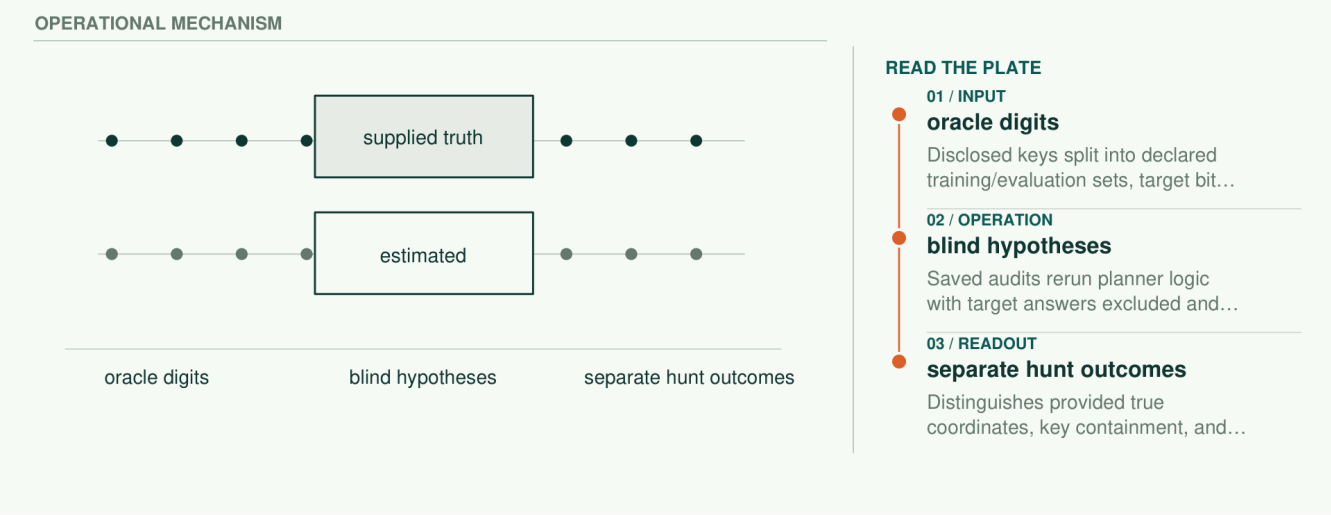
SECTION 04 / PREDICTORS, HELD-OUT VALIDATION AND CORRECTIVE AUDITS

Oracle-versus-blind hunt comparison

M066

EVIDENCE STATUS AUDITED/CORRECTED

FIGURE / METHOD MECHANISM



HOW IT WORKS

Separate three tasks: reconstruct from supplied digits, include the answer in a candidate interval, and actually discover it by target verification.

INPUTS NEEDED

Disclosed keys split into declared training/evaluation sets, target bit widths, fixed model choices and correctly matched null controls.

WHAT WAS ACTUALLY DONE

Saved audits rerun planner logic with target answers excluded and compare oracle/full-slot results. A true-key-assisted coordinate lookup is labeled oracle rather than blind recovery.

WHAT THE RESULT MEANS

Distinguishes provided true coordinates, key containment, and actual address/public-point discovery.

MATHEMATICAL FORMULATION

$$\mathcal{S}_{\text{oracle}} = \mathcal{S}(\theta_{\text{true}}), \quad \mathcal{S}_{\text{blind}} = \mathcal{S}(\hat{\theta})$$

Separate correctness conditional on true coordinates from prediction of those coordinates; each case still needs a verified target comparison.

EVIDENCE S222 / S066

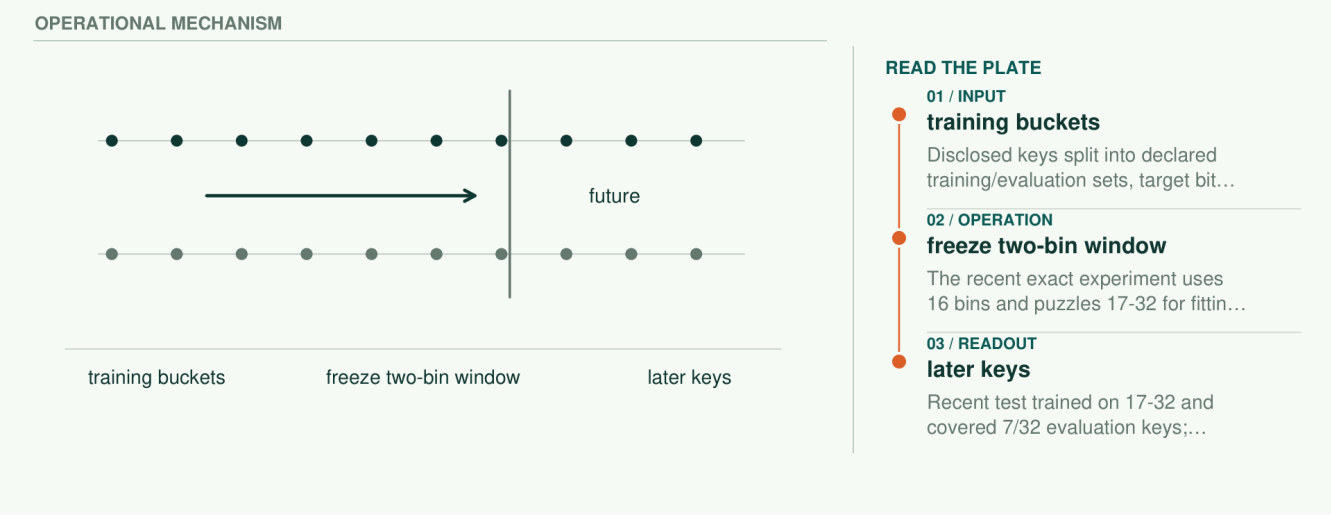
SECTION 04 / PREDICTORS, HELD-OUT VALIDATION AND CORRECTIVE AUDITS

Frozen ninth-root remainder window

M067

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

Normalize the ninth-root bucket remainder, fit a two-bin window on training keys, then freeze it for later keys.

INPUTS NEEDED

Disclosed keys split into declared training/evaluation sets, target bit widths, fixed model choices and correctly matched null controls.

WHAT WAS ACTUALLY DONE

The recent exact experiment uses 16 bins and puzzles 17-32 for fitting, then tests 33-64. It records both hits and the actual fraction of scalar candidates selected.

WHAT THE RESULT MEANS

Recent test trained on 17-32 and covered 7/32 evaluation keys; uncorrected p about 0.116, not convincing evidence.

MATHEMATICAL FORMULATION

$$r = \lfloor k^{1/9} \rfloor, \quad u = \frac{k - r^9}{(r+1)^9 - r^9}$$

Normalize the remainder within its exact ninth-root bucket. Fit a share window on the frozen training set and test it on excluded indices.

EVIDENCE S009

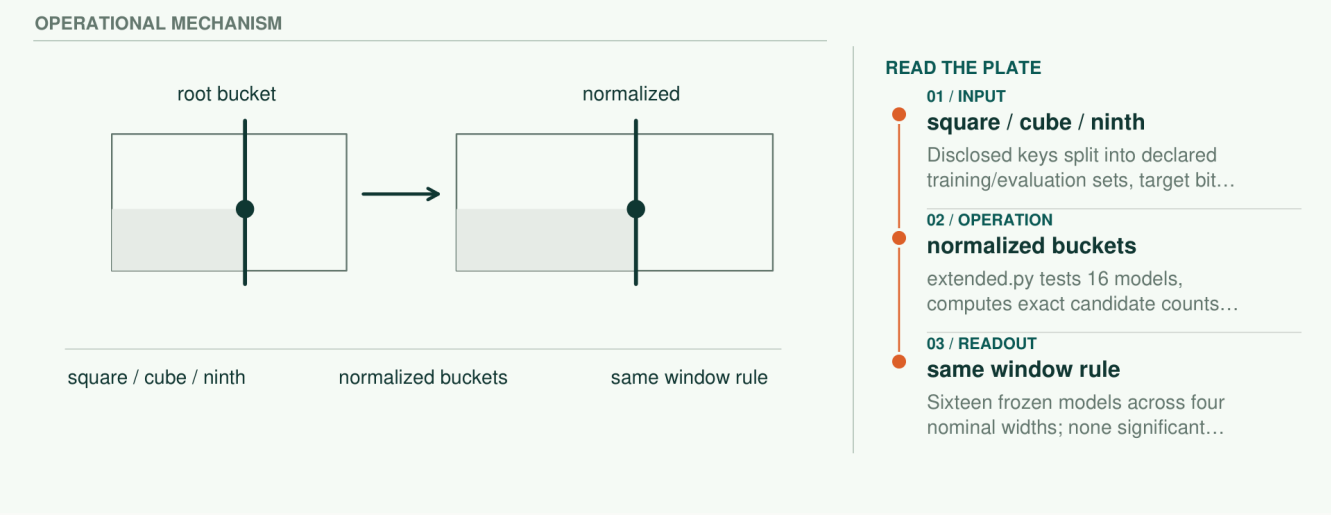
SECTION 04 / PREDICTORS, HELD-OUT VALIDATION AND CORRECTIVE AUDITS

Square/cube/ninth-root window family

M068

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

Repeat the same declared window-fitting rule for square, cube, ninth-root and ordinary range coordinates at four widths.

INPUTS NEEDED

Disclosed keys split into declared training/evaluation sets, target bit widths, fixed model choices and correctly matched null controls.

WHAT WAS ACTUALLY DONE

extended.py tests 16 models, computes exact candidate counts and applies a multiple-model correction. It does not keep tuning windows after examining each evaluation result.

WHAT THE RESULT MEANS

Sixteen frozen models across four nominal widths; none significant after multiple-model correction.

MATHEMATICAL FORMULATION

$$r_b = \lfloor k^{1/b} \rfloor, \quad u_b = \frac{k - r_b^b}{(r_b+1)^b - r_b^b}, \quad b \in \{2, 3, 9\}$$

Compare frozen bucket-share models across exponents and widths. Correct the reported evidence for the full family that was tried.

EVIDENCE S008

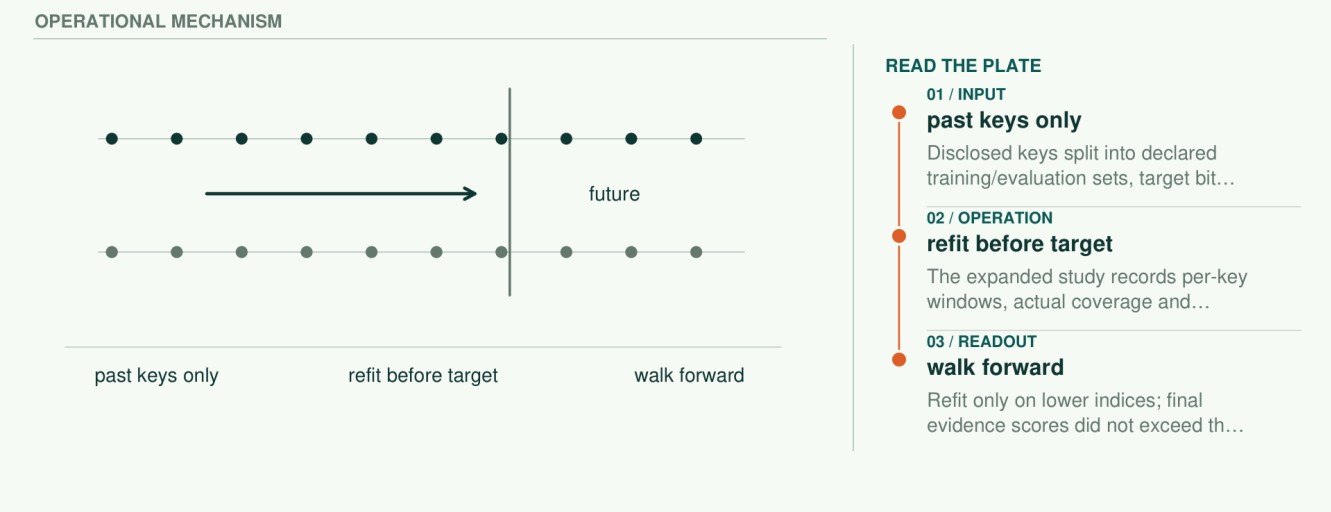
SECTION 04 / PREDICTORS, HELD-OUT VALIDATION AND CORRECTIVE AUDITS

Walk-forward adaptive remainder windows

M069

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

Before predicting each evaluation key, refit only on smaller indices, making every forecast independent of that target's stored answer.

INPUTS NEEDED

Disclosed keys split into declared training/evaluation sets, target bit widths, fixed model choices and correctly matched null controls.

WHAT WAS ACTUALLY DONE

The expanded study records per-key windows, actual coverage and sequential evidence scores. Previously inspected data still make this retrospective rather than pristine prospective validation.

WHAT THE RESULT MEANS

Refit only on lower indices; final evidence scores did not exceed the family threshold.

MATHEMATICAL FORMULATION

$$\theta_N = \mathrm{Fit}\{(i,k_i) : i < N\}, \quad B_N = \mathrm{Predict}(\theta_N, N)$$

A walk-forward model excludes the current and future keys at each evaluation step. Its selected width and adaptation policy also require honest calibration.

EVIDENCE S010

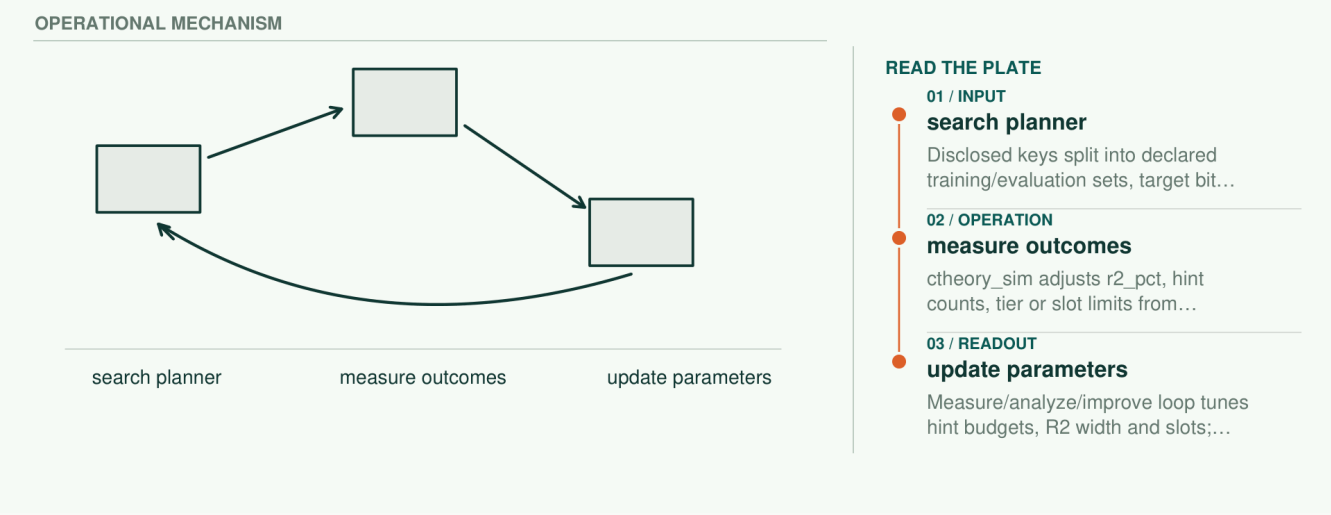
SECTION 04 / PREDICTORS, HELD-OUT VALIDATION AND CORRECTIVE AUDITS

Self-improving CTheory parameter loop

M070

EVIDENCE STATUS IMPLEMENTED

FIGURE / METHOD MECHANISM



HOW IT WORKS

Alternate measurement, statistical analysis and parameter changes to tune a search planner's hint/window budgets.

INPUTS NEEDED

Disclosed keys split into declared training/evaluation sets, target bit widths, fixed model choices and correctly matched null controls.

WHAT WAS ACTUALLY DONE

ctheory_sim adjusts r2_pct, hint counts, tier or slot limits from recorded performance. It optimizes the tested planner; biased/leaky metrics would teach it the wrong objective.

WHAT THE RESULT MEANS

Measure/analyze/improve loop tunes hint budgets, R2 width and slots; usefulness is limited by honest target inclusion.

MATHEMATICAL FORMULATION

$$\theta_{t+1} = \mathrm{Update}(\theta_t, \mathrm{metrics}_t)$$

theta contains hint budgets, widths and slot limits. Parameter adaptation is an optimization procedure; cryptographic progress requires measured target containment and verification.

EVIDENCE S089 / S251

PAPERS PAPER-020

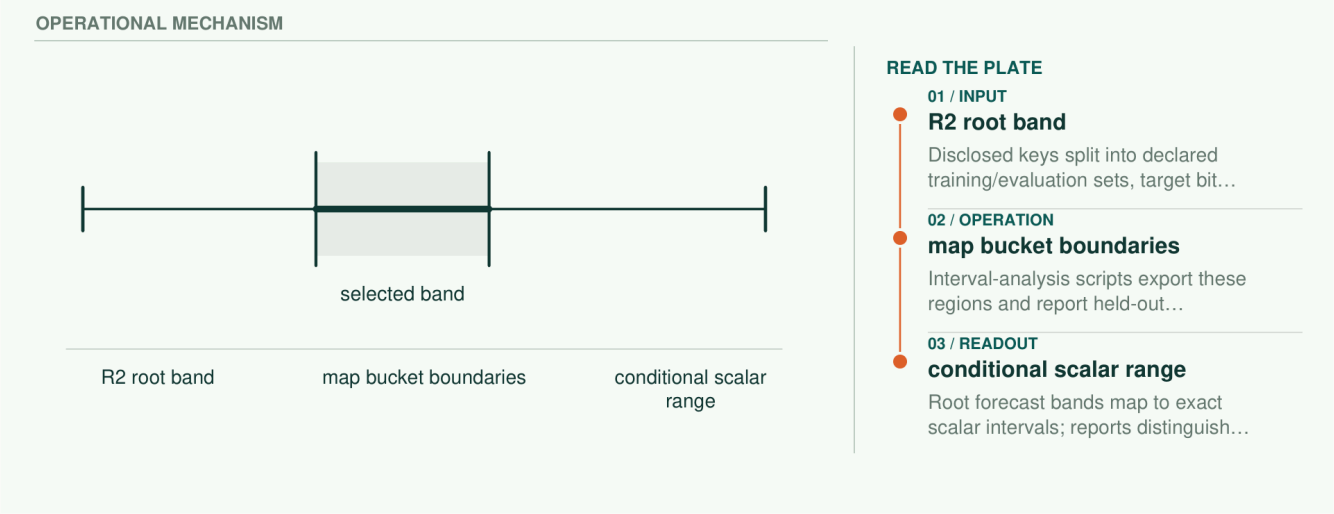
SECTION 04 / PREDICTORS, HELD-OUT VALIDATION AND CORRECTIVE AUDITS

R2 intervals as conditional search prioritization

M071

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

A root band [a,b] corresponds exactly to scalars from a^9 through (b+1)^9-1, clipped to puzzle bounds.

INPUTS NEEDED

Disclosed keys split into declared training/evaluation sets, target bit widths, fixed model choices and correctly matched null controls.

WHAT WAS ACTUALLY DONE

Interval-analysis scripts export these regions and report held-out coverage. The conversion is exact, but whether the unknown target belongs to the forecast band remains a probabilistic hypothesis.

WHAT THE RESULT MEANS

Root forecast bands map to exact scalar intervals; reports distinguish conditional narrowing from guaranteed coverage and statistical advantage.

MATHEMATICAL FORMULATION

$$k \in [a^9, (b+1)^9) \cap I \quad \Longleftrightarrow \quad R_2 \in \{a, \ldots, b\}$$

The equivalence uses the canonical ninth-root coordinate and integer a,b. Count the clipped scalar interval to measure conditional coverage.

EVIDENCE S067 / S163

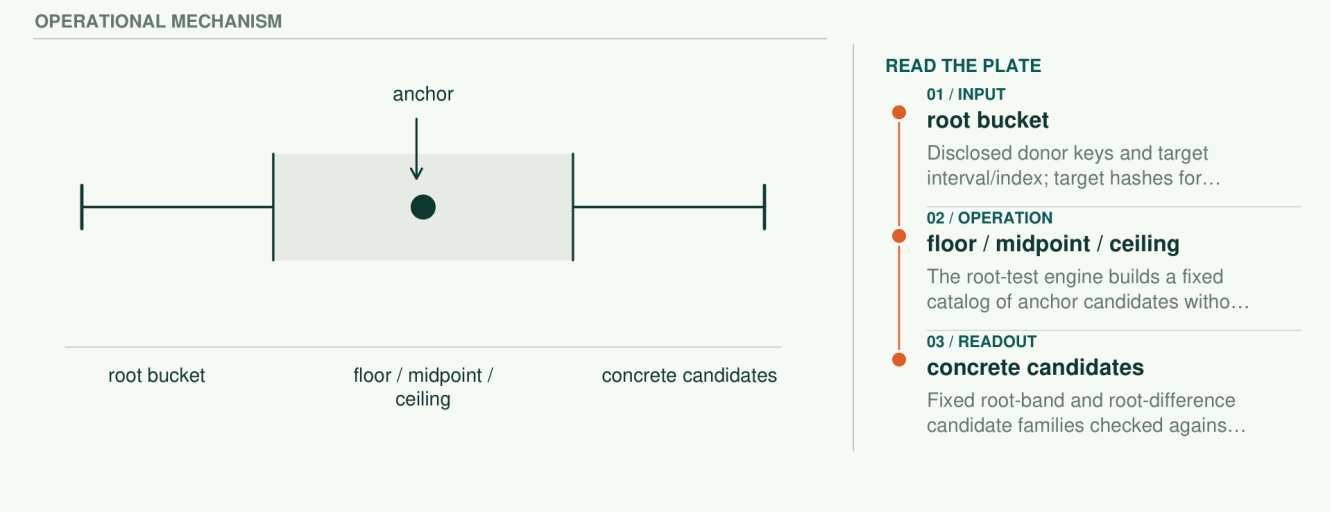
SECTION 05 / ROOT-DIFFERENCE, CARRY, INTERPOLATION AND NEAR-MISS ROUTES

Root floor/midpoint/ceiling candidate anchors

M072

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

Turn plausible root locations into concrete integers using floors, midpoints, ceilings or differences from disclosed roots.

INPUTS NEEDED

Disclosed donor keys and target interval/index; target hashes for candidate verification, with hidden answers used only for evaluation.

WHAT WAS ACTUALLY DONE

The root-test engine builds a fixed catalog of anchor candidates without reading the target key. Each resulting scalar is checked against that target's actual HASH160.

WHAT THE RESULT MEANS

Fixed root-band and root-difference candidate families checked against known and unknown targets.

MATHEMATICAL FORMULATION

$$a_{\text{floor}} = r^b, \quad a_{\text{mid}} = r^b + \left\lfloor \frac{(r+1)^b - r^b}{2} \right\rfloor$$

r is a legal integer b-th root. Lower edge, midpoint and upper edge are candidate anchors; they do not exhaust the bucket.

EVIDENCE S050 / S045

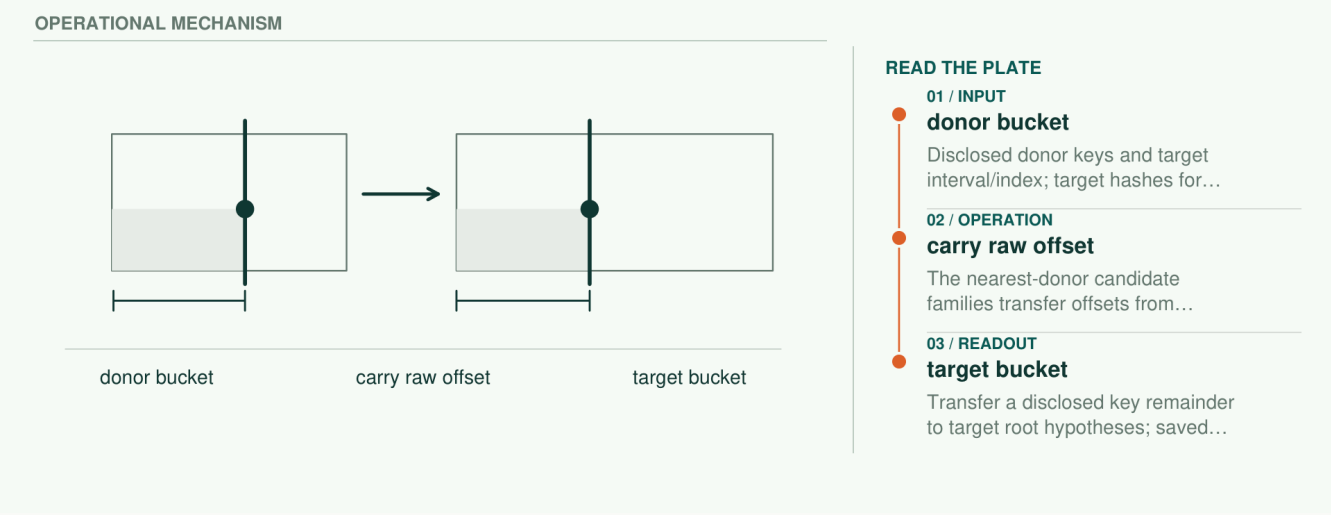
SECTION 05 / ROOT-DIFFERENCE, CARRY, INTERPOLATION AND NEAR-MISS ROUTES

Nearest-donor raw remainder carry

M073

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

Express a donor key as donor_root^9 plus an offset, then place that same offset above a legal root for the target.

INPUTS NEEDED

Disclosed donor keys and target interval/index; target hashes for candidate verification, with hidden answers used only for evaluation.

WHAT WAS ACTUALLY DONE

The nearest-donor candidate families transfer offsets from neighboring disclosed puzzle indices. They test an offset-inheritance hypothesis, not a consequence of ninth-root arithmetic.

WHAT THE RESULT MEANS

Transfer a disclosed key remainder to target root hypotheses; saved unknown-target candidate set contains no matches.

MATHEMATICAL FORMULATION

$$e_d = k_d - r_d^9, \quad \hat{k}_t = r_t^9 + e_d$$

Transfer a donor's raw remainder to a target root hypothesis. Check interval and canonical-bucket membership, then verify the candidate address.

EVIDENCE S050 / S045

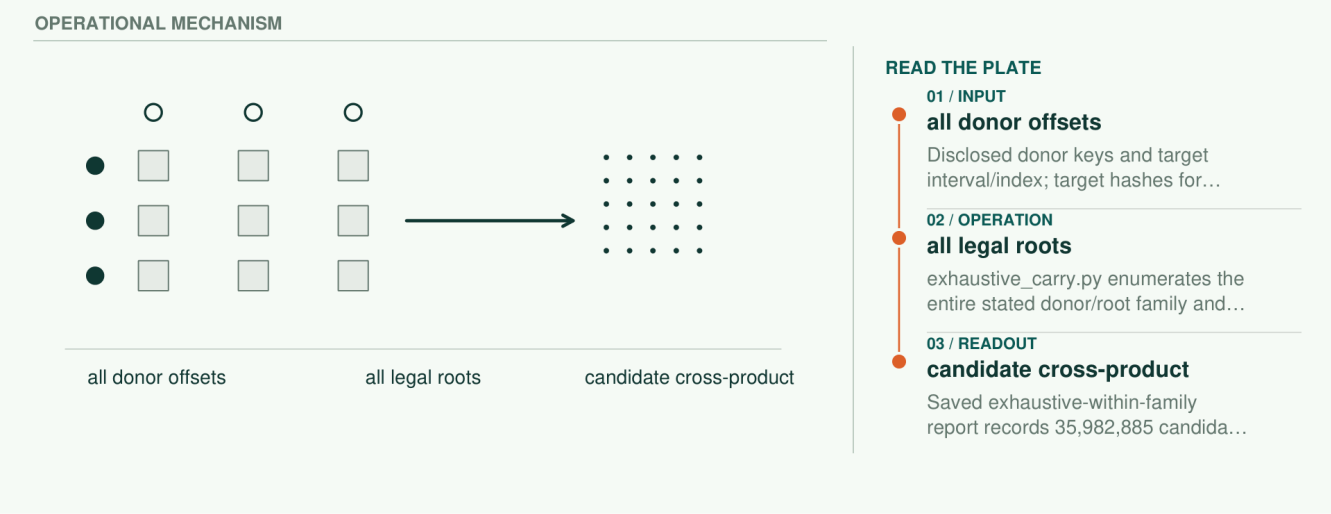
SECTION 05 / ROOT-DIFFERENCE, CARRY, INTERPOLATION AND NEAR-MISS ROUTES

All-donor carry over every legal ninth root

M074

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

Broaden the carry test to every disclosed donor and every legal target root, eliminating the nearest-donor selection restriction.

INPUTS NEEDED

Disclosed donor keys and target interval/index; target hashes for candidate verification, with hidden answers used only for evaluation.

WHAT WAS ACTUALLY DONE

exhaustive_carry.py enumerates the entire stated donor/root family and verifies candidates. Zero hits rejects those concrete carried candidates, not all possible remainder-generation laws.

WHAT THE RESULT MEANS

Saved exhaustive-within-family report records 35,982,885 candidate checks and zero target matches.

MATHEMATICAL FORMULATION

$$\mathcal{C}_t = \left\{ r^9 + e_d : r \in \mathcal{R}_t,\ d \in \mathcal{D} \right\} \cap I_t$$

Enumerate all declared donor remainders and legal target roots, removing duplicates. Exhaustion applies only to this candidate family.

EVIDENCE S041 / S042

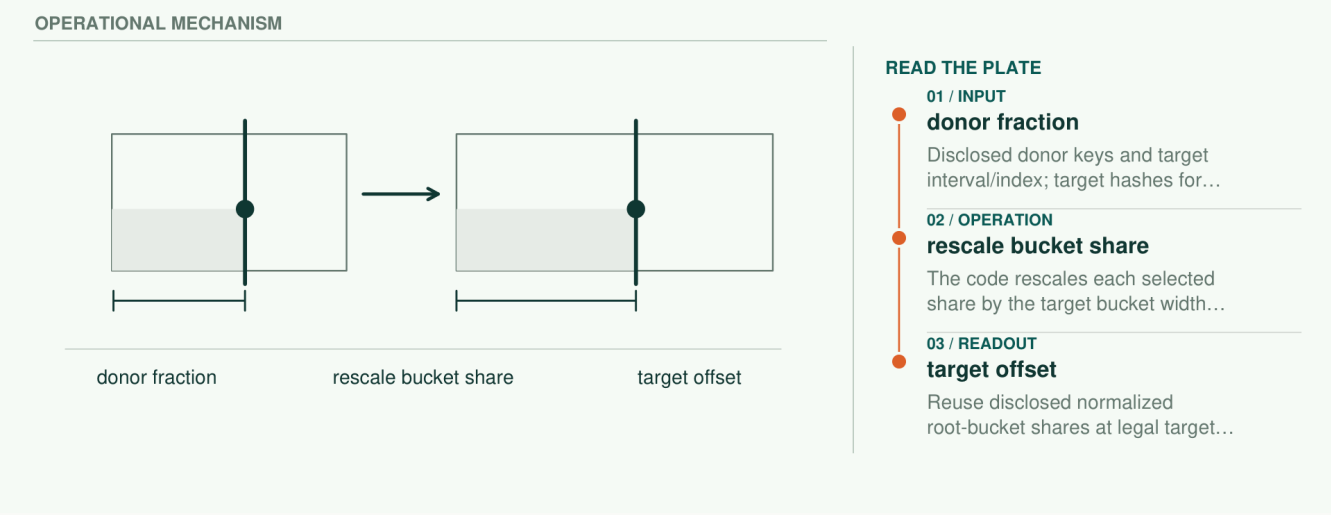
SECTION 05 / ROOT-DIFFERENCE, CARRY, INTERPOLATION AND NEAR-MISS ROUTES

Transferred within-bucket share

M075

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

Transfer a fraction of a donor bucket rather than its raw offset so buckets of different sizes can be compared.

INPUTS NEEDED

Disclosed donor keys and target interval/index; target hashes for candidate verification, with hidden answers used only for evaluation.

WHAT WAS ACTUALLY DONE

The code rescales each selected share by the target bucket width and rounds to an integer candidate. Any rounding and donor choice are part of the tested hypothesis.

WHAT THE RESULT MEANS

Reuse disclosed normalized root-bucket shares at legal target roots; tests do not cover arbitrary offset functions.

MATHEMATICAL FORMULATION

$$u_d = \frac{k_d - r_d^9}{(r_d+1)^9 - r_d^9}, \quad \widehat{k}_t = r_t^9 + \lfloor u_d W_t \rfloor$$

W_t=(r_t+1)^9-r_t^9. This transfers a normalized bucket share rather than an unchanged integer remainder.

EVIDENCE S041

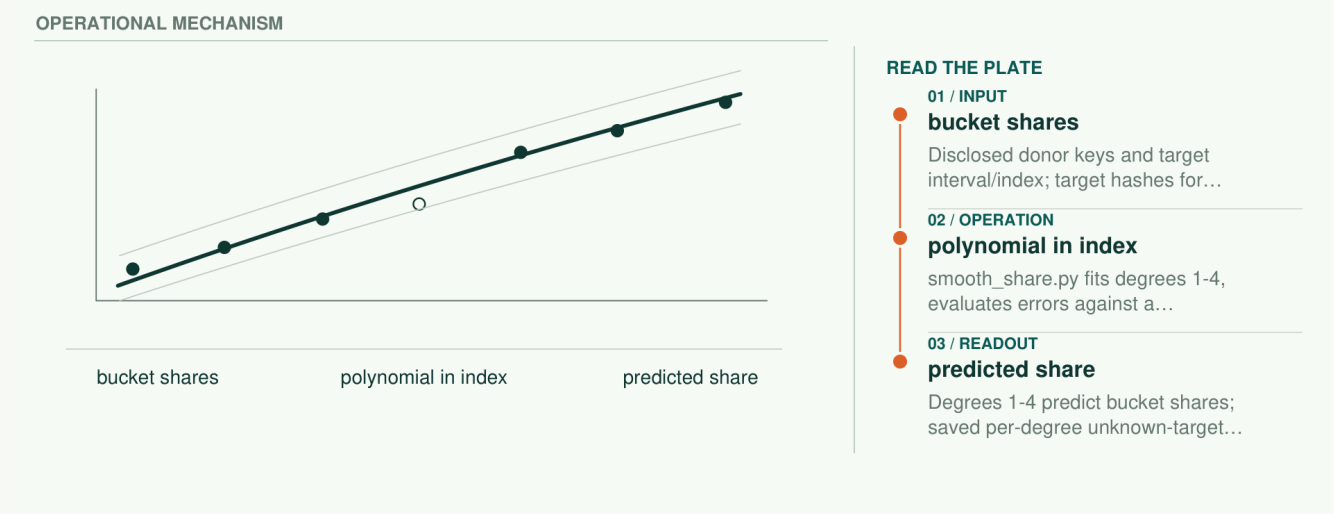
SECTION 05 / ROOT-DIFFERENCE, CARRY, INTERPOLATION AND NEAR-MISS ROUTES

Smooth polynomial share in puzzle index

M076

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

Assume normalized bucket share is a low-degree polynomial in puzzle index and use fitted values to generate target offsets.

INPUTS NEEDED

Disclosed donor keys and target interval/index; target hashes for candidate verification, with hidden answers used only for evaluation.

WHAT WAS ACTUALLY DONE

smooth_share.py fits degrees 1-4, evaluates errors against a uniform-share baseline and checks candidates over legal target roots. Numerical fit quality is reported separately from address hits.

WHAT THE RESULT MEANS

Degrees 1-4 predict bucket shares; saved per-degree unknown-target tests report no matches.

MATHEMATICAL FORMULATION

$$\widehat{u}(N) = \sum_{j=0}^{p} a_j N^j, \quad p \in \{1,2,3,4\}$$

A fitted share polynomial proposes a target bucket position. Handle predictions outside [0,1) explicitly and verify reconstructed candidates.

EVIDENCE S046 / S047

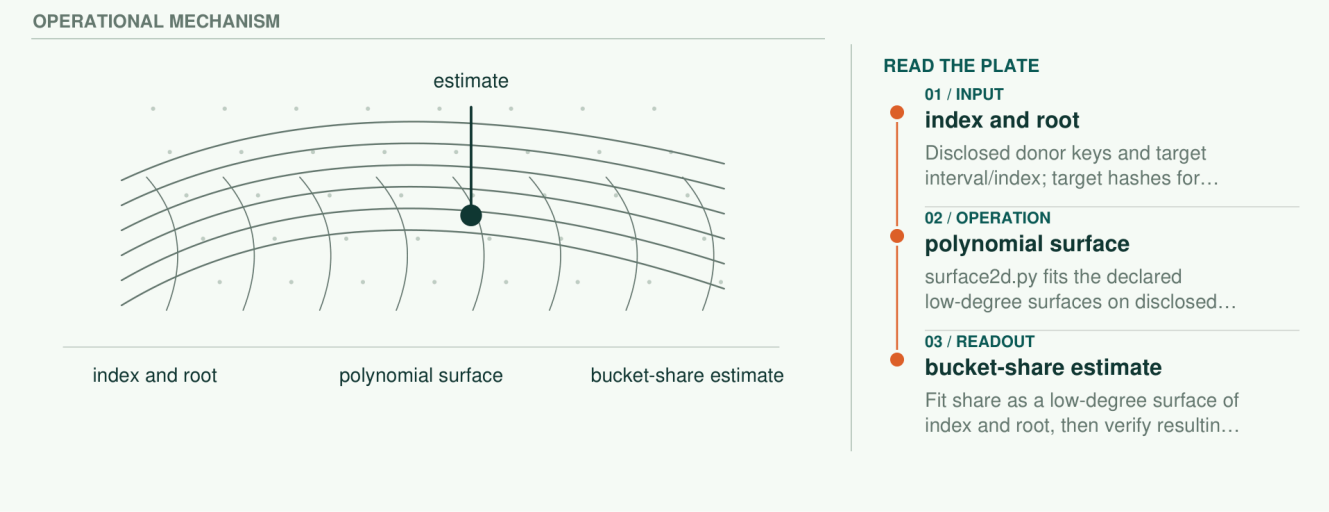
SECTION 05 / ROOT-DIFFERENCE, CARRY, INTERPOLATION AND NEAR-MISS ROUTES

Two-dimensional index/root share surface

M077

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

Include both puzzle index and legal root as inputs to a polynomial surface predicting the bucket share.

WHAT WAS ACTUALLY DONE

surface2d.py fits the declared low-degree surfaces on disclosed data and verifies reconstructed candidates. Additional model dimensions do not by themselves resolve an unknown remainder.

INPUTS NEEDED

Disclosed donor keys and target interval/index; target hashes for candidate verification, with hidden answers used only for evaluation.

WHAT THE RESULT MEANS

Fit share as a low-degree surface of index and root, then verify resulting candidates; no saved unknown-target hits.

MATHEMATICAL FORMULATION

$$\widehat{u}(N,r) = \sum_{i+j\le p} a_{ij} N^i r^j$$

A low-degree surface uses both index and root. This is the model family, not a guarantee that an unknown bucket share lies on that surface.

EVIDENCE S048 / S049

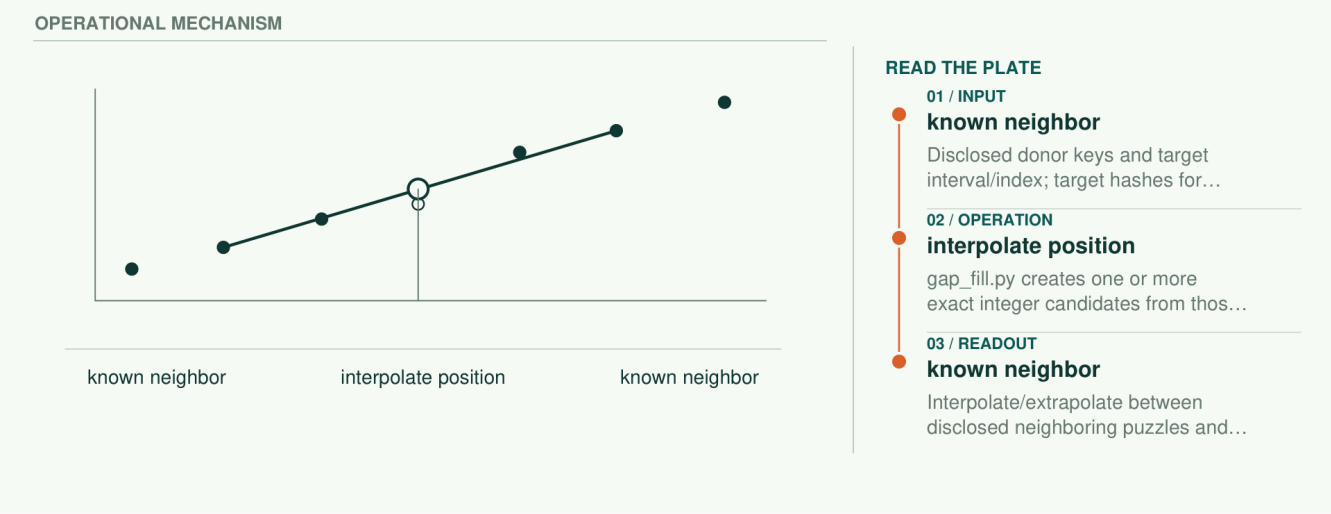
SECTION 05 / ROOT-DIFFERENCE, CARRY, INTERPOLATION AND NEAR-MISS ROUTES

Gap-fill normalized-position interpolation

M078

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

Assume normalized interval position changes smoothly between the nearest disclosed neighbors. Interpolate that position into the target interval.

INPUTS NEEDED

Disclosed donor keys and target interval/index; target hashes for candidate verification, with hidden answers used only for evaluation.

WHAT WAS ACTUALLY DONE

gap_fill.py creates one or more exact integer candidates from those neighboring positions. The hypothesis is smoothness; the verifier, not visual continuity, determines success.

WHAT THE RESULT MEANS

Interpolate/extrapolate between disclosed neighboring puzzles and verify candidates; no recorded matches.

MATHEMATICAL FORMULATION

$$u_t = (1-\alpha)u_a + \alpha u_b, \quad \alpha = \frac{t-a}{b-a}$$

Interpolate normalized tier positions between disclosed indices a,b, or extrapolate when alpha is outside [0,1]. Convert to integers and verify.

EVIDENCE S043 / S044

SECTION 05 / ROOT-DIFFERENCE, CARRY, INTERPOLATION AND NEAR-MISS ROUTES

Gap-fill linear/geometric root interpolation

M079

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM

HOW IT WORKS

Interpolate roots linearly or geometrically, then map the forecast root back to an integer scalar.

WHAT WAS ACTUALLY DONE

The gap-fill test generates candidates from both root interpolation rules and clips to the target range. A nearby root can still correspond to a very large scalar error.

INPUTS NEEDED

Disclosed donor keys and target interval/index; target hashes for candidate verification, with hidden answers used only for evaluation.

WHAT THE RESULT MEANS

Interpolate root coordinates and reconstruct candidate powers; tested candidates did not recover unknown keys.

MATHEMATICAL FORMULATION

$$\hat{r}_t = (1-\alpha)r_a + \alpha r_b \quad \text{or} \quad r_a^{1-\alpha} r_b^{\alpha}$$

Linear and geometric interpolation are alternative root hypotheses. Integer rounding and reconstructed offsets determine the actual candidate set.

EVIDENCE S043

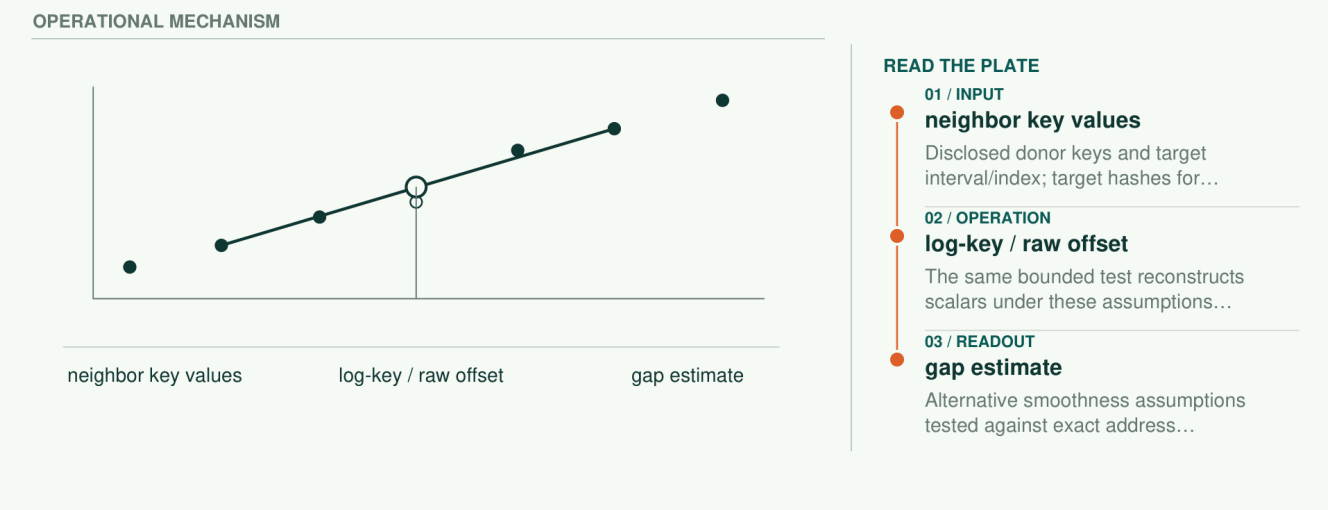
SECTION 05 / ROOT-DIFFERENCE, CARRY, INTERPOLATION AND NEAR-MISS ROUTES

Gap-fill log-key and raw-offset interpolation

M080

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

Interpolate log2(key) or the raw ninth-root residual as an alternative notion of smooth progression.

WHAT WAS ACTUALLY DONE

The same bounded test reconstructs scalars under these assumptions and compares addresses. Log closeness or matching early values is not accepted as a recovered key.

INPUTS NEEDED

Disclosed donor keys and target interval/index; target hashes for candidate verification, with hidden answers used only for evaluation.

WHAT THE RESULT MEANS

Alternative smoothness assumptions tested against exact address hashes, not judged by numerical proximity alone.

MATHEMATICAL FORMULATION

$$\log \hat{k}_t = (1-\alpha)\log k_a + \alpha \log k_b$$

Log-key interpolation is one family; directly interpolating raw offsets is another. Neither smoothness assumption follows from address data.

EVIDENCE S043

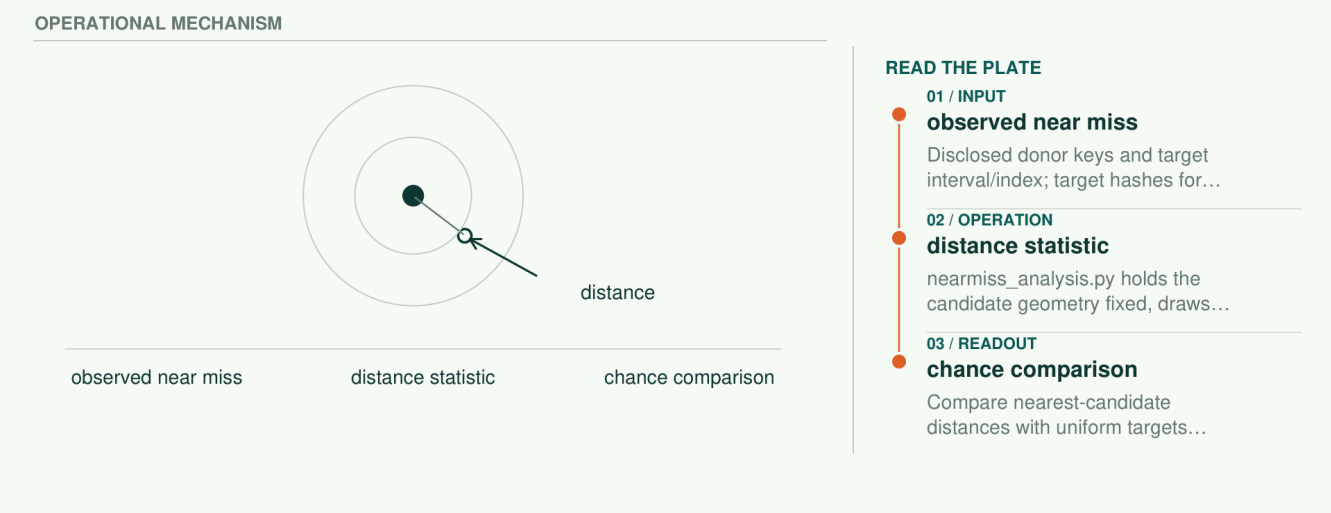
SECTION 05 / ROOT-DIFFERENCE, CARRY, INTERPOLATION AND NEAR-MISS ROUTES

Near-miss anomaly testing

M081

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

Judge proximity relative to the density and placement of the entire candidate set. Dense sets naturally create close numerical neighbors.

INPUTS NEEDED

Disclosed donor keys and target interval/index; target hashes for candidate verification, with hidden answers used only for evaluation.

WHAT WAS ACTUALLY DONE

nearmiss_analysis.py holds the candidate geometry fixed, draws uniform targets in the same interval and compares minimum distances. This is a diagnostic of chance closeness, not a new candidate verifier.

WHAT THE RESULT MEANS

Compare nearest-candidate distances with uniform targets against the same candidate geometry; closeness alone is not a verified key.

MATHEMATICAL FORMULATION

$$\delta(k, \mathcal{C}) = \min_{c \in \mathcal{C}} |k - c|$$

Compare this nearest-candidate distance with distances of tier-uniform control targets against the same candidate geometry. A small distance is not a hash match.

EVIDENCE S039 / S040

SECTION 05 / ROOT-DIFFERENCE, CARRY, INTERPOLATION AND NEAR-MISS ROUTES

All-donor carry proximity null

M082

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM

HOW IT WORKS

Ask whether any donor-offset/root combination lands unusually near a held-out key compared with identical carry geometry under uniform targets.

INPUTS NEEDED

Disclosed donor keys and target interval/index; target hashes for candidate verification, with hidden answers used only for evaluation.

WHAT WAS ACTUALLY DONE

all_donor_carry.py excludes that target from donor construction and computes observed versus null proximity. A favorable minimum is not an address match and may require selection adjustment.

WHAT THE RESULT MEANS

Compare minimum carry distance to uniform controls while excluding the known target from its donor set.

MATHEMATICAL FORMULATION

$$\delta_i = \min_{c \in \mathcal{C}(\mathcal{D} \setminus \{i\})} |k_i - c|$$

Exclude the known target from its donor set before measuring proximity. Repeat donor construction and distance scoring for matched controls.

EVIDENCE S038 / S037

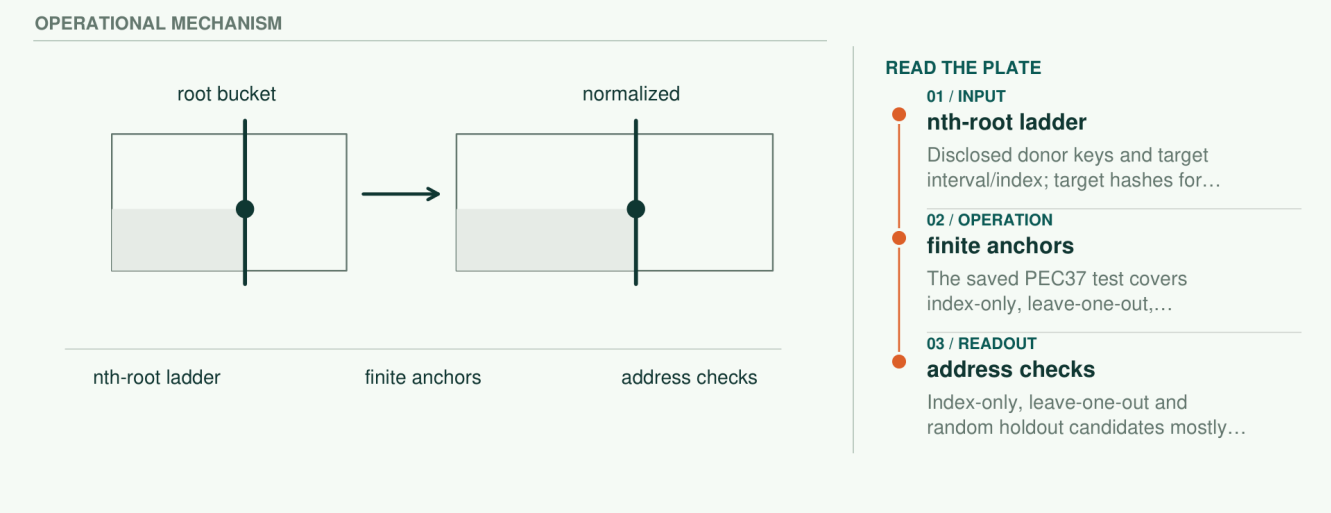
SECTION 05 / ROOT-DIFFERENCE, CARRY, INTERPOLATION AND NEAR-MISS ROUTES

Blind nth-root ladder address tests

M083

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

Build nth-root scheduled anchors and difference-based forecasts without supplying the hidden scalar to the candidate generator.

INPUTS NEEDED

Disclosed donor keys and target interval/index; target hashes for candidate verification, with hidden answers used only for evaluation.

WHAT WAS ACTUALLY DONE

The saved PEC37 test covers index-only, leave-one-out, random-holdout and unknown-target candidates. Reported matches are confined to tiny trivial intervals rather than scalable unknown-key recovery.

WHAT THE RESULT MEANS

Index-only, leave-one-out and random holdout candidates mostly match only trivial tiny puzzles; no unknown-target matches recorded.

MATHEMATICAL FORMULATION

$$r_b = \lfloor k^{1/b} \rfloor, \quad \widehat{k} = \widehat{r}_b^{\,b} + \widehat{e}_b$$

A root-ladder proposal must supply both the root estimate and remainder rule. Verify the resulting finite integer candidates exactly.

EVIDENCE S259
PAPERS PAPER-014

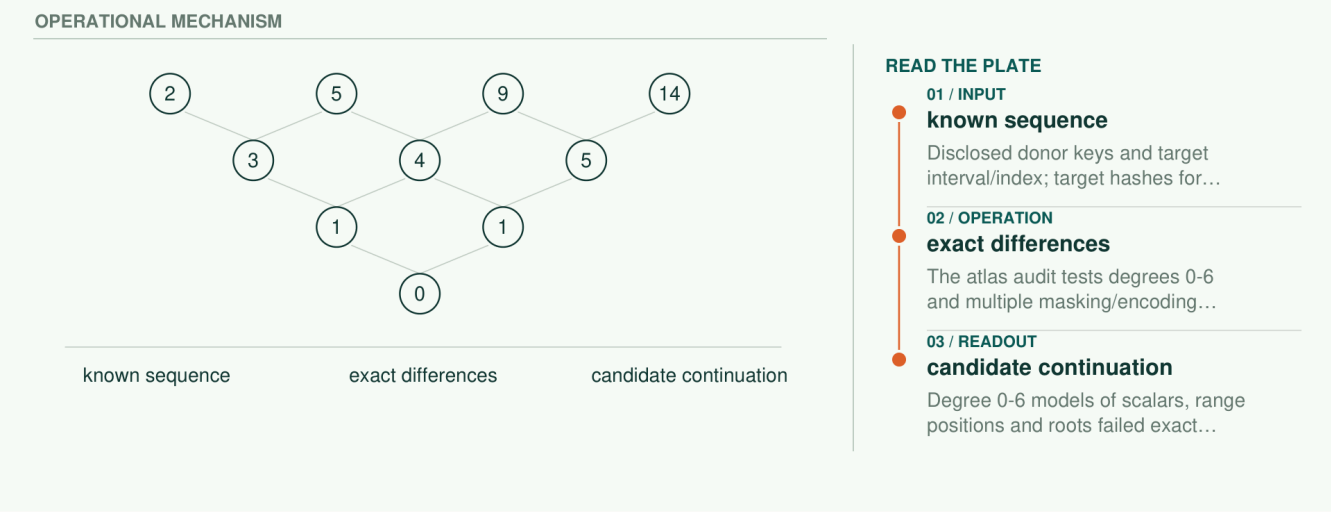
SECTION 05 / ROOT-DIFFERENCE, CARRY, INTERPOLATION AND NEAR-MISS ROUTES

Exact polynomial/difference continuation

M084

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

Fit exact rational polynomials to known scalars, roots or normalized positions, then evaluate them at held-out indices.

INPUTS NEEDED

Disclosed donor keys and target interval/index; target hashes for candidate verification, with hidden answers used only for evaluation.

WHAT WAS ACTUALLY DONE

The atlas audit tests degrees 0-6 and multiple masking/encoding variants. It uses published answers afterward for scoring and checks target-derived addresses for concrete candidate attempts.

WHAT THE RESULT MEANS

Degree 0-6 models of scalars, range positions and roots failed exact higher-index recovery in the saved tests.

MATHEMATICAL FORMULATION

$$\Delta^{d+1} f(i) = 0 \quad \text{for a degree } d \text{ polynomial}$$

The identity applies on equally spaced indices. Fitting or continuing a low-degree polynomial is a hypothesis about the data-generating rule.

EVIDENCE S071 / S068

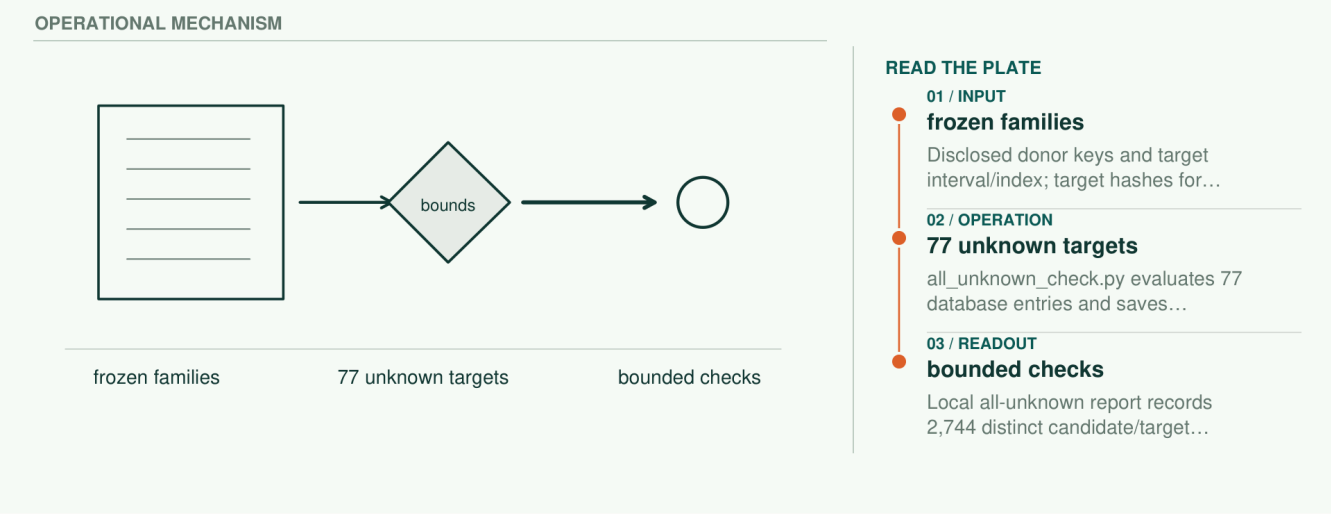
SECTION 05 / ROOT-DIFFERENCE, CARRY, INTERPOLATION AND NEAR-MISS ROUTES

Frozen candidate families on 77 unknown targets

M085

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

Freeze a finite candidate recipe, apply it to every locally undisclosed puzzle, and compare each output with that puzzle's target hash.

INPUTS NEEDED

Disclosed donor keys and target interval/index; target hashes for candidate verification, with hidden answers used only for evaluation.

WHAT WAS ACTUALLY DONE

all_unknown_check.py evaluates 77 database entries and saves candidate/target counts. A completed finite recipe search is not an exhaustive search of all their intervals.

WHAT THE RESULT MEANS

Local all-unknown report records 2,744 distinct candidate/target checks and no matches.

MATHEMATICAL FORMULATION

$$\mathcal{C}_i = cup_{f \in \mathcal{F}} \mathcal{C}_{i,f}, \quad H = \sum_i \sum_{k \in \mathcal{C}_i} 1\{h(k) = h_i\}$$

Freeze the finite family F before checking unknown targets and deduplicate candidate/target pairs. Counts cover only that declared union.

EVIDENCE S069 / S066

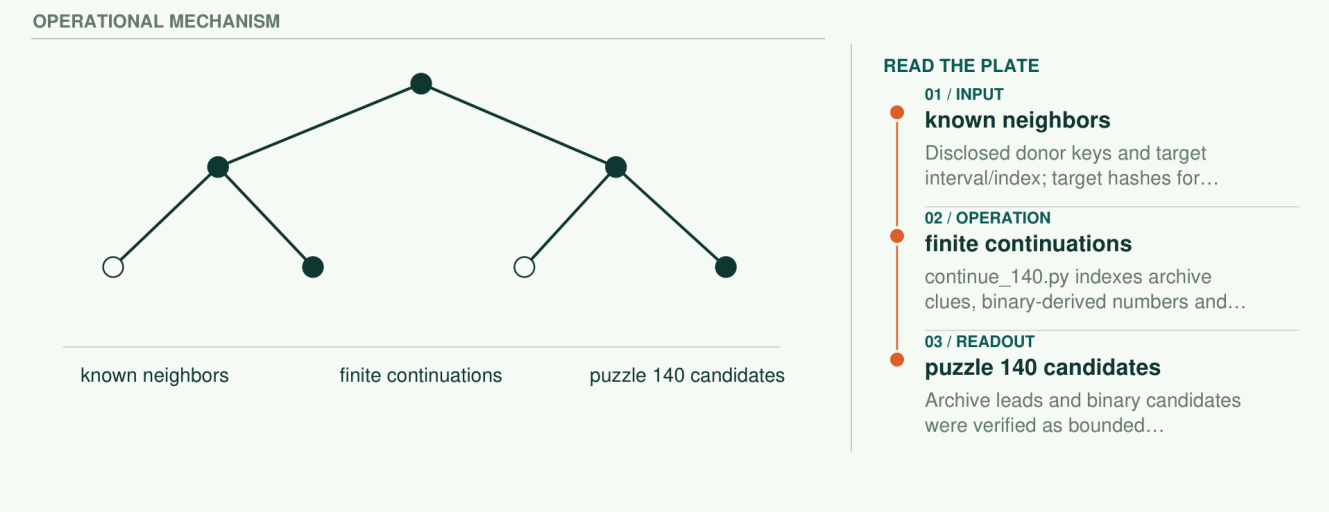
SECTION 05 / ROOT-DIFFERENCE, CARRY, INTERPOLATION AND NEAR-MISS ROUTES

Finite continuation attempts specifically for #140

M086

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

Collect new finite leads specifically for a public-point target, derive candidate scalars or intervals and perform exact comparisons.

INPUTS NEEDED

Disclosed donor keys and target interval/index; target hashes for candidate verification, with hidden answers used only for evaluation.

WHAT WAS ACTUALLY DONE

continue_140.py indexes archive clues, binary-derived numbers and signature relations. It records bounded attempts; finding a plausible story or interval does not count as discovering #140.

WHAT THE RESULT MEANS

Archive leads and binary candidates were verified as bounded hypotheses, not exhaustive interval searches.

MATHEMATICAL FORMULATION

$$\mathcal{C}_{140} = \left\{ f(140) : f \in \mathcal{F}_{\text{declared}} \right\} \cap I_{140}$$

These are finite continuation hypotheses and artifact-derived candidates. An unsuccessful test excludes these candidates, not the full tier.

EVIDENCE S074 / S073

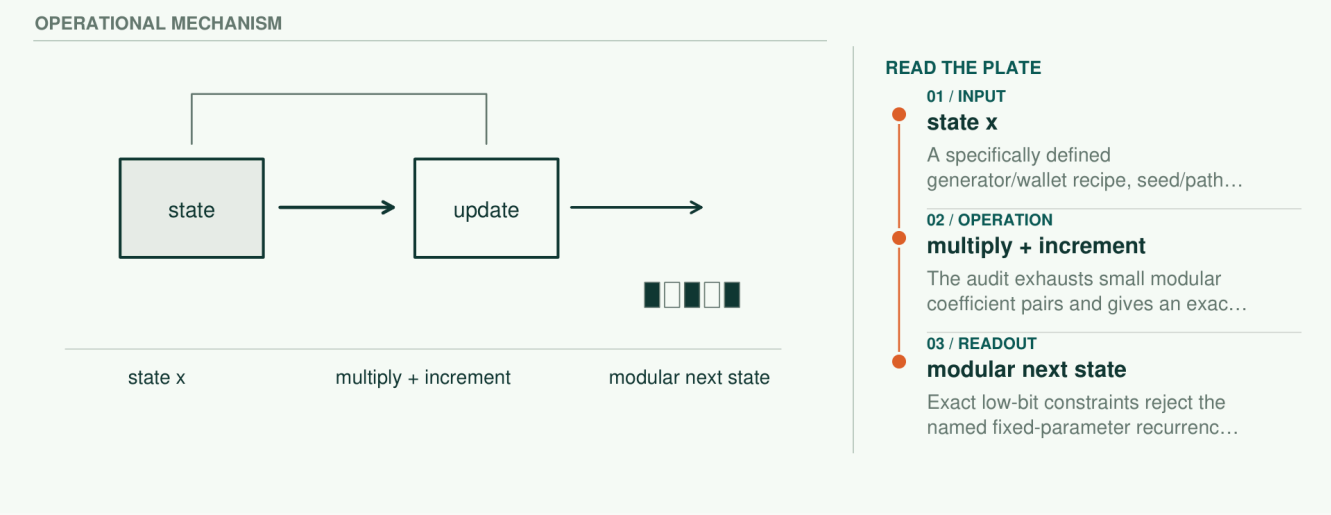
SECTION 06 / GENERATOR RECONSTRUCTION, MASKING AND DETERMINISTIC SEQUENCES

Fixed affine/LCG key recurrence

M087

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

A fixed affine recurrence maps the same low-bit input to the same output. Contradictory observed transitions can therefore reject that recurrence family.

WHAT WAS ACTUALLY DONE

The audit exhausts small modular coefficient pairs and gives an exact parity obstruction. It does not exclude hidden state, changing coefficients, discarded outputs or unrelated deterministic wallets.

INPUTS NEEDED

A specifically defined generator/wallet recipe, seed/path hypotheses and known keys or public targets for independent comparison.

WHAT THE RESULT MEANS

Exact low-bit constraints reject the named fixed-parameter recurrence family; hidden state and different generators remain outside scope.

MATHEMATICAL FORMULATION

$$x_{t+1} \equiv a x_t + c \pmod{2^b}$$

Fixed affine recurrence parameters must satisfy every applicable observed low-bit constraint under the declared masking and output schedule.

EVIDENCE S068 / S079
PAPERS PAPER-015

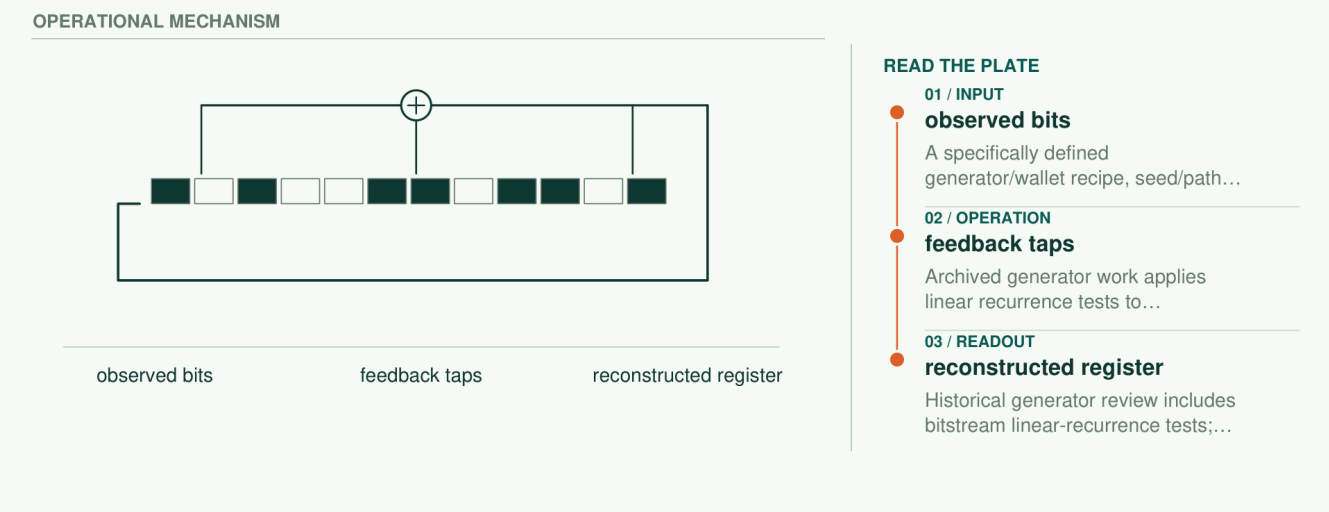
SECTION 06 / GENERATOR RECONSTRUCTION, MASKING AND DETERMINISTIC SEQUENCES

LFSR / Berlekamp-Massey reconstruction

M088

EVIDENCE STATUS RECORDED RESEARCH

FIGURE / METHOD MECHANISM



HOW IT WORKS

An LFSR predicts later bits from a fixed linear combination of earlier bits. Berlekamp-Massey finds a shortest recurrence fitting a supplied bitstream.

INPUTS NEEDED

A specifically defined generator/wallet recipe, seed/path hypotheses and known keys or public targets for independent comparison.

WHAT WAS ACTUALLY DONE

Archived generator work applies linear recurrence tests to concatenated known keys. Fitting the observed stream is insufficient unless the same state and output schedule predict unseen keys.

WHAT THE RESULT MEANS

Historical generator review includes bitstream linear-recurrence tests; no transferable unknown-key generator established.

MATHEMATICAL FORMULATION

$$s_t = \sum_{j=1}^{\ell} a_j s_{t-j} \quad \text{in } \mathbf{F}_2$$

Berlekamp-Massey finds a shortest binary linear recurrence fitting the supplied stream. A compatible fit requires independent predictive validation.

EVIDENCE S079 / S135

PAPERS PAPER-002 / PAPER-004 / PAPER-005 / PAPER-006 / PAPER-007 / PAPER-015

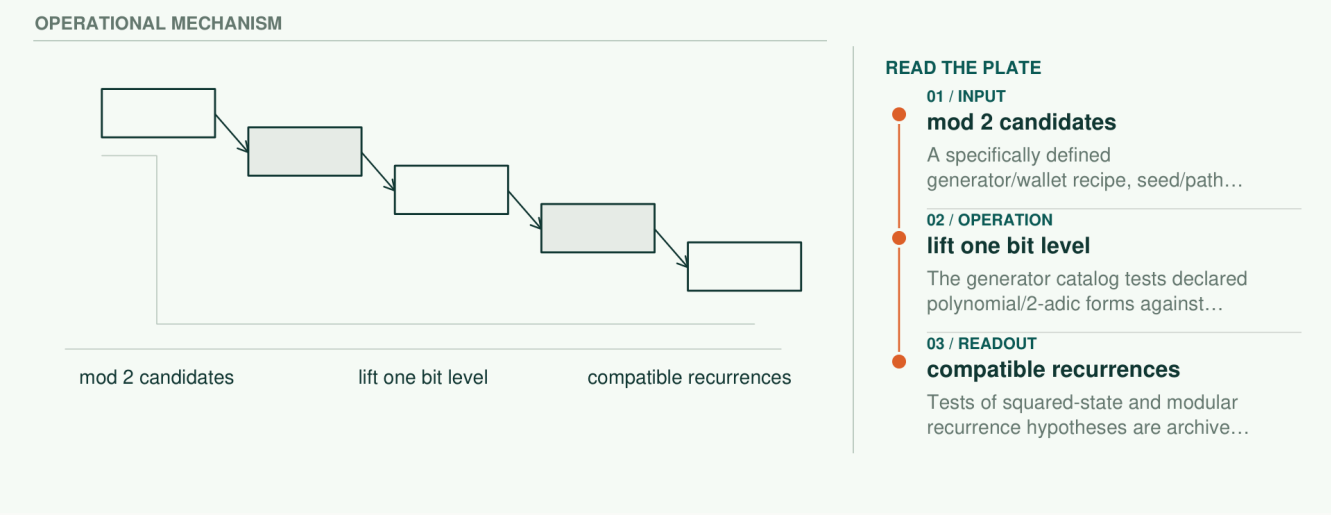
SECTION 06 / GENERATOR RECONSTRUCTION, MASKING AND DETERMINISTIC SEQUENCES

Two-adic/Hensel polynomial recurrence

M089

EVIDENCE STATUS RECORDED RESEARCH

FIGURE / METHOD MECHANISM



HOW IT WORKS

A polynomial recurrence modulo powers of two can constrain parameters one bit at a time; lifting extends compatible low-bit solutions.

INPUTS NEEDED

A specifically defined generator/wallet recipe, seed/path hypotheses and known keys or public targets for independent comparison.

WHAT WAS ACTUALLY DONE

The generator catalog tests declared polynomial/2-adic forms against known puzzle transitions. It searches a restricted family, not all possible deterministic key generators.

WHAT THE RESULT MEANS

Tests of squared-state and modular recurrence hypotheses are archived; finite failures are not impossibility theorems.

MATHEMATICAL FORMULATION

$$f(x_j) \equiv 0 \pmod{2^j}, \quad x_{j+1} = x_j + \epsilon 2^j$$

For epsilon in {0,1}, test which lifts satisfy the polynomial congruence at the next precision. Applicability depends on the specified recurrence hypothesis.

EVIDENCE S079 / S135

PAPERS PAPER-002 / PAPER-004 / PAPER-005 / PAPER-006 / PAPER-007 / PAPER-015

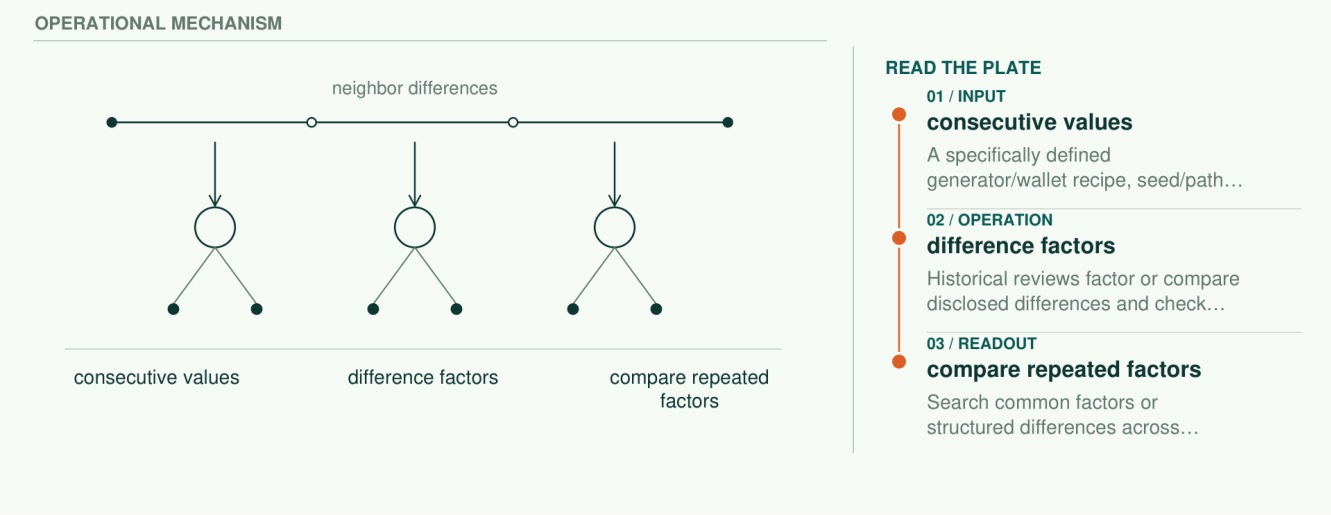
SECTION 06 / GENERATOR RECONSTRUCTION, MASKING AND DETERMINISTIC SEQUENCES

Consecutive-difference factor patterns

M090

EVIDENCE STATUS RECORDED RESEARCH

FIGURE / METHOD MECHANISM



HOW IT WORKS

If consecutive keys share a multiplicative or arithmetic construction, their differences may expose common factors or repeated relations.

INPUTS NEEDED

A specifically defined generator/wallet recipe, seed/path hypotheses and known keys or public targets for independent comparison.

WHAT WAS ACTUALLY DONE

Historical reviews factor or compare disclosed differences and check proposed recurrences. The known interval sizes themselves create scale trends that must be distinguished from secret generator structure.

WHAT THE RESULT MEANS

Search common factors or structured differences across published puzzle keys; no validated target predictor recorded.

MATHEMATICAL FORMULATION

$$g = \gcd(k_2 - k_1, k_3 - k_2, \ldots)$$

Common factors of observed differences can diagnose a proposed recurrence family. They do not by themselves predict a future key or establish a creator algorithm.

EVIDENCE S135

PAPERS PAPER-002 / PAPER-004 / PAPER-005 / PAPER-006 / PAPER-007 / PAPER-008

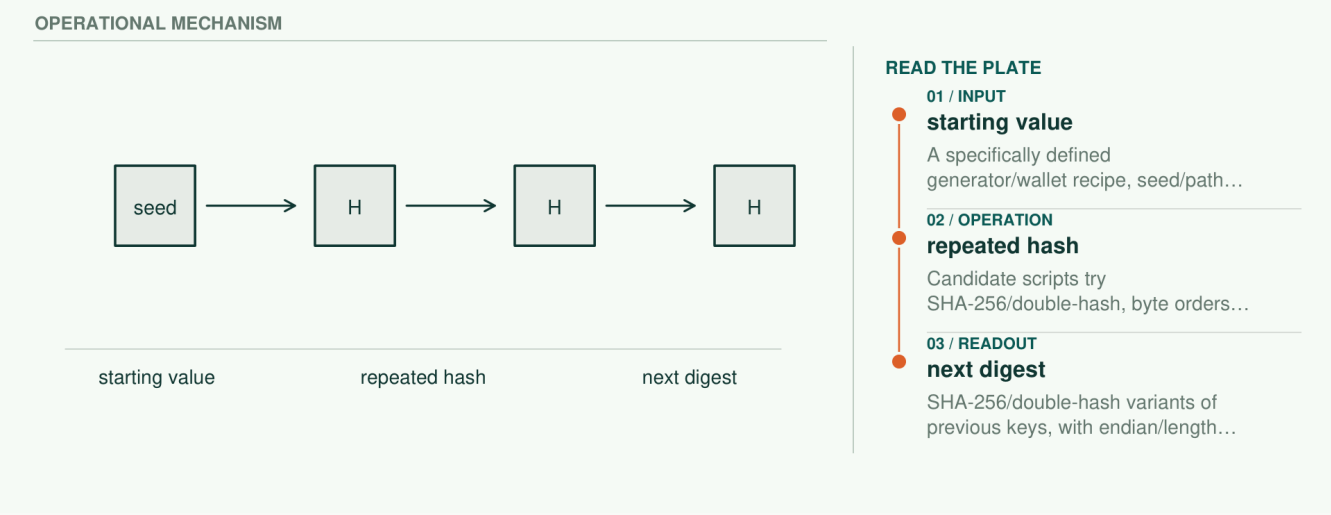
SECTION 06 / GENERATOR RECONSTRUCTION, MASKING AND DETERMINISTIC SEQUENCES

Hash-chain continuation

M091

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

Hash a preceding key in a specified encoding, then mask the digest into the next puzzle's range.

WHAT WAS ACTUALLY DONE

Candidate scripts try SHA-256/double-hash, byte orders and minimal/32-byte lengths. Every recipe is deterministic; zero target matches applies to those encodings and transitions only.

INPUTS NEEDED

A specifically defined generator/wallet recipe, seed/path hypotheses and known keys or public targets for independent comparison.

WHAT THE RESULT MEANS

SHA-256/double-hash variants of previous keys, with endian/length choices and masks, failed named holdout and target checks.

MATHEMATICAL FORMULATION

$$\widehat{k}_{i+1} = \mathrm{Mask}_{i+1}(H(\mathrm{enc}(k_i)))$$

H, byte order, input length and masking are all part of the tested family. Different choices produce different candidate chains.

EVIDENCE S070 / S068

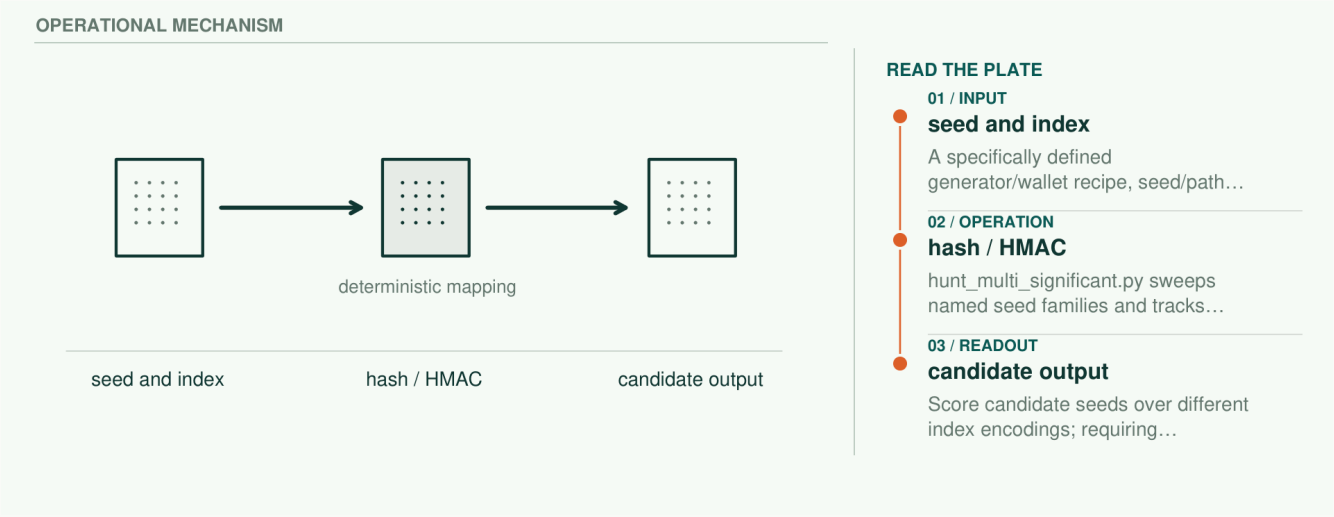
SECTION 06 / GENERATOR RECONSTRUCTION, MASKING AND DETERMINISTIC SEQUENCES

Seed-plus-index hash/HMAC families

M092

EVIDENCE STATUS IMPLEMENTED

FIGURE / METHOD MECHANISM



HOW IT WORKS

Derive a candidate from a small seed and an index encoding using hashing or HMAC, then compare many disclosed tiers.

INPUTS NEEDED

A specifically defined generator/wallet recipe, seed/path hypotheses and known keys or public targets for independent comparison.

WHAT WAS ACTUALLY DONE

hunt_multi_significant.py sweeps named seed families and tracks nontrivial matches. Matching a one- or two-bit puzzle is expected by chance and is not a recovered creator seed.

WHAT THE RESULT MEANS

Score candidate seeds over different index encodings; requiring significant later-tier matches avoids trivial small-range coincidences.

MATHEMATICAL FORMULATION

$$\hat{k}_i = \mathrm{Mask}_i(\mathrm{HMAC}_s(\mathrm{enc}(i)))$$

A candidate seed s, index encoding and hash construction specify a reproducible family. Ordinary hash variants are separate hypotheses.

EVIDENCE S092

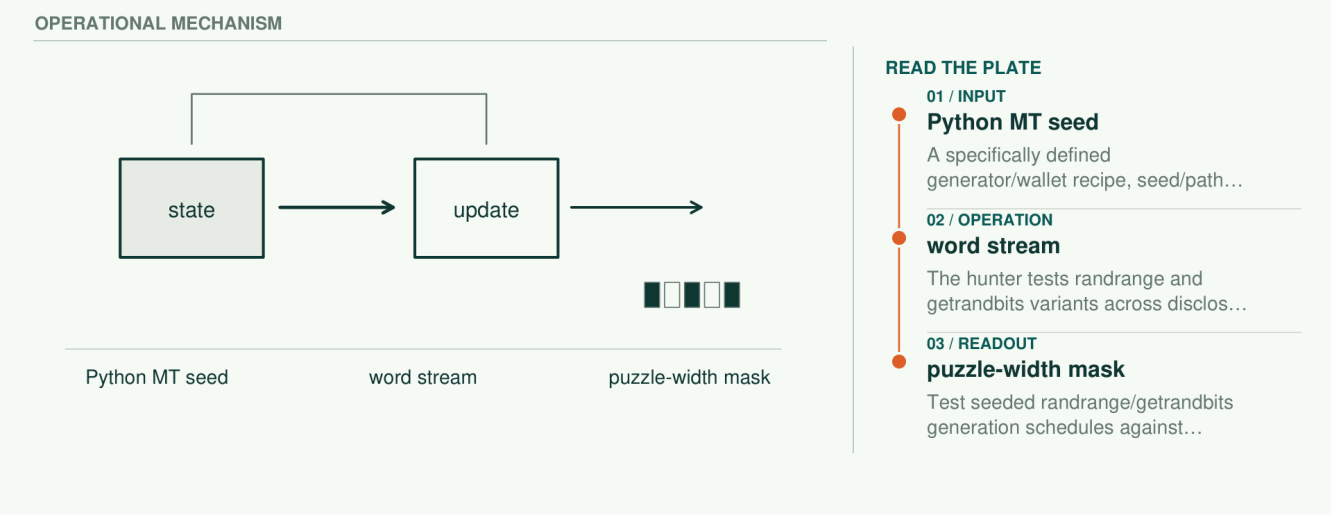
SECTION 06 / GENERATOR RECONSTRUCTION, MASKING AND DETERMINISTIC SEQUENCES

Python MT19937 masked-key replay

M093

EVIDENCE STATUS IMPLEMENTED

FIGURE / METHOD MECHANISM



HOW IT WORKS

Replay Python's seeded PRNG under a specific call schedule and force outputs into each tier.

WHAT WAS ACTUALLY DONE

The hunter tests randrange and getrandbits variants across disclosed keys. Exact software version, seeding and draw order are part of the hypothesis, not generic consequences of MT19937.

INPUTS NEEDED

A specifically defined generator/wallet recipe, seed/path hypotheses and known keys or public targets for independent comparison.

WHAT THE RESULT MEANS

Test seeded randrange/getrandbits generation schedules against published puzzle keys.

MATHEMATICAL FORMULATION

$$\widehat{k}_i = \mathrm{Mask}_i(\mathrm{MT19937}(s; \mathrm{schedule}_i))$$

The seed, getrandbits/randrange recipe and prior output consumption determine each candidate. A generic MT label does not specify the stream.

EVIDENCE S092

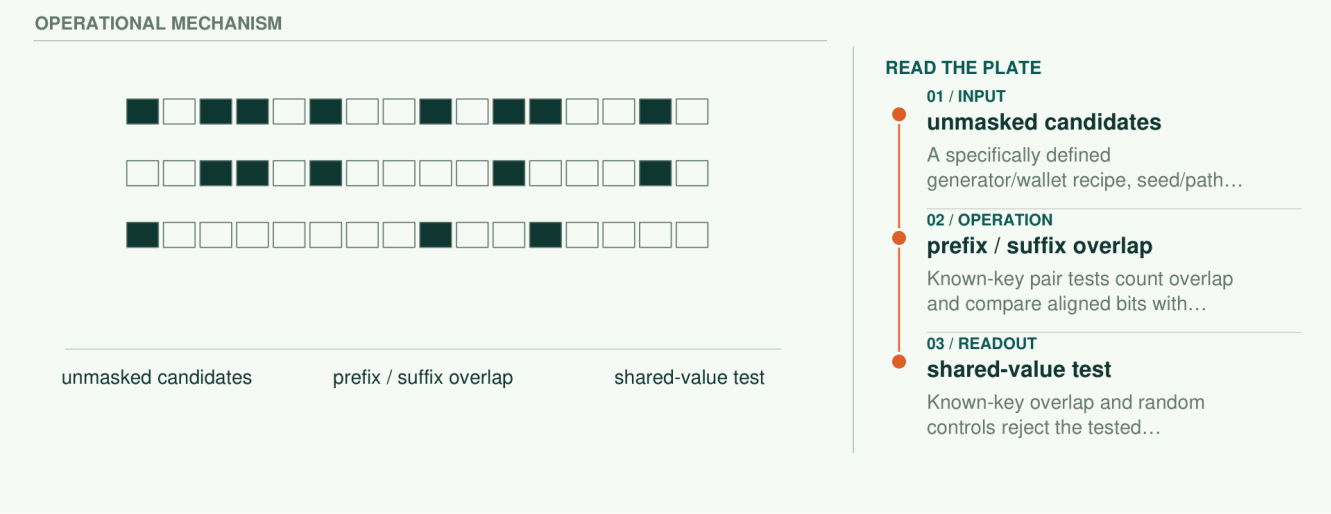
SECTION 06 / GENERATOR RECONSTRUCTION, MASKING AND DETERMINISTIC SEQUENCES

Shared unmasked value, prefix and suffix tests

M094

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

Re-masking one common integer would force predictable prefix/suffix agreement among tiers. Independent draws need not share those bits.

INPUTS NEEDED

A specifically defined generator/wallet recipe, seed/path hypotheses and known keys or public targets for independent comparison.

WHAT WAS ACTUALLY DONE

Known-key pair tests count overlap and compare aligned bits with random controls. They test the common-value construction; ordinary BIP32 child masking does not imply that same shared-value model.

WHAT THE RESULT MEANS

Known-key overlap and random controls reject the tested common-prefix/suffix hypotheses, not every wallet generator.

MATHEMATICAL FORMULATION

$$k_i \equiv K \pmod{2^{b_i}}$$

A shared-unmasked-value hypothesis requires agreement on every jointly observed low bit. Prefix and suffix variants impose different overlap constraints.

EVIDENCE S167

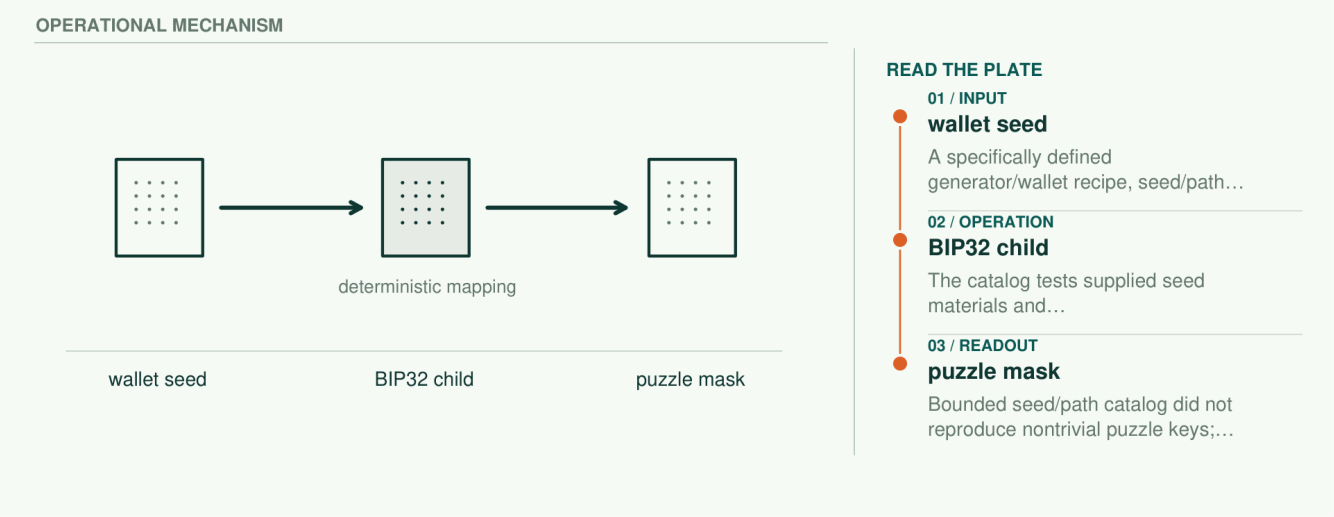
SECTION 06 / GENERATOR RECONSTRUCTION, MASKING AND DETERMINISTIC SEQUENCES

CoinVault/mSIGNA BIP32 plus mask

M095

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

BIP32 derives a child scalar using HMAC-SHA512 and chain codes; the proposed creator mask keeps the required leading bit and lower child bits.

INPUTS NEEDED

A specifically defined generator/wallet recipe, seed/path hypotheses and known keys or public targets for independent comparison.

WHAT WAS ACTUALLY DONE

The catalog tests supplied seed materials and CoinVault/mSIGNA-style paths, with synthetic positive controls. No tested catalog entry reproduces the nontrivial public puzzle sequence.

WHAT THE RESULT MEANS

Bounded seed/path catalog did not reproduce nontrivial puzzle keys; synthetic known-wallet recovery validates the checker.

MATHEMATICAL FORMULATION

$$\hat{k}_i = \mathrm{Mask}_i(\mathrm{CKD}(\mathrm{Master}(s), p_i))$$

Reproduce the wallet's seed-to-master and BIP32 child path p_i exactly before applying the proposed puzzle mask.

EVIDENCE S146

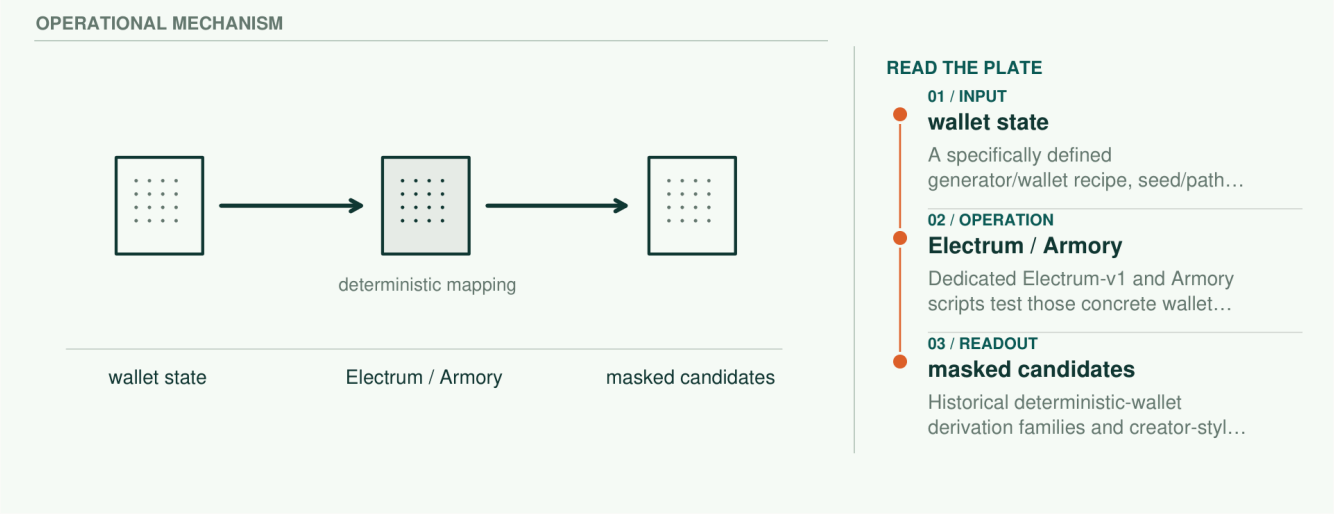
SECTION 06 / GENERATOR RECONSTRUCTION, MASKING AND DETERMINISTIC SEQUENCES

Electrum-v1 and Armory masking models

M096

EVIDENCE STATUS IMPLEMENTED

FIGURE / METHOD MECHANISM



HOW IT WORKS

Historical deterministic wallets can derive children by algorithms different from BIP32. Apply each modeled derivation before the puzzle mask.

INPUTS NEEDED

A specifically defined generator/wallet recipe, seed/path hypotheses and known keys or public targets for independent comparison.

WHAT WAS ACTUALLY DONE

Dedicated Electrum-v1 and Armory scripts test those concrete wallet families. Their existence does not establish that the creator used either wallet or that its seed is enumerable.

WHAT THE RESULT MEANS

Historical deterministic-wallet derivation families and creator-style masks have dedicated test scripts.

MATHEMATICAL FORMULATION

$$\widehat{k}_i = \mathrm{Mask}_i(\mathrm{Derive}_{\mathrm{wallet}}(s, i))$$

Electrum-v1 and Armory have distinct historical derivation procedures. This interface states their required inputs without treating them as interchangeable.

EVIDENCE S230 / S229

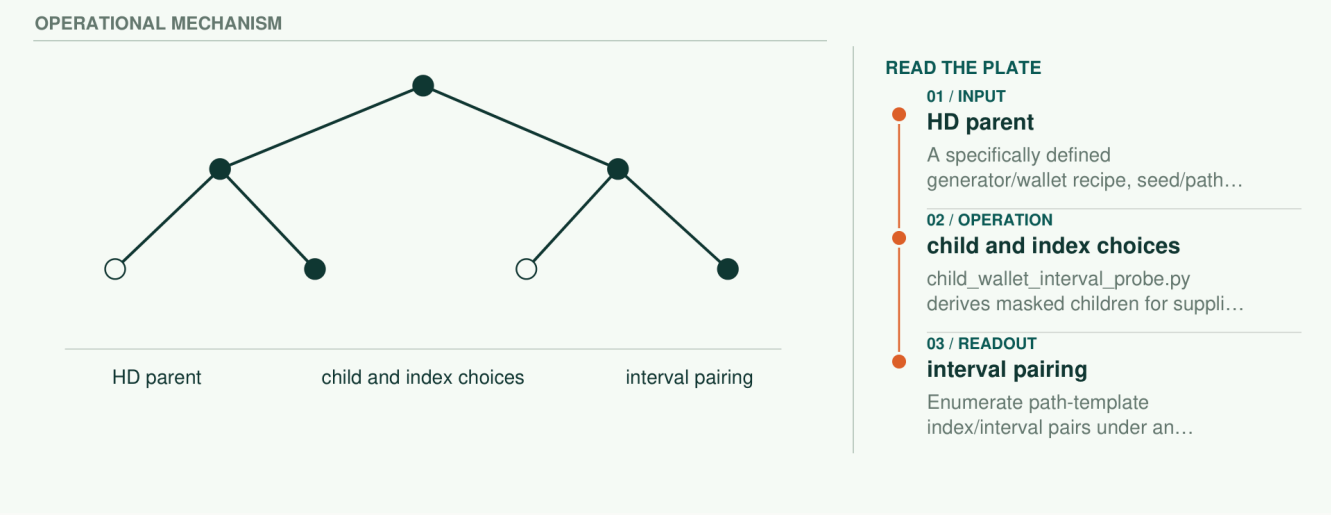
SECTION 06 / GENERATOR RECONSTRUCTION, MASKING AND DETERMINISTIC SEQUENCES

BIP32 child/index interval pairing

M097

EVIDENCE STATUS IMPLEMENTED

FIGURE / METHOD MECHANISM



HOW IT WORKS

Treat child derivation indices as a pair such as start index and interval, and test path templates built from those integers.

INPUTS NEEDED

A specifically defined generator/wallet recipe, seed/path hypotheses and known keys or public targets for independent comparison.

WHAT WAS ACTUALLY DONE

child_wallet_interval_probe.py derives masked children for supplied seeds and compares public targets. A synthetic known-template test checks the probe; unknown creator provenance remains missing.

WHAT THE RESULT MEANS

Enumerate path-template index/interval pairs under an explicitly supplied seed; synthetic self-test is separate from creator identification.

MATHEMATICAL FORMULATION

$$(p, i, N) \longmapsto \mathrm{Mask}_N(\mathrm{CKD}(s, p/i))$$

Enumerate the declared path templates, child indices and puzzle intervals under the supplied seed. Synthetic tests do not identify the puzzle creator's seed.

EVIDENCE S218

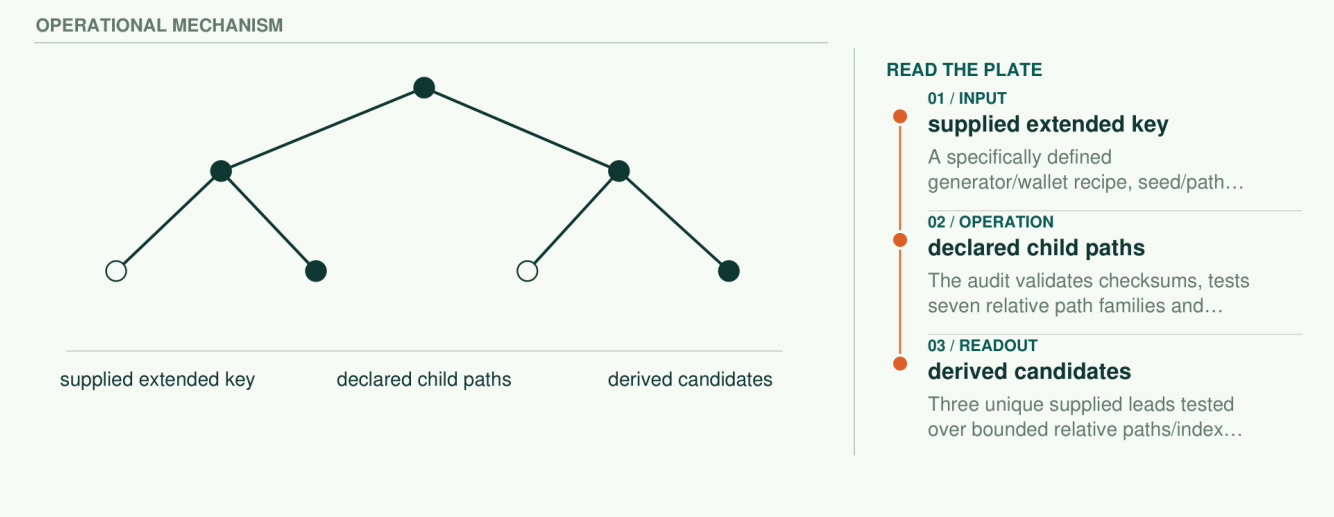
SECTION 06 / GENERATOR RECONSTRUCTION, MASKING AND DETERMINISTIC SEQUENCES

Supplied extended-key derivation leads

M098

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

An explicitly supplied extended private node allows deterministic child derivation along a bounded list of relative paths.

INPUTS NEEDED

A specifically defined generator/wallet recipe, seed/path hypotheses and known keys or public targets for independent comparison.

WHAT WAS ACTUALLY DONE

The audit validates checksums, tests seven relative path families and index shifts, then compares initial nontrivial probe keys. It never prints or republishes the supplied secret nodes.

WHAT THE RESULT MEANS

Three unique supplied leads tested over bounded relative paths/index shifts; no reproduction of nontrivial probe keys. Secrets are not included here.

MATHEMATICAL FORMULATION

$$\mathcal{C}_N = \left\{ \mathrm{Mask}_N(\mathrm{CKD}(x, p)) : p \in \mathcal{P} \right\}$$

x denotes the supplied extended-key lead, omitted from the report. P is a finite set of relative paths and shifts.

EVIDENCE S072 / S076

PAPERS PAPER-008

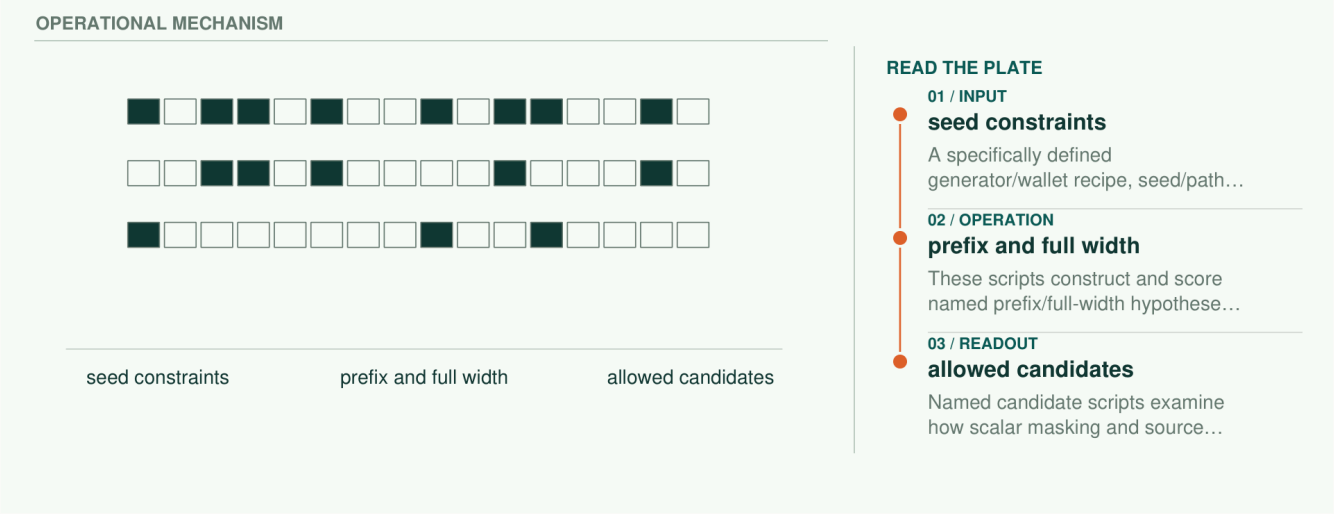
SECTION 06 / GENERATOR RECONSTRUCTION, MASKING AND DETERMINISTIC SEQUENCES

Entropy-prefix and full-width constraints

M099

EVIDENCE STATUS IMPLEMENTED

FIGURE / METHOD MECHANISM



HOW IT WORKS

A known source-entropy prefix or full-width relation could restrict which masked outputs are possible. It must be derived independently of the hidden key.

INPUTS NEEDED

A specifically defined generator/wallet recipe, seed/path hypotheses and known keys or public targets for independent comparison.

WHAT WAS ACTUALLY DONE

These scripts construct and score named prefix/full-width hypotheses. They do not turn a public difficulty interval into additional leaked secret bits without matching evidence.

WHAT THE RESULT MEANS

Named candidate scripts examine how scalar masking and source entropy could constrain keys; no generic recovery follows.

MATHEMATICAL FORMULATION

$$k_N = 2^{N-1} + (K \bmod 2^{N-1})$$

The creator-style mask fixes the leading interval bit and exposes low bits of a hypothetical source K. Full-width appearance does not prove full source entropy.

EVIDENCE S231 / S232

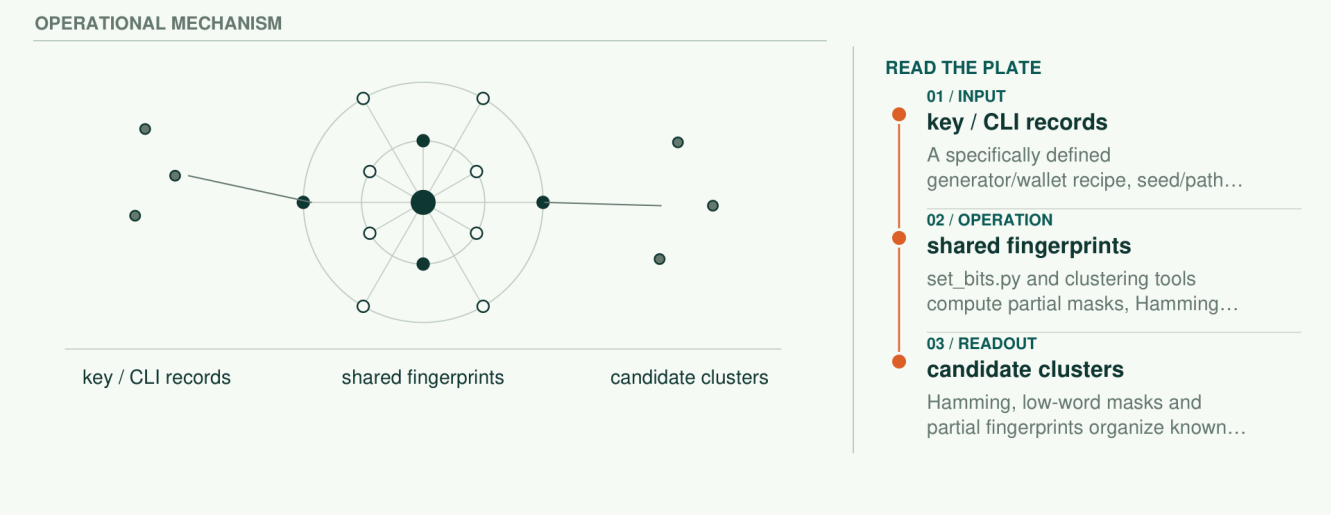
SECTION 06 / GENERATOR RECONSTRUCTION, MASKING AND DETERMINISTIC SEQUENCES

CLI fingerprint and key clustering

M100

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

A short fingerprint of known components can group similar CLI outputs or candidate tuples. It helps organize evidence and detect duplicate hypotheses.

INPUTS NEEDED

A specifically defined generator/wallet recipe, seed/path hypotheses and known keys or public targets for independent comparison.

WHAT WAS ACTUALLY DONE

set_bits.py and clustering tools compute partial masks, Hamming statistics and groups. A matching fingerprint is a lossy summary, not proof of an exact scalar or valid search cut.

WHAT THE RESULT MEANS

Hamming, low-word masks and partial fingerprints organize known coordinates; not established target narrowing.

MATHEMATICAL FORMULATION

$$d_H(x,y) = \mathrm{popcount}(x \oplus y)$$

Hamming distances and masked fingerprints compare aligned known coordinates. A cluster is descriptive unless it predicts independently verified targets.

EVIDENCE S122 / S078

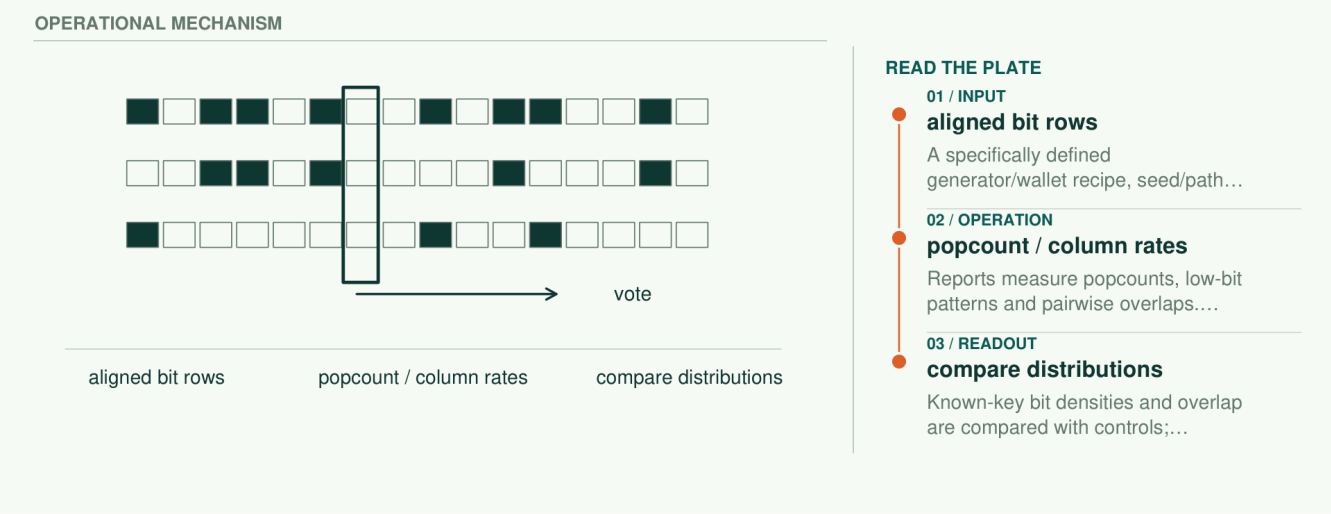
SECTION 06 / GENERATOR RECONSTRUCTION, MASKING AND DETERMINISTIC SEQUENCES

Popcount and aligned-bit distribution tests

M101

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

Compare the number of set bits and aligned bit agreements with the distribution expected from uniform interval keys.

INPUTS NEEDED

A specifically defined generator/wallet recipe, seed/path hypotheses and known keys or public targets for independent comparison.

WHAT WAS ACTUALLY DONE

Reports measure popcounts, low-bit patterns and pairwise overlaps. Statistical similarity/difference concerns the tested corpus; it does not directly provide the unknown bits of the next key.

WHAT THE RESULT MEANS

Known-key bit densities and overlap are compared with controls; observed appearance is not a scalar equation.

MATHEMATICAL FORMULATION

$$w(k) = \sum_j b_j, \quad \widehat{p}_j = \tfrac{1}{m} \sum_i b_{ij}$$

Popcount w and aligned-bit frequencies summarize disclosed keys; matched controls must respect tier widths and fixed leading bits.

EVIDENCE S185
PAPERS PAPER-002

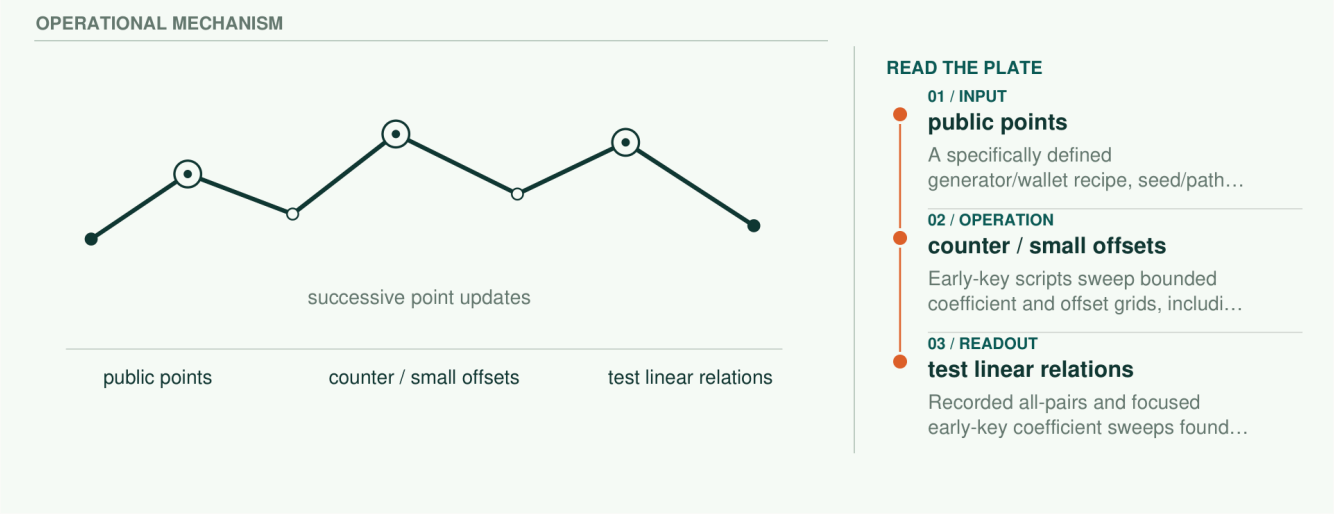
SECTION 06 / GENERATOR RECONSTRUCTION, MASKING AND DETERMINISTIC SEQUENCES

Counter/small-affine early public-key relations

M102

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

Small scalar relations are visible as public-point relations: alpha*P_i+beta*P_j+gamma*G equals the identity if the corresponding scalars satisfy them.

INPUTS NEEDED

A specifically defined generator/wallet recipe, seed/path hypotheses and known keys or public targets for independent comparison.

WHAT WAS ACTUALLY DONE

Early-key scripts sweep bounded coefficient and offset grids, including nonce-to-signer pairs. A genuine relation could help, but the saved finite sweeps report no such target recovery.

WHAT THE RESULT MEANS

Recorded all-pairs and focused early-key coefficient sweeps found no relation in their finite coefficient/offset bounds.

MATHEMATICAL FORMULATION

$$Q_j \stackrel{?}{=} aQ_i + bG$$

Search only the declared coefficient and offset bounds. A verified relation would imply k_j congruent to a k_i+b modulo subgroup order n.

EVIDENCE S155 / S156

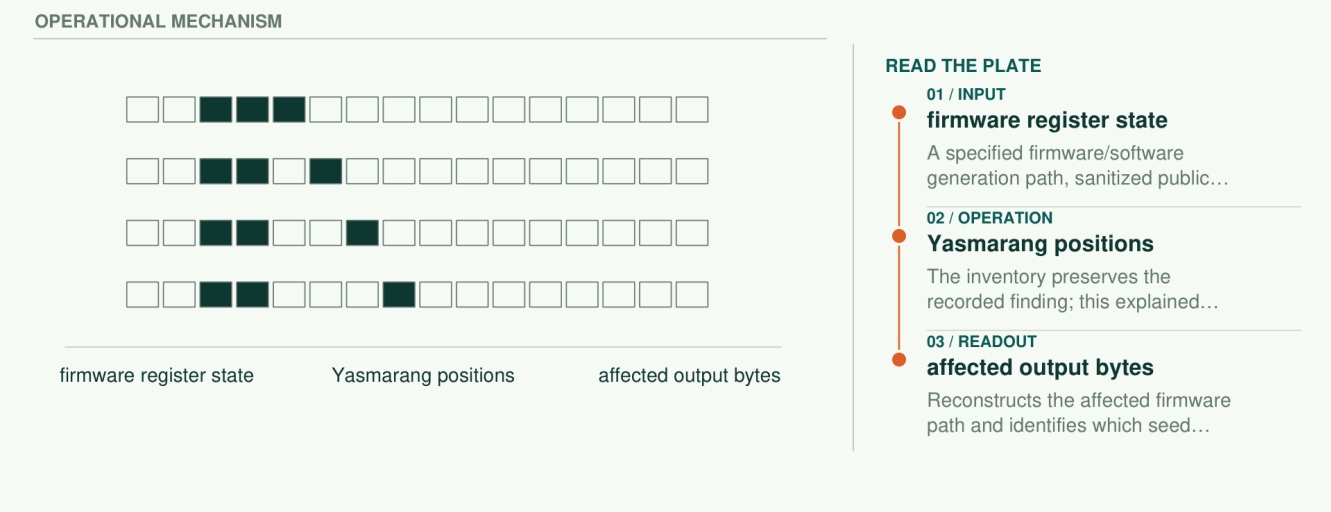
SECTION 07 / FIRMWARE ENTROPY AND READ-ONLY WALLET FORENSICS

Firmware-accurate Coldcard Yasmarang entropy-position analysis

M103

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

The method is defined by the named evidence files and recorded scope.

WHAT WAS ACTUALLY DONE

The inventory preserves the recorded finding; this explained edition does not rerun or independently certify the source.

INPUTS NEEDED

A specified firmware/software generation path, sanitized public address/index data and explicit read-only chain or balance evidence; no private keys are required or reproduced.

WHAT THE RESULT MEANS

Reconstructs the affected firmware path and identifies which seed registers and skip positions influence generated bytes; it does not establish wallet recovery or an address match.

MATHEMATICAL FORMULATION

No explicit formula has been editorially assigned to this newer method entry.

EVIDENCE S061

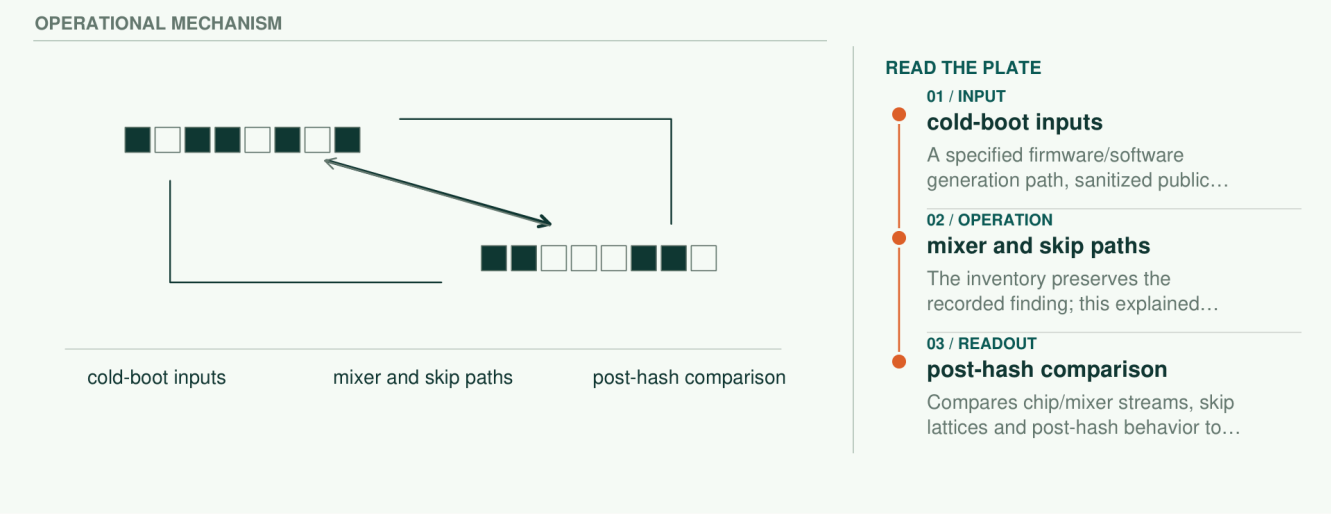
SECTION 07 / FIRMWARE ENTROPY AND READ-ONLY WALLET FORENSICS

Cold-boot entropy propagation and mixer analysis

M104

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

The method is defined by the named evidence files and recorded scope.

WHAT WAS ACTUALLY DONE

The inventory preserves the recorded finding; this explained edition does not rerun or independently certify the source.

INPUTS NEEDED

A specified firmware/software generation path, sanitized public address/index data and explicit read-only chain or balance evidence; no private keys are required or reproduced.

WHAT THE RESULT MEANS

Compares chip/mixer streams, skip lattices and post-hash behavior to distinguish low-entropy inputs from claims that the mixer itself creates an address family; no spend path or address matching is performed.

MATHEMATICAL FORMULATION

No explicit formula has been editorially assigned to this newer method entry.

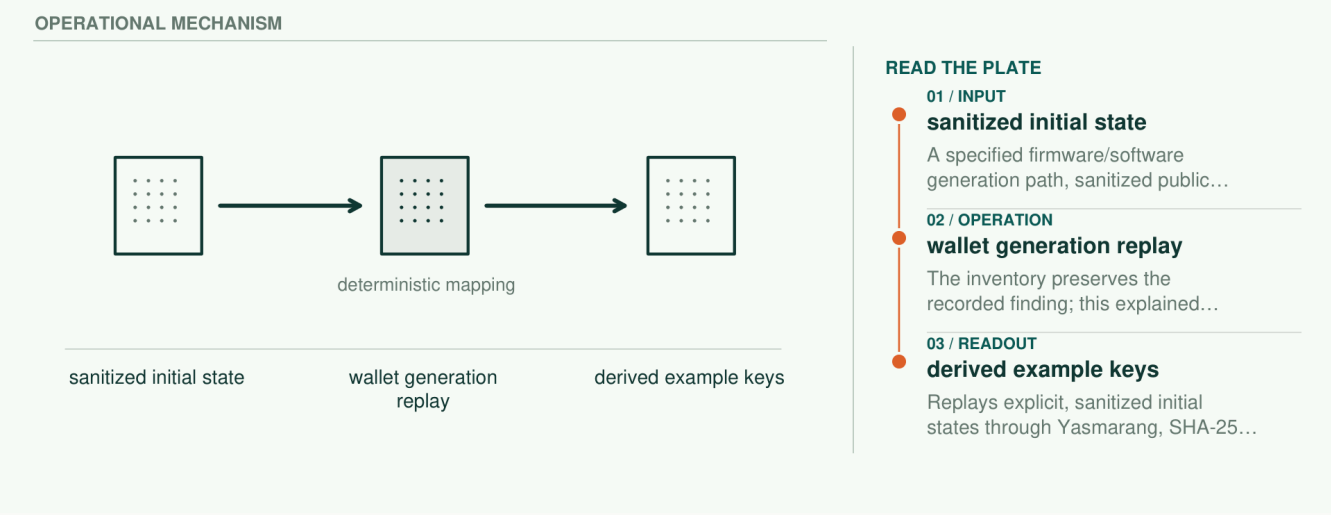
EVIDENCE S060

SECTION 07 / FIRMWARE ENTROPY AND READ-ONLY WALLET FORENSICS

Local Coldcard wallet-generation replay M105

EVIDENCE STATUS IMPLEMENTED

FIGURE / METHOD MECHANISM



HOW IT WORKS

The method is defined by the named evidence files and recorded scope.

WHAT WAS ACTUALLY DONE

The inventory preserves the recorded finding; this explained edition does not rerun or independently certify the source.

INPUTS NEEDED

A specified firmware/software generation path, sanitized public address/index data and explicit read-only chain or balance evidence; no private keys are required or reproduced.

WHAT THE RESULT MEANS

Replays explicit, sanitized initial states through Yasmarang, SHA-256 whitening and BIP-39 formatting for implementation analysis; generated examples are not evidence of a historical wallet seed.

MATHEMATICAL FORMULATION

No explicit formula has been editorially assigned to this newer method entry.

EVIDENCE S059

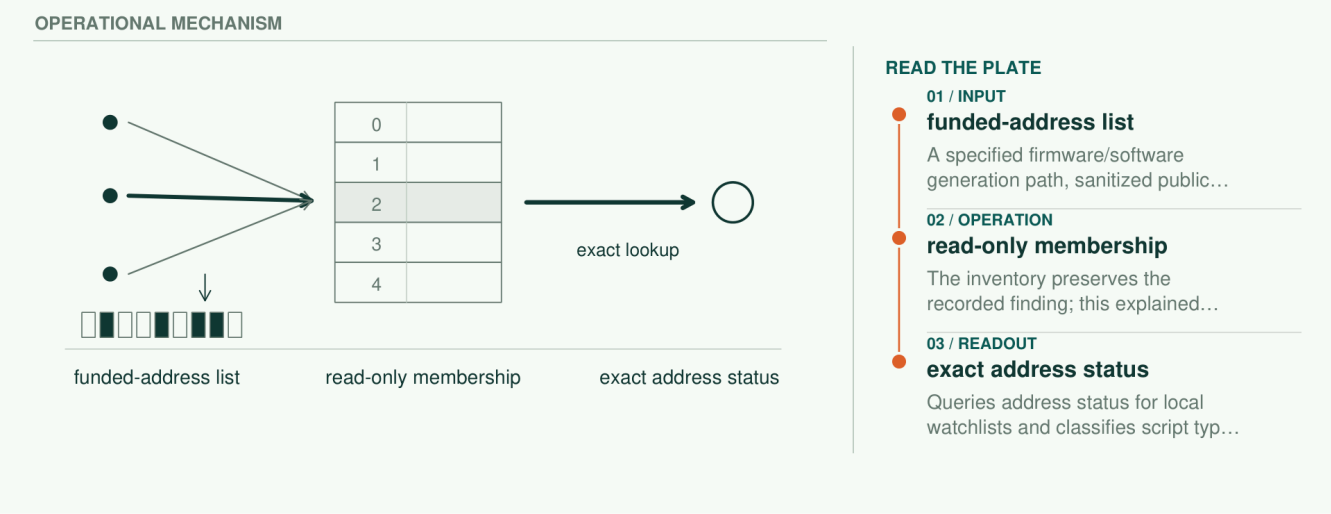
SECTION 07 / FIRMWARE ENTROPY AND READ-ONLY WALLET FORENSICS

Read-only funded-address membership analysis

M106

EVIDENCE STATUS IMPLEMENTED

FIGURE / METHOD MECHANISM



HOW IT WORKS

The method is defined by the named evidence files and recorded scope.

WHAT WAS ACTUALLY DONE

The inventory preserves the recorded finding; this explained edition does not rerun or independently certify the source.

INPUTS NEEDED

A specified firmware/software generation path, sanitized public address/index data and explicit read-only chain or balance evidence; no private keys are required or reproduced.

WHAT THE RESULT MEANS

Queries address status for local watchlists and classifies script types; membership is not proof of seed ownership, key recovery or a valid spend path.

MATHEMATICAL FORMULATION

No explicit formula has been editorially assigned to this newer method entry.

EVIDENCE S062

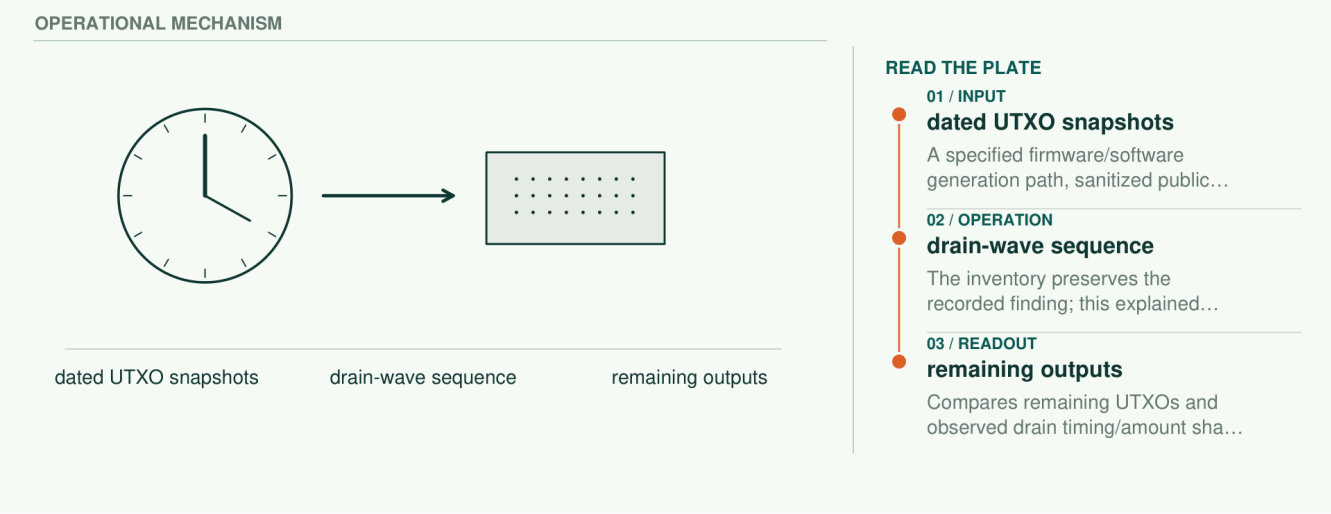
SECTION 07 / FIRMWARE ENTROPY AND READ-ONLY WALLET FORENSICS

Read-only residual-UTXO and drain-wave analysis

M107

EVIDENCE STATUS IMPLEMENTED

FIGURE / METHOD MECHANISM



HOW IT WORKS

The method is defined by the named evidence files and recorded scope.

WHAT WAS ACTUALLY DONE

The inventory preserves the recorded finding; this explained edition does not rerun or independently certify the source.

INPUTS NEEDED

A specified firmware/software generation path, sanitized public address/index data and explicit read-only chain or balance evidence; no private keys are required or reproduced.

WHAT THE RESULT MEANS

Compares remaining UTXOs and observed drain timing/amount shape against a declared block boundary; chain observations do not identify a private key.

MATHEMATICAL FORMULATION

No explicit formula has been editorially assigned to this newer method entry.

EVIDENCE S063

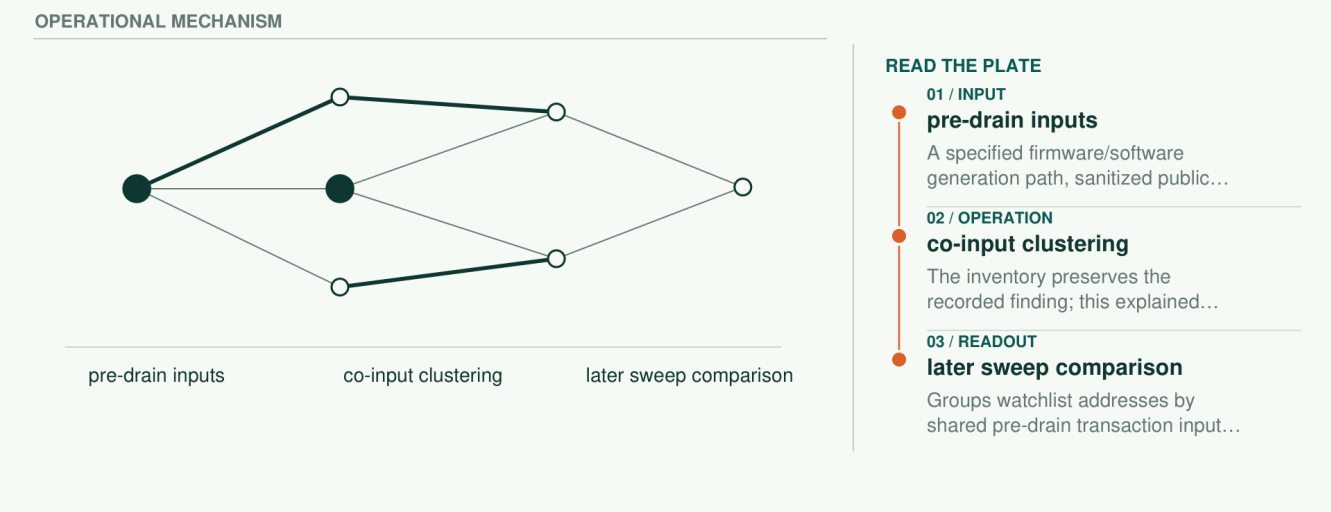
SECTION 07 / FIRMWARE ENTROPY AND READ-ONLY WALLET FORENSICS

Pre-drain co-input and address-cluster analysis

M108

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

The method is defined by the named evidence files and recorded scope.

WHAT WAS ACTUALLY DONE

The inventory preserves the recorded finding; this explained edition does not rerun or independently certify the source.

INPUTS NEEDED

A specified firmware/software generation path, sanitized public address/index data and explicit read-only chain or balance evidence; no private keys are required or reproduced.

WHAT THE RESULT MEANS

Groups watchlist addresses by shared pre-drain transaction inputs and separates those from later sweep co-inputs; the saved result is chain evidence, not proof of common seed derivation.

MATHEMATICAL FORMULATION

No explicit formula has been editorially assigned to this newer method entry.

EVIDENCE S065 / S064

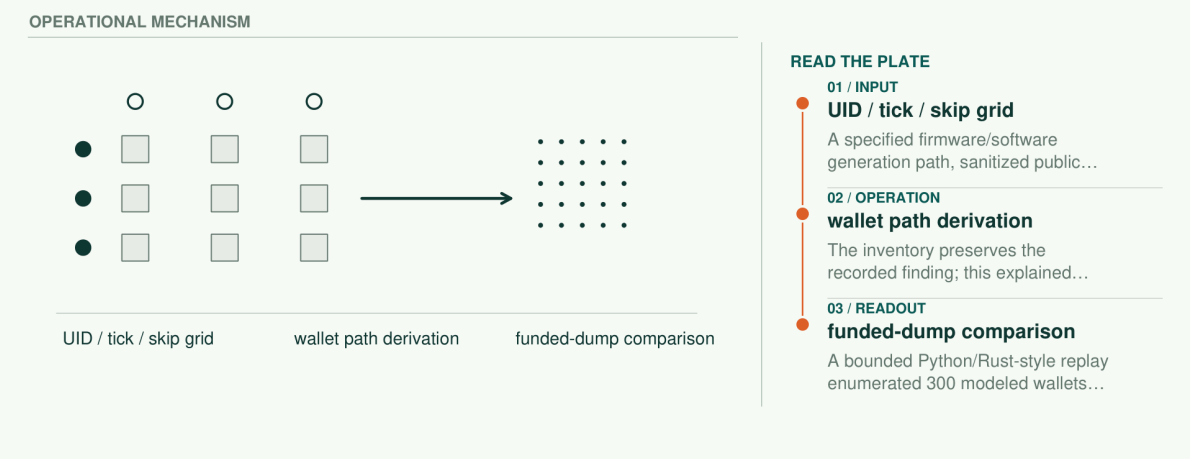
SECTION 07 / FIRMWARE ENTROPY AND READ-ONLY WALLET FORENSICS

Bounded Coldcard entropy-grid enumeration against a funded-address dump

M109

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

The method is defined by the named evidence files and recorded scope.

WHAT WAS ACTUALLY DONE

The inventory preserves the recorded finding; this explained edition does not rerun or independently certify the source.

INPUTS NEEDED

A specified firmware/software generation path, sanitized public address/index data and explicit read-only chain or balance evidence; no private keys are required or reproduced.

WHAT THE RESULT MEANS

A bounded Python/Rust-style replay enumerated 300 modeled wallets across explicit UID, SysTick, skip and reseed grids, deriving BIP44/49/84 receive/change paths and comparing against a dated address-balance dump; listed hits validate membership only and do not establish seed ownership or exhaustive coverage.

MATHEMATICAL FORMULATION

No explicit formula has been editorially assigned to this newer method entry.

EVIDENCE S052 / S053

PAPERS PAPER-014

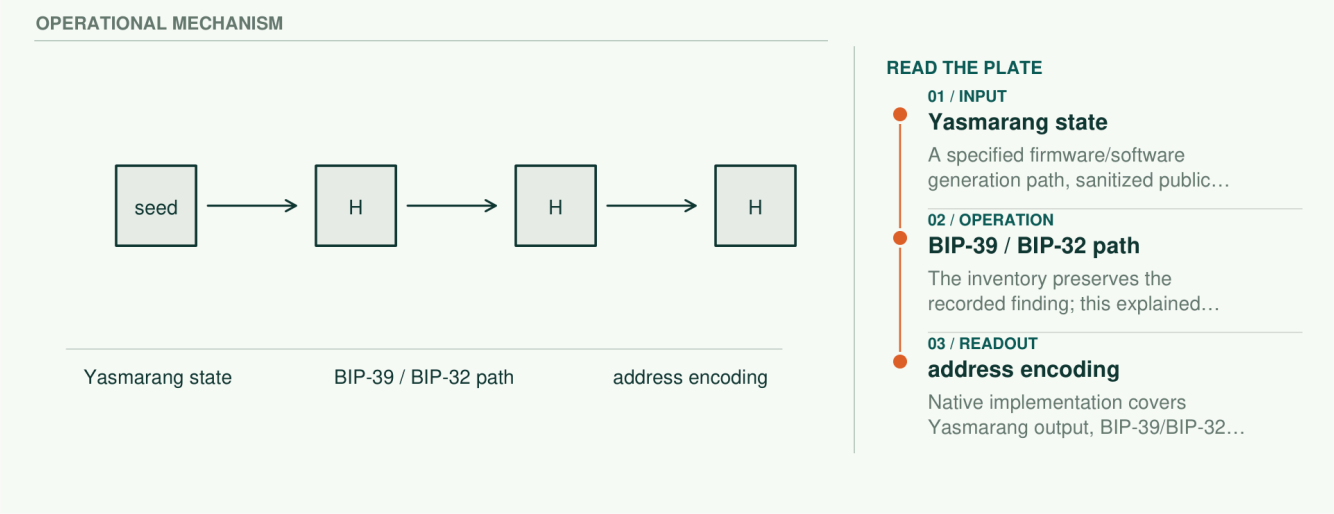
SECTION 07 / FIRMWARE ENTROPY AND READ-ONLY WALLET FORENSICS

Rust Coldcard address-derivation backend

M110

EVIDENCE STATUS IMPLEMENTED

FIGURE / METHOD MECHANISM



HOW IT WORKS

The method is defined by the named evidence files and recorded scope.

WHAT WAS ACTUALLY DONE

The inventory preserves the recorded finding; this explained edition does not rerun or independently certify the source.

INPUTS NEEDED

A specified firmware/software generation path, sanitized public address/index data and explicit read-only chain or balance evidence; no private keys are required or reproduced.

WHAT THE RESULT MEANS

Native implementation covers Yasmarang output, BIP-39/BIP-32 derivation and P2PKH/P2WPKH address encoding for bounded enumeration; backend existence is not proof of a historical match.

MATHEMATICAL FORMULATION

No explicit formula has been editorially assigned to this newer method entry.

EVIDENCE S055 / S054 / S051

PAPERS PAPER-014

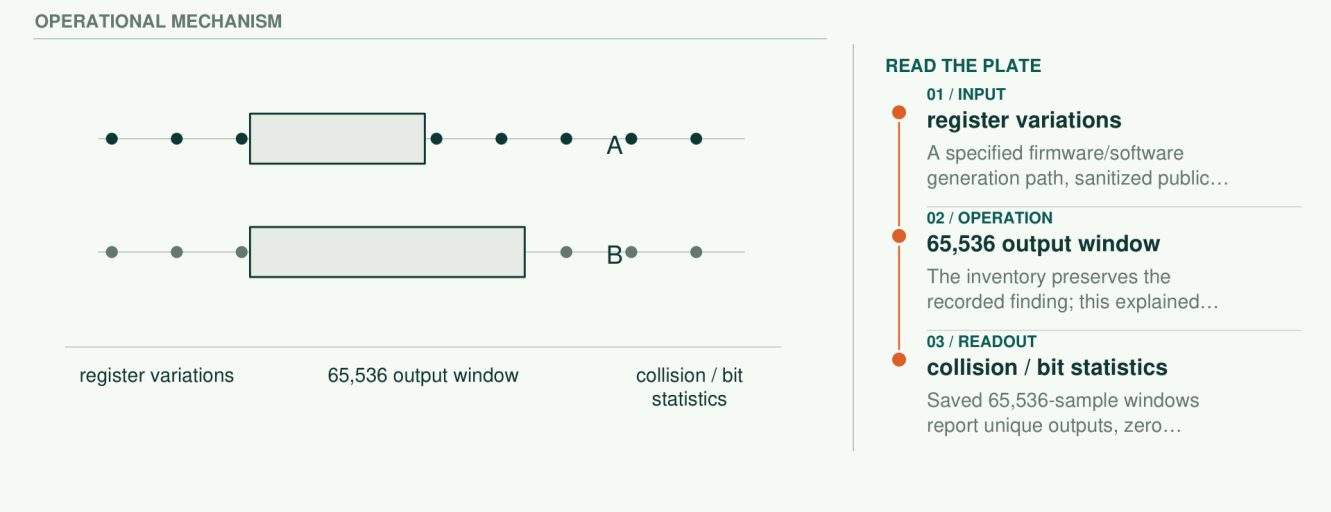
SECTION 07 / FIRMWARE ENTROPY AND READ-ONLY WALLET FORENSICS

Coldcard entropy statistical variation sweep

M111

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

The method is defined by the named evidence files and recorded scope.

WHAT WAS ACTUALLY DONE

The inventory preserves the recorded finding; this explained edition does not rerun or independently certify the source.

INPUTS NEEDED

A specified firmware/software generation path, sanitized public address/index data and explicit read-only chain or balance evidence; no private keys are required or reproduced.

WHAT THE RESULT MEANS

Saved 65,536-sample windows report unique outputs, zero observed collisions and near-balanced first-byte/bit statistics under synthetic register variations; this measures the modeled output distribution, not wallet recovery.

MATHEMATICAL FORMULATION

No explicit formula has been editorially assigned to this newer method entry.

EVIDENCE S057 / S056 / S058

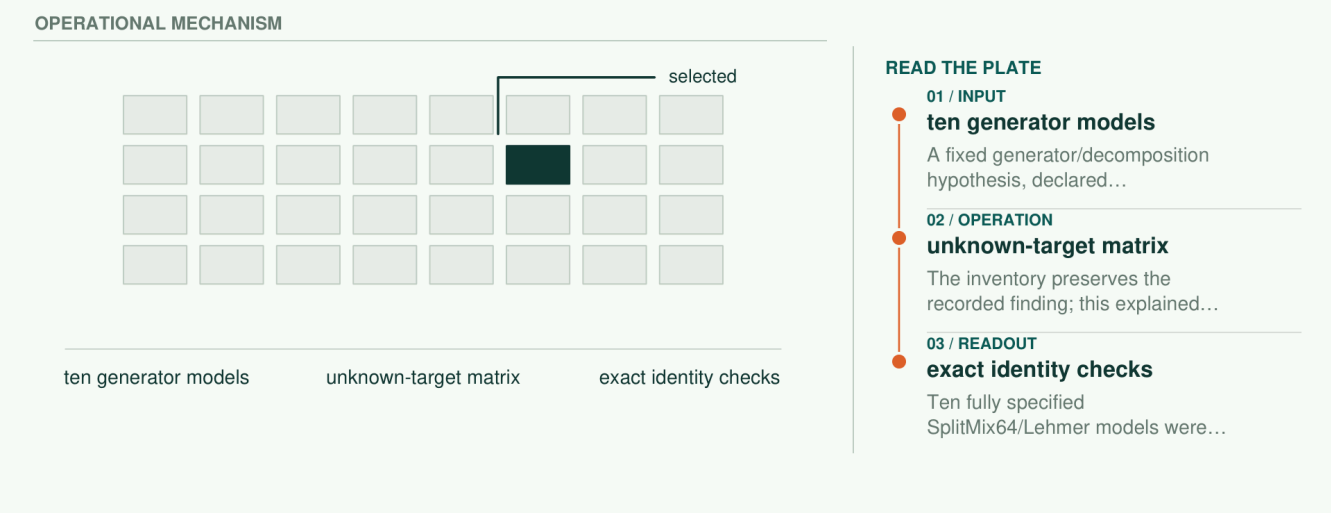
SECTION 08 / DATED GENERATOR AND DECOMPOSITION FOLLOW-UP AUDITS

All-unknown-target generator matrix

M112

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

The method is defined by the named evidence files and recorded scope.

WHAT WAS ACTUALLY DONE

The inventory preserves the recorded finding; this explained edition does not rerun or independently certify the source.

INPUTS NEEDED

A fixed generator/decomposition hypothesis, declared solved/unsolved corpus, target identity checks and range-matched controls where statistical claims are made.

WHAT THE RESULT MEANS

Ten fully specified SplitMix64/Lehmer models were checked against 770 unknown-target identities, plus 97,216 low-bit-consistent candidates where feasible; zero matches, with prior solved-key failures limiting interpretation.

MATHEMATICAL FORMULATION

No explicit formula has been editorially assigned to this newer method entry.

EVIDENCE S206 / S160

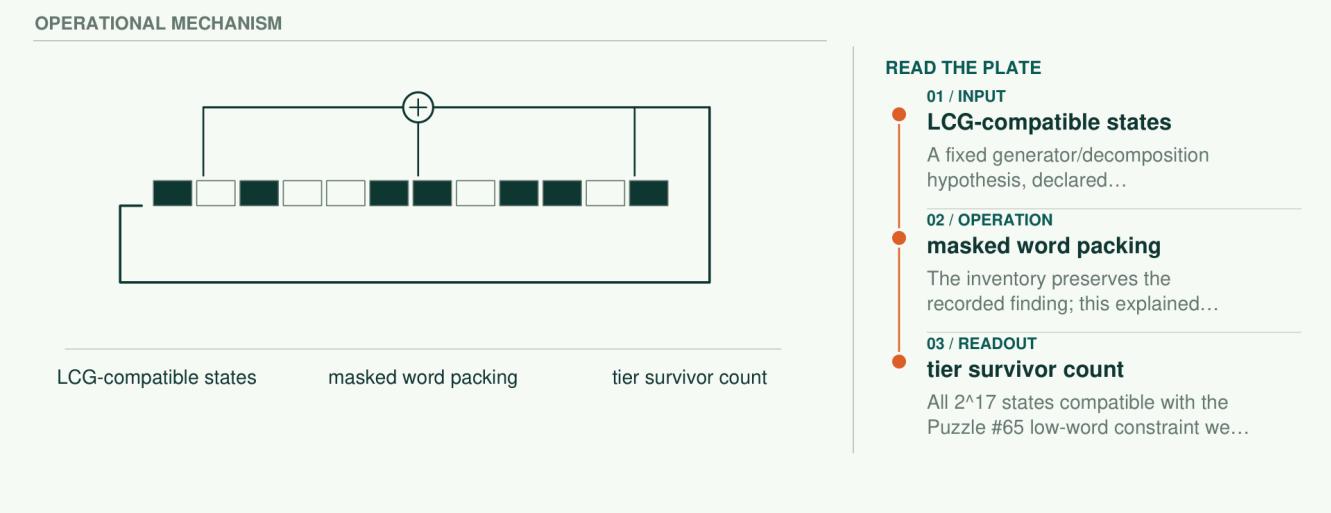
SECTION 08 / DATED GENERATOR AND DECOMPOSITION FOLLOW-UP AUDITS

Wallace LCG masked-word backtest

M113

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

The method is defined by the named evidence files and recorded scope.

WHAT WAS ACTUALLY DONE

The inventory preserves the recorded finding; this explained edition does not rerun or independently certify the source.

INPUTS NEEDED

A fixed generator/decomposition hypothesis, declared solved/unsolved corpus, target identity checks and range-matched controls where statistical claims are made.

WHAT THE RESULT MEANS

All 2^17 states compatible with the Puzzle #65 low-word constraint were tested under 17/18-word packing; survivors fell to zero by tier 67 under the stated mask and schedule.

MATHEMATICAL FORMULATION

No explicit formula has been editorially assigned to this newer method entry.

EVIDENCE S216 / S194

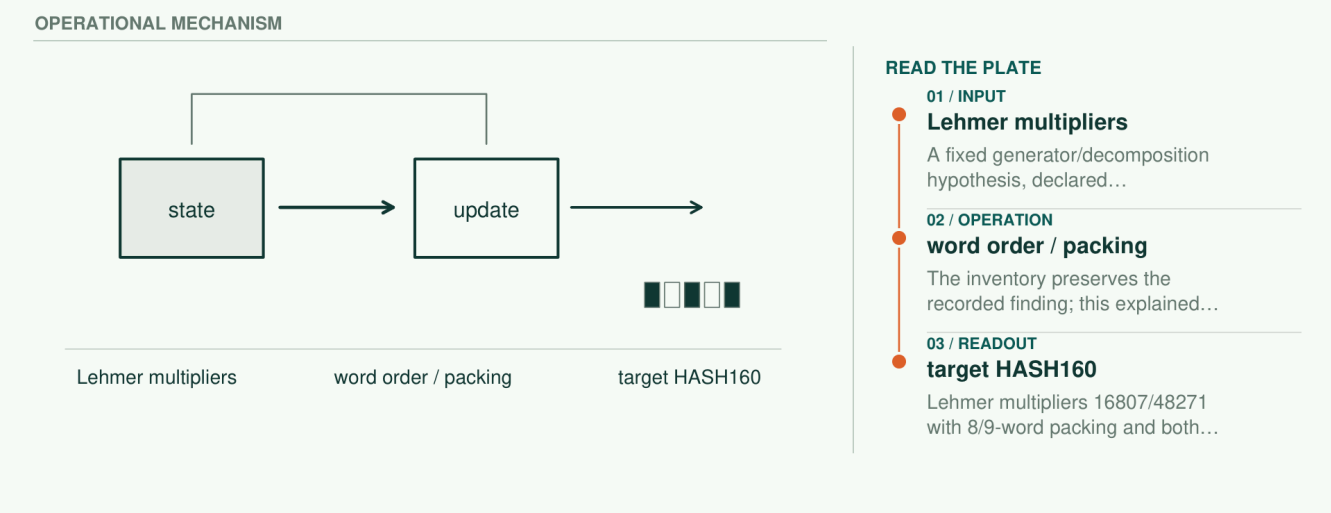
SECTION 08 / DATED GENERATOR AND DECOMPOSITION FOLLOW-UP AUDITS

Park-Miller packed-word target checks

M114

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

The method is defined by the named evidence files and recorded scope.

WHAT WAS ACTUALLY DONE

The inventory preserves the recorded finding; this explained edition does not rerun or independently certify the source.

INPUTS NEEDED

A fixed generator/decomposition hypothesis, declared solved/unsolved corpus, target identity checks and range-matched controls where statistical claims are made.

WHAT THE RESULT MEANS

Lehmer multipliers 16807/48271 with 8/9-word packing and both orders were checked against #66, #71 and #140; candidate target identities did not match.

MATHEMATICAL FORMULATION

No explicit formula has been editorially assigned to this newer method entry.

EVIDENCE S210 / S174

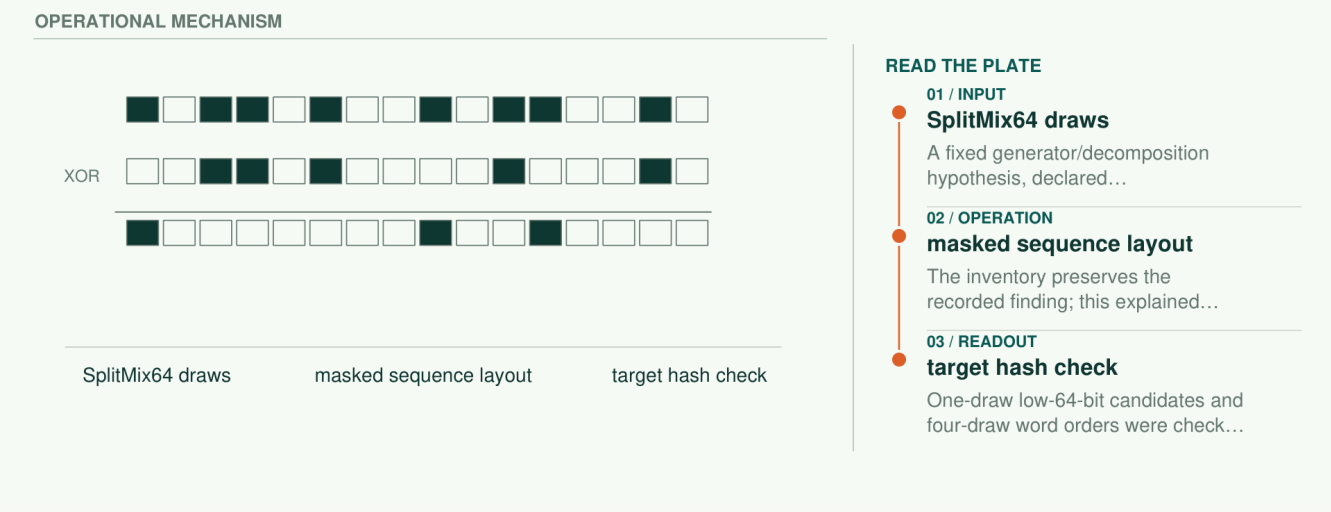
SECTION 08 / DATED GENERATOR AND DECOMPOSITION FOLLOW-UP AUDITS

SplitMix64 masked-sequence target checks

M115

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

The method is defined by the named evidence files and recorded scope.

WHAT WAS ACTUALLY DONE

The inventory preserves the recorded finding; this explained edition does not rerun or independently certify the source.

INPUTS NEEDED

A fixed generator/decomposition hypothesis, declared solved/unsolved corpus, target identity checks and range-matched controls where statistical claims are made.

WHAT THE RESULT MEANS

One-draw low-64-bit candidates and four-draw word orders were checked against target HASH160s; evaluated candidates produced no matches.

MATHEMATICAL FORMULATION

No explicit formula has been editorially assigned to this newer method entry.

EVIDENCE S214 / S188

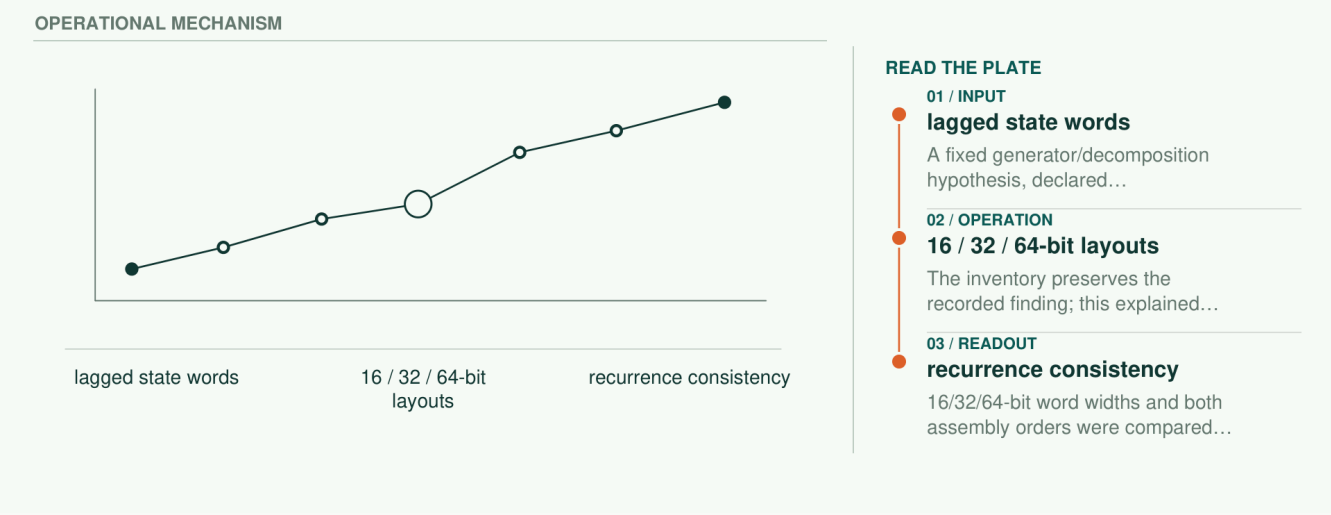
SECTION 08 / DATED GENERATOR AND DECOMPOSITION FOLLOW-UP AUDITS

Lagged-RNG multiword wallet-key audit

M116

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

The method is defined by the named evidence files and recorded scope.

WHAT WAS ACTUALLY DONE

The inventory preserves the recorded finding; this explained edition does not rerun or independently certify the source.

INPUTS NEEDED

A fixed generator/decomposition hypothesis, declared solved/unsolved corpus, target identity checks and range-matched controls where statistical claims are made.

WHAT THE RESULT MEANS

16/32/64-bit word widths and both assembly orders were compared with archived lagged recurrences; the listed contiguous layouts failed their available equations, while underdetermined layouts remain untested.

MATHEMATICAL FORMULATION

No explicit formula has been editorially assigned to this newer method entry.

EVIDENCE S205 / S138

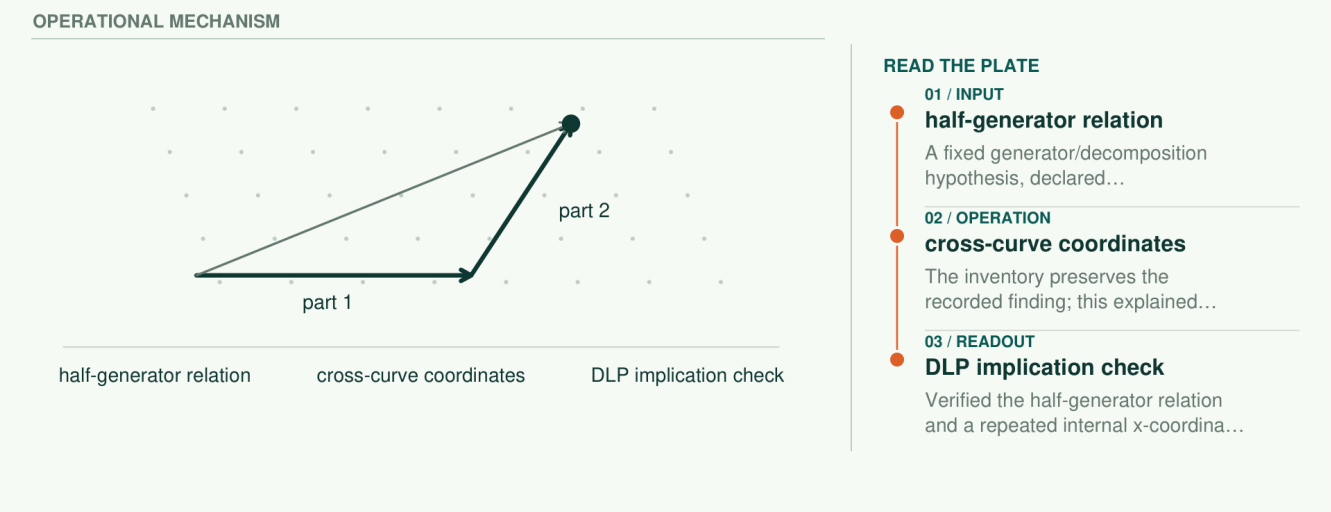
SECTION 08 / DATED GENERATOR AND DECOMPOSITION FOLLOW-UP AUDITS

Half-generator cross-curve coordinate analysis

M117

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

The method is defined by the named evidence files and recorded scope.

WHAT WAS ACTUALLY DONE

The inventory preserves the recorded finding; this explained edition does not rerun or independently certify the source.

INPUTS NEEDED

A fixed generator/decomposition hypothesis, declared solved/unsolved corpus, target identity checks and range-matched controls where statistical claims are made.

WHAT THE RESULT MEANS

Verified the half-generator relation and a repeated internal x-coordinate pattern across four standard curves; the known generator relabeling does not reduce the Bitcoin puzzle discrete-log search space.

MATHEMATICAL FORMULATION

No explicit formula has been editorially assigned to this newer method entry.

EVIDENCE S207 / S162

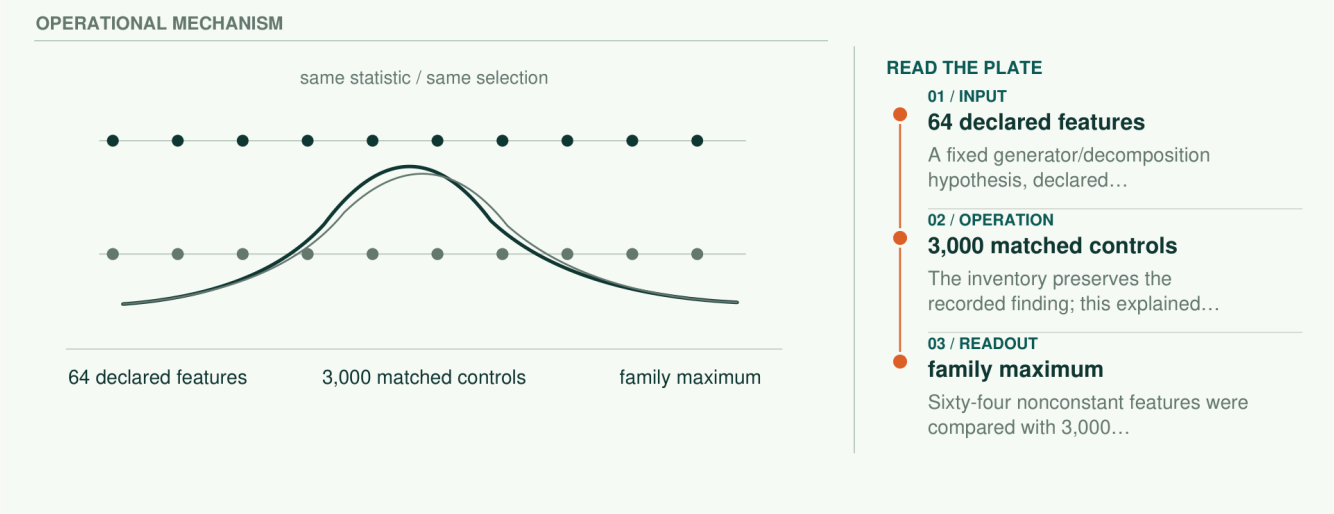
SECTION 08 / DATED GENERATOR AND DECOMPOSITION FOLLOW-UP AUDITS

Deeper remainder/permutation-tree null calibration

M118

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

The method is defined by the named evidence files and recorded scope.

WHAT WAS ACTUALLY DONE

The inventory preserves the recorded finding; this explained edition does not rerun or independently certify the source.

INPUTS NEEDED

A fixed generator/decomposition hypothesis, declared solved/unsolved corpus, target identity checks and range-matched controls where statistical claims are made.

WHAT THE RESULT MEANS

Sixty-four nonconstant features were compared with 3,000 range-matched control corpora; the observed family maximum was not unusual, so no beacon was established.

MATHEMATICAL FORMULATION

No explicit formula has been editorially assigned to this newer method entry.

EVIDENCE S211 / S180

SECTION 08 / DATED GENERATOR AND DECOMPOSITION FOLLOW-UP AUDITS

Low-bit affine and linear-state generator audit

M119

EVIDENCE STATUS AUDITED/CORRECTED

FIGURE / METHOD MECHANISM

HOW IT WORKS

The method is defined by the named evidence files and recorded scope.

WHAT WAS ACTUALLY DONE

The inventory preserves the recorded finding; this explained edition does not rerun or independently certify the source.

INPUTS NEEDED

A fixed generator/decomposition hypothesis, declared solved/unsolved corpus, target identity checks and range-matched controls where statistical claims are made.

WHAT THE RESULT MEANS

Direct affine recurrence and at-most-32-bit binary-linear-state families fail under the stated mask and indexing assumptions; this does not exclude wider, nonlinear, hashed or skipped-state generators.

MATHEMATICAL FORMULATION

No explicit formula has been editorially assigned to this newer method entry.

EVIDENCE S201 / S209 / S169

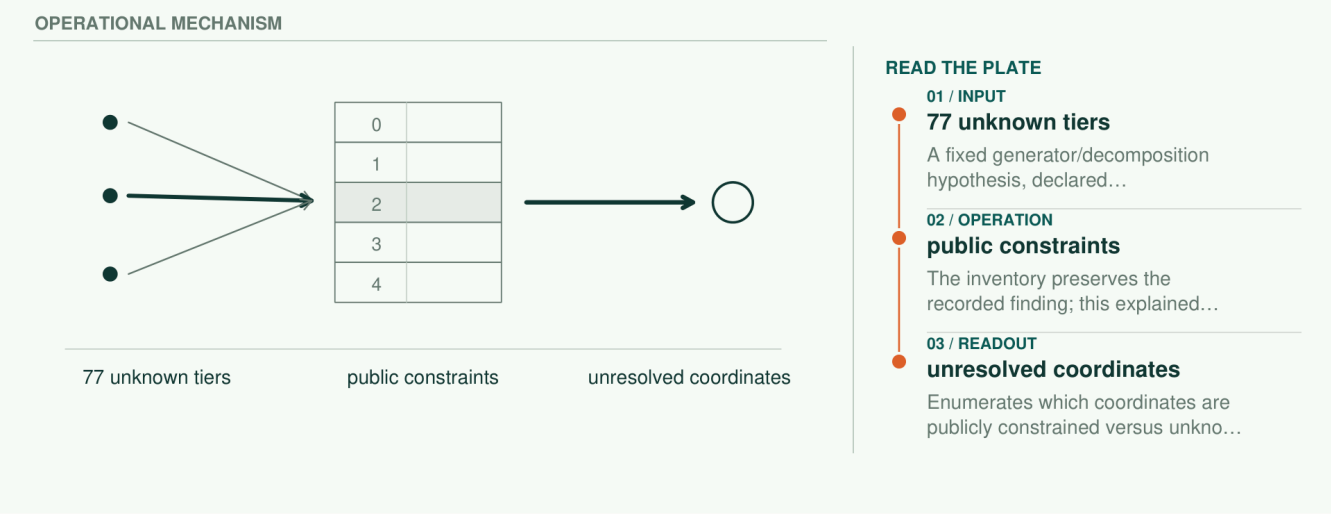
SECTION 08 / DATED GENERATOR AND DECOMPOSITION FOLLOW-UP AUDITS

Unknown-parameter inventory through Puzzle #160

M120

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

The method is defined by the named evidence files and recorded scope.

WHAT WAS ACTUALLY DONE

The inventory preserves the recorded finding; this explained edition does not rerun or independently certify the source.

INPUTS NEEDED

A fixed generator/decomposition hypothesis, declared solved/unsolved corpus, target identity checks and range-matched controls where statistical claims are made.

WHAT THE RESULT MEANS

Enumerates which coordinates are publicly constrained versus unknown across 77 unsolved tiers; fixed high decomposition coordinates are range consequences, not recovered private-key information.

MATHEMATICAL FORMULATION

No explicit formula has been editorially assigned to this newer method entry.

EVIDENCE S215 / S193

PAPERS PAPER-002 / PAPER-008

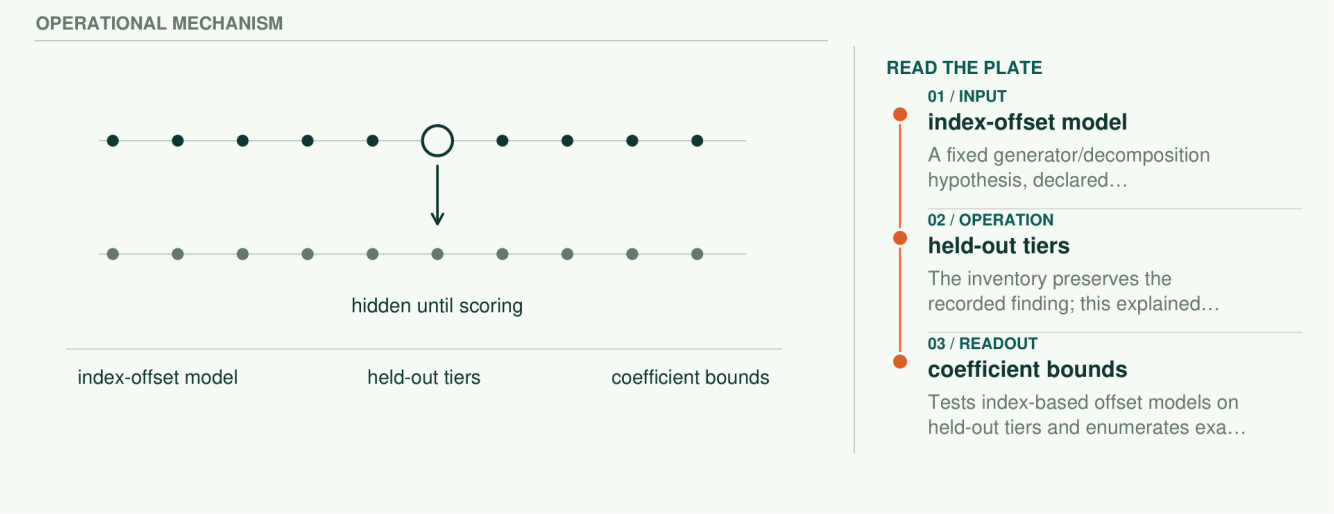
SECTION 08 / DATED GENERATOR AND DECOMPOSITION FOLLOW-UP AUDITS

Index-offset holdout and coefficient-interval audits

M121

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

The method is defined by the named evidence files and recorded scope.

WHAT WAS ACTUALLY DONE

The inventory preserves the recorded finding; this explained edition does not rerun or independently certify the source.

INPUTS NEEDED

A fixed generator/decomposition hypothesis, declared solved/unsolved corpus, target identity checks and range-matched controls where statistical claims are made.

WHAT THE RESULT MEANS

Tests index-based offset models on held-out tiers and enumerates exact decomposition coefficient possibilities over unsolved intervals; bounded intervals do not identify a scalar.

MATHEMATICAL FORMULATION

No explicit formula has been editorially assigned to this newer method entry.

EVIDENCE S208 / S166 / S145

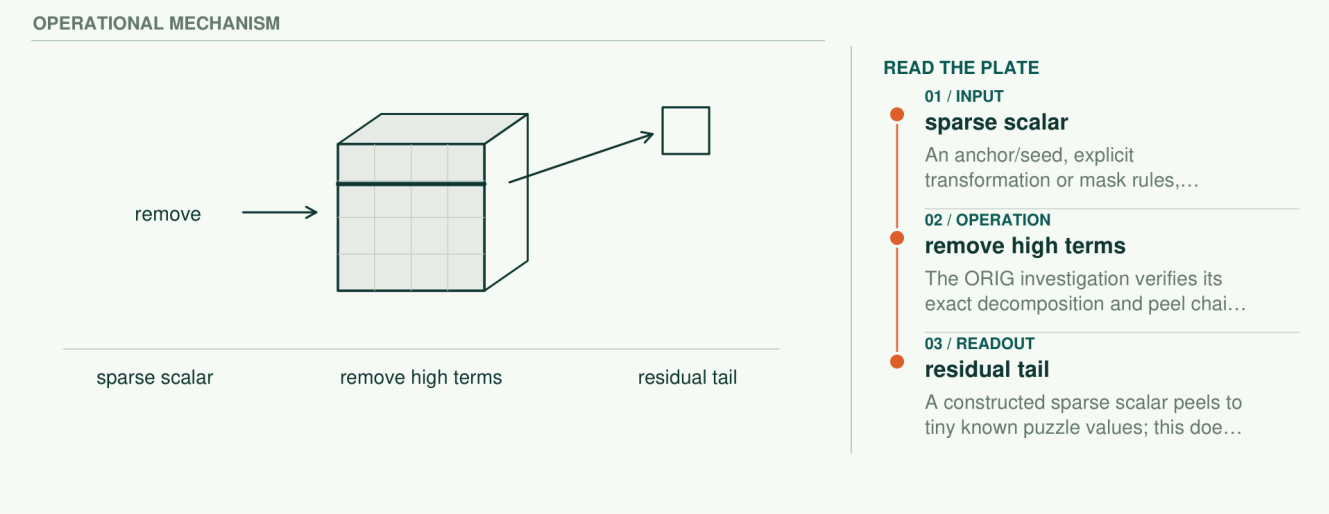
SECTION 09 / SPARSE KEYS, BIT WALKS AND SYNTHESIS ORBITS

Sparse ORIG decomposition and peel chain

M122

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

Describe a supplied sparse integer as a few shifted terms plus a tail, then reverse those terms to expose nested smaller values.

INPUTS NEEDED

An anchor/seed, explicit transformation or mask rules, bounded traversal settings and the target hash for a real candidate check.

WHAT WAS ACTUALLY DONE

The ORIG investigation verifies its exact decomposition and peel chain to tiny public keys. Constructing an integer around a known tail does not explain generation of later independent puzzles.

WHAT THE RESULT MEANS

A constructed sparse scalar peels to tiny known puzzle values; this does not generate the numbered puzzle sequence.

MATHEMATICAL FORMULATION

$$k = \sum_{j \in S} 2^j$$

A sparse construction specifies a small set S of occupied bit positions. Peeling a chosen highest bit or block recovers the construction's tail, not a general puzzle sequence.

EVIDENCE S187

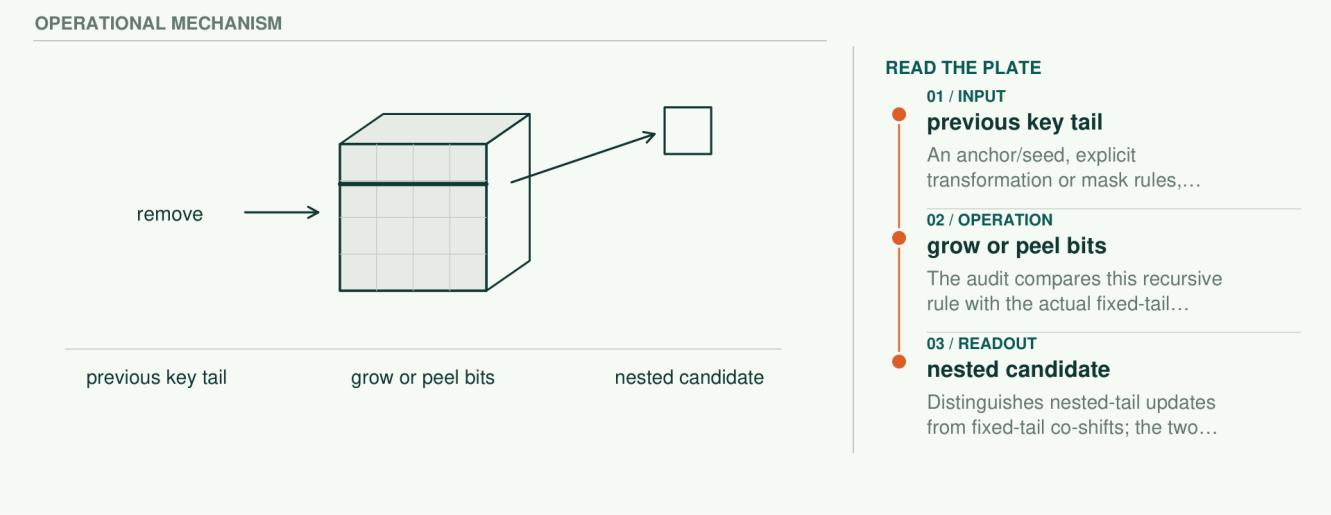
SECTION 09 / SPARSE KEYS, BIT WALKS AND SYNTHESIS ORBITS

Nested previous-key tail grow/peel

M123

EVIDENCE STATUS AUDITED/CORRECTED

FIGURE / METHOD MECHANISM



HOW IT WORKS

Build the next sparse value by embedding the previous entire value under a new frame, and reverse that operation by peeling the frame.

INPUTS NEEDED

An anchor/seed, explicit transformation or mask rules, bounded traversal settings and the target hash for a real candidate check.

WHAT WAS ACTUALLY DONE

The audit compares this recursive rule with the actual fixed-tail expression. They coincide at one chosen step but produce different subsequent integers, exposing a conflated model.

WHAT THE RESULT MEANS

Distinguishes nested-tail updates from fixed-tail co-shifts; the two agree at one construction step, not generally.

MATHEMATICAL FORMULATION

$$k_{t+1} = 2^{s_t} k_t + u_t$$

A nested grow model shifts the entire previous key and appends a tail u_t smaller than 2^s_t. Peeling reverses this specified construction.

EVIDENCE S186

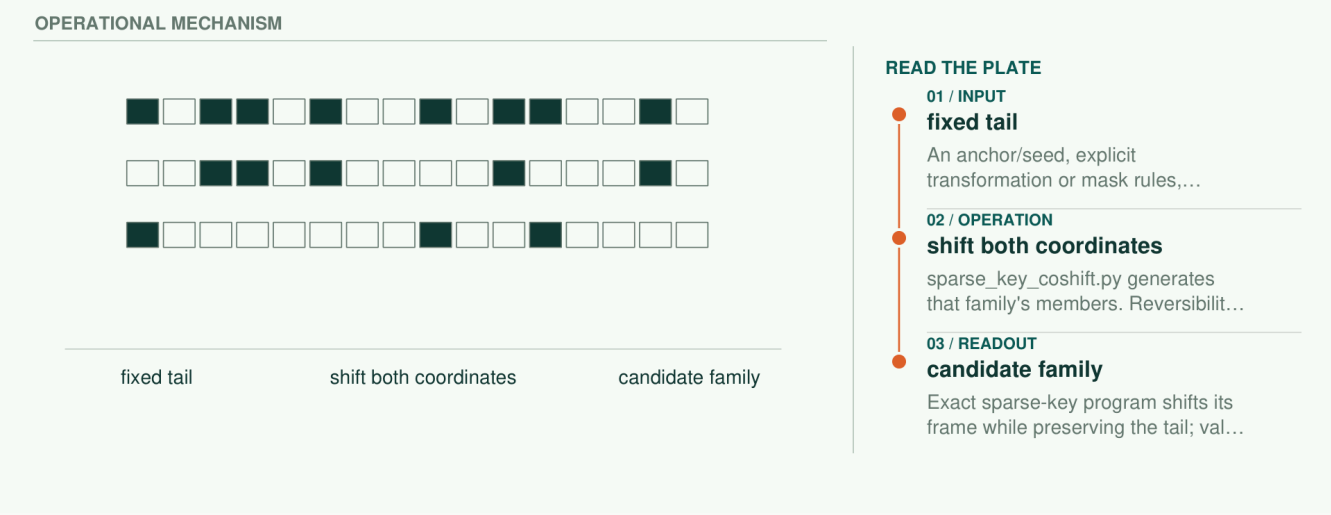
SECTION 09 / SPARSE KEYS, BIT WALKS AND SYNTHESIS ORBITS

Fixed-tail co-shift family

M124

EVIDENCE STATUS IMPLEMENTED

FIGURE / METHOD MECHANISM



HOW IT WORKS

Keep a small tail constant while shifting selected high and middle terms together. This defines a precise sparse family.

INPUTS NEEDED

An anchor/seed, explicit transformation or mask rules, bounded traversal settings and the target hash for a real candidate check.

WHAT WAS ACTUALLY DONE

sparse_key_coshift.py generates that family's members. Reversibility within the constructed family is a program property, not evidence that numbered Bitcoin puzzles lie on its orbit.

WHAT THE RESULT MEANS

Exact sparse-key program shifts its frame while preserving the tail; valid program structure is not a Bitcoin predictor.

MATHEMATICAL FORMULATION

$$k_t = 2^{s_t} F + u$$

A fixed frame F is shifted while tail u stays fixed. This differs from shifting the complete prior key at every step.

EVIDENCE S238 / S186

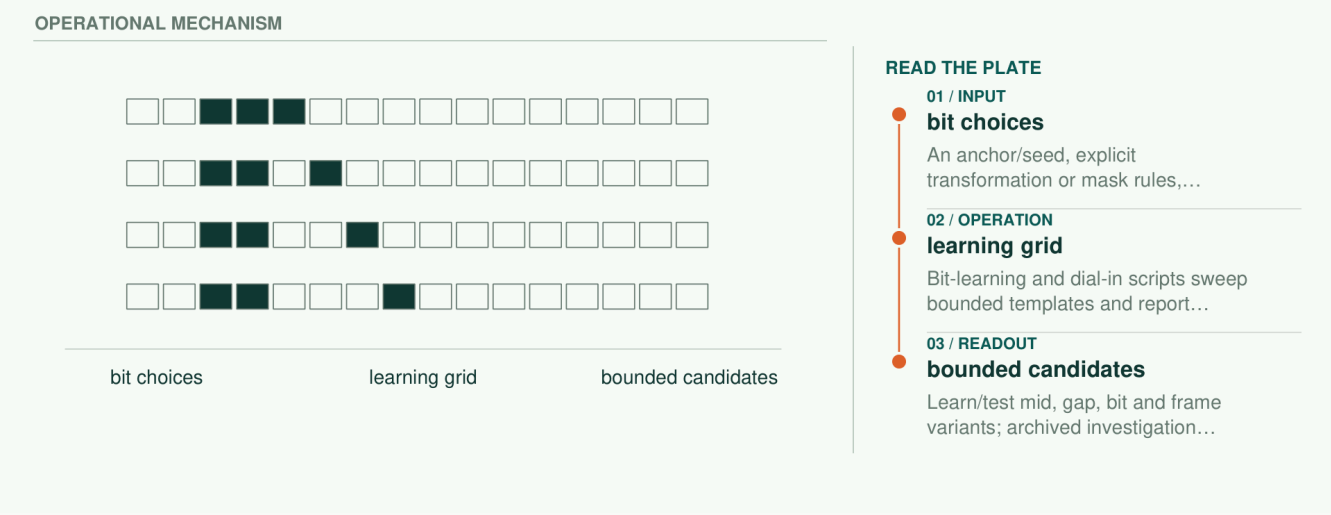
SECTION 09 / SPARSE KEYS, BIT WALKS AND SYNTHESIS ORBITS

Sparse bit-learning and dial-in grids

M125

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

Fit simple frame positions, bit rules or mid/gap parameters to known sparse values and test whether they extend across puzzle keys.

INPUTS NEEDED

An anchor/seed, explicit transformation or mask rules, bounded traversal settings and the target hash for a real candidate check.

WHAT WAS ACTUALLY DONE

Bit-learning and dial-in scripts sweep bounded templates and report coverage. The later investigation separates fitted tiny-key coincidences from actual unseen-address hits.

WHAT THE RESULT MEANS

Learn/test mid, gap, bit and frame variants; archived investigation reports poor known-key recall and no unknown address hits.

MATHEMATICAL FORMULATION

$$\mathcal{C} = \{F(\theta) : \theta \in \Theta_{\text{grid}}\}$$

theta selects frame, gap and bit parameters in a finite sparse family. Report exact recall and target checks across the declared grid.

EVIDENCE S236 / S240 / S187

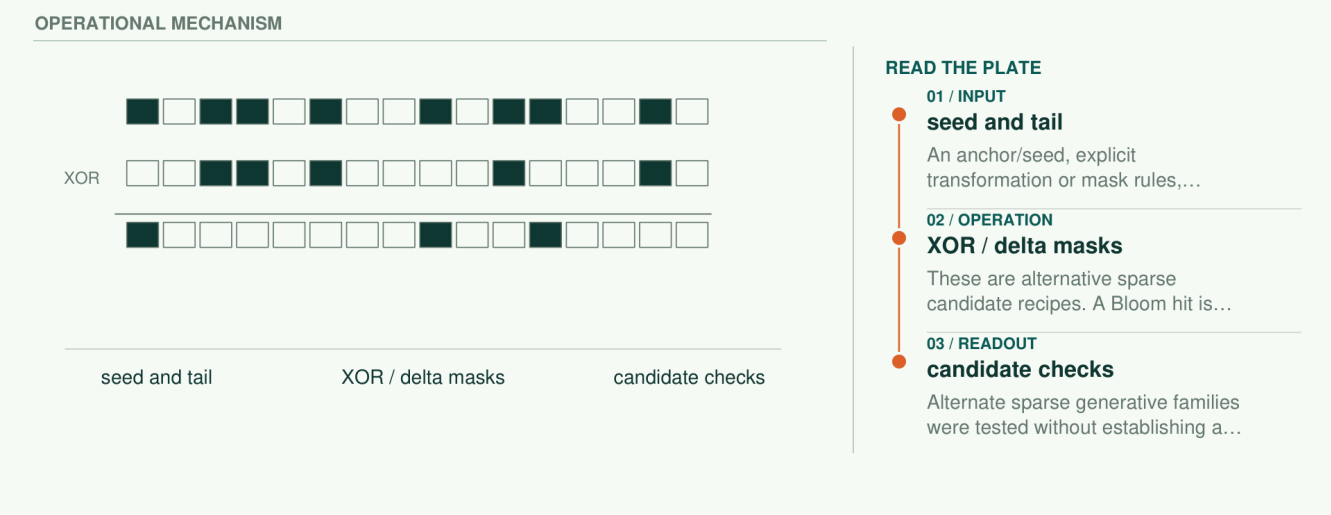
SECTION 09 / SPARSE KEYS, BIT WALKS AND SYNTHESIS ORBITS

Sparse XOR-tail / delta / Bloom variants

M126

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

Change the recurrence using XOR tails, deltas or Bloom-filter-assisted membership screening and compare its generated values with targets.

INPUTS NEEDED

An anchor/seed, explicit transformation or mask rules, bounded traversal settings and the target hash for a real candidate check.

WHAT WAS ACTUALLY DONE

These are alternative sparse candidate recipes. A Bloom hit is only a screening event; the scalar still needs exact address verification, and saved summaries do not establish a transferable generator.

WHAT THE RESULT MEANS

Alternate sparse generative families were tested without establishing a transferable key-generation rule.

MATHEMATICAL FORMULATION

$$k = F(\theta) \oplus \tau$$

XOR tails tau and difference variants create alternative sparse candidate families. A Bloom prefilter needs exact membership confirmation afterward.

EVIDENCE S241 / S239 / S237

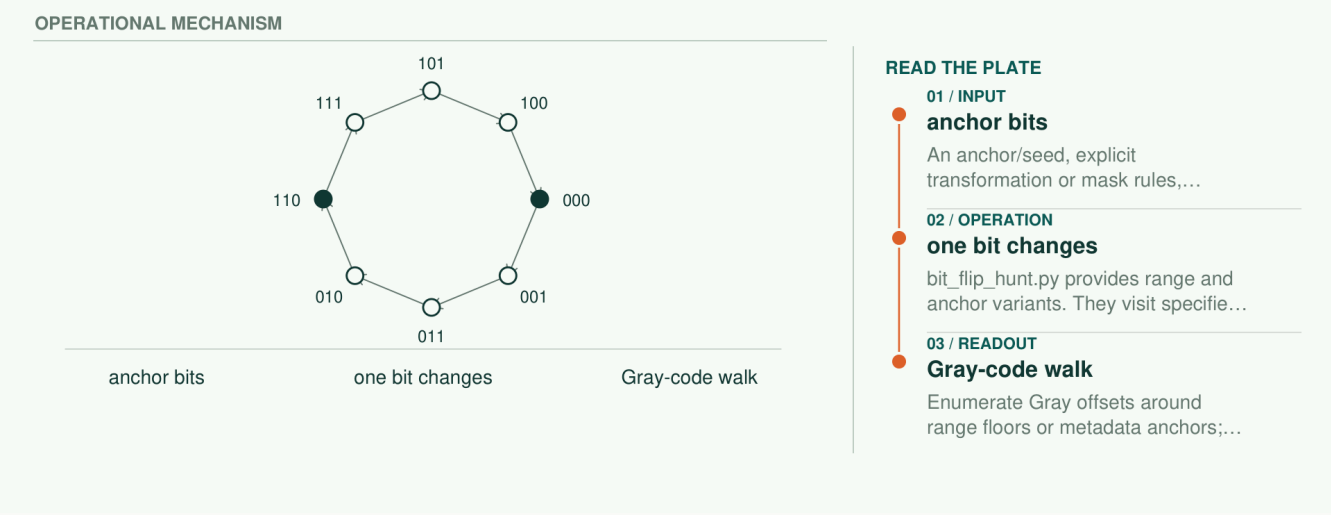
SECTION 09 / SPARSE KEYS, BIT WALKS AND SYNTHESIS ORBITS

Gray-code range and anchor walks

M127

EVIDENCE STATUS IMPLEMENTED

FIGURE / METHOD MECHANISM



HOW IT WORKS

Gray code orders bitstrings so adjacent offsets differ in one bit; adding or XORing offsets around an anchor defines a traversal.

INPUTS NEEDED

An anchor/seed, explicit transformation or mask rules, bounded traversal settings and the target hash for a real candidate check.

WHAT WAS ACTUALLY DONE

bit_flip_hunt.py provides range and anchor variants. They visit specified candidates in a different order and do not prove the key is near the selected anchor.

WHAT THE RESULT MEANS

Enumerate Gray offsets around range floors or metadata anchors; this changes order rather than entropy.

MATHEMATICAL FORMULATION

$$g(t) = t \oplus \lfloor t/2 \rfloor, \quad k_t = L + g(t)$$

Binary-reflected Gray code visits all offsets of a complete power-of-two block. Clip or explicitly map offsets for other interval sizes.

EVIDENCE S095

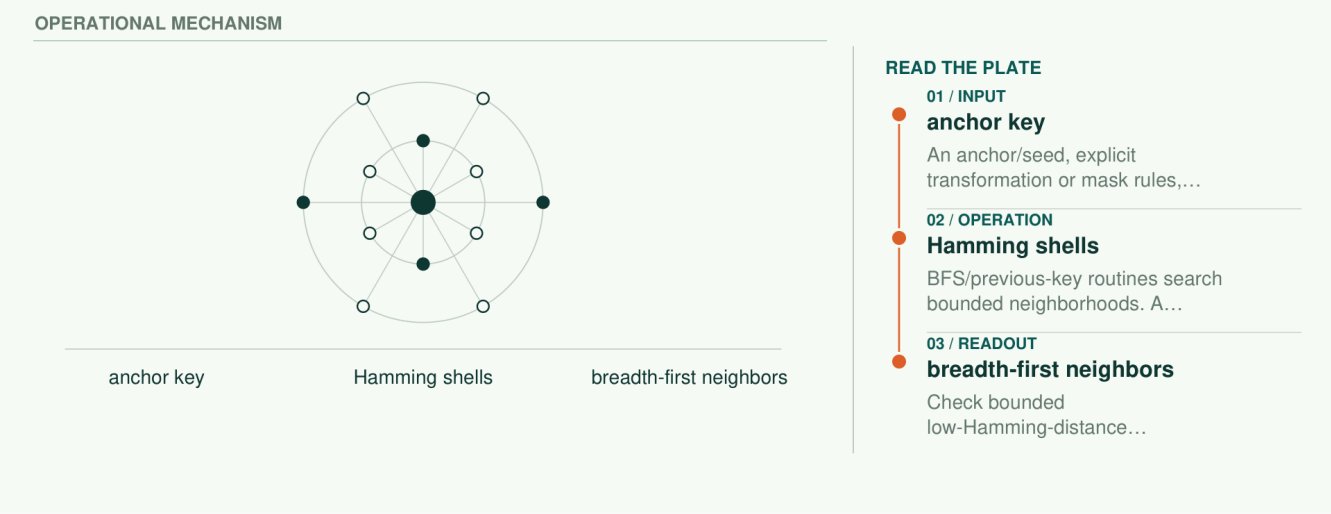
SECTION 09 / SPARSE KEYS, BIT WALKS AND SYNTHESIS ORBITS

BFS Hamming balls and previous-key XOR

M128

EVIDENCE STATUS IMPLEMENTED

FIGURE / METHOD MECHANISM



HOW IT WORKS

Enumerate all masks up to a chosen Hamming weight around a seed, or XOR selected bits into a previous known key.

INPUTS NEEDED

An anchor/seed, explicit transformation or mask rules, bounded traversal settings and the target hash for a real candidate check.

WHAT WAS ACTUALLY DONE

BFS/previous-key routines search bounded neighborhoods. A completed neighborhood search says nothing about integers outside that radius unless independent target-distance information exists.

WHAT THE RESULT MEANS

Check bounded low-Hamming-distance neighborhoods around seeds or prior keys; completeness is limited to that neighborhood.

MATHEMATICAL FORMULATION

$$\mathcal{B}_h(a) = \{a \oplus e : \mathrm{popcount}(e) \le h\}$$

This bounded Hamming neighborhood has sum_{j=0}^h binomial(b,j) offsets for b eligible bit positions, before range clipping.

EVIDENCE S095

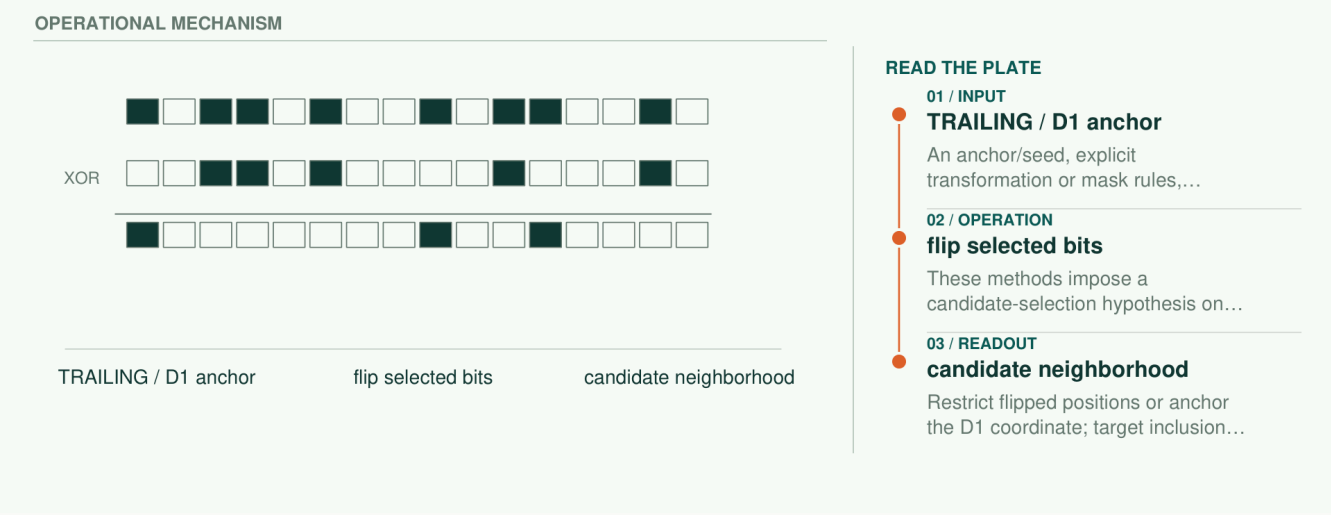
SECTION 09 / SPARSE KEYS, BIT WALKS AND SYNTHESIS ORBITS

TRAILING-mask and FUSED-D1 bit flips

M129

EVIDENCE STATUS IMPLEMENTED

FIGURE / METHOD MECHANISM



HOW IT WORKS

Choose flip positions from TRAILING bits or apply flips within a D1 coordinate derived from a FUSED anchor.

INPUTS NEEDED

An anchor/seed, explicit transformation or mask rules, bounded traversal settings and the target hash for a real candidate check.

WHAT WAS ACTUALLY DONE

These methods impose a candidate-selection hypothesis on where the unknown bits differ. Their bounds are computational choices, not leaked bit positions from the target.

WHAT THE RESULT MEANS

Restrict flipped positions or anchor the D1 coordinate; target inclusion is unproven.

MATHEMATICAL FORMULATION

$$k = a \oplus e, \quad e \,\mathrm{AND}\, \neg M = 0$$

The mask M declares which anchor bits may flip. A D1 version applies the flips to the residual coordinate before reconstructing the scalar.

EVIDENCE S095

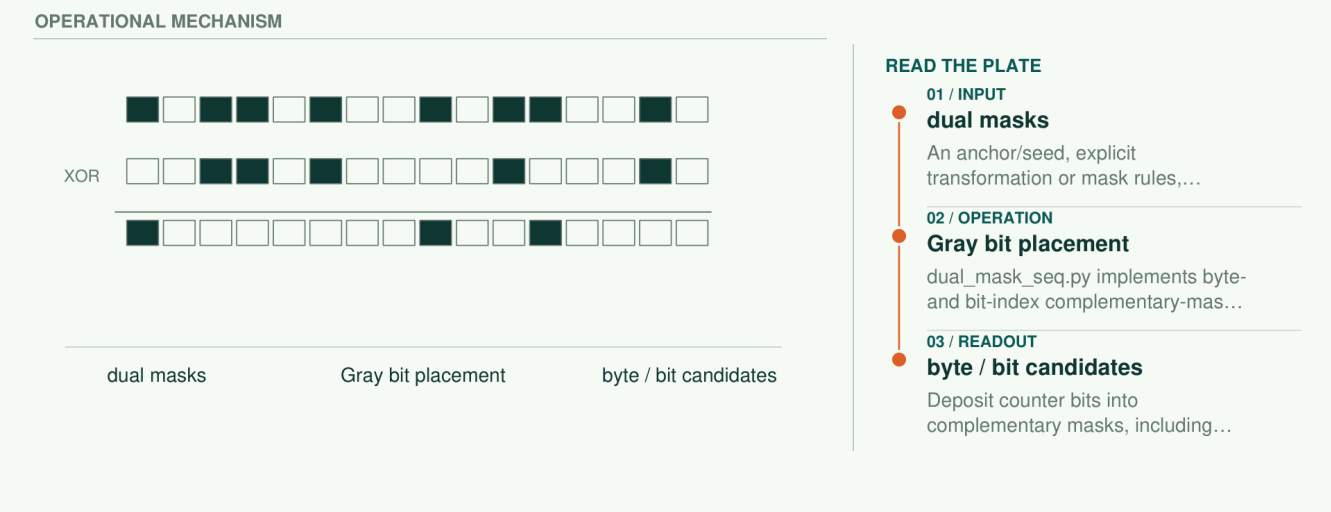
SECTION 09 / SPARSE KEYS, BIT WALKS AND SYNTHESIS ORBITS

Dual-mask byte/bit and Gray-PDEP walks

M130

EVIDENCE STATUS IMPLEMENTED

FIGURE / METHOD MECHANISM



HOW IT WORKS

Deposit counter bits into chosen mask positions; optionally Gray-code the counter so consecutive deposits change sparsely.

INPUTS NEEDED

An anchor/seed, explicit transformation or mask rules, bounded traversal settings and the target hash for a real candidate check.

WHAT WAS ACTUALLY DONE

dual_mask_seq.py implements byte- and bit-index complementary-mask schedules. The masks describe allowed moves; their symbolic organization does not create missing target information.

WHAT THE RESULT MEANS

Deposit counter bits into complementary masks, including Gray ordering; these are deterministic traversal families.

MATHEMATICAL FORMULATION

$$k = a \oplus \mathrm{PDEP}(g(t), M)$$

PDEP deposits successive counter bits into the set positions of M; Gray order g changes traversal. Complementary masks can define a product of such walks.

EVIDENCE S103

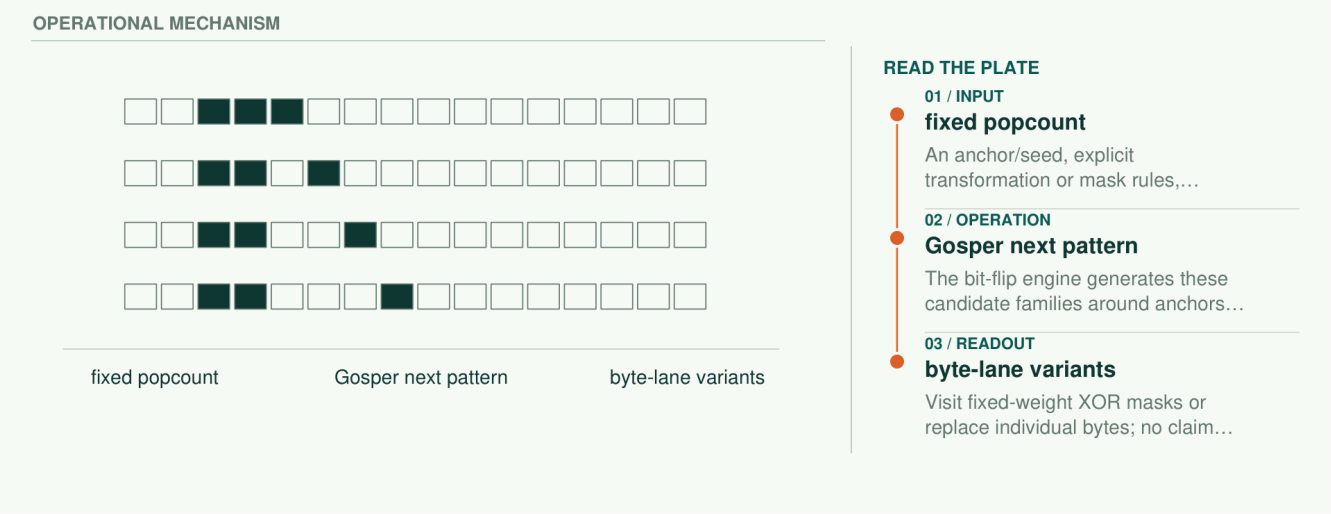
SECTION 09 / SPARSE KEYS, BIT WALKS AND SYNTHESIS ORBITS

Gosper constant-weight and byte-lane variants

M131

EVIDENCE STATUS IMPLEMENTED

FIGURE / METHOD MECHANISM



HOW IT WORKS

Gosper's rule enumerates masks of fixed Hamming weight. A byte-lane walk substitutes every byte value at one position.

INPUTS NEEDED

An anchor/seed, explicit transformation or mask rules, bounded traversal settings and the target hash for a real candidate check.

WHAT WAS ACTUALLY DONE

The bit-flip engine generates these candidate families around anchors and verifies addresses. Their coverage is explicitly restricted to the selected mask weights or byte substitutions.

WHAT THE RESULT MEANS

Visit fixed-weight XOR masks or replace individual bytes; no claim that these neighborhoods contain the target.

MATHEMATICAL FORMULATION

$$E_h = \{e : \mathrm{popcount}(e) = h\}, \quad |E_h| = \binom{b}{h}$$

Gosper-style traversal enumerates constant-weight b-bit masks. Byte-lane replacement is a separate bounded neighborhood.

EVIDENCE S095 / S103

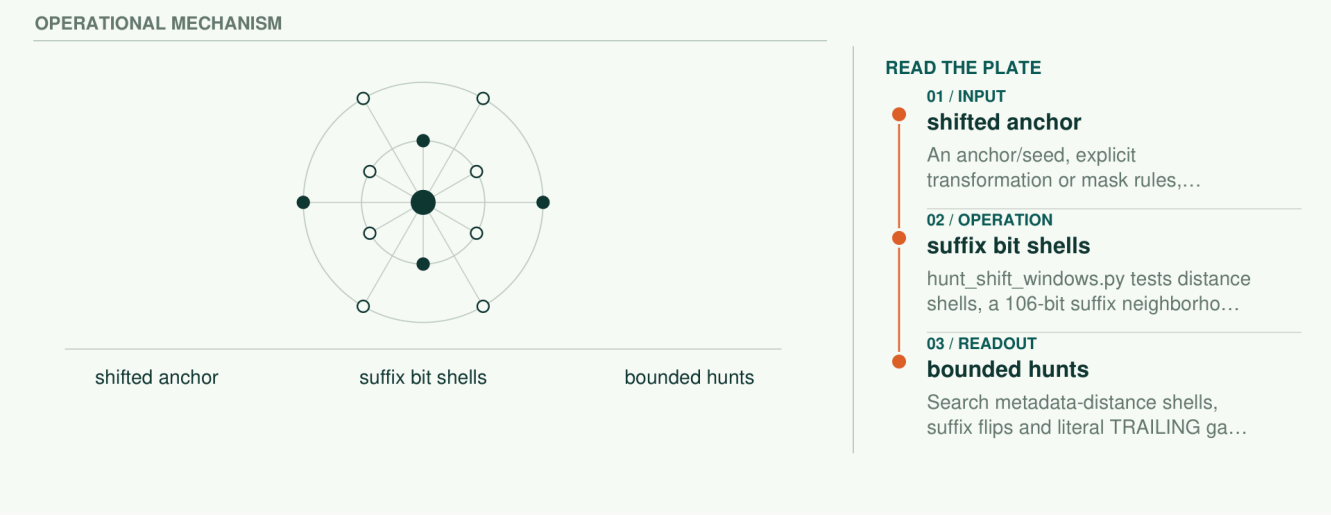
SECTION 09 / SPARSE KEYS, BIT WALKS AND SYNTHESIS ORBITS

Shift-shell and suffix Hamming hunts

M132

EVIDENCE STATUS IMPLEMENTED

FIGURE / METHOD MECHANISM



HOW IT WORKS

Search literal scalar shells around a metadata distance, or flip a limited number of bits in a chosen suffix.

INPUTS NEEDED

An anchor/seed, explicit transformation or mask rules, bounded traversal settings and the target hash for a real candidate check.

WHAT WAS ACTUALLY DONE

hunt_shift_windows.py tests distance shells, a 106-bit suffix neighborhood and small TRAILING gaps. Each is a bounded anchor hypothesis, not an exhaustive private-key interval search.

WHAT THE RESULT MEANS

Search metadata-distance shells, suffix flips and literal TRAILING gaps around selected anchors.

MATHEMATICAL FORMULATION

$$\mathcal{C} = \{(a+\delta)\oplus e : \delta \in D,\ e \in E\} \cap I$$

D gives declared shift shells and E gives eligible suffix flips. Completeness refers only to this finite neighborhood.

EVIDENCE S093

PAPERS PAPER-002 / PAPER-011

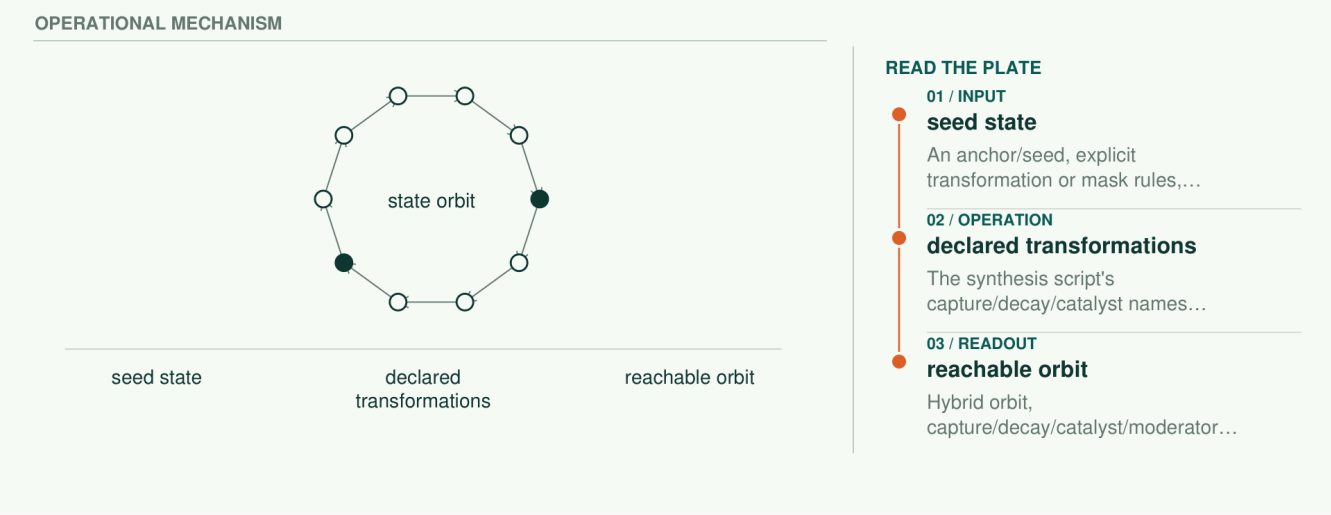
SECTION 09 / SPARSE KEYS, BIT WALKS AND SYNTHESIS ORBITS

Synthesis transformation orbit engine

M133

EVIDENCE STATUS IMPLEMENTED

FIGURE / METHOD MECHANISM



HOW IT WORKS

Apply a registry of deterministic mixing transformations repeatedly to produce candidate orbits, then rank the resulting integers.

INPUTS NEEDED

An anchor/seed, explicit transformation or mask rules, bounded traversal settings and the target hash for a real candidate check.

WHAT WAS ACTUALLY DONE

The synthesis script's capture/decay/catalyst names denote ordinary bit/arithmetic operations. It generates and scores candidate scalars; the names do not represent physical or quantum processes.

WHAT THE RESULT MEANS

Hybrid orbit, capture/decay/catalyst/moderator transformations rank unused-byte novelty; terminology does not imply quantum physics.

MATHEMATICAL FORMULATION

$$\mathcal{O}_T(a) = \{ f_t \circ \cdots \circ f_1(a) : t \le T,\ f_i \in \mathcal{F} \}$$

The transformation catalog F and depth limit T define an orbit search. Ranking states by novelty does not give a target-key likelihood without validation.

EVIDENCE S233

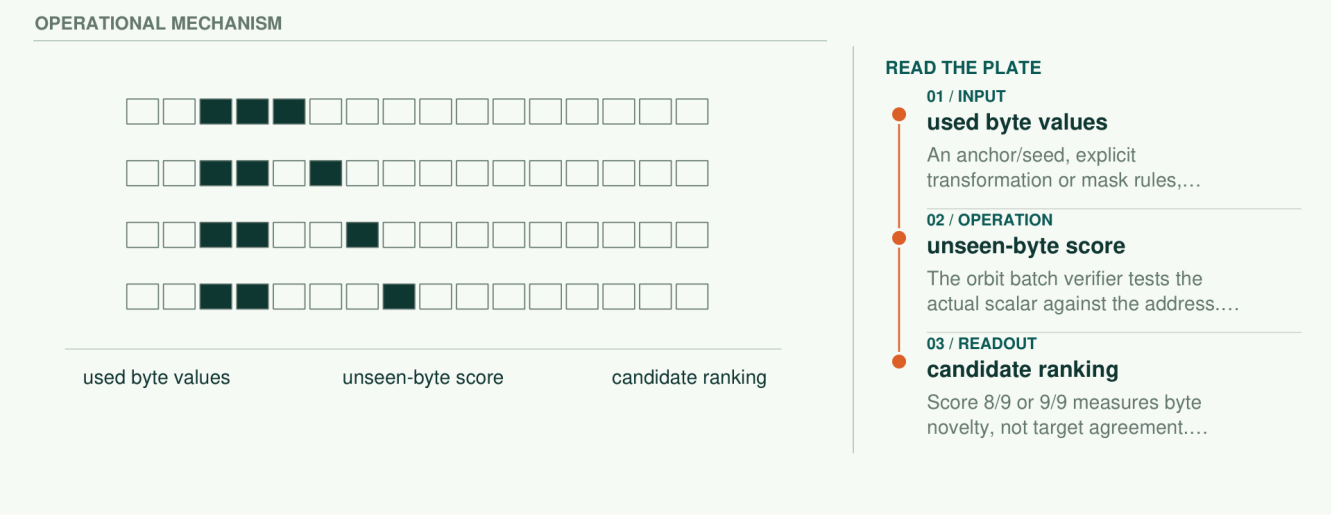
SECTION 09 / SPARSE KEYS, BIT WALKS AND SYNTHESIS ORBITS

Unused-byte novelty scoring

M134

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

Count byte positions whose values have not appeared in the solved-key corpus, favoring combinations labeled novel.

INPUTS NEEDED

An anchor/seed, explicit transformation or mask rules, bounded traversal settings and the target hash for a real candidate check.

WHAT WAS ACTUALLY DONE

The orbit batch verifier tests the actual scalar against the address. High novelty scores do not measure target agreement; some archived candidates were even below the valid puzzle interval.

WHAT THE RESULT MEANS

Score 8/9 or 9/9 measures byte novelty, not target agreement. Archived 1,800-candidate verification had zero hits.

MATHEMATICAL FORMULATION

$$\nu(k) = \sum_{j=1}^{m} 1\{b_j(k) \notin U\}$$

This representative novelty score counts bytes outside the declared used-byte set U. Its meaning is byte novelty, not agreement with a Bitcoin target.

EVIDENCE S173

SECTION 10 / HP-RTE, TOPOLOGY AND PATTERN-AVOIDANCE EXPERIMENTS

Original HP-RTE weighted shard sampling

M135

EVIDENCE STATUS AUDITED/CORRECTED

FIGURE / METHOD MECHANISM

HOW IT WORKS

The original recipe samples a key, derives a byte/nibble label and uses its assigned weight to move the output into a biased subrange.

INPUTS NEEDED

Candidate integers/bytes and explicitly defined numerical or pattern rules; mathematical diagnostics do not themselves supply target information.

WHAT WAS ACTUALLY DONE

Historical code uses floating-point offsets, and successive versions exclude different parts of the interval. The repaired generator removes that unsupported exclusion and uses exact integer sampling.

WHAT THE RESULT MEANS

Historical float-based biased generator excludes part of the advertised interval; corrected pipeline uses full-support integer partitions.

MATHEMATICAL FORMULATION

$$\Pr(k \in S^c) = 0 \quad \text{when} \quad \operatorname{support}(P) = S \subsetneq I$$

A generator whose support S is a proper subset of the advertised interval cannot reach excluded candidates. The old weighted/float construction required a support audit.

EVIDENCE S002 / S003

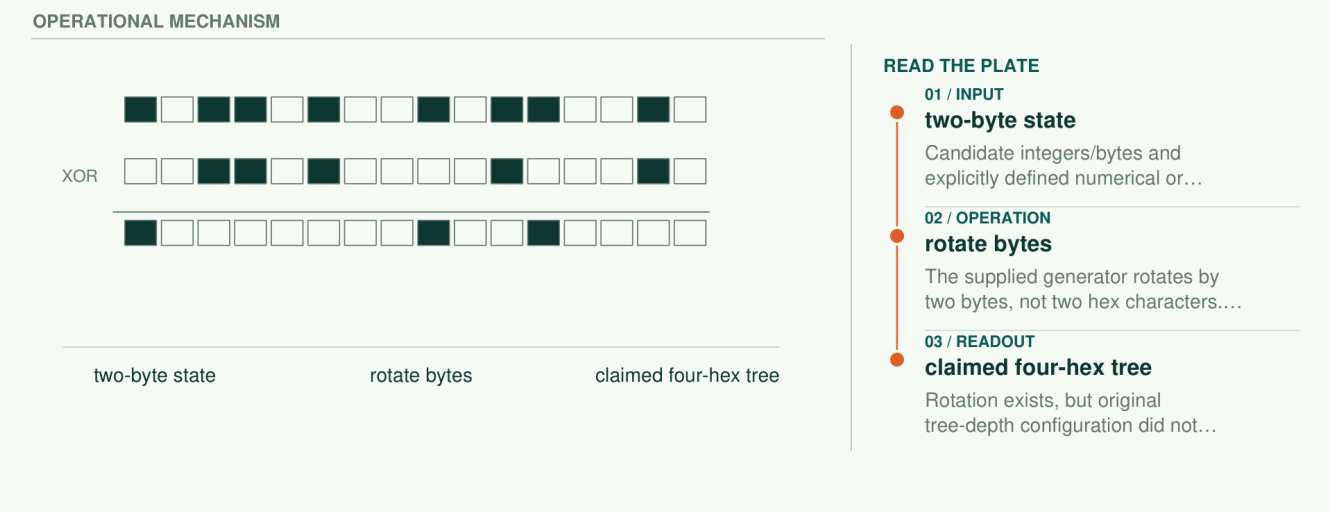
SECTION 10 / HP-RTE, TOPOLOGY AND PATTERN-AVOIDANCE EXPERIMENTS

Two-byte rotation and four-hex-tree claims

M136

EVIDENCE STATUS AUDITED/CORRECTED

FIGURE / METHOD MECHANISM



HOW IT WORKS

Rotate a byte string to change which byte is inspected. A genuine four-hex tree would also define branching and traversal over four hex digits.

INPUTS NEEDED

Candidate integers/bytes and explicitly defined numerical or pattern rules; mathematical diagnostics do not themselves supply target information.

WHAT WAS ACTUALLY DONE

The supplied generator rotates by two bytes, not two hex characters. Its tree-depth name is not used to implement the claimed tree partitioning.

WHAT THE RESULT MEANS

Rotation exists, but original tree-depth configuration did not implement the claimed tree search.

MATHEMATICAL FORMULATION

$$\mathrm{rot}_{16}(x, r) = ((x \ll r) | (x \gg (16 - r))) \,\mathrm{AND}\, (2^{16} - 1)$$

For a 16-bit word and 0<r<16, rotation permutes bits. A claimed hexadecimal tree additionally needs explicit depth and branching coverage.

EVIDENCE S002 / S003

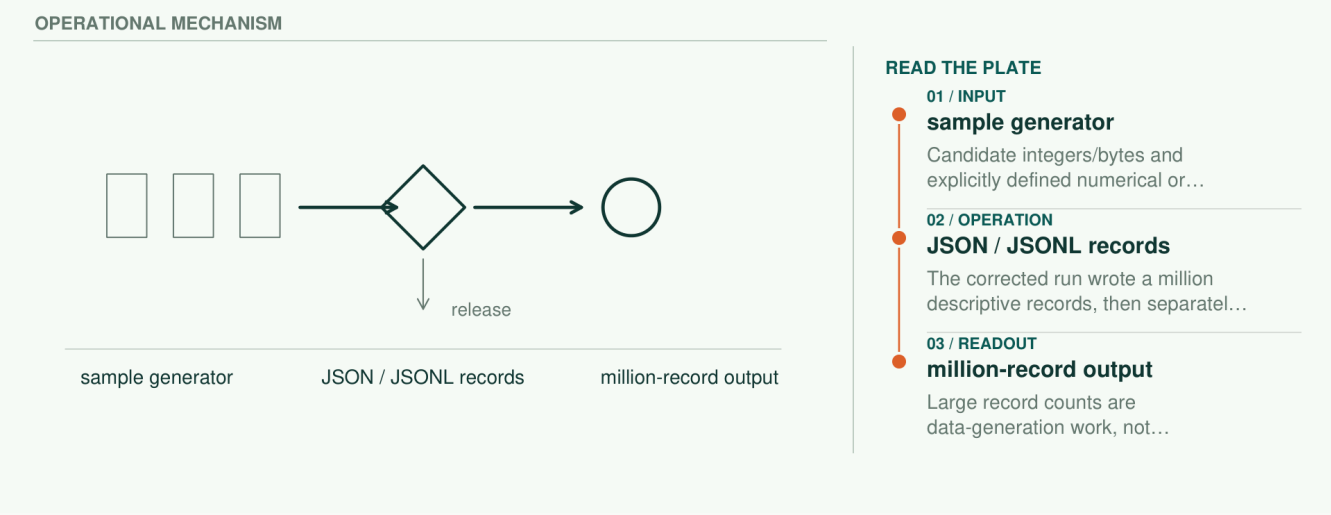
SECTION 10 / HP-RTE, TOPOLOGY AND PATTERN-AVOIDANCE EXPERIMENTS

Million-record JSON/JSONL generation

M137

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

Serialize one record per generated candidate or interval. JSONL allows records to be written one at a time.

INPUTS NEEDED

Candidate integers/bytes and explicitly defined numerical or pattern rules; mathematical diagnostics do not themselves supply target information.

WHAT WAS ACTUALLY DONE

The corrected run wrote a million descriptive records, then separately verified their sample keys and boundaries. File size and record count measure storage and generation, not exhaustive cryptographic coverage.

WHAT THE RESULT MEANS

Large record counts are data-generation work, not cryptographic search coverage; streaming JSONL avoids all-record memory use.

MATHEMATICAL FORMULATION

$$N_{\text{records}} \neq N_{\text{verified scalars}} \quad \text{in general}$$

A record can contain boundaries and samples without checking them. Verification counts require actual scalar-to-point/hash comparisons.

EVIDENCE S014 / S003

SECTION 10 / HP-RTE, TOPOLOGY AND PATTERN-AVOIDANCE EXPERIMENTS

Weighted modular-diameter diversity

M138

EVIDENCE STATUS AUDITED/CORRECTED

FIGURE / METHOD MECHANISM

HOW IT WORKS

The old statistic computes pairwise numeric spread and multiplies it by a subset-dependent average bias weight.

INPUTS NEEDED

Candidate integers/bytes and explicitly defined numerical or pattern rules; mathematical diagnostics do not themselves supply target information.

WHAT WAS ACTUALLY DONE

The original uint64 conversion cannot hold 71-bit scalars; unsigned subtraction also wraps. Fixing integer overflow alone does not prove that the weighted expression satisfies diversity axioms.

WHAT THE RESULT MEANS

uint64 overflow, unsigned differences and unproved subset weights invalidate original diversity claims.

MATHEMATICAL FORMULATION

$$d_m(x,y) = \min(r, m-r), \quad r = (x-y) \bmod m$$

This defines circular modular distance using exact integer subtraction. Weighted subset-diameter claims require separate definitions and cannot rely on overflowing unsigned arithmetic.

EVIDENCE S002 / S022

SECTION 10 / HP-RTE, TOPOLOGY AND PATTERN-AVOIDANCE EXPERIMENTS

Finite-sample Cauchy/convergence claims

M139

EVIDENCE STATUS AUDITED/CORRECTED

FIGURE / METHOD MECHANISM

HOW IT WORKS

True Cauchyness and convergence require bounds over arbitrary sufficiently late terms of an infinite sequence, with the proposed limit included for convergence.

WHAT WAS ACTUALLY DONE

The old functions sample subsets of a finite whole history and compare an arbitrary threshold. They are finite clustering tests mislabeled as asymptotic guarantees and cannot locate an unknown private key.

INPUTS NEEDED

Candidate integers/bytes and explicitly defined numerical or pattern rules; mathematical diagnostics do not themselves supply target information.

WHAT THE RESULT MEANS

Sampling a finite history is not an asymptotic tail criterion; clustering cannot identify the unknown scalar.

MATHEMATICAL FORMULATION

$$\forall \epsilon > 0 \; \exists N \; \forall p, q \geq N : |x_p - x_q| < \epsilon$$

This is the Cauchy tail criterion. A finite history cannot certify this universal statement without an argument controlling every future term.

EVIDENCE S022 / S003

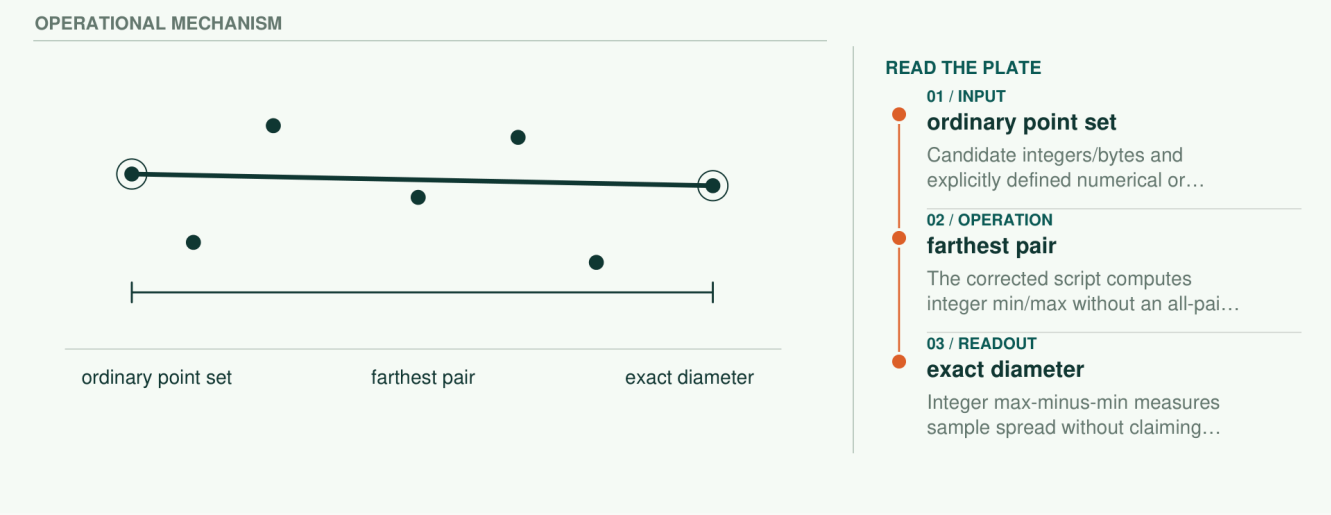
SECTION 10 / HP-RTE, TOPOLOGY AND PATTERN-AVOIDANCE EXPERIMENTS

Exact ordinary diameter diagnostic

M140

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

Diameter is the largest value minus the smallest. With fixed normalization it describes a finite sample's coordinate spread exactly.

INPUTS NEEDED

Candidate integers/bytes and explicitly defined numerical or pattern rules; mathematical diagnostics do not themselves supply target information.

WHAT WAS ACTUALLY DONE

The corrected script computes integer min/max without an all-pairs matrix. It labels the result descriptively and makes no claim of convergence toward the cryptographic target.

WHAT THE RESULT MEANS

Integer max-minus-min measures sample spread without claiming convergence or cryptographic proximity.

MATHEMATICAL FORMULATION

$$\operatorname{diam}(S) = \max S - \min S$$

For a nonempty finite set of ordinary integers, exact max-minus-min measures its spread. It says nothing about distance to an unknown scalar.

EVIDENCE S011 / S017

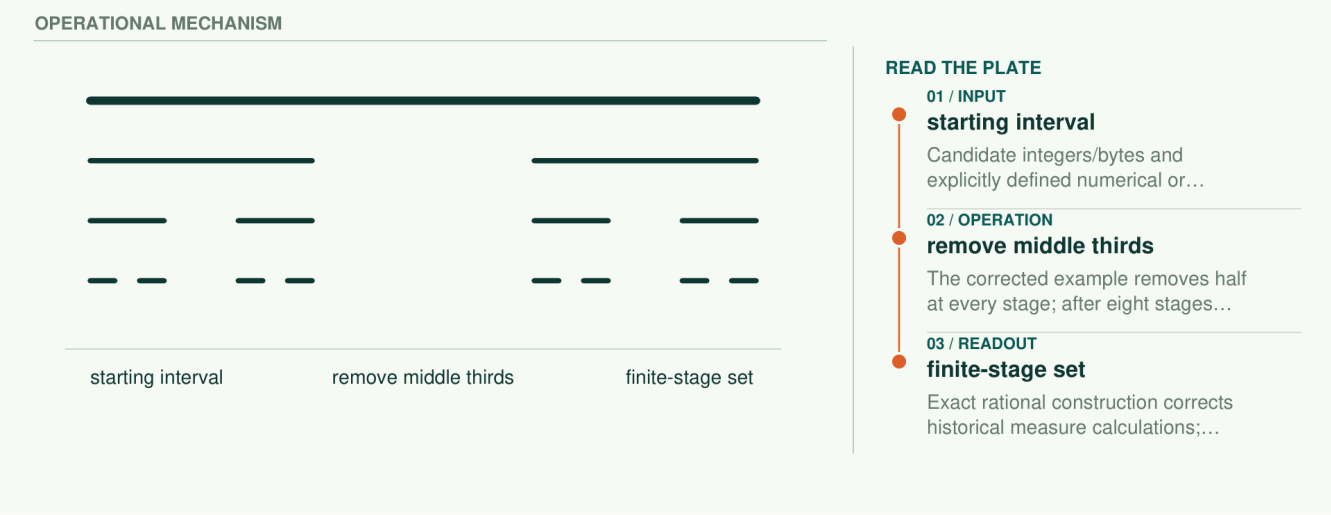
SECTION 10 / HP-RTE, TOPOLOGY AND PATTERN-AVOIDANCE EXPERIMENTS

Cantor-like finite-stage constructions

M141

EVIDENCE STATUS AUDITED/CORRECTED

FIGURE / METHOD MECHANISM



HOW IT WORKS

Repeatedly remove a middle interval from each remaining closed interval, tracking exact rational endpoints and total surviving length.

INPUTS NEEDED

Candidate integers/bytes and explicitly defined numerical or pattern rules; mathematical diagnostics do not themselves supply target information.

WHAT WAS ACTUALLY DONE

The corrected example removes half at every stage; after eight stages length is 1/256. The infinite limit has zero measure, unlike the earlier claimed fat-Cantor interpretation.

WHAT THE RESULT MEANS

Exact rational construction corrects historical measure calculations; constant half-removal tends to zero measure.

MATHEMATICAL FORMULATION

$$\mu(C_n) = \prod_{j=1}^{n} (1 - \alpha_j)$$

In a construction removing fraction alpha_j of every surviving interval at stage j, the remaining total length is this product. Constant half-removal gives 2^-n.

EVIDENCE S011 / S003

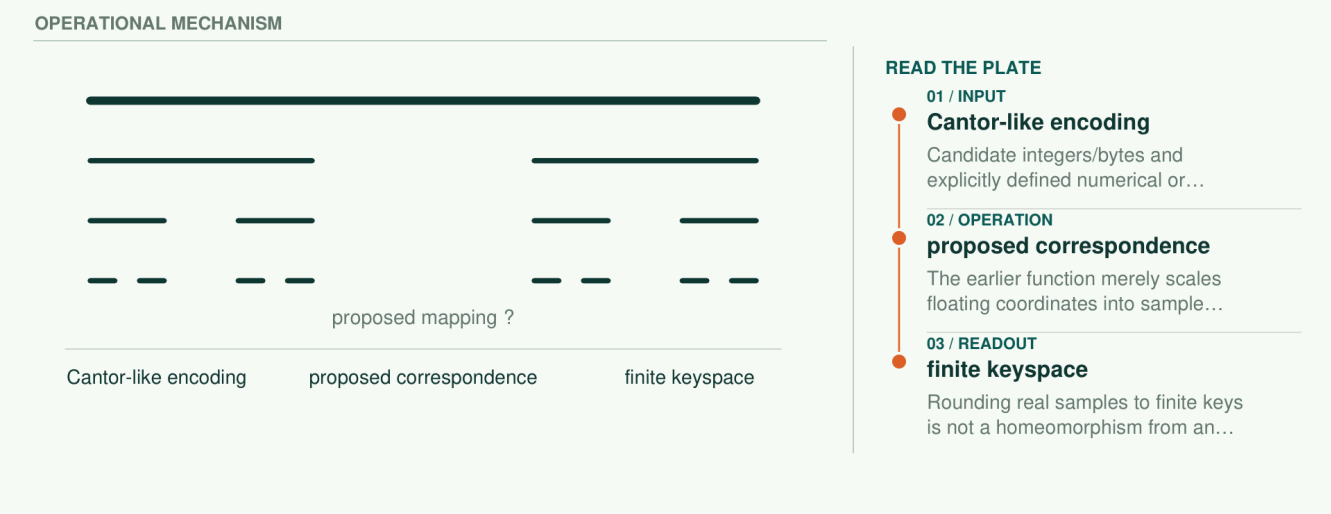
SECTION 10 / HP-RTE, TOPOLOGY AND PATTERN-AVOIDANCE EXPERIMENTS

Cantor-to-keyspace homeomorphism claim

M142

EVIDENCE STATUS AUDITED/CORRECTED

FIGURE / METHOD MECHANISM



HOW IT WORKS

A homeomorphism must be a continuous bijection with a continuous inverse. An infinite Cantor set cannot be bijective with a finite integer keyspace.

INPUTS NEEDED

Candidate integers/bytes and explicitly defined numerical or pattern rules; mathematical diagnostics do not themselves supply target information.

WHAT WAS ACTUALLY DONE

The earlier function merely scales floating coordinates into sample min/max bounds and rounds to integers. It is a lossy candidate map, not the source paper's homeomorphism construction.

WHAT THE RESULT MEANS

Rounding real samples to finite keys is not a homeomorphism from an infinite Cantor set.

MATHEMATICAL FORMULATION

$$q(x) = \mathrm{round}(Ax + B) \in \mathcal{K}_{\mathrm{finite}}$$

Quantization from an infinite real set to a finite key set cannot be injective and hence cannot be a homeomorphism.

EVIDENCE S022 / S003

SECTION 10 / HP-RTE, TOPOLOGY AND PATTERN-AVOIDANCE EXPERIMENTS

Byte-value arithmetic-progression avoidance

M143

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM

HOW IT WORKS

Search the set of distinct byte values for a nonconstant arithmetic progression such as 10,20,30. Every requested term must be present.

INPUTS NEEDED

Candidate integers/bytes and explicitly defined numerical or pattern rules; mathematical diagnostics do not themselves supply target information.

WHAT WAS ACTUALLY DONE

Corrected code extracts nine actual bytes, ignores order/multiplicity for this test, and records a Boolean label. It does not treat pattern avoidance as evidence that the key is more likely.

WHAT THE RESULT MEANS

Corrected diagnostic detects nonconstant APs among byte values; it is not used to discard search candidates.

MATHEMATICAL FORMULATION

$$a + c = 2b, \quad a, b, c \text{ distinct}$$

This detects a nonconstant three-term arithmetic progression among byte values. Reporting its presence does not justify rejecting the key.

EVIDENCE S011 / S017

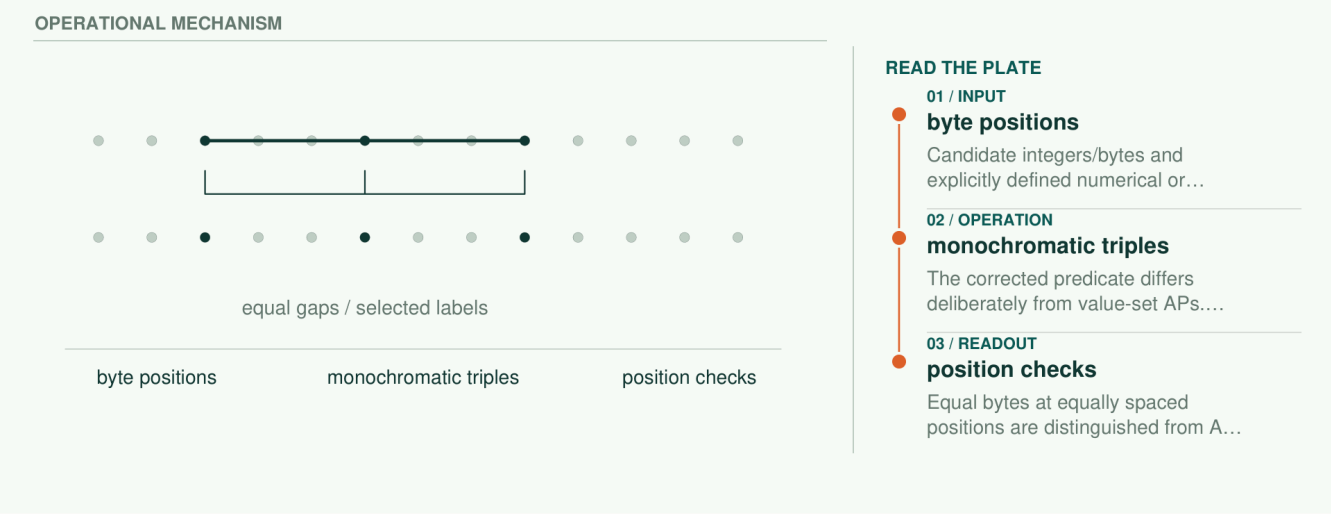
SECTION 10 / HP-RTE, TOPOLOGY AND PATTERN-AVOIDANCE EXPERIMENTS

Monochromatic byte-position progression checks

M144

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

Treat byte positions as the arithmetic progression and ask whether equally spaced positions carry the same byte value.

INPUTS NEEDED

Candidate integers/bytes and explicitly defined numerical or pattern rules; mathematical diagnostics do not themselves supply target information.

WHAT WAS ACTUALLY DONE

The corrected predicate differs deliberately from value-set APs. Repeated byte values can trigger a positional pattern even when the value set has no nonconstant arithmetic progression.

WHAT THE RESULT MEANS

Equal bytes at equally spaced positions are distinguished from APs in the values themselves.

MATHEMATICAL FORMULATION

$$b_i = b_{i+d} = b_{i+2d}, \quad d > 0$$

Equal bytes at equally spaced positions form a monochromatic position progression. This is different from arithmetic progression in the byte values.

EVIDENCE S011

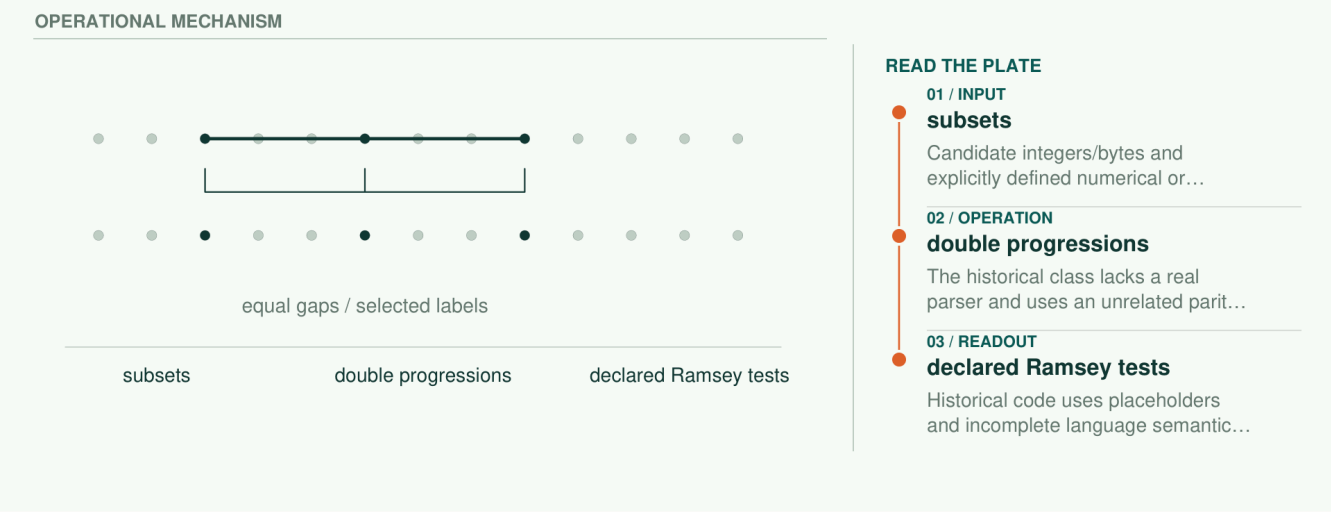
SECTION 10 / HP-RTE, TOPOLOGY AND PATTERN-AVOIDANCE EXPERIMENTS

Double-AP and RamseyScript subset claims

M145

EVIDENCE STATUS AUDITED/CORRECTED

FIGURE / METHOD MECHANISM



HOW IT WORKS

Ramsey search recursively extends combinatorial objects while rejecting forbidden structures; a faithful interpreter must implement the specified objects and filter semantics.

WHAT WAS ACTUALLY DONE

The historical class lacks a real parser and uses an unrelated parity placeholder for double APs. Its general k-AP detector can accept an incomplete progression; later versions repair only some defects.

INPUTS NEEDED

Candidate integers/bytes and explicitly defined numerical or pattern rules; mathematical diagnostics do not themselves supply target information.

WHAT THE RESULT MEANS

Historical code uses placeholders and incomplete language semantics; original filter claims do not establish a cryptographic advantage.

MATHEMATICAL FORMULATION

$$i+k=2j, \quad x_i+x_k=2x_j, \quad i<j<k$$

A double progression imposes both position and value relations. This states the mathematical pattern; placeholder code does not establish a complete implementation or useful filter.

EVIDENCE S002 / S022

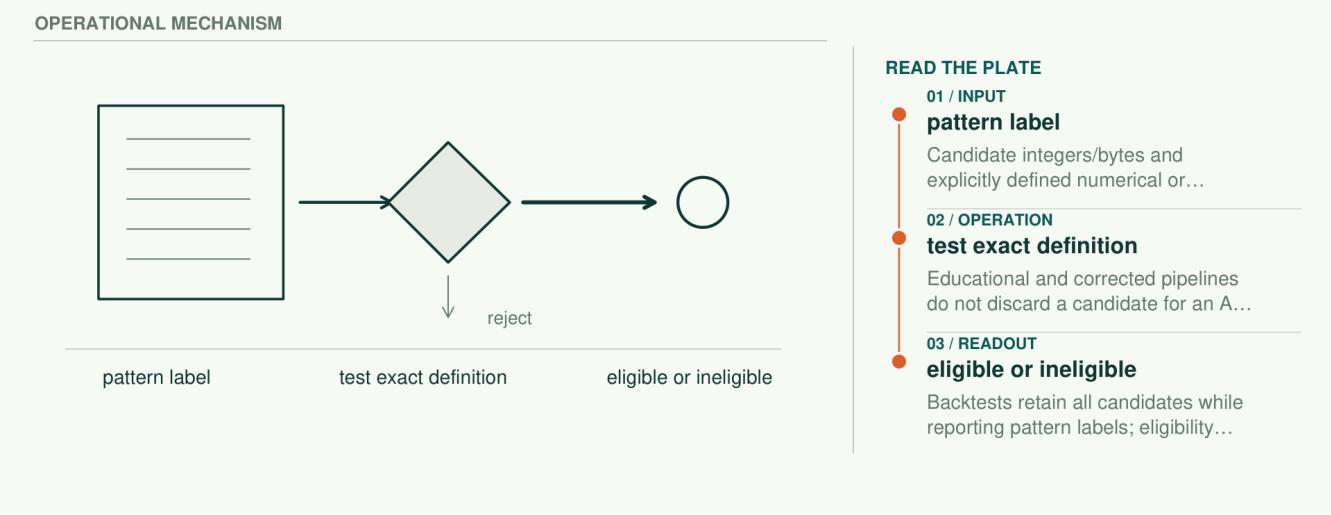
SECTION 10 / HP-RTE, TOPOLOGY AND PATTERN-AVOIDANCE EXPERIMENTS

Pattern-label eligibility correction

M146

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

Keep pattern classification as metadata while preserving eligibility of every integer in the advertised interval.

INPUTS NEEDED

Candidate integers/bytes and explicitly defined numerical or pattern rules; mathematical diagnostics do not themselves supply target information.

WHAT WAS ACTUALLY DONE

Educational and corrected pipelines do not discard a candidate for an AP label. Known-key tests separately measure label preferences and full-range eligibility, which must not be conflated.

WHAT THE RESULT MEANS

Backtests retain all candidates while reporting pattern labels; eligibility is not evidence that a pattern predicts keys.

MATHEMATICAL FORMULATION

$$\mathrm{eligible}(k) = 1, \quad \ell(k) = \mathrm{patternLabel}(k)$$

Keep labels as diagnostics while retaining every candidate. Eligibility and pattern observations are distinct from a learned target predictor.

EVIDENCE S024 / S023

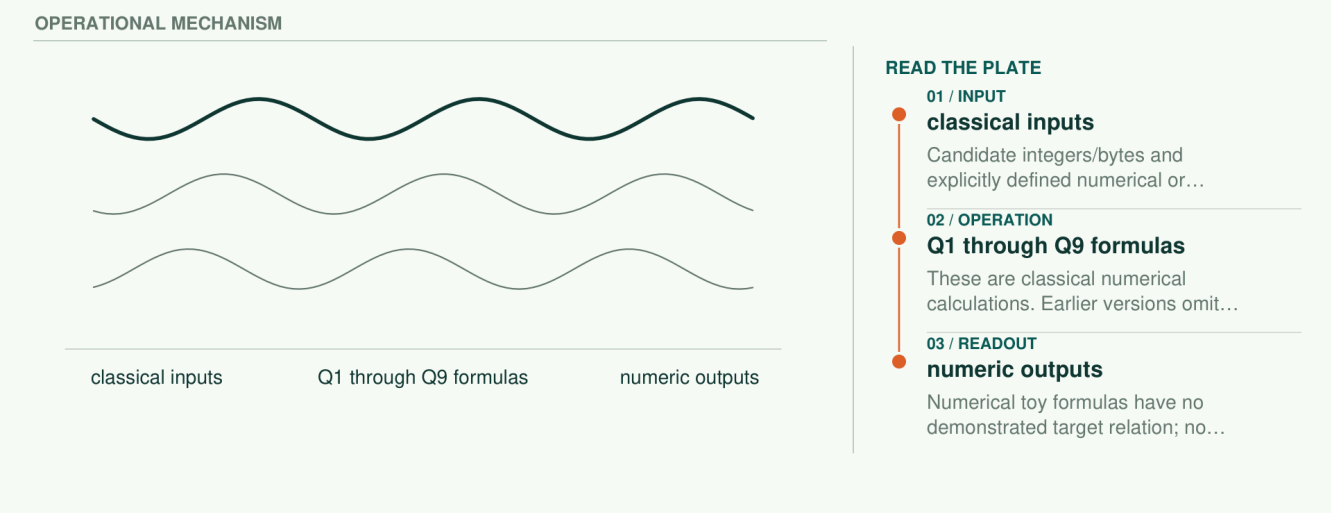
SECTION 10 / HP-RTE, TOPOLOGY AND PATTERN-AVOIDANCE EXPERIMENTS

Classical hyper-phase Q1-Q9 formulas

M147

EVIDENCE STATUS AUDITED/CORRECTED

FIGURE / METHOD MECHANISM



HOW IT WORKS

Evaluate trigonometric/exponential recurrence formulas indexed by state and bit position, then compare a magnitude with a chosen threshold.

INPUTS NEEDED

Candidate integers/bytes and explicitly defined numerical or pattern rules; mathematical diagnostics do not themselves supply target information.

WHAT WAS ACTUALLY DONE

These are classical numerical calculations. Earlier versions omit states; later formulas fill them but still neither ingest the target hash nor construct a quantum circuit or valid key-extraction equation.

WHAT THE RESULT MEANS

Numerical toy formulas have no demonstrated target relation; no quantum circuit or recovery is implemented.

MATHEMATICAL FORMULATION

$$z_j = F_j(x;\theta), \quad j = 1, \ldots, 9$$

F_j denotes each archived classical Q1-Q9 numerical formula. These toy transforms have no verified scalar-extraction equation or executed quantum circuit in the inventory.

EVIDENCE S002 / S022

PAPERS PAPER-022

SECTION 10 / HP-RTE, TOPOLOGY AND PATTERN-AVOIDANCE EXPERIMENTS

Bit-position periodic table / CFSG labels

M148

EVIDENCE STATUS CONVERSATION PROPOSAL

FIGURE / METHOD MECHANISM

HOW IT WORKS

Organize bit indices into a table with powers of two, symbolic families and heuristic scores. Such a table can be an interface for diagnostics.

INPUTS NEEDED

Candidate integers/bytes and explicitly defined numerical or pattern rules; mathematical diagnostics do not themselves supply target information.

WHAT WAS ACTUALLY DONE

The supplied transcript assigns random simulated biases and CFSG-inspired labels. For bits below 71, reducing 2^b modulo the large group order changes nothing; no hidden algebraic advantage follows.

WHAT THE RESULT MEANS

Supplied transcript uses simulated random biases and organizational labels; no local validated cryptanalytic result was established.

MATHEMATICAL FORMULATION

$$\widehat{p}_j = \frac{1}{m}\sum_{i=1}^{m} b_{ij}$$

Per-position empirical biases can organize a table of bit labels. Simulated biases and group-theory labels do not establish a target-specific probability model.

EVIDENCE S003

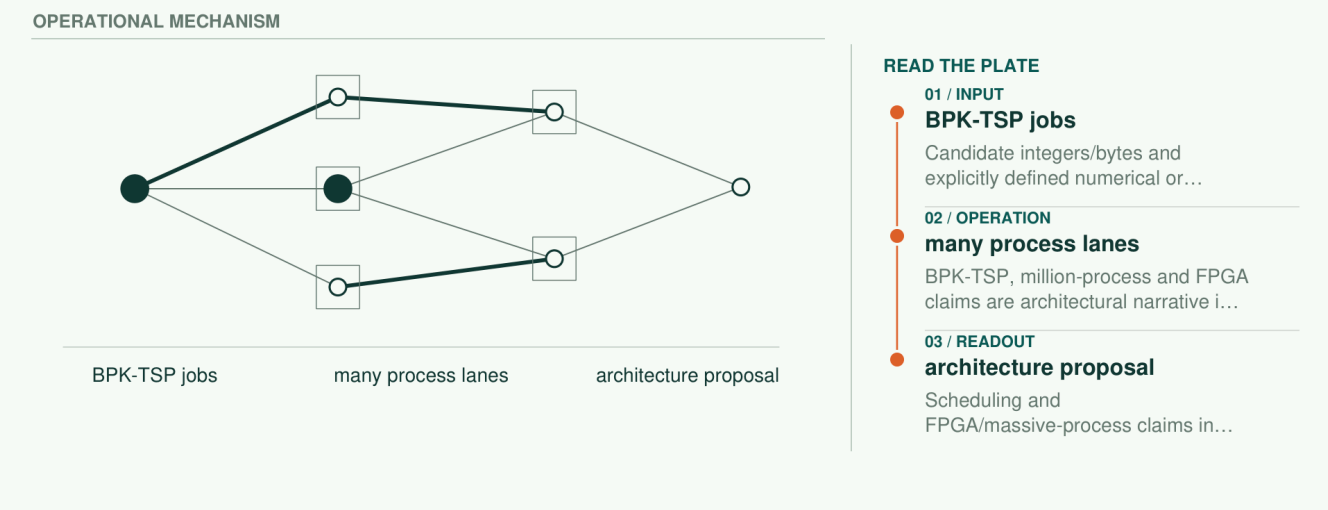
SECTION 10 / HP-RTE, TOPOLOGY AND PATTERN-AVOIDANCE EXPERIMENTS

BPK-TSP and million-process architecture

M149

EVIDENCE STATUS CONVERSATION PROPOSAL

FIGURE / METHOD MECHANISM



HOW IT WORKS

A scheduler could distribute generation, verification and artifact collection across workers, with hardware-specific executors.

INPUTS NEEDED

Candidate integers/bytes and explicitly defined numerical or pattern rules; mathematical diagnostics do not themselves supply target information.

WHAT WAS ACTUALLY DONE

BPK-TSP, million-process and FPGA claims are architectural narrative in the supplied material. Concrete implementations elsewhere are documented separately; this entry has no corroborated complete large-scale execution.

WHAT THE RESULT MEANS

Scheduling and FPGA/massive-process claims in supplied reports are proposals without corroborated execution evidence in this inventory.

MATHEMATICAL FORMULATION

$$T_{\text{ideal}} \geq \frac{N_{\text{work}}}{p\,v}$$

With p workers and per-worker rate v, this idealized lower bound ignores coordination and hardware limits. Process-count proposals require measured throughput and coverage.

EVIDENCE S003

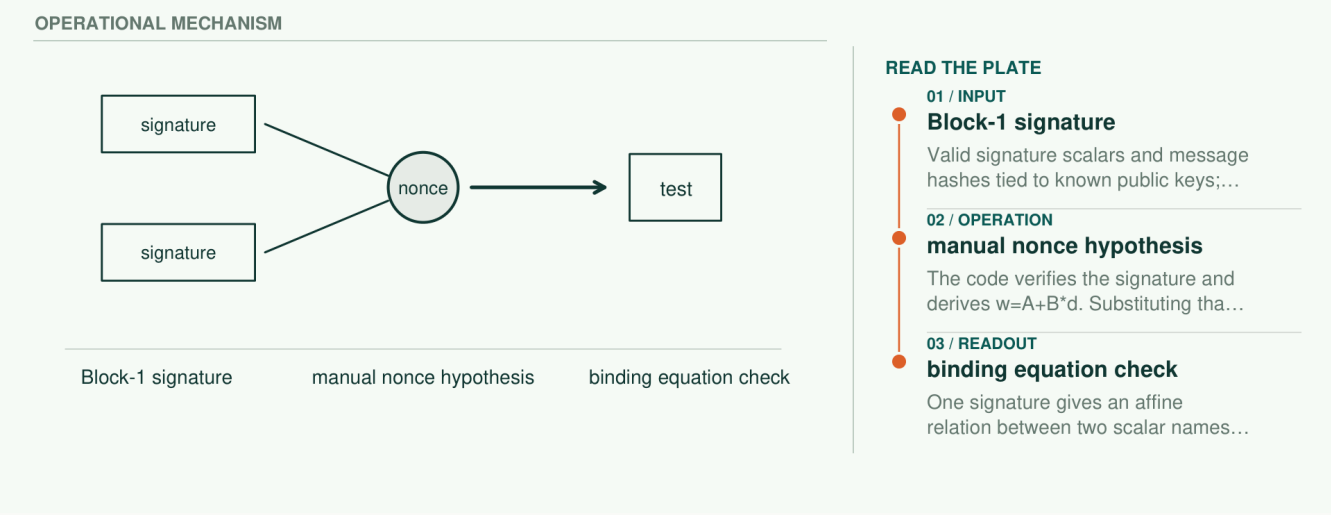
SECTION 11 / ECDSA, NONCE RELATIONS AND SIGNATURE-DATA SEARCHES

Aranyn Block-1 manual-nonce binding

M150

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

ECDSA gives s*w=z+r*d modulo the group order when the signing nonce is chosen as the Block-1 scalar w.

INPUTS NEEDED

Valid signature scalars and message hashes tied to known public keys; additional relations/leakage when the selected attack requires them.

WHAT WAS ACTUALLY DONE

The code verifies the signature and derives w=A+B*d. Substituting that relation back into the same signature adds no independent equation; one unknown scalar remains behind public-point multiplication.

WHAT THE RESULT MEANS

One signature gives an affine relation between two scalar names but leaves an ECDLP-equivalent unknown.

MATHEMATICAL FORMULATION

$$s\nu \equiv z + rd \pmod{n}$$

A single ECDSA signature relates signing key d and nonce nu, with message scalar z and signature r,s. Renaming the two unknowns does not determine them.

EVIDENCE S149
PAPERS PAPER-009

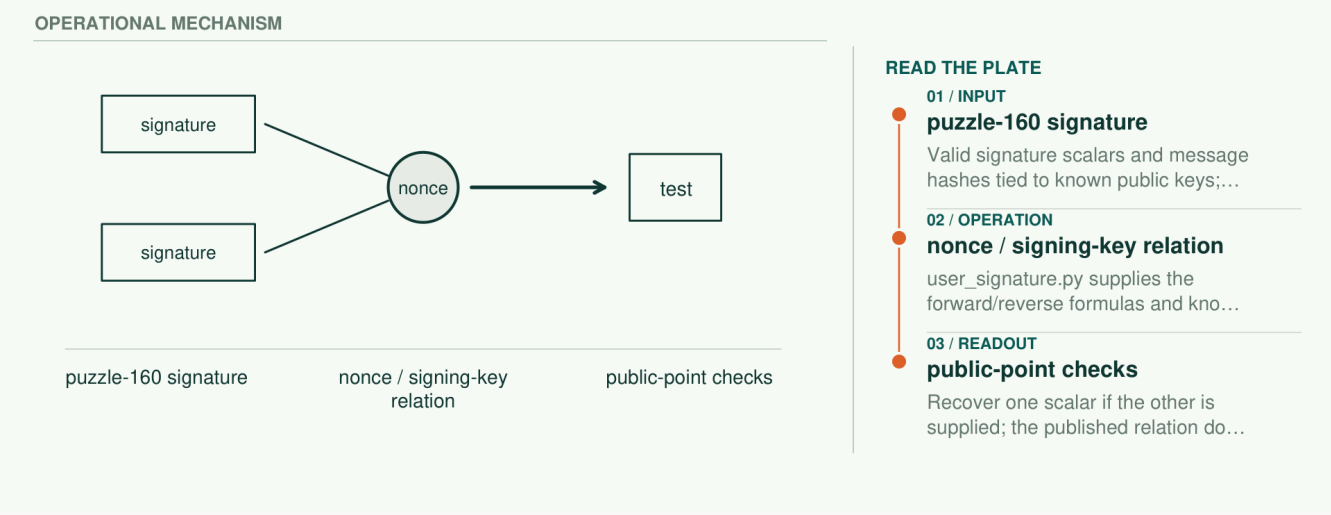
SECTION 11 / ECDSA, NONCE RELATIONS AND SIGNATURE-DATA SEARCHES

Puzzle-160 nonce/signing-key relation

M151

EVIDENCE STATUS IMPLEMENTED

FIGURE / METHOD MECHANISM



HOW IT WORKS

If an ECDSA nonce is the Puzzle-160 key, knowledge of either that nonce or the signer scalar determines the other algebraically.

INPUTS NEEDED

Valid signature scalars and message hashes tied to known public keys; additional relations/leakage when the selected attack requires them.

WHAT WAS ACTUALLY DONE

user_signature.py supplies the forward/reverse formulas and known public-point checks. Neither published relation alone reveals an undisclosed nonce; solving for it still requires new information or ECDLP work.

WHAT THE RESULT MEANS

Recover one scalar if the other is supplied; the published relation does not independently reveal either unknown key.

MATHEMATICAL FORMULATION

$$d \equiv r^{-1}(s\nu - z) \pmod{n}$$

One known nonce yields the signing scalar d when r is invertible. Without that extra input, the signature relation still has an unknown nonce.

EVIDENCE S127

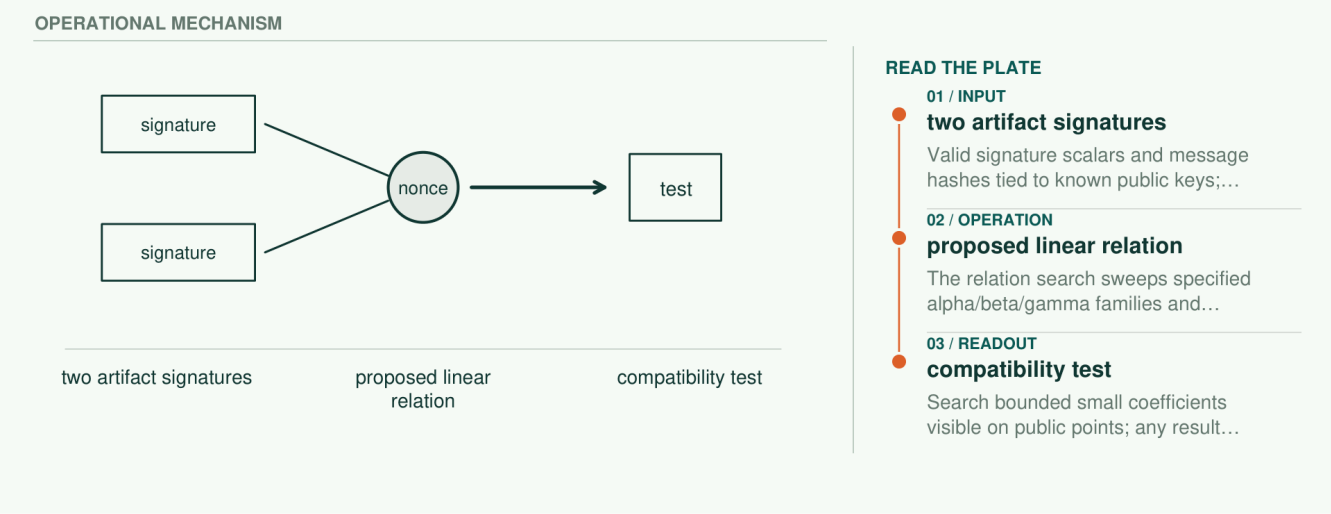
SECTION 11 / ECDSA, NONCE RELATIONS AND SIGNATURE-DATA SEARCHES

Cross-signature #160 / Block-1 linear relations

M152

EVIDENCE STATUS IMPLEMENTED

FIGURE / METHOD MECHANISM



HOW IT WORKS

Combine the two affine signature relations and test small coefficients directly on their public points.

WHAT WAS ACTUALLY DONE

The relation search sweeps specified alpha/beta/gamma families and verifies candidate equations. This can detect a genuine small relation without knowing either scalar, but cannot assume one exists.

INPUTS NEEDED

Valid signature scalars and message hashes tied to known public keys; additional relations/leakage when the selected attack requires them.

WHAT THE RESULT MEANS

Search bounded small coefficients visible on public points; any result would need independent target verification.

MATHEMATICAL FORMULATION

$$R_2 \stackrel{?}{=} aR_1 + bG$$

R1 and R2 here are nonce points, not CTheory roots. A bounded public-point relation would constrain nonces; it must be found and verified rather than assumed.

EVIDENCE S213 / S179

PAPERS PAPER-012

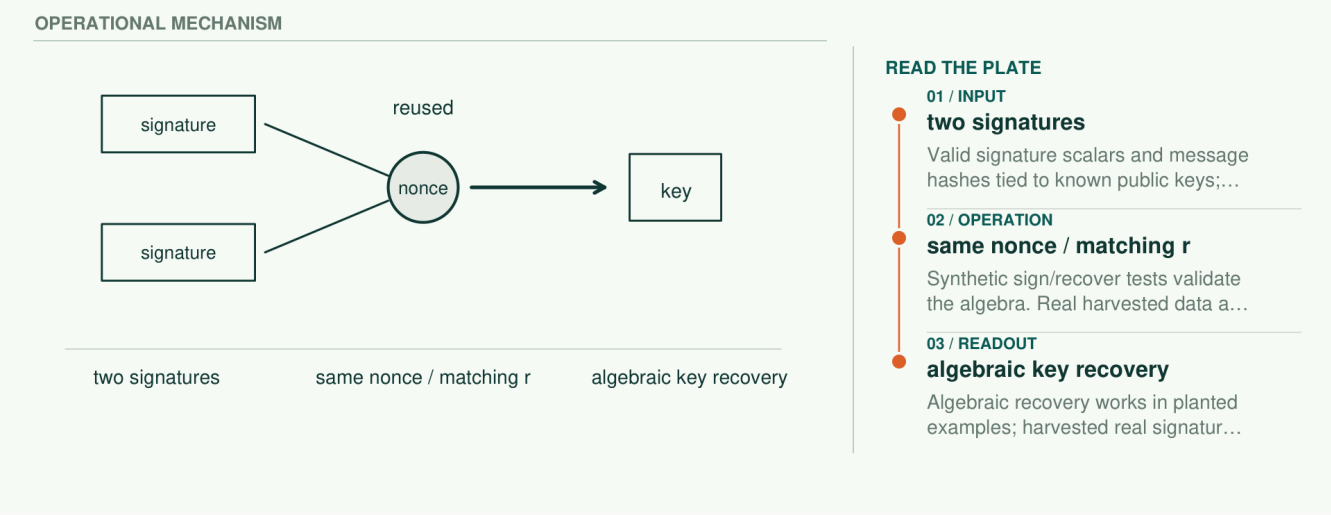
SECTION 11 / ECDSA, NONCE RELATIONS AND SIGNATURE-DATA SEARCHES

ECDSA nonce-reuse recovery

M153

EVIDENCE STATUS SYNTHETIC/RECORDED TESTS

FIGURE / METHOD MECHANISM



HOW IT WORKS

Reusing the same nonce for different messages creates two equations with the same unknown nonce, allowing elimination and scalar recovery.

INPUTS NEEDED

Valid signature scalars and message hashes tied to known public keys; additional relations/leakage when the selected attack requires them.

WHAT WAS ACTUALLY DONE

Synthetic sign/recover tests validate the algebra. Real harvested data are grouped by signer and r; the archived samples did not provide the required exploitable reuse pair.

WHAT THE RESULT MEANS

Algebraic recovery works in planted examples; harvested real signatures showed no matching reuse groups in recorded samples.

MATHEMATICAL FORMULATION

$$\nu \equiv (z_1 - z_2)(s_1 - s_2)^{-1} \pmod{n}$$

$$d \equiv (s_1\nu - z_1)r^{-1} \pmod{n}$$

These equations assume the same signing key, the same nonce, nonzero denominators and compatible signature signs. Repeated r alone also requires considering opposite nonce signs.

EVIDENCE S104 / S147

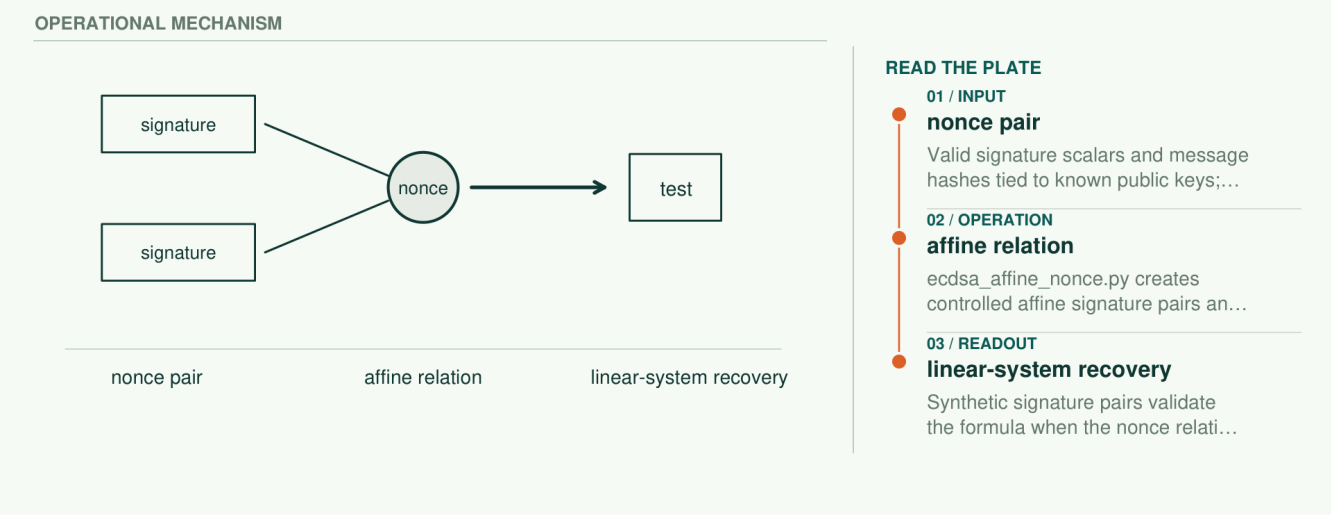
SECTION 11 / ECDSA, NONCE RELATIONS AND SIGNATURE-DATA SEARCHES

Known affine-related nonce recovery

M154

EVIDENCE STATUS SYNTHETIC TESTS

FIGURE / METHOD MECHANISM



HOW IT WORKS

A known relation k2=a*k1+b between two ECDSA nonces supplies an extra equation that makes the signer scalar solvable.

INPUTS NEEDED

Valid signature scalars and message hashes tied to known public keys; additional relations/leakage when the selected attack requires them.

WHAT WAS ACTUALLY DONE

ecdsa_affine_nonce.py creates controlled affine signature pairs and recovers the planted key. The puzzle target does not gain that relation merely because the implementation works on planted examples.

WHAT THE RESULT MEANS

Synthetic signature pairs validate the formula when the nonce relation is actually known. Applicability requires such a pair.

MATHEMATICAL FORMULATION

$$d \equiv \frac{s_1 z_2 - a s_2 z_1 - b s_1 s_2}{a s_2 r_1 - s_1 r_2} \pmod{n}$$

Assume nonce relation nu2=a nu1+b and the same signing key; division means modular inversion of a nonzero denominator. The affine relation is required input.

EVIDENCE S104 / S220

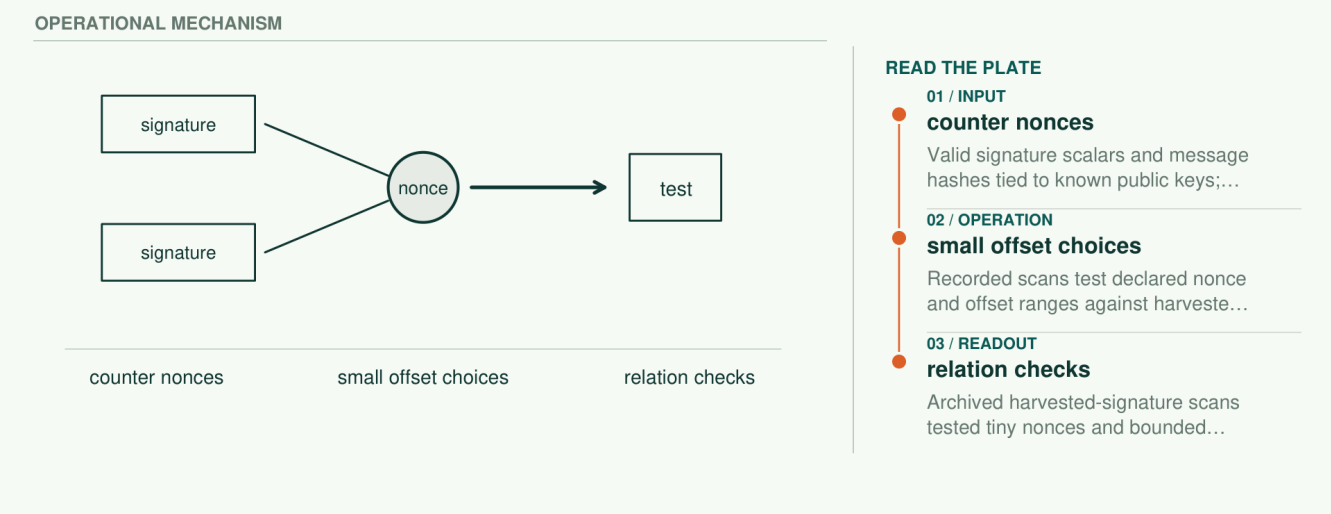
SECTION 11 / ECDSA, NONCE RELATIONS AND SIGNATURE-DATA SEARCHES

Counter nonces and small offset relation tests

M155

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

Guess tiny nonce values or small differences between two nonce values, then use ECDSA equations to derive a signer candidate.

INPUTS NEEDED

Valid signature scalars and message hashes tied to known public keys; additional relations/leakage when the selected attack requires them.

WHAT WAS ACTUALLY DONE

Recorded scans test declared nonce and offset ranges against harvested signatures and verify candidates. A null result covers those guesses, not arbitrary unknown nonce relations.

WHAT THE RESULT MEANS

Archived harvested-signature scans tested tiny nonces and bounded offsets; no verified wallet-key recovery reported.

MATHEMATICAL FORMULATION

$$\nu_2 = \nu_1 + \delta \pmod{n}, \quad \delta \in D$$

Test a bounded offset set D under the ECDSA equations. Without an applicable signature pair, an offset hypothesis supplies no recovered key.

EVIDENCE S147

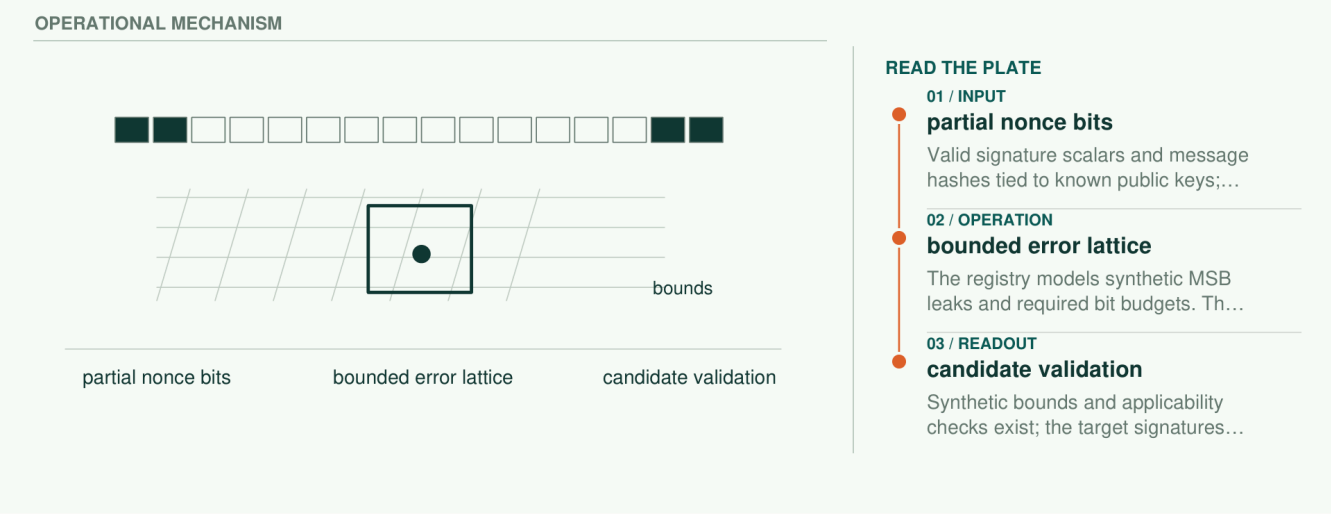
SECTION 11 / ECDSA, NONCE RELATIONS AND SIGNATURE-DATA SEARCHES

Hidden-number/partial-MSB nonce leakage models

M156

EVIDENCE STATUS SYNTHETIC/APPLICABILITY

FIGURE / METHOD MECHANISM



HOW IT WORKS

Multiple signatures with known nonce bits create inequalities that can support Hidden Number Problem/lattice recovery or smaller interval searches.

WHAT WAS ACTUALLY DONE

The registry models synthetic MSB leaks and required bit budgets. The available target data do not include the corresponding independent bit leaks or adequate applicable signatures.

INPUTS NEEDED

Valid signature scalars and message hashes tied to known public keys; additional relations/leakage when the selected attack requires them.

WHAT THE RESULT MEANS

Synthetic bounds and applicability checks exist; the target signatures do not supply the required leakage measurements.

MATHEMATICAL FORMULATION

$$s_i\nu_i \equiv z_i + r_i d \pmod{n}, \quad |\nu_i - \tilde{\nu}_i| < B_i$$

A partial-nonce hidden-number model combines signatures with genuine nonce estimates and error bounds. Signatures alone do not supply those leakage bounds.

EVIDENCE S108 / S149

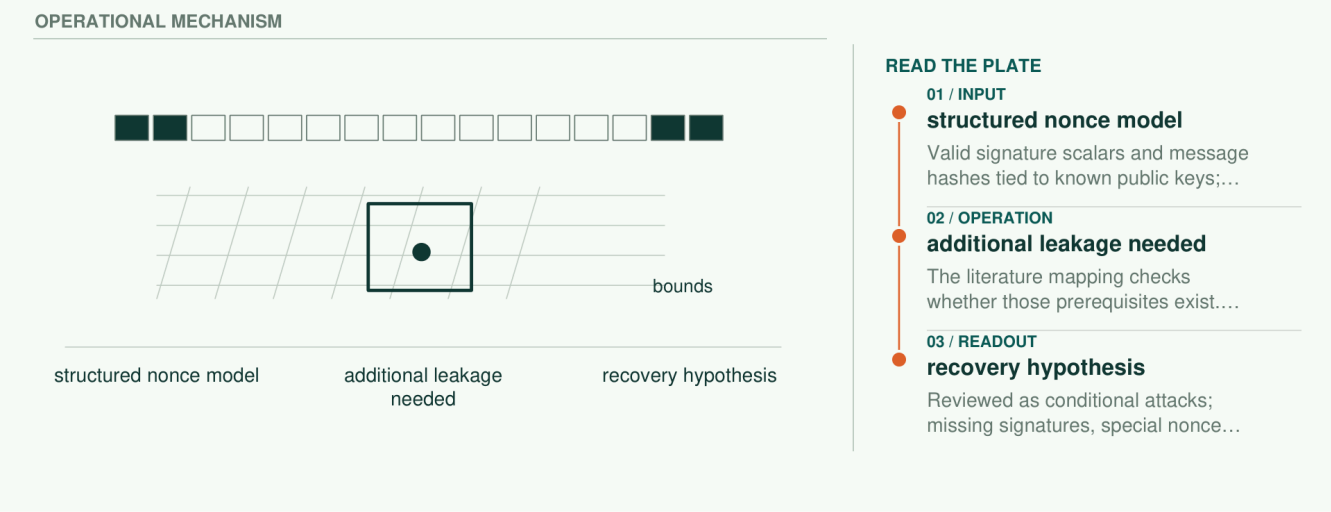
SECTION 11 / ECDSA, NONCE RELATIONS AND SIGNATURE-DATA SEARCHES

Polynonce/half-half/side-channel possibilities

M157

EVIDENCE STATUS APPLICABILITY REVIEW

FIGURE / METHOD MECHANISM



HOW IT WORKS

Polynomial nonce recurrences, concatenated half-nonces or measured side-channel leakage provide special extra structure beyond ordinary ECDSA.

WHAT WAS ACTUALLY DONE

The literature mapping checks whether those prerequisites exist. This is an applicability review; it does not execute a successful lattice/side-channel attack on a single ordinary signature.

INPUTS NEEDED

Valid signature scalars and message hashes tied to known public keys; additional relations/leakage when the selected attack requires them.

WHAT THE RESULT MEANS

Reviewed as conditional attacks; missing signatures, special nonce structure or measurements block their use on the cited target.

MATHEMATICAL FORMULATION

$$\nu_{i+1} = f(\nu_i) \pmod{n}$$

This represents a conditional structured-nonce family, such as a low-degree polynomial recurrence. Half-half and side-channel attacks have different required structure or measurements.

EVIDENCE S149

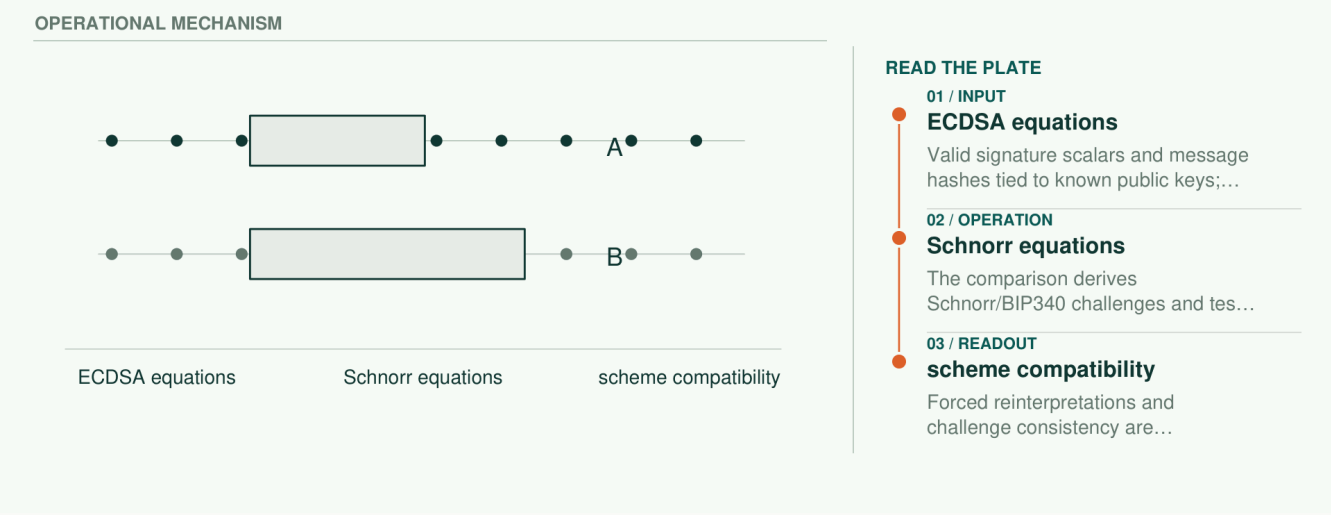
SECTION 11 / ECDSA, NONCE RELATIONS AND SIGNATURE-DATA SEARCHES

ECDSA versus Schnorr/BIP340 checks

M158

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

Schnorr and ECDSA use different equations and message challenges. A signature must satisfy the correct scheme's verification relation.

INPUTS NEEDED

Valid signature scalars and message hashes tied to known public keys; additional relations/leakage when the selected attack requires them.

WHAT WAS ACTUALLY DONE

The comparison derives Schnorr/BIP340 challenges and tests forced interpretations of the ECDSA constants. Reinterpreting bytes under another scheme does not provide a second valid signature equation.

WHAT THE RESULT MEANS

Forced reinterpretations and challenge consistency are compared; changing signature labels does not create a new independent equation.

MATHEMATICAL FORMULATION

$$sG = R + eP \quad \text{versus} \quad s\nu \equiv z + rd \pmod{n}$$

The first is Schnorr verification with scheme-defined challenge e and point parity rules; the second is ECDSA. Reinterpreting bytes does not create another independent signature.

EVIDENCE S212 / S182

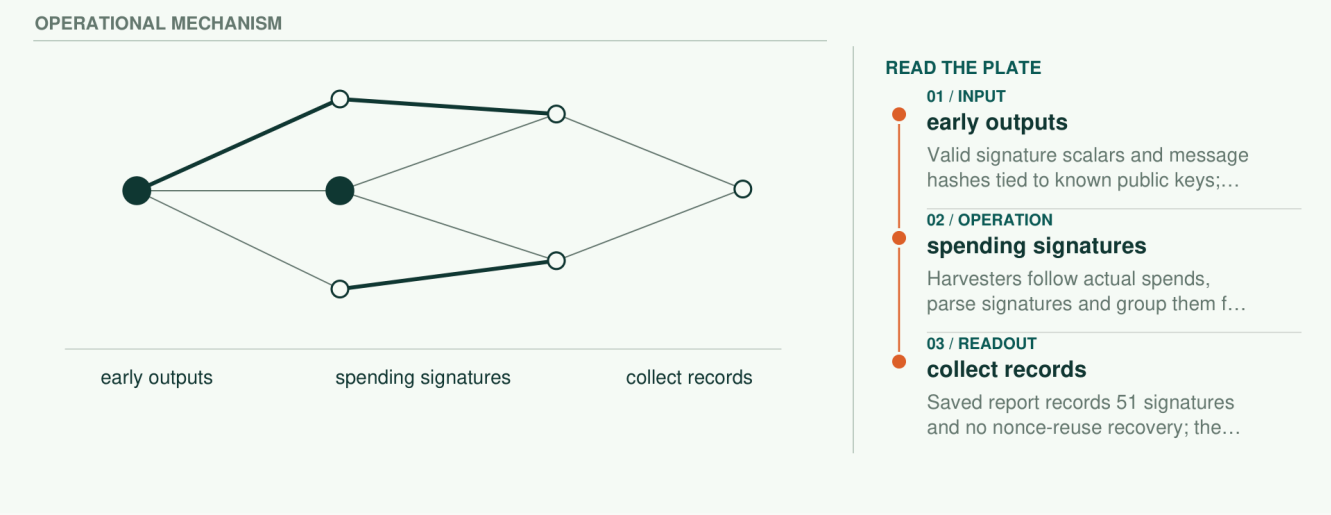
SECTION 11 / ECDSA, NONCE RELATIONS AND SIGNATURE-DATA SEARCHES

Early coinbase signature harvesting

M159

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

Extract signature data from spent early coinbase outputs and identify the signer/public point needed for nonce analysis.

INPUTS NEEDED

Valid signature scalars and message hashes tied to known public keys; additional relations/leakage when the selected attack requires them.

WHAT WAS ACTUALLY DONE

Harvesters follow actual spends, parse signatures and group them for reuse/target-r tests. The saved 51-signature sample provides data, not previously unknown recovered wallet keys.

WHAT THE RESULT MEANS

Saved report records 51 signatures and no nonce-reuse recovery; the graph is a data source, not a solve.

MATHEMATICAL FORMULATION

$$\Sigma = \left\{ (P_i, z_i, r_i, s_i, \text{source}_i) \right\}_{i=1}^{m}$$

Harvest signatures together with their exact signing public key and sighash context. A collected tuple is input to later tests, not a recovery result.

EVIDENCE S154

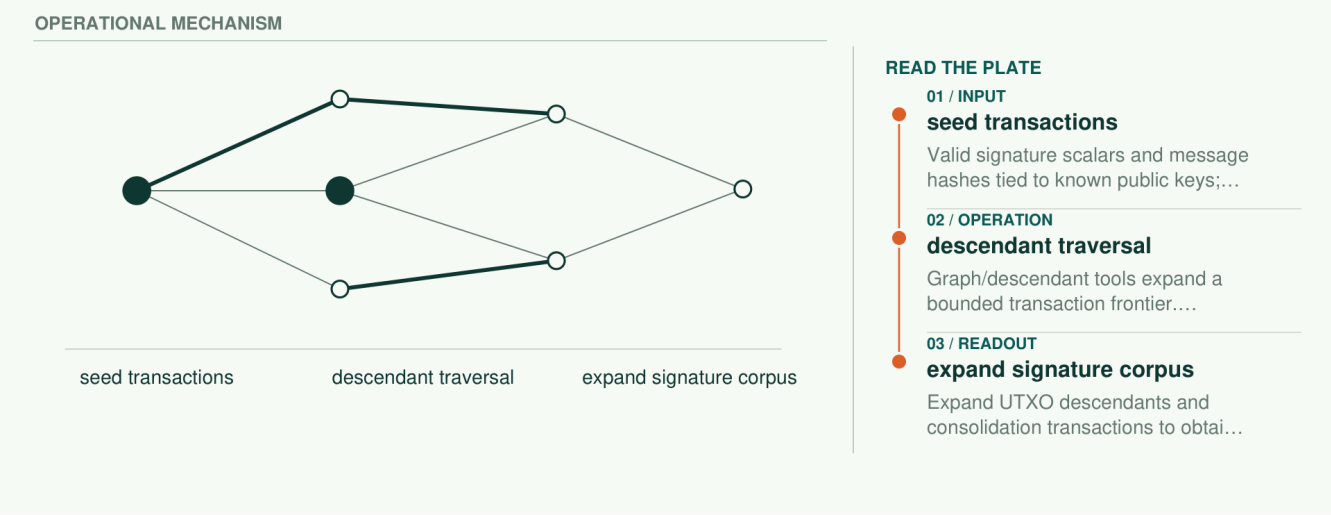
SECTION 11 / ECDSA, NONCE RELATIONS AND SIGNATURE-DATA SEARCHES

Descendant/graph signature expansion

M160

EVIDENCE STATUS IMPLEMENTED

FIGURE / METHOD MECHANISM



HOW IT WORKS

Follow descendants and consolidation transactions to collect more signatures associated with the original outputs.

INPUTS NEEDED

Valid signature scalars and message hashes tied to known public keys; additional relations/leakage when the selected attack requires them.

WHAT WAS ACTUALLY DONE

Graph/descendant tools expand a bounded transaction frontier. Additional transaction relationships can increase the sample but do not prove that every downstream signer shares a private scalar.

WHAT THE RESULT MEANS

Expand UTXO descendants and consolidation transactions to obtain additional signature samples.

MATHEMATICAL FORMULATION

$$V_{t+1} = V_t \cup \{v : (u, v) \in E,\ u \in V_t\}$$

Expand a transaction/UTXO graph from declared seeds, then extract usable signatures from reached spends. Define traversal depth and deduplicate transactions.

EVIDENCE S227 / S228

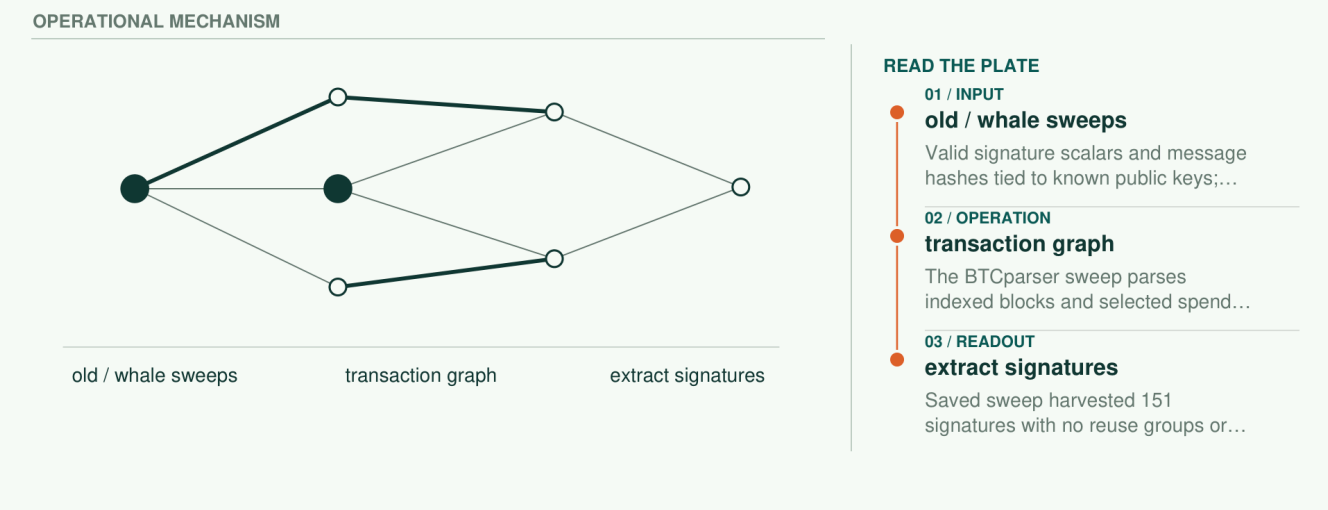
SECTION 11 / ECDSA, NONCE RELATIONS AND SIGNATURE-DATA SEARCHES

BTCparser sleeping/whale sweep signatures

M161

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

Use sleeping-address or whale-sweep indexes to select transactions likely to contain useful signature collections.

INPUTS NEEDED

Valid signature scalars and message hashes tied to known public keys; additional relations/leakage when the selected attack requires them.

WHAT WAS ACTUALLY DONE

The BTCparser sweep parses indexed blocks and selected spend transactions, then tests collected r values. It is a bounded data-gathering strategy, not evidence of biased nonces.

WHAT THE RESULT MEANS

Saved sweep harvested 151 signatures with no reuse groups or target-r hits in that sample.

MATHEMATICAL FORMULATION

$$\mathcal{G}_{P,r} = \{\sigma_i : P_i = P,\ r_i = r\}$$

Group harvested signatures by signing key and nonce x-coordinate indicator r. Any proposed reuse group needs exact sighash and sign-aware verification.

EVIDENCE S143

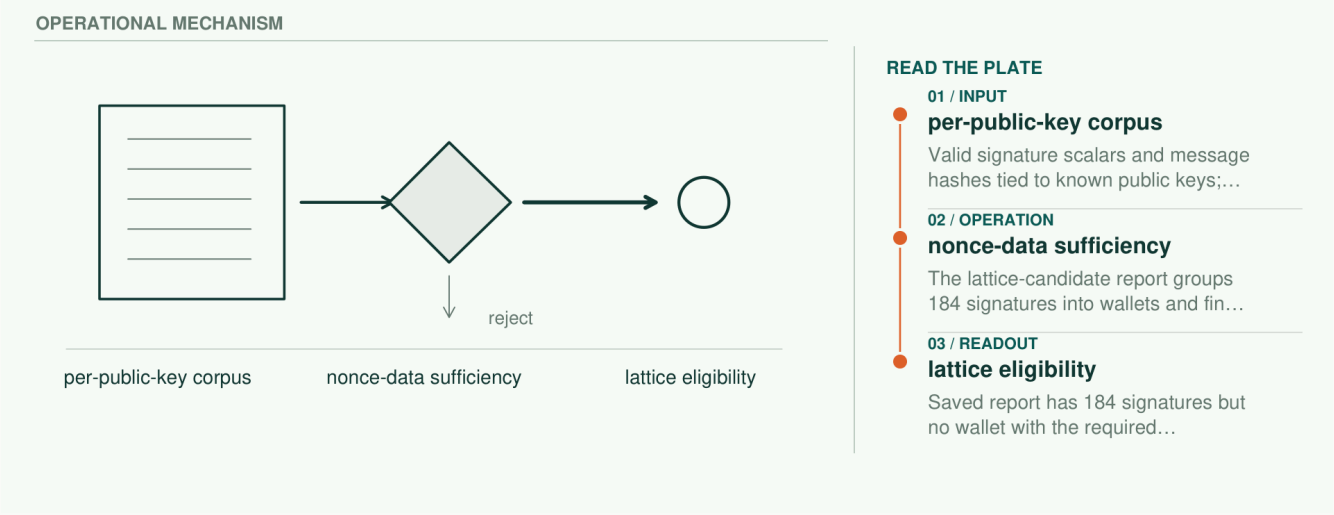
SECTION 11 / ECDSA, NONCE RELATIONS AND SIGNATURE-DATA SEARCHES

Lattice-candidate sufficiency scan

M162

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

Count usable signatures per signer before attempting attacks that require repeated nonce information or leakage.

INPUTS NEEDED

Valid signature scalars and message hashes tied to known public keys; additional relations/leakage when the selected attack requires them.

WHAT WAS ACTUALLY DONE

The lattice-candidate report groups 184 signatures into wallets and finds none meeting its multi-signature threshold. No target lattice recovery is performed simply by detecting that data are insufficient.

WHAT THE RESULT MEANS

Saved report has 184 signatures but no wallet with the required multi-signature count; no lattice recovery executed.

MATHEMATICAL FORMULATION

$$m_P = \#\{i : P_i = P\}, \quad m_P \geq m_{\text{required}}$$

Count usable signatures per signer under a specified leakage model. Passing a count threshold is necessary for that model, not sufficient for lattice recovery.

EVIDENCE S168

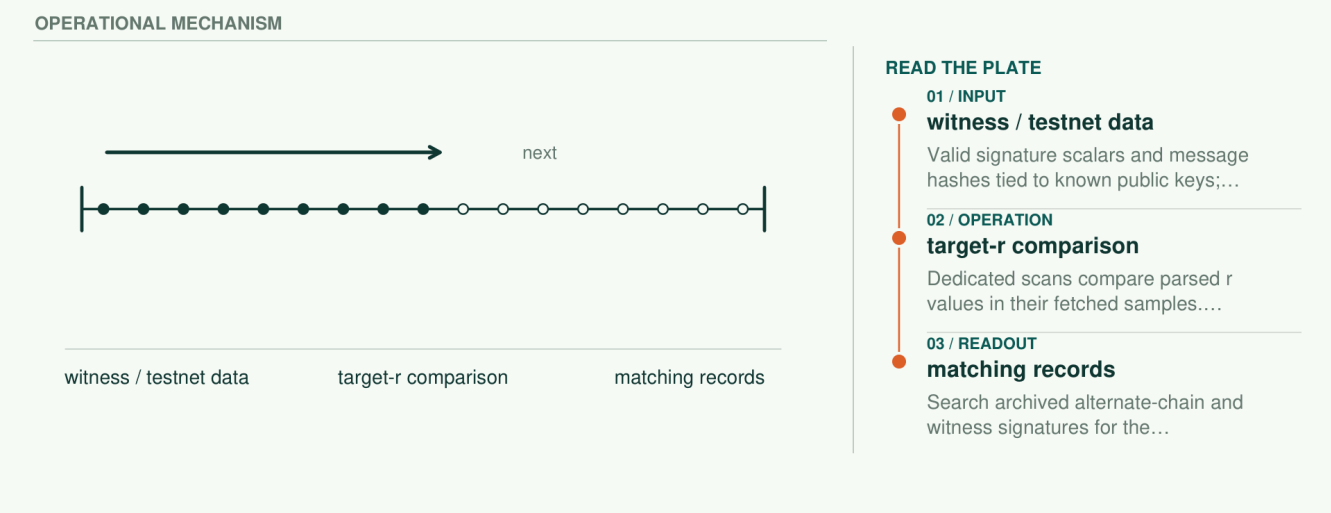
SECTION 11 / ECDSA, NONCE RELATIONS AND SIGNATURE-DATA SEARCHES

SegWit/testnet target-r scans

M163

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

Search additional witness and testnet signature data for the same nonce x-coordinate or related target public material.

INPUTS NEEDED

Valid signature scalars and message hashes tied to known public keys; additional relations/leakage when the selected attack requires them.

WHAT WAS ACTUALLY DONE

Dedicated scans compare parsed r values in their fetched samples. Matching a signature field would need scheme and signer verification; absence in a sample is not a global chain proof.

WHAT THE RESULT MEANS

Search archived alternate-chain and witness signatures for the manual-nonce x-coordinate; finite scan scope only.

MATHEMATICAL FORMULATION

$$r_i \stackrel{?}{=} r_*$$

Search the declared chain/witness signature corpus for the target signature component. A match requires further signer and nonce-relation checks.

EVIDENCE S184 / S189

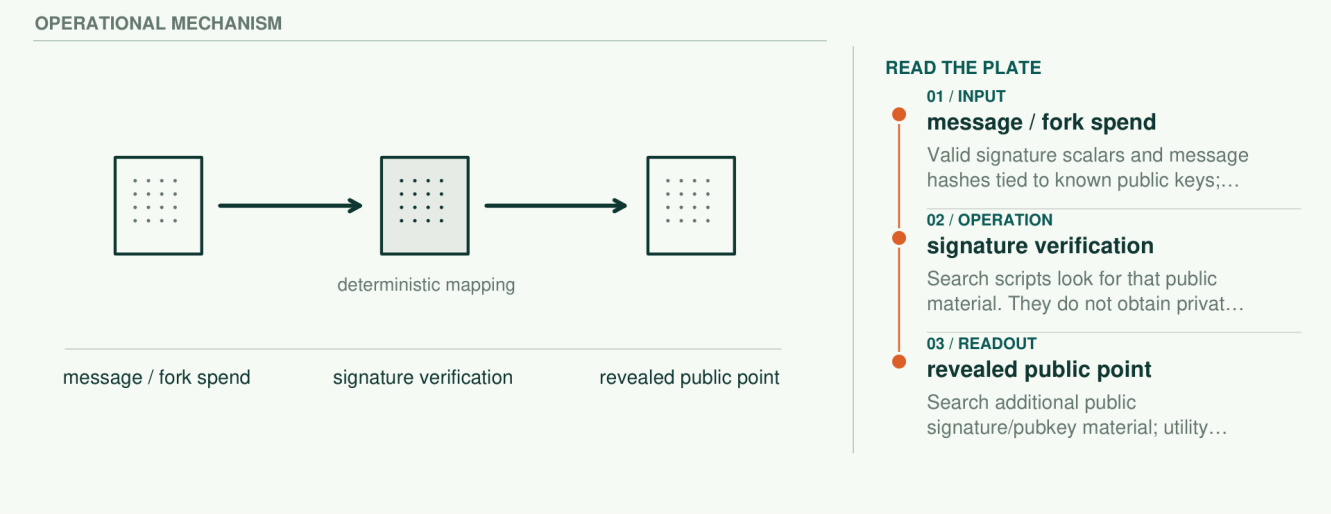
SECTION 11 / ECDSA, NONCE RELATIONS AND SIGNATURE-DATA SEARCHES

Signed-message and fork public-key leaks

M164

EVIDENCE STATUS IMPLEMENTED

FIGURE / METHOD MECHANISM



HOW IT WORKS

Extra signed messages or fork spends may reveal a public point or an independently usable signature absent from the current target record.

INPUTS NEEDED

Valid signature scalars and message hashes tied to known public keys; additional relations/leakage when the selected attack requires them.

WHAT WAS ACTUALLY DONE

Search scripts look for that public material. They do not obtain private keys from a public-point leak alone; they may only enable a different, still expensive recovery method.

WHAT THE RESULT MEANS

Search additional public signature/pubkey material; utility depends on finding independently applicable data.

MATHEMATICAL FORMULATION

$$\mathrm{Verify}_{\mathrm{scheme}}(P, m, \sigma) = 1$$

Signed messages and fork records supply useful evidence only when parsed under the correct message digest, scheme and public key.

EVIDENCE S235 / S234

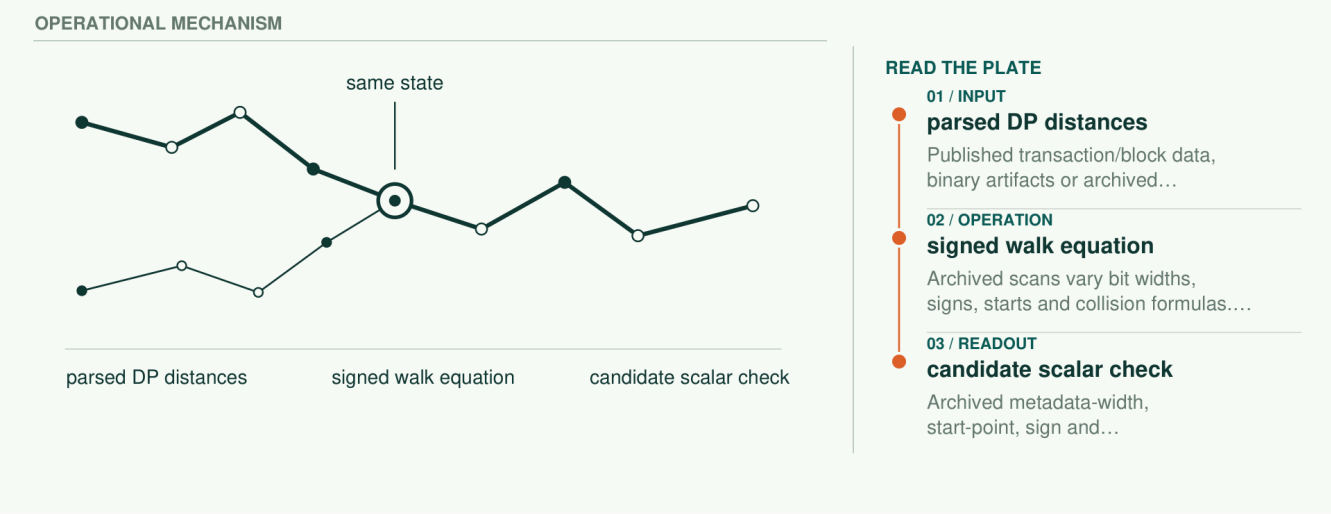
SECTION 12 / ON-CHAIN, METADATA AND CLUE RELATIONSHIPS

DP metadata distances and signed collision variants

M165

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

Interpret candidate metadata slices as walk distances and test whether they satisfy the claimed tame/wild public-point equations.

INPUTS NEEDED

Published transaction/block data, binary artifacts or archived messages and a fixed target comparison. External facts need separate authentication.

WHAT WAS ACTUALLY DONE

Archived scans vary bit widths, signs, starts and collision formulas. A distance-looking field is rejected unless its derived point or scalar matches the target.

WHAT THE RESULT MEANS

Archived metadata-width, start-point, sign and collision-formula tests found no verified puzzle scalar.

MATHEMATICAL FORMULATION

$$k \equiv \epsilon t - w \pmod{n}, \quad \epsilon \in \{-1, 1\}$$

At a same-x tame/wild record, epsilon accounts for point parity in tG=epsilon(Q+wG). Test declared distance layouts and verify each resulting scalar.

EVIDENCE S171

PAPERS PAPER-011

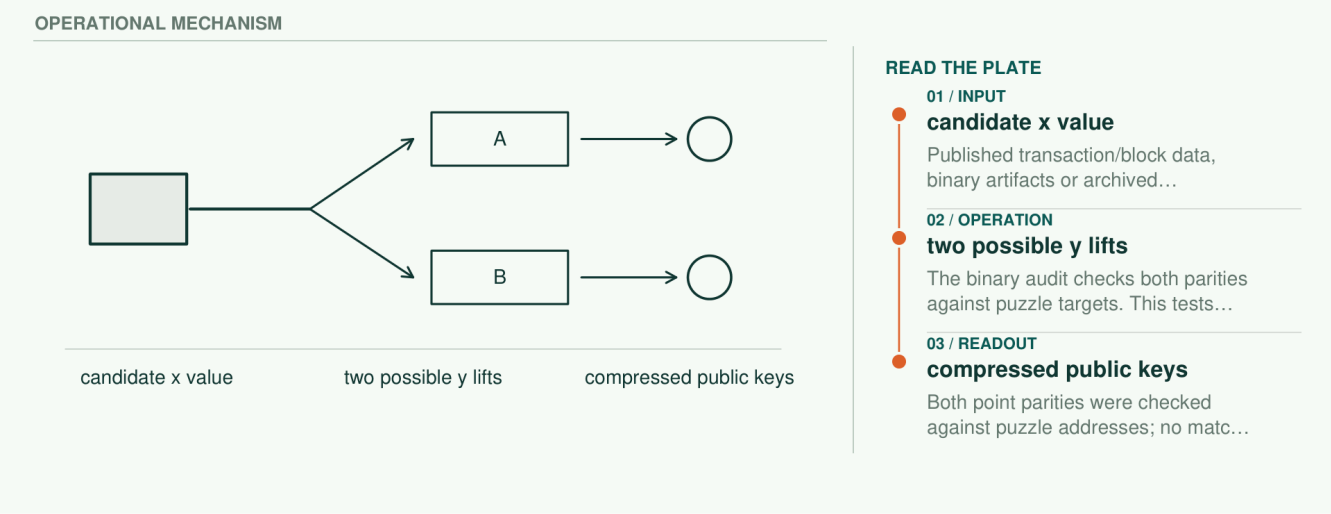
SECTION 12 / ON-CHAIN, METADATA AND CLUE RELATIONSHIPS

DP x-coordinates as candidate public keys

M166

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

A valid x-coordinate has up to two curve-point lifts, each yielding a possible compressed public key and address hash.

INPUTS NEEDED

Published transaction/block data, binary artifacts or archived messages and a fixed target comparison. External facts need separate authentication.

WHAT WAS ACTUALLY DONE

The binary audit checks both parities against puzzle targets. This tests whether the dump literally contains the target public key, not whether arbitrary points can be inverted into scalars.

WHAT THE RESULT MEANS

Both point parities were checked against puzzle addresses; no match in the recorded binary sample.

MATHEMATICAL FORMULATION

$$y^2 \equiv x^3 + 7 \pmod{p}$$

A valid secp256k1 x-coordinate has candidate point parities y and -y. Serialize each valid point and compare its address hash; an arbitrary x need not lift.

EVIDENCE S171

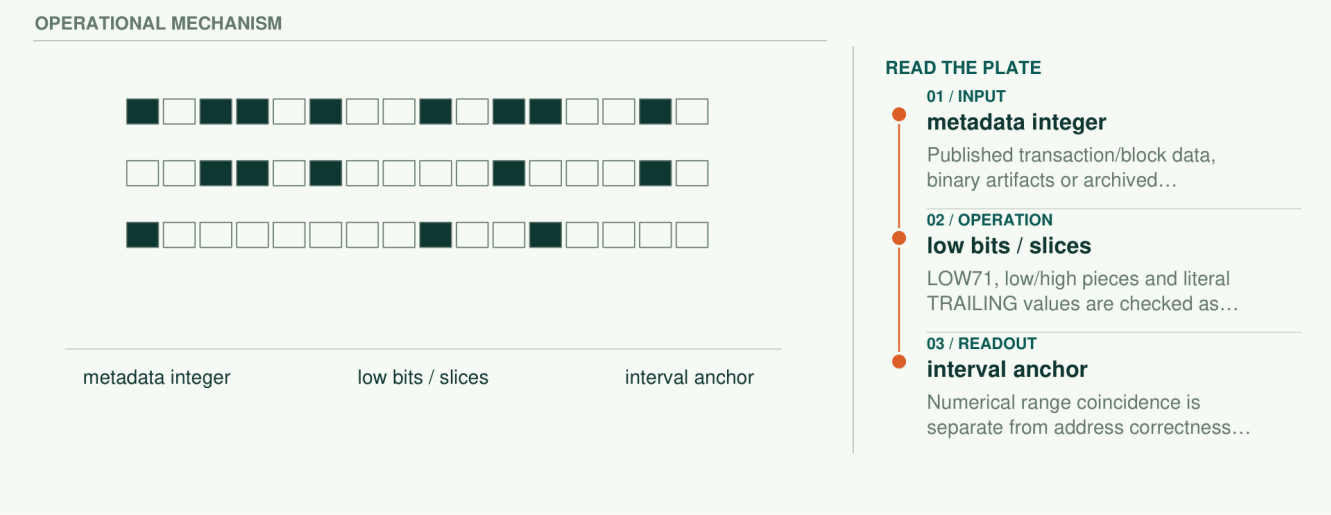
SECTION 12 / ON-CHAIN, METADATA AND CLUE RELATIONSHIPS

TRAILING / LOW71 / bit-slice anchors

M167

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

Slice or reinterpret a trailing metadata integer to produce anchor scalars for selected puzzle widths.

WHAT WAS ACTUALLY DONE

LOW71, low/high pieces and literal TRAILING values are checked as concrete candidates. Their bit length or range membership alone is not evidence of target identity.

INPUTS NEEDED

Published transaction/block data, binary artifacts or archived messages and a fixed target comparison. External facts need separate authentication.

WHAT THE RESULT MEANS

Numerical range coincidence is separate from address correctness; recorded anchor checks failed target verification.

MATHEMATICAL FORMULATION

$$a_b = 2^{b-1} + (M \bmod 2^{b-1})$$

A metadata word M can be masked into a b-bit puzzle interval. Being in range establishes no relation to the target hash.

EVIDENCE S137 / S171

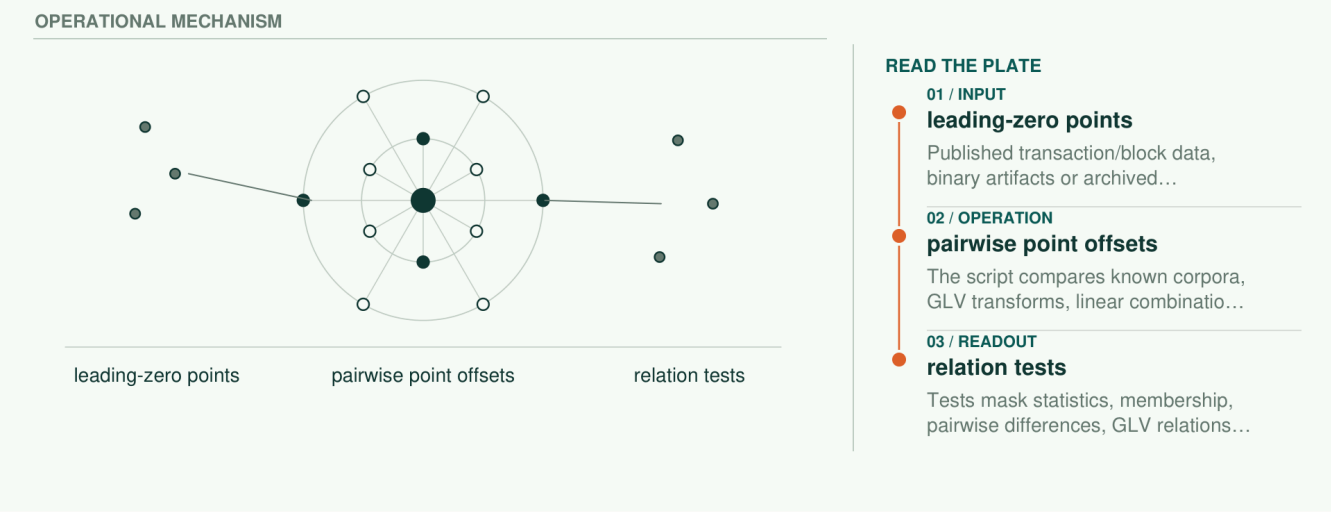
SECTION 12 / ON-CHAIN, METADATA AND CLUE RELATIONSHIPS

Leading-zero public-key collection analysis

M168

EVIDENCE STATUS IMPLEMENTED

FIGURE / METHOD MECHANISM



HOW IT WORKS

Analyze a collection selected for leading-zero x-coordinate masks, then search membership, pairwise offsets and small public-point relations.

INPUTS NEEDED

Published transaction/block data, binary artifacts or archived messages and a fixed target comparison. External facts need separate authentication.

WHAT WAS ACTUALLY DONE

The script compares known corpora, GLV transforms, linear combinations, bounded BSGS differences and generator recipes. Mask rarity must be evaluated conditional on the deliberate point-selection rule.

WHAT THE RESULT MEANS

Tests mask statistics, membership, pairwise differences, GLV relations, linear combinations and bounded BSGS offsets.

MATHEMATICAL FORMULATION

$$P_j \stackrel{?}{=} P_i + \delta G, \quad \delta \in D$$

Bounded-offset and other point-relation tests need declared search sets. A collection selected for leading zero bits is not evidence of scalar proximity.

EVIDENCE S203

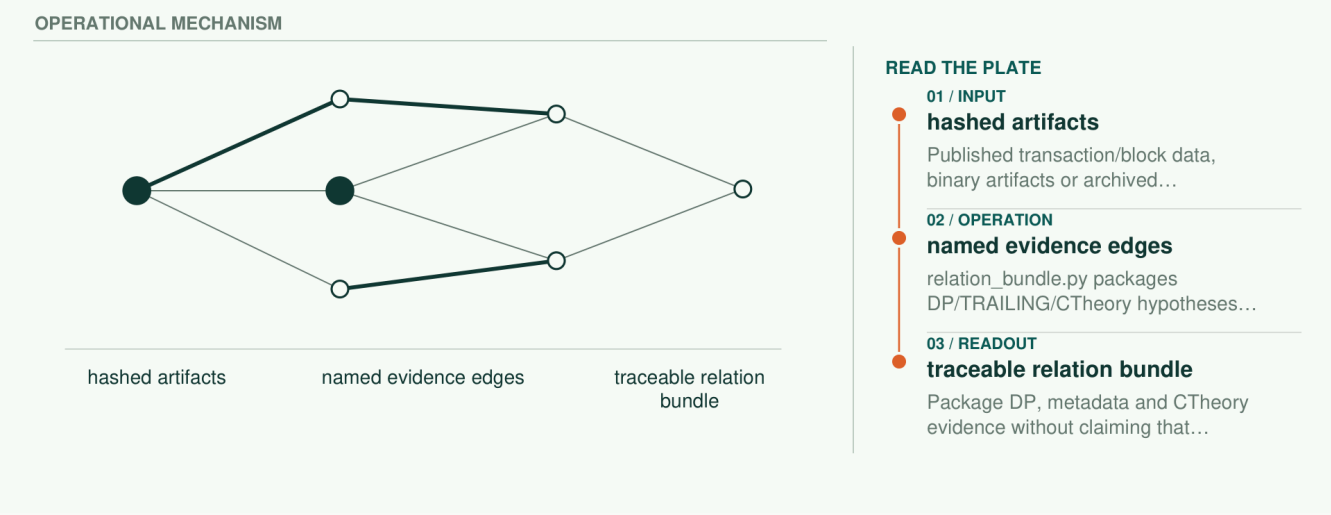
SECTION 12 / ON-CHAIN, METADATA AND CLUE RELATIONSHIPS

Cross-artifact relation bundles

M169

EVIDENCE STATUS IMPLEMENTED

FIGURE / METHOD MECHANISM



HOW IT WORKS

Bundle source hashes, parsed values and named relationships so later investigations can trace every clue to its artifact.

INPUTS NEEDED

Published transaction/block data, binary artifacts or archived messages and a fixed target comparison. External facts need separate authentication.

WHAT WAS ACTUALLY DONE

relation_bundle.py packages DP/TRAILING/CTheory hypotheses and metadata. It preserves evidence lineage without claiming that colocated numbers imply a key equation.

WHAT THE RESULT MEANS

Package DP, metadata and CTheory evidence without claiming that correlated artifacts reveal a key.

MATHEMATICAL FORMULATION

$$\mathcal{E} = \mathcal{E}_{\mathrm{DP}} \cup \mathcal{E}_{\mathrm{metadata}} \cup \mathcal{E}_{\mathrm{CT}}$$

Bundle artifacts with their provenance and dependencies. Combining correlated evidence does not make it independent scalar information.

EVIDENCE S118

PAPERS PAPER-009

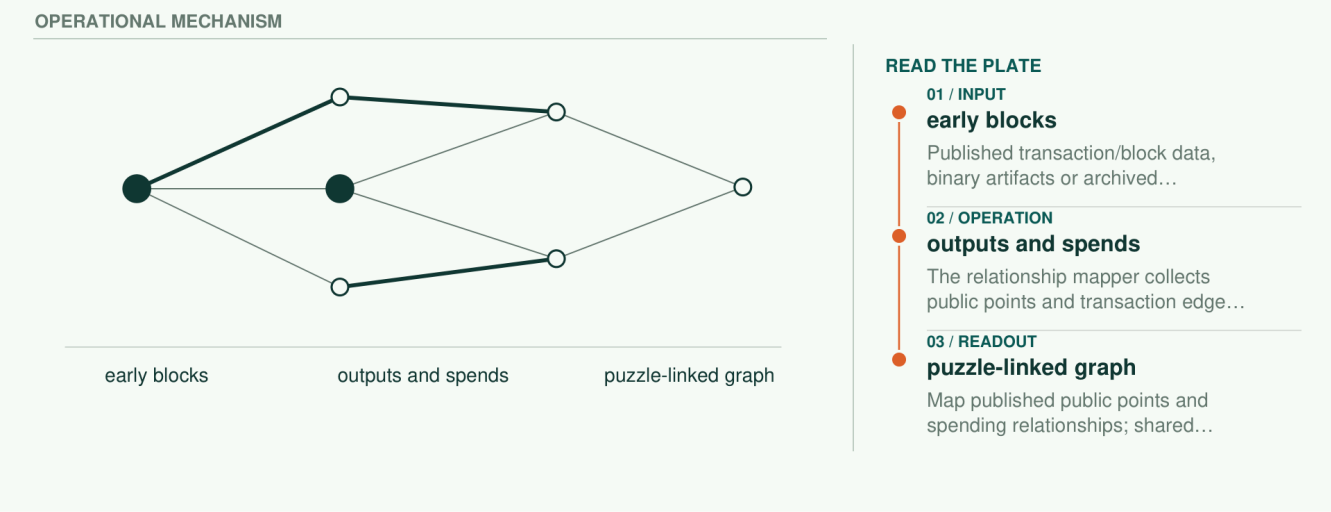
SECTION 12 / ON-CHAIN, METADATA AND CLUE RELATIONSHIPS

Genesis/early-coinbase relationship graph

M170

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

Build a graph linking early blocks, output public keys, spends and known puzzle entities.

WHAT WAS ACTUALLY DONE

The relationship mapper collects public points and transaction edges. A shared transaction or funder is a graph link; recovering a shared scalar requires separate cryptographic evidence.

INPUTS NEEDED

Published transaction/block data, binary artifacts or archived messages and a fixed target comparison. External facts need separate authentication.

WHAT THE RESULT MEANS

Map published public points and spending relationships; shared transactions need not imply shared private keys.

MATHEMATICAL FORMULATION

$$\mathcal{G} = (V, E), \quad E \subseteq V \times V$$

Nodes represent declared blocks, transactions or public-key objects; edges encode observed relationships. A graph edge does not imply equal private scalars.

EVIDENCE S144
PAPERS PAPER-002

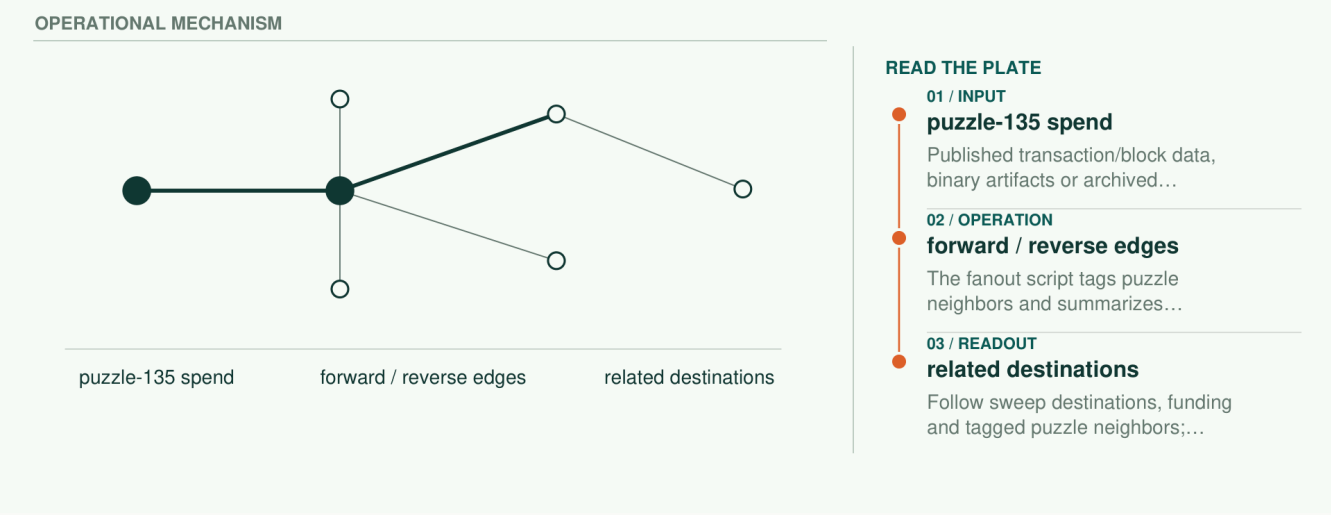
SECTION 12 / ON-CHAIN, METADATA AND CLUE RELATIONSHIPS

Puzzle-135 sweep/fanout tracing

M171

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

Follow the known Puzzle-135 spending destination forward and backward to identify related addresses or messages.

INPUTS NEEDED

Published transaction/block data, binary artifacts or archived messages and a fixed target comparison. External facts need separate authentication.

WHAT WAS ACTUALLY DONE

The fanout script tags puzzle neighbors and summarizes transaction flows. It tracks an observed sweep; it does not establish that this local code discovered the spent key.

WHAT THE RESULT MEANS

Follow sweep destinations, funding and tagged puzzle neighbors; movement of funds is not local proof of solving the puzzle.

MATHEMATICAL FORMULATION

$$\mathrm{Reach}_d(v) = \{u : \mathrm{dist}_{\mathcal{G}}(v,u) \le d\}$$

Follow bounded sweep/fanout relationships in the dated chain snapshot. Observed fund movement is distinct from a locally verified key-discovery event.

EVIDENCE S158

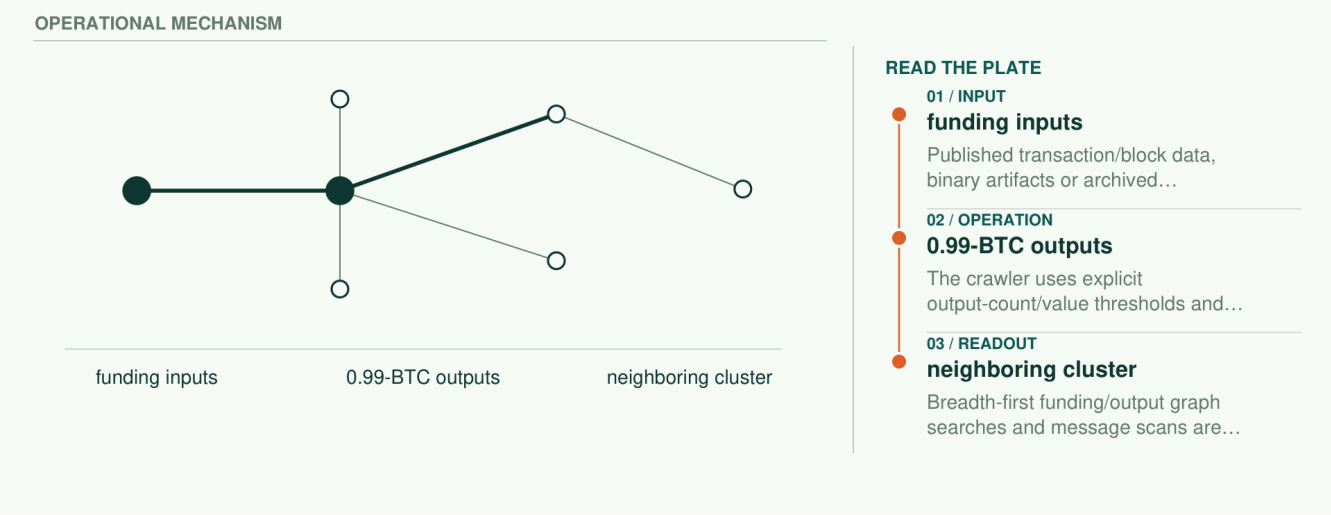
SECTION 12 / ON-CHAIN, METADATA AND CLUE RELATIONSHIPS

0.99-BTC fanout cluster tracing

M172

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

Detect transactions with repeated 0.99-BTC outputs and traverse their funding/fanout neighbors.

WHAT WAS ACTUALLY DONE

The crawler uses explicit output-count/value thresholds and bounded BFS expansion. This identifies a transaction pattern or cluster, not an algebraic relation among its private keys.

INPUTS NEEDED

Published transaction/block data, binary artifacts or archived messages and a fixed target comparison. External facts need separate authentication.

WHAT THE RESULT MEANS

Breadth-first funding/output graph searches and message scans are archived.

MATHEMATICAL FORMULATION

$$E_{0.99} = \{(u, v) \in E : \text{value}(u, v) = 0.99 \text{ BTC}\}$$

This is an example value-conditioned graph filter. State the actual amount tolerance and traversal bounds used by the archived cluster scan.

EVIDENCE S159 / S157

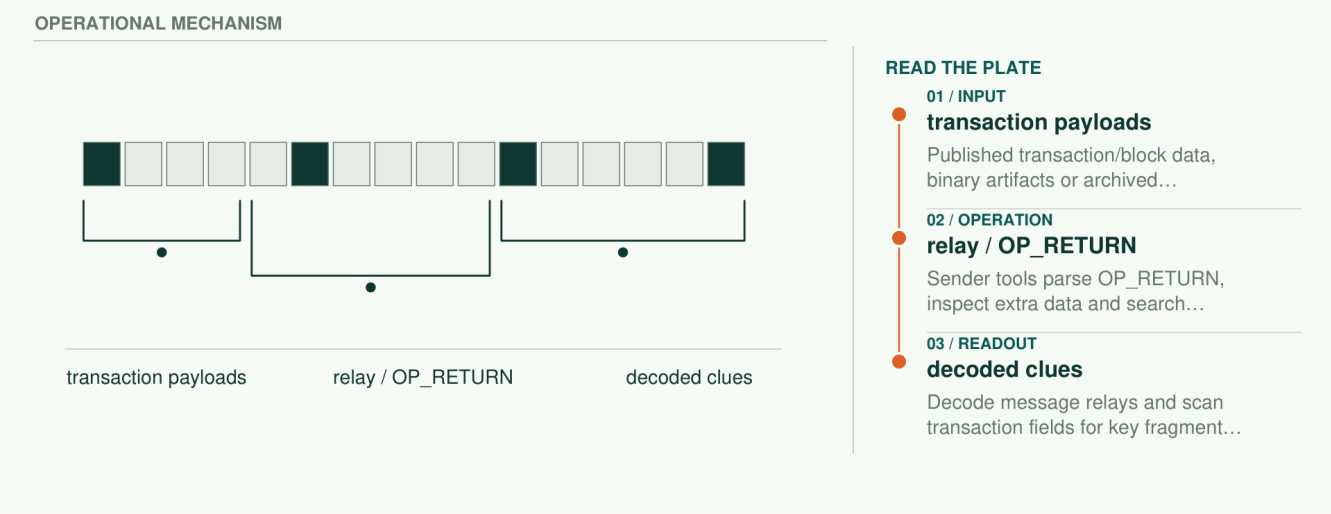
SECTION 12 / ON-CHAIN, METADATA AND CLUE RELATIONSHIPS

Sender relay / OP_RETURN / extra-data clues

M173

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

Decode message-bearing transaction fields and follow a relay chain for possible puzzle instructions or artifacts.

INPUTS NEEDED

Published transaction/block data, binary artifacts or archived messages and a fixed target comparison. External facts need separate authentication.

WHAT WAS ACTUALLY DONE

Sender tools parse OP_RETURN, inspect extra data and search references. Public text is treated as a lead requiring verification, not executable instructions or proven secret leakage.

WHAT THE RESULT MEANS

Decode message relays and scan transaction fields for key fragments or target relations.

MATHEMATICAL FORMULATION

$$\mathcal{M} = \{\mathrm{decode}(x) : x \in \mathrm{payloads}(\mathcal{T})\}$$

Decode declared transaction fields, including OP_RETURN payloads, and inspect for concrete candidate material. Decoded text alone is not a scalar relation.

EVIDENCE S190 / S191

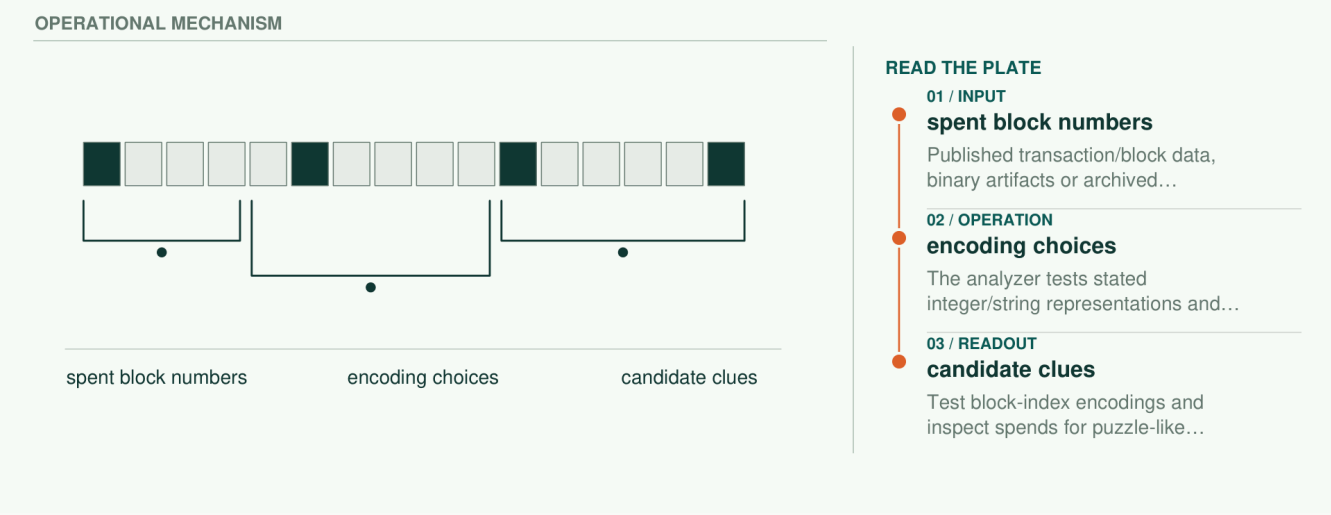
SECTION 12 / ON-CHAIN, METADATA AND CLUE RELATIONSHIPS

Early spent-block number encodings

M174

EVIDENCE STATUS IMPLEMENTED

FIGURE / METHOD MECHANISM



HOW IT WORKS

Convert lists of spent block numbers or transaction fields into candidate encodings and compare with known puzzle clues.

INPUTS NEEDED

Published transaction/block data, binary artifacts or archived messages and a fixed target comparison. External facts need separate authentication.

WHAT WAS ACTUALLY DONE

The analyzer tests stated integer/string representations and inspects spends. Finding readable text or a similar hex fragment is exploratory evidence, not a verified key.

WHAT THE RESULT MEANS

Test block-index encodings and inspect spends for puzzle-like hexadecimal data.

MATHEMATICAL FORMULATION

$$\mathcal{C} = \left\{ \mathrm{Encode}_f(h) : h \in \mathcal{H},\ f \in \mathcal{F} \right\}$$

Test a bounded collection of block-height encodings and transformations. Exact candidate verification controls coincidences in hexadecimal representations.

EVIDENCE S204

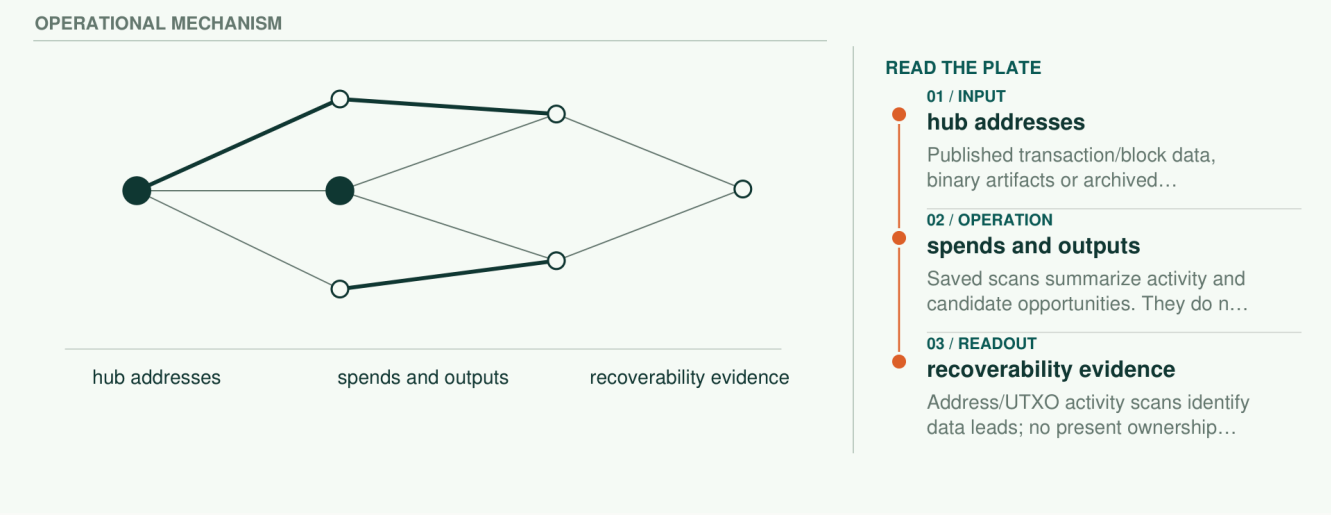
SECTION 12 / ON-CHAIN, METADATA AND CLUE RELATIONSHIPS

Hub-address and recoverability scans

M175

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

Query selected addresses and outputs to find spends, public keys, old funds or other data relevant to recoverability hypotheses.

INPUTS NEEDED

Published transaction/block data, binary artifacts or archived messages and a fixed target comparison. External facts need separate authentication.

WHAT WAS ACTUALLY DONE

Saved scans summarize activity and candidate opportunities. They do not freshly establish present balances, authority over wallets or a successful cryptographic recovery.

WHAT THE RESULT MEANS

Address/UTXO activity scans identify data leads; no present ownership or balance conclusions are newly verified here.

MATHEMATICAL FORMULATION

$$V(A) = \sum_{u \in \mathrm{UTXO}(A)} \mathrm{value}(u)$$

This defines snapshot balance for address A. Activity or hub status can guide evidence collection but does not show possession or recoverability of its private key.

EVIDENCE S164 / S178

PAPERS PAPER-026

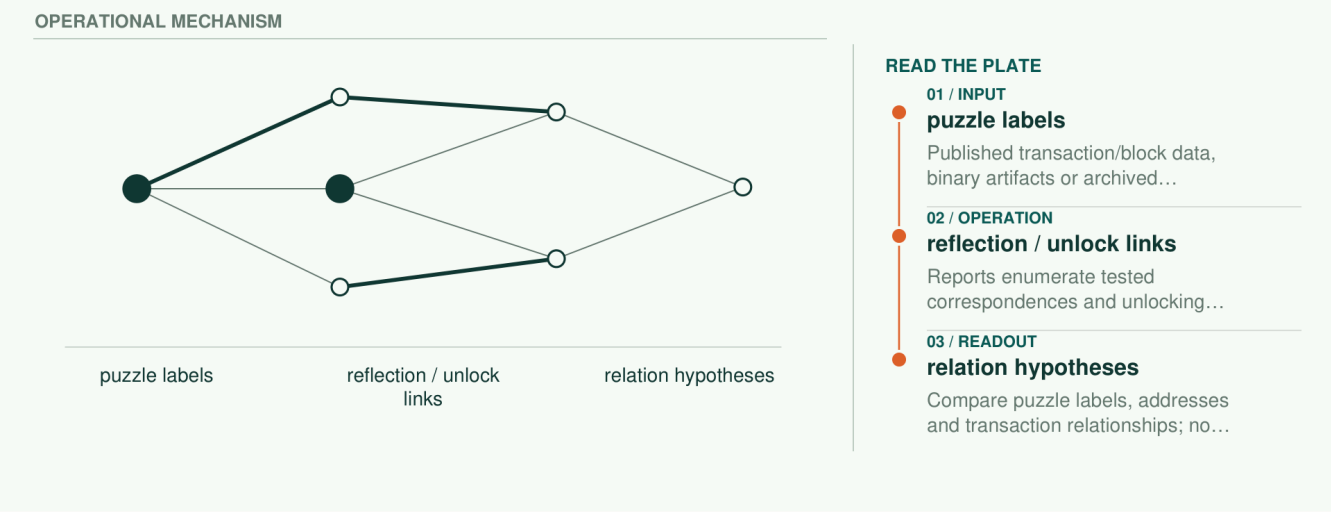
SECTION 12 / ON-CHAIN, METADATA AND CLUE RELATIONSHIPS

Puzzle reflection and unlock relationship scans

M176

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

Look for reflected labels, addresses or tier relationships suggesting that one puzzle might encode information useful for another.

INPUTS NEEDED

Published transaction/block data, binary artifacts or archived messages and a fixed target comparison. External facts need separate authentication.

WHAT WAS ACTUALLY DONE

Reports enumerate tested correspondences and unlocking ideas. A relation between labels is distinct from a demonstrated scalar transformation that reproduces both targets.

WHAT THE RESULT MEANS

Compare puzzle labels, addresses and transaction relationships; no validated universal unlocking rule established.

MATHEMATICAL FORMULATION

$$\widehat{Q}_j = aQ_i + bG$$

Reflection/unlock proposals need a specified mapping, such as this affine point hypothesis, and held-out exact checks. No universal relation is assumed.

EVIDENCE S176 / S177

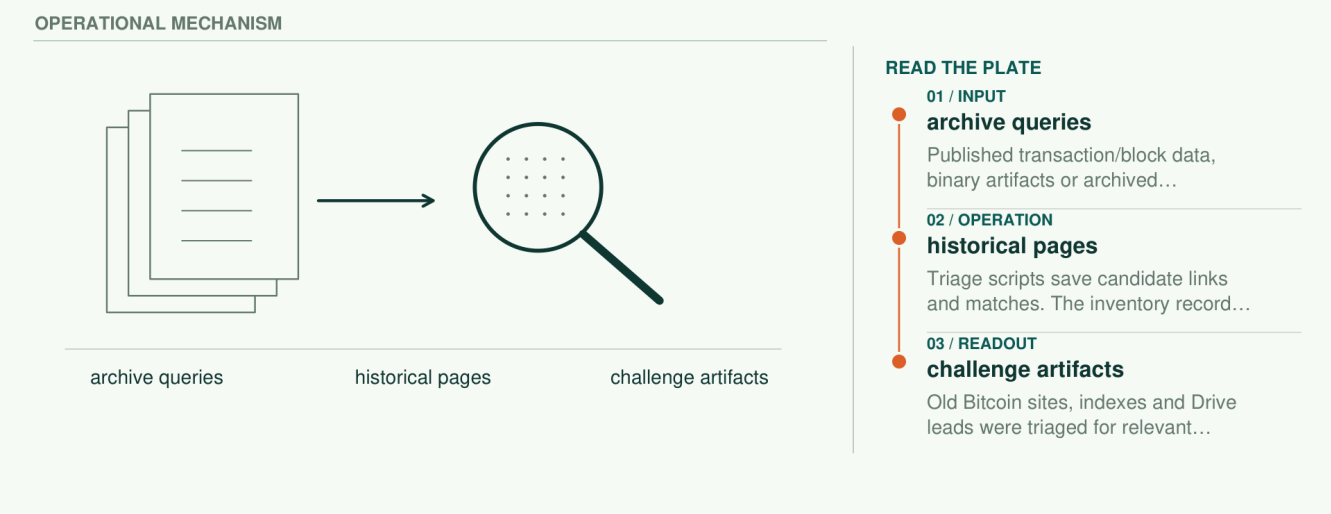
SECTION 12 / ON-CHAIN, METADATA AND CLUE RELATIONSHIPS

Historical website/archive and directory searches

M177

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

Search historical sites and directory/archive listings for original challenge files, creator software or missing public parameters.

INPUTS NEEDED

Published transaction/block data, binary artifacts or archived messages and a fixed target comparison. External facts need separate authentication.

WHAT WAS ACTUALLY DONE

Triage scripts save candidate links and matches. The inventory records those searches without treating every archived webpage as authenticated creator provenance or a successful lead.

WHAT THE RESULT MEANS

Old Bitcoin sites, indexes and Drive leads were triaged for relevant artifacts.

MATHEMATICAL FORMULATION

$$\mathcal{A} = cup_{q \in \mathcal{Q}} \mathrm{Results}(q)$$

A finite query catalog collects candidate public artifacts for source triage. Search-index coverage and relevance require separate assessment.

EVIDENCE S172 / S165 / S161

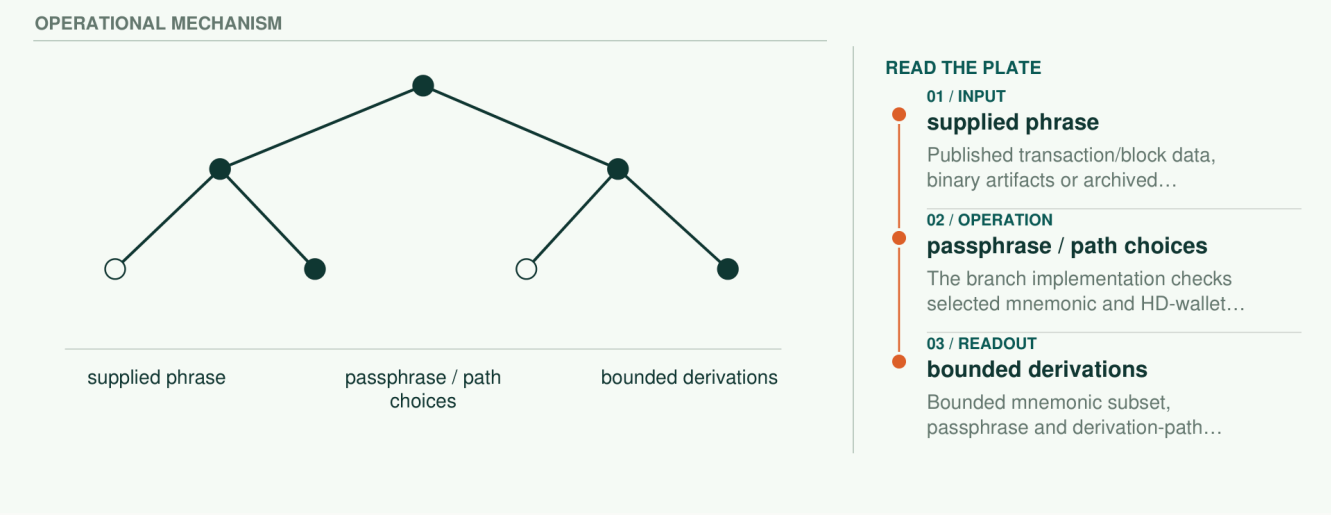
SECTION 12 / ON-CHAIN, METADATA AND CLUE RELATIONSHIPS

Time-Will-Tell phrase/path hypotheses

M178

EVIDENCE STATUS IMPLEMENTED/BRANCH RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

Try bounded interpretations of an explicitly supplied recovery phrase, including word-subset and passphrase/path choices.

INPUTS NEEDED

Published transaction/block data, binary artifacts or archived messages and a fixed target comparison. External facts need separate authentication.

WHAT WAS ACTUALLY DONE

The branch implementation checks selected mnemonic and HD-wallet variants against named targets. Private phrases are omitted from the report, and unrun full search coverage is not inferred.

WHAT THE RESULT MEANS

Bounded mnemonic subset, passphrase and derivation-path tests are documented; private phrases are excluded from this report.

MATHEMATICAL FORMULATION

$$k_p = \mathrm{Derive}(\mathrm{Mnemonic}, \mathrm{Passphrase}, p)$$

The phrase hypothesis is a bounded derivation-path experiment. Sensitive phrase material stays outside the report; successful derivation still requires an exact target match.

EVIDENCE S217 / S253

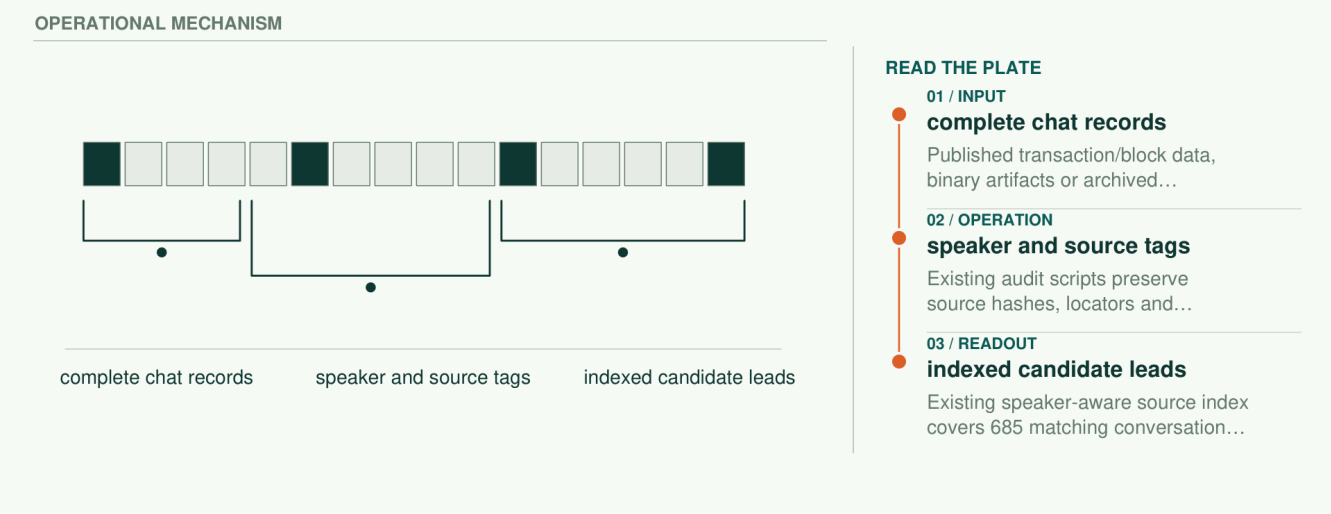
SECTION 12 / ON-CHAIN, METADATA AND CLUE RELATIONSHIPS

Complete chat-lead indexing and candidate extraction

M179

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

Index complete available user/assistant messages, distinguish speakers and extract only relevant candidate claims or supplied leads.

INPUTS NEEDED

Published transaction/block data, binary artifacts or archived messages and a fixed target comparison. External facts need separate authentication.

WHAT WAS ACTUALLY DONE

Existing audit scripts preserve source hashes, locators and redacted witnesses. Chat assertions can guide tests, but a model's claim of running code is not substituted for execution artifacts.

WHAT THE RESULT MEANS

Existing speaker-aware source index covers 685 matching conversations; assistant assertions are not execution evidence.

MATHEMATICAL FORMULATION

$$\mathcal{L} = \{(\text{speaker}, \text{source}, \text{candidate})\}$$

Index extracted leads with speaker and source provenance. A statement in an assistant message is not itself a completed computation.

EVIDENCE S068 / S075

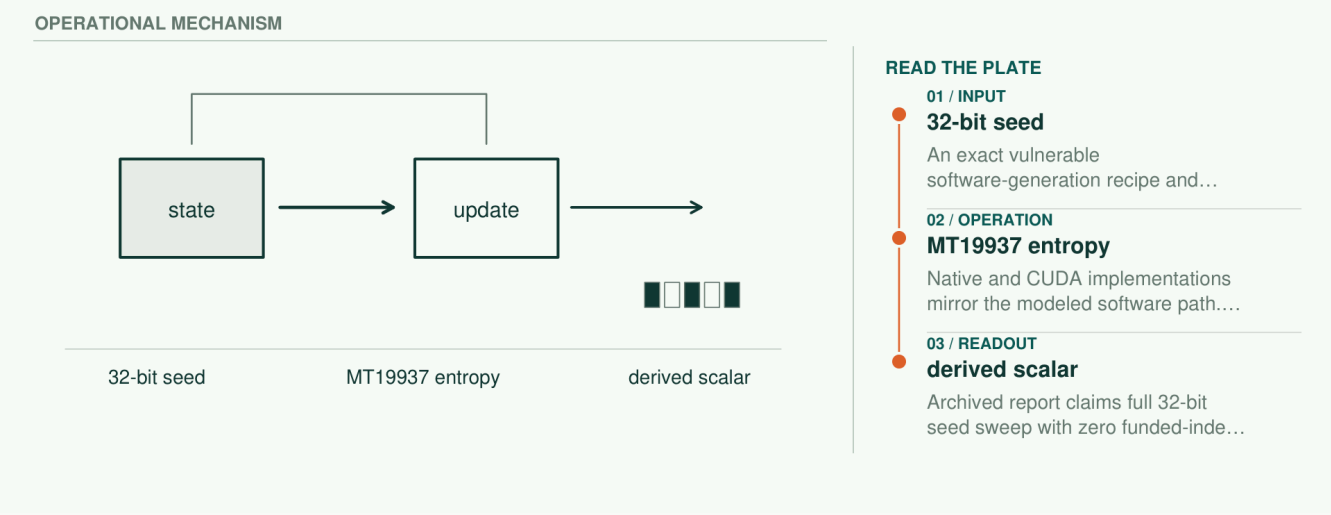
SECTION 13 / WEAK-ENTROPY WALLET AND PRNG RESEARCH

Milk Sad raw MT19937 seed expansion

M180

EVIDENCE STATUS RECORDED GPU SWEEP

FIGURE / METHOD MECHANISM



HOW IT WORKS

Reproduce the exact weak MT19937 entropy stream produced from a 32-bit libbitcoin seed, then derive the corresponding scalar/public point.

INPUTS NEEDED

An exact vulnerable software-generation recipe and bounded seed/state assumptions; a dated target/index for replay verification.

WHAT WAS ACTUALLY DONE

Native and CUDA implementations mirror the modeled software path. The saved raw-entropy sweep compares supported address hashes with a dated funded index, rather than attacking arbitrary secure seeds.

WHAT THE RESULT MEANS

Archived report claims full 32-bit seed sweep with zero funded-index hits; this inventory does not rerun or independently certify that sweep.

MATHEMATICAL FORMULATION

$$e = \mathrm{MT19937}(s)_{0:256}, \quad s \in \{0, \ldots, 2^{32} - 1\}$$

The precise libbitcoin output extraction/order defines the 256 entropy bits e. A full-length output retains at most the enumerable seed uncertainty.

EVIDENCE S170 / S249

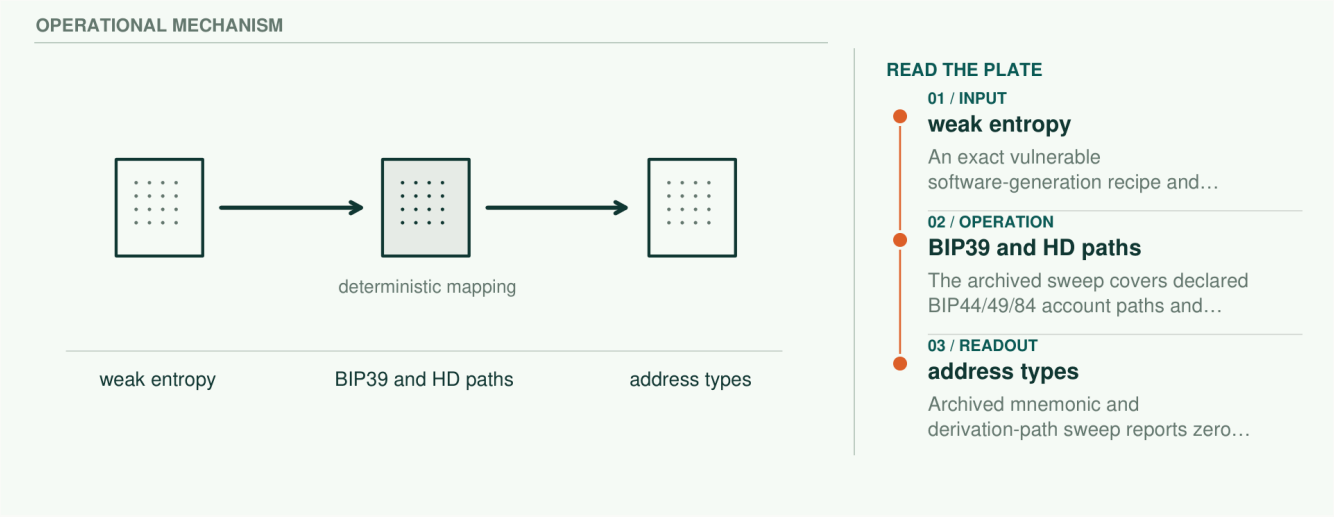
SECTION 13 / WEAK-ENTROPY WALLET AND PRNG RESEARCH

Milk Sad BIP39/BIP44/49/84 derivations

M181

EVIDENCE STATUS RECORDED GPU SWEEP

FIGURE / METHOD MECHANISM



HOW IT WORKS

Turn the weak entropy into a BIP39 mnemonic seed, derive specified HD paths and compare supported address types.

INPUTS NEEDED

An exact vulnerable software-generation recipe and bounded seed/state assumptions; a dated target/index for replay verification.

WHAT WAS ACTUALLY DONE

The archived sweep covers declared BIP44/49/84 account paths and serializations. PBKDF2 and child derivation make this substantially different from checking the raw entropy as a scalar.

WHAT THE RESULT MEANS

Archived mnemonic and derivation-path sweep reports zero funded-index hits; bounded historical index and path coverage only.

MATHEMATICAL FORMULATION

$$\text{seed} = \text{PBKDF2}_{\text{HMAC-SHA512}}(m, \text{salt}, 2048)$$

BIP39 mnemonic m comes from the modeled weak entropy; the salt includes the declared passphrase. Then derive the specified BIP44/49/84 paths and address encodings.

EVIDENCE S170 / S250

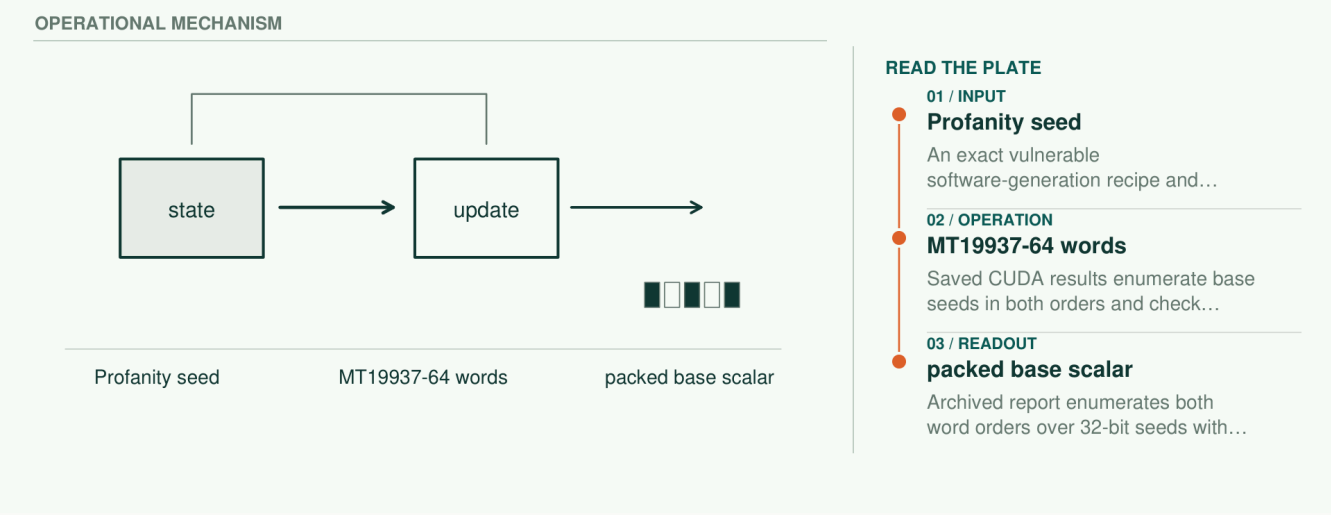
SECTION 13 / WEAK-ENTROPY WALLET AND PRNG RESEARCH

Profanity MT19937-64 base-key replay

M182

EVIDENCE STATUS RECORDED GPU SWEEP

FIGURE / METHOD MECHANISM



HOW IT WORKS

Replay Profanity's small seed expansion to reconstruct its base scalar under a specified output word order.

INPUTS NEEDED

An exact vulnerable software-generation recipe and bounded seed/state assumptions; a dated target/index for replay verification.

WHAT WAS ACTUALLY DONE

Saved CUDA results enumerate base seeds in both orders and check three hash variants. Offset-zero coverage does not automatically cover keys reached by later vanity-search increments.

WHAT THE RESULT MEANS

Archived report enumerates both word orders over 32-bit seeds with zero funded-index hits at offset zero.

MATHEMATICAL FORMULATION

$$k_0 = \mathrm{Pack}(\mathrm{MT19937}{-}64(s), \mathrm{wordOrder})$$

Reproduce Profanity's exact base-key expansion for each candidate seed and output order. This equation concerns offset zero.

EVIDENCE S175

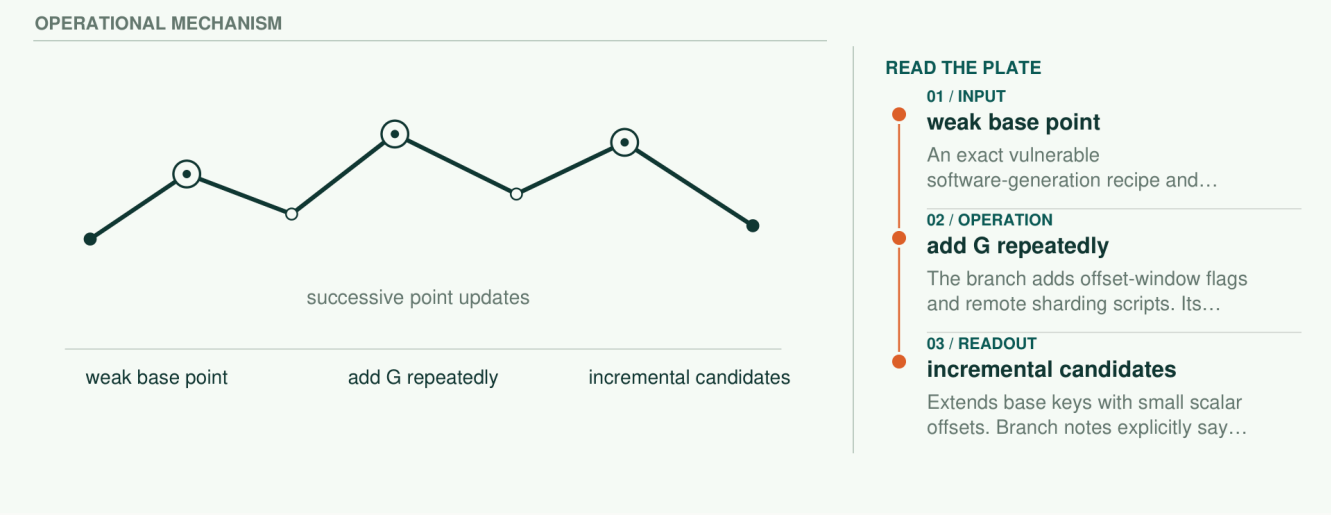
SECTION 13 / WEAK-ENTROPY WALLET AND PRNG RESEARCH

Profanity incremental offset grinding

M183

EVIDENCE STATUS BRANCH/IMPLEMENTATION

FIGURE / METHOD MECHANISM



HOW IT WORKS

Starting from a reconstructed weak base key, add small scalar offsets and reuse point additions to check nearby vanity-search states efficiently.

INPUTS NEEDED

An exact vulnerable software-generation recipe and bounded seed/state assumptions; a dated target/index for replay verification.

WHAT WAS ACTUALLY DONE

The branch adds offset-window flags and remote sharding scripts. Its documentation explicitly separates Python-side tests from real-GPU compile/verification that was still outstanding there.

WHAT THE RESULT MEANS

Extends base keys with small scalar offsets. Branch notes explicitly say real-GPU compile/verification was not yet done there.

MATHEMATICAL FORMULATION

$$k_t = k_0 + t \pmod{n}, \quad Q_{t+1} = Q_t + G$$

Search declared incremental vanity offsets from each modeled weak base. Covered offsets and seed intervals must both be reported.

EVIDENCE S252

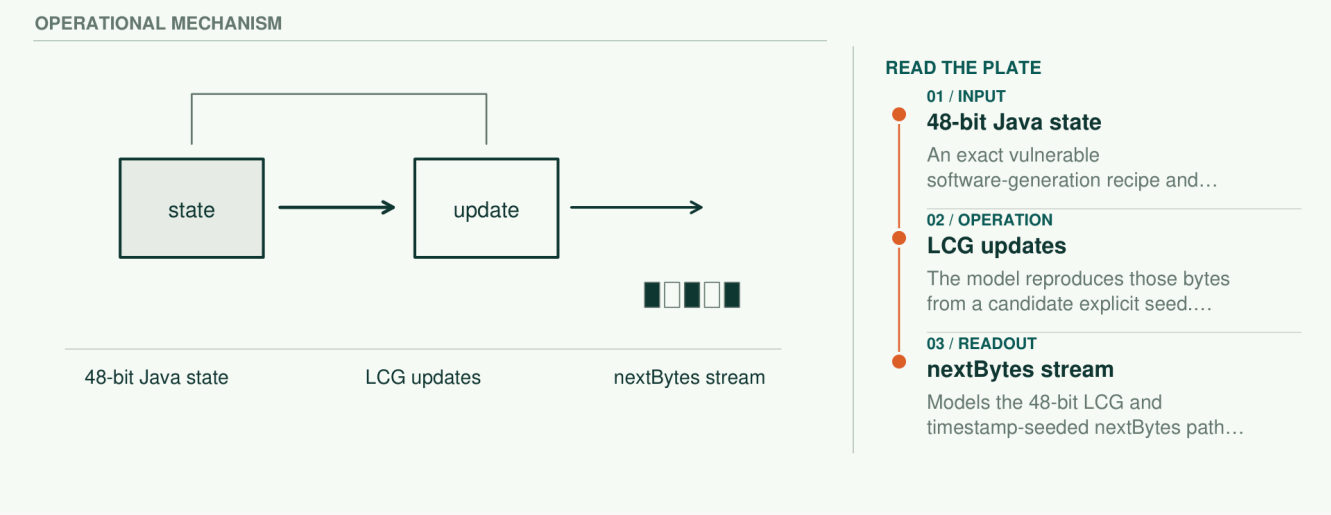
SECTION 13 / WEAK-ENTROPY WALLET AND PRNG RESEARCH

Java util.Random replay

M184

EVIDENCE STATUS IMPLEMENTED/REGISTRY

FIGURE / METHOD MECHANISM



HOW IT WORKS

Java Random updates a 48-bit linear-congruential state; nextBytes exposes a particular sequence of output bytes.

INPUTS NEEDED

An exact vulnerable software-generation recipe and bounded seed/state assumptions; a dated target/index for replay verification.

WHAT WAS ACTUALLY DONE

The model reproduces those bytes from a candidate explicit seed. Timestamp-window enumeration tests only programs that really seeded and consumed that generator in the modeled way.

WHAT THE RESULT MEANS

Models the 48-bit LCG and timestamp-seeded nextBytes path; registered runtime vectors distinguish model fidelity from a recovery.

MATHEMATICAL FORMULATION

$$x_{t+1} = (25214903917 x_t + 11) \bmod 2^{48}$$

Java util.Random's state transition is deterministic; reproduce seed scrambling, output-bit extraction and nextBytes order to model a key.

EVIDENCE S248

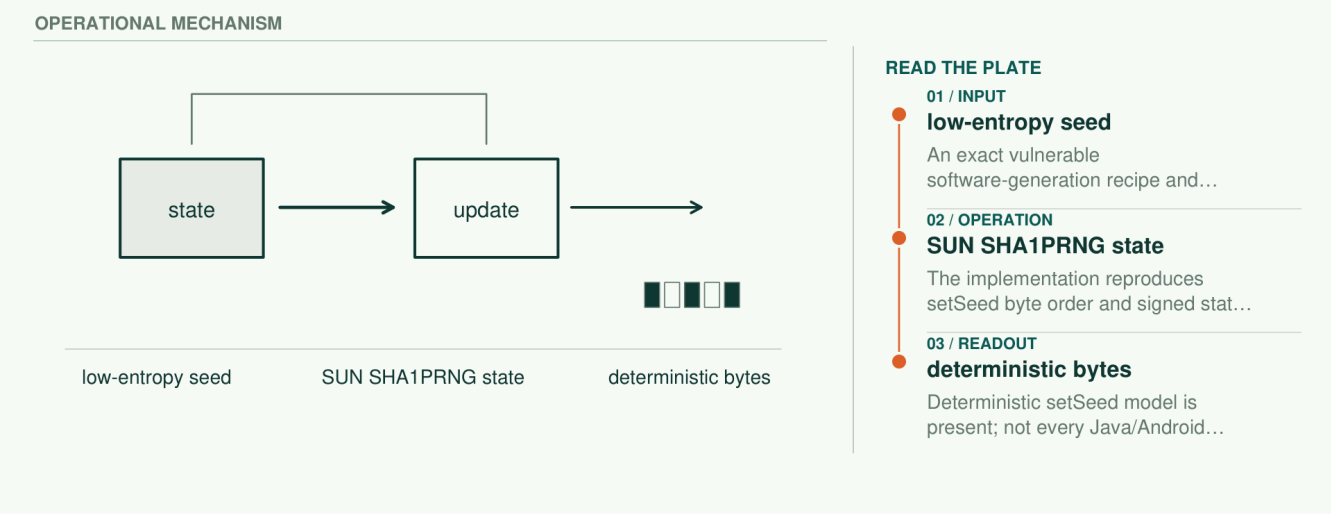
SECTION 13 / WEAK-ENTROPY WALLET AND PRNG RESEARCH

Java SUN SHA1PRNG low-entropy seed replay

M185

EVIDENCE STATUS IMPLEMENTED/REGISTRY

FIGURE / METHOD MECHANISM



HOW IT WORKS

SUN SHA1PRNG with an explicit low-entropy seed evolves deterministic SHA-1-derived state and emits predictable bytes.

INPUTS NEEDED

An exact vulnerable software-generation recipe and bounded seed/state assumptions; a dated target/index for replay verification.

WHAT WAS ACTUALLY DONE

The implementation reproduces setSeed byte order and signed state updates. It does not assume every SecureRandom call is weak; normal unobserved platform entropy changes the problem.

WHAT THE RESULT MEANS

Deterministic setSeed model is present; not every Java/Android wallet follows that path.

MATHEMATICAL FORMULATION

$$(S_{t+1}, o_t) = \mathrm{SUN\text{-}SHA1PRNG}(S_t)$$

This denotes the implementation-specific signed state update and emitted bytes after explicit seed initialization. It is not a generic claim that all SHA-1 PRNG seeding is enumerable.

EVIDENCE S248

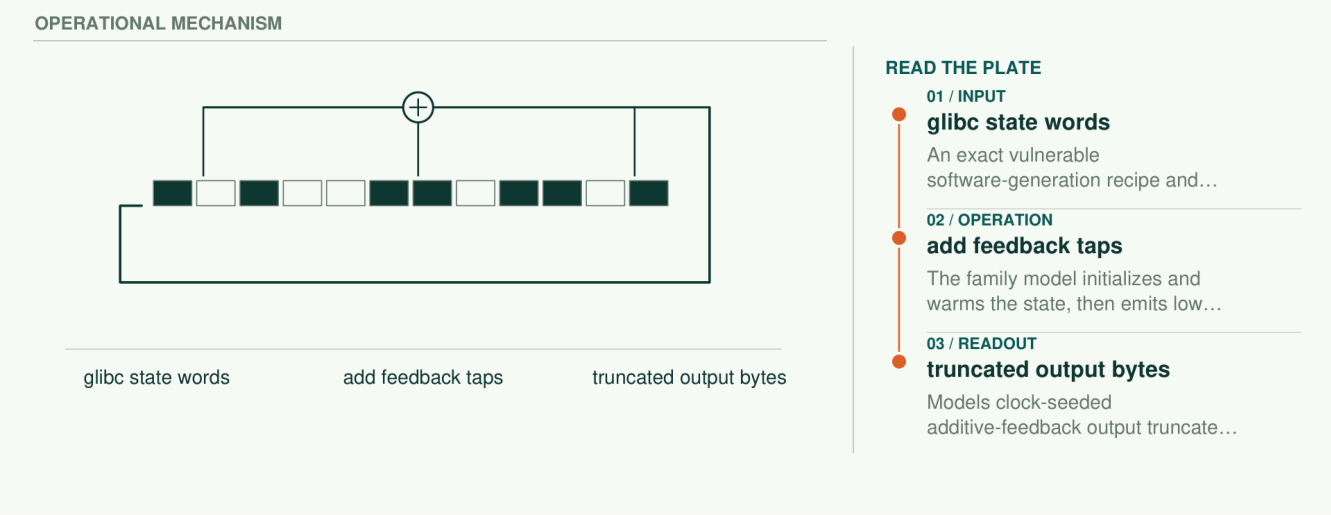
SECTION 13 / WEAK-ENTROPY WALLET AND PRNG RESEARCH

glibc rand/random byte-fill replay

M186

EVIDENCE STATUS IMPLEMENTED/REGISTRY

FIGURE / METHOD MECHANISM



HOW IT WORKS

glibc rand/random uses additive-feedback state; truncating its outputs to bytes can make keys reproducible from a small seed.

INPUTS NEEDED

An exact vulnerable software-generation recipe and bounded seed/state assumptions; a dated target/index for replay verification.

WHAT WAS ACTUALLY DONE

The family model initializes and warms the state, then emits low bytes under the catalog's byte-fill recipe. Generator fidelity is tested separately from applicability to a target wallet.

WHAT THE RESULT MEANS

Models clock-seeded additive-feedback output truncated into key bytes.

MATHEMATICAL FORMULATION

$$x_t = x_{t-31} + x_{t-3} \pmod{2^{32}}, \quad o_t = x_t \gg 1$$

This describes the modeled default glibc additive-feedback stream after its prescribed initialization/warm-up; the key byte-fill rule must also match.

EVIDENCE S248

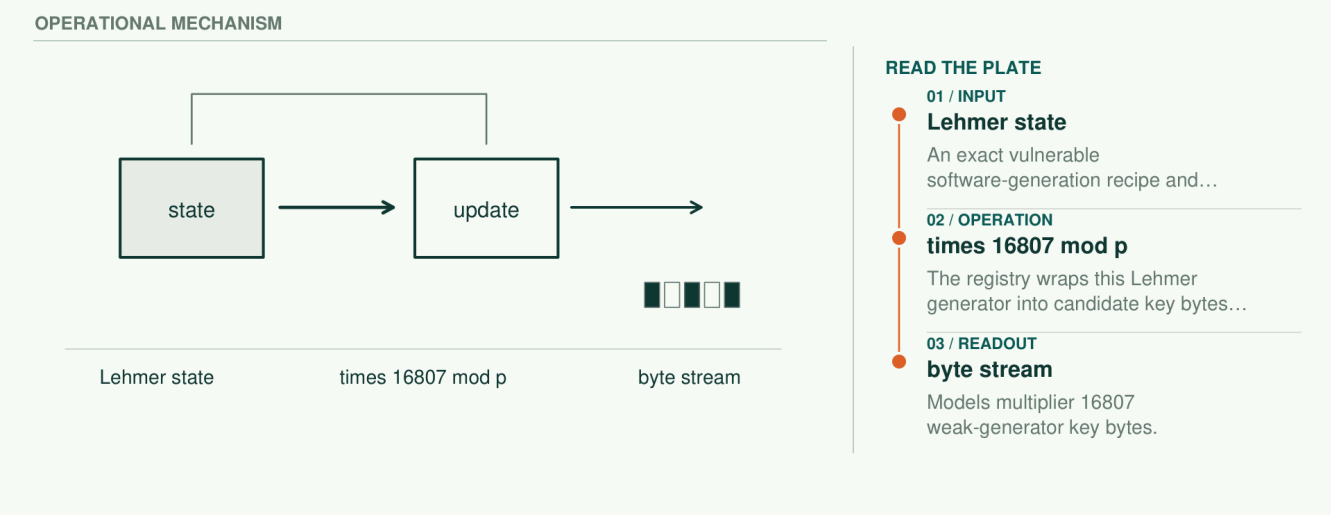
SECTION 13 / WEAK-ENTROPY WALLET AND PRNG RESEARCH

Lehmer minstd_rand0 replay

M187

EVIDENCE STATUS IMPLEMENTED/REGISTRY

FIGURE / METHOD MECHANISM



HOW IT WORKS

minstd_rand0 multiplies state by 16807 modulo 2^31-1. An enumerable state determines the byte stream.

INPUTS NEEDED

An exact vulnerable software-generation recipe and bounded seed/state assumptions; a dated target/index for replay verification.

WHAT WAS ACTUALLY DONE

The registry wraps this Lehmer generator into candidate key bytes and runtime vectors. The attack premise is misuse for wallet entropy, not a weakness of secp256k1 itself.

WHAT THE RESULT MEANS

Models multiplier 16807 weak-generator key bytes.

MATHEMATICAL FORMULATION

$$x_{t+1} = 16807 x_t \pmod{2^{31} - 1}$$

minstd_rand0 is a Lehmer generator. Specify seed normalization and byte extraction in addition to the recurrence.

EVIDENCE S248

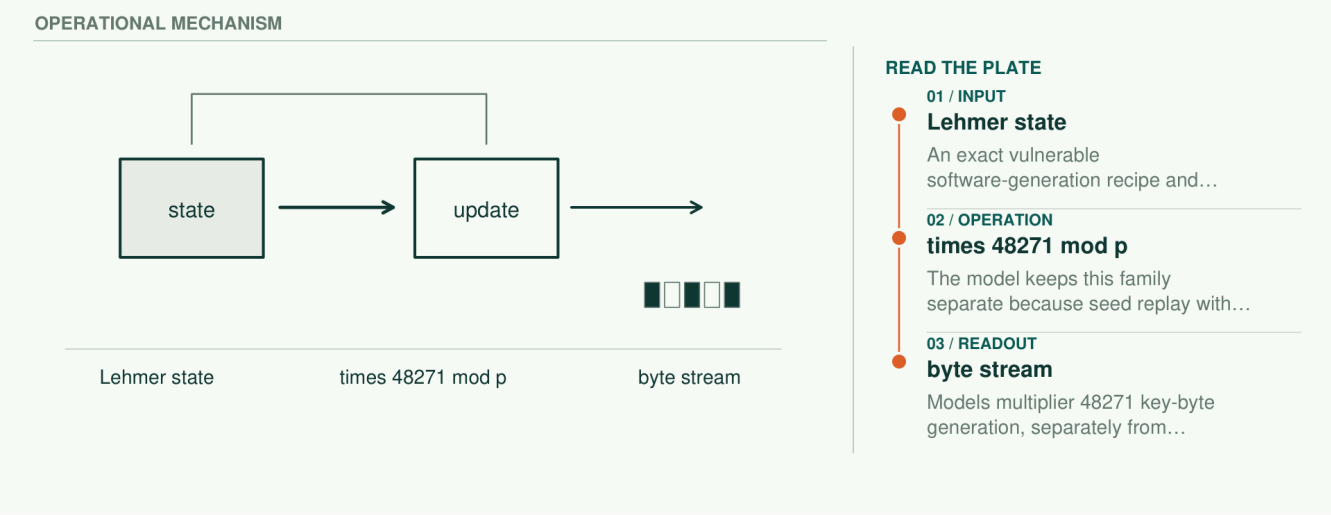
SECTION 13 / WEAK-ENTROPY WALLET AND PRNG RESEARCH

Lehmer minstd_rand replay

M188

EVIDENCE STATUS IMPLEMENTED/REGISTRY

FIGURE / METHOD MECHANISM



HOW IT WORKS

minstd_rand uses multiplier 48271 with the same prime modulus, producing a different deterministic stream.

INPUTS NEEDED

An exact vulnerable software-generation recipe and bounded seed/state assumptions; a dated target/index for replay verification.

WHAT WAS ACTUALLY DONE

The model keeps this family separate because seed replay with the wrong multiplier produces different keys. A sweep must identify the exact generator and byte extraction recipe it covers.

WHAT THE RESULT MEANS

Models multiplier 48271 key-byte generation, separately from minstd_rand0.

MATHEMATICAL FORMULATION

$$x_{t+1} = 48271 x_t \pmod{2^{31} - 1}$$

minstd_rand uses a different multiplier from minstd_rand0. Their key streams are not interchangeable even with the same nominal seed.

EVIDENCE S248

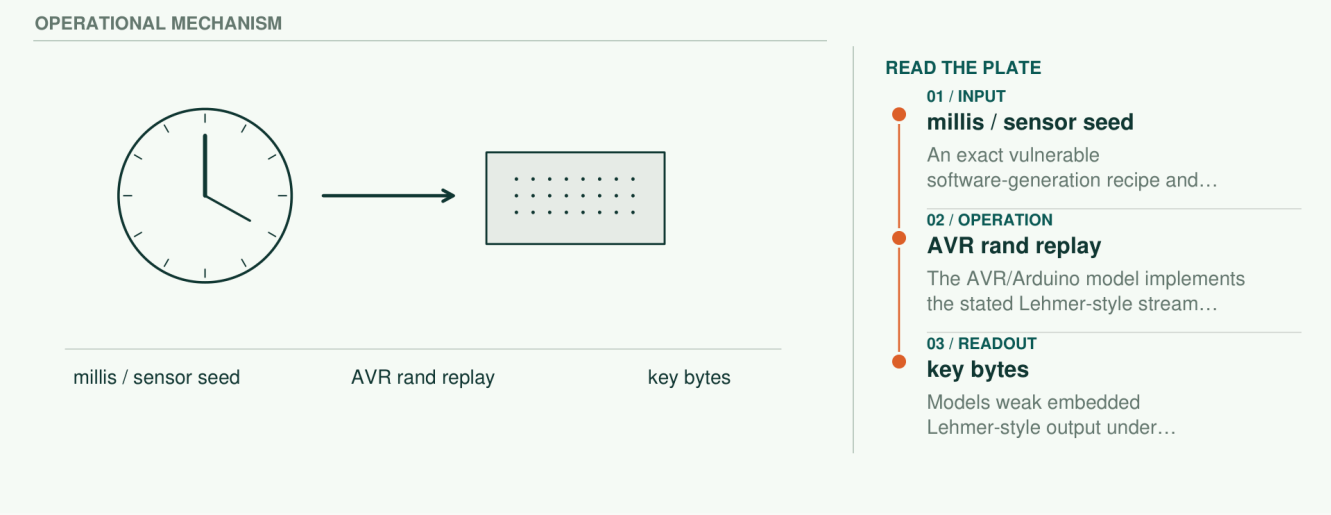
SECTION 13 / WEAK-ENTROPY WALLET AND PRNG RESEARCH

AVR/Arduino rand replay

M189

EVIDENCE STATUS IMPLEMENTED/REGISTRY

FIGURE / METHOD MECHANISM



HOW IT WORKS

Embedded sketches may derive key bytes from a deterministic rand stream seeded by millis or limited sensor readings.

INPUTS NEEDED

An exact vulnerable software-generation recipe and bounded seed/state assumptions; a dated target/index for replay verification.

WHAT WAS ACTUALLY DONE

The AVR/Arduino model implements the stated Lehmer-style stream under candidate seeds. It does not model unknown hardware noise measurements or every embedded random library.

WHAT THE RESULT MEANS

Models weak embedded Lehmer-style output under enumerable seeds.

MATHEMATICAL FORMULATION

$$k = \mathrm{ByteFill}(\mathrm{PRNG}_{\mathrm{AVR}}(s))$$

The embedded model depends on the exact library, seed range and rand-to-byte recipe. A timer or sensor hypothesis must specify those inputs.

EVIDENCE S248

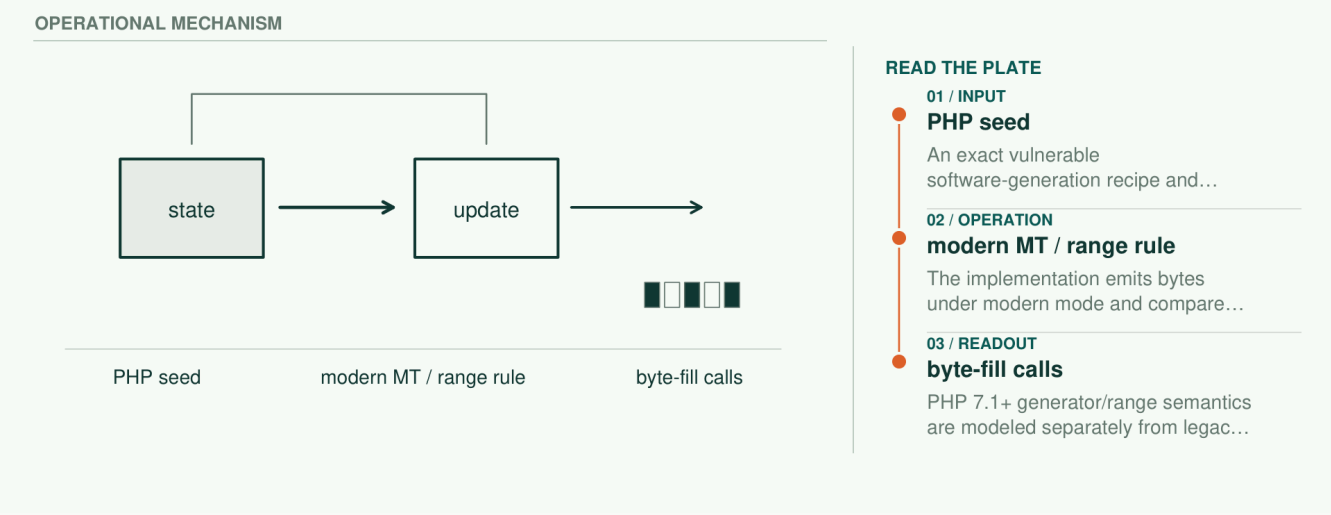
SECTION 13 / WEAK-ENTROPY WALLET AND PRNG RESEARCH

Modern PHP MT19937 byte-fill replay

M190

EVIDENCE STATUS IMPLEMENTED/REGISTRY

FIGURE / METHOD MECHANISM



HOW IT WORKS

Modern PHP mt_rand uses MT19937 and particular range-output rules. Repeated byte requests determine a key from its seed.

INPUTS NEEDED

An exact vulnerable software-generation recipe and bounded seed/state assumptions; a dated target/index for replay verification.

WHAT WAS ACTUALLY DONE

The implementation emits bytes under modern mode and compares with runtime vectors. A generic MT19937 sequence with different range scaling is not the same candidate family.

WHAT THE RESULT MEANS

PHP 7.1+ generator/range semantics are modeled separately from legacy PHP.

MATHEMATICAL FORMULATION

$$k_j = \mathrm{PHP}\text{–}\,\mathrm{range}(\mathrm{MT}_{\mathrm{modern}}(s), 0, 255)_j$$

Each byte request consumes state according to the modern PHP range-sampling behavior. Generic MT words without that rule define a different family.

EVIDENCE S248

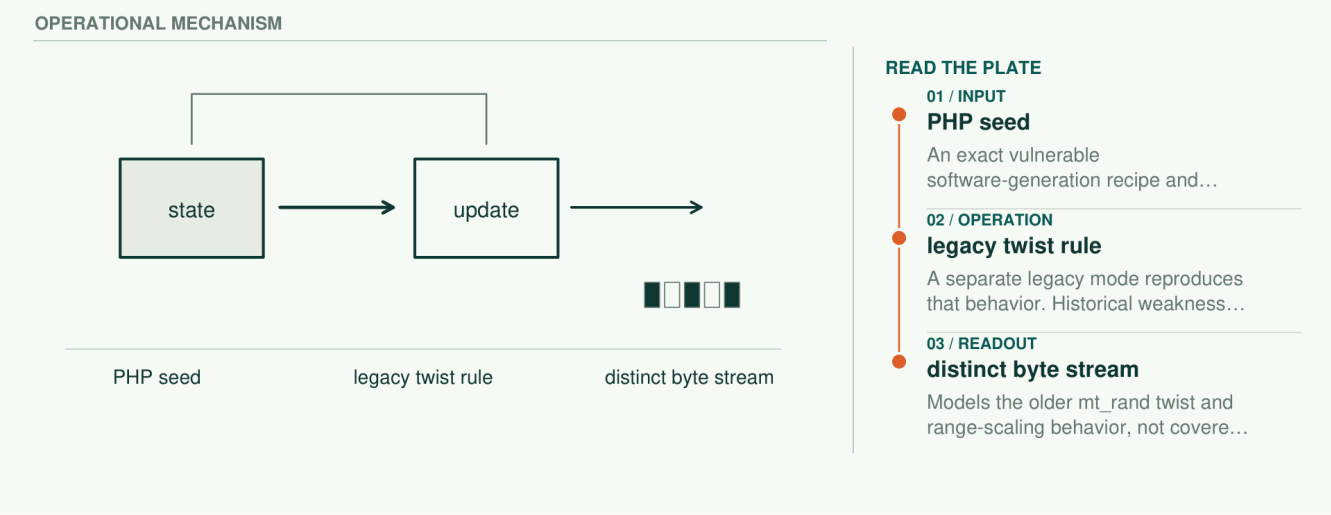
SECTION 13 / WEAK-ENTROPY WALLET AND PRNG RESEARCH

Legacy PHP broken-twist replay

M191

EVIDENCE STATUS IMPLEMENTED/REGISTRY

FIGURE / METHOD MECHANISM



HOW IT WORKS

Older PHP used a different twist/range behavior, so a small seed produces a distinct key stream from modern PHP.

INPUTS NEEDED

An exact vulnerable software-generation recipe and bounded seed/state assumptions; a dated target/index for replay verification.

WHAT WAS ACTUALLY DONE

A separate legacy mode reproduces that behavior. Historical weakness labels alone are insufficient without matching the actual PHP version and key-generation recipe.

WHAT THE RESULT MEANS

Models the older mt_rand twist and range-scaling behavior, not covered by ordinary MT19937 assumptions.

MATHEMATICAL FORMULATION

$$k_j = \mathrm{PHP}\text{–}\,\mathrm{range}(\mathrm{MT}_{\mathrm{legacy}}(s), 0, 255)_j$$

The historical broken-twist mode and its range scaling need version-matched replay. Seed enumeration covers only that specified mode.

EVIDENCE S248

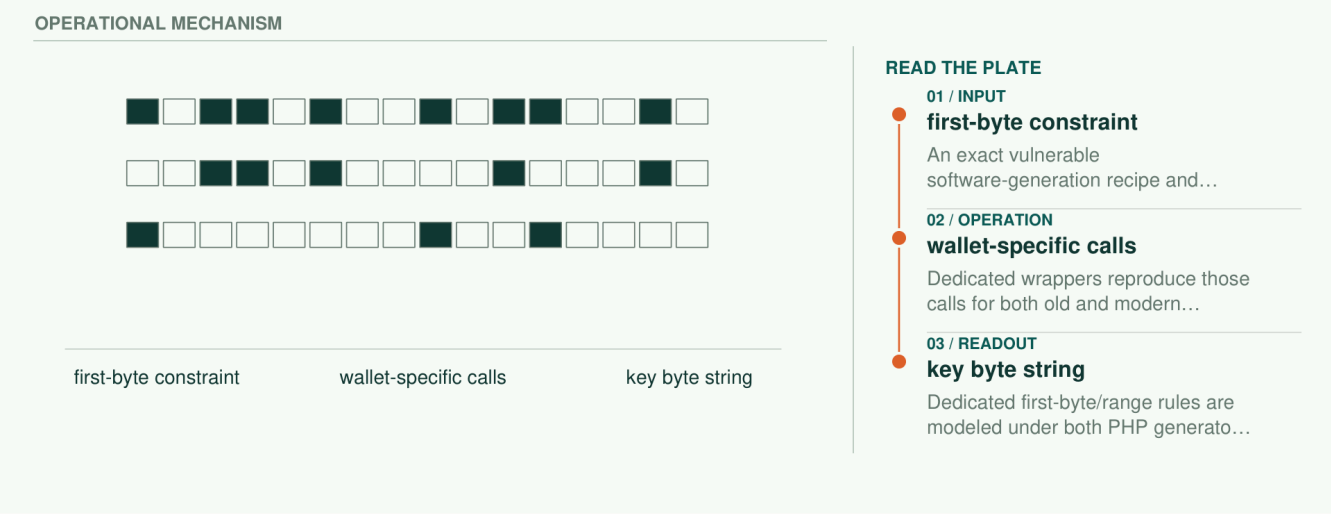
SECTION 13 / WEAK-ENTROPY WALLET AND PRNG RESEARCH

PHPCoinAddress legacy and modern keygen

M192

EVIDENCE STATUS IMPLEMENTED/REGISTRY

FIGURE / METHOD MECHANISM



HOW IT WORKS

PHPCoinAddress applies its own bounded byte calls, including a special first-byte range, around PHP's generator.

INPUTS NEEDED

An exact vulnerable software-generation recipe and bounded seed/state assumptions; a dated target/index for replay verification.

WHAT WAS ACTUALLY DONE

Dedicated wrappers reproduce those calls for both old and modern modes. Replaying plain mt_rand bytes would omit this wallet-specific transformation and could miss its generated keys.

WHAT THE RESULT MEANS

Dedicated first-byte/range rules are modeled under both PHP generator modes.

MATHEMATICAL FORMULATION

$$k_0 \in A_{\text{first}}, \quad k_j = \text{WalletByteCall}(s, j)$$

PHPCoinAddress has wallet-specific first-byte bounds and subsequent calls. Reproduce those calls using the declared legacy or modern PHP mode.

EVIDENCE S248

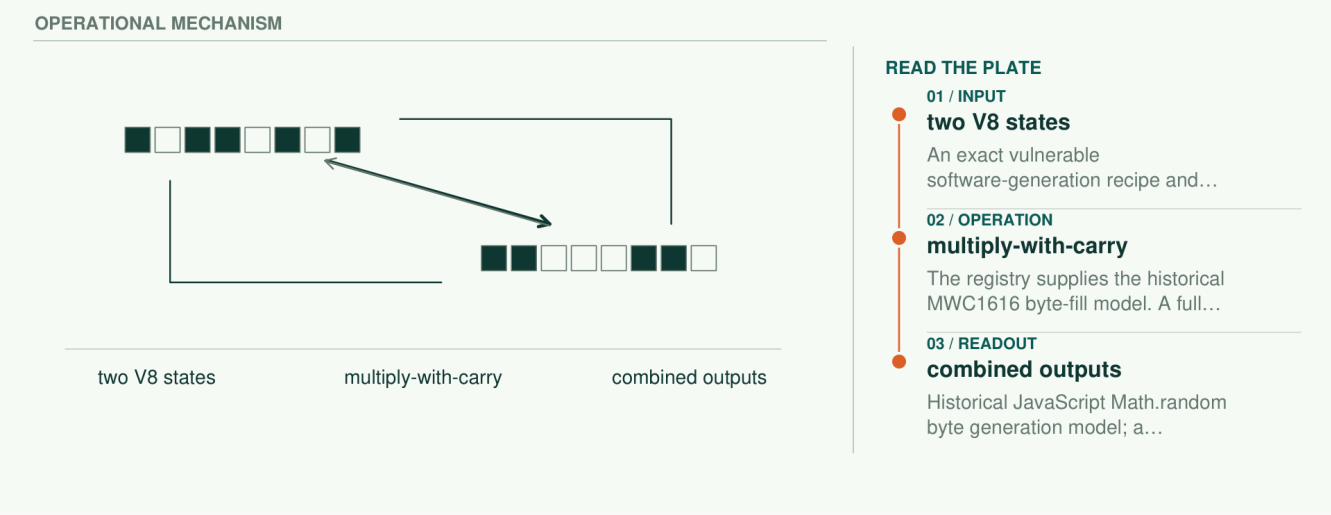
SECTION 13 / WEAK-ENTROPY WALLET AND PRNG RESEARCH

V8 MWC1616 / Randstorm-era byte fill

M193

EVIDENCE STATUS IMPLEMENTED/REGISTRY

FIGURE / METHOD MECHANISM



HOW IT WORKS

Old V8 Math.random combines multiply-with-carry state; weakly seeded outputs can be reproduced and packed into key bytes.

INPUTS NEEDED

An exact vulnerable software-generation recipe and bounded seed/state assumptions; a dated target/index for replay verification.

WHAT WAS ACTUALLY DONE

The registry supplies the historical MWC1616 byte-fill model. A full Randstorm wallet route may include additional entropy mixing and consumption order not implied by that one modeled stream.

WHAT THE RESULT MEANS

Historical JavaScript Math.random byte generation model; a vulnerable-family model is not attribution to a target wallet.

MATHEMATICAL FORMULATION

$$x' = 18030(x \text{ AND } 65535) + (x \gg 16)$$

$$y' = 30903(y \text{ AND } 65535) + (y \gg 16)$$

These are the historical V8 MWC1616 state updates with unsigned word arithmetic. The output packing, seeding and wallet entropy mixing must also be modeled.

EVIDENCE S248

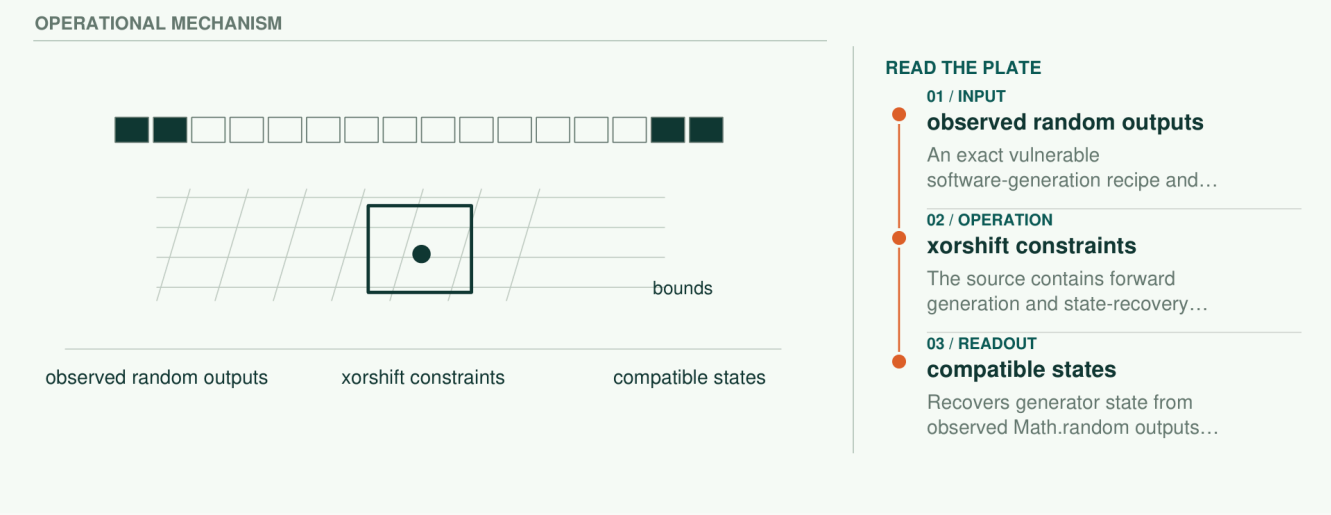
SECTION 13 / WEAK-ENTROPY WALLET AND PRNG RESEARCH

V8 xorshift128 state recovery

M194

EVIDENCE STATUS IMPLEMENTED

FIGURE / METHOD MECHANISM



HOW IT WORKS

Observed floating Math.random outputs expose some generator bits. Constraint solving can recover compatible xorshift state under the modeled engine order.

WHAT WAS ACTUALLY DONE

The source contains forward generation and state-recovery helpers. It requires actual usable observations; no such target observations are created by merely knowing a Bitcoin address.

INPUTS NEEDED

An exact vulnerable software-generation recipe and bounded seed/state assumptions; a dated target/index for replay verification.

WHAT THE RESULT MEANS

Recovers generator state from observed Math.random outputs under the modeled engine semantics; no target observations established here.

MATHEMATICAL FORMULATION

$$\mathcal{S} = \{ s : \mathrm{Obs}(F^t(s)) = y_t \text{ for all } t \}$$

Recover compatible xorshift states from actual observed floating outputs y_t under the engine's output order. Without observations, the constraints are absent.

EVIDENCE S248

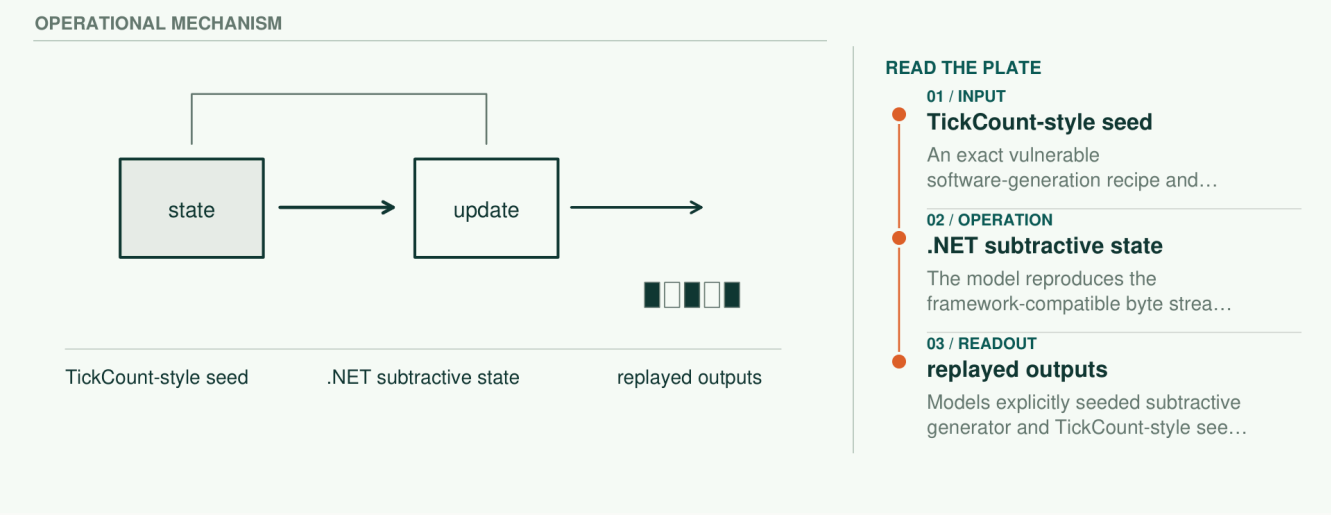
SECTION 13 / WEAK-ENTROPY WALLET AND PRNG RESEARCH

.NET System.Random replay

M195

EVIDENCE STATUS IMPLEMENTED/REGISTRY

FIGURE / METHOD MECHANISM



HOW IT WORKS

Explicitly seeded .NET System.Random evolves subtractive state; low-entropy TickCount-style seeds allow replay.

INPUTS NEEDED

An exact vulnerable software-generation recipe and bounded seed/state assumptions; a dated target/index for replay verification.

WHAT WAS ACTUALLY DONE

The model reproduces the framework-compatible byte stream and supports runtime vectors. Newer runtime semantics or additional entropy require a different model.

WHAT THE RESULT MEANS

Models explicitly seeded subtractive generator and TickCount-style seed scope.

MATHEMATICAL FORMULATION

$$(S_{t+1}, o_t) = \mathrm{DotNetRandom}_{\mathrm{version}}(S_t)$$

The historical subtractive-state algorithm and explicit seed determine the byte stream. State size, byte mapping and runtime version are part of the model.

EVIDENCE S248

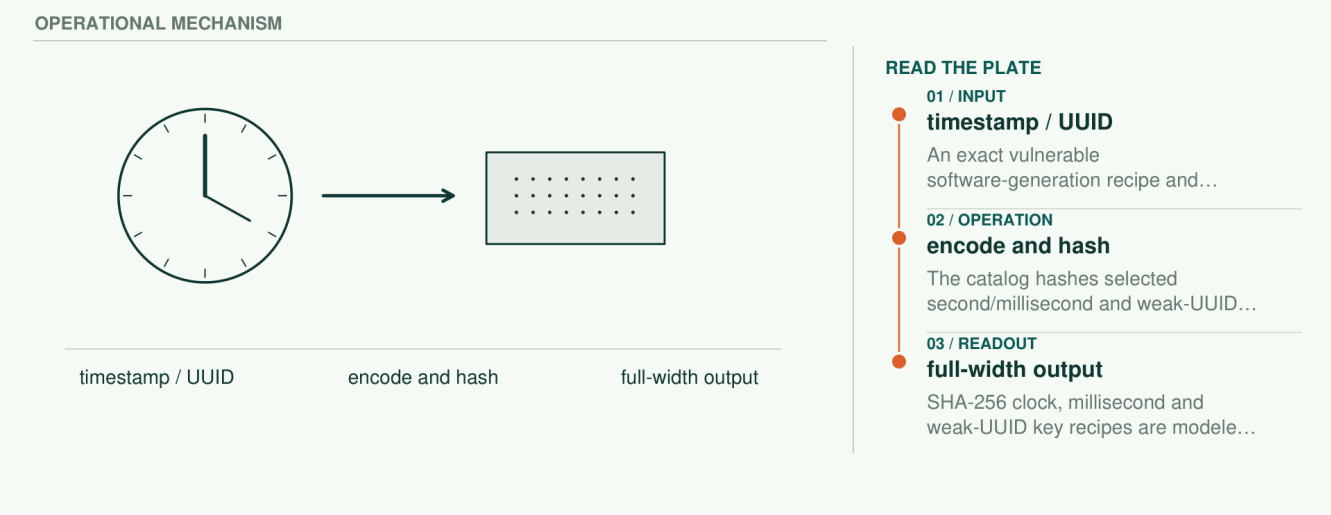
SECTION 13 / WEAK-ENTROPY WALLET AND PRNG RESEARCH

Timestamp/UUID hash-derived keys

M196

EVIDENCE STATUS IMPLEMENTED

FIGURE / METHOD MECHANISM



HOW IT WORKS

Hashing a predictable timestamp or weak UUID produces a full-length string but retains the small input uncertainty.

INPUTS NEEDED

An exact vulnerable software-generation recipe and bounded seed/state assumptions; a dated target/index for replay verification.

WHAT WAS ACTUALLY DONE

The catalog hashes selected second/millisecond and weak-UUID encodings into candidate keys. It searches their input choices, not the full 256-bit digest space.

WHAT THE RESULT MEANS

SHA-256 clock, millisecond and weak-UUID key recipes are modeled as bounded generator hypotheses.

MATHEMATICAL FORMULATION

$$k = \mathrm{Int}(H(\mathrm{enc}(t, \mathrm{UUID})))$$

Enumerate the declared low-entropy timestamp/UUID input choices. Hash output width does not enlarge the uncertainty in those inputs.

EVIDENCE S248

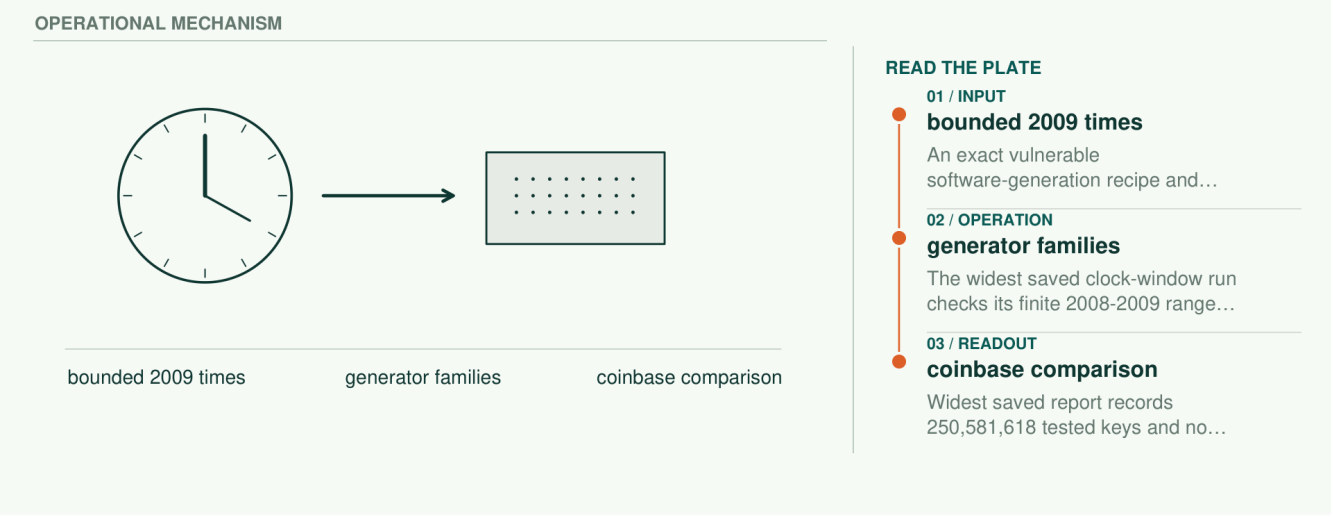
SECTION 13 / WEAK-ENTROPY WALLET AND PRNG RESEARCH

Early-2009 clock-window seed replay

M197

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

Guess creation-time values, feed them into named weak generators/encodings and compare with early coinbase public keys.

INPUTS NEEDED

An exact vulnerable software-generation recipe and bounded seed/state assumptions; a dated target/index for replay verification.

WHAT WAS ACTUALLY DONE

The widest saved clock-window run checks its finite 2008-2009 range and reports no matches. Actual OpenSSL machine-state seeding is not reproduced by these timestamp hypotheses.

WHAT THE RESULT MEANS

Widest saved report records 250,581,618 tested keys and no early-coinbase target recovery; true machine-state entropy is not enumerated.

MATHEMATICAL FORMULATION

$$\mathcal{C} = \{F_g(t) : t \in [t_0, t_1],\ g \in \mathcal{G}\}$$

A clock-window sweep combines a finite time range with named generator/encoding recipes G. It does not model arbitrary operating-system entropy.

EVIDENCE S142

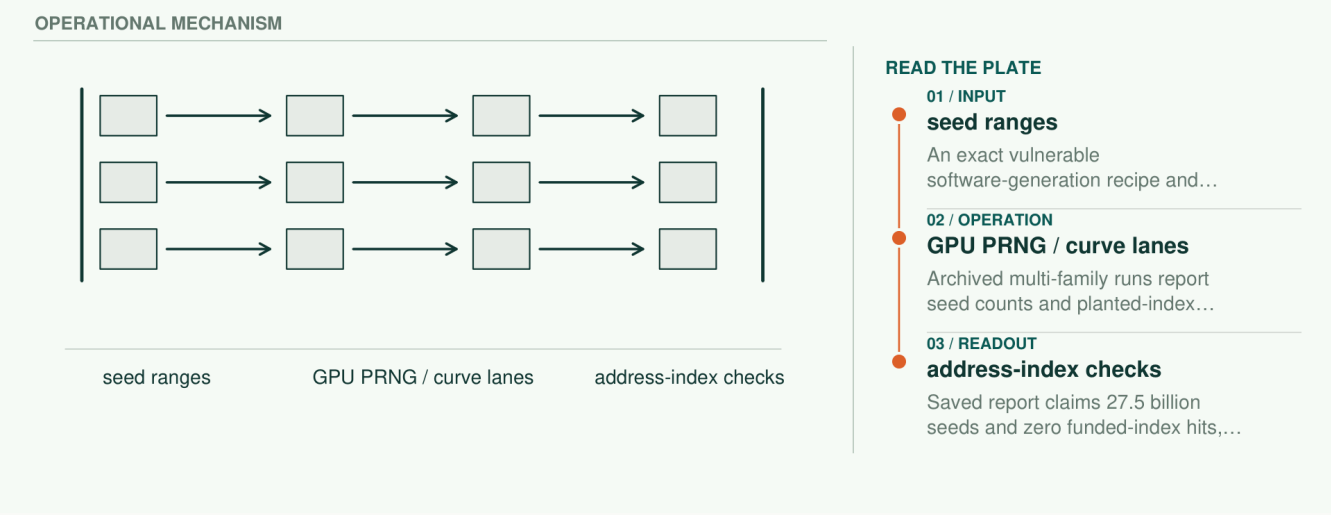
SECTION 13 / WEAK-ENTROPY WALLET AND PRNG RESEARCH

Weak-entropy multi-family CUDA sweep

M198

EVIDENCE STATUS RECORDED GPU SWEEP

FIGURE / METHOD MECHANISM



HOW IT WORKS

Parallel GPU threads map seeds through PRNG, curve multiplication and hash comparison against a supported address index.

INPUTS NEEDED

An exact vulnerable software-generation recipe and bounded seed/state assumptions; a dated target/index for replay verification.

WHAT WAS ACTUALLY DONE

Archived multi-family runs report seed counts and planted-index detection checks. Their coverage is bounded by the selected generator families, seed intervals and supported address encodings.

WHAT THE RESULT MEANS

Saved report claims 27.5 billion seeds and zero funded-index hits, with planted-index detection tests. Historical record, not rerun here.

MATHEMATICAL FORMULATION

$$\mathcal{C} = cup_{g \in \mathcal{G}} \{ F_g(s) : s \in S_g \}$$

GPU enumeration covers selected generator families and seed sets. Deduplicate candidates when reporting distinct scalar coverage.

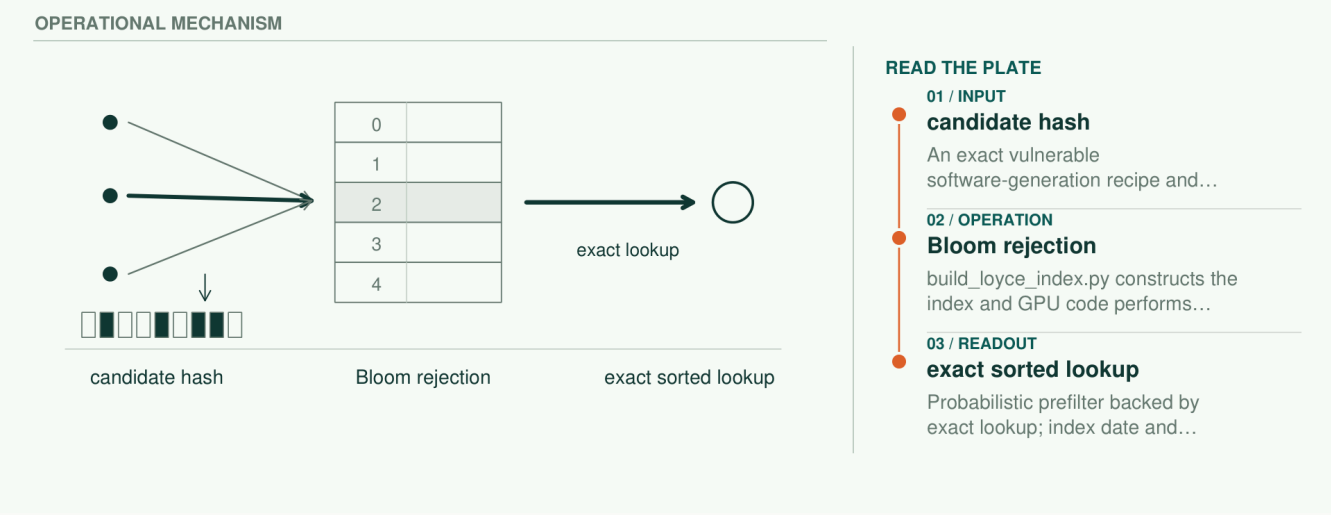
EVIDENCE S195

SECTION 13 / WEAK-ENTROPY WALLET AND PRNG RESEARCH

Bloom plus sorted funded-address index M199

EVIDENCE STATUS IMPLEMENTED

FIGURE / METHOD MECHANISM



HOW IT WORKS

A Bloom filter rapidly rejects most nonmembers; positive candidates then undergo exact lookup in a sorted hash160 index.

INPUTS NEEDED

An exact vulnerable software-generation recipe and bounded seed/state assumptions; a dated target/index for replay verification.

WHAT WAS ACTUALLY DONE

build_loyce_index.py constructs the index and GPU code performs membership checks. Bloom positives alone are not hits; the source index's date and skipped address types limit conclusions.

WHAT THE RESULT MEANS

Probabilistic prefilter backed by exact lookup; index date and supported address types bound conclusions.

MATHEMATICAL FORMULATION

$$\mathrm{hit}(h) = \mathrm{Bloom}(h) \wedge \mathrm{ExactLookup}(h)$$

Every Bloom positive needs exact confirmation. With a independent hash functions, N indexed entries and m filter bits, the ideal false-positive rate is approximately (1-exp(-aN/m))^a.

EVIDENCE S082 / S195

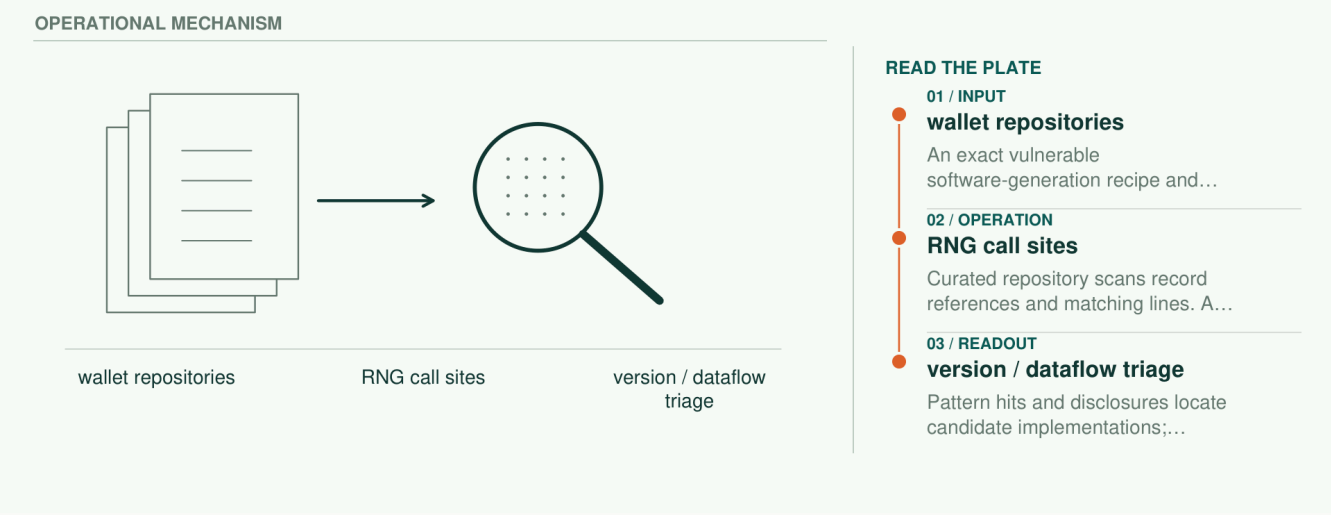
SECTION 13 / WEAK-ENTROPY WALLET AND PRNG RESEARCH

Weak-RNG repository-source triage

M200

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

Search wallet source for weak RNG calls and correlate findings with known disclosures and affected versions.

INPUTS NEEDED

An exact vulnerable software-generation recipe and bounded seed/state assumptions; a dated target/index for replay verification.

WHAT WAS ACTUALLY DONE

Curated repository scans record references and matching lines. A weak RNG in tests, an old dependency or a non-keygen path is not automatic evidence that current wallet keys are vulnerable.

WHAT THE RESULT MEANS

Pattern hits and disclosures locate candidate implementations; historical/current refs and dependency bugs must be distinguished.

MATHEMATICAL FORMULATION

$$\mathcal{F} = \{(\text{repo}, \text{version}, \text{callsite}, \text{dataflow})\}$$

A source finding matters when weak outputs flow into production key entropy. A text match alone does not establish that dataflow.

EVIDENCE S196 / S197

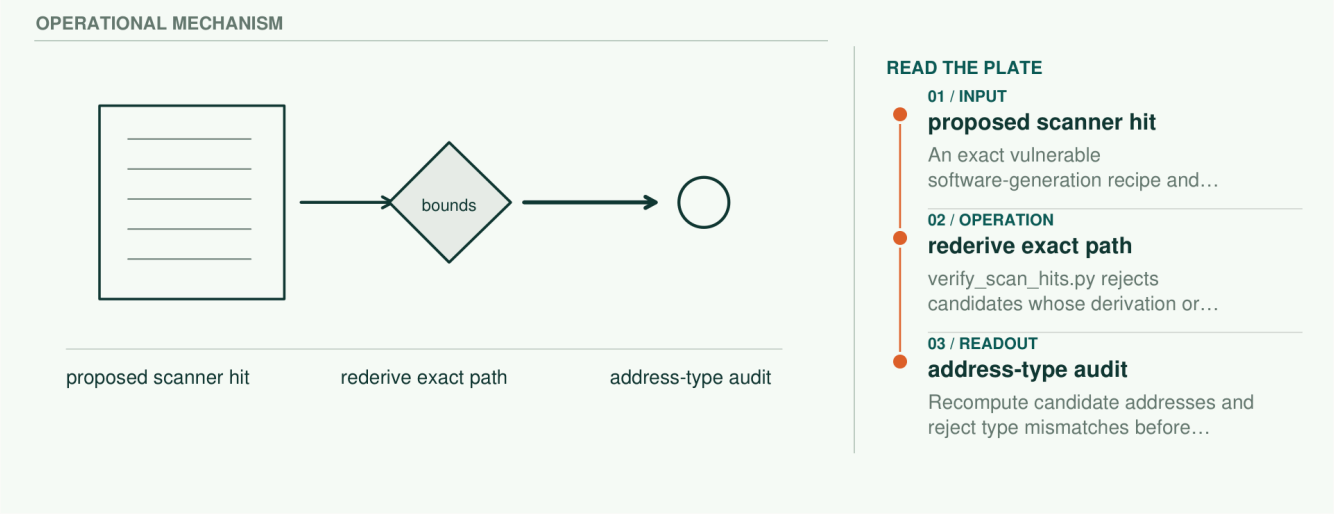
SECTION 13 / WEAK-ENTROPY WALLET AND PRNG RESEARCH

Recoverable-hit rederivation/address-type audit

M201

EVIDENCE STATUS AUDITED/CORRECTED

FIGURE / METHOD MECHANISM



HOW IT WORKS

Recompute the complete derivation and exact address type before accepting a scanner's proposed recovery.

INPUTS NEEDED

An exact vulnerable software-generation recipe and bounded seed/state assumptions; a dated target/index for replay verification.

WHAT WAS ACTUALLY DONE

verify_scan_hits.py rejects candidates whose derivation or P2PKH/P2SH/witness interpretation does not match. It corrects the earlier tendency to label raw index associations as recovered wallets.

WHAT THE RESULT MEANS

Recompute candidate addresses and reject type mismatches before reporting a recovery; prior raw hits were not all valid.

MATHEMATICAL FORMULATION

$$\text{Accept} = 1\{\text{Address}_{\text{type}}(\text{Derive}(s,p)) = A_*\}$$

Recompute seed/path derivation and the exact address type before accepting a hit. Index associations or Bloom positives alone fail this criterion.

EVIDENCE S246 / S256

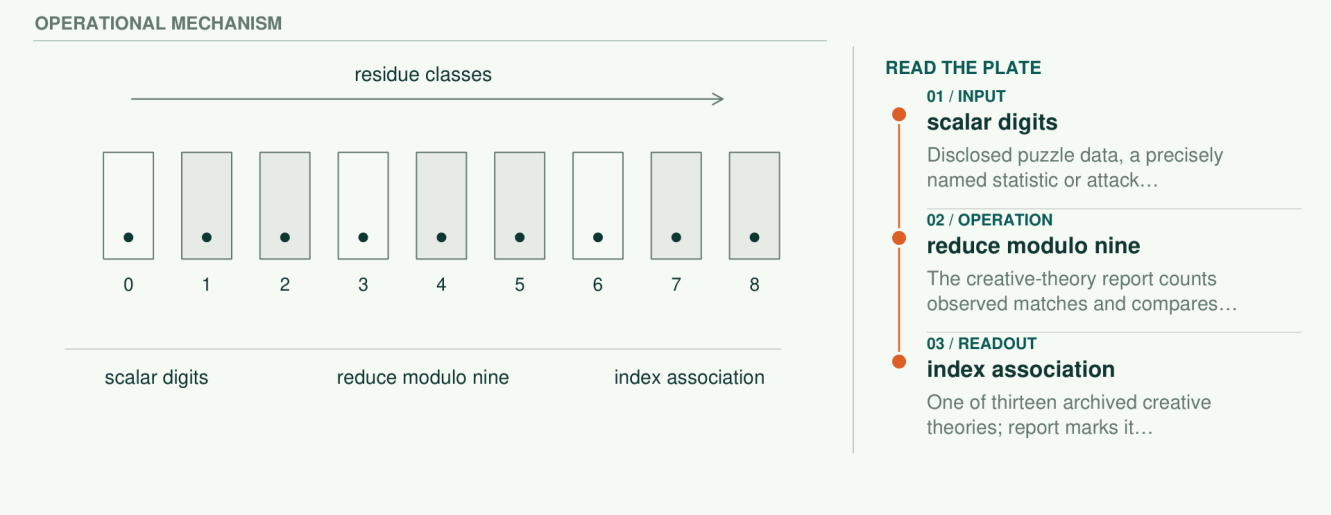
SECTION 14 / EXPLORATORY NUMBER THEORY, STATISTICS AND LITERATURE-INSPIRED PROBES

Digital-root puzzle-index association

M202

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

Reduce a scalar's decimal digits repeatedly and compare the result with a puzzle-index rule.

WHAT WAS ACTUALLY DONE

The creative-theory report counts observed matches and compares with controls. A small digital-root match imposes at most a weak residue condition, not the scalar's full value.

INPUTS NEEDED

Disclosed puzzle data, a precisely named statistic or attack prerequisite and appropriate random/control data.

WHAT THE RESULT MEANS

One of thirteen archived creative theories; report marks it nonsignificant, not a recovered-key method.

MATHEMATICAL FORMULATION

$$\mathrm{dr}(k) = 1 + ((k-1) \bmod 9), \quad k > 0$$

The decimal digital root is essentially a modulo-nine condition. A matching puzzle-index rule constrains only this small residue class.

EVIDENCE S148

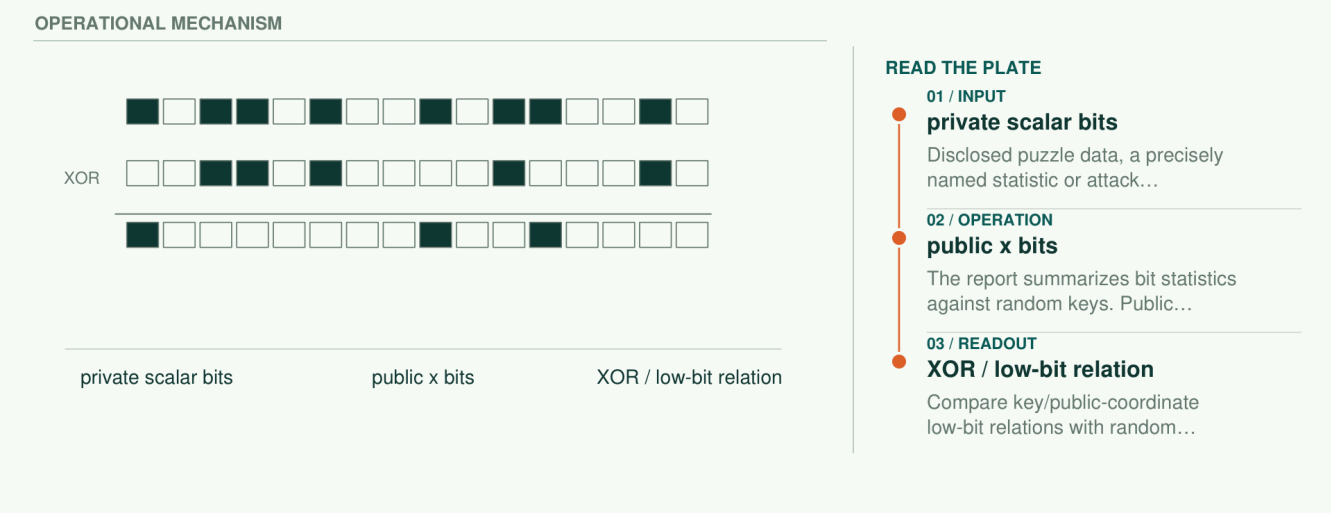
SECTION 14 / EXPLORATORY NUMBER THEORY, STATISTICS AND LITERATURE-INSPIRED PROBES

Public-key x-coordinate steganography

M203

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

Compare selected low bits or XOR patterns of k and the public x-coordinate for signs of intentional embedded information.

INPUTS NEEDED

Disclosed puzzle data, a precisely named statistic or attack prerequisite and appropriate random/control data.

WHAT WAS ACTUALLY DONE

The report summarizes bit statistics against random keys. Public multiplication is nonlinear; apparent low-bit similarity must predict unseen scalars before it can guide recovery.

WHAT THE RESULT MEANS

Compare key/public-coordinate low-bit relations with random controls; no convincing result recorded.

MATHEMATICAL FORMULATION

$$T_b(k) = (k \oplus x(kG)) \bmod 2^b$$

This is an example low-bit XOR statistic for testing scalar/public-x associations. Use matched random keys; no inverse map is supplied by the statistic.

EVIDENCE S148

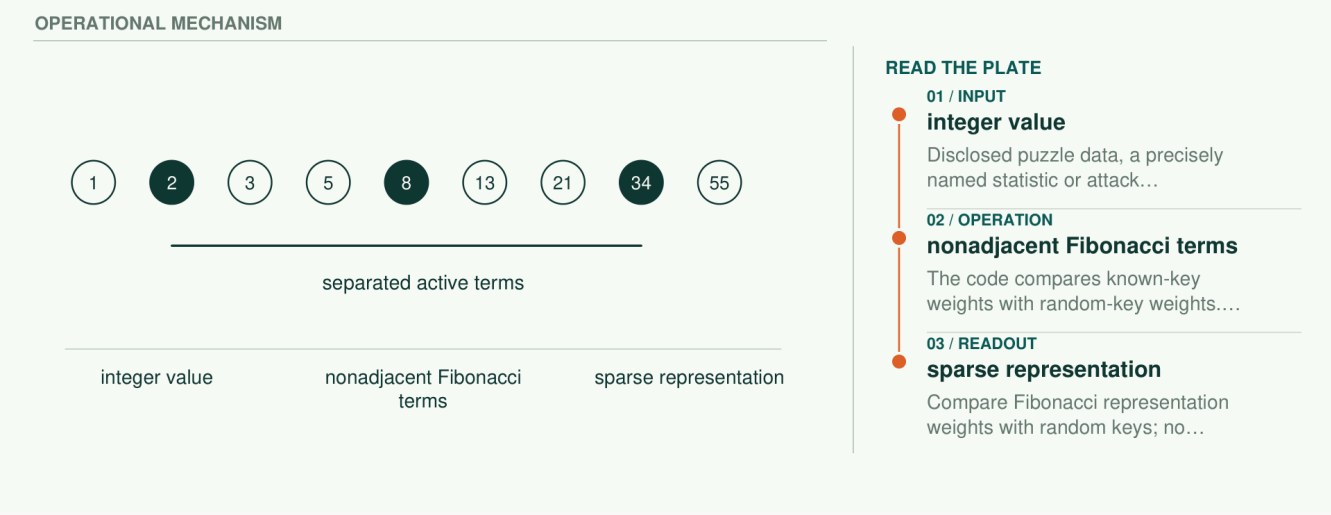
SECTION 14 / EXPLORATORY NUMBER THEORY, STATISTICS AND LITERATURE-INSPIRED PROBES

Zeckendorf/Fibonacci sparsity

M204

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

Represent integers as sums of nonadjacent Fibonacci numbers and measure how sparse that representation is.

INPUTS NEEDED

Disclosed puzzle data, a precisely named statistic or attack prerequisite and appropriate random/control data.

WHAT WAS ACTUALLY DONE

The code compares known-key weights with random-key weights. Changing numeral systems can change visible sparsity without reducing the number of possible keys.

WHAT THE RESULT MEANS

Compare Fibonacci representation weights with random keys; no significant anomaly reported.

MATHEMATICAL FORMULATION

$$k=\sum_j \epsilon_j F_j, \quad \epsilon_j \in \{0,1\}, \quad \epsilon_j \epsilon_{j+1}=0$$

Zeckendorf uses the conventional Fibonacci list 1,2,3,5,... and forbids adjacent terms. Weight counts must be compared with the same encoding on random keys.

EVIDENCE S148

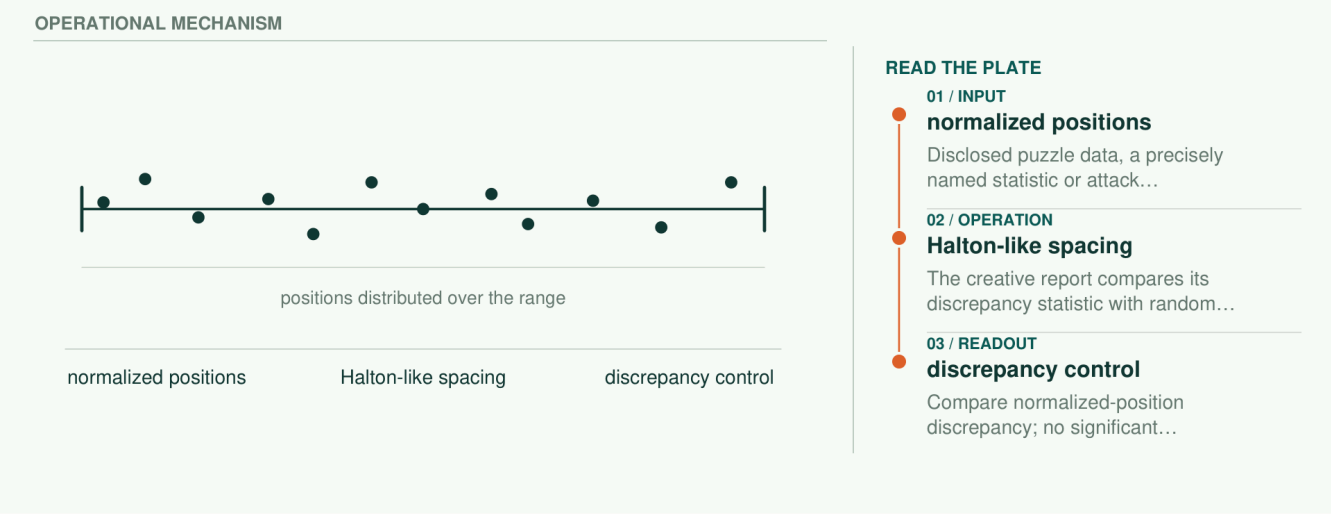
SECTION 14 / EXPLORATORY NUMBER THEORY, STATISTICS AND LITERATURE-INSPIRED PROBES

Low-discrepancy / Halton-like tier placement

M205

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

Normalize key positions to [0,1) and measure gaps or star discrepancy against structured low-discrepancy schedules.

INPUTS NEEDED

Disclosed puzzle data, a precisely named statistic or attack prerequisite and appropriate random/control data.

WHAT WAS ACTUALLY DONE

The creative report compares its discrepancy statistic with random controls. Evenly spread finite observations do not establish a deterministic next-key schedule.

WHAT THE RESULT MEANS

Compare normalized-position discrepancy; no significant deterministic schedule established.

MATHEMATICAL FORMULATION

$$D_m^* = \sup_{0 \le u \le 1} \left| \frac{1}{m} \sum_i 1\{u_i < u\} - u \right|$$

Star discrepancy measures spread of normalized tier positions u_i. Low discrepancy in a finite sample does not determine the next scalar.

EVIDENCE S148
PAPERS PAPER-004

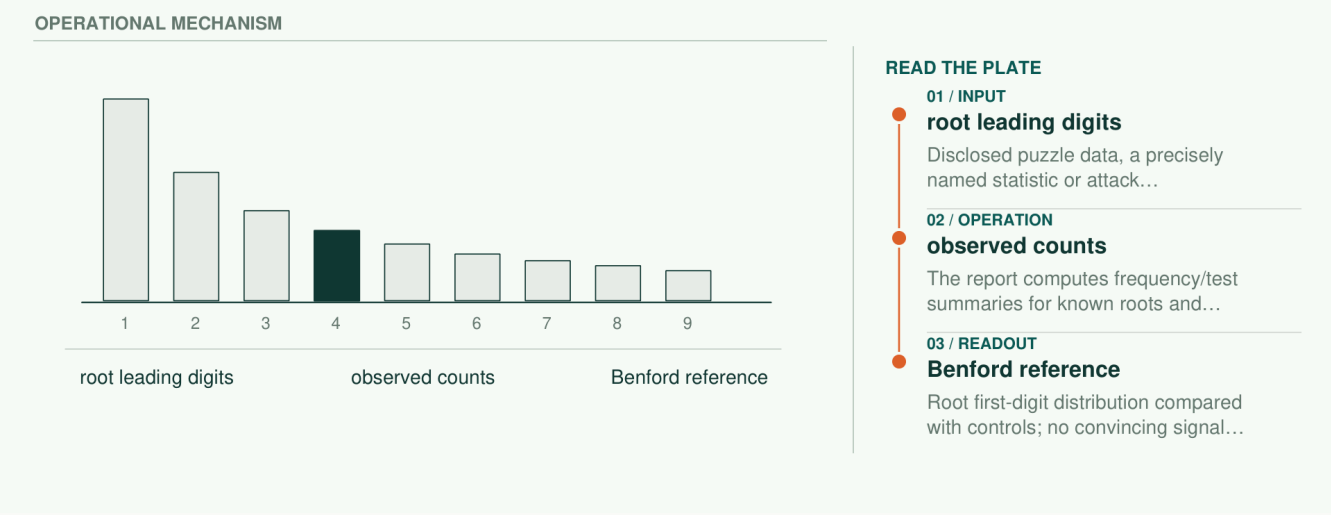
SECTION 14 / EXPLORATORY NUMBER THEORY, STATISTICS AND LITERATURE-INSPIRED PROBES

Benford leading digits of roots

M206

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

Compare first decimal digits of root values with a Benford-type reference distribution.

WHAT WAS ACTUALLY DONE

The report computes frequency/test summaries for known roots and controls. Agreement or disagreement is a distribution diagnostic, not enough information to determine unknown remainder digits.

INPUTS NEEDED

Disclosed puzzle data, a precisely named statistic or attack prerequisite and appropriate random/control data.

WHAT THE RESULT MEANS

Root first-digit distribution compared with controls; no convincing signal reported.

MATHEMATICAL FORMULATION

$$\Pr(D=d)=\log_{10}(1+1/d),\quad d=1,\ldots,9$$

This is the Benford reference for a leading decimal digit. Finite root ranges need a matched reference; the formula is not automatically their correct null.

EVIDENCE S148

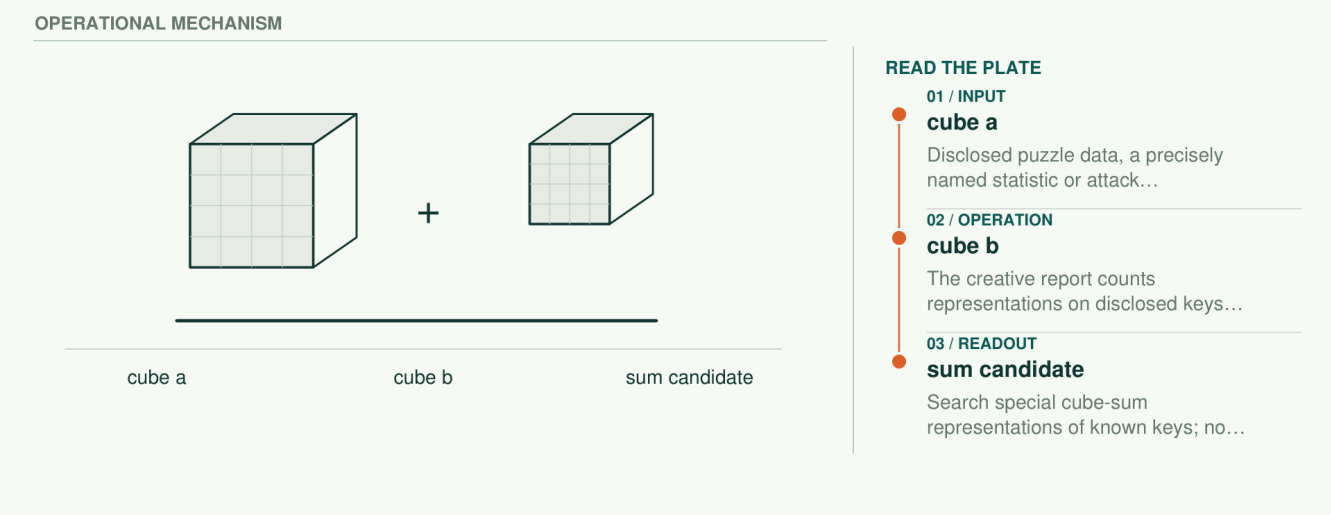
SECTION 14 / EXPLORATORY NUMBER THEORY, STATISTICS AND LITERATURE-INSPIRED PROBES

Taxicab sums of two cubes

M207

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

Search whether a scalar is a sum of two integer cubes, potentially exposing a special generator family.

INPUTS NEEDED

Disclosed puzzle data, a precisely named statistic or attack prerequisite and appropriate random/control data.

WHAT WAS ACTUALLY DONE

The creative report counts representations on disclosed keys and random controls. A special representation would still need independent evidence that the target was generated by that family.

WHAT THE RESULT MEANS

Search special cube-sum representations of known keys; no transferable high-index key relation.

MATHEMATICAL FORMULATION

$$k = a^3 + b^3$$

Test whether disclosed scalars have integer two-cube representations in the declared range. Representation counts need matched controls and a justified target-generation premise.

EVIDENCE S148
PAPERS PAPER-002

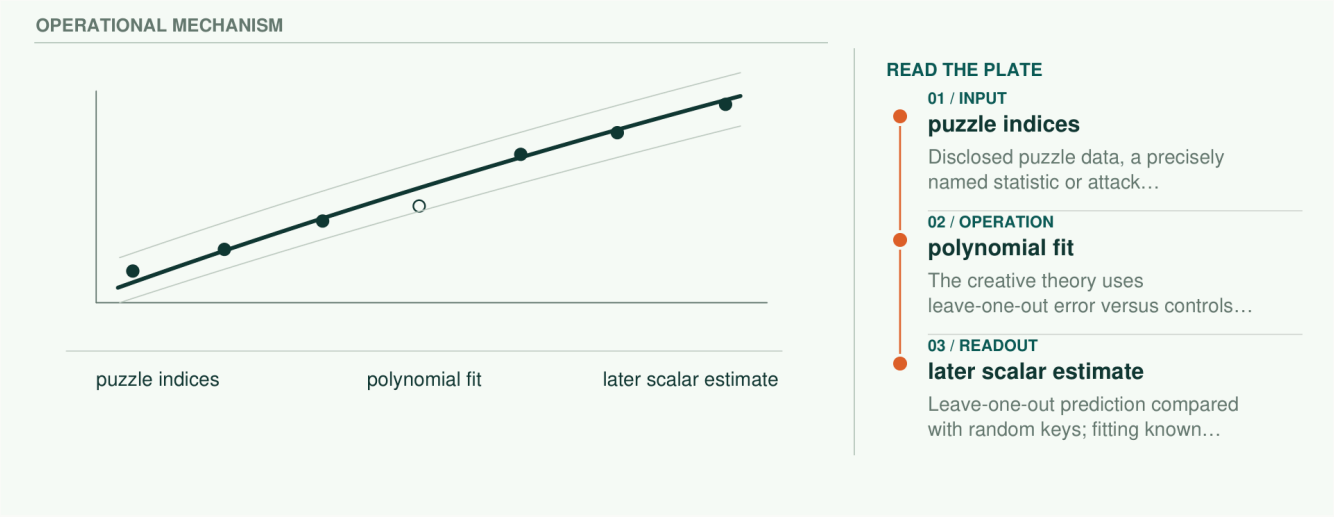
SECTION 14 / EXPLORATORY NUMBER THEORY, STATISTICS AND LITERATURE-INSPIRED PROBES

Polynomial in puzzle number

M208

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

Fit a low-degree function of puzzle number and forecast later key positions or values.

WHAT WAS ACTUALLY DONE

The creative theory uses leave-one-out error versus controls; separate exact continuation tests verify concrete candidates. A polynomial fitting all known points has no automatic predictive authority.

INPUTS NEEDED

Disclosed puzzle data, a precisely named statistic or attack prerequisite and appropriate random/control data.

WHAT THE RESULT MEANS

Leave-one-out prediction compared with random keys; fitting known data does not determine the next scalar.

MATHEMATICAL FORMULATION

$$\widehat{k}_N = \sum_{j=0}^{d} a_j N^j$$

A polynomial in index N is a hypothesized predictor. Its fitted degree, target scale and evaluation split determine what can be inferred.

EVIDENCE S148

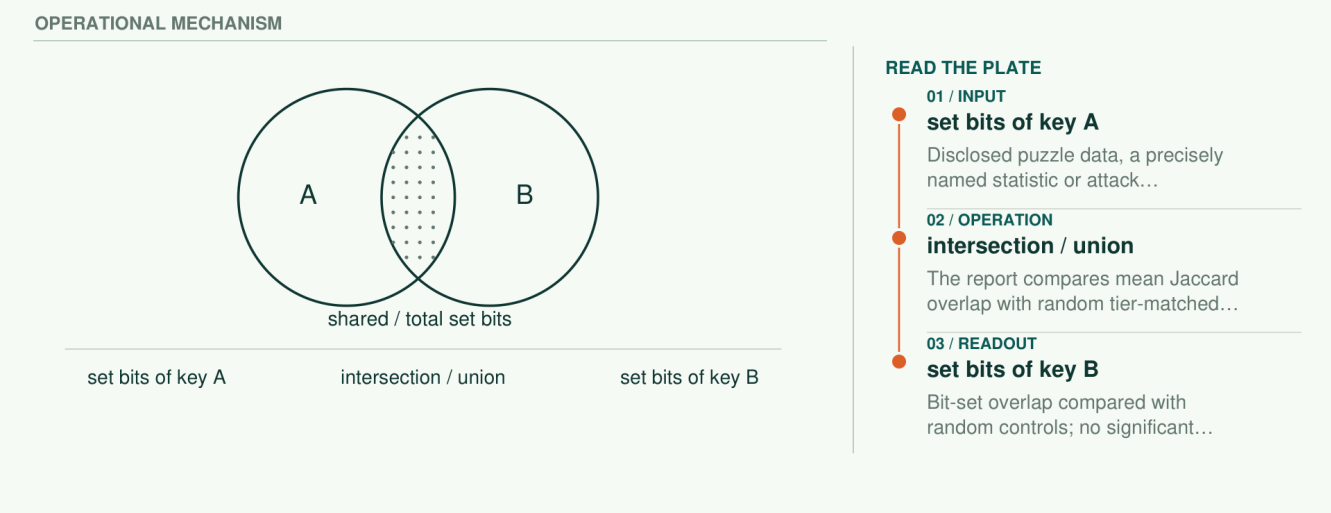
SECTION 14 / EXPLORATORY NUMBER THEORY, STATISTICS AND LITERATURE-INSPIRED PROBES

Consecutive bit-Jaccard similarity

M209

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

View set-bit positions as sets and compute intersection divided by union for consecutive puzzle keys.

INPUTS NEEDED

Disclosed puzzle data, a precisely named statistic or attack prerequisite and appropriate random/control data.

WHAT WAS ACTUALLY DONE

The report compares mean Jaccard overlap with random tier-matched keys. Similarity caused by bit widths must not be mistaken for inherited secret bits.

WHAT THE RESULT MEANS

Bit-set overlap compared with random controls; no significant inheritance relation reported.

MATHEMATICAL FORMULATION

$$J(A,B)=\frac{|A\cap B|}{|A\cup B|}$$

A and B are sets of occupied bit positions in consecutive keys. Tier lengths and fixed leading bits affect the null overlap.

EVIDENCE S148

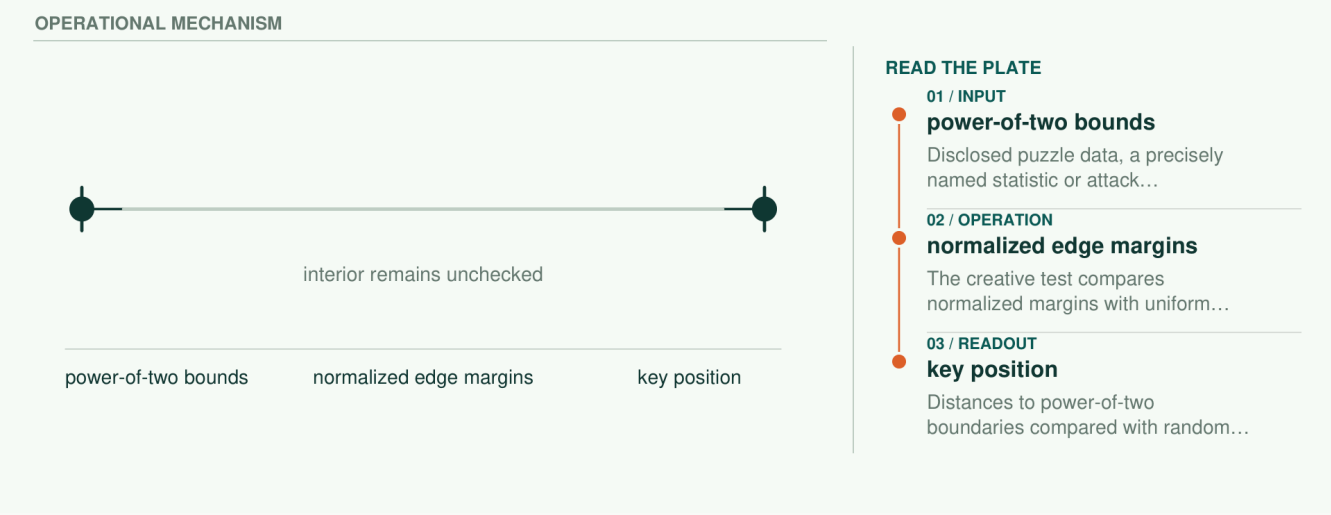
SECTION 14 / EXPLORATORY NUMBER THEORY, STATISTICS AND LITERATURE-INSPIRED PROBES

Mersenne-adjacent margin hypothesis

M210

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

Measure whether keys lie unusually close to powers of two or all-ones Mersenne-style boundaries.

WHAT WAS ACTUALLY DONE

The creative test compares normalized margins with uniform controls. Small margins would identify a candidate region only under a validated generator prior.

INPUTS NEEDED

Disclosed puzzle data, a precisely named statistic or attack prerequisite and appropriate random/control data.

WHAT THE RESULT MEANS

Distances to power-of-two boundaries compared with random ranges.

MATHEMATICAL FORMULATION

$$\delta(k) = \frac{\min(k-L, U-k)}{U-L+1}$$

This normalized distance to tier endpoints is one margin diagnostic. Other power-of-two or all-ones boundary families must be declared separately.

EVIDENCE S148

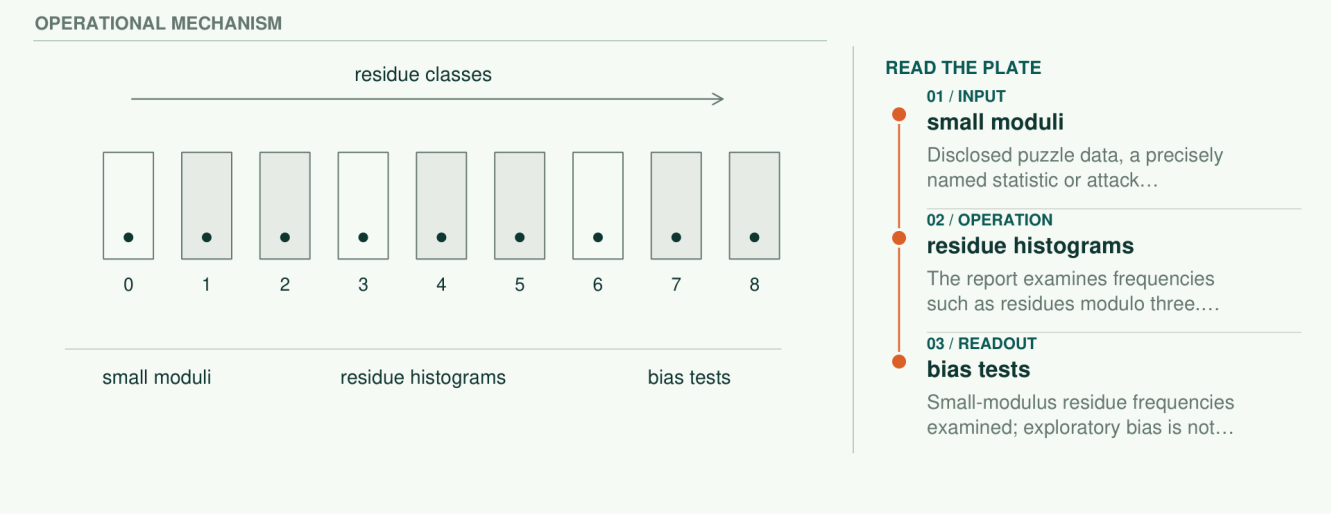
SECTION 14 / EXPLORATORY NUMBER THEORY, STATISTICS AND LITERATURE-INSPIRED PROBES

Shared CRT residue / digital modular biases

M211

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

Count small-modulus residues and ask whether a common congruence distinguishes disclosed keys from random intervals.

INPUTS NEEDED

Disclosed puzzle data, a precisely named statistic or attack prerequisite and appropriate random/control data.

WHAT WAS ACTUALLY DONE

The report examines frequencies such as residues modulo three. Several weak residue preferences can arise by chance; they do not justify applying an unvalidated CRT restriction to the target.

WHAT THE RESULT MEANS

Small-modulus residue frequencies examined; exploratory bias is not proof of a target constraint.

MATHEMATICAL FORMULATION

$$\chi^2 = \sum_{r=0}^{m-1} \frac{(O_r - E_r)^2}{E_r}$$

Compare residue counts with the correct tier-induced expected counts E_r. Trying many moduli requires accounting for that selection before imposing CRT restrictions.

EVIDENCE S148

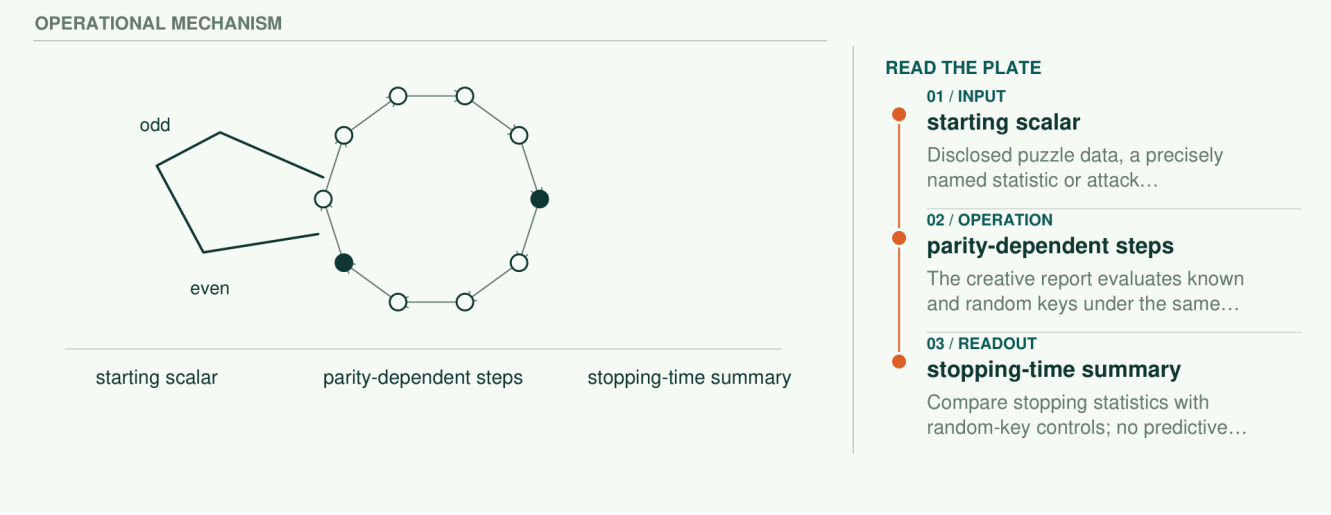
SECTION 14 / EXPLORATORY NUMBER THEORY, STATISTICS AND LITERATURE-INSPIRED PROBES

Collatz stopping-time association

M212

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

Map each key through the Collatz iteration and compare stopping-time statistics.

WHAT WAS ACTUALLY DONE

The creative report evaluates known and random keys under the same rule. The transformed statistic is not shown to be invertible from a target address or predictive of its scalar.

INPUTS NEEDED

Disclosed puzzle data, a precisely named statistic or attack prerequisite and appropriate random/control data.

WHAT THE RESULT MEANS

Compare stopping statistics with random-key controls; no predictive result reported.

MATHEMATICAL FORMULATION

$$C(x) = x/2 \text{ if even}; \quad C(x) = 3x + 1 \text{ if odd}$$

Iterate to the stated stopping condition or cap and compare stopping times against matched controls. No general termination assumption is needed for a capped experiment.

EVIDENCE S148

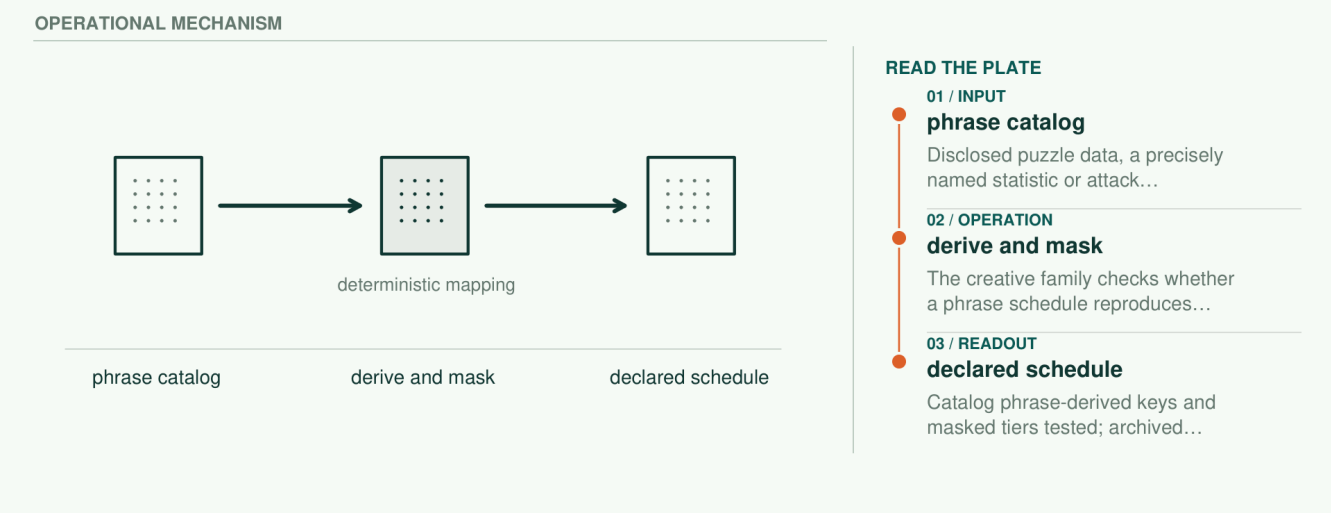
SECTION 14 / EXPLORATORY NUMBER THEORY, STATISTICS AND LITERATURE-INSPIRED PROBES

Brainwallet/BIP39 phrase schedule

M213

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

Hash or derive keys from a fixed catalog of phrases/mnemonics and apply the puzzle's interval mask.

INPUTS NEEDED

Disclosed puzzle data, a precisely named statistic or attack prerequisite and appropriate random/control data.

WHAT WAS ACTUALLY DONE

The creative family checks whether a phrase schedule reproduces significant known tiers. Tiny-tier coincidences are separated from evidence that one phrase generates later unknown keys.

WHAT THE RESULT MEANS

Catalog phrase-derived keys and masked tiers tested; archived creative report records no nontrivial hits.

MATHEMATICAL FORMULATION

$$\hat{k}_N = \mathrm{Mask}_N(\mathrm{Derive}(\mathrm{phrase}, p_N))$$

A phrase catalog and path schedule p_N define a finite candidate family. Tiny-tier coincidences do not establish a rule for later indices.

EVIDENCE S148

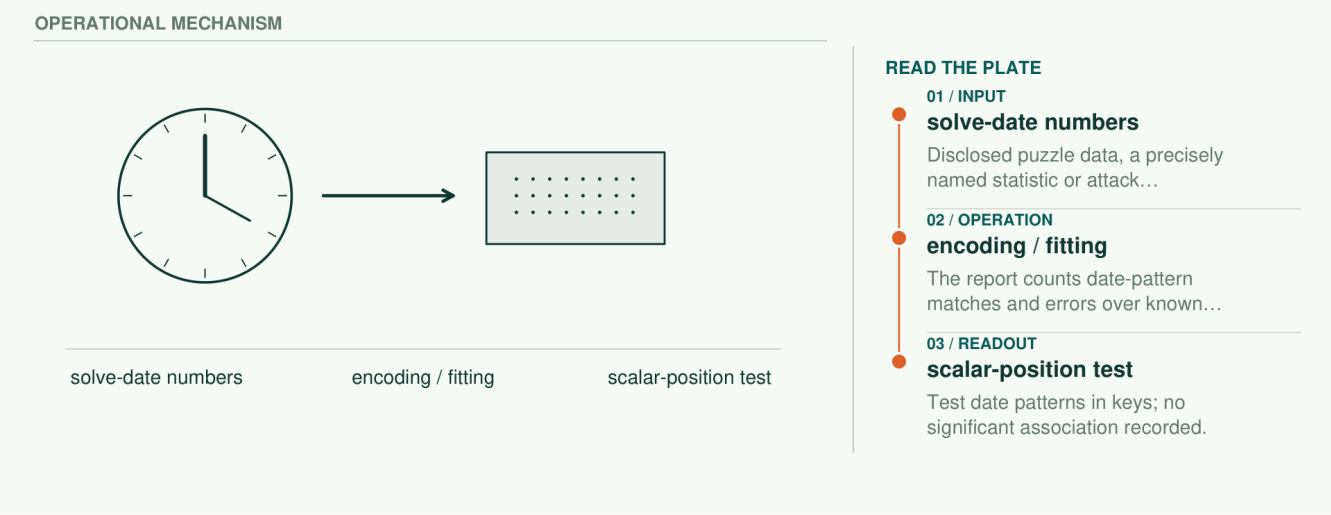
SECTION 14 / EXPLORATORY NUMBER THEORY, STATISTICS AND LITERATURE-INSPIRED PROBES

Solve-date scalar encoding

M214

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

Encode dates in scalars or compare normalized scalar positions with date-related numbers.

WHAT WAS ACTUALLY DONE

The report counts date-pattern matches and errors over known solve records. Solve date can reflect search effort rather than generation, so correlation would require independent validation.

INPUTS NEEDED

Disclosed puzzle data, a precisely named statistic or attack prerequisite and appropriate random/control data.

WHAT THE RESULT MEANS

Test date patterns in keys; no significant association recorded.

MATHEMATICAL FORMULATION

$$u_N = \frac{k_N - 2^{N-1}}{2^{N-1}}, \quad \widehat{u}_N = f(\mathrm{date}_N)$$

Date encodings propose normalized tier positions or literal integers. Solve dates are observations after generation and can introduce misleading correlations.

EVIDENCE S148

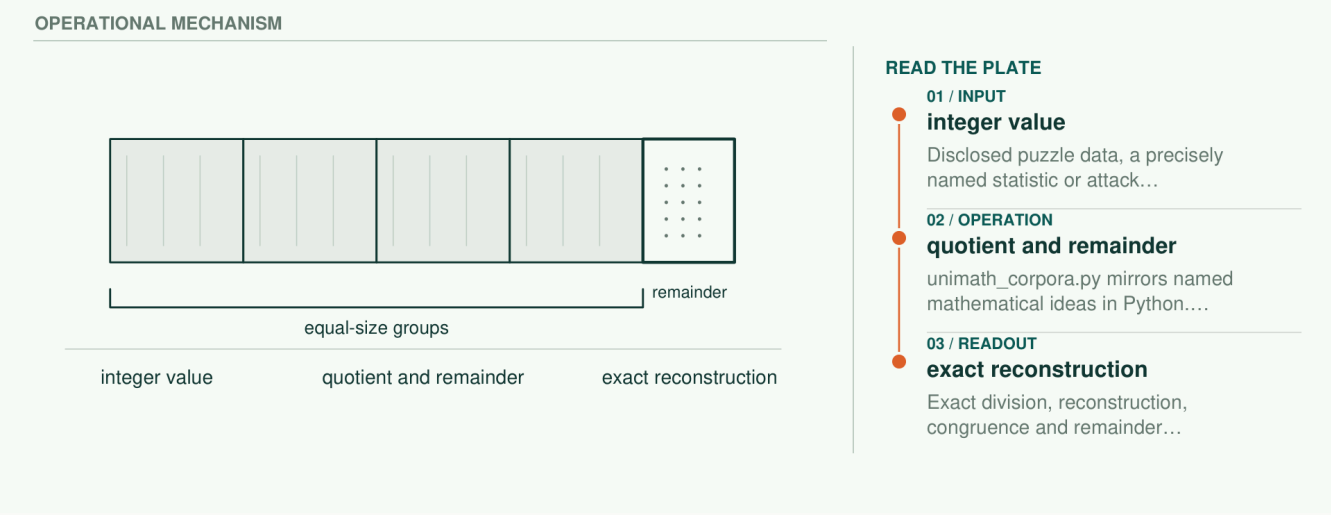
SECTION 14 / EXPLORATORY NUMBER THEORY, STATISTICS AND LITERATURE-INSPIRED PROBES

UniMath-inspired arithmetic identities

M215

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

Re-express arithmetic facts such as Euclidean division and reconstruction as executable exact assertions on known data.

INPUTS NEEDED

Disclosed puzzle data, a precisely named statistic or attack prerequisite and appropriate random/control data.

WHAT WAS ACTUALLY DONE

unimath_corpora.py mirrors named mathematical ideas in Python. Passing these checks validates that implementation and corpus, not a machine-checked Agda proof or an unknown-key extractor.

WHAT THE RESULT MEANS

Exact division, reconstruction, congruence and remainder assertions on known keys; arithmetic checks are not an Agda proof or prediction.

MATHEMATICAL FORMULATION

$$a = bq + r, \quad 0 \le r < b, \quad b > 0$$

Euclidean division and exact reconstruction are executable arithmetic checks. Passing them is not a formal proof assistant verification of a key-extraction theorem.

EVIDENCE S125 / S192

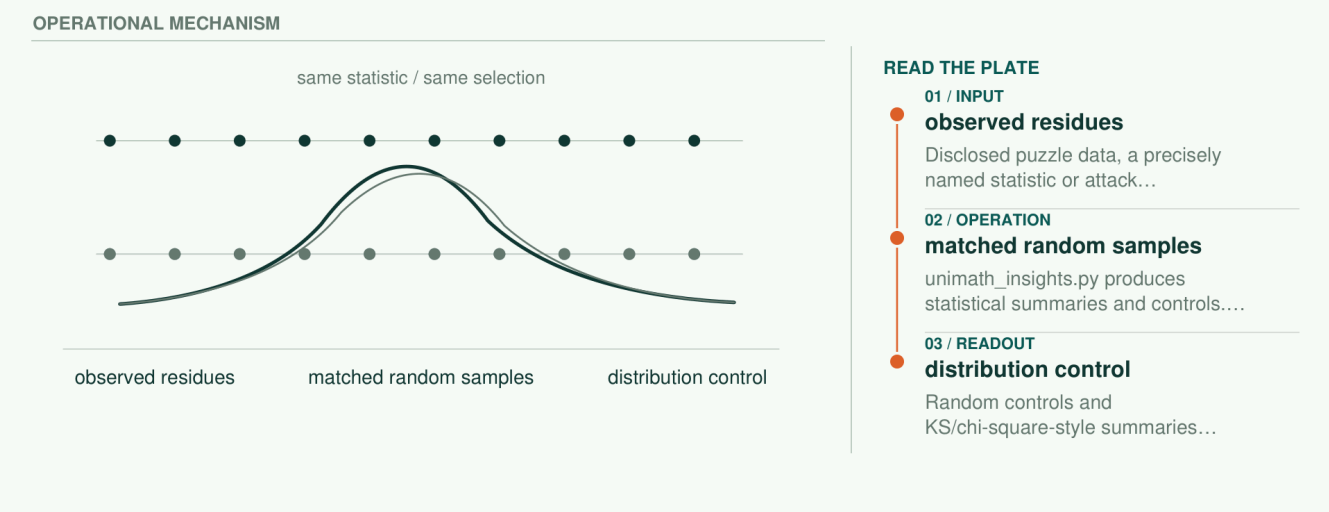
SECTION 14 / EXPLORATORY NUMBER THEORY, STATISTICS AND LITERATURE-INSPIRED PROBES

Residue/Legendre/coefficient distribution controls

M216

EVIDENCE STATUS IMPLEMENTED

FIGURE / METHOD MECHANISM



HOW IT WORKS

Compare coefficient, residue, Legendre-symbol and remainder distributions with matched random samples.

INPUTS NEEDED

Disclosed puzzle data, a precisely named statistic or attack prerequisite and appropriate random/control data.

WHAT WAS ACTUALLY DONE

unimath_insights.py produces statistical summaries and controls. It tests distributional hypotheses rather than deriving new bits directly from the target public point.

WHAT THE RESULT MEANS

Random controls and KS/chi-square-style summaries distinguish transformation artifacts from candidate key bias.

MATHEMATICAL FORMULATION

$$\left(\tfrac{a}{p}\right) \equiv a^{(p-1)/2} \pmod{p}$$

For odd prime p and a not divisible by p, Euler's criterion encodes the Legendre sign. Compare such symbols, residues and coefficients with matched controls.

EVIDENCE S126

SECTION 14 / EXPLORATORY NUMBER THEORY, STATISTICS AND LITERATURE-INSPIRED PROBES

DP hashing/birthday/URAND jump probes

M217

EVIDENCE STATUS IMPLEMENTED

FIGURE / METHOD MECHANISM

HOW IT WORKS

Measure hash-table collisions, birthday behavior and candidate jump tables to improve practical walk mechanics.

INPUTS NEEDED

Disclosed puzzle data, a precisely named statistic or attack prerequisite and appropriate random/control data.

WHAT WAS ACTUALLY DONE

Bitsavers-inspired probes model open addressing and first-collision counts, then compare URAND-derived jumps. These are solver engineering experiments, not secret-generator observations.

WHAT THE RESULT MEANS

Bitsavers-inspired hash-table, collision and jump-distribution experiments assess solver mechanics.

MATHEMATICAL FORMULATION

$$\Pr(\text{collision}) \approx 1 - \exp\left(-\frac{t(t-1)}{2M}\right)$$

This birthday approximation assumes t independent uniform draws into M buckets. It calibrates storage/jump engineering, not target-scalar prediction.

EVIDENCE S096 / S140

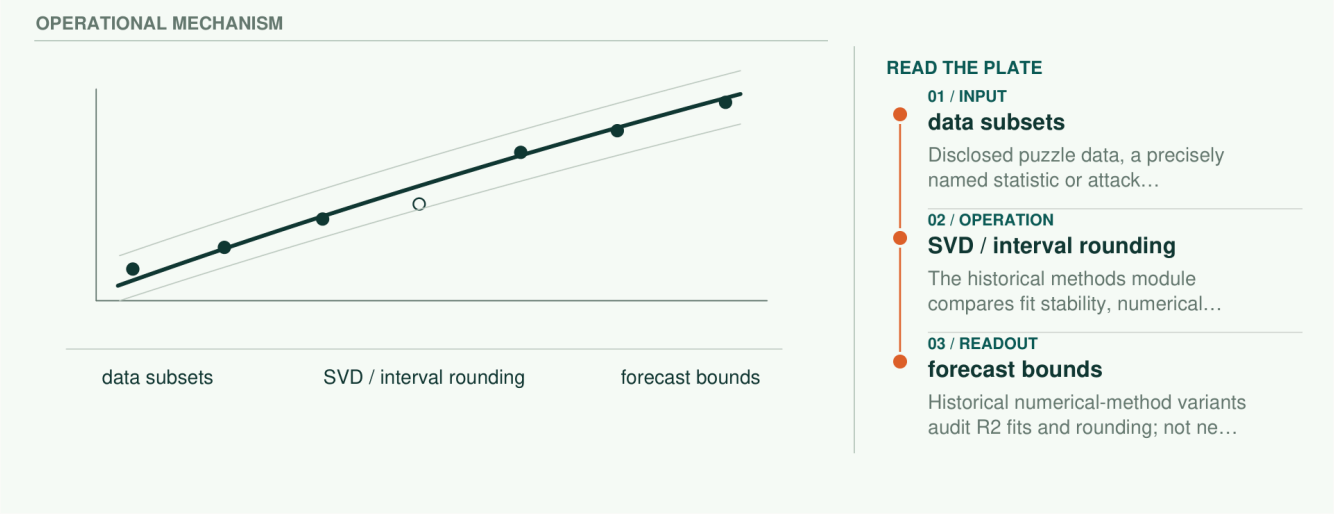
SECTION 14 / EXPLORATORY NUMBER THEORY, STATISTICS AND LITERATURE-INSPIRED PROBES

Regression SVD/subset/interval arithmetic variants

M218

EVIDENCE STATUS IMPLEMENTED

FIGURE / METHOD MECHANISM



HOW IT WORKS

Refit numerical predictors using alternative regression subsets/SVD and propagate rounding intervals through formulas.

INPUTS NEEDED

Disclosed puzzle data, a precisely named statistic or attack prerequisite and appropriate random/control data.

WHAT WAS ACTUALLY DONE

The historical methods module compares fit stability, numerical bounds and selected regressions. It can diagnose approximation error but cannot manufacture predictive information absent from the data.

WHAT THE RESULT MEANS

Historical numerical-method variants audit R2 fits and rounding; not new information about the target.

MATHEMATICAL FORMULATION

$$X = U\Sigma V^{\mathsf{T}}, \quad \widehat{\beta} = X^{+} y$$

SVD and the pseudoinverse estimate regression parameters; near-zero singular values reveal instability. Numerical stabilization does not create new predictive data.

EVIDENCE S096

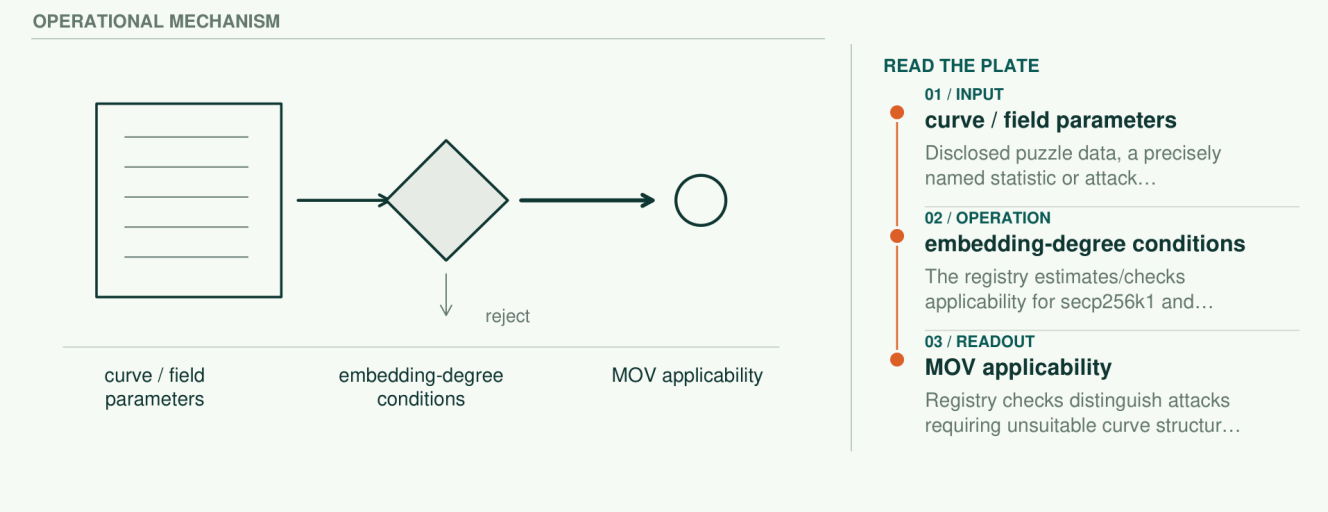
SECTION 14 / EXPLORATORY NUMBER THEORY, STATISTICS AND LITERATURE-INSPIRED PROBES

MOV and finite-field index-calculus applicability

M219

EVIDENCE STATUS APPLICABILITY REVIEW

FIGURE / METHOD MECHANISM



HOW IT WORKS

MOV reduces certain elliptic-curve logs to finite fields when the embedding degree is suitably small; index calculus has domain-specific requirements.

INPUTS NEEDED

Disclosed puzzle data, a precisely named statistic or attack prerequisite and appropriate random/control data.

WHAT WAS ACTUALLY DONE

The registry estimates/checks applicability for secp256k1 and records why named structural attacks do not straightforwardly apply. It is not an executed transfer solving the target log.

WHAT THE RESULT MEANS

Registry checks distinguish attacks requiring unsuitable curve structure from applicable interval methods.

MATHEMATICAL FORMULATION

$$e = \min\{j \geq 1 : n \mid p^j - 1\}$$

e is the embedding degree of an order-n subgroup over the base field of size p. MOV applicability depends on tractable e and the target finite-field log cost.

EVIDENCE S112

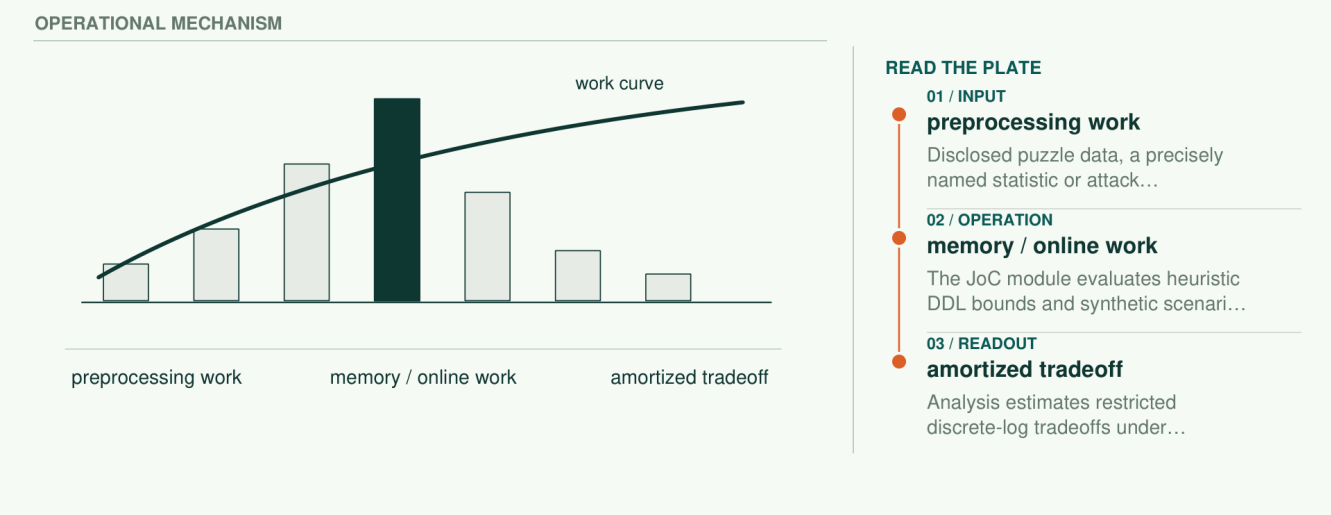
SECTION 14 / EXPLORATORY NUMBER THEORY, STATISTICS AND LITERATURE-INSPIRED PROBES

DDL preprocessing tradeoff bounds

M220

EVIDENCE STATUS SYNTHETIC/APPLICABILITY

FIGURE / METHOD MECHANISM



HOW IT WORKS

Restricted discrete-log methods trade preprocessing, memory and online work under stated target/setup assumptions.

INPUTS NEEDED

Disclosed puzzle data, a precisely named statistic or attack prerequisite and appropriate random/control data.

WHAT WAS ACTUALLY DONE

The JoC module evaluates heuristic DDL bounds and synthetic scenarios. Estimated online savings must include required preprocessing and oracles before being compared with an actual puzzle search.

WHAT THE RESULT MEANS

Analysis estimates restricted discrete-log tradeoffs under additional setup assumptions; no blind puzzle recovery demonstrated.

MATHEMATICAL FORMULATION

$$T_{\text{total}}(q) = T_{\text{pre}} + qT_{\text{online}}$$

Preprocessing tradeoffs must include memory, setup and the number q of targets. An online cost estimate alone is not a complete recovery bound.

EVIDENCE S108

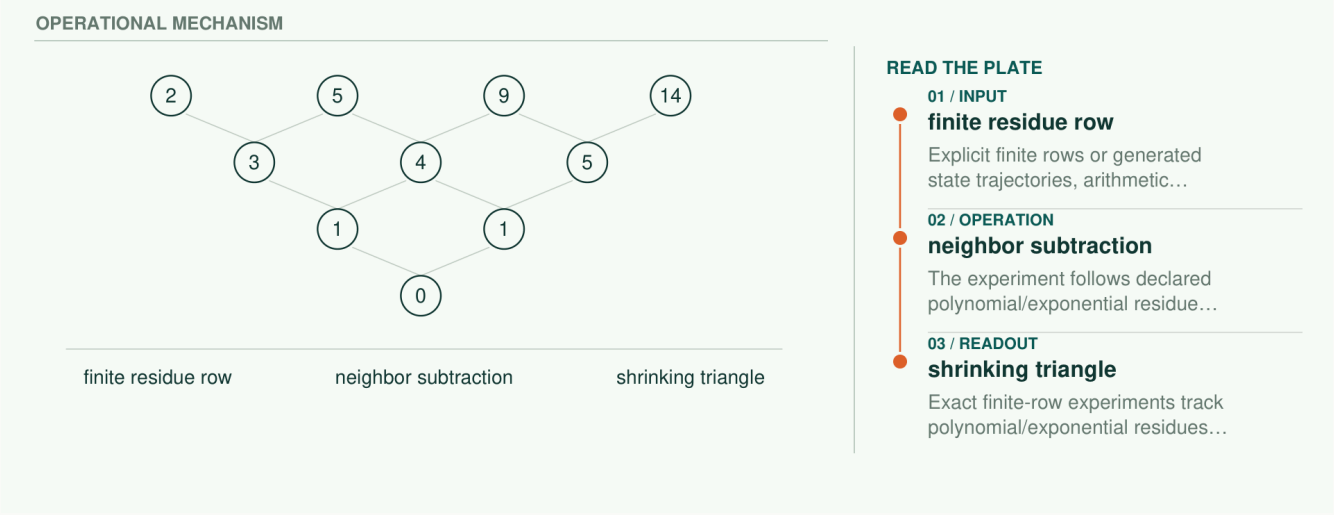
SECTION 15 / MODULAR DIFFERENCES AND GENERAL TRANSFORMATION FRAMEWORKS

Finite modular difference triangles

M221

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

Repeatedly subtract neighboring residues modulo m; finite rows shrink by one entry at each level.

WHAT WAS ACTUALLY DONE

The experiment follows declared polynomial/exponential residue families through their full finite triangles. Reaching zero on supplied samples does not determine an unknown continuation without a recurrence assumption.

INPUTS NEEDED

Explicit finite rows or generated state trajectories, arithmetic conventions, transformation parameters and finite resource limits.

WHAT THE RESULT MEANS

Exact finite-row experiments track polynomial/exponential residues through all levels; they do not reconstruct an unknown puzzle key.

MATHEMATICAL FORMULATION

$$D_i^{(t+1)} = D_{i+1}^{(t)} - D_i^{(t)} \pmod{m}$$

Ordinary finite difference rows shorten by one entry per level. Zero rows are conditional on the supplied sequence and do not fix arbitrary continuations.

EVIDENCE S036

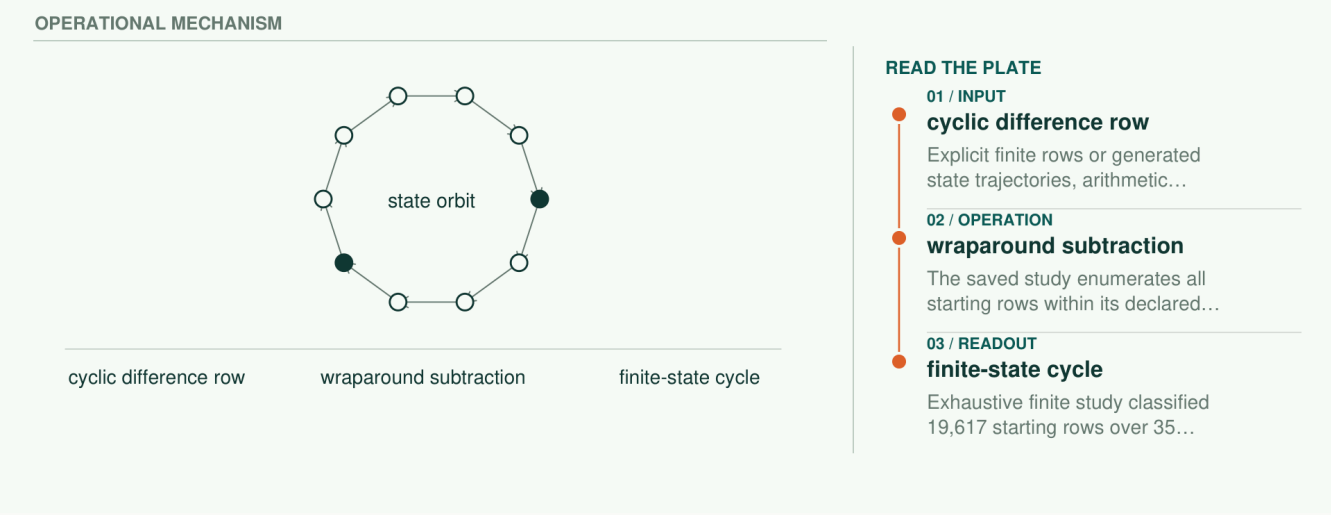
SECTION 15 / MODULAR DIFFERENCES AND GENERAL TRANSFORMATION FRAMEWORKS

Cyclic difference orbit classification

M222

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

Wrap the difference operator cyclically so row length stays fixed, then detect exact zero or repeated states in a finite state graph.

INPUTS NEEDED

Explicit finite rows or generated state trajectories, arithmetic conventions, transformation parameters and finite resource limits.

WHAT WAS ACTUALLY DONE

The saved study enumerates all starting rows within its declared small configurations and classifies their orbits. This is an exhaustive finite dynamics result, not cryptographic inversion.

WHAT THE RESULT MEANS

Exhaustive finite study classified 19,617 starting rows over 35 configurations into zero/cycles.

MATHEMATICAL FORMULATION

$$(Dx)_i = x_{i+1} - x_i \pmod{m}, \quad i \bmod \ell$$

Cyclic differences preserve row length ell. The finite state space has m^ell rows, so every orbit eventually repeats; exhaustive results apply to the declared sizes.

EVIDENCE S036

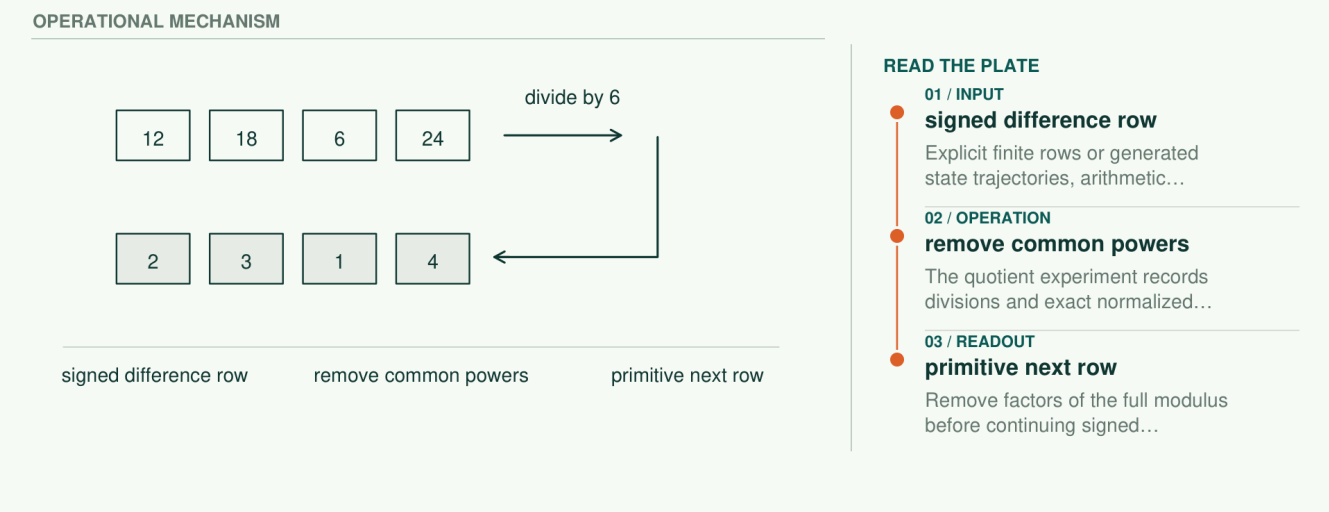
SECTION 15 / MODULAR DIFFERENCES AND GENERAL TRANSFORMATION FRAMEWORKS

Signed quotient hierarchy with common-factor removal

M223

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

Keep signed integer differences and divide a row by m whenever every entry shares that factor before taking the next difference.

INPUTS NEEDED

Explicit finite rows or generated state trajectories, arithmetic conventions, transformation parameters and finite resource limits.

WHAT WAS ACTUALLY DONE

The quotient experiment records divisions and exact normalized states. Removing common factors exposes structure but can still leave growing values or unresolved trajectories.

WHAT THE RESULT MEANS

Remove factors of the full modulus before continuing signed differences; zero, cycle and capped outcomes remain separate.

MATHEMATICAL FORMULATION

$$Q_m(x) = x/m^{v_m(x)}, \quad v_m(x) = \max\{e : m^e \mid x_i \text{ for all } i\}$$

For a nonzero integer row, remove the largest common power of m. Combine normalization with signed differences in the exact order specified by the experiment; handle the all-zero row separately.

EVIDENCE S034

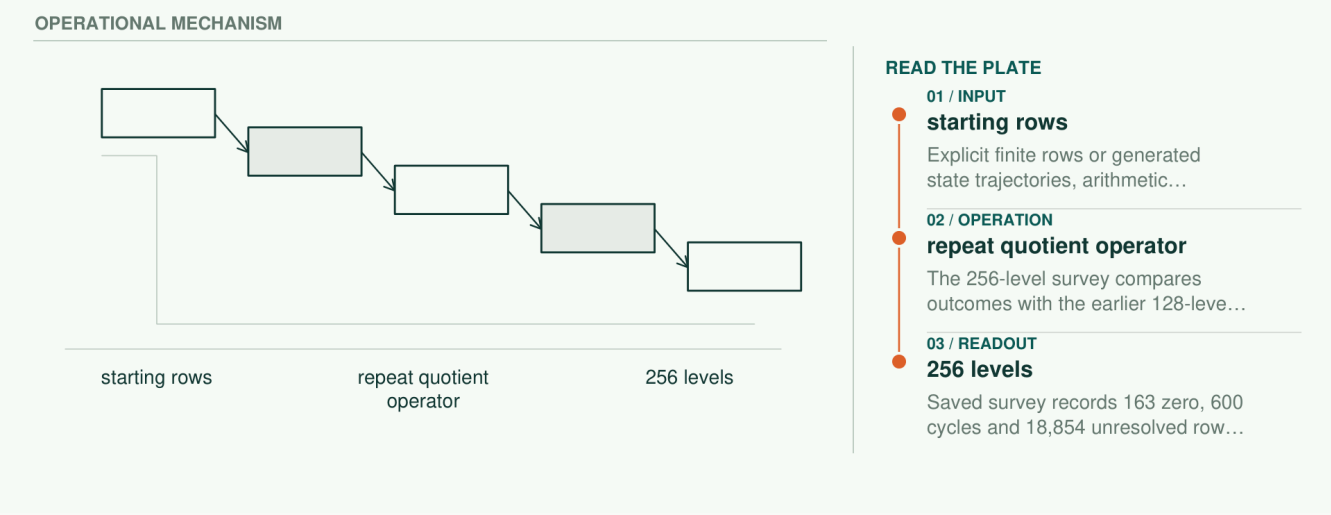
SECTION 15 / MODULAR DIFFERENCES AND GENERAL TRANSFORMATION FRAMEWORKS

256-level quotient continuation

M224

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

Extend the same bounded quotient experiment to more levels without changing the starting rows or normalization rule.

INPUTS NEEDED

Explicit finite rows or generated state trajectories, arithmetic conventions, transformation parameters and finite resource limits.

WHAT WAS ACTUALLY DONE

The 256-level survey compares outcomes with the earlier 128-level cap. Capped rows remain unresolved; lack of termination by the cap is not proof of infinite nontermination.

WHAT THE RESULT MEANS

Saved survey records 163 zero, 600 cycles and 18,854 unresolved rows; no new resolutions after the earlier cap.

MATHEMATICAL FORMULATION

$$x_{t+1} = F(x_t), \quad 0 \le t < 256$$

F is the declared quotient/difference update. Reaching the iteration cap gives a finite unresolved trajectory, not proof of nontermination.

EVIDENCE S035

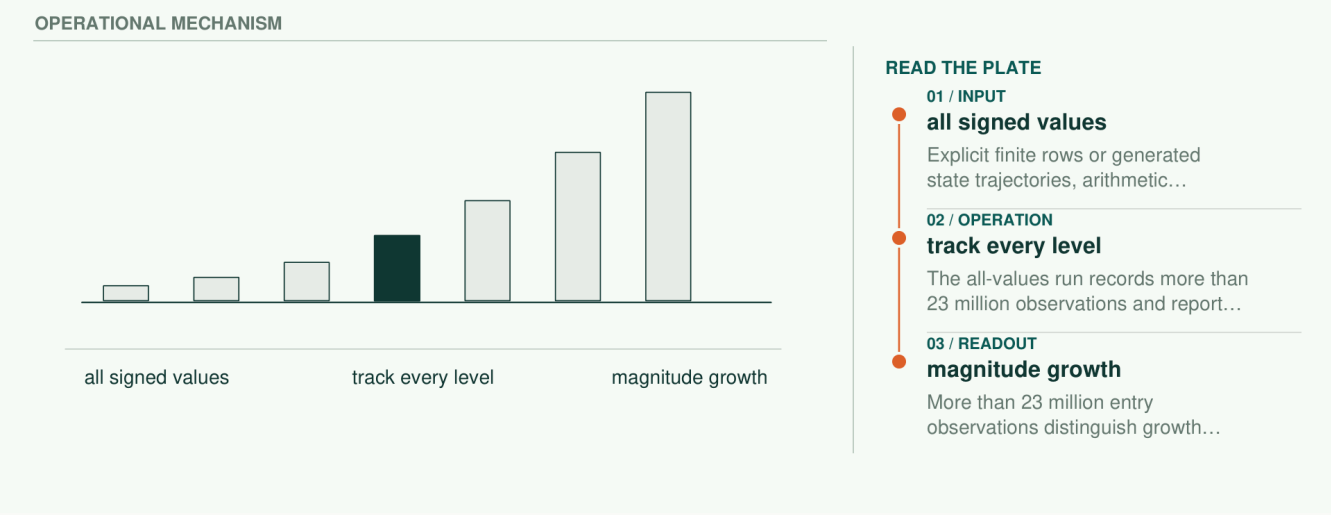
SECTION 15 / MODULAR DIFFERENCES AND GENERAL TRANSFORMATION FRAMEWORKS

All-value magnitude/growth tracking

M225

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

Track magnitudes, signs, divisibility and growth of every entry across every difference level, including after an earlier event.

INPUTS NEEDED

Explicit finite rows or generated state trajectories, arithmetic conventions, transformation parameters and finite resource limits.

WHAT WAS ACTUALLY DONE

The all-values run records more than 23 million observations and reports extreme bit lengths. It measures behavior of those exact finite trajectories, not secret-key information gained.

WHAT THE RESULT MEANS

More than 23 million entry observations distinguish growth from termination; unresolved finite caps are not nontermination proofs.

MATHEMATICAL FORMULATION

$$B_t = \max_i \left\lceil \log_2 \left(|x_i^{(t)}| + 1 \right) \right\rceil$$

Track maximum magnitude bits along with signs and divisibility for every finite row. Statistics describe those trajectories only.

EVIDENCE S032

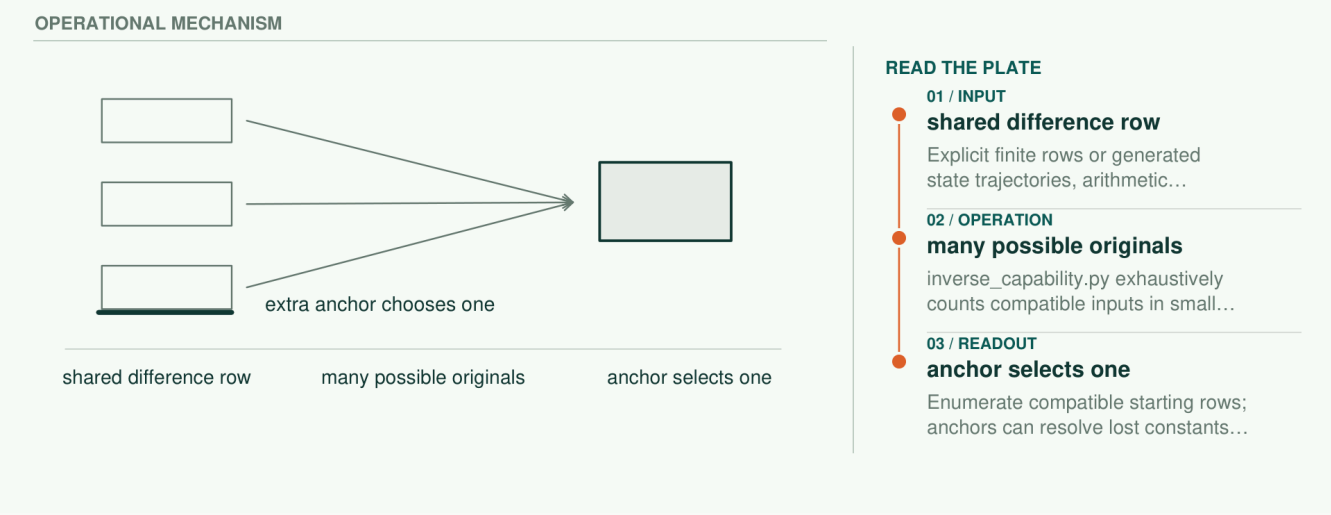
SECTION 15 / MODULAR DIFFERENCES AND GENERAL TRANSFORMATION FRAMEWORKS

Inverse-difference kernel/anchor tests

M226

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

Group all starting rows that produce the same observed difference row. Additional anchors can select among these inverse candidates.

INPUTS NEEDED

Explicit finite rows or generated state trajectories, arithmetic conventions, transformation parameters and finite resource limits.

WHAT WAS ACTUALLY DONE

inverse_capability.py exhaustively counts compatible inputs in small finite systems. If several remain, the transformed row alone has lost information and cannot uniquely reconstruct the original.

WHAT THE RESULT MEANS

Enumerate compatible starting rows; anchors can resolve lost constants, but the residual hierarchy supplies no missing observations.

MATHEMATICAL FORMULATION

$$\mathcal{F}_y = \{x : Dx = y\}, \quad D(x + c1) = Dx$$

Differences erase constant offsets. Additional anchors can reduce an inverse fiber; uniqueness must be checked under the declared domain and modulus.

EVIDENCE S033
PAPERS PAPER-002

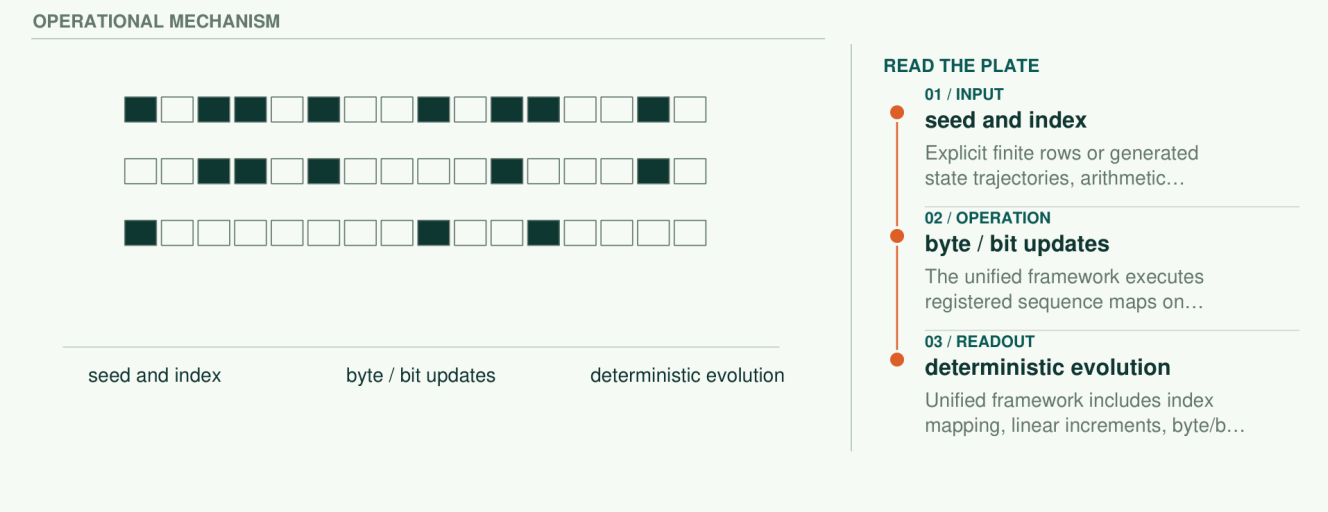
SECTION 15 / MODULAR DIFFERENCES AND GENERAL TRANSFORMATION FRAMEWORKS

Deterministic index, byte and bit evolution

M227

EVIDENCE STATUS IMPLEMENTED

FIGURE / METHOD MECHANISM



HOW IT WORKS

Define reversible or simple deterministic maps from an index into bitstrings, including increments and fixed byte/bit changes.

INPUTS NEEDED

Explicit finite rows or generated state trajectories, arithmetic conventions, transformation parameters and finite resource limits.

WHAT WAS ACTUALLY DONE

The unified framework executes registered sequence maps on artificial states. It generates controlled trajectories rather than learning the unknown puzzle creator's generation rule.

WHAT THE RESULT MEANS

Unified framework includes index mapping, linear increments, byte/bit steps and monotonic traversal; code is not an unknown-key predictor.

MATHEMATICAL FORMULATION

$$x_{t+1} = F_t(x_t), \quad x_0 = x_{\text{seed}}$$

A specified deterministic evolution generates controlled artificial trajectories. It does not infer an unknown real-world generator from its output hash.

EVIDENCE S226
PAPERS PAPER-018

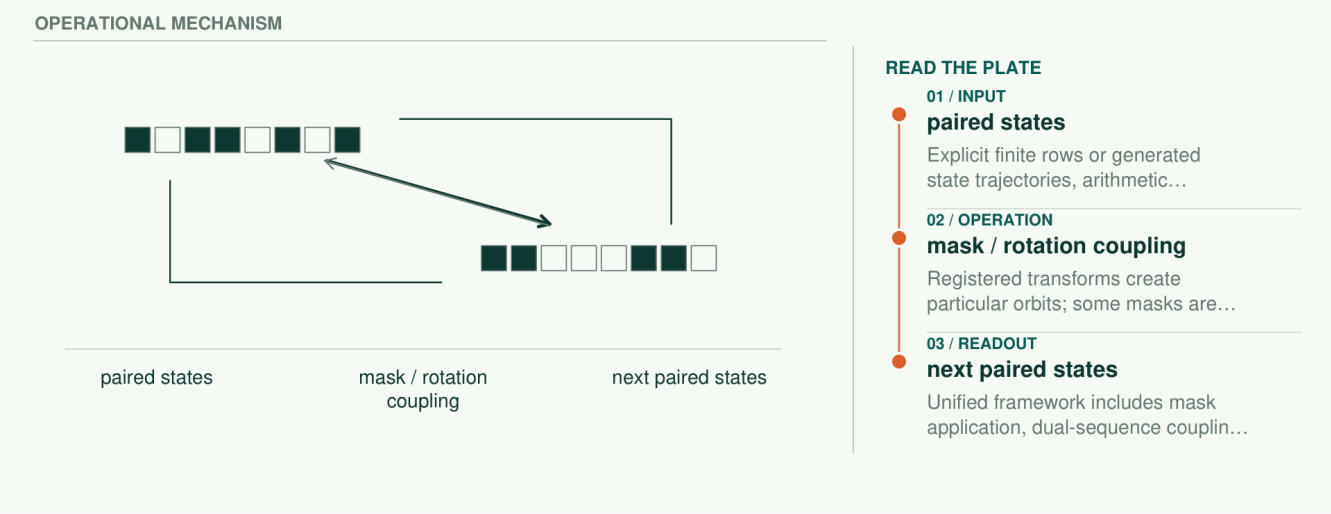
SECTION 15 / MODULAR DIFFERENCES AND GENERAL TRANSFORMATION FRAMEWORKS

Masking, rotation and coupled sequences

M228

EVIDENCE STATUS IMPLEMENTED

FIGURE / METHOD MECHANISM



HOW IT WORKS

Apply masks, rotate bit positions or couple two evolving states according to an explicit deterministic rule.

INPUTS NEEDED

Explicit finite rows or generated state trajectories, arithmetic conventions, transformation parameters and finite resource limits.

WHAT WAS ACTUALLY DONE

Registered transforms create particular orbits; some masks are lossy. Comparing their shapes cannot establish that a real target key belongs to one of those orbits.

WHAT THE RESULT MEANS

Unified framework includes mask application, dual-sequence coupling, mask rotation and structured 256-step schedules.

MATHEMATICAL FORMULATION

$$x' = \mathrm{rot}_b(x \oplus M, r)$$

This fixed XOR-mask/rotation example is reversible on b-bit words. AND masks are generally lossy; coupled-state transforms need their full joint update.

EVIDENCE S226

PAPERS PAPER-018

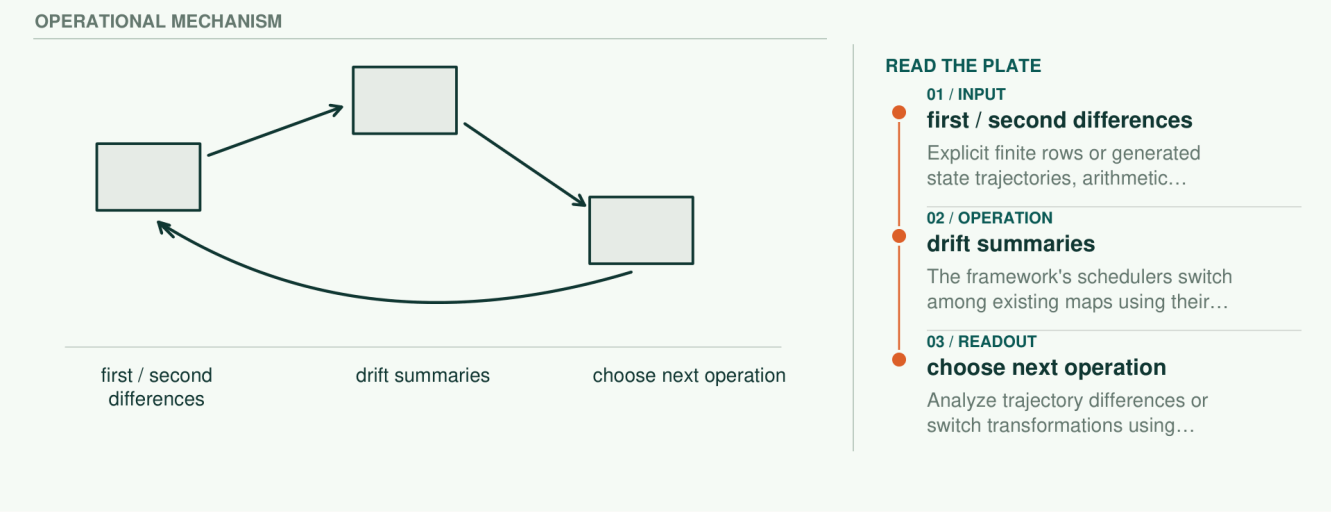
SECTION 15 / MODULAR DIFFERENCES AND GENERAL TRANSFORMATION FRAMEWORKS

First/second-difference and drift schedulers

M229

EVIDENCE STATUS IMPLEMENTED

FIGURE / METHOD MECHANISM



HOW IT WORKS

Measure first/second differences and choose the next transformation based on drift or entropy summaries.

INPUTS NEEDED

Explicit finite rows or generated state trajectories, arithmetic conventions, transformation parameters and finite resource limits.

WHAT WAS ACTUALLY DONE

The framework's schedulers switch among existing maps using their configured rules. Adaptation optimizes trajectory diagnostics; it is not rewarded by an independently discovered target-key relation.

WHAT THE RESULT MEANS

Analyze trajectory differences or switch transformations using entropy thresholds; diagnostic evolution is not key recovery.

MATHEMATICAL FORMULATION

$$\Delta x_t = x_{t+1} - x_t, \quad \Delta^2 x_t = \Delta x_{t+1} - \Delta x_t$$

Drift schedulers choose subsequent transformations from these summaries. Arithmetic must specify whether differences are ordinary signed or modular.

EVIDENCE S226
PAPERS PAPER-018

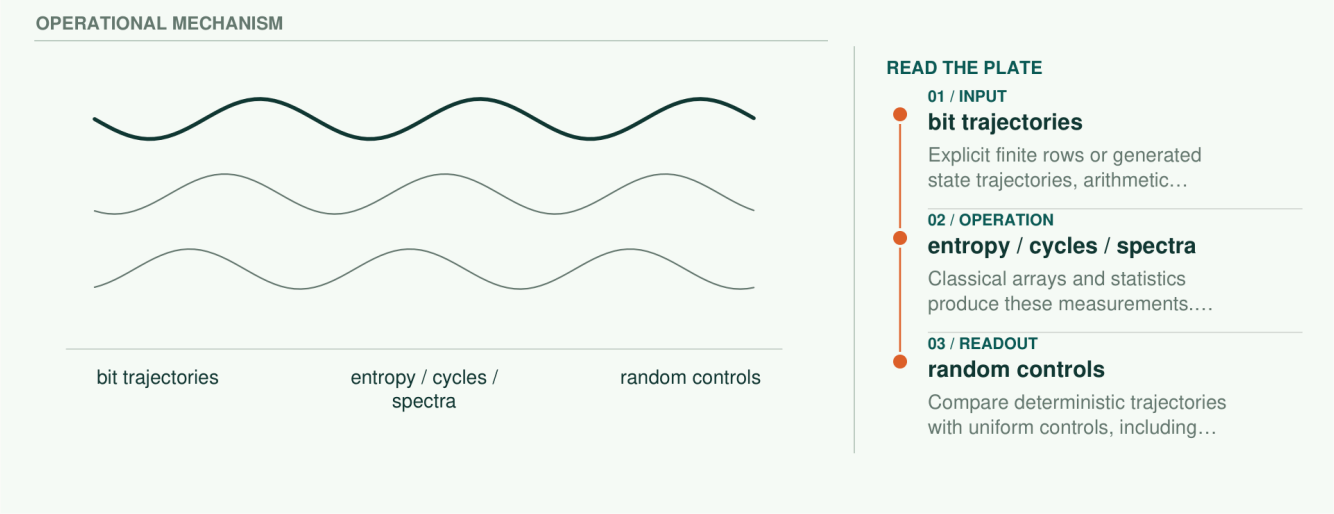
SECTION 15 / MODULAR DIFFERENCES AND GENERAL TRANSFORMATION FRAMEWORKS

Entropy, cycles, compression and spectral diagnostics

M230

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

Compute bit entropy, compression, cycles, nearest-neighbor spacings and spectral summaries, then compare with random trajectories.

INPUTS NEEDED

Explicit finite rows or generated state trajectories, arithmetic conventions, transformation parameters and finite resource limits.

WHAT WAS ACTUALLY DONE

Classical arrays and statistics produce these measurements. Quantum-chaos terminology denotes analogy/diagnostics, not a quantum computation or an inverse map from address hashes to private scalars.

WHAT THE RESULT MEANS

Compare deterministic trajectories with uniform controls, including cycle tests, spacings and spectral summaries; no quantum execution implied.

MATHEMATICAL FORMULATION

$$h(p) = -p\log_2 p - (1-p)\log_2 (1-p)$$

Binary marginal entropy h uses observed bit frequency p and 0 log 0=0. It is one diagnostic, distinct from joint entropy, compressibility, cycles and spectral summaries.

EVIDENCE S226 / S181
PAPERS PAPER-018

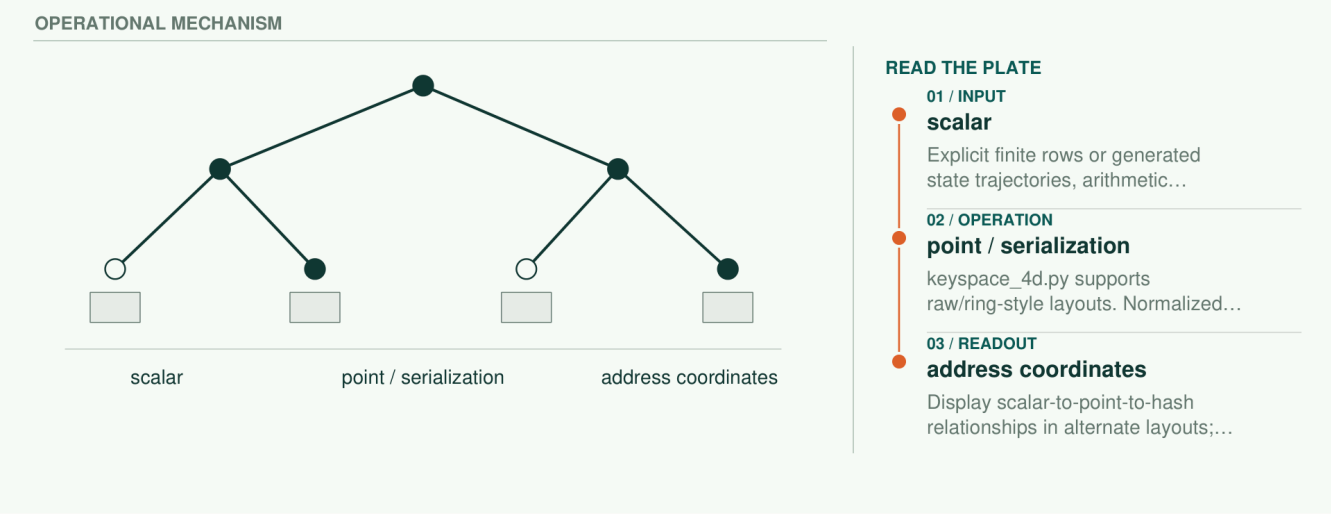
SECTION 15 / MODULAR DIFFERENCES AND GENERAL TRANSFORMATION FRAMEWORKS

Four-dimensional key/public/address visualization

M231

EVIDENCE STATUS IMPLEMENTED

FIGURE / METHOD MECHANISM



HOW IT WORKS

Place scalar, public-point and address-hash objects into shared visualization coordinates and connect their derivation edges.

INPUTS NEEDED

Explicit finite rows or generated state trajectories, arithmetic conventions, transformation parameters and finite resource limits.

WHAT WAS ACTUALLY DONE

keyspace_4d.py supports raw/ring-style layouts. Normalized coordinates can illustrate hashing's loss of visible geometry without revealing an exploitable inverse relationship.

WHAT THE RESULT MEANS

Display scalar-to-point-to-hash relationships in alternate layouts; no invertible address-to-scalar map is established.

MATHEMATICAL FORMULATION

$$k \longmapsto (k, kG, \mathrm{ser}(kG), \mathrm{HASH160}(\mathrm{ser}(kG)))$$

A visualization embeds these distinct derived objects into chosen display coordinates. Spatial proximity in that display is not a cryptographic inverse relation.

EVIDENCE S110

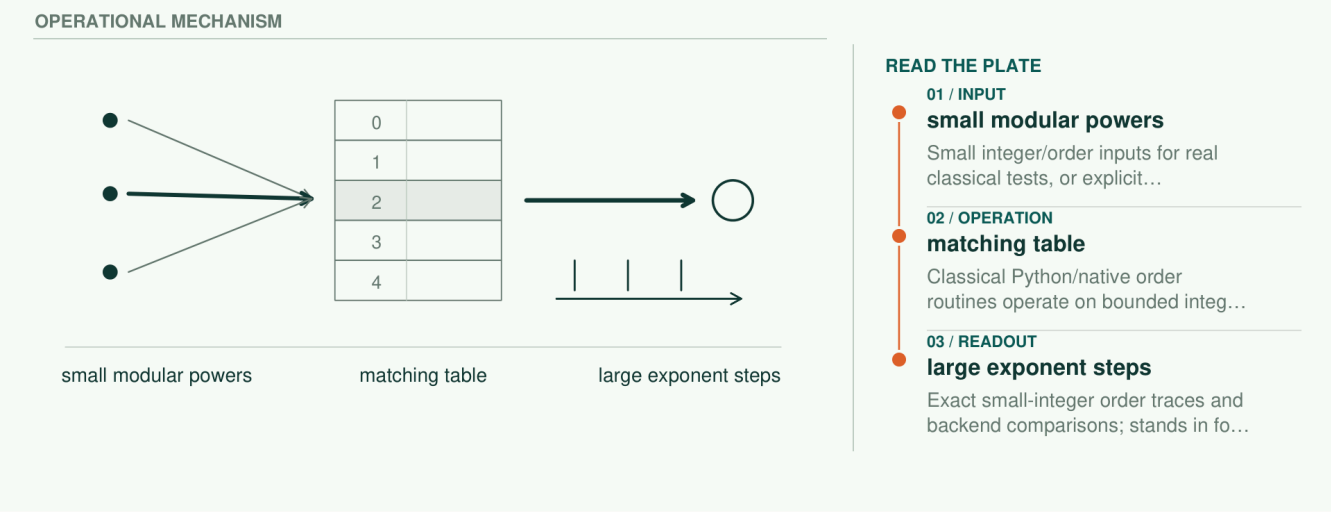
SECTION 16 / SHOR, CLASSICAL FACTORING AND QUANTUM-RESOURCE RESEARCH

Classical BSGS multiplicative-order finder

M232

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

BSGS finds an exponent r with a^r=1 modulo N by matching stored small powers against larger steps.

WHAT WAS ACTUALLY DONE

Classical Python/native order routines operate on bounded integer examples. In the factoring reduction they replace the quantum order-finding oracle; no QFT is executed.

INPUTS NEEDED

Small integer/order inputs for real classical tests, or explicit oracle/resource parameters for simulations and estimates.

WHAT THE RESULT MEANS

Exact small-integer order traces and backend comparisons; stands in for an order oracle, not a quantum device.

MATHEMATICAL FORMULATION

$$a^{im+j} \equiv 1 \pmod{N}, \quad m \approx \sqrt{R}$$

BSGS matches small powers with inverse giant steps to seek a positive order within bound R; require gcd(a,N)=1 and reduce a found multiple to the actual order.

EVIDENCE S264

PAPERS PAPER-003 / PAPER-021 / PAPER-022 / PAPER-027

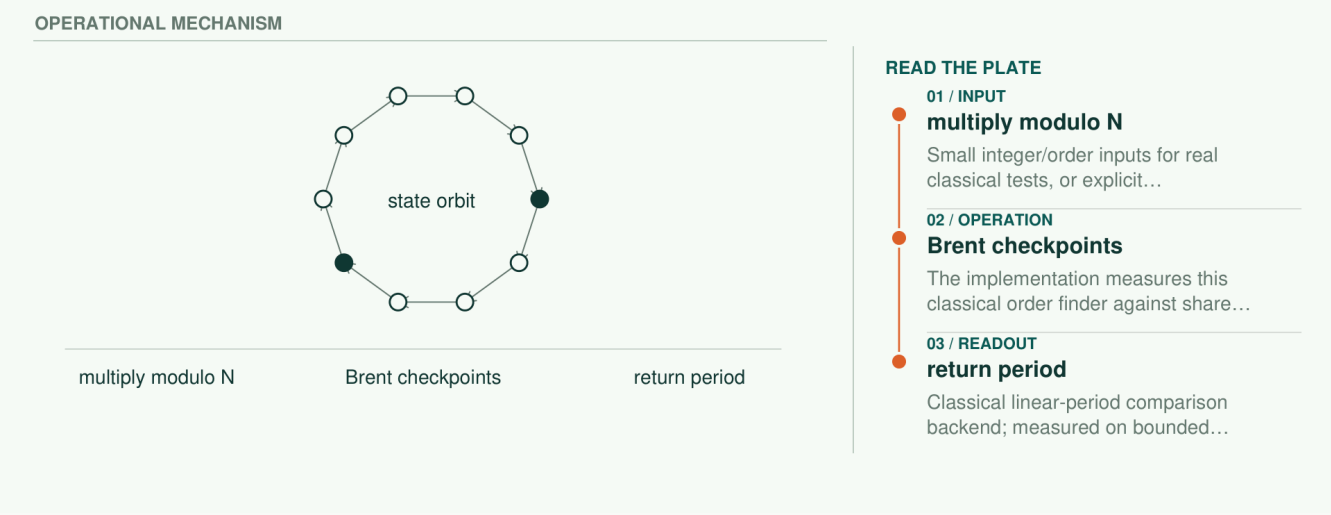
SECTION 16 / SHOR, CLASSICAL FACTORING AND QUANTUM-RESOURCE RESEARCH

Brent cycle-detection order finder

M233

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

Brent's cycle detector follows repeated multiplication modulo N while using a low-memory schedule to detect a return period.

INPUTS NEEDED

Small integer/order inputs for real classical tests, or explicit oracle/resource parameters for simulations and estimates.

WHAT WAS ACTUALLY DONE

The implementation measures this classical order finder against shared small inputs. Its runtime grows with the period and should not be mistaken for Shor's quantum complexity.

WHAT THE RESULT MEANS

Classical linear-period comparison backend; measured on bounded small semiprimes.

MATHEMATICAL FORMULATION

$$x_{t+1} = a x_t \pmod{N}, \quad x_0 = 1$$

For a coprime to N, this multiplication orbit has period ord_N(a). Brent's schedule detects its cycle using small memory and classical iteration.

EVIDENCE S263

PAPERS PAPER-003 / PAPER-021 / PAPER-027

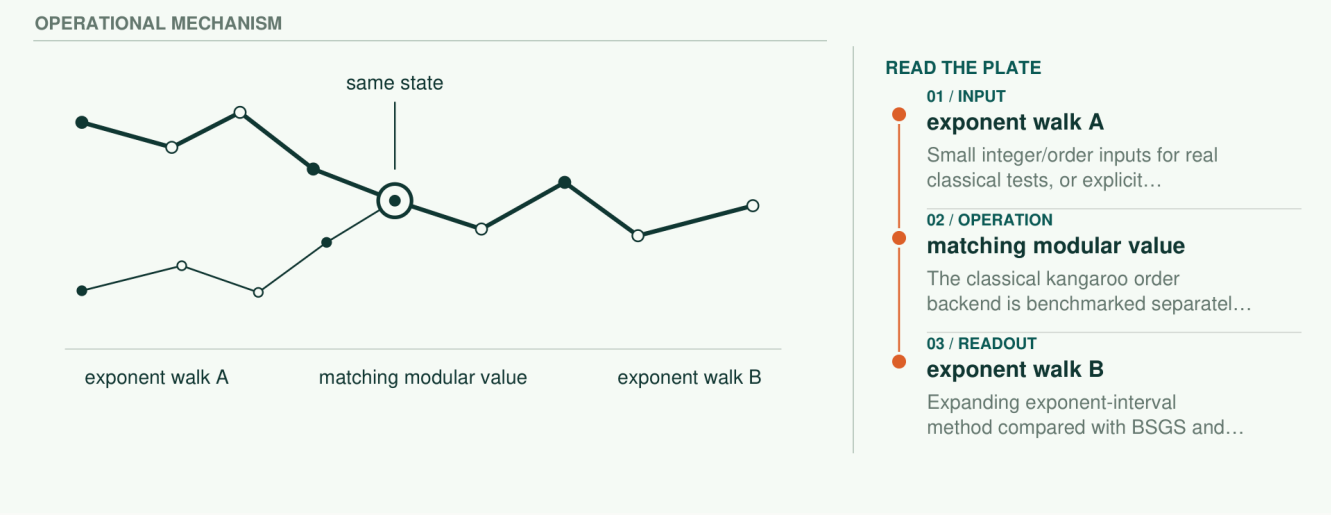
SECTION 16 / SHOR, CLASSICAL FACTORING AND QUANTUM-RESOURCE RESEARCH

Kangaroo multiplicative-order finder

M234

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

Search successively larger exponent intervals with collision walks to locate a multiple/order relation.

WHAT WAS ACTUALLY DONE

The classical kangaroo order backend is benchmarked separately from elliptic-curve kangaroo. Matching the name does not mean these factoring experiments recovered Bitcoin private keys.

INPUTS NEEDED

Small integer/order inputs for real classical tests, or explicit oracle/resource parameters for simulations and estimates.

WHAT THE RESULT MEANS

Expanding exponent-interval method compared with BSGS and Brent; not a secp256k1 scalar recovery.

MATHEMATICAL FORMULATION

$$a^t \equiv a^s \pmod{N} \quad \Longrightarrow \quad \mathrm{ord}_N(a) \mid (t-s)$$

A nontrivial collision gives an order multiple. Interval walks must find a nonzero exponent difference and then verify/reduce that multiple.

EVIDENCE S263

PAPERS PAPER-003 / PAPER-026 / PAPER-027

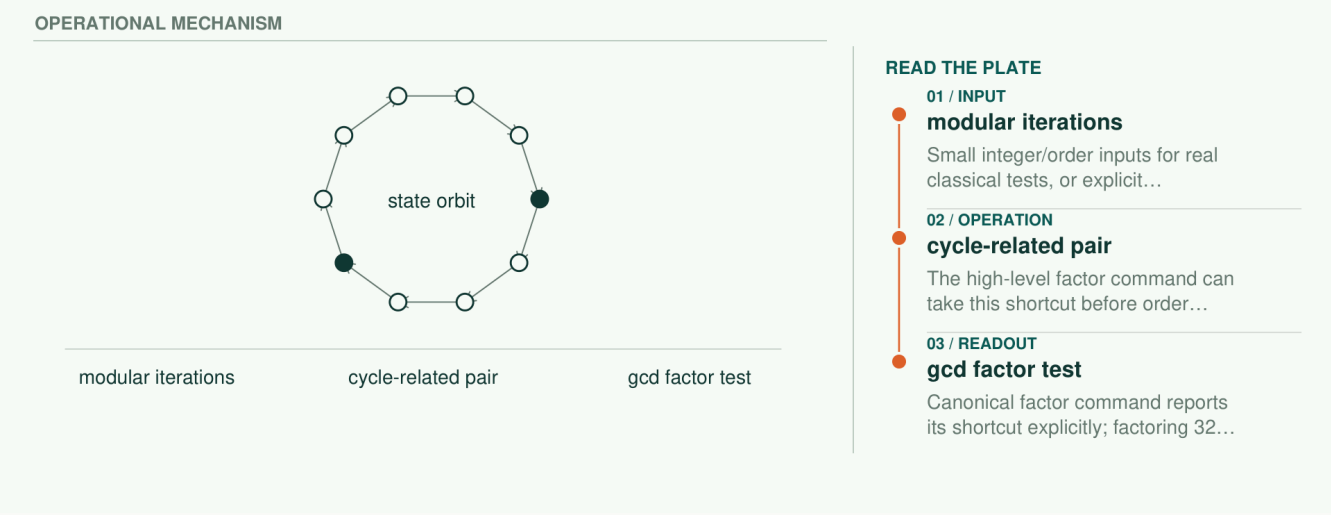
SECTION 16 / SHOR, CLASSICAL FACTORING AND QUANTUM-RESOURCE RESEARCH

Pollard-rho factoring fast path

M235

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

Pollard rho searches for a nontrivial common divisor through repeated modular iteration and cycle-related gcds.

INPUTS NEEDED

Small integer/order inputs for real classical tests, or explicit oracle/resource parameters for simulations and estimates.

WHAT WAS ACTUALLY DONE

The high-level factor command can take this shortcut before order finding. Its reported factorization of 3233 is correctly labeled rho, not evidence of quantum or even BSGS execution on that path.

WHAT THE RESULT MEANS

Canonical factor command reports its shortcut explicitly; factoring 3233 must not be relabeled as a quantum Shor run.

MATHEMATICAL FORMULATION

$$x_{t+1} = f(x_t) \pmod{N}, \quad d = \gcd(|x - y|, N)$$

Pollard rho seeks 1<d<N using suitably separated iterates x,y. This classical factoring path is distinct from quantum order finding.

EVIDENCE S264

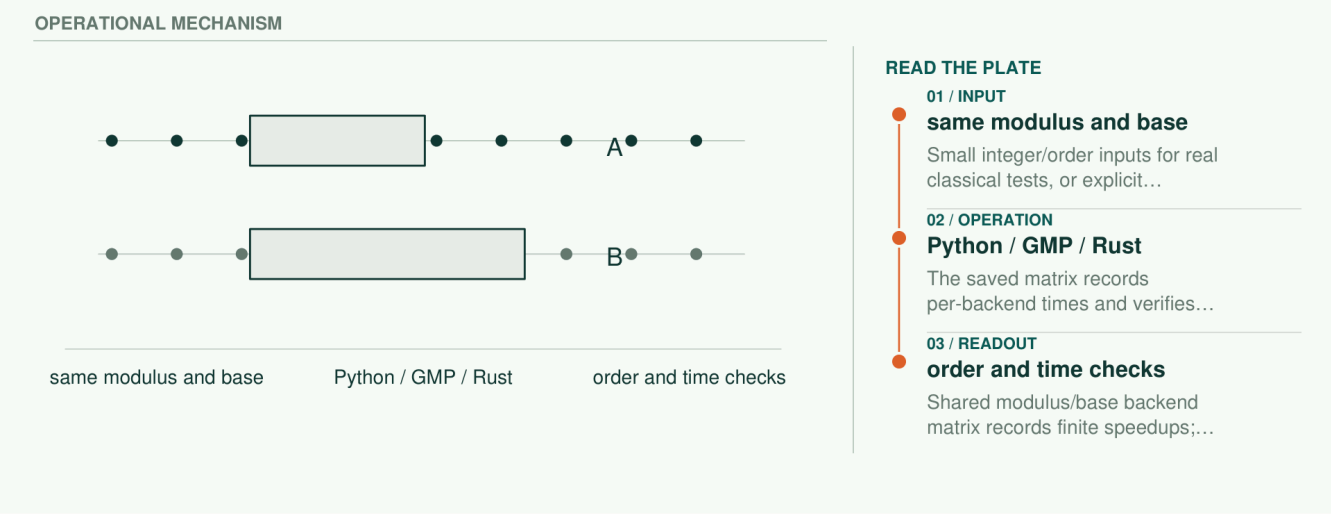
SECTION 16 / SHOR, CLASSICAL FACTORING AND QUANTUM-RESOURCE RESEARCH

Python/GMP/Rust controlled order benchmarks

M236

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

Hold modulus, base and repetition schedule fixed while replacing the order-finding arithmetic backend.

INPUTS NEEDED

Small integer/order inputs for real classical tests, or explicit oracle/resource parameters for simulations and estimates.

WHAT WAS ACTUALLY DONE

The saved matrix records per-backend times and verifies identical orders. This isolates local implementation constants; differences across operand sizes are not a clean universal scaling-law estimate.

WHAT THE RESULT MEANS

Shared modulus/base backend matrix records finite speedups; source/runtime differences are pinned in historical manifests.

MATHEMATICAL FORMULATION

$$S_b = \frac{t_{\text{reference}}}{t_b}, \quad r_b = r_{\text{reference}}$$

Compare backend times on identical modulus/base/order tasks while checking returned orders. Report operand sizes and repetition schedule.

EVIDENCE S262

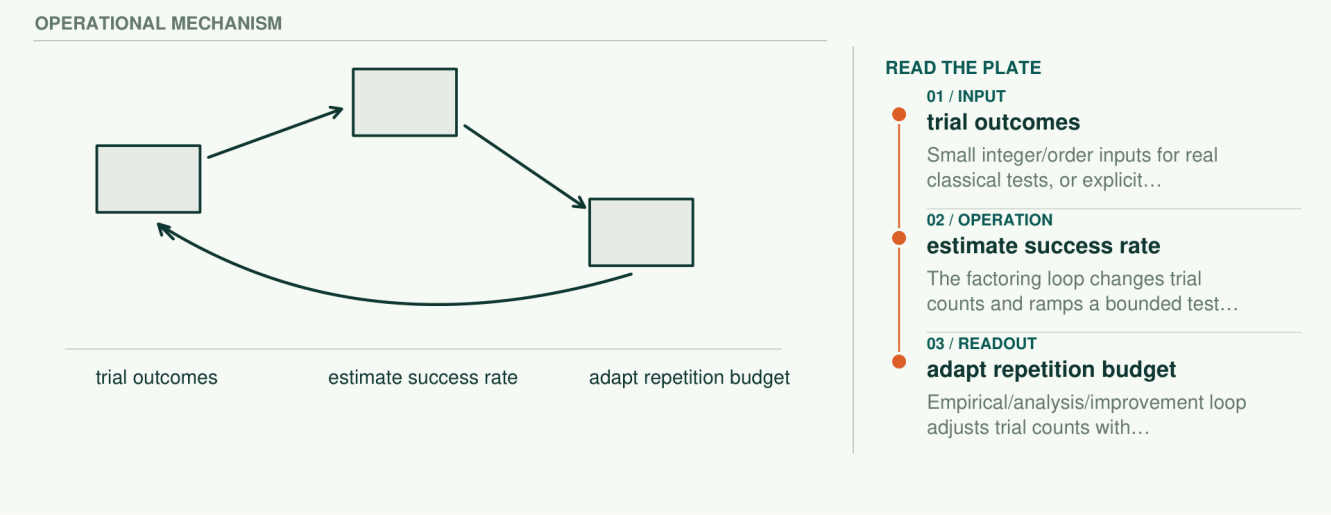
SECTION 16 / SHOR, CLASSICAL FACTORING AND QUANTUM-RESOURCE RESEARCH

Adaptive Shor simulation trial budgets

M237

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

Measure trial success, compute a confidence-aware required budget and adjust the number of repetitions to meet a target.

INPUTS NEEDED

Small integer/order inputs for real classical tests, or explicit oracle/resource parameters for simulations and estimates.

WHAT WAS ACTUALLY DONE

The factoring loop changes trial counts and ramps a bounded test size. The feedback improves a classical experimental procedure, not the underlying cryptographic problem's entropy.

WHAT THE RESULT MEANS

Empirical/analysis/improvement loop adjusts trial counts with Wilson-bound and runtime metrics.

MATHEMATICAL FORMULATION

$$B \geq \left\lceil \frac{\log(1-\tau)}{\log(1-p)} \right\rceil$$

Under independent trials with fixed success probability 0<p<1, B trials achieve success probability at least tau. An estimated p requires uncertainty and model calibration.

EVIDENCE S263 / S244

PAPERS PAPER-003 / PAPER-020 / PAPER-027

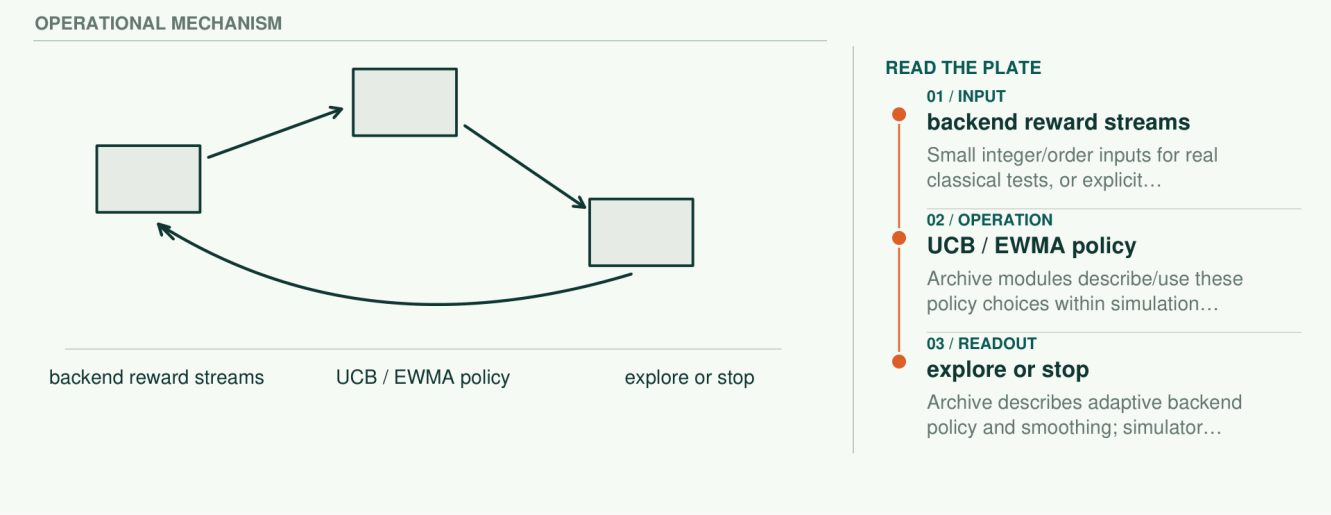
SECTION 16 / SHOR, CLASSICAL FACTORING AND QUANTUM-RESOURCE RESEARCH

UCB/EWMA/early-stop method selection

M238

EVIDENCE STATUS BRANCH/IMPLEMENTATION

FIGURE / METHOD MECHANISM



HOW IT WORKS

An upper-confidence policy explores backend choices while EWMA smooths noisy measurements and early stopping limits unnecessary trials.

INPUTS NEEDED

Small integer/order inputs for real classical tests, or explicit oracle/resource parameters for simulations and estimates.

WHAT WAS ACTUALLY DONE

Archive modules describe/use these policy choices within simulation loops. Better simulated reward can reflect assumptions in the model and needs real execution validation before any cryptanalytic interpretation.

WHAT THE RESULT MEANS

Archive describes adaptive backend policy and smoothing; simulator optimization is not target-scale quantum cryptanalysis.

MATHEMATICAL FORMULATION

$$U_i = \overline{r}_i + c\sqrt{\frac{\log t}{n_i}}, \quad \mu_t = \alpha r_t + (1-\alpha)\mu_{t-1}$$

UCB scores balance exploration and reward; EWMA smooths observations. Handle untried arms explicitly; the reward's meaning depends on the experiment.

EVIDENCE S255

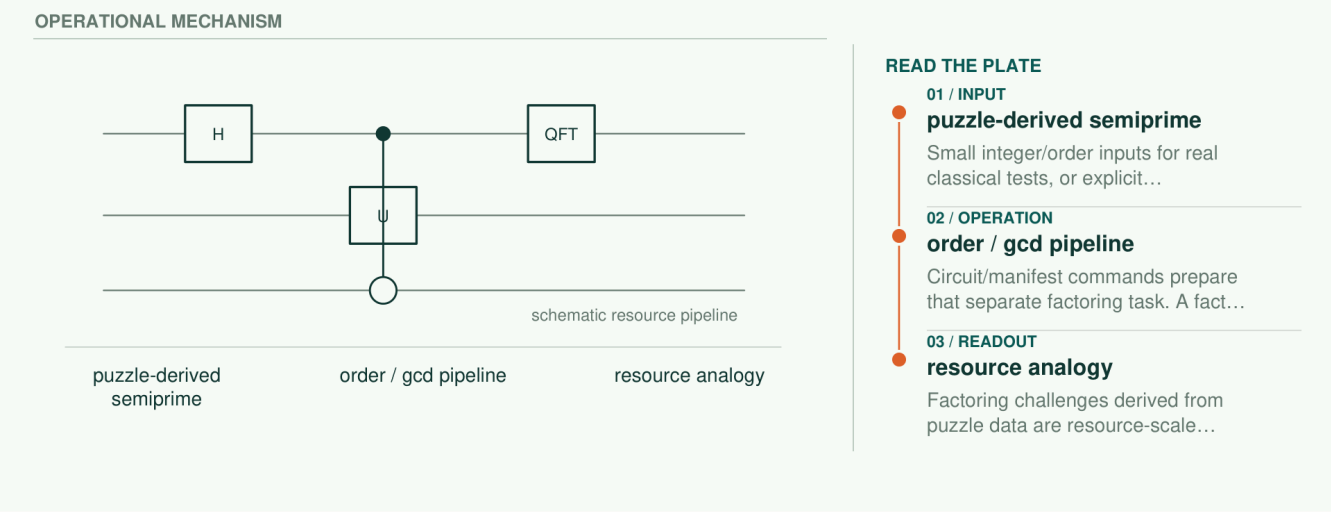
SECTION 16 / SHOR, CLASSICAL FACTORING AND QUANTUM-RESOURCE RESEARCH

Semiprime-per-puzzle circuit/manifest analogy

M239

EVIDENCE STATUS AUDITED/CORRECTED

FIGURE / METHOD MECHANISM



HOW IT WORKS

Construct a deterministic semiprime challenge from puzzle data to illustrate a comparable input size for factoring-resource calculations.

INPUTS NEEDED

Small integer/order inputs for real classical tests, or explicit oracle/resource parameters for simulations and estimates.

WHAT WAS ACTUALLY DONE

Circuit/manifest commands prepare that separate factoring task. A factor of the constructed integer is not mathematically shown to disclose the original elliptic-curve scalar.

WHAT THE RESULT MEANS

Factoring challenges derived from puzzle data are resource-scale analogies, not reductions recovering puzzle scalars.

MATHEMATICAL FORMULATION

$$N = pq, \quad \gcd(a^{r/2} \pm 1, N)$$

The factoring reduction uses an even multiplicative order r with nontrivial gcd outcomes. Factoring a semiprime constructed from puzzle data does not thereby yield the elliptic-curve scalar.

EVIDENCE S255

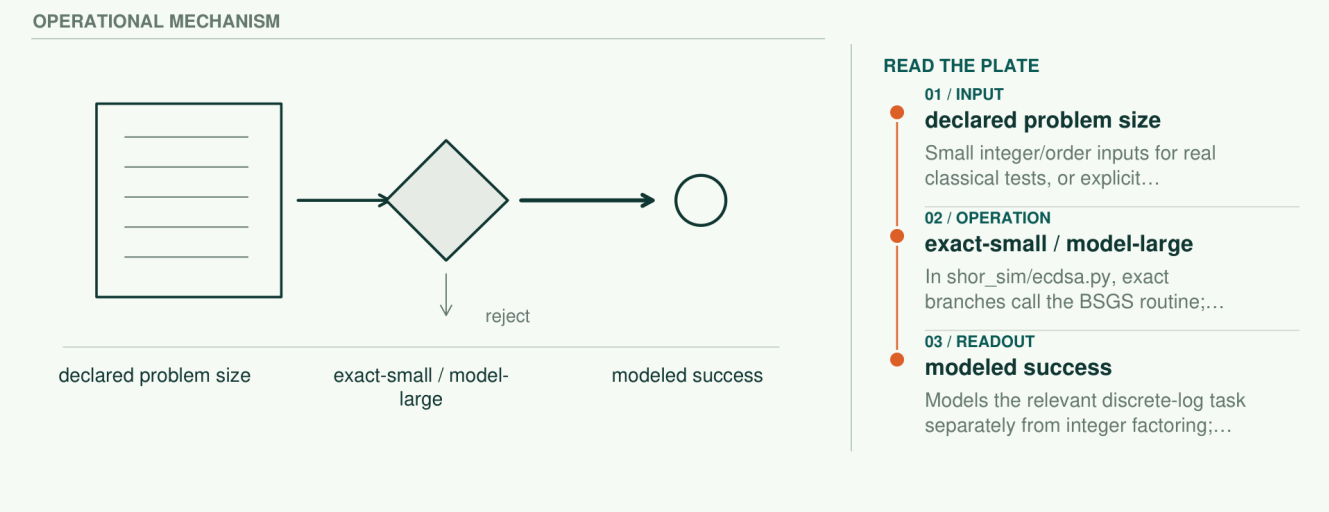
SECTION 16 / SHOR, CLASSICAL FACTORING AND QUANTUM-RESOURCE RESEARCH

secp256k1 quantum ECDLP resource mode

M240

EVIDENCE STATUS IMPLEMENTED/MODEL

FIGURE / METHOD MECHANISM



HOW IT WORKS

The ECDLP mode combines real small-range BSGS with a probability-and-time oracle model beyond configured exact limits.

INPUTS NEEDED

Small integer/order inputs for real classical tests, or explicit oracle/resource parameters for simulations and estimates.

WHAT WAS ACTUALLY DONE

In shor_sim/ecdsa.py, exact branches call the BSGS routine; larger Shor/hybrid trials draw success from rng.random() and return formula-based elapsed time. Even the small kangaroo-labeled profile calls BSGS in its exact branch.

WHAT THE RESULT MEANS

Models the relevant discrete-log task separately from integer factoring; no executed target-scale quantum circuit is established.

MATHEMATICAL FORMULATION

$$X = 1\{U < p_{\text{model}}\}, \quad U \sim \text{Unif}[0,1)$$

This describes the simulator's success draw beyond its exact small-range branch. A model draw is not an executed quantum computation or recovered target scalar.

EVIDENCE S243

PAPERS PAPER-022

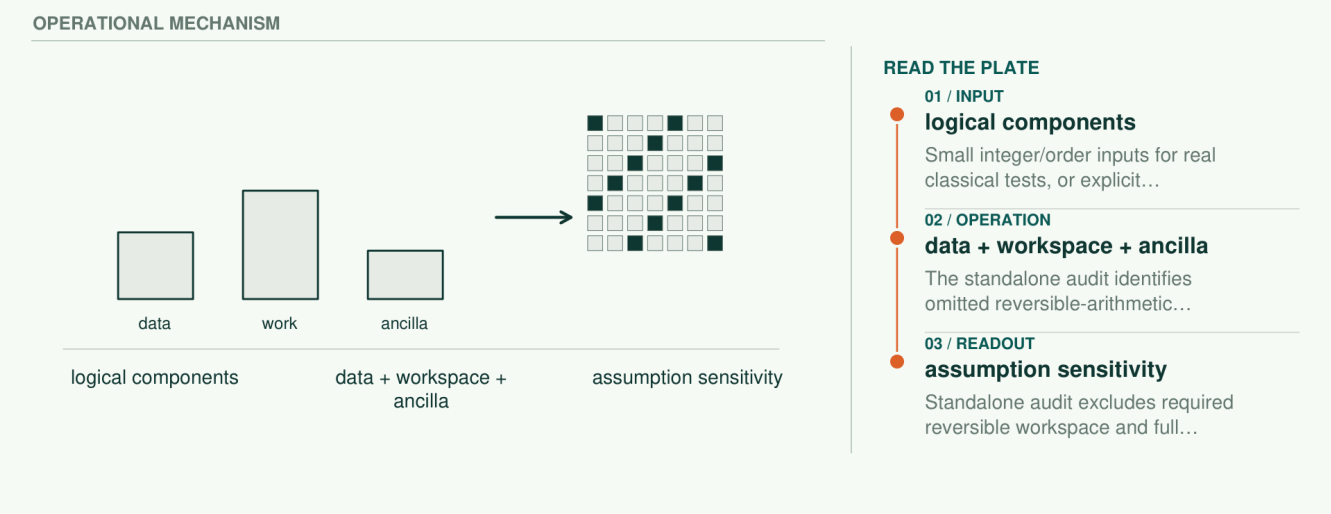
SECTION 16 / SHOR, CLASSICAL FACTORING AND QUANTUM-RESOURCE RESEARCH

520-logical-qubit resource sensitivity model

M241

EVIDENCE STATUS AUDITED/CORRECTED

FIGURE / METHOD MECHANISM



HOW IT WORKS

Estimate logical resources under a simplified model and vary physical error/architecture assumptions to see their effect.

INPUTS NEEDED

Small integer/order inputs for real classical tests, or explicit oracle/resource parameters for simulations and estimates.

WHAT WAS ACTUALLY DONE

The standalone audit identifies omitted reversible-arithmetic workspace and other architecture costs. A 520-qubit headline is a sensitivity-model output, not a complete executable Bitcoin-breaking circuit.

WHAT THE RESULT MEANS

Standalone audit excludes required reversible workspace and full architecture; headline counts are not a complete Bitcoin-breaking machine.

MATHEMATICAL FORMULATION

$$Q_{\text{logical}} = Q_{\text{data}} + Q_{\text{workspace}} + Q_{\text{ancilla}}$$

A complete logical-qubit budget must state which registers are included and whether lifetimes overlap. A sensitivity headline can omit load-bearing reversible-arithmetic workspace.

EVIDENCE S198

PAPERS PAPER-003 / PAPER-027

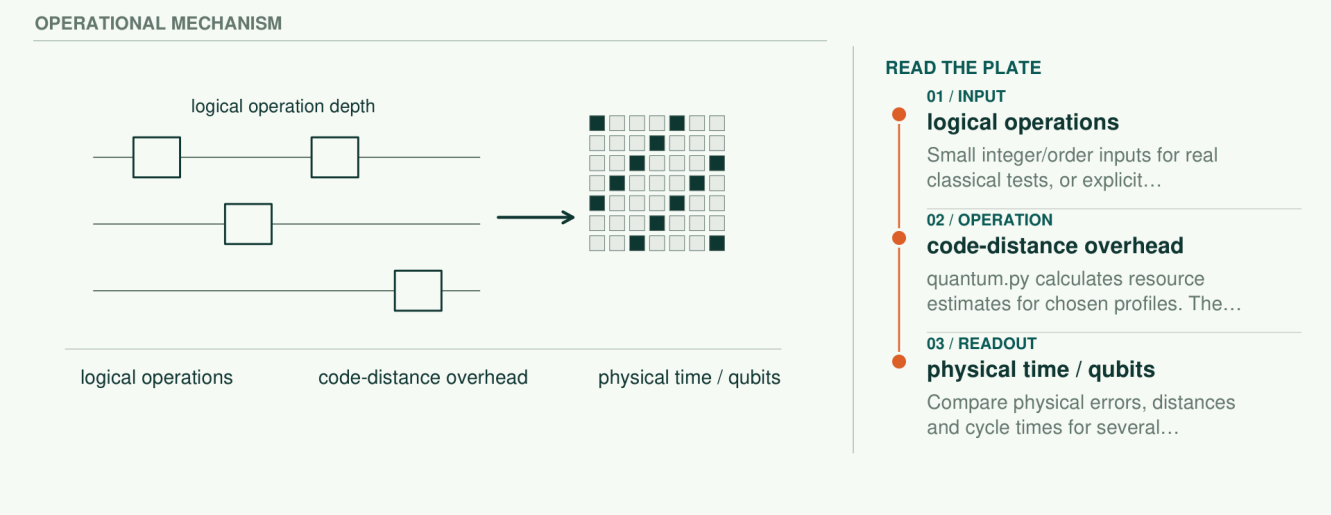
SECTION 16 / SHOR, CLASSICAL FACTORING AND QUANTUM-RESOURCE RESEARCH

Surface-code/platform timing estimates

M242

EVIDENCE STATUS IMPLEMENTED/MODEL

FIGURE / METHOD MECHANISM



HOW IT WORKS

Map logical operation counts to physical qubits and runtime using code distance, gate/cycle assumptions and platform parameters.

INPUTS NEEDED

Small integer/order inputs for real classical tests, or explicit oracle/resource parameters for simulations and estimates.

WHAT WAS ACTUALLY DONE

quantum.py calculates resource estimates for chosen profiles. The outputs are model-dependent arithmetic, not measured throughput on an available quantum computer.

WHAT THE RESULT MEANS

Compare physical errors, distances and cycle times for several platforms; estimates depend on explicit architecture assumptions.

MATHEMATICAL FORMULATION

$$t_{\text{wall}} \approx D_{\text{logical}}\,\tau_{\text{logical}}, \quad Q_{\text{physical}} = Q_{\text{logical}}\,\eta(d)$$

Logical depth, logical cycle time and code-distance overhead eta(d) determine resource-model estimates; scheduling and architecture assumptions control their accuracy.

EVIDENCE S245 / S255

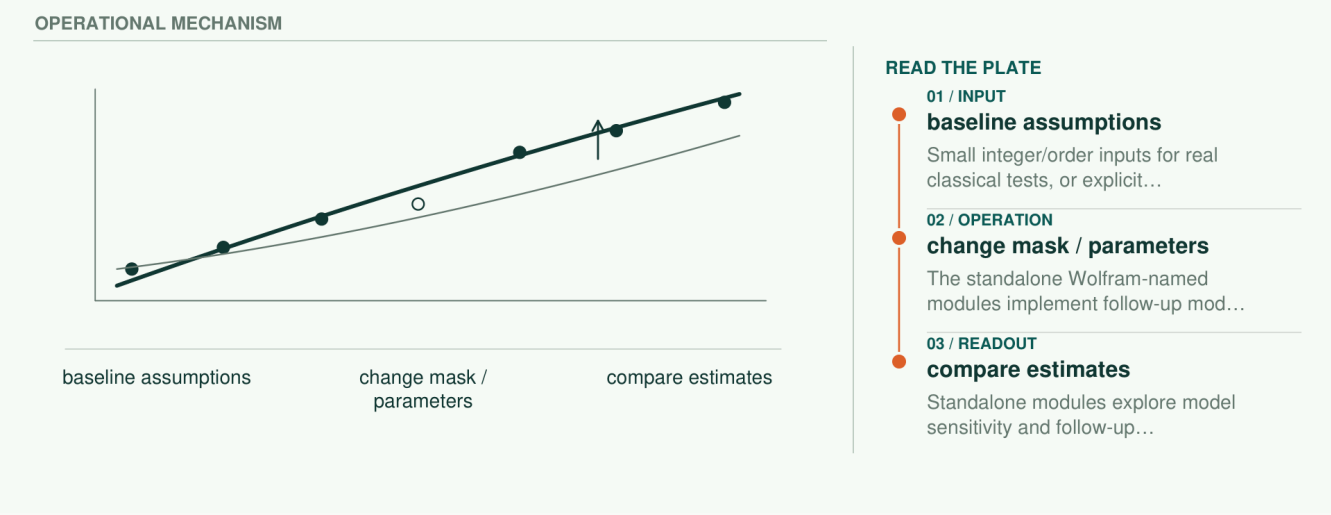
SECTION 16 / SHOR, CLASSICAL FACTORING AND QUANTUM-RESOURCE RESEARCH

Wolfram/mask sensitivity extensions

M243

EVIDENCE STATUS IMPLEMENTED/MODEL

FIGURE / METHOD MECHANISM



HOW IT WORKS

Feed alternative masks, assumptions or external analytical outputs into the resource/sensitivity model and compare derived estimates.

INPUTS NEEDED

Small integer/order inputs for real classical tests, or explicit oracle/resource parameters for simulations and estimates.

WHAT WAS ACTUALLY DONE

The standalone Wolfram-named modules implement follow-up model calculations. They do not construct or execute a target-sized quantum state evolution that reveals a puzzle key.

WHAT THE RESULT MEANS

Standalone modules explore model sensitivity and follow-up calculations, not recovered puzzle keys.

MATHEMATICAL FORMULATION

$$R(\theta + \delta\theta) - R(\theta)$$

A finite sensitivity change in assumptions theta measures how the resource model R responds. It is a comparison of modeled outputs, not quantum state evolution.

EVIDENCE S199 / S200

PAPERS PAPER-003 / PAPER-027

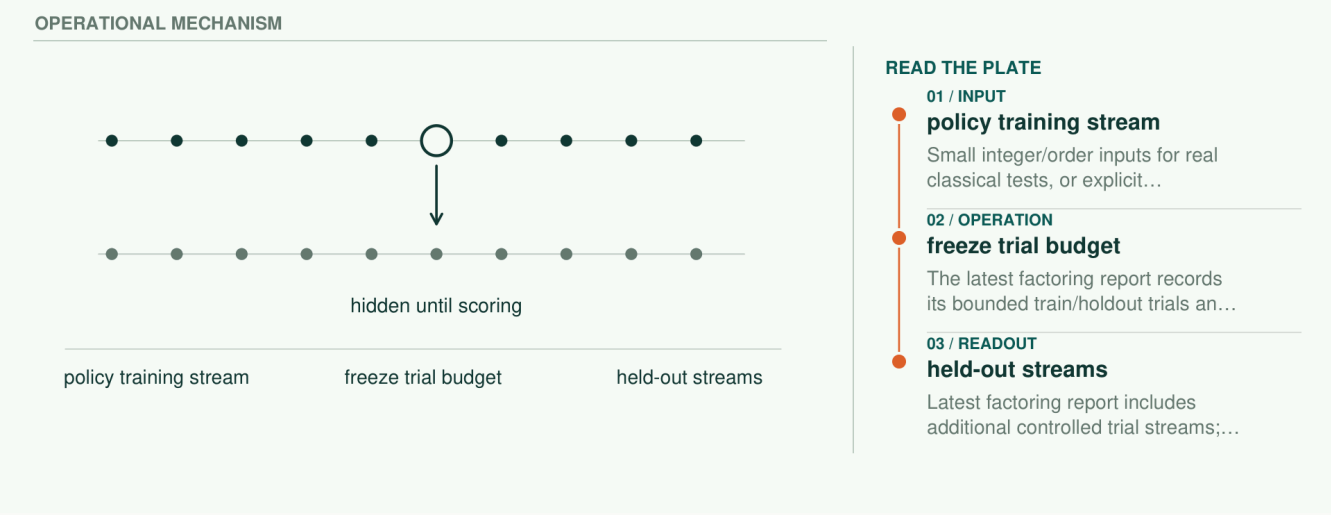
SECTION 16 / SHOR, CLASSICAL FACTORING AND QUANTUM-RESOURCE RESEARCH

Train/holdout adaptive-trial study

M244

EVIDENCE STATUS EXECUTED RECORD

FIGURE / METHOD MECHANISM



HOW IT WORKS

Freeze a trial-budget policy on one stream, then evaluate it on additional held-out random streams to check whether tuning generalizes.

INPUTS NEEDED

Small integer/order inputs for real classical tests, or explicit oracle/resource parameters for simulations and estimates.

WHAT WAS ACTUALLY DONE

The latest factoring report records its bounded train/holdout trials and limitations. Repeated success on small semiprimes validates that experiment, not quantum-scale secp256k1 recovery.

WHAT THE RESULT MEANS

Latest factoring report includes additional controlled trial streams; conclusions remain limited to classical simulation.

MATHEMATICAL FORMULATION

$$\widehat{p}_{\text{test}} = \frac{1}{m}\sum_{i=1}^{m} 1\{\text{success}_i \mid \theta_{\text{frozen}}\}$$

Freeze the trial policy after training, then evaluate on independent holdout streams. Small-instance experimental success does not establish target-scale quantum recovery.

EVIDENCE S263

PAPERS PAPER-003 / PAPER-020 / PAPER-027

PAPER LITERATURE

Research papers in the inventory

27 non-empty local papers are cataloged here. The paper IDs printed on method cards refer to these entries. Abstracts state the papers' own claims; later corrections and method evidence labels remain controlling.

PAPER-001 The Arithmetic of C-Theory Which Bitcoin Puzzle Regularities Are Theorems, and What a Cubic Tower Can Cost

Cory L. Thorbeck / 12 pages

papers/ctheory_arithmetic.pdf

PAPER-STATED ABSTRACT

C-Theory re-encodes a Bitcoin puzzle private key k as a nested tower of cubic forms, k = R3 1 + C1R2 1 + D1 with R1 = R3 2 + C2R2 2 + Δ2 and deterministic anchors Sℓ= ⌊2(N−1)/3ℓ⌋, and reports that a battery of “universal laws” holds on all 82 solved keys. This paper supplies the missing arithmetic. We prove that the tower is a mixed-radix positional representation of k in a key-dependent base: the coefficient bound C ∈{0, 1, 2, 3} and the remainder bound D < R2 are theorems about integers, tight and independent of how the key was generated, and the maximal coefficient occurs with probability below 1/R. We prove that iterated cube roots collapse, Sℓ= ⌊k1/3ℓ⌋, so the anchor tower is a single radical and the deterministic anchor formula is exact precisely when no perfect 3ℓ-th power lies in (2N−1, k]; this predicts the lone observed miss (S3 at N = 63, where 527 falls inside the tier and breaks the top 38.4% of it) and explains why levels 4–8 match on 82/82: they are constant across each entire tier. Five draws of uniform random keys reproduce every Tier I law at the same rates, and Kolmogorov–Smirnov and χ2 tests on the normalised digits reject nothing at the 1% level, so the corpus evidence for designed structure is empty. We then price the tower. Covering an M-bit interval with slots of width w and one interval kangaroo per slot costs κ2M/√w operations, so exhausting the cubic tower costs 22M/3 against 2M/2 for a single full-interval kangaroo: 290.1 versus 268.1 at puzzle #135. A hunt breaks even only by supplying M/6 bits of correct prior information about R1 (22.3 bits at #135), which is also the largest speedup a perfect anchor oracle can ever buy.

METHODS M040

PAPER LITERATURE / CONTINUED

PAPER-002 C-Theory: A Lossless Five-Level Cubic Hierarchy for Bitcoin Puzzle Private Keys Structural Decomposition, Deterministic Anchors, and Falsification-Driven Search Planning

Cory L. Thorbeck / 40 pages

papers/ctheory_main.pdf

PAPER-STATED ABSTRACT

We introduce C-Theory, a five-level hierarchical cubic decomposition of secp256k1 private keys used in the Bitcoin puzzle challenge series. Given a solved puzzle scalar k at bit-length tier N, C-Theory re-encodes k as a nested tower of cubic forms k = R3 1 + C1R2 1 + D1 with R1 = R3 2 + C2R2 2 + Δ2 and three further anchor levels S3–S5, each ter- minating in deterministic puzzle-tier anchors Sℓ= ⌊2(N−1)/3ℓ⌋. On a corpus of 82 solved puzzle keys (N = 1..130), round-trip reconstruction is exact in 82/82 cases (Evidence Tier I). For hunt tiers N ≥67, coefficients satisfy C1, C2 ∈{0, 1, 2} with D1 < R2 1 and Δ2 < R2 2 in all 16 keys. Deterministic anchors S4–S8 match exactly on 82/82 keys; S3 matches on 81/82 (single miss at N = 63). We show that C-Theory is a lossless re-indexing, not an ECDLP break: given k, reconstruction is trivial; predicting an unsolved key requires separate Tier II heuristics (FUSED R2 ensemble, two-phase lattice narrowing) whose leave-one-out hit rates we report honestly (53.5% within ±1% for R2 prediction). For puz- zle #135—solved on 2026-07-30 with private key 0x6d9392...07eb2—anchors collapse to {S3:31, S4:3, S5..S8:1}, yielding a feasible R2 lattice of 2,429 values. FUSED R2 ≈31,341 was −2.69% from the true R2 = 32,207 (rank 1 732 by absolute distance, but inside the ±3% win- dow). Phase 3 recovery via direct D1 Pollard kangaroo costs ~247 group operations per slot, superseding an earlier wrapped-CT estimate of ~228.9 that we falsified (0/27 live hits at N ≥105). All results are reproducible from the open-source distinguished-kangaroo repository with 376 automated tests.

METHODS M025, M026, M027, M035, M038, M039, M045, M056, M088, M089, M090, M101, M120, M132, M170, M207, M226

PAPER-003 Classical Order Finding at the Edge of the Shor Simulation Regime: An Adaptive BSGS Benchmark with a 520-Qubit Sensitivity Model

Cory L. Thorbeck / 14 pages

papers/main.pdf

PAPER-STATED ABSTRACT

We present a reproducible classical order-finding benchmark and an adaptive experiment framework for simulating the period-finding subroutine of Shor’s algorithm. The primary contribution is empirical: an iterative-deepening baby-step/giant-step (BSGS) implemen- tation, checked against direct order enumeration, together with a closed-loop procedure for tuning trial counts and problem size. The experiments establish bounded performance of this implementation; they do not establish the difficulty of integer factorisation against number-field-sieve or other classical algorithms. For comparison, we retain a transparent 520-logical-qubit parameter in a separate sensi- tivity model. It is the chosen textbook register expression Qreg = 2b + ⌈log2 b⌉at b = 256, not a universal minimum, a complete reversible circuit, or a physical-machine prediction. The shor-sim system couples three subprocesses—empirical Monte-Carlo execution, closed-form statistical analysis, and rule-based parameter improvement—in a closed loop that scales the target semiprime as performance permits. Its optimised baby-step/giant-step (BSGS) order finder runs in O(√r) time, outperforms Brent cycle detection by orders of magnitude where both are tractable, and enables practical simulation to 36 bits in the reproducible benchmark configuration on a single CPU node. The displayed adaptive success counters include classical shortcuts and are not used as a shortcut-free estimate of Shor-trial reliability. The quantum section reports only conditional arithmetic outputs from this parameterised model: 520 logical register qubits, 650,000 data-patch units under the stated convention, and a cycle-limited service-rate scenario; the single-factory diagnostic remains undetermined.

METHODS M232, M233, M234, M237, M241, M243, M244

PAPER LITERATURE / CONTINUED

PAPER-004 Deterministic Anchor Towers in C-Theory: Puzzle-Tier Geometry Without Key Leakage

Cory L. Thorbeck / 25 pages

papers/series/paper02_anchor_tower.pdf

PAPER-STATED ABSTRACT

We study deterministic anchor towers in C-Theory: values Sℓ= ⌊2(N−1)/3ℓ⌋for ℓ∈ {3, . . . , 8} that depend only on puzzle tier N, not on the private key k. On 82 solved Bitcoin puzzle keys, anchors S4–S8 match exactly in 82/82 cases; S3 matches in 81/82 (single miss at N=63). At N=135 the tower collapses to {S3:31, S4:3, S5:1, S6:1, S7:1, S8:1}, providing tier indexing without additional search freedom. We prove no key leakage from published anchors and document that puzzle #135 remains unsolved.

METHODS M025, M030, M035, M038, M045, M088, M089, M090, M205

PAPER-005 FUSED R_2: An Ensemble Predictor for C-Theory Second-Level Residues

Cory L. Thorbeck / 18 pages

papers/series/paper03_fused_r2.pdf

PAPER-STATED ABSTRACT

We present FUSED R2, an ensemble predictor for C-Theory second-level residues fused_r2(N) = mean(gap5, gap15, 20.1115N−0.0871, 2(N−1)/9). Leave-one-out validation on 43 puzzles (N ≥40) yields mean absolute error 2.56%, with only 53.5% of keys landing in a ±1% window (Tier II—insufficient alone for production narrowing). At N=135, FUSED R2 ≈31,341 ranks first in the feasible lattice. Puzzle #135 remains UNSOLVED.

METHODS M025, M035, M038, M045, M056, M088, M089, M090

PAPER-006 Feasible R_2 Lattices: Constraint-Based Enumeration for C-Theory Phase 1 Search

Cory L. Thorbeck / 255 pages

papers/series/paper04_feasible_r2_lattice.pdf

PAPER-STATED ABSTRACT

We enumerate all R2 values consistent with fixed S3 anchor and cubic band con- straints (γ3, δ3) for puzzle tier N. At N=135 with S3=31, the lattice contains 2,429 distinct feasible R2 values in [30,339, 32,767]. FUSED ranking places TRAILING-derived R2=32,003 at rank 1324 (+2.11%). Puzzle #135 remains UNSOLVED.

METHODS M024, M025, M035, M038, M041, M045, M088, M089, M090

PAPER LITERATURE / CONTINUED

PAPER-007 Two-Phase R_2 Narrowing: Composable Filters with Quantified LOO Risk

Cory L. Thorbeck / 24 pages

papers/series/paper05_two_phase_narrowing.pdf

PAPER-STATED ABSTRACT

Two-phase R2 narrowing unions Phase 1 (γ3=1∩FUSED±1%, 626 values) and Phase 2 (γ3=2∩FUSED±3%\ Phase 1, 569 values) for 1,195 total (50.8% reduction from 2,429). LOO on N≥75 (12 puzzles): 6/12 for two_phase vs 4/12 for FUSED±1% alone. De- fault GPU narrow export: 96 slots, ~347h at published external RTX 4090 throughput; provider price required.

METHODS M025, M035, M038, M045, M050, M088, M089, M090

PAPER-008 CLI Fingerprints: Unique Cubic Decomposition Signatures Across 82 Bitcoin Puzzle Keys

Cory L. Thorbeck / 22 pages

papers/series/paper07_cli_fingerprint.pdf

PAPER-STATED ABSTRACT

CLI ngerprint tuple (γ3, C1, C2, quantile(D1/R2 1), popcount(D1), D1∧0xFFFF) is unique on 82/82 keys (Tier I). Popcount ±15% narrowing: 5/16 hit falsi ed for search (Tier III). Valid uses: cataloging, deduplication, audit trails.

METHODS M013, M035, M038, M045, M090, M098, M120

PAPER-009 Relation Bundles: A Typed Evidence Schema for Bitcoin ECDLP Hunt Campaigns

Cory L. Thorbeck / 16 pages

papers/series/paper08_relation_bundle.pdf

PAPER-STATED ABSTRACT

Relation Bundle (RB) is a JSON artifact linking distinguished-point metadata, TRAIL- ING bit splits, C-Theory hypotheses, and ranked search windows. TRAILING-as-#135-key: FALSE (Tier III). Schema version in relation_bundle.py; enables reproducible GPU eet manifests.

METHODS M036, M150, M169

PAPER-010 Bounded ECDLP Certificates: Tamper-Evident Audit Envelopes for Structured Kangaroo Hunts

Cory L. Thorbeck / 15 pages

papers/series/paper09_bec.pdf

PAPER-STATED ABSTRACT

Bounded ECDLP Certi cate (BEC) wraps GPU hunt manifests with corpus version hash, sigma calibration references (FUSED, D1 bands), per-slot attestations, and embed- ded falsi cation registry (wrapped CT, TRAILING-as-key, popcount narrowing). Bridges Tier I/II/III research claims to operational con gs.

METHODS M023, M028, M036

PAPER LITERATURE / CONTINUED

PAPER-011 ECDLP Reduction Window Ranking: Cost-Ordered Search from Distinguished-Point Metadata

Cory L. Thorbeck / 22 pages

papers/series/paper10_ecdlp_reduction.pdf

PAPER-STATED ABSTRACT

We rank candidate search windows from TRAILING metadata by estimated Pollard kangaroo cost. 66-bit TRAILING window vs full 256-bit: ~295 ops speedup estimate. GLV: 0/6 solved keys N≥105 have both halves < 64 bits. 11 DP x-coordinates; 0 collisions. Metadata-as-key: Tier III falsi ed.

METHODS M021, M022, M024, M036, M041, M132, M165

PAPER-012 Wrapped CT Kangaroo: A Negative Result for Clustered D_1 Search at N≥105

Cory L. Thorbeck / 28 pages

papers/series/paper11_wrapped_ct_falsified.pdf

PAPER-STATED ABSTRACT

Wrapped CT clustered shifted D1 residues into span ≈3R1W with optimistic ~228.9 ops/slot. Live hunt: 0/27 hits at W=7 for N≥105. Corrected costing: ~247 ops/slot via direct D1 kangaroo. Algebraic structure on historical puzzles ≠ automatic compression for future puzzles.

METHODS M026, M027, M035, M045, M152

PAPER-013 Strategic Δ_2 Hints: Sequence-Based Priors for C-Theory Phase 2 Enumeration

Cory L. Thorbeck / 24 pages

papers/series/paper12_strategic_delta2.pdf

PAPER-STATED ABSTRACT

Δ2 candidate prioritization uses trailing-prime decomposition and gap-5 R2 growth ratios. True Δ2 for #130 included in hint set. TRAILING Δ2 for #135: 55,703,163 (metadata). Phase 2 feasibility: ~59M ops per slot. Puzzle #135 UNSOLVED.

METHODS M035, M041, M045, M058

PAPER-014 Bit-Flip Enumeration for Address-Only Bitcoin Puzzles: A Systematic Falsification Study

Cory L. Thorbeck / 19 pages

papers/series/paper13_bit_flip_hunt.pdf

PAPER-STATED ABSTRACT

Six alternative key enumeration strategies for puzzle #71 (no public key): Gray-code walks, BFS Hamming balls, TRAILING-mask ips, FUSED-derived D1 ips. LOW71±24b scan: 16.8M keys, 0 hits. Puzzle #71 remains unsolved at ~270.5 classical complexity.

METHODS M002, M003, M035, M083, M109, M110

PAPER LITERATURE / CONTINUED

PAPER-015 The Creator-Method Oracle: Falsifying Shared Generators in the Bitcoin Puzzle Series

Cory L. Thorbeck / 40 pages

papers/series/paper14_creator_oracle.pdf

PAPER-STATED ABSTRACT

We catalog fast deterministic generators and test whether any ts keys 1..70 and predicts 75..130. No generator found. Evidence compatible with independent uniform draws per tier. 849-line audit in analyze_creator_generator.py. Rules out large class of creator- backdoor hypotheses.

METHODS M035, M087, M088, M089

PAPER-016 A Falsification-Driven Hypothesis Framework for Bitcoin Puzzle Research

Cory L. Thorbeck / 25 pages

papers/series/paper15_hypothesis_framework.pdf

PAPER-STATED ABSTRACT

Constraint-graph hypothesis runner separates ECDSA algebraic constraints (1 DOF) from puzzle key-generation statistics. 35 distinct theories tested: ~1 3 Tier I, ~1 3 Tier II, ~1 3 Tier III. Evidence tier taxonomy as scienti c practice in open cryptanalytic research.

METHODS No method assigned by the matching pass.

PAPER-017 Cubic Primitive Layers: From C-Theory Structural Laws to Composable ECDLP Search Artifacts

Cory L. Thorbeck / 16 pages

papers/series/paper16_cubic_primitive_layers.pdf

PAPER-STATED ABSTRACT

Five composable layers: L0 CLI codec (Tier I), L1 tier surface Σ(N) (Tier II), L2 Hunt Slot, L3 Relation Bundle, L4 BEC, L5 distributed orchestration (future). 79 distinct tier- code classes for 82 keys. L2 L4 implemented.

METHODS No method assigned by the matching pass.

PAPER-018 A Unified Keyspace Framework for Deterministic secp256k1 Trajectory Analysis

Cory L. Thorbeck / 25 pages

papers/series/paper17_unified_keyspace.pdf

PAPER-STATED ABSTRACT

18 callable transformations Fk over 256-bit state Si = (Hi, Bi, Xi, Mi) with schedulers and uniform-random null model. ~900 lines in unified_keyspace_framework_algorithm.py. No dedicated pytest; not applied to solve puzzles rigorous trajectory vs null-model tool.

METHODS M227, M228, M229, M230

PAPER LITERATURE / CONTINUED

PAPER-019 An Integrated Distinguished-Point Pipeline for Bitcoin Puzzle ECDLP Recovery

Cory L. Thorbeck / 33 pages

papers/series/paper18_kangaroo_pipeline.pdf

PAPER-STATED ABSTRACT

End-to-end integration: DP parse →tame/wild classi cation →collision recovery → JeanLucPons/RCKangaroo export. √-law calibration: ops ≈2W/2+1.06 (puzzles #65 #130). 0 collisions on 11-DP artifact. Puzzle #135 UNSOLVED.

METHODS M021, M022, M024, M035

PAPER-020 Autonomous Self-Improving Classical Simulation of Shor's Algorithm

Cory L. Thorbeck / 12 pages

papers/series/paper19_shor_loop.pdf

PAPER-STATED ABSTRACT

Closed loop: empirical Monte-Carlo →Wilson-bound analysis →rule-based tuning → bit-length ramp (+1 bit/generation). Maintains aggregate success ≥0.92 while scaling to 28+ bits. Note: separate from main C-Theory line; see also papers/main.tex.

METHODS M070, M237, M244

PAPER-021 Iterative-Deepening Baby-Step Giant-Step for Classical Shor Order Finding

Cory L. Thorbeck / 12 pages

papers/series/paper20_bsgs_rust.pdf

PAPER-STATED ABSTRACT

BSGS order nding in O(√r) vs Brent cycle detection. Benchmarks: 28 1300× faster than Brent at 16 26 bits. Enables practical simulation to 36+ bits on single CPU. Rust crate: rust/shor_fast/.

METHODS M028, M232, M233

PAPER-022 Quantum vs Classical Resource Analysis for 256-Bit ECDLP

Cory L. Thorbeck / 12 pages

papers/series/paper21_quantum_resources.pdf

PAPER-STATED ABSTRACT

Surface-code fault tolerance model: 256-bit semiprime ~1.2 min quantum (depth-limited) vs ~1022 years classical. Anchored to Gidney (2021, 2025) and Google Quantum AI (March 2026). Quantum feasibility; classical simulator establishes necessity of quantum formulation.

METHODS M147, M232, M240

PAPER LITERATURE / CONTINUED

PAPER-023 Node State Variables: A Typed Composition Layer for Governed Agentic Execution

Cory L. Thorbeck / 89 pages

papers/series/paper22_nsv.pdf

PAPER-STATED ABSTRACT

Each NSV has in_type →out_type ports; compounds decompose from steps or auto- split UPPER_SNAKE names. Port compatibility checking enables composable chains. 217 lines of tests in tests/test_nsv.py.

METHODS No method assigned by the matching pass.

PAPER-024 AgentFlow: Governance-First Execution for Production Autonomous Agents

Cory L. Thorbeck / 12 pages

papers/series/paper23_agentflow_governance.pdf

PAPER-STATED ABSTRACT

Pre-execution policy →post-action gates →SQLite hash-chain audit trail →evidence bundle export. Primitives: Policy (allow/deny), Gate (postconditions), Ledger (verify_chain()). vs LangGraph/CrewAI: veri able execution over chat orchestration.

METHODS No method assigned by the matching pass.

PAPER-025 The .af Language: Verified Action Graphs with Compile-Time Gate Validation

Cory L. Thorbeck / 11 pages

papers/series/paper26_af_language.pdf

PAPER-STATED ABSTRACT

Human/agent-authored programs with responsibility, depends_on, produces, compile- time gate predicates. 368 lines in tests/test_af.py. Examples: security_scan, init_security_memory_process composites.

METHODS No method assigned by the matching pass.

PAPER-026 Distinguished Kangaroo, C-Theory, and ECDLP Reduction for Bitcoin Puzzle Recovery

Cory L. Thorbeck / 386 pages

papers/series/paper27_umbrella.pdf

PAPER-STATED ABSTRACT

Integrated technical report summarizing C-Theory ve-level decomposition, anchor tow- ers, FUSED R2, feasible lattices, two-phase narrowing, D1 bands, CLI ngerprints, relation bundles, BEC audit envelopes, falsi ed wrapped CT and bit- ip hunts, and agent ow gover- nance tooling. 376 automated tests. Puzzle #135, #71, Block 1 w: UNSOLVED. Expands bitcoin_research_manuscript.md.

METHODS M021, M022, M027, M035, M045, M175, M234

PAPER LITERATURE / CONTINUED

PAPER-027 Classical Order Finding at the Edge of the Shor Simulation Regime: An Adaptive BSGS Benchmark with a 520-Qubit Sensitivity Model

Cory L. Thorbeck / 14 pages

papers/standalone_520_qubit/main.pdf

PAPER-STATED ABSTRACT

We present a reproducible classical order-finding benchmark and an adaptive experiment framework for simulating the period-finding subroutine of Shor’s algorithm. The primary contribution is empirical: an iterative-deepening baby-step/giant-step (BSGS) implementa- tion, checked against direct order enumeration, together with a closed-loop procedure for tuning trial counts and problem size. The experiments establish bounded performance of this implementation; they do not establish the difficulty of integer factorisation against number-field-sieve or other classical algorithms. For comparison, we retain a transparent 520-logical-qubit parameter in a separate sensi- tivity model. It is the chosen textbook register expression Qreg = 2b + ⌈log2 b⌉at b = 256, not a universal minimum, a complete reversible circuit, or a physical-machine prediction. The shor-sim system couples three subprocesses—empirical Monte-Carlo execution, closed-form statistical analysis, and rule-based parameter improvement—in a closed loop that scales the target semiprime as performance permits. Its optimised baby-step/giant- step (BSGS) order finder runs in O(√r) time, outperforms Brent cycle detection by orders of magnitude where both are tractable, and enables practical simulation to 36 bits in the reproducible benchmark configuration on a single CPU node. The quantum section reports only conditional arithmetic outputs from this parameterised model: 520 logical register qubits, 650,000 data-patch units under the stated convention, and two service-rate scenarios. No claim is made that one compiled circuit simultaneously realises those qubit, gate, depth, factory, routing, and feedback assumptions.

METHODS M232, M233, M234, M237, M241, M243, M244

PROVISIONAL SYSTEMS ARCHIVE

Machine-wide
systems + script lineage

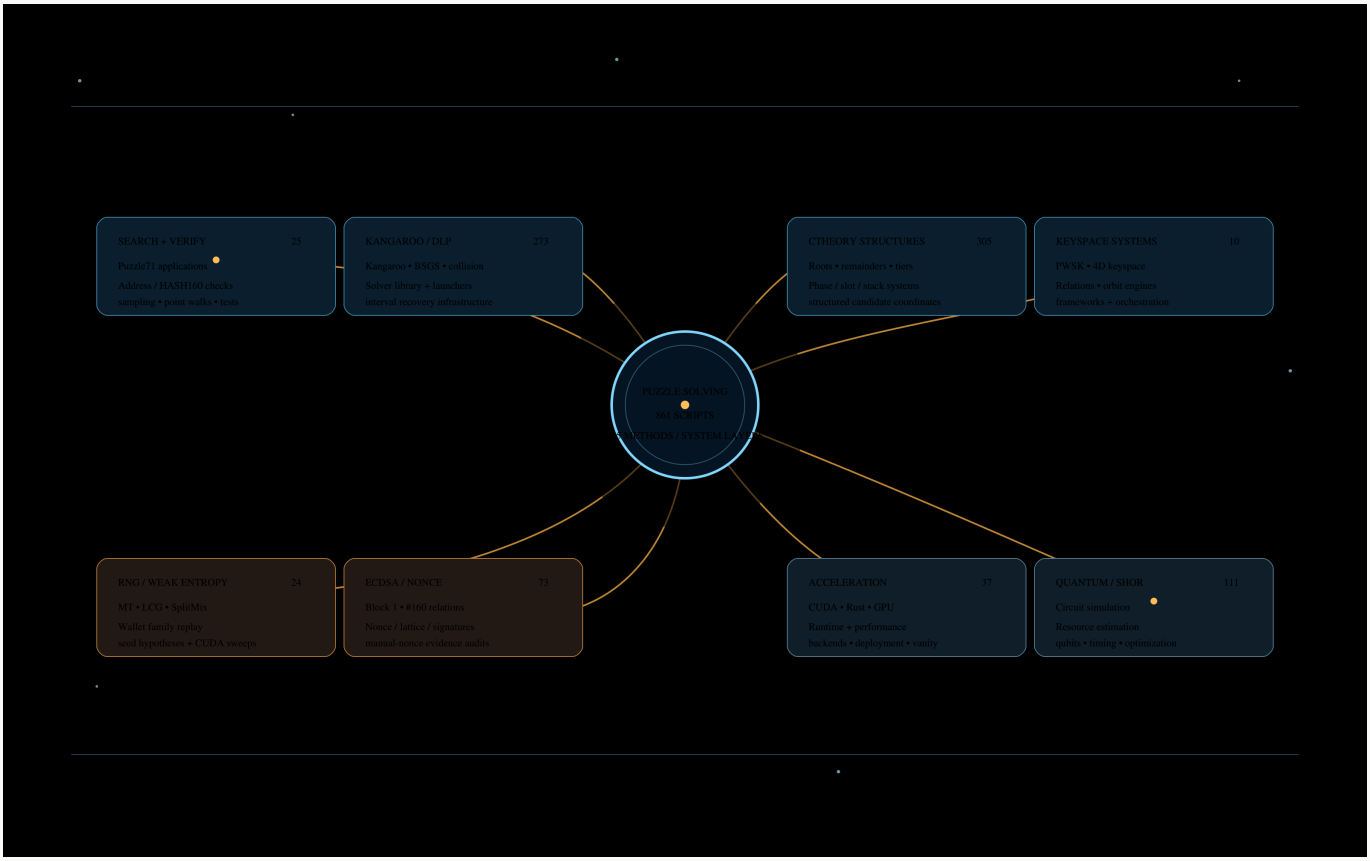
The following section is a review surface for larger systems, execution infrastructure, wrappers, tests, and scripts found on this machine. It is separate from the official 244-method inventory.

PREVIEW ONLY • NOT OFFICIAL INVENTORY

PREVIEW / NOT OFFICIAL INVENTORY

Machine-wide Bitcoin puzzle-solving systems

Architecture map for review before inclusion



REVIEW / COLLAPSE DUPLICATES FIRST

System lineage before method inclusion

The same algorithm may appear as a library, application, test, launcher, backend, or cloud wrapper. This view keeps those relationships visible so the official inventory counts ideas once and cites implementations underneath.

01 Search and verification	25
02 Kangaroo and discrete logarithm	273
03 CTheory and structured decompositions	305
04 Keyspace frameworks and orchestration	10
05 RNG and weak entropy	24
06 ECDSA, nonce, and signature relations	73
08 Bit, byte, sequence, and statistical structure	2
09 GPU, CUDA, Rust, and performance backends	37
10 Quantum and Shor simulation	111
11 OSINT, chain, and artifact analysis	1

PREVIEW / 861 FILES / NOT OFFICIAL INVENTORY

Every classified executable file

The following pages preserve implementations, tests, launchers, backends, and supporting scripts. A primary group is assigned for navigation; cross-system tags remain in the companion JSON. This appendix is a review surface, not a claim that every file is an independent method.

01 SEARCH AND VERIFICATION

/home/arialmohan/bitcoin-key-ascii-2026-10-21/decode_streams.py [implementation]
/home/arialmohan/puzzle71-corrected/benchmark_point_walk.py [implementation]
/home/arialmohan/puzzle71-corrected/continuous_stream.py [implementation]
/home/arialmohan/puzzle71-corrected/pipeline.py [implementation]
/home/arialmohan/puzzle71-corrected/point_walk.py [implementation]
/home/arialmohan/puzzle71-corrected/stream_check.py [implementation]
/home/arialmohan/puzzle71-corrected/verify_samples.py [implementation]
/home/arialmohan/puzzle71-educational/affine_point_walk.py [implementation]
/home/arialmohan/puzzle71-educational/audit_cutoff.py [implementation]
/home/arialmohan/puzzle71-educational/backtest.py [implementation]
/home/arialmohan/puzzle71-educational/backtest_continuous.py [implementation]
/home/arialmohan/puzzle71-educational/backtest_continuous_17_32.py [implementation]
/home/arialmohan/puzzle71-educational/candidate_sampling.py [implementation]
/home/arialmohan/puzzle71-educational/check_keys.py [implementation]
/home/arialmohan/puzzle71-educational/checks/benchmark_point_walk.py [implementation]
/home/arialmohan/puzzle71-educational/checks/continue_fused.py [implementation]
/home/arialmohan/puzzle71-educational/checks/fused_status.py [implementation]
/home/arialmohan/puzzle71-educational/checks/fused_status_window.py [implementation]
/home/arialmohan/puzzle71-educational/checks/successful_status_window.py [implementation]
/home/arialmohan/puzzle71-educational/continuous_check.py [implementation]
/home/arialmohan/puzzle71-educational/random_until_found.py [implementation]
/home/arialmohan/puzzle71-educational/test_affine_point_walk.py [test]
/home/arialmohan/puzzle71-educational/test_unique_sampling.py [test]
/home/arialmohan/scratch/modular-difference-levels/inverse_capability.py [implementation]
/home/arialmohan/scratch/modular-difference-levels/render_value_analysis.py [implementation]

02 KANGAROO AND DISCRETE LOGARITHM

/home/arialmohan/bitcoin-key-ascii-2026-10-21/convert_keys.py [implementation]
/home/arialmohan/scratch/puzzle-root-solve-test/gap_fill.py [implementation]
/home/arialmohan/scratch/puzzle-root-solve-test/surface2d.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/agentflow/__init__.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/agentflow/__main__.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/agentflow/admin/__init__.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/agentflow/admin/executor.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/agentflow/admin/handlers.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/agentflow/admin/registry.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/agentflow/admin/ui.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/agentflow/af_ast.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/agentflow/af_compile.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/agentflow/af_emit.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/agentflow/af_parser.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/agentflow/af_schema.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/agentflow/af_typecheck_errors.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/agentflow/agent.py [implementation]

PREVIEW / SCRIPT CATALOG

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/agentflow/agents.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/agentflow/analysis.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/agentflow/anchor.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/agentflow/auth.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/agentflow/characterization.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/agentflow/cli.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/agentflow/config.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/agentflow/context.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/agentflow/dashboard.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/agentflow/evidence.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/agentflow/exporters.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/agentflow/file_semantics.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/agentflow/gates.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/agentflow/graph_architecture.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/agentflow/graph_compile.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/agentflow/graph_events.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/agentflow/graph_lifecycle.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/agentflow/graph_parser.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/agentflow/graph_schema.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/agentflow/harness.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/agentflow/life_adapters/__init__.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/agentflow/life_adapters/_util.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/agentflow/life_adapters/init_ops.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/agentflow/life_adapters/security_ops.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/agentflow/life_map.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/agentflow/life_planner.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/agentflow/nsv.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/agentflow/orchestrator.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/agentflow/parser.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/agentflow/policy.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/agentflow/reports.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/agentflow/runbook.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/agentflow/runner.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/agentflow/runstore.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/agentflow/scaffold.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/agentflow/schema.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/agentflow/spec_loader.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/agentflow/spec_toml.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/agentflow/steptypes.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/agentflow/tools.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/agentflow/tracing.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/agentflow/verbosity.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/analyze_cli_clustering.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/analyze_ec_decompose.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/analyze_key_masking_hypothesis.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/analyze_leading_zero_pubs.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/analyze_spent_block_encoding.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/assets/js/app.js [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/assets/js/tools.js [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/benchmark_cpu_ec.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/benchmarks/benchmark_cpu_ec.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/candidate_variant_test.py [implementation]

PREVIEW / SCRIPT CATALOG

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/check_range_applicability.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/clawbot8.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/console/__init__.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/console/__main__.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/console/inproc.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/console/jobs.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/continue_traces.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/ecdlp_reduction_research.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/entropy_prefix_test.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/examples/characterize_files.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/examples/governance_demo.sh [launcher]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/examples/run_life_af.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/examples/run_life_framework.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/examples/run_nsv.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/examples/run_runbook.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/fanout_puzzle135.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/follow_099_fanouts.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/fullwidth_constraint_test.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/harvest_early_coinbase_descendants.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/harvest_early_coinbase_graph.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/harvest_early_coinbase_sigs.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/hunt_bit_flip.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/hunt_puzzle135.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/hunt_puzzle71.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/hunt_signer.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/kangaroo_solver/__init__.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/kangaroo_solver/__main__.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/kangaroo_solver/bip32.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/kangaroo_solver/cli.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/kangaroo_solver/collision.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/kangaroo_solver/dual_mask_seq.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/kangaroo_solver/export.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/kangaroo_solver/fast_ec.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/kangaroo_solver/glv.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/kangaroo_solver/phase.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/kangaroo_solver/puzzle_db.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/kangaroo_solver/secp256k1.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/map_chain_relationships.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/mock_keyspace_network.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/cloud-gpu-solver/kangaroo_solver/__init__.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/cloud-gpu-solver/kangaroo_solver/bip32.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/cloud-gpu-solver/kangaroo_solver/collision.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/cloud-gpu-solver/kangaroo_solver/export.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/cloud-gpu-solver/kangaroo_solver/fast_ec.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/cloud-gpu-solver/kangaroo_solver/glv.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/cloud-gpu-solver/kangaroo_solver/phase.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/cloud-gpu-solver/kangaroo_solver/puzzle_db.py [implementation]

PREVIEW / SCRIPT CATALOG

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/cloud-gpu-solver/kangaroo_solver/secp256k1.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/cloud-gpu-solver/scripts/setup_psckangaroo_local.sh [launcher]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/cloud-gpu-solver/scripts/setup_rckangaroo_local.sh [launcher]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/cloud-gpu-solver/vultr/rckangaroo_cmd.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/cloud-gpu-solver/vultr/verify_solution.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/puzzle_unlock_relationships.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/__init__.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/analysis/__init__.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/analysis/affine_lowbits_check_20261019.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/analysis/analyze_bip32_glv.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/analysis/archive_rng_recurrence_check_20261020.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/analysis/archive_rng_wordstream_check_20261020.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/analysis/generator_all_unknown_targets_20261020.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/analysis/index_offset_holdout_20261019.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/analysis/lowbit_linear_state_20261019.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/analysis/marsaglia_mother_full_seed_scan.c [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/analysis/marsaglia_mother_source_seed_check_20261020.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/analysis/park_miller_target_check_20261020.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/analysis/puzzle140_center_scan_20261020.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/analysis/splitmix64_mask_sequence_20261020.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/analysis/splitmix64_target_check_20261020.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/analysis/wallace_lcg_mask_backtest_20261020.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/chain/__init__.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/chain/multi_esplora.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/child_wallet_interval_probe.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/frameworks/__init__.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/harvest/__init__.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/puzzles/__init__.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/puzzles/armory_mask_test.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/puzzles/audit_chatgpt_parity.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/puzzles/candidate_variant_test.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/puzzles/electrum_v1_mask_test.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/puzzles/fullwidth_constraint_test.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/puzzles/hunt_puzzle71.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/scans/__init__.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/scans/scan_early_50btc_spent.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/scans/scan_fork_pubkey_leaks.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/scans/search_signed_bitcoin_messages.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/sparse_key_backtest.py [implementation]

PREVIEW / SCRIPT CATALOG

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/sparse_key_bitlearn.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/sparse_key_bloom.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/sparse_key_coshift.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/sparse_key_delta_hunt.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/sparse_key_dial_in.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/sparse_key_grow.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/sparse_key_hardthink.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/sparse_key_investigate.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/sparse_key_peel_tree.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/sparse_key_program_test.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/sparse_key_xor_tail.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/runpod/deploy_gpu_bench.sh [launcher]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/runpod/deploy_seed_crack.sh [launcher]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/runpod/deploy_vanity_btc.sh [launcher]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/runpod/vanity_bootstrap.sh [launcher]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/scan_btcorg_address.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/scan_btcparser_sleeping.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/scan_btcparser_sweep_signatures.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/scan_early_50btc_spent.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/scan_hub_address.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/scan_puzzle160_messages.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/scan_recoverable_bitcoin.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/scan_reused_r.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/scan_segwit_aranyn_r.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/scan_sender_extra_data.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/scan_testnet_aranyn.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/scripts/analyze_slot_scaling.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/scripts/build_target_index.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/scripts/export_points_1_160.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/scripts/export_points_all.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/scripts/gen_ref_pages.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/scripts/loyce_grep_derived.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/scripts/new_user.sh [launcher]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/scripts/publish-agentflow-repo.sh [launcher]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/scripts/run_creative_theories.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/scripts/setup_psckangaroo_local.sh [launcher]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/scripts/setup_rckangaroo_local.sh [launcher]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/scripts/start-dashboard.sh [launcher]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/scripts/verify_unimath_corpora.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/search_early_key_relations.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/server/auth.js [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/server/cafe.js [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/server/db.js [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/server/groups.js [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/server/index.js [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/server/messages.js [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/server/rateLimit.js [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/synthesis_puzzle71_solver.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/__init__.py [test]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_admin.py [test]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_af.py [test]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_af_parser_errors.py [test]

PREVIEW / SCRIPT CATALOG

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_af_typecheck.py [test]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_agent.py [test]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_analysis.py [test]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_armory_mask_test.py [test]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_auth.py [test]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_bip32_coinvault.py [test]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_build_target_index.py [test]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_candidate_variant_test.py [test]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_chatgpt_parity_audit.py [test]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_claim_recovered_btc.py [test]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_clawbot8.py [test]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_confidence.py [test]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_config.py [test]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_creative_theories.py [test]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_dashboard.py [test]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_dual_mask_seq.py [test]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_electrum_v1_mask_test.py [test]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_entropy_prefix_test.py [test]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_env_http.py [test]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_exporters.py [test]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_fullwidth_constraint_test.py [test]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_gates.py [test]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_glv_research_path.py [test]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_glv_scalar_mul.py [test]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_governance_v2.py [test]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_gpu_bench.py [test]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_gpu_cost.py [test]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_graph_spec.py [test]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_graph_system.py [test]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_harness.py [test]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_harness_runner.py [test]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_kangaroo_native_hits.py [test]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_life_adapters.py [test]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_life_af.py [test]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_life_map.py [test]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_missing_cue_audit.py [test]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_mock_keyspace_network.py [test]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_new_user.py [test]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_nsv.py [test]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_orchestrator.py [test]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_parser.py [test]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_policy.py [test]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_reports_webhook.py [test]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_runbook_remote.py [test]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_runbook_runner.py [test]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_scaffold.py [test]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_search_narrowing.py [test]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_shift_window_hunt.py [test]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_spec_toml.py [test]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_steptypes.py [test]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_time_will_tell_seed.py [test]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_unimath_corpora.py [test]

PREVIEW / SCRIPT CATALOG

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_vanity_btc.py [test]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_vanity_evm.py [test]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_verbosity.py [test]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_verify_scan_hits.py [test]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/trace_fanout_cc6f.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/trace_sender.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/try_all_puzzle_keys.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/uncover_relations.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/verify_scan_hits.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/vultr/benchmark_gpu.sh [launcher]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/vultr/rckangaroo_cmd.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/vultr/setup_psckangaroo.sh [launcher]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/vultr/setup_rckangaroo.sh [launcher]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/vultr/verify_solution.py [implementation]

03 CTHEORY AND STRUCTURED DECOMPOSITIONS

/home/arialmohan/bitcoin-key-ascii-2026-10-21/verify_x_coordinate.py [implementation]
/home/arialmohan/puzzle71-corrected/ctheory-test/experiment.py [implementation]
/home/arialmohan/puzzle71-corrected/ctheory-test/extended.py [implementation]
/home/arialmohan/puzzle71-educational/candidate_sampling_ctheory.py [implementation]
/home/arialmohan/puzzle71-educational/checks/advanced_selective.py [implementation]
/home/arialmohan/puzzle71-educational/checks/blind_fused_30_64.py [implementation]
/home/arialmohan/puzzle71-educational/continuous_check_ctheory.py [implementation]
/home/arialmohan/puzzle71-educational/ctheory-suite/candidate_sampling_ctheory.py [implementation]
/home/arialmohan/puzzle71-educational/ctheory-suite/check_keys.py [implementation]
/home/arialmohan/puzzle71-educational/ctheory-suite/continuous_check_ctheory.py [implementation]
/home/arialmohan/puzzle71-educational/ctheory-suite/ctheory_suite.py [implementation]
/home/arialmohan/puzzle71-educational/random_until_found_ctheory.py [implementation]
/home/arialmohan/puzzle71-educational/run_33_64_ctheory.py [implementation]
/home/arialmohan/scratch/modular-difference-levels/make_analysis_pdf.py [implementation]
/home/arialmohan/scratch/puzzle-root-solve-test/smooth_share.py [implementation]
/home/arialmohan/scratch/puzzle-root-solve-test/test.py [test]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/analyze_ctheory_decompose.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/analyze_ctheory_puzzle131.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/analyze_ctheory_puzzle135.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/analyze_ctheory_puzzle71.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/analyze_ctheory_sequence.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/analyze_d1_narrowing.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/analyze_deeper_decomposition_limits.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/analyze_missing_cue.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/analyze_phase23_remainders.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/analyze_puzzle135_sidebyside.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/analyze_puzzle140_breakdown.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/analyze_search_narrowing.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/analyze_set_bits.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/analyze_undecomposed_levels.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/backtest_all_keys_slots.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/backtest_arxiv_methods.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/backtest_bitsavers_methods.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/backtest_csp_orderings.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/backtest_ctheory.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/backtest_deeper_decomposition.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/backtest_hunt_pipeline.py [implementation]

PREVIEW / SCRIPT CATALOG

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/backtest_integrated_phase3.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/backtest_joc_methods.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/backtest_oreilly_methods.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/backtest_phase3_space.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/checks/build_value_atlas_20260920.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/checks/ctheory_forward_familywise_20261008.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/checks/ctheory_forward_null_20260920.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/checks/ctheory_forward_slice_family_20261008.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/checks/ctheory_math_20260920.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/checks/ctheory_remainder_precision_20260920.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/console/app.js [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/console/briefing.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/console/math_tower.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/ctheory_phase_solver.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/ctheory_sim/analysis.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/ctheory_sim/cli.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/ctheory_sim/empirical.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/ctheory_sim/loop.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/ctheory_sim/reporting.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/decompose.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/diagnose_ctheory135.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/export_ctheory_gpu_d1.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/export_ctheory_phase2.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/export_ctheory_phase3.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/export_ctheory_wrapped.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/export_p135_r2_ranked.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/export_p140_d1_gpu.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/export_p140_r2_ranked.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/hunt_ctheory_puzzle135.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/kangaroo_solver/bec.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/kangaroo_solver/bit_flip_hunt.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/kangaroo_solver/bitsavers_methods.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/kangaroo_solver/cli_clustering.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/kangaroo_solver/confidence.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/kangaroo_solver/csp/__init__.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/kangaroo_solver/csp/backtest.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/kangaroo_solver/csp/constraints.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/kangaroo_solver/csp/graph.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/kangaroo_solver/csp/solver.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/kangaroo_solver/ctheory.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/kangaroo_solver/ctheory_arithmetic.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/kangaroo_solver/ctheory_csp.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/kangaroo_solver/ctheory_stack/bec_cert.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/kangaroo_solver/ctheory_stack/cli_index.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/kangaroo_solver/ctheory_stack/hunt_slot.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/kangaroo_solver/ctheory_stack/ports.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/kangaroo_solver/ctheory_stack/tier_surface.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/kangaroo_solver/d1_narrowing.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/kangaroo_solver/ec_decompose.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/kangaroo_solver/executors/__init__.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/kangaroo_solver/external_schema.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/kangaroo_solver/gpu_bench.py [implementation]

PREVIEW / SCRIPT CATALOG

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/kangaroo_solver/gpu_cost.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/kangaroo_solver/hunt_plan.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/kangaroo_solver/joc_methods.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/kangaroo_solver/kangaroo.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/kangaroo_solver/keyspace_4d.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/kangaroo_solver/miss_ladder.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/kangaroo_solver/oreilly_methods.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/kangaroo_solver/phase2.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/kangaroo_solver/phase2_search.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/kangaroo_solver/pwsk.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/kangaroo_solver/pwsk_budget.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/kangaroo_solver/relation_bundle.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/kangaroo_solver/research.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/kangaroo_solver/set_bits.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/kangaroo_solver/slot_scaling.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/kangaroo_solver/unimath_corpora.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/kangaroo_solver/unimath_insights.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/kangaroo_solver/wrapped_export.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/kangaroo_solver/wrapped_kangaroo.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/build_ctheory_kangaroo_package.sh [launcher]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/cloud-gpu-solver/kangaroo_solver/bec.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/cloud-gpu-solver/kangaroo_solver/bit_flip_hunt.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/cloud-gpu-solver/kangaroo_solver/cli_clustering.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/cloud-gpu-solver/kangaroo_solver/confidence.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/cloud-gpu-solver/kangaroo_solver/ctheory.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/cloud-gpu-solver/kangaroo_solver/d1_narrowing.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/cloud-gpu-solver/kangaroo_solver/ec_decompose.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/cloud-gpu-solver/kangaroo_solver/gpu_cost.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/cloud-gpu-solver/kangaroo_solver/kangaroo.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/cloud-gpu-solver/kangaroo_solver/phase2.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/cloud-gpu-solver/kangaroo_solver/phase2_search.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/cloud-gpu-solver/kangaroo_solver/pwsk.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/cloud-gpu-solver/kangaroo_solver/relation_bundle.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/cloud-gpu-solver/kangaroo_solver/research.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/cloud-gpu-solver/kangaroo_solver/set_bits.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/cloud-gpu-solver/kangaroo_solver/wrapped_export.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/cloud-gpu-solver/kangaroo_solver/wrapped_kangaroo.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/cloud-gpu-solver/scripts/recover_k135_from_d1.py [implementation]

PREVIEW / SCRIPT CATALOG

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/ctheory-kangaroo/build_rust_base.sh [launcher]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/ctheory-kangaroo/kangaroo_solver/__init__.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/ctheory-kangaroo/kangaroo_solver/__main__.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/ctheory-kangaroo/kangaroo_solver/arxiv_methods.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/ctheory-kangaroo/kangaroo_solver/bec.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/ctheory-kangaroo/kangaroo_solver/bip32.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/ctheory-kangaroo/kangaroo_solver/bit_flip_hunt.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/ctheory-kangaroo/kangaroo_solver/bitsavers_methods.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/ctheory-kangaroo/kangaroo_solver/block1.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/ctheory-kangaroo/kangaroo_solver/cli.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/ctheory-kangaroo/kangaroo_solver/cli_clustering.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/ctheory-kangaroo/kangaroo_solver/collision.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/ctheory-kangaroo/kangaroo_solver/confidence.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/ctheory-kangaroo/kangaroo_solver/csp/__init__.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/ctheory-kangaroo/kangaroo_solver/csp/backtest.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/ctheory-kangaroo/kangaroo_solver/csp/constraints.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/ctheory-kangaroo/kangaroo_solver/csp/graph.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/ctheory-kangaroo/kangaroo_solver/csp/solver.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/ctheory-kangaroo/kangaroo_solver/ctheory.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/ctheory-kangaroo/kangaroo_solver/ctheory_arithmetic.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/ctheory-kangaroo/kangaroo_solver/ctheory_csp.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/ctheory-kangaroo/kangaroo_solver/ctheory_stack/bec_cert.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/ctheory-kangaroo/kangaroo_solver/ctheory_stack/cli_index.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/ctheory-kangaroo/kangaroo_solver/ctheory_stack/hunt_slot.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/ctheory-kangaroo/kangaroo_solver/ctheory_stack/ports.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/ctheory-kangaroo/kangaroo_solver/ctheory_stack/tier_surface.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/ctheory-kangaroo/kangaroo_solver/d1_narrowing.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/ctheory-kangaroo/kangaroo_solver/dp_gap_decoder.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/ctheory-kangaroo/kangaroo_solver/dual_mask_seq.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/ctheory-kangaroo/kangaroo_solver/ec_decompose.py [implementation]

PREVIEW / SCRIPT CATALOG

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/ctheory-kangaroo/kangaroo_solver/ecdsa_affine_nonce.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/ctheory-kangaroo/kangaroo_solver/executors/__init__.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/ctheory-kangaroo/kangaroo_solver/export.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/ctheory-kangaroo/kangaroo_solver/external_schema.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/ctheory-kangaroo/kangaroo_solver/fast_ec.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/ctheory-kangaroo/kangaroo_solver/glv.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/ctheory-kangaroo/kangaroo_solver/gpu_bench.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/ctheory-kangaroo/kangaroo_solver/gpu_cost.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/ctheory-kangaroo/kangaroo_solver/hunt_plan.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/ctheory-kangaroo/kangaroo_solver/joc_methods.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/ctheory-kangaroo/kangaroo_solver/kangaroo.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/ctheory-kangaroo/kangaroo_solver/keyspace_4d.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/ctheory-kangaroo/kangaroo_solver/miss_ladder.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/ctheory-kangaroo/kangaroo_solver/oreilly_methods.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/ctheory-kangaroo/kangaroo_solver/phase.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/ctheory-kangaroo/kangaroo_solver/phase2.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/ctheory-kangaroo/kangaroo_solver/phase2_search.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/ctheory-kangaroo/kangaroo_solver/puzzle_db.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/ctheory-kangaroo/kangaroo_solver/pwsk.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/ctheory-kangaroo/kangaroo_solver/pwsk_budget.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/ctheory-kangaroo/kangaroo_solver/reduction.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/ctheory-kangaroo/kangaroo_solver/reduction_export.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/ctheory-kangaroo/kangaroo_solver/relation_bundle.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/ctheory-kangaroo/kangaroo_solver/research.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/ctheory-kangaroo/kangaroo_solver/secp256k1.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/ctheory-kangaroo/kangaroo_solver/set_bits.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/ctheory-kangaroo/kangaroo_solver/slot_scaling.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/ctheory-kangaroo/kangaroo_solver/unimath_corpora.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/ctheory-kangaroo/kangaroo_solver/unimath_insights.py [implementation]

PREVIEW / SCRIPT CATALOG

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/ctheory-kangaroo/kangaroo_solver/user_signature.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/ctheory-kangaroo/kangaroo_solver/wrapped_export.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/ctheory-kangaroo/kangaroo_solver/wrapped_kangaroo.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/ctheory-kangaroo/scripts/allocate_pwsk_slot_budgets.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/ctheory-kangaroo/scripts/ctheory_runtime_status.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/ctheory-kangaroo/scripts/gate_unified_hunt.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/papers/generate_paper_series.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/predict_ctheory_puzzle140.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/analysis/analyze_ec_decompose.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/analysis/half_generator_cross_curve_20261020.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/analysis/half_generator_known_keys_backtest_20261020.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/analysis/puzzle140_r2_template_batch_20261020.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/analysis/r2_window_target_audit_20261020.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/analysis/remainder_holdout_20261019.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/analysis/remainder_tree_beacon_20261020.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/analysis/unknown_parameter_inventory_20261020.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/chain/crack_time_will_tell_seed.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/ctheory/__init__.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/ctheory/analyze_ctheory_decompose.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/ctheory/analyze_ctheory_puzzle135.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/ctheory/analyze_ctheory_puzzle71.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/ctheory/analyze_ctheory_sequence.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/ctheory/analyze_puzzle135_sidebyside.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/ctheory/backtest_arxiv_methods.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/ctheory/backtest_bitsavers_methods.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/ctheory/backtest_csp_orderings.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/ctheory/backtest_ctheory.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/ctheory/backtest_hunt_pipeline.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/ctheory/backtest_joc_methods.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/ctheory/backtest_oreilly_methods.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/ctheory/blind_loo_backtest.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/ctheory/cpu_band_d1_hunt.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/ctheory/ctheory_phase_solver.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/ctheory/diagnose_ctheory135.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/ctheory/export_ctheory_gpu_d1.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/ctheory/export_ctheory_phase3.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/ctheory/export_ctheory_wrapped.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/ctheory/hunt_ctheory_puzzle135.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/ctheory/predict_puzzle.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/ctheory/run_forward.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/ctheory/solve_puzzle135.py [implementation]

PREVIEW / SCRIPT CATALOG

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/ctheory/validate_ctheory_confidence.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/interval_r2_analyzer.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/r2_predictor_eval.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/r2_residual_analysis.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/scans/search_bitsavers_ctheory.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/scans/search_ctheory_structure_web.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/run_ctheory.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/run_forward.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/runpod/deploy_band_d1_batch2.sh [launcher]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/runpod/deploy_p135_max_gpu.sh [launcher]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/runpod/deploy_p140_max_gpu.sh [launcher]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/runpod/worker_bootstrap.sh [launcher]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/runpod/worker_bootstrap_band140.sh [launcher]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/runpod/worker_bootstrap_p140.sh [launcher]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/scripts/allocate_pwsk_slot_budgets.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/scripts/audit_narrowing_20260908.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/scripts/compile_sign_ctheory_sim.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/scripts/ctheory_runtime_status.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/scripts/export_d1_anchor.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/scripts/gate_unified_hunt.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/scripts/gen_research_launchers.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/scripts/generate_puzzle140_factsheet.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/scripts/heldout_r2_vs_midpoint.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/scripts/investigate_trailing.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/scripts/plot_ctheory_figures.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/scripts/recover_k135_from_d1.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/scripts/recover_k_from_d1.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/scripts/run_ctheory_phase_solver.sh [launcher]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/scripts/run_local_hunt.sh [launcher]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/scripts/setup_puzzle140.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/scripts/verify_ctheory_theorems.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/solve_puzzle135.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_bec_export.py [test]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_bit_flip_hunt.py [test]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_bitsavers_ctheory_search.py [test]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_bitsavers_methods.py [test]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_cli_clustering.py [test]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_console.py [test]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_csp_solver.py [test]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_ctheory_arithmetic.py [test]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_ctheory_csp.py [test]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_ctheory_phase_solver.py [test]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_ctheory_pipeline.py [test]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_ctheory_sim_loop.py [test]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_ctheory_structure_web_search.py [test]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_d1_narrowing.py [test]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_decompose_cli.py [test]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_deeper_decomposition.py [test]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_file_semantics.py [test]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_fused_r2_ranking.py [test]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_integrated_phase3_backtest.py [test]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_joc_methods.py [test]

PREVIEW / SCRIPT CATALOG

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_oreilly_methods.py [test]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_phase23_remainders.py [test]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_phase2_search.py [test]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_phase3_space_backtest.py [test]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_puzzle131_analysis.py [test]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_puzzle140_setup.py [test]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_r2_coverage.py [test]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_runtime_rust_base.py [test]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_scaling.py [test]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_set_bits_analysis.py [test]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_undecomposed_levels.py [test]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_unified_hunt.py [test]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/validate_ctheory_confidence.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/verify_orbit_candidates.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/vultr/bootstrap.sh [launcher]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/vultr/deploy_gpu_anchor_api.sh [launcher]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/vultr/gpu_benchmark.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/vultr/run_all_gpus.sh [launcher]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/vultr/run_anchor_hunt.sh [launcher]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/vultr/run_gpu_d1_batch.sh [launcher]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/vultr/run_phase3_batch.sh [launcher]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/vultr/run_puzzle140_hunt.sh [launcher]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/vultr/run_wrapped_all.sh [launcher]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/vultr/run_wrapped_batch.sh [launcher]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/vultr/run_wrapped_slot.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/vultr/userdata-32cpu.sh [launcher]

04 KEYSPACE FRAMEWORKS AND ORCHESTRATION

/home/arialmohan/puzzle71-educational/test_pipeline.py [test]
/home/arialmohan/scratch/modular-difference-levels/experiment.py [implementation]
/home/arialmohan/scratch/modular-difference-levels/value_analysis.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/demo_pwsk.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/export_relation_bundle.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_keyspace_4d.py [test]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_pwsk.py [test]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_revelation_discovery.py [test]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/unified_keyspace_framework_algorithm.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/visualize_keyspace_4d.py [implementation]

05 RNG AND WEAK ENTROPY

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/analyze_creator_generator.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/derive_milksad_addresses.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/hunt_multi_significant.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/scans/scan_btcparser_sweep_signatures.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/scans/scan_reused_r.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/runpod/deploy_profanity_offset.sh [launcher]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/runpod/profanity_worker_bootstrap.sh [launcher]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/scan_weak_entropy_addresses.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/scan_weak_entropy_families.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/scan_weak_entropy_repos.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/scripts/derive_combos2_scan.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/scripts/derive_combos3_scan.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/scripts/derive_combos_scan.py [implementation]

PREVIEW / SCRIPT CATALOG

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/scripts/discover_revelation_methods.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_profanity_closest.py [test]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_profanity_scan.py [test]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_weak_entropy_cuda_port.py [test]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_weak_entropy_derive.py [test]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_weak_entropy_families.py [test]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/weak_entropy_cluster.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/weak_entropy_families.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/weak_entropy_gen.cpp [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/weak_entropy_prng.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/weak_entropy_wallet.py [implementation]

06 ECDSA, NONCE, AND SIGNATURE RELATIONS

/home/arialmohan/bitcoin-key-ascii-2026-10-21/check_transactions.py [implementation]
/home/arialmohan/scratch/puzzle-root-solve-test/exhaustive_carry.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/console/server.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/console/status.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/demo_ecdlp_reproducible.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/hunt_block1.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/hunt_shift_windows.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/kangaroo_solver/arxiv_methods.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/kangaroo_solver/block1.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/kangaroo_solver/dp_gap_decoder.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/kangaroo_solver/ecdsa_affine_nonce.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/kangaroo_solver/reduction.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/kangaroo_solver/reduction_export.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/kangaroo_solver/user_signature.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/output/early_known_nonce_check_20261020/audit_early.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/output/early_known_nonce_check_20261020/check_manual_160.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/output/early_known_nonce_check_20261020/inputs/block1.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/output/early_known_nonce_check_20261020/inputs/user_signature.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/cloud-gpu-solver/kangaroo_solver/block1.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/cloud-gpu-solver/kangaroo_solver/dp_gap_decoder.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/cloud-gpu-solver/kangaroo_solver/reduction.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/cloud-gpu-solver/kangaroo_solver/reduction_export.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/cloud-gpu-solver/one_shot_gpu.sh [launcher]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/cloud-gpu-solver/vultr/gpu_solve.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/puzzle_research.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/analysis/analyze_leading_zero_pubs.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/analysis/analyze_spent_block_encoding.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/analysis/check_range_applicability.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/analysis/ecdlp_reduction_research.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/analysis/schnorr_ecdsa_aranyn_compare.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/analysis/search_160_block1_relation.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/analysis/search_early_key_relations.py [implementation]

PREVIEW / SCRIPT CATALOG

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/chain/continue_traces.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/chain/follow_099_fanouts.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/chain/map_chain_relationships.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/chain/puzzle_unlock_relationships.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/chain/trace_fanout_cc6f.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/chain/trace_sender.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/chain/uncover_relations.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/harvest/harvest_early_coinbase_descendants.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/harvest/harvest_early_coinbase_graph.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/harvest/harvest_early_coinbase_sigs.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/puzzles/hunt_block1.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/puzzles/hunt_puzzle135.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/puzzles/hunt_signer.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/puzzles/synthesis_puzzle71_solver.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/puzzles/try_all_puzzle_keys.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/scans/scan_btcorg_address.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/scans/scan_btcparser_sleeping.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/scans/scan_hub_address.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/scans/scan_old_bitcoin_sites.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/scans/scan_puzzle160_messages.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/scans/scan_recoverable_bitcoin.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/scans/scan_segwit_aranyn_r.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/scans/scan_sender_extra_data.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/scans/scan_testnet_aranyn.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/scan_block1_keygen_seeds.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/scan_puzzle_reflections.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/schnorr_ecdsa_aranyn_compare.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/scripts/claim_recovered_btc.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/scripts/generate_low_entropy_benchmark.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/search_160_block1_relation.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/solve.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/solve_user_signature.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_160_block1_relation.py [test]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_block1_keygen_seeds.py [test]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_early_key_relations.py [test]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_ecdsa_affine_nonce.py [test]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_reduction_export.py [test]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_scan_reused_r.py [test]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/vgen_puzzle_adapter.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/vultr/gpu_solve.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/vultr/one_shot_gpu.sh [launcher]

08 BIT, BYTE, SEQUENCE, AND STATISTICAL STRUCTURE

/home/arialmohan/scratch/modular-difference-levels/quotient_experiment.py [implementation]
/home/arialmohan/scratch/modular-difference-levels/quotient_experiment_256.py [implementation]

09 GPU, CUDA, RUST, AND PERFORMANCE BACKENDS

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/cuda/ctheory_modules/d1_search/d1_search.cu [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/cuda/ctheory_modules/herd_step/herd_step.cu [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/cuda/keystream_dump.cpp [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/cuda/run_bounded.sh [launcher]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/cuda/run_sweeps.sh [launcher]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/cuda/spike_index.py [implementation]

PREVIEW / SCRIPT CATALOG

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/cuda/validate_addresses.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/cuda/validate_keystreams.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/cuda/vanity/vanity_btc_cuda.cu [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/cuda/vanity/vanity_btc_host_test.cpp [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/cuda/vanity/vanity_evm_cuda.cu [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/cuda/vanity/vanity_evm_host_test.cpp [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/cuda/weak_entropy_cuda.cu [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/kangaroo_solver/runtime/__init__.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/kangaroo_solver/runtime/backends_cuda.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/kangaroo_solver/runtime/backends_rust.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/kangaroo_solver/runtime/base.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/kangaroo_solver/runtime/modules.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/ctheory-kangaroo/cuda/ctheory_modules/d1_search/d1_search.cu [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/ctheory-kangaroo/cuda/ctheory_modules/herd_step/herd_step.cu [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/ctheory-kangaroo/kangaroo_solver/runtime/__init__.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/ctheory-kangaroo/kangaroo_solver/runtime/backends_cuda.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/ctheory-kangaroo/kangaroo_solver/runtime/backends_rust.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/ctheory-kangaroo/kangaroo_solver/runtime/base.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/ctheory-kangaroo/kangaroo_solver/runtime/modules.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/ctheory-kangaroo/rust/kangaroo_fast/src/lib.rs [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/rust/kangaroo_fast/src/lib.rs [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/rust/profanity_chain/src/bin/addr_check.rs [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/rust/profanity_chain/src/main.rs [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/rust/profanity_scan/cuda/lowkey_cuda.cu [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/rust/profanity_scan/cuda/milksad_bip39_cuda.cu [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/rust/profanity_scan/cuda/milksad_cuda.cu [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/rust/profanity_scan/cuda/profanity_cuda.cu [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/rust/profanity_scan/src/lib.rs [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/rust/profanity_scan/src/main.rs [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/rust/shor_fast/src/bin/bench.rs [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/rust/shor_fast/src/lib.rs [implementation]

10 QUANTUM AND SHOR SIMULATION

/home/arialmohan/puzzle71-educational/ctheory-suite/pipeline.py [implementation]
/home/arialmohan/puzzle71-educational/pipeline.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/agentflow/af_gates.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/agentflow/af_typecheck.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/agentflow/registry.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/analyze_lattice_candidates.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/benchmarks/benchmark.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/benchmarks/shor_backend_matrix.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/build_loyce_index.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/checks/exhaustive_bsgs_validation.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/console/math.js [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/console/math_catalog.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/console/tools.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/ctheory_sim/__init__.py [implementation]

PREVIEW / SCRIPT CATALOG

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/ctheory_sim/improvement.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/examples/autonomous_dev_agent.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/kangaroo_solver/creative_theories.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/kangaroo_solver/ctheory_stack/__init__.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/kangaroo_solver/ctheory_stack/hunt_stack.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/kangaroo_solver/ctheory_stack/shortcuts.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/kangaroo_solver/dp_parser.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/kangaroo_solver/revelation_discovery.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/kangaroo_solver/slot_contract.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/kangaroo_solver/tier_discipline.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/osint/nevercast-404-loader.deobfuscated.js [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/build_cloud_gpu_package.sh [launcher]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/cloud-gpu-solver/kangaroo_solver/dp_parser.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/ctheory-kangaroo/kangaroo_solver/creative_theories.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/ctheory-kangaroo/kangaroo_solver/ctheory_stack/__init__.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/ctheory-kangaroo/kangaroo_solver/ctheory_stack/hunt_stack.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/ctheory-kangaroo/kangaroo_solver/ctheory_stack/shortcuts.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/ctheory-kangaroo/kangaroo_solver/dp_parser.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/ctheory-kangaroo/kangaroo_solver/revelation_discovery.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/ctheory-kangaroo/kangaroo_solver/slot_contract.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/ctheory-kangaroo/kangaroo_solver/tier_discipline.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/packages/ctheory-kangaroo/scripts/compose_gpu_hunt_stack.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/papers/content_loader.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/papers/standalone_520_qubit/benchmarks/benchmark.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/papers/standalone_520_qubit/checks/exhaustive_bsgs_validation.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/papers/standalone_520_qubit/run_quantum.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/papers/standalone_520_qubit/shor_sim/__init__.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/papers/standalone_520_qubit/shor_sim/analysis.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/papers/standalone_520_qubit/shor_sim/bitcoin.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/papers/standalone_520_qubit/shor_sim/circuit.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/papers/standalone_520_qubit/shor_sim/cli.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/papers/standalone_520_qubit/shor_sim/core.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/papers/standalone_520_qubit/shor_sim/ecdsa.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/papers/standalone_520_qubit/shor_sim/empirical.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/papers/standalone_520_qubit/shor_sim/improvement.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/papers/standalone_520_qubit/shor_sim/loop.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/papers/standalone_520_qubit/shor_sim/manifest.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/papers/standalone_520_qubit/shor_sim/optimizer_agent.py [implementation]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/papers/standalone_520_qubit/shor_sim/primes.py [implementation]

PREVIEW / SCRIPT CATALOG

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/papers/standalone_520_qubit/shor_sim/quantum.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/papers/standalone_520_qubit/shor_sim/reporting.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/papers/standalone_520_qubit/shor_sim/wolfram.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/papers/standalone_520_qubit/shor_sim/wolfram_followup.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/papers/standalone_520_qubit/shor_sim/wolfram_masks.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/papers/standalone_520_qubit/shor_sim/wolfram_mix.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/papers/standalone_520_qubit/tests/test_quantum.py [test]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/papers/standalone_520_qubit/tests/test_quantum_profiles.py [test]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/analysis/marsaglia_mother_direct_check_20261020.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/analysis/marsaglia_mother_two_draw_check_20261020.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/chain/fanout_puzzle135.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/frameworks/unified_keyspace_framework_algorithm.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/puzzles/entropy_prefix_test.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/research/scans/scan_block1_keygen_seeds.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/run_continuous_ordering.sh [launcher]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/scripts/compose_gpu_hunt_stack.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/scripts/fetch_loyce_dump.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/scripts/gen_wiki_branch_catalog.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/scripts/publish-to-agentflow.sh [launcher]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/scripts/validate_ctheory_math.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/shor_sim/__init__.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/shor_sim/analysis.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/shor_sim/bitcoin.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/shor_sim/circuit.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/shor_sim/cli.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/shor_sim/core.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/shor_sim/ecdsa.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/shor_sim/empirical.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/shor_sim/improvement.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/shor_sim/loop.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/shor_sim/manifest.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/shor_sim/optimizer_agent.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/shor_sim/primes.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/shor_sim/quantum.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/shor_sim/reporting.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/shor_sim/wolfram.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/shor_sim/wolfram_followup.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/shor_sim/wolfram_masks.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/shor_sim/wolfram_mix.py [implementation]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_backend_matrix.py [test]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_circuit.py [test]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_core.py [test]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_ctheory_stack_gpu.py [test]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_ecdsa.py [test]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_factor_fast.py [test]

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_improvement.py [test]

PREVIEW / SCRIPT CATALOG

/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_ledger.py [test]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_loop.py [test]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_manifest.py [test]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_optimizer_agent.py [test]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_quantum.py [test]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_quantum_profiles.py [test]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_reporting.py [test]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_shor_analysis.py [test]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_wiki_knowledge_base.py [test]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_wolfram.py [test]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_wolfram_masks.py [test]
/home/arialmohan/workspace/projects/distinguished-kangaroo-main/tests/test_wolfram_mix.py [test]

11 OSINT, CHAIN, AND ARTIFACT ANALYSIS

/home/arialmohan/puzzle71-corrected/test_pipeline.py [test]

INTERPRETATION

Corrections that travel with the results

01 CTheory reconstruction identities hold for ordinary integers. Exact reconstruction on known keys does not predict unknown digits.

02 Old unconditional CTheory and wrapped-leaf speedups are retracted or superseded. Total work must include all unknown slots, misses and setup.

03 The old Delta2 7/7 calibration result leaked target information. The documented corrected blind high-index full-slot coverage was 0/7.

04 Ninth-root clustering requires a transformed uniform-key baseline. A narrow-looking root interval may cover much of the scalar interval.

05 A conditional search band can be operationally useful while missing the target. Interval containment, prefix checks and exhaustive recovery are different evidence.

06 The original HP-RTE, diversity, Cantor and Ramsey claims contain coding or mathematical defects. Corrected diagnostics do not establish a key-recovery shortcut.

07 Historical GPU sweep results apply to their dated indexes, address types, generator families and finite seed scopes. They do not cover all possible wallets.

08 Classical Shor-style factoring simulations and qubit estimates are not executed target-scale quantum secp256k1 key recovery.

09 A disclosed #135 key or moved funds is calibration/chain evidence, not proof that the local project independently discovered that key.

10 Broad archive statements that all future structural approaches are impossible exceed bounded tests. Missing signatures or creator artifacts remain possible new evidence.

MATH APPENDIX / READER GUIDE

What the mathematics is for

The equations at the end are tools for checking three kinds of research claim. Start with the question below, then read the named method sheets and their evidence status. A formula can prove an arithmetic identity or reject a fully specified model; it cannot, by itself, recover an unknown Bitcoin key.

IS A PATTERN FORCED BY THE RANGE?

Use the range-forced feature test when a root, prefix, or other statistic appears constant across solved keys. Calculate the statistic over the entire allowed tier, then ask whether the observed agreement is automatic.

Read M040, M061, M065 and the first test on the next page.

DOES A GENERATOR HYPOTHESIS SURVIVE?

Write down the exact seed/state, word order, mask, schedule, and public observations before comparing a generator prediction. A mismatch can reject that specified model; a match or an underdetermined system does not establish the creator's generator.

Read M087, M093-M096, M112 and M116.

IS A DECOMPOSITION SIGNAL REAL?

Compare every claimed beacon with random integers drawn from the same range. Repeat the entire feature-selection procedure in each control replicate, including the search over many candidate features.

Read M038-M040, M059, M061-M064 and the second test.

DOES A NARROW SLOT SAVE TOTAL WORK?

Use the CTheory pages to reconstruct every candidate exactly, count every feasible coefficient/remainder branch, and include the unsearched complement and per-branch solver cost.

Read M041-M048, M056-M060 and the CTheory references.

Reading rule: definition -> required input -> calculation -> observed result -> scope of the conclusion. The glossary after the references translates the notation and research shorthand used throughout the book.

MATH APPENDIX / 01

Mathematical tools behind three tests

These equations formalize useful diagnostics from the archive. They are test patterns, not newly established lemmas or key-recovery results.

01 RANGE-FORCED FEATURE TEST

For an N-bit candidate, let the allowed interval and a feature be:

$$I_N = \{2^{N-1}, \ldots, 2^N - 1\}$$

$$f(k) = c \quad \forall k \in I_N$$

Then the feature carries no information about which candidate was selected within that interval:

$$H(f(K_N) \mid K_N \in I_N) = 0$$

This conclusion is only about f on the specified interval. It does not imply that the key itself has zero entropy or that every other feature is constant.

Use it: if all keys in a tier share the same root anchor, that shared anchor is not evidence of a generator. Check the whole tier before testing historical keys. See M040 and M065.

02 REJECTING A SPECIFIED GENERATOR

Under the stated mask hypothesis, a candidate K_N predicts a public value P_N and its low bits:

$$P_N = 2^{N-1} + (K_N \bmod 2^{N-1})$$

$$m_N = P_N - 2^{N-1} = K_N \bmod 2^{N-1}$$

For a lagged-add word generator with b-bit words, the proposed recurrence is:

$$x_t \equiv x_{t-r} + x_{t-s} \pmod{2^b}$$

A mismatch in observed bits rejects that fully specified generator, state, schedule, and word order:

$$\widehat{m}_N \neq m_N \pmod{2^b}$$

With no usable recurrence equations, the model is untested. A rejection applies only to the exact model checked.

Use it: freeze the proposed generator and compare its predicted words with observations not used to choose the model. Record a mismatch as a rejection of that configuration, not every generator in the family. See M112 and M116.

MATH APPENDIX / 02

Testing a decomposition beacon

A representation can look unusual simply because the candidate interval and number format create structure. The beacon diagnostic compares the observed statistic against range-matched random controls.

DECOMPOSITION AND NULL CONTROLS

$$x = r^3 + cr^2 + d$$

$$K_i^{(q)} \sim \mathrm{Unif}(I_i)$$

For control replicate q, compute all m predeclared features and retain the largest absolute standardized score:

$$M_q = \max_{1 \le j \le m} |Z_j^{(q)}|$$

The plus-one Monte Carlo estimate adjusts for the search across those features:

$$\widehat{p} = \frac{1 + \#\{q : M_q \ge M_{\mathrm{obs}}\}}{B + 1}$$

The archived range-matched test reports familywise p approximately 0.988, so its observed beacon is not unusual under those controls. This finite null test does not prove that every possible representation is uninformative.

INTERPRETATION

A low p-value would flag an anomaly for further study; it would not establish a causal generator, identify a private key, or validate a predictive method. The result depends on the declared features, control distribution, and sampling procedure.

Use it: run the identical feature-search pipeline on each range-matched control set. Compare the observed maximum with the distribution of control maxima, not with a single unadjusted feature. See M061-M064.

CTHEORY / DEFINITIONS & NOTATION

CTheory as a radix tower

CTheory (Cubic Theory) is a lossless integer decomposition used to organize candidate scalars into nested coordinates. It is not an elliptic-curve inversion method. The decomposition can be applied to any positive integer, including a uniformly random one.

Notation mapping used in this section: $Y_1=R_1$, $Y_2=R_2$, $Y_3=S_3$, $Y_4=S_4$, $Y_5=S_5$. The implementation names the first two coefficients C_1,C_2 and the next coefficients $\gamma_3,\gamma_4,\gamma_5$; here these are c_1,c_2,c_3,c_4,c_5. The optional deeper tower continues with c_6,c_7,c_8 (and can continue further). Remainders are d_1, Δ_2, δ_3,… .

THE FIRST FIVE LEVELS

$$k = Y_1^3 + c_1 Y_1^2 + d_1$$

$$Y_1 = Y_2^3 + c_2 Y_2^2 + d_2$$

$$Y_2 = Y_3^3 + c_3 Y_3^2 + d_3$$

$$Y_3 = Y_4^3 + c_4 Y_4^2 + d_4$$

$$Y_4 = Y_5^3 + c_5 Y_5^2 + d_5$$

Thus $d_1=D_1$ and $d_2=\Delta_2$ in the code. Each line is a reversible decomposition of its left-hand integer; substituting from the bottom upward reconstructs k exactly.

CTHEORY / OPTIONAL DEEPER LEVELS

Continuing the tower through c_8

The core report stops after $Y_4=Y_5^3+c_5Y_5^2+d_5$. The implementation also permits further exact splits. Under the notation mapping above, the next three are:

$$Y_5 = Y_6^3 + c_6 Y_6^2 + d_6$$

$$Y_6 = Y_7^3 + c_7 Y_7^2 + d_7$$

$$Y_7 = Y_8^3 + c_8 Y_8^2 + d_8$$

Each level repeats the same integer procedure: take the floor cube root for the next Y, divide the excess by Y^2 to obtain c, and keep the division remainder d. The c_i values are not fixed digits in a conventional positional numeral system: their feasible bounds depend on the current root Y_i.

CLOSED-FORM ROOTS AND TIER ANCHORS

$$Y_j = \lfloor k^{1/3^j} \rfloor$$

This nested-root identity is exact for the canonical decomposition. For a tier-N interval, the lower-edge anchor A_j=floor(2^((N−1)/3^j)) is a cheap candidate-scale reference, not automatically the exact Y_j throughout the tier. Exact root bounds come from applying integer roots to both interval endpoints.

$$\lfloor (2^{N-1})^{1/3^j} \rfloor \le Y_j \le \lfloor (2^N - 1)^{1/3^j} \rfloor$$

At sufficiently deep levels, many tier anchors equal 1. Those levels then add no useful branching information for that tier, even though the decomposition remains mathematically valid. A match of an anchor formula on solved examples is expected when the entire tier lies below the next perfect-power threshold; it is not an independent law of key generation.

WHY DECOMPOSE A REMAINDER?

A recursive split can expose a component for visualization, storage, constraint programming, or an experiment about digit statistics. To use it for search, every possible coefficient and remainder branch must still be counted. If a branch is omitted, that is a heuristic restriction that needs held-out coverage evidence.

CTHEORY / EXACT PROCEDURE

How each decomposition is computed

For a positive integer x, take its integer cube root r=floor(cuberoot(x)). Divide the excess x−r³ by r². The quotient is the coefficient c and the remainder is d. This is Euclidean division, so the result is unique under the stated root convention.

$$r = \lfloor \sqrt[3]{x} \rfloor, \quad x - r^3 = cr^2 + d$$

$$c = \left\lfloor \frac{x - r^3}{r^2} \right\rfloor, \quad d = x - r^3 - cr^2$$

$$0 \le d < r^2$$

Apply this same operation successively to k, then Y_1, then Y_2, then each later root. Exact integer-root routines avoid floating-point rounding; reconstruction is a round-trip check, not a prediction check.

WHY COEFFICIENTS ARE NOT ALWAYS 0, 1, OR 2

The next cube boundary is (r+1)³. Therefore the excess may be almost 3r²+3r, which permits c=3 for sufficiently large r. The often-used set {0,1,2} is an observed restriction on particular ranges/corpora, not a universal consequence of this division algorithm.

$$0 \le x - r^3 < (r+1)^3 - r^3 = 3r^2 + 3r + 1$$

$$c \le \left\lfloor \frac{(r+1)^3 - r^3 - 1}{r^2} \right\rfloor$$

Small roots can have still larger coefficient bounds. Consequently, hard-coding c∈{0,1,2} can drop valid integers unless the target range has separately justified that restriction.

RECURSIVE OR SUB-REMAINDER SPLITS

Any remainder can itself be split by the same rule. This introduces another root, coefficient and residual at each depth. It can make coordinates smaller and easier to store or enumerate, while preserving every possibility.

$$d_i = u_i^3 + a_i u_i^2 + \epsilon_i, \quad 0 \le \epsilon_i < u_i^2$$

CTHEORY / INTERPRETATION

What the tower tells us—and what it does not

Nested integer cube roots collapse to a single higher root. This gives compact scale coordinates. It does not mean the coordinates are exposed by a Bitcoin address or that an elliptic-curve point can be cube-rooted to obtain its discrete logarithm.

$$Y_j = \lfloor k^{1/3^j} \rfloor \quad \text{for the canonical root tower}$$

A tier N denotes the scalar interval [2^(N−1),2^N). A tier-only root estimate such as floor(2^((N−1)/3^j)) is a lower-edge anchor. It equals the true root only until the interval crosses the next perfect 3^j-th power. The interval bounds the root; the tier label alone need not determine it.

$$A_j = \lfloor 2^{(N-1)/3^j} \rfloor$$

For fixed Y and c, the residual d still ranges over 0,...,Y²−1. The corresponding scalar block is contiguous. All coefficient and remainder branches together partition the original candidate set; the tree is a coordinate change, not a sieve.

$$k = Y^3 + cY^2 + d, \quad d \in \{0, \ldots, Y^2 - 1\}$$

If a feature is a deterministic function of the tier and is constant throughout that tier, it cannot distinguish candidates within the tier. A skew in a digit histogram can also be caused by the decomposition map itself; it must be compared with range-matched random integers before being interpreted as key-generation structure.

THE CRITICAL CRYPTOGRAPHIC GAP

With public point Q=kG, knowing candidate coordinates lets us compute a residual point. It does not reveal the unknown coordinates. A guessed (Y_1,c_1) creates a residual discrete-log search for d_1; only a verified scalar satisfying the public-key or address check is a solution.

$$Q - (Y_1^3 + c_1 Y_1^2)G = d_1 G$$

CTHEORY / PHASES & EVIDENCE

How the search procedure uses the coordinates

PHASE 1 — PROPOSE ROOT CANDIDATES

A predictor or a complete enumeration supplies candidate Y_2 values (R_2). FUSED/global-local regressions and gap-based sequence rules rank or propose them from solved-key data. They are empirical heuristics, not consequences of the cubic identities. A narrow band is a priority list unless the omitted complement is also searched.

PHASE 2 — EXPAND THE NEXT COORDINATE

For each proposed (Y_2,c_2), enumerate or prioritize feasible d_2 values, form $Y_1=Y_2^3+c_2Y_2^2+d_2$, and discard branches whose possible scalar block misses the advertised puzzle interval. A sequence-derived hint changes order; it does not certify the correct d_2.

$$Y_1 = Y_2^3 + c_2 Y_2^2 + d_2, \quad k = Y_1^3 + c_1 Y_1^2 + d_1$$

PHASE 3 — SEARCH THE RESIDUAL

For each branch (Y_1,c_1), subtract its known base point from Q and solve the bounded discrete log for d_1. Pollard kangaroo cost is proportional to the square root of the interval width. Any guessed narrow d_1 band can miss; full coverage requires searching its complement.

$$Q_{\text{res}} = Q - (Y_1^3 + c_1 Y_1^2)G = d_1 G$$

The total work includes the number of (Y_2,c_2,d_2,c_1) branches as well as the per-branch kangaroo. A smaller leaf interval does not by itself imply a faster total search.

VALIDATION CHECKLIST

- Arithmetic: exact reconstruction and interval-bound checks on each decomposition.
- Prediction: train only on information available before the held-out target; report exact scalar coverage and missed cases.
- Null: repeat the full feature-selection procedure on range-matched uniform integers and correct for trying many features.
- Search: account for every branch, setup, misses, and any unsearched complement.
- Success: verify the recovered scalar against the target public point or address.

The reviewed archive reports exact decomposition/reconstruction and several useful audit procedures. It does not validate the tower as an unknown-key predictor. Historical strong CTheory speedup and wrapped-leaf cost claims were corrected or superseded; apply the cited dated reports and their stated scopes.

REFERENCES / HOW TO USE THEM

The CTheory evidence map

These are the local documents and code behind the preceding explanation, not eight additional methods or eight independent confirmations. The notes below put their useful content directly on the page. Use the method IDs to reach the inventory sheets; use the file names only to trace provenance.

CT1 MATHEMATICAL REVIEW / 20 SEP 2026

USE: Use for the corrected boundary between exact decomposition and unknown-key prediction; it also reviews root geometry and search-cost claims.

METHOD SHEETS: M038, M040, M048, M059-M061

LIMIT: A review of what the record supports, not a certificate of a solved target.

Archive file: CTHEORY_MATH_REVIEW_20260920.md

CT2 EXACT ARITHMETIC

USE: Use to derive legal coefficient and remainder ranges, distinguish tier anchors from true roots, and count complete slot work.

METHOD SHEETS: M038-M040, M047-M048

LIMIT: An exact identity changes coordinates; it does not make an unknown coordinate observable.

Archive file: ctheory_arithmetic.tex

CT3 MAIN PAPER

USE: Use for the original definitions and phase-by-phase proposal: form candidate roots, expand branches, then test the residual.

METHOD SHEETS: M038-M045

LIMIT: Read historical proposal language alongside the later corrections in CT1.

Archive file: ctheory_main.tex

CT4 FIVE-LEVEL DECOMPOSITION

USE: Use for worked tower examples, coefficient tables, and checks that a known scalar reconstructs from its coordinates.

METHOD SHEETS: M039-M040, M055

LIMIT: Reconstructing known keys is an arithmetic check, not held-out prediction.

Archive file: ctheory_decompose_full.md

REFERENCES / CONTINUED

From arithmetic to a bounded search

The remaining files show how coordinates are computed and passed to a search routine. They are useful for reproducibility, but code presence and exact round-trips do not establish target recovery.

CT5 DEEPER DECOMPOSITION LIMITS

USE: Use when extending the tower or splitting a remainder: track residual possibilities and compare apparent patterns with range-matched controls.

METHOD SHEETS: M046, M061-M064

LIMIT: More levels can improve organization without reducing total search work.

Archive file: deeper_decomposition_limits.md

CT6 INTEGER DECOMPOSITION CODE

USE: Use to reproduce exact cube splitting, tower construction, scalar reconstruction, and the arithmetic side of a candidate check.

METHOD SHEETS: M038-M040, M046

LIMIT: Source code documents a mechanism; a target run needs its own saved evidence.

Archive file: ctheory.py

CT7 PHASE-2 FEASIBILITY CODE

USE: Use to enumerate feasible Delta2 values and turn a proposed R2 coordinate into possible R1 coordinates.

METHOD SHEETS: M041, M043, M047

LIMIT: A feasible branch is not necessarily the target branch; account for all legal branches.

Archive file: phase2.py

CT8 PHASE-2 SEARCH HANDOFF

USE: Use to inspect residual public-point construction and the handoff from a candidate branch to a bounded D1 search.

METHOD SHEETS: M041-M045, M026

LIMIT: The bounded solver needs the target public point and a valid interval; an address alone is insufficient.

Archive file: phase2_search.py

PLAIN-LANGUAGE READING KEY

Extended glossary

Terms are defined for this inventory, not as universal labels for every implementation. Method IDs point to fuller examples. Especially important distinctions: an address is not a public point; exact reconstruction is not unknown-key prediction; a recorded run is not independent reproduction.

Address	A Bitcoin payment identifier derived from a script or key-related hash. An address is not the same input as a full public point. See M001-M002.
Anchor	A reference coordinate used to center or compare candidate ranges. A tier-derived anchor may be known without revealing the target coordinate. See M040, M052.
Artifact	A saved file, log, table, program, or paper inspected for this inventory. Its presence is evidence of a record, not automatic proof of its claims.
BIP32	A hierarchical deterministic wallet derivation scheme. Testing it requires the relevant generation path, seed scope, and version assumptions. See M095.
Blind test	A test in which the target answer is withheld while the method and candidate rule are fixed. It distinguishes forecasting from fitting known answers. See M059-M060.
Bloom filter	A compact set-membership filter that can say “possibly present” or “definitely absent”; a positive requires an exact check. See M199.
Bounded discrete log	Recovering k from Q=kG when k is known to lie in a specified interval. The full target public point and interval are essential inputs. See M023, M026.
BSGS	Baby-step giant-step: a time-memory method for a bounded discrete logarithm. Its memory and setup costs matter alongside its search steps.
Calibration leakage	Using a held-out target answer, directly or indirectly, to choose a method parameter. This invalidates the claimed blind score. See M059.
Candidate	A possible private scalar proposed for verification. A candidate is not a recovered key until it passes the exact target check.
Checkpoint	Saved search state that allows a run to resume and helps distinguish tested coverage from planned coverage.
Claim scope	The exact targets, inputs, versions, budget, and procedure to which a result applies. A finite rejection does not automatically generalize.
Coefficient c	The quotient in $x=r^3+cr^2+d$ after fixing the canonical integer cube root r. Its feasible values depend on r. See M038-M040.
Complement	Candidates outside a prioritized band or slot. A narrow successful-looking band is not complete unless the complement is also accounted for.
Compressed public key	A compact encoding of an elliptic-curve point that stores x and a parity bit. It is not an address hash.
Containment	Whether a proposed interval or slot actually includes the known or held-out target. Cost is meaningful only with coverage. See M047, M059.
Corrected claim	An earlier conclusion narrowed or withdrawn after an audit. The later scope governs interpretation, even if an older paper uses stronger language.
CTheory	The archive’s cubic integer-coordinate system for scalar candidates. Exact decomposition is established; unknown-key prediction is a separate question. See M038-M052.

EXTENDED GLOSSARY / CONTINUED

Curve order	The number n of scalar multiples in the secp256k1 generator subgroup. Private scalars are interpreted modulo n.
D1	The first residual d_1 after a scalar is split as $k=R_1^3+c_1R_1^2+D1$. Searching D1 still requires the correct outer coordinates. See M026, M043.
Delta2	The second residual d_2 in $R_1=R_2^3+c_2R_2^2+Delta2$. A feasible Delta2 range can be enumerated without knowing which value is the target. See M041.
Distinguished point	A public point meeting a chosen bit pattern and saved as a collision marker in a walk-based search. Its storage rule does not reveal a key by itself.
ECDLP	Elliptic-curve discrete logarithm problem: given G and Q=kG, determine k. The bounded case adds a known candidate interval.
ECDSA	Elliptic Curve Digital Signature Algorithm. Recoverability from signatures requires a specific nonce relation or leakage, not merely the existence of signatures. See M153-M157.
Entropy	Uncertainty in a generator or candidate distribution under a stated model. It is not established by visual randomness alone.
Evidence status	The inventory label distinguishing executed records, implementations, conditional models, proposals, and corrections. Read it before the result text.
Executed record	A saved output describing a run. The inventory preserves the record but does not independently rerun every experiment.
Exhaustive search	Checking every candidate in a defined finite range under a stated verification rule. Outside that range remains untested. See M002.
False positive	A candidate flagged by a filter or partial check that fails exact verification. Bloom hits and prefix matches need a final target check.
Family-wise correction	Accounting for the fact that the largest apparent signal was selected from many features or tests. See M061-M064.
Feasible branch	A coordinate branch not excluded by arithmetic and range bounds. Feasible means possible, not likely or correct. See M041, M049.
Firmware replay	Reproducing a named wallet or device generation path under a specified version and seed/entropy scope. See M103-M116.
Full-slot work	The search cost after adding all legal slots, solver setup, failed branches, and any complement; not just the cheapest conditional leaf. See M048.
Generator hypothesis	A fully specified process claimed to produce candidate keys. A valid test fixes its state, output schedule, transformations, and observations in advance.
Generator point G	The standard secp256k1 point multiplied by private scalar k to obtain public point Q. Not a wallet random-number generator.
GPU sweep	A finite parallel candidate test. Its coverage is determined by the exact generator families, index ranges, targets, and saved run limits.
Hamming distance	The number of bit positions where two equal-length strings differ. A small Hamming ball is only a local neighborhood. See M128.
HASH160	RIPEMD-160 applied to SHA-256 of a serialized public key. It is used in common key-hash address constructions; it is not the public point.
Held-out target	A case excluded from model selection and used only for a final test. Forward-time holdout is stronger than refitting after seeing the result.

EXTENDED GLOSSARY / CONTINUED

Implemented	Relevant code exists. This label alone does not say that a target run completed or recovered a key.
Input contract	The data a method genuinely requires: for example a public point, an interval, or a leaked nonce relation. Missing inputs cannot be supplied by a similar-looking address.
Integer cube root	The greatest integer r with $r^3 \le x$. Exact integer arithmetic avoids floating-point boundary errors in the CTheory split.
Interval	A declared contiguous candidate range. Its width, endpoints, and whether the target lies inside determine the meaning of a bounded search.
Kangaroo	Pollard's interval discrete-log search using pseudorandom walks and collisions; expected work scales roughly with the square root of interval width. See M023.
Key derivation	A specified transformation from seed or parent material to key bytes. The exact wallet version and derivation path matter.
Known-key round-trip	Decompose a known scalar and reconstruct it exactly. This checks arithmetic, not prediction of an unknown scalar. See M038-M039.
Lagged recurrence	A generator rule in which a new word depends on earlier words at fixed lags. It is testable only when word alignment and state assumptions are specified. See M116.
Mask	A bit-level transformation or selection rule applied to generator output or candidate scalars. Different masks define different hypotheses.
Method ID	The M-number assigned to one inventory entry. It is a navigation key, not a ranking or success score.
Monte Carlo p-value	The fraction of simulated null outcomes at least as extreme as the observed statistic, with finite-sample adjustment as specified. It is not a recovery probability.
Negative result	A failed or null test under a documented scope. It rules out or weakens that scope without ruling out every related idea.
Nonce	An ephemeral ECDSA signing scalar. Reuse, algebraic relation, or partial leakage can matter; an ordinary signature does not expose it.
Null control	A comparison set generated under a stated no-signal model. For puzzle keys, the range and selection procedure must be matched. See M061.
Paper ID	The PAPER-number for a local research document catalogued in the paper chapter. Papers can contain dated claims later narrowed by audits.
Phase 2	In this CTheory workflow, expansion from a proposed R_2 coordinate through feasible Delta2 values into R_1 candidates. See M041-M042.
Pollard kangaroo	A bounded discrete-log algorithm based on walks and collision detection. It is conditional on a suitable public-point target and interval.
Private key	A secret scalar k that determines a public point Q=kG. This inventory does not treat a public address as a private key.
PRNG	Pseudorandom number generator: a deterministic process whose outputs depend on state and seed. Weak or known state can create a testable key family.
Proposal	A documented method idea without enough saved execution to call it a completed test. Its predicted benefit remains conditional.
Public point	The elliptic-curve point Q=kG. It provides the algebraic target needed by bounded ECDLP tools; an address hash usually does not.
Quantum resource estimate	A model-dependent count of fault-tolerant hardware and time. It is not evidence that such a machine ran the attack.

EXTENDED GLOSSARY / CONTINUED

R1 / R2	The first and second integer cube-root coordinates in the CTheory tower. The PDF also denotes them Y_1 and Y_2. See M039.
Range-matched	Drawn from the same allowed candidate interval as the observed keys. This prevents interval geometry from masquerading as a signal.
Recovery	A candidate scalar that passes the exact target public-point or address derivation check, with the applicable input and provenance recorded.
Remainder d	The leftover after subtracting r^3+cr^2 in the canonical cubic split; by construction $0 \le d < r^2$. See M038.
Residual point	The point remaining after subtracting a guessed base scalar times G from Q. Solving its log still depends on a valid branch and interval.
Scalar	An integer used to multiply the secp256k1 generator point. Candidate scalar, private key, and discrete log refer to different roles for this integer.
Search slot	A candidate subrange defined by coordinate choices or constraints. Its apparent small size is conditional on the correct slot being included.
secp256k1	The elliptic curve and generator subgroup used by Bitcoin public keys. The inventory's $Q=kG$ equations refer to this group.
Seed	Initial material for a deterministic key generator or wallet. Testing a seed family is meaningful only within its explicit size and derivation scope.
Shor-style simulation	A classical program modeling pieces of a quantum order-finding workflow. It is not a physical quantum recovery run. See M232-M244.
Signature leakage	Extra information about an ECDSA nonce or signing key. An attack needs the precise leakage or nonce relation its equations assume.
Source ID	The S-number for a directly cited local evidence file. The source index gives its location; the method sheet explains why it matters.
Square-root cost	The characteristic approximate growth of bounded generic discrete-log search with interval width. Total work also includes branch count and setup.
Target	The address, hash, public point, or signature relation being tested. Different targets expose different information and permit different methods.
Tier	An N-bit puzzle scalar interval, here [2^(N−1), 2^N). It is an allowed range, not a generator fingerprint.
Tower	Nested exact cubic splits of k and its successive roots. All coordinates together reconstruct k; unknown coordinates remain unknown. See M039.
Validation	The final exact check that a candidate produces the specified target, plus separate checks that the method's claimed coverage and cost were recorded.
Wallet	Software or hardware that generates, stores, and derives keys. A device-family hypothesis requires its exact implementation and configuration.
Weak entropy	A key-generation situation with less effective uncertainty than intended under a specified model. It is not inferred solely from a patterned address.
Word order	The byte/word arrangement used to interpret generator outputs. A different order can change every predicted key in a replay test.
Zero entropy of a feature	A statistic constant over the declared interval contains no within-interval distinguishing information; it says nothing by itself about the key's entropy.

TRACEABILITY

Source index

Source IDs identify evidence files in the accompanying archive. The published index intentionally uses sanitized filenames rather than local machine paths. Compilation-time SHA-256 hashes and fuller provenance remain in the private working inventory.

S001	evidence/bitcoin_scale5_results.json
S002	evidence/bitcoin_puzzle_71_master_pipeline.py
S003	evidence/README.md
S004	evidence/WORKING_NOTES.md
S005	evidence/benchmark_point_walk.py
S006	evidence/status.json
S007	evidence/continuous_stream.py
S008	evidence/EXTENDED_REPORT.md
S009	evidence/REPORT.md
S010	evidence/results.json
S011	evidence/pipeline.py
S012	evidence/point-walk-benchmark.json
S013	evidence/point_walk.py
S014	evidence/run-1M.log
S015	evidence/stream-verification-20261019.json
S016	evidence/stream_check.py
S017	evidence/test_pipeline.py
S018	evidence/verification-1M.json
S019	evidence/verification-boundaries-1M.json
S020	evidence/verify_samples.py
S021	evidence/ACTUAL_BACKTEST.md
S022	evidence/AUDIT.md
S023	evidence/BACKTEST.md
S024	evidence/backtest.py
S025	evidence/backtest_continuous_17_32.py
S026	evidence/candidate_sampling.py

SOURCE INDEX / CONTINUED

S027	evidence/check_keys.py
S028	evidence/benchmark_point_walk.py
S029	evidence/REPORT.md
S030	evidence/REPORT.md
S031	evidence/test_unique_sampling.py
S032	evidence/ALL_VALUES_REPORT_256.md
S033	evidence/INVERSE_CAPABILITY_REPORT.md
S034	evidence/QUOTIENT_REPORT.md
S035	evidence/QUOTIENT_REPORT_256.md
S036	evidence/REPORT.md
S037	evidence/all_donor_carry.json
S038	evidence/all_donor_carry.py
S039	evidence/nearmiss_analysis.py
S040	evidence/nearmiss_results.json
S041	evidence/exhaustive_carry.py
S042	evidence/exhaustive_carry_report.json
S043	evidence/gap_fill.py
S044	evidence/gap_fill_report.json
S045	evidence/report.json
S046	evidence/smooth_share.py
S047	evidence/smooth_share_report.json
S048	evidence/surface2d.py
S049	evidence/surface2d_report.json
S050	evidence/test.py
S051	evidence/Cargo.toml
S052	evidence/check_experiments_vs_balance.py
S053	evidence/coldcard_experiments_vs_balance.json
S054	evidence/derive.rs
S055	evidence/yasmarang.rs
S056	evidence/stats-continuous-output.txt

SOURCE INDEX / CONTINUED

S057 evidence/stats-range-output.txt

S058 evidence/stats-variations-output.txt

S059 evidence/coldcard_make_new_wallet.py

S060 evidence/entropy_patterns.py

S061 evidence/entropy_position.py

S062 evidence/funded_membership.py

S063 evidence/residual_utxo.py

S064 evidence/seed_addr_cluster.json

S065 evidence/seed_addr_cluster.py

S066 evidence/OTHER_KEYS_AND_BACKTESTS.md

S067 evidence/R2_SEARCH_INTERVALS.md

S068 evidence/REPORT.md

S069 evidence/all_unknown_check.py

S070 evidence/candidate_models.py

S071 evidence/check_routes.py

S072 evidence/check_wallet_leads.py

S073 evidence/results.json

S074 evidence/continue_140.py

S075 evidence/scan_chats.py

S076 evidence/wallet_lead_results.json

S077 evidence/README.md

S078 evidence/analyze_cli_clustering.py

S079 evidence/analyze_creator_generator.py

S080 evidence/backtest_all_keys_slots.py

S081 evidence/benchmark_cpu_ec.py

S082 evidence/build_loyce_index.py

S083 evidence/build_value_atlas_20260920.py

S084 evidence/ctheory_forward_familywise_20261008.py

S085 evidence/ctheory_forward_null_20260920.py

S086 evidence/ctheory_forward_slice_family_20261008.py

SOURCE INDEX / CONTINUED

S087	evidence/ctheory_math_20260920.py
S088	evidence/ctheory_remainder_precision_20260920.py
S089	evidence/loop.py
S090	evidence/d1_search.cu
S091	evidence/demo_pwsk.py
S092	evidence/hunt_multi_significant.py
S093	evidence/hunt_shift_windows.py
S094	evidence/bec.py
S095	evidence/bit_flip_hunt.py
S096	evidence/bitsavers_methods.py
S097	evidence/collision.py
S098	evidence/solver.py
S099	evidence/ctheory.py
S100	evidence/ctheory_arithmetic.py
S101	evidence/d1_narrowing.py
S102	evidence/dp_parser.py
S103	evidence/dual_mask_seq.py
S104	evidence/ecdsa_affine_nonce.py
S105	evidence/export.py
S106	evidence/fast_ec.py
S107	evidence/gpu_bench.py
S108	evidence/joc_methods.py
S109	evidence/kangaroo.py
S110	evidence/keyspace_4d.py
S111	evidence/miss_ladder.py
S112	evidence/oreilly_methods.py
S113	evidence/phase.py
S114	evidence/phase2.py
S115	evidence/phase2_search.py
S116	evidence/pwsk.py

SOURCE INDEX / CONTINUED

S117 evidence/pwsk_budget.py

S118 evidence/relation_bundle.py

S119 evidence/research.py

S120 evidence/backends_cuda.py

S121 evidence/base.py

S122 evidence/set_bits.py

S123 evidence/slot_contract.py

S124 evidence/slot_scaling.py

S125 evidence/unimath_corpora.py

S126 evidence/unimath_insights.py

S127 evidence/user_signature.py

S128 evidence/wrapped_export.py

S129 evidence/wrapped_kangaroo.py

S130 evidence/mock_keyspace_network.py

S131 evidence/CTHEORY_FORWARD_RESEARCH_20260920.md

S132 evidence/CTHEORY_MATH_REVIEW_20260920.md

S133 evidence/FEASIBILITY_REVALIDATION_20260918.md

S134 evidence/PRIVATE_KEY_RELATION_AUDIT_20260919.md

S135 evidence/PUZZLE_STATUS_NOTE.md

S136 evidence/all_keys_slot_backtest.md

S137 evidence/any_relation_report.md

S138 evidence/archive_rng_wordstream_audit_20261020.md

S139 evidence/bip32_glv_probe.md

S140 evidence/bitsavers_methods_backtest.md

S141 evidence/blind_loo_backtest_20260919.md

S142 evidence/block1_keygen_seed_sweep_2008.md

S143 evidence/btcparser_sweep_signature_scan.md

S144 evidence/chain_relationship_map.md

S145 evidence/coefficient_interval_audit_20261019.md

S146 evidence/coinvault_bip32_oracle.md

SOURCE INDEX / CONTINUED

S147 evidence/counter_nonce_analysis.md

S148 evidence/creative_theories.md

S149 evidence/crypto_non_bruteforce_methods.md

S150 evidence/csp_ordering_backtest.md

S151 evidence/ctheory_phase_ab_summary.md

S152 evidence/d1_narrowing_audit.md

S153 evidence/deeper_decomposition_limits.md

S154 evidence/early_coinbase_sig_harvest.md

S155 evidence/early_key_relations.md

S156 evidence/early_key_relations_focus.md

S157 evidence/fanout_cc6f_trace.md

S158 evidence/fanout_puzzle135.md

S159 evidence/follow_099_fanouts.md

S160 evidence/generator_all_unknown_targets_report_20261020.md

S161 evidence/google_drive_puzz_triage.md

S162 evidence/half_generator_cross_curve_report_20261020.md

S163 evidence/heldout_r2_vs_midpoint.md

S164 evidence/hub_3EEJF_scan.md

S165 evidence/index_of_directories_scan.md

S166 evidence/index_offset_holdout_20261019.json

S167 evidence/key_masking_hypothesis.md

S168 evidence/lattice_candidate_scan.md

S169 evidence/lowbit_generator_audit_20261019.md

S170 evidence/milksad_cuda_sweep.md

S171 evidence/non_bruteforce_audit_20260831.md

S172 evidence/old_bitcoin_sites_scan.md

S173 evidence/orbit_candidates_report.md

S174 evidence/park_miller_target_check_20261020.json

S175 evidence/profanity_cuda_sweep.md

S176 evidence/puzzle_reflection_scan.md

SOURCE INDEX / CONTINUED

S177 evidence/puzzle_unlock_relationships.md

S178 evidence/recoverable_bitcoin_scan.md

S179 evidence/relation_160_block1.md

S180 evidence/remainder_tree_beacon_report_20261020.md

S181 evidence/revelation_discovery.md

S182 evidence/schnorr_ecdsa_aranyn_compare.md

S183 evidence/search_narrowing_ladder.md

S184 evidence/segwit_aranyn_r_scan.md

S185 evidence/set_bits_audit.md

S186 evidence/sparse_key_hardthink.md

S187 evidence/sparse_key_investigate.md

S188 evidence/splitmix64_target_check_20261020.json

S189 evidence/testnet_aranyn_scan.md

S190 evidence/trace_sender_extended.md

S191 evidence/trace_sender_extra_data_scan.md

S192 evidence/unimath_corpora.md

S193 evidence/unknown_parameter_inventory_20261020.json

S194 evidence/wallace_lcg_mask_backtest_20261020.json

S195 evidence/weak_entropy_cuda_sweep.md

S196 evidence/weak_entropy_repos.md

S197 evidence/weak_entropy_unverified.md

S198 evidence/AUDIT.md

S199 evidence/wolfram_masks.py

S200 evidence/wolfram_mix.py

S201 evidence/affine_lowbits_check_20261019.py

S202 evidence/analyze_ec_decompose.py

S203 evidence/analyze_leading_zero_pubs.py

S204 evidence/analyze_spent_block_encoding.py

S205 evidence/archive_rng_wordstream_check_20261020.py

S206 evidence/generator_all_unknown_targets_20261020.py

SOURCE INDEX / CONTINUED

S207 evidence/half_generator_cross_curve_20261020.py

S208 evidence/index_offset_holdout_20261019.py

S209 evidence/lowbit_linear_state_20261019.py

S210 evidence/park_miller_target_check_20261020.py

S211 evidence/remainder_tree_beacon_20261020.py

S212 evidence/schnorr_ecdsa_aranyn_compare.py

S213 evidence/search_160_block1_relation.py

S214 evidence/splitmix64_target_check_20261020.py

S215 evidence/unknown_parameter_inventory_20261020.py

S216 evidence/wallace_lcg_mask_backtest_20261020.py

S217 evidence/crack_time_will_tell_seed.py

S218 evidence/child_wallet_interval_probe.py

S219 evidence/analyze_ctheory_decompose.py

S220 evidence/backtest_arxiv_methods.py

S221 evidence/backtest_ctheory.py

S222 evidence/backtest_hunt_pipeline.py

S223 evidence/cpu_band_d1_hunt.py

S224 evidence/ctheory_phase_solver.py

S225 evidence/export_ctheory_gpu_d1.py

S226 evidence/unified_keyspace_framework_algorithm.py

S227 evidence/harvest_early_coinbase_descendants.py

S228 evidence/harvest_early_coinbase_graph.py

S229 evidence/armory_mask_test.py

S230 evidence/electrum_v1_mask_test.py

S231 evidence/entropy_prefix_test.py

S232 evidence/fullwidth_constraint_test.py

S233 evidence/synthesis_puzzle71_solver.py

S234 evidence/scan_fork_pubkey_leaks.py

S235 evidence/search_signed_bitcoin_messages.py

S236 evidence/sparse_key_bitlearn.py

SOURCE INDEX / CONTINUED

S237 evidence/sparse_key_bloom.py

S238 evidence/sparse_key_coshift.py

S239 evidence/sparse_key_delta_hunt.py

S240 evidence/sparse_key_dial_in.py

S241 evidence/sparse_key_xor_tail.py

S242 evidence/lib.rs

S243 evidence/ecdsa.py

S244 evidence/loop.py

S245 evidence/quantum.py

S246 evidence/verify_scan_hits.py

S247 evidence/GPU_PLAYBOOK.md

S248 evidence/weak_entropy_families.py

S249 evidence/weak_entropy_prng.py

S250 evidence/weak_entropy_wallet.py

S251 evidence/Branch-cursor-ctheory-self-improve-agents-ac93.md

S252 evidence/Branch-cursor-profanity-offset-grind-8a23.md

S253 evidence/Branch-cursor-time-will-tell-seed-b049.md

S254 evidence/GPU-Deployment.md

S255 evidence/Shor-Simulation.md

S256 evidence/Weak-Entropy-Research.md

S257 evidence/empirical_key_forensics.json

S258 evidence/empirical_key_forensics.py

S259 evidence/nth_root_blind_address_test.json

S260 evidence/synthetic_chain_evaluation.csv

S261 evidence/evaluate_synthetic_chain.py

S262 evidence/BACKEND_MATRIX_2026-09-10.md

S263 evidence/FACTORING_RUN_REPORT_2026-10-10.md

S264 evidence/RUN_REPORT_2026-09-09.md